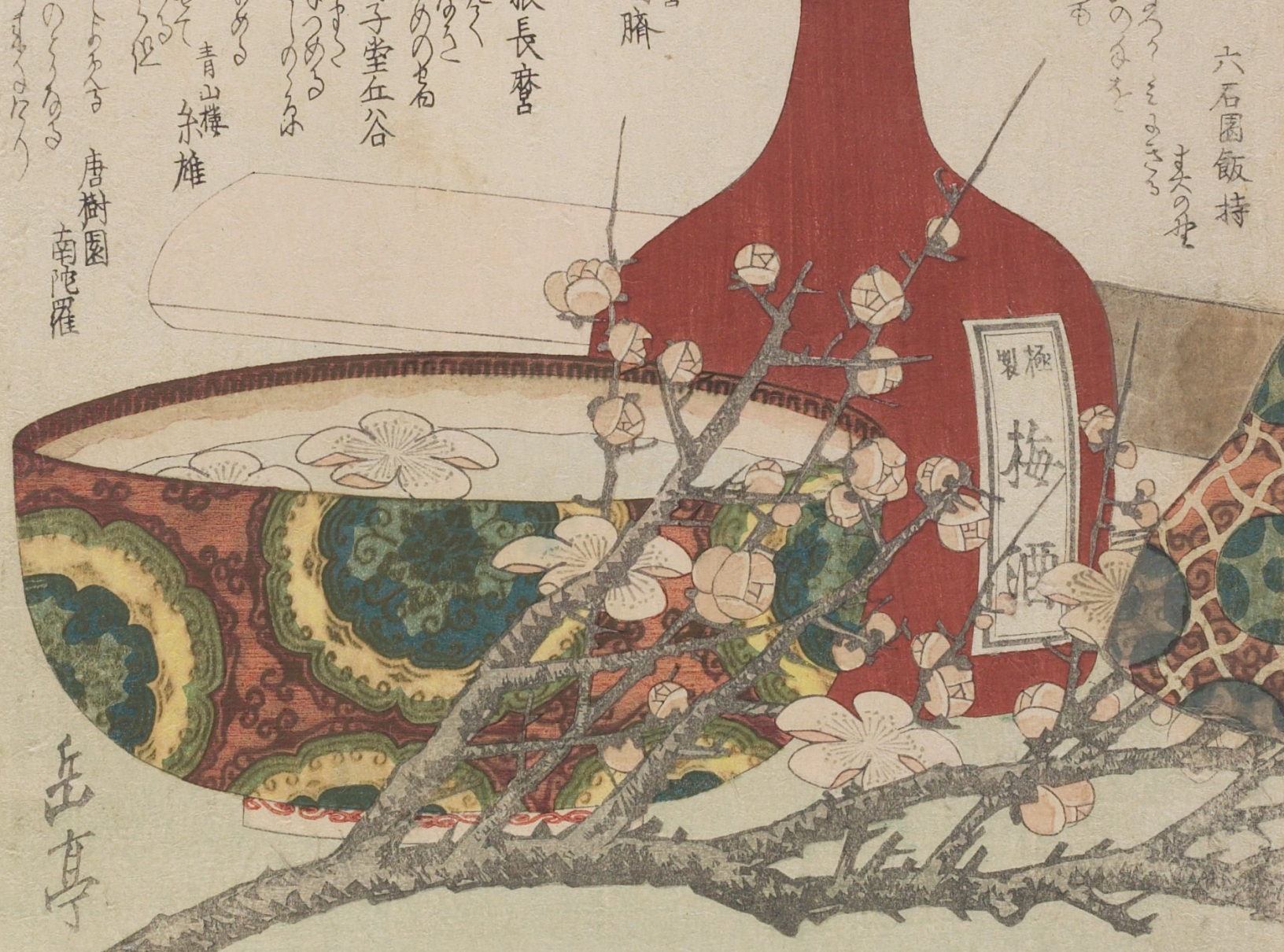




Piensa en Raku

Cómo pensar como un científico de la computación

Laurent Rosenfeld,
con Allen B. Downey

Piensa en Raku

Cómo Pensar Como un Científico de la Computación

2da Edición, Versión 0.6

Enero 2020

Piensa en Raku

Cómo Pensar Como un Científico de la Computación

2da Edición, Versión 0.6

Enero 2020

Laurent Rosenfeld, con Allen B. Downey

Green Tea Press

Needham, Massachusetts

Copyright © 2017 Allen Downey, Laurent Rosenfeld.

Green Tea Press
9 Washburn Ave
Needham MA 02492

Se concede permiso para copiar, distribuir y/o modificar este documento bajo los términos de Creative Commons Attribution-NonCommercial 3.0 Unported License, la cual está disponible en <http://creativecommons.org/licenses/by-nc/3.0/>.

La forma original de este libro es código fuente $\text{\LaTeX}$. La compilación de este código $\text{\LaTeX}$ tiene el efecto de generar una representación independiente a cualquier dispositivo de este libro de texto, la cual puede ser convertida a otros formatos e impresa.

El código $\text{\LaTeX}$ de este libro está disponible en <https://gitlab.com/uzluisf/piensa-raku/>. Puedes encontrar el código $\text{\LaTeX}$ de la versión en inglés en https://github.com/LaurentRosenfeld/think_raku.

Título original: *Think Raku: How to Think Like a Computer Scientist*

Cubierta por Luis F. Uceta.

La imagen usada en la cubierta es un ejemplo de *ukiyo-e*, un género de grabados realizados mediante técnica de grabado en madera, producidos en Japón entre los siglos XVII y XX. El artista de esta pieza en particular es Yashima Gakutei.

La fuente utilizada es Libertinus Serif, <https://github.com/alif-type/libertinus>.

Traducción por Luis F. Uceta, <https://gitlab.com/uzluisf>

Correctores de la traducción

La traducción de este libro no habría sido la misma sin la ayuda de las siguientes personas:

- Boyd Duffee, <https://gitlab.com/duffee>

Dedico esta traducción a mi adorada madre Flerida y
querida abuela Aurelia.

— Luis F. Uceta

Tabla de Contenido

Introducción	xxi
I Comenzando con lo Básico	1
1 La Forma del Programa	5
1.1 ¿Qué es un Programa?	5
1.2 Ejecutando Raku	6
1.3 El Primer Programa	8
1.4 Operadores Aritméticos	8
1.5 Valores y Tipos	9
1.6 Lenguajes Formales y Naturales	11
1.7 Depuración de Programas	13
1.8 Glosario	13
1.9 Ejercicios	14
2 Variables, Expresiones y Sentencias	17
2.1 Sentencias de Asignación	17
2.2 Nombres de las Variables	19
2.3 Expresiones y Sentencias	20
2.4 Modo Script	22
2.5 Modo de una sola línea	24
2.6 Orden de Operaciones	24
2.7 Operaciones de Cadena de Texto	25
2.8 Comentarios	26

2.9	Depuración de Programas	27
2.10	Glosario	27
2.11	Ejercicios	28
3	Funciones	31
3.1	Llamadas de Funciones	31
3.2	Funciones y Métodos	34
3.3	Funciones Matemáticas	34
3.4	Composición	36
3.5	Agregar Nuevas Funciones (o Subrutinas)	37
3.6	Definiciones y Usos	39
3.7	Flujo de Ejecución	39
3.8	Parámetros y Argumentos	40
3.9	Las Variables y los Parámetros son Locales	41
3.10	Diagramas de Pila	42
3.11	Funciones Fructuosas y Funciones Voids	42
3.12	Signaturas de Funciones	44
3.13	Parámetros Inmutables y Mutables	45
3.14	Funciones y Subrutinas: Ciudadanos de Primera Clase	46
3.15	¿Por qué Funciones y Subrutinas?	48
3.16	Depuración de Programas	49
3.17	Glosario	50
3.18	Ejercicios	51
4	Bucles, Condicionales y Recursión	55
4.1	División de Enteros y Módulo	55
4.2	Expresiones Booleanas	56
4.3	Operadores Lógicos	58
4.4	Ejecuciones Condicionales	60
4.5	Ejecución Alternativa	60
4.6	Condicionales Encadenadas	61
4.7	Condicionales Anidadas	62
4.8	Condicionales if como Modificadores de Sentencias	63

Tabla de Contenido		xi
4.9	Sentencia Condicional Unless	63
4.10	Bucles For	64
4.11	Recursión	65
4.12	Diagramas de Pilas para Subrutinas Recursivas	67
4.13	Recursión Infinita	68
4.14	Entrada del Teclado	68
4.15	Argumentos del Programa y la Subrutina MAIN	69
4.16	Depuración de programas	69
4.17	Glosario	71
4.18	Ejercicios	72
5	Subrutinas Fructuosas	75
5.1	Valores de Retorno	75
5.2	Desarrollo Incremental	77
5.3	Composición	79
5.4	Funciones Booleanas	80
5.5	Un Lenguaje de Programación Completo	81
5.6	Más Recursión	82
5.7	Acto de Fe	84
5.8	Otro Ejemplo Más	84
5.9	Chequeo de Tipos	85
5.10	Subrutines multi	87
5.11	Depuración de programas	88
5.12	Glosario	89
5.13	Ejercicios	90
6	Iteración	93
6.1	Asignación Versus Igualdad	93
6.2	Reasignación	94
6.3	Actualización de las Variables	94
6.4	La Sentencia while	95
6.5	Variables Locales y Ámbito de una Variable	97
6.6	Sentencias de Flujo de Control (last, next, etc.)	100

6.7	Raíces Cuadradas	101
6.8	Algoritmos	103
6.9	Depuración de Programas	104
6.10	Glosario	104
6.11	Ejercicios	105
7	Cadenas de Texto	107
7.1	Una Cadena de Texto es una Secuencia	107
7.2	Operadores Comunes de las Cadenas de Texto	108
7.2.1	Longitud de una Cadena de Texto	108
7.2.2	Búsqueda de una Subcadena Dentro de una Cadena de Texto	109
7.2.3	Extrayendo una Subcadena de una Cadena de Texto	109
7.2.4	Otras Funciones o Métodos Útiles de las Cadenas de Texto	110
7.3	Recorrido de una Cadena con un bucle <code>while</code> o <code>for</code>	113
7.4	Bucles y Conteos	114
7.5	Expresiones Regulares (Regexes)	114
7.6	Uso de los Regexes	116
7.7	Cómo Construir tus Patrones Regex	117
7.7.1	Coincidencia Literal	118
7.7.2	Comodines y Categorías de Caracteres	118
7.7.3	Cuantificadores	119
7.7.4	Anclas y Aserciones	120
7.7.5	Alternancia	122
7.7.6	Grupos y Capturas	123
7.7.7	Adverbios (o Modificadores)	123
7.7.8	Ejercicios con Regexes	124
7.8	Combinando Todas las Piezas	125
7.8.1	Extracción de Fechas	125
7.8.2	Extracción de una dirección IP	127
7.9	Sustituciones	128
7.9.1	El Método <code>subst</code>	128
7.9.2	La construcción <code>s/búsqueda/reemplazo/</code>	128

7.9.3	Uso de Capturas	129
7.9.4	Adverbios	129
7.10	Depuración de Programas	130
7.11	Glosario	132
7.12	Ejercicios	133
8	Caso Práctico: Juego de Palabras	137
8.1	Lectura de y Escritura a un Archivo	137
8.2	Lectura de Listas de Palabras	139
8.3	Ejercicios	140
8.4	Búsqueda	141
8.4.1	Palabras con Más de 20 Caracteres (Solución)	141
8.4.2	Palabras Sin “e” (Solución)	142
8.4.3	Evitar Otras Letras (Solución)	143
8.4.4	Usando Solo Algunas Letras (Solución)	144
8.4.5	Usando Todas las Letras de una Lista (Solución)	144
8.4.6	Orden Alfabético (Solución)	145
8.4.7	Otro Ejemplo de Reducción a un Problema Previamente Solucionado	145
8.5	Depuración de Programas	146
8.6	Glosario	147
8.7	Ejercicios	147
9	Arrays y Listas	149
9.1	Las Listas y los Arrays Son Secuencias	149
9.2	Los Arrays Son Mutables	151
9.3	Cómo Agregar o Remover Elementos de un Array	153
9.4	Pilas y Colas	155
9.5	Otras Formas de Modificar un Array	156
9.6	Cómo Recorrer una Lista	158
9.7	Nuevas Construcciones de Repetición	160
9.8	Asociaciones, Filtros y Reducciones	161
9.8.1	Reducir una Lista a un Valor	161
9.8.2	El Metaoperador de Reducción	162

9.8.3	Asociando una Lista a Otra Lista	163
9.8.4	Filtrando los Elementos de una Lista	164
9.8.5	Funciones de Orden Superior y la Programación Funcional	165
9.9	Arrays de Tamaños Fijos y Tipados	166
9.10	Arrays Multidimensionales	167
9.11	Ordenando Arrays o Listas	168
9.12	Técnicas de Ordenamiento Más Avanzadas	169
9.13	Depuración de Programas	172
9.14	Glosario	173
9.15	Ejercicios	173
10	Hashes	177
10.1	Un Hash es un Mapeo	177
10.2	Operaciones Comunes con Hashes	180
10.3	Un Hash como una Colección de Contadores	181
10.4	Bucles y Hashes	182
10.5	Búsqueda Inversa	183
10.6	Comprobando la Existencia	184
10.7	Las Claves de un Hash son Únicas	185
10.8	Los Hashes y los Arrays	186
10.9	Memos	188
10.10	Los Hashes como Tablas de Despacho	190
10.11	Variables Globales	191
10.12	Depuración de Programas	192
10.13	Glosario	193
10.14	Ejercicios	194
11	Caso Práctico: Selección de Estructura de Datos	195
11.1	El Operador Condicional Ternario	195
11.2	La Sentencia “Switch” given when	196
11.3	Condicionales Múltiples con Uniones	198
11.4	Subrutinas con Parámetros Nombrados y Opcionales	199
11.4.1	Parámetros Nombrados	199

11.4.2	Parámetros Opcionales	200
11.5	Análisis de la Frecuencia de Palabras	200
11.6	Números Aleatorios	201
11.7	Histograma de Palabras	202
11.8	Palabras Más Comunes	205
11.9	Parámetros Opcionales	206
11.10	Diferencia de Hashes	206
11.11	Construir Operadores Nuevos	207
11.12	Sets, Bags y Mixes	208
11.13	Palabras Aleatorias	210
11.14	Análisis de Markov	211
11.15	Estructuras de Datos	214
11.16	Construyendo tus Propias Estructuras de Datos	215
11.16.1	Listas Enlazadas	215
11.16.2	Árboles	216
11.16.3	Montículos Binarios	217
11.17	Depuración de Programas	220
11.18	Glosario	221
11.19	Ejercicio: Codificación Huffman	222
11.19.1	Código de Longitud Variable	222
11.19.2	La Tabla de Frecuencias	223
11.19.3	Construyendo el Código Huffman	223
II	Hacia Adelante	227
12	Clases y Objetos	231
12.1	Objetos, Métodos y Programación Orientada a Objetos	231
12.2	Tipos Definidos por el Programador	233
12.3	Atributos	234
12.4	Creando Métodos	236
12.5	Rectángulos y Composición de Objetos	239
12.6	Instancias como Valores de Retorno	240

12.7	Herencia	241
12.7.1	La Clase Pixel	241
12.7.2	La Clase PuntoMovable	243
12.7.3	Herencia Múltiple: Atractiva Pero, es de Sabio Utilizarla?	245
12.8	Roles y Composición	246
12.8.1	Clases y Roles: Un Ejemplo	247
12.8.2	Composición de un Rol y Reutilización de Código	248
12.8.3	Roles, Clases, Objetos, y Tipos	249
12.9	Delegación de Método	250
12.10	Polimorfismo	251
12.11	Encapsulación	253
12.11.1	Métodos Privados	254
12.11.2	Construir Objetos con Atributos Privados	254
12.12	Interfaz e Implementación	257
12.13	Programación Orientada a Objetos: Una Fábula	258
12.13.1	La Fábula del Pastor	258
12.13.2	La Moraleja	259
12.14	Depuración de Programas	259
12.14.1	El Depurador de Raku	260
12.14.2	Cómo Conseguir Ayuda	260
12.14.3	Recorriendo el Código	260
12.14.4	Parando en el Lugar Correcto con Puntos de Interrupción	261
12.14.5	Registrar Información con Puntos de Trace	261
12.14.6	Recorriendo una Coincidencia Regex	262
12.15	Glosario	263
13	Regexes y Gramáticas	265
13.1	Una Breve Actualización	265
13.2	Programación Declarativa	267
13.3	Capturas	267
13.4	Reglas Nombradas (o Subreglas)	268
13.5	Gramáticas	271

13.6	Herencia de una Gramática	273
13.7	Objetos de Acciones	274
13.8	Una Gramática para Analizar JSON	275
13.8.1	El Formato JSON	276
13.8.2	Nuestra Muestra de JSON	276
13.8.3	Creando la Gramática de JSON Paso a Paso	277
13.8.4	La Gramática de JSON	279
13.8.5	Agregando Acciones	280
13.9	Herencia y las Gramáticas Mutables	283
13.10	Depuración de Programas	283
13.11	Glosario	285
13.12	Ejercicio: Una Gramática para una Calculadora Aritmética	286
14	Programación Funcional en Raku	287
14.1	Funciones de Orden Superior	287
14.1.1	Breve Actualización: Funciones como Objetos de Primera Clase . . .	287
14.1.2	Subrutinas Anónimas y Lambdas	289
14.1.3	Clausuras	290
14.2	Procesamiento de Listas y Programación de Tuberías	292
14.2.1	Los Operadores Feed y Backward Feed	293
14.2.2	El Metaoperador de Reducción	294
14.2.3	El Hiperoperador	294
14.2.4	Los Operadores Cruz (X) y Zip (Z)	295
14.2.5	Un Resumen de Los Operadores de Listas	296
14.2.6	Creando Nuevos Operadores	296
14.3	Creando tus Funciones Similares a Map	300
14.3.1	Versiones Personalizadas de map, grep, etc.	300
14.3.2	Nuestra Propia Versión de la Función sort	301
14.3.3	Una Versión Iterador de map	302
14.3.4	Una Versión Iterador de grep	303
14.4	Las Construcciones gather y take	306
14.5	Listas Perezosas y el Operador de Secuencia	308

14.5.1	El Operador de Secuencia	308
14.5.2	Listas Infinitas	309
14.5.3	Uso de un Generador Explícito	310
14.6	Currificación y el Operador Whatever	311
14.6.1	Creando una Subrutina Currificada	312
14.6.2	Currificar una Subrutina Existente con el Método <code>assuming</code>	313
14.6.3	Currificando con el Parámetro Whatever Star	313
14.7	Usando un Estilo de Programación Funcional	314
14.7.1	El Algoritmo de Ordenamiento por Mezcla	315
14.7.2	Una Implementación No Funcional de Ordenamiento por Mezcla	315
14.7.3	Una Implementación Funcional de Ordenamiento por Mezcla	317
14.8	Depuración de Programas	318
14.9	Glosario	322
14.10	Ejercicio: Ordenamiento Rápido	323
15	Algunos Consejos Finales	325
15.1	Hazlo Claro, Mantenlo Simple	325
15.2	Qué Hacer y Qué No Hacer	326
15.3	Usa Expresiones Idiomáticas	328
15.4	¿Qué sigue?	331
A	Soluciones a los Ejercicios	333
A.1	Ejercicios del Capítulo 3: Funciones y Subrutinas	333
A.1.1	Ejercicio 3.1: Subrutina <code>alinear-a-derecha</code> (p. 51)	333
A.1.2	Ejercicio 3.2: Subrutina <code>do-twice</code> (p. 51)	335
A.1.3	Ejercicio 3.3: Subrutina <code>imprimir-cuadr</code> (p. 52)	336
A.2	Ejercicios del Capítulo 4: Condicionales y Recursión	338
A.2.1	Subrutina <code>hacer-n-veces</code> , Ejercicio Sugerido en la Sección 4.12 (p. 68)	338
A.2.2	Ejercicio 4.1: <code>Días, Horas, Minutos, y Segundos</code> (p. 72)	338
A.2.3	Ejercicio 4.2: Teorema de Fermat (p. 72)	340
A.2.4	Ejercicio 4.3: ¿Es un triángulo? (p. 72)	340
A.2.5	Ejercicio 4.4: Los Números Fibonacci (p. 73)	341
A.2.6	Ejercicio 4.5: La Subrutina <code>recurse</code> (p. 73)	342

A.3	Ejercicios del Capítulo 5: Funciones Fructuosas	344
A.3.1	Comparar, ejercicio al final de la Sección 5.1 (p. 77)	344
A.3.2	Hipotenusa, ejercicio al final de la Sección 5.2 (p. 79)	344
A.3.3	Operadores de Relación Encadenados (en la Sección 5.4)	345
A.3.4	La Función Ackermann (Ejercicio 5.2)	346
A.3.5	Palíndromos (Ejercicio 5.3)	346
A.3.6	Potencias (Ejercicio 5.4)	347
A.3.7	Encontrando el MCD de Dos Números, Ejercicio 5.5 (p. 91)	348
A.4	Ejercicios del Capítulo 6: Iteración	351
A.4.1	Ejercicio 6.1: Raíz Cuadrada (p. 105)	351
A.4.2	Ejercicio 6.2: Estimado de Pi (p. 105)	352
A.5	Ejercicios del Capítulo 7: Cadenas de Texto	353
A.5.1	Ejercicio en la Sección 7.3: Recorrido de una Cadena de Texto (p. 113)	353
A.5.2	Ejercicio en la Sección 7.3: The Ducklings (p. 113)	354
A.5.3	Ejercicio en la Sección 7.3: Contando las Letras de una Cadena de Texto (p. 113)	355
A.5.4	Sección 7.5: Simulando un Regex con un Bloque (p. 115)	355
A.5.5	Ejercicios en la Subsección 7.7.8: Ejercicios de Regexes (p. 124)	357
A.5.6	Ejercicio en la Sección 7.10: Subrutina es-inversa (p. 132)	360
A.5.7	Ejercicio 7.1: Contando Letras (p. 133)	360
A.5.8	Ejercicio 7.2: Letras Minúsculas (p. 133)	361
A.5.9	Ejercicio 7.3: Cifrado César (p. 135)	362
A.6	Ejercicios del Capítulo 8: Juego de Palabras	364
A.6.1	Ejercicio 8.7: Letras Dobles Consecutivas (p. 147)	364
A.6.2	Ejercicio 8.8: Palíndromos en los Odómetros (p. 147)	365
A.6.3	Ejercicio 8.9: Palíndromos en las Edades (p. 148)	366
A.7	Ejercicios del Capítulo 9: Arrays y Listas	367
A.7.1	Ejercicio en la Sección 9.4: Implementando una Cola (Queue) (p. 156)	367
A.7.2	Ejercicio de la Sección 9.5: Otras Maneras de Modificar un Array (p. 158)	372
A.7.3	Ejercicio de la Sección 9.8: Mapeando y Filtrando los Elementos de una Lista (p. 165)	373
A.7.4	Ejercicio de la Sección 9.12: Técnicas Avanzadas de Ordenamiento (p. 171)	374

A.7.5	Sección 9.1: Suma Anidada (p. 173)	375
A.7.6	Ejercicio 9.2: Suma Acumulativa (p. 173)	376
A.7.7	Ejercicio 9.3: Medio (p. 173)	377
A.7.8	Ejercicio 9.4: Corta (p. 173)	377
A.7.9	Ejercicio 9.5: Subrutina está-ordenada (p. 174)	377
A.7.10	Ejercicio 9.6: Subrutina es-anagrama (p. 174)	378
A.7.11	Ejercicio 9.7: Subrutina tiene-duplicados (p. 174)	379
A.7.12	Ejercicio 9.8: Simulando la Paradoja del Cumpleaños (p. 174)	380
A.7.13	Ejercicio 9.9: Comparando a push y unshift (p. 174)	382
A.7.14	Ejercicio 9.10: Búsqueda Binaria en una Lista (p. 174)	383
A.7.15	Ejercicio 9.11: Invirtiendo Pares de Palabras (p. 175)	385
A.7.16	Ejercicio 9.12: Entrelazando Palabras (p. 175)	388
A.8	Ejercicios del Capítulo 10: Hashes	389
A.8.1	Ejercicio al Final de la Sección 10.1: Un hash es un mapeo (p. 179)	389
A.8.2	Ejercicio 10.1: Almacenar la Lista de Palabras en un Hash (p. 194)	389
A.8.3	Ejercicio 10.2: Memoizando la Función Ackermann (p. 194)	390
A.8.4	Ejercicio 10.3: Encontrando Duplicados con un Hash (p. 194)	391
A.8.5	Ejercicio 10.4: Rotar Pares (p. 194)	392
A.8.6	Ejercicio 10.5: Homófonas (p. 194)	392
A.9	Ejercicio del Capítulo 11: Estructuras de Datos	394
A.9.1	Ejercicio de la Sección 11.2: La Sentencia Switch given ... when (p. p. 197)	394
A.9.2	Ejercicio de la Sección 11.11: Creando Nuevo Operadores p. 208)	396
A.9.3	Ejercicio de la Sección 11.12: Sets, Bags y Mixes (p. 210)	397
A.9.4	Ejercicio de la Sección 11.13: Palabras Aleatorias (p. 211)	399
A.9.5	Ejercicio de la Sección 11.14: Análisis de Markov (p. 213)	399
A.9.6	Ejercicio sobre Código Huffman de la Sección 11.19 (p. 222)	402
A.10	Ejercicios del Capítulo 13: Regexes y Gramáticas	409
A.10.1	Ejercicio de la Sección 13.1: Obteniendo las Fechas de Febrero Correctamente (p. 271)	409
A.10.2	Ejercicio 13.12 (p. 286): Una Gramática para una Calculador Aritmética	411
A.11	Ejercicios del Capítulo 14: Programación Funcional	414
A.11.1	Ejercicio 14.10: Creando una Implementación Funcional de Ordenamiento Rápido	414

Introducción

Bienvenido al arte de la programación de computadoras y al nuevo lenguaje Raku. Este es uno de los primeros libros que usa Raku, un lenguaje de programación poderoso, expresivo, maleable y altamente extensible. Pero este libro es menos acerca de Raku, y más sobre cómo aprender a escribir programas de computadora.

Este libro está destinado para principiantes y no requiere ningún conocimiento previo de programación, aunque espera que aquellos con experiencia de programación puedan aún así beneficiarse.

El Objetivo de este Libro

El objetivo de este libro no es primariamente enseñar Raku, sino enseñar el arte de la programación, usando el lenguaje Raku. Después de haber completado el libro, con suerte deberías ser capaz de escribir programas para resolver problemas relativamente difíciles en Raku, pero mi objetivo principal es enseñar la ciencia de la computación, la programación de software, y la resolución de problemas más allá de solamente enseñar el lenguaje Raku por sí mismo.

Esto significa que no abarcaré cada aspecto de Raku, pero solo un subconjunto (relativamente grande pero incompleto) del lenguaje. Este libro no pretende ser una referencia del lenguaje.

No es posible aprender programación o aprender un nuevo lenguaje de programación solo con leer un libro; la práctica es esencial. Este libro contiene muchos ejercicios. Se te anima a que haga un esfuerzo real para resolverlos. Independiente a si eres capaz de resolver los ejercicios, deberías mirar las soluciones en el Apéndice, dado que, muy a menudo, se sugieren varias soluciones con una extensa discusión de la materia y los asuntos relacionados. Algunas veces, la sección de soluciones del Apéndice introduce ejemplos de temas que serán cubiertos en el siguiente capítulo—y algunas veces cosas que no se discutirán en otras partes del libro. Así que, para sacarle provecho a este libro, te sugiero a que trates de solucionar los ejercicios y revisar las soluciones e intentarlas.

Hay más de mil ejemplos de código en este libro; estúdialos, asegúrate de entenderlos, y ejecútalos. Si es posible, trata de cambiarlos y observa que sucede. Es probable que aprendas mucho de este proceso.

La Historia de este Libro

Hay un algo muy simple que necesitas saber para entender lo que explico en los siguientes párrafos. El lenguaje de programación Raku no siempre se ha llamado Raku. El proyecto comenzó con el nombre Perl 6, dado que fue originalmente concebido como la siguiente versión de Perl, un sucesor de Perl 5. Sin embargo, con el pasar del tiempo, el lenguaje ha evolucionado en un lenguaje sintácticamente diferente a Perl, aunque aún mantiene el espíritu de Perl y avanza los ideales de la comunidad de Perl. Debido a estas diferencias, en Octubre del 2019 se decidió cambiar el nombre de Perl 6 a Raku. El libro que está leyendo ahora originalmente se llamaba *Think Perl 6*. Así que esta nueva edición refleja el cambio de imagen del lenguaje.

En el transcurso de los últimos tres a cuatro años antes del comienzo del 2016, he traducido o adaptado al francés un número de tutoriales y artículos acerca de lo que en aquel entonces se llamaba Perl 6, y también he escrito algunos artículos totalmente nuevos en francés.¹ Juntos, estos documentos representaban para el final del 2015 entre 250 y 300 páginas de material sobre Raku. Para ese entonces, probablemente había hecho público más material sobre este nuevo lenguaje en francés que todos los otros escritores juntos.

A fines del 2015, comencé a pensar que un documento de Raku para principiantes era algo que faltaba y que estaba dispuesto a llevar a cabo. Busqué alrededor y encontré que aparentemente no existía nada igual en inglés. Me vino la idea que, después de todo, sería más útil escribir tal documento inicialmente en inglés, para dárselo a una audiencia más amplia. Así fue que comencé a contemplar escribir una introducción para la programación de Raku destinada para principiantes. En aquel entonces, mi idea era sobre un tutorial de 50 a 70 páginas y comencé a recopilar material e ideas en esta dirección.

Entonces, algo que cambió mis planes ocurrió.

En Diciembre del 2015, algunos amigos míos estaban contemplando traducir al francés *Think Python, Second Edition* de Allen B. Downey². Había leído una versión previa de ese libro excelente y totalmente apoyé la idea de traducirlo³. Como resultado, terminé como un co-traductor y el editor técnico de la traducción al francés de ese libro⁴.

Mientras trabajaba en la traducción al francés del libro sobre Python de Allen, se me ocurrió una idea. En lugar de escribir un tutorial sobre Raku, sería más útil hacer un “traducción a Raku” de *Think Python*. Debido a que estaba en contacto con Allen en el contexto de la traducción al francés, yo le sugerí la idea a Allen, quién acogió favorablemente la idea. Así fue como comencé a escribir este libro a final de Enero del 2016, poco tiempo después de haber completado la traducción al francés de su libro sobre Python.

Tuvo que usar el nombre “Perl 6” en los párrafos anteriores, dado que esa era la manera en la cual fue llamado en ese punto en la historia. De ahora en adelante, usaré el nuevo nombre del lenguaje, *Raku*, a menos que me refiera a eventos prior al cambio de imagen.

De tal manera, este libro es mayormente un derivado de *Think Python* de Allen, pero adaptado a Raku. Como sucedió, es también algo más que una “traducción a Raku” del libro de Allen: con suficiente nuevo material, se ha convertido en un libro totalmente nuevo, con

¹Ver por ejemplo <http://perl.developpez.com/cours/#TutorielsPerl6>.

²Ver <http://greenteapress.com/wp/think-python-2e/>.

³Lo sé, es acerca de Python, no Perl o Raku. Sin embargo, no creo en las “guerras de lenguajes” y pienso que todos debemos aprender de otros lenguajes; para mí, el lema de Perl, “hay más de una forma para hacer algo,” también significa que hacerlo en Python (o cualquier otro lenguaje) es realmente una posibilidad aceptable.

⁴Ver <http://allen-downey.developpez.com/livres/python/pensez-python/>.

una gran deuda al libro de Allen, pero aún un libro nuevo por el cual tomo total responsabilidad. Cualquier error es mío, no de Allen.

Mi esperanza es que esto será útil para la comunidad de Raku, y generalmente para la comunidad de código abierto (*open source*) y la comunidad de la programación de computadoras. En una entrevista, con *LinuxVoice* (July 2015), Larry Wall, el creador de Raku, dijo: “Nosotros pensamos que Perl 6 se podrá aprender como un primer lenguaje”. ¡Esperemos que este libro contribuya a lograr este cometido!

Reconocimientos

Realmente no sé cómo podría agradecerle a Larry Wall al nivel de gratitud que él se merece por haber creado Perl en primer lugar, y más recientemente Raku. ¡Qué seas bendecido por toda la eternidad, Larry!

Y gracias a todos ustedes que fueron parte de esta aventura (en ningún orden en particular), Tom, Damian, chromatic, Nathan, brian, Jan, Jarkko, John, Johan, Randall, Mark Jason, Ovid, Nick, Tim, Andy, Chip, Matt, Michael, Tatsuhiko, Dave, Rafael, Chris, Stevan, Saraty, Malcolm, Graham, Leon, Ricardo, Gurusamy, Scott y muchísimos otros.

Todas mis gracias también a aquellos quienes creyeron en el proyecto de Perl 6 y subsiguientemente Raku, y que lo hicieron posible, incluyendo a aquellos que abandonaron en algún momento pero que contribuyeron por algún tiempo; sé que no fue siempre fácil.

Muchas gracias a Allen Downey, quien amablemente apoyó la idea de adaptar su libro a este lenguaje y me ayudó en muchos aspectos, pero también se abstuvo de interferir en lo que yo ponía en este libro.

Le agradezco sinceramente a la gente de O'Reilly quienes aceptaron la idea de este libro y sugirieron muchas correcciones o mejoras. Quiero agradecer especialmente a Dawn Schanafelt, mi editor de O'Reilly, cuyos consejos han contribuido a hacer este un mejor libro. Muchas gracias también a Charles Roumeliotis, el editor de copia, y Kristen Brown, la editora de producción, quien arregló muchos problemas tipográficos y faltas ortográficas.

De antemano le doy gracias a todos los lectores quienes ofrecieron comentarios o enviaron sugerencias o correcciones, al igual que palabras de aliento.

Si ves algo que necesita ser corregido o que podría ser mejorado, por favor amablemente envía tus comentarios a `think.perl6(at)gmail.com`.

Nota del traductor: Alternativamente, puedes enviar tus correos electrónicos amablemente a `uzluisf(arroba)protonmail.com` si se trata de algo relacionado directamente con esta traducción.

Lista de Contribuciones

Me gustaría agradecer especialmente a Moritz Lenz y Elizabeth Mattijsen, quienes revisaron los detalles en los borradores de este libro y sugirieron un sin número de mejoras y correcciones. Liz invirtió mucho tiempo en una revisión detallada del contenido completo de este libro y le agradezco eternamente por sus numerosos y muy útiles comentarios. Gracias también a Timo Paulssen y ryanschoppe quienes también revisaron los primeros

borradores y proveyeron algunos comentarios útiles. Muchísimas gracias también a Uri Guttman, quien revisó este libro y sugirió un gran número de pequeñas correcciones y mejoras un poco antes de la publicación.

Kamimura, James Lenz, y Jeff McClelland cada uno sometieron algunas correcciones en la lista de errata en el sitio web de O'Reilly. zengargoyle señaló un carácter falso en un regex y lo arregló en el repositorio Github del libro. zengargoyle también sugirió una aclaración en el capítulo acerca de la programación funcional. Otro James (segundo nombre que no conozco) sometió una errata al sitio web de O'Reilly. Mikhail Korenev sugirió correcciones precisas para tres fragmento de código. Sébastien Dorey, Jean Forget, y Roland Schmitz enviaron algunos correos electrónicos sugiriendo algunas correcciones o mejoras útiles. Luis F. Uceta tradujo este libro al español y corrigió varios errores tipográficos en el repositorio de Github. Gil Magno, zoffixznet y Joaquin Ferrero también sugirieron varias correcciones en Github. Threadless-screw arregló un par de referencias defectuosas acerca de los hashes. Boyd Duffee encontró dos muestras de código que funcionaban bien cuando las escribí, pero que ya no funcionan, debido a cambios en la implementación.

Parte I

Comenzando con lo Básico

Este libro está dividido en dos partes. La razón principal por esta decisión es que yo quería hacer una distinción entre las nociones relativamente básicas que cualquier programador necesita al usar Raku y los conceptos avanzados que un buen programador necesita saber y que pueden ser necesarios en el día a día de un programador.

Los primeros once capítulos (aproximadamente 200 páginas) que constituyen la primera parte están diseñados para enseñar los conceptos básicos que cada programador debe conocer: variables, expresiones, condicionales, recursión, precedencia de operadores, bucles, etc., y también las estructuras de datos básicas usadas comúnmente, y los algoritmos más útiles. Creo que estos capítulos pueden ser la base de un curso introductorio a la programación para un semestre.

Por supuesto, el profesor o instructor que desee utilizar este material es libre de saltar algunos detalles en la Parte 1 (y también incluir de la Parte 2). Por lo menos, he incluido algunas recomendaciones sobre como este libro podría ser usado para enseñar programación usando el lenguaje Raku.

La segunda parte se enfoca en paradigmas diferentes y algunas técnicas avanzadas de programación que son en mi opinión de gran importancia, pero que deberían ser estudiadas en el contexto de un segundo semestre más avanzado.

Por ahora, comencemos con lo básico. Espero que disfrutes la travesía.

La Forma del Programa

La meta de este libro es enseñarte a pensar como un científico de la computación. Esta manera de pensar combina algunas de las mejores características de las matemáticas, la ingeniería, y las ciencias naturales. Al igual que los matemáticos, los científicos de la computación usan lenguajes formales para denotar ideas (específicamente computaciones). Al igual que los ingenieros, ellos diseñan cosas, ensamblan componentes en sistemas y evalúan las compensaciones entre las alternativas. Al igual que los científicos, ellos observan el comportamiento de sistemas complejos, formulan hipótesis, y prueban las predicciones.

La habilidad más importante de un científico de la computación es la **resolución de problemas**. La resolución de problemas se refiere a la habilidad de formular problemas, pensar creativamente sobre las soluciones, y expresar una solución clara y precisamente. Como sucede, el proceso de aprender a programar es una oportunidad excelente para practicar las habilidades de resolución de problemas. Por esta razón, este capítulo se titula, “La Forma del Programa”.

En un nivel, aprenderás a programar, la cual es una habilidad útil por sí misma. En otro nivel, aprenderás a usar la programación como un medio para un fin. A medida que avancemos, este fin se esclarecerá.

1.1 ¿Qué es un Programa?

Un **programa** es una secuencia de instrucciones que especifica como realizar una computación. La computación podría involucrar algo matemático, tal como resolver un sistema de ecuaciones o encontrar las raíces de un polinomio, pero también puede ser algo simbólico, tal como buscar y reemplazar texto en un documento, o algo gráfico, como el procesamiento de una imagen o la reproducción de un video.

Los detalles lucen diferentes en lenguajes distintos, pero hay algunas instrucciones básicas que aparecen en casi todos los lenguajes:

Entrada Obtener datos desde el teclado, un archivo, la red, un sensor, un chip de GPS o algún otro dispositivo.

Salida Mostrar información en la pantalla, guardarla en un archivo, enviarla a través de la red, actuar en un dispositivo mecánico, etc.

Matemática Realizar operaciones matemáticas básicas tales como la adición y la multiplicación.

Ejecución condicional Revisar ciertas condiciones y ejecutar el código apropiadamente.

Repetición Realizar alguna acción repetidamente, usualmente con algún tipo de variación.

Lo creas o no, eso es todo. Cada programa que has usado, sin importar que tan complicado sea, está compuesto de instrucciones que lucen exactamente como estas. Por lo tanto, puedes imaginarte la programación como el proceso de fragmentar una tarea grande y compleja en piezas más pequeñas las cuales se pueden ejecutar con una de estas operaciones básicas.

Al usar o llamar estas piezas, es posible crear varios niveles de *abstracción*. Probablemente te han dicho que las computadoras solamente usan 0s y 1s al nivel más fundamental; pero usualmente no nos preocupamos acerca de esto. Cuando usamos un procesador de texto para escribir una carta o un reporte, estamos interesados en archivos con textos y algunas instrucciones de formato, y con comandos para cambiar el archivo o imprimirlo;afortunadamente, no tenemos que preocuparnos sobre los 0s y 1s; el procesador de texto nos ofrece una vista más general (archivos, comandos, etc.) que oculta todos los detalles insignificantes del usuario.

Similarmente, cuando escribimos un programa, usualmente usamos y/o creamos varias capas de abstracción, para que, por ejemplo, una vez que hayamos creado una tarea pequeña que consulta una base de datos y guarda la información relevante en la memoria, no tengamos que preocuparnos sobre los detalles técnicos de la tarea. Podemos usarla como una caja negra la cual realizará la operación deseada para nosotros. La esencia de la programación es en gran parte este arte de crear estas capas sucesivas de abstracción de tal manera que realizar tareas de niveles más altos se vuelva relativamente más fácil.

1.2 Ejecutando Raku

Uno de los retos de comenzar con Raku es que podrías tener que instalar Raku y cualquier software relacionado en tu computadora. Si estás familiarizado con tu sistema operativo, y especialmente te sientes cómodo con el intérprete de comandos (o shell), no tendrás problemas instalando Raku. Para los principiantes, puede ser un poco difícil aprender sobre la administración de sistema y programación al mismo tiempo.

Para evitar ese problema, podrías comenzar ejecutando Raku en el navegador. Podrías usar un motor de búsquedas para encontrar tal sitio. Por el momento, la forma más fácil es probablemente conectarse al sitio <https://repl.it/languages/raku>, donde puedes entrar código Raku en el editor de texto de la ventana principal, ejecutarlo, y observar el resultado en la ventana para la salida de texto. Otra posibilidad que podrías probar es el sitio <https://glot.io/new/raku>. Probablemente existen otros interpretadores de Raku en el navegador.

Tarde o temprano, sin embargo, tendrás que instalar Raku en tu computadora.

La manera más fácil de instalar Raku en tu sistema es descargar Rakudo Star (una distribución de Raku que contiene el **compilador** Rakudo de Raku, documentación y módulos útiles): sigue las instrucciones para tu sistema operativo en <http://rakudo.org/how-to-get-rakudo/> y en <https://raku.org/downloads/>. También podrías probar paquetes de terceros usando Docker o Chocolatey, por ejemplo.

Al momento de escribir este libro, la especificación más reciente del lenguaje es Raku versión 6d (v6.d), y el lanzamiento más reciente disponible para la descarga es Rakudo Star 2019.03; los ejemplos en este libro deberían funcionar con esta versión. Puedes encontrar la versión instalada al ejecutar el siguiente comando en el intérprete de comandos de tu sistema operativo:

```
$ raku -v
This is Rakudo Star version 2019.03.1 built on MoarVM version 2019.03
implementing Perl 6.d.
```

No obstante, deberías descargar e instalar la versión más reciente que puedas encontrar. La salida (advertencias, mensajes de errores, etc.) que obtendrás de tu versión de Raku podría en algunos casos diferir de la que se encuentra en este libro, pero estas diferencias deberían ser esencialmente cosméticas.

Raku se deriva de Perl, e intenta avanzar los grandes ideales de la comunidad de Perl. Pero, comparado con Perl 5, Raku no es una nueva versión de Perl. El mismo no intenta reemplazar a Perl 5. Raku es realmente un nuevo lenguaje de programación, con una sintaxis similar a Perl 5.

El **intérprete** de Raku es un programa que lee y ejecuta código de Raku. Algunas veces se le llama REPL (**en.** “read, evaluate, print, loop”). Dependiendo de tu entorno, podría iniciar el intérprete al cliquear un ícono, o al escribir `raku` en el intérprete de comandos.

Cuando comience, deberías ver algo similar a esto:

```
To exit type 'exit' or '^D'
(Possibly some information about Raku and related software)
>
```

La última línea con `>` es un **prompt** que indica que el REPL está listo para que entres código. Si tecleas una línea de código y presiona la tecla Enter, el intérprete muestra el resultado:

```
> 1 + 1
2
>
```

Puedes teclear `exit` en el prompt de REPL para salir del REPL.

Ahora estás listo para comenzar. De aquí en adelante, asumimos que sabes como iniciar el REPL de Raku y ejecutar código.

1.3 El Primer Programa

Tradicionalmente, el primer programa que escribes en un nuevo lenguaje se llama “Hello, World” porque todo lo que hace es mostrar las palabras “Hello, World”. En Raku, luce de la siguiente manera:

```
> say "Hello, World";  
Hello, World  
>
```

Este es un ejemplo de lo que usualmente se conoce como una **sentencia de impresión**, aunque actualmente no imprime nada sobre el papel y ni siquiera usa la palabra clave `print`¹ (Las palabras claves son palabras que tienen un significado especial en el lenguaje y son usadas por el interpretador para reconocer la estructura del programa). La sentencia de impresión muestra un resultado en la pantalla. En este caso, el resultado son las palabras Hello World. Las comillas inglesas (o comillas dobles) en el programa indican el comienzo y final del texto a ser mostrado; ellas no aparecen en el resultado.

El punto y medio “;” al final de la línea indica que este es el final de la sentencia actual. Aunque un punto y medio no es técnicamente necesario al ejecutar código simple en el REPL, es usualmente necesario cuando se escribe un programa con varias líneas de código, así que deberías ir acostumbrándote a finalizar instrucciones de código con un punto y medio.

Otros lenguajes de programación podrían requerir paréntesis alrededor de la sentencia que se quiere imprimir, pero esto es usualmente no necesario en Raku.

1.4 Operadores Aritméticos

Después de “Hello, World”, el siguiente paso es aritmética. Raku provee **operadores**, los cuales son símbolos especiales que representan computaciones tales como la adición y la multiplicación.

Los operadores `+`, `-`, `*`, y `/` realizan adición, sustracción, multiplicación y división, como en los siguientes ejemplos en el REPL:

```
> 40 + 2  
42  
> 43 - 1  
42  
> 6 * 7  
42  
> 84 / 2  
42
```

Dado que usamos el REPL, no necesitamos una sentencia de impresión explícita en estos ejemplos, debido a que el REPL automáticamente imprime el resultado de las sentencias. En un programa real, necesitarías una sentencia de impresión para mostrar el resultado,

¹Raku también tiene una función `print`, pero la función integrada `say` es usada aquí porque agrega un carácter de nueva línea a la salida.

como veremos más adelante. Similarmente, si ejecutas sentencias de Raku en el navegador mencionado en la Sección 1.2, necesitarás una sentencia de impresión para mostrar el resultado de estas operaciones. Por ejemplo:

```
say 40 + 2;    # -> 42
```

Finalmente, el operador `**` realiza la potenciación; es decir que eleva un número a un exponente:

```
> 6**2 + 6
42
```

En otros lenguajes, el signo de intercalación (“^”) o el acento circunflejo es usado para la potenciación, pero en Raku se utiliza para otros propósitos.

1.5 Valores y Tipos

Un **valor** es una de las cosas básicas con la cual un programa funciona, como una letra o un número. Entre algunos de los valores que hemos visto hasta ahora están 2, 42, y "Hello, World".

Estos valores pertenecen a diferentes **tipos**: 2 es un número **entero**, `40 + 2` es también un entero, `84/2` es un número **racional**, y `'Hello, World'` es una **cadena de texto**, llamada así porque los caracteres que contiene están unidos conjuntamente.

Si no estás seguro del tipo que un valor tiene, Raku puede decirte:

```
> say 42.WHAT;
(Int)
> say (40 + 2).WHAT;
(Int)
> say (84 / 2).WHAT;
(Rat)
> say (42.0).WHAT
(Rat)
> say ("Hello, World").WHAT;
(Str)
>
```

En estas instrucciones, `a.WHAT` se le conoce como un método de introspección, el cual es un tipo de método que te dice *a cual* tipo la expresión precedente pertenece. `42.WHAT` es un ejemplo de la sintaxis del punto usada para la invocación de método: invoca el método integrado `.WHAT` sobre la expresión “42” (el invocante) y provee la función `say` con el resultado de esta invocación, que en este caso es el tipo de la expresión.

No resulta sorprendente que los números enteros pertenecen al tipo `Int`, las cadenas de texto pertenecen a `Str`, y los números racionales pertenecen a `Rat`.

Aunque `40 + 2` y `84 / 2` parecen arrojar el mismo resultado (42), la primera expresión devuelve un entero (`Int`), y la segunda devuelve un número racional (`Rat`). El número `42.0` es también un número racional.

Estos valores son cadenas de texto porque están definidos dentro de comillas dobles. Aunque Raku usualmente realizará las conversiones necesarias para ti, es generalmente buena práctica no usar comillas dobles si tu valor está supuesto a ser un número.

Cuando escribes un número entero largo, podrías estar tentado a usar comas entre grupos de dígitos, como en 1,234,567. Esto no es un *entero* legal en Raku, pero sí es una expresión legal:

```
> 1,234,567
(1 234 567)
>
```

¡Eso es actualmente una lista de tres números enteros diferentes, y no lo que esperábamos!

```
> say (1,234,567).WHAT
(List)
```

Raku interpreta 1,234,567 como una secuencia de tres números enteros separados por coma. Como veremos más adelante, la coma es un separador usado para construir listas.

No obstante, puedes separar grupos de dígitos con una barra baja “_” para mejorar la legibilidad y para obtener un entero propio:

```
> 1_234_567
1234567
> say 1_234_567.WHAT
(Int)
>
```

1.6 Lenguajes Formales y Naturales

Los **lenguajes naturales** son los lenguajes que las personas hablan, como el inglés, el español y el francés. Ellos no fueron diseñados por personas (aunque la gente trata de imponer cierto orden sobre ellos); ellos evolucionaron naturalmente.

Los **lenguajes formales** son lenguajes que son diseñados por personas para aplicaciones específicas. Por ejemplo, la notación que los matemáticos utilizan es un lenguaje formal que es particularmente buena para denotar relaciones entre números y símbolos. Los químicos usan un lenguaje formal para representar la estructura química de las moléculas. Y más importante:

Los lenguajes de programación son lenguajes formales que han sido diseñados para expresar computaciones.

Los lenguajes formales tienen usualmente **reglas sintácticas** estrictas que gobiernan la estructura de las sentencias. Por ejemplo, en las matemáticas la sentencia $3 + 3 = 6$ es sintácticamente correcta pero $3 + = 3\$6$ no lo es. En química H_2O es una fórmula sintácticamente correcta, pero $_2Zz$ no lo es.

Las reglas sintácticas vienen en dos formas, pertenecientes a los **tókenes** y la **estructura**. Los tókenes son los elementos básicos del lenguaje, tales como las palabras, números, y

elementos químicos. Uno de los problemas con $3+ = 3\$6$ es que \$ no es un token legal en matemáticas (al menos eso es lo que sé). Similarmente, ${}_2Zz$ no es legal porque no hay elemento químico con la abreviación Zz.

La estructura es el segundo tipo de la regla sintáctica, la cual se concierne con la forma en que los tókenes son combinados. La ecuación $3+ = 3$ es ilegal en matemáticas porque, aunque $+$ y $=$ son tókenes legales, no puedes tenerlos uno detrás del otro. De manera similar, en una fórmula química, el subíndice que representa el número de átomos en un compuesto químico viene después del nombre del elemento, no antes.

Esto es una oración en español bien estructurada con tókenes inválidos. En esta oración todos tókenes los válidos son, pero la inválida estructura es.

Cuando lees una oración en español o una sentencia en un lenguaje formal, debes descifrar la estructura (aunque en un lenguaje natural lo haces inconscientemente). Este proceso es conocido como **análisis sintáctico** (en. parsing).

Aunque los lenguajes formales y los lenguajes naturales comparten muchas características en común—tókenes, estructura, y sintaxis—también hay algunas diferencias:

Ambigüedad Los lenguajes naturales están llenos de ambigüedades, y las personas lidian con ellas a través del uso de contexto y cualquier información adicional. Los lenguajes formales son diseñados para no tener ambigüedades, de tal manera que cualquier sentencia tiene exactamente un solo significado.

Redundancia Para compensar por las ambigüedades y reducir los malentendidos, los lenguajes naturales hacen uso de muchas redundancias. Como resultado, ellos tienden a ser verbosos. En cambio, los lenguajes formales son menos redundantes y más concisos.

Literalidad Los lenguajes naturales están llenos de modismos y metáforas. Si decimos, “Hablar por los codos”, es probable que nadie está hablando por los codos literalmente (este modismo significa que una persona habla demasiado). Los lenguajes formales hacen referencias exactas a lo que quieren decir.

Debido a que todos crecemos hablando lenguajes naturales, es un poco difícil ajustarse a los lenguajes formales. La diferencia entre un lenguaje formal y un lenguaje natural es como la diferencia entre la poesía y la prosa, pero más definida:

Poesía Las palabras son usadas tanto por sus sonidos como sus significados, y el poema completo crea un efecto o respuesta emocional. En este caso, la ambigüedad no es solo común sino usualmente deliberada.

Prosa El significado literal de las palabras es más importante, y la estructura contribuye más al significado. La prosa es más amena para el análisis que la poesía aunque tiende a ser ambigua en muchas ocasiones.

Programa El significado de un programa de computadora es literal y no tiene ambigüedades. Dicho programa puede ser comprendido enteramente a través del análisis de los tókenes y la estructura.

Los lenguajes formales son mucho más densos que los lenguajes naturales, y por lo tanto se tarda más en leerlos. De igual manera, la estructura es importante y algunas veces, no siempre es eficiente leerlos de arriba hacia abajo o de izquierda a derecha. Por lo contrario,

mejor aprende a parsear el programa en tu cabeza, identificando los tókenes e interpretando la estructura. Finalmente, los detalles importan. Pequeños errores en la ortografía y la puntuación que no son de mucha importancia en los lenguajes naturales, pueden hacer una gran diferencia en un lenguaje formal.

1.7 Depuración de Programas

Los programadores cometen errores. A los errores de programación se les conoce usualmente como **bichos** (en. bugs) y el proceso de eliminación de estos errores se llama **depuración de programas** (en. debugging).

La programación, y especialmente la depuración de programas, algunas veces puede aflorar emociones fuertes. Si estás batallando con un error difícil, podrías sentirte enojada/o, avergonzada/o o desanimada/o.

Existe evidencia que sugiere que las personas responden naturalmente a las computadoras como si fueran personas. Cuando funcionan adecuadamente, las consideramos nuestras compañeras, y cuando son obstinadas y groseras, nosotros nos dirigimos hacia ellas de la misma manera que lo haríamos con personas obstinadas y groseras².

Prepararse de antemano para lidiar con estas reacciones puede ser beneficioso. Una estrategia es tratar de pensar acerca de la computadora como si fuera un empleado con ciertas fortalezas, como la velocidad y la precisión, y con debilidades particulares tales como la falta de empatía y la inhabilidad para comprender la perspectiva general.

Tu trabajo es ser un buen gerente: debes encontrar maneras de tomar ventaja de las fortalezas y mitigar las debilidades. Y encontrar maneras de usar tus emociones para familiarizarte y envolverte con el problema sin dejar que tus reacciones interfieran con tu habilidad de trabajar efectivamente.

Aprender a depurar puede ser frustrante, pero es una habilidad valiosa que será muy útil para muchas actividades más allá de la programación. Al final de cada capítulo hay una sección, como esta, con sugerencias para la depuración. ¡Espero que sean útiles!

1.8 Glosario

Resolución de problemas El proceso de formular un problema, encontrar una solución y expresarla adecuadamente.

Abstracción Una manera de proveer una perspectiva de alto nivel de una tarea y ocultar los detalles técnicos con el propósito de hacer la tarea más simple.

Interpretador Un programa que lee otro programa y lo ejecuta.

Compilador Un programa que lee otro programa y lo transforma en código ejecutable de computadora; solía ser que había una gran diferencia entre lenguajes interpretados y lenguajes compilados, sin embargo esta distinción se ha vuelto más difusa en las últimas dos décadas.

²Reeves and Nass, *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*, (Center for the Study of Language and Information, 2003).

Prompt El interpretador muestra caracteres en la pantalla para indicar que está listo para recibir entrada de texto por el usuario.

Programa Un conjunto de instrucciones que especifica una computación.

Sentencia de impresión Una instrucción que causa que el interpretador de Raku muestre un valor en la pantalla.

Operador Un símbolo especial que representa una simple computación como la adición, la multiplicación, o la concatenación de una cadena de texto.

Valor Una de las unidades básicas de dato, como un número o una cadena de texto, que un programa manipula.

Tipo Una categoría de valores. Los tipos que hemos visto hasta ahora son números enteros (tipo `Int`), números racionales (tipo `Rat`), y cadenas de texto (tipo `Str`).

Entero (Int) Un tipo que representa valores enteros.

Racional (Rat) Un tipo que representa números con partes fraccionales. Internamente, Raku almacena un racional como dos números enteros que representan respectivamente el numerador y el denominador de una fracción.

Cadena de texto (Str) Un tipo que representa secuencias de caracteres.

Lenguaje natural Cualquiera de los lenguajes que las personas hablan y que han evolucionado naturalmente.

Lenguaje formal Cualquiera de los lenguajes que las personas han diseñado para propósitos específicos tales como la representación de ideas matemáticas o programas de computadoras; todos los lenguajes de programación son lenguajes formales.

Token Uno de los elementos básicos de la estructura sintáctica de un programa. El concepto es análogo a una palabra en un lenguaje natural.

Sintaxis Las reglas que dictan la estructura de un programa.

Análisis sintáctico Examinar un programa y analizar su estructura sintáctica.

Bicho Un error en un programa.

Depuración El proceso por el cual se encuentran y corrigen errores de programación.

1.9 Ejercicios

Ejercicio 1.1. *Es una buena idea que lea este libro al frente de una computadora para que puedas intentar los ejemplos según avanzas.*

Cuando experimentes con una característica nueva, trata de cometer errores. Por ejemplo, en el programa “Hello, world!”, ¿qué pasa si no incluye una de las comillas dobles? ¿Qué pasa si no deleetras `say` correctamente?

Este tipo de experimento te ayuda a recordar lo que lees; además te ayuda cuando está programando porque te acostumbras y aprendes lo que significan los mensajes de error. Es mejor cometer errores ahora a propósito que cometerlos después por accidente.

Ten pendiente que la mayoría de los ejercicios en este libro tienen soluciones que se encuentran en el apéndice. No obstante, la intención de los ejercicios en este capítulo y en el siguiente no es para resolver un problema actual. Están diseñados para que experimentes con el interpretador de Raku; no hay una solución correcta, solo intenta lo que se propone en los ejercicios para obtener un sentido de cómo funciona.

1. Si intentas imprimir una cadena de texto, ¿qué pasa si no incluye una de las comillas dobles, o ambas?
2. Puedes usar el signo de menos para hacer números negativos como -2 . ¿Qué pasa si pones un signo de adición frente al número? ¿Qué acerca de $2++2$?
3. En notación matemática, los ceros al principio son aceptables, como en 02. ¿Qué pasa si intentas esto en Raku?
4. ¿Qué pasa si tienes dos valores sin ningún operador entre ellos, como en `say 2 2`;

Ejercicio 1.2. Comienza con el interpretador REPL de Raku y úsalo como una calculadora.

1. ¿Cuántos segundos hay en 42 minutos, 42 segundos?
2. ¿Cuántas millas hay en 10 kilómetros? Pista: Hay 1.61 kilómetros en una milla.
3. Si haces una carrera de 10 kilómetros en 42 minutos, 42 segundos, cuál es tu rapidez promedio (tiempo por milla en minutos y segundos)? ¿Cuál es tu rapidez promedio en millas por horas?

Variables, Expresiones y Sentencias

Una de las más poderosas características de un lenguaje de programación es la habilidad para manipular **variables**. En términos generales, una variable es el nombre que hace referencia a un valor. Sería más preciso decir que una variable es un contenedor que tiene un nombre y almacena un valor.

2.1 Sentencias de Asignación

Una **sentencia de asignación** usa el signo de igualdad = y asigna un valor a una variable, pero, antes de que asignes un valor a una variable, debes primero crear la variable a través de una declaración (si todavía no existe):

```
> my $mensaje;           # declaración de una variable, aún sin valor
> $mensaje = 'And now for something completely different';
And now for something completely different
> my $numero = 42;        # declaración de una variable y asignación
42
> $numero = 17;           # nueva asignación
17
> my $phi = 1.618033988;
1.618033988
>
```

Este ejemplo lleva a cabo cuatro asignaciones. La primera asigna una cadena de texto a una nueva variable llamada `$mensaje`, la segunda asigna el número entero 42 a `$numero`, la tercera reasigna el entero 17 a `$numero`, y la cuarta asigna el valor aproximado del número áureo a `$phi`.

Aquí hay dos características sintácticas importantes que debes entender.

Primero, en Raku, los nombres de las variables comienzan con un símbolo conocido como un *sigilo*, es decir, un carácter no alfanumérico como \$, @, %, &, y otros más. Este carácter especial nos informa a nosotros y al compilador de Raku (el programa que lee el código de nuestro programa y lo transforma en instrucciones de computadoras) el tipo de variable que es. Por ejemplo, el carácter \$ indica que las variables más arriba son todas *variables escalares*, lo cual significa que ellas pueden almacenar un solo valor en cualquier momento. Veremos más adelante otros tipos de variables las cuales pueden contener más de un solo valor.

Segundo, nota como las tres variables más arriba son introducidas con la palabra clave *my*, que es una manera de declarar una variable nueva. Cuando tu creas una variable nueva en Raku, necesitas *declararla*, o sea, le dejas saber a Raku que vas a usar una variable nueva; esto se hace comúnmente con la palabra clave *my*, la cual declara una variable *léxica*. Explicaremos luego lo que es una variable léxica pero por ahora, digamos que te permite crear una variable local en una parte limitada de tu código. Una de las consecuencias buenas del requerimiento de declarar variables antes de que las uses es que, si tu cometes un error al escribir el nombre de una variable, el compilador usualmente será capaz de decirte que estás usando una variable que no ha sido declarada y como resultado te ayudará a encontrar tu error. Esto tiene grandes complicaciones las cuales examinaremos más tarde.

Cuando escribimos al inicio de esta sección que una variable debe ser declarada antes de ser usada (o solo cuando se usa), esto significa que la declaración tiene que ser antes (o al punto de) del primer uso de la variable en el archivo de texto que contiene el programa. Veremos más adelante que los programas no son ejecutados necesariamente de arriba hacia abajo en el orden en el que las líneas o código aparecen en el programa; aún así, la declaración de la variable debe ocurrir antes de su uso en el archivo de texto que contiene el programa.

Si olvidas declarar una variable, obtienes un error sintáctico:

```
> $numero = 5;
===SORRY!=== Error while compiling <unknown file>
Variable '$numero' is not declared
at <unknown file>:1
-----> <BOL><HERE>$numero = 5;
>
```

Ten presente que podrías obtener mensajes diferentes de error dependiendo de la versión de Rakudo que ejecutes. El mensaje más arriba se obtuvo en Febrero 2016; con una versión más nueva (Octubre 2016), el mismo error se muestra en una forma algo más organizada:

```
>
> $numero = 5;
===SORRY!=== Error while compiling:
Variable '$numero' is not declared
at line 2
-----> <BOL><HERE>$numero = 5;
>
```

Una manera común de representar variables en papel es escribir el nombre con una flecha apuntando a su valor. Este tipo de dibujo se le conoce como un **diagrama de estado** porque muestra en qué estado cada una de las variables se encuentra (imagínalo como el estado mental de la variable). La Figura 2.1 muestra el resultado del ejemplo anterior.

```
$mensaje → 'And now for something completely different'
$número  → 17
$phi     → 1.618033988
```

Fig. 2.1: Diagrama de estado.

2.2 Nombres de las Variables

Los programadores generalmente eligen nombres significativos para sus variables—los nombres documentan el uso de la variable.

Los nombres de las variables pueden ser tan largos como tu desees. Ellos pueden contener letras y números, pero las variables definidas por el usuario no pueden comenzar con un número. Los nombres de las variables son sensibles al uso de letras mayúsculas y minúsculas, es decir, `$mensaje` no es la misma variable que `$Mensaje` o `$MENSAJE`. Es legal usar letras mayúsculas, pero es convencional usar solo letras minúsculas para la mayoría de los nombres de las variables. No obstante, algunas personas prefieren usar `$TipoTítulo` (en. camel case) para los nombres de sus variables o hasta `$MAYÚSCULAS` para algunas variables especiales.

A diferencia de muchos otros lenguajes de programación, Raku no requiere que las letras y dígitos en los nombres de las variables sean únicamente ASCII. Puedes usar cualquier tipo de letras Unicode, es decir, letras de cualquier otro lenguaje en el mundo, así que, por ejemplo, `$brücke`, `$payé` o `$niño` son nombres de variables válidos, los cuales pueden ser usados por programadores que hablan otros idiomas diferentes al inglés (siempre y cuando estos caracteres Unicode sean correctamente manipulados por tu editor de texto y tu configuración de pantalla). De la misma manera, en vez de usar `$phi` para el nombre de la variable del número áureo, podríamos haber usado la *letra griega minúscula phi*, φ (Unicode code point U+03C6). Igualmente, podríamos haber usado la *letra griega minúscula pi*, π , para la muy conocida razón de la circunferencia del círculo al diámetro:

```
> my $φ = (5 ** .5 + 1)/2;      # número áureo
1.61803398874989
> say 'Variable $φ = ', $φ;
Variable $φ = 1.61803398874989
> my $π = 4 * atan 1;
3.14159265358979
> # podrías también las constante integrada pi o π:
> say pi
3.14159265358979
```

El carácter de barra baja, `_`, puede aparecer en cualquier parte del nombre de una variable. Usualmente se usa en nombres con múltiples palabras, tal como `$tu_nombre` o `$airspeed_of_unladen_swallow`.

Hasta puedes usar guiones para crear lo que se conoce como “kebab case”¹ y nombrar esas variables `$tu_nombre` o `$airspeed-of-unladen-swallow`, y esto las puede hacer más legibles: un guion `-` es válido en una variable siempre y cuando sea inmediatamente seguido

¹Llamado así porque las variables aparecen estar atravesadas como las piezas de comida preparadas para una barbacoa.

por un carácter alfanumérico. Por ejemplo, `$doble-clic` or `$la-niña` son nombres legítimos de variables. Análogamente, puedes usar un apóstrofo `'` (también conocido como comilla simple) entre las letras, así que `$isn't` o `$o'brien's-age` son identificadores válidos.

Si les da un nombre ilegal a una variable, obtienes un error sintáctico:

```
> my $76trombones = 'big parade'
===SORRY!=== Error while compiling <unknown file>
Cannot declare a numeric variable
at <unknown file>:1
-----> my $76<HERE>trombones = "big parade";
>
> my $more$ = 100000;
===SORRY!=== Error while compiling <unknown file>
Bogus postfix
at <unknown file>:1
-----> my $more<HERE>$ = 100000;
(...)
```

`$76trombones` es ilegal porque comienza con un número. `$more$` es ilegal porque contiene un carácter ilegal, `$`.

Si alguna vez has usado otro lenguaje de programación y te has tropezado con un mensaje terso tal como "SyntaxError: invalid syntax", notarás que los diseñadores de Raku han hecho un gran esfuerzo para proveerte con mensajes de error que sean detallados, útiles y significativos.

Muchos lenguajes de programación tienen *palabras claves* o *palabras reservadas* que son parte de la sintaxis, tales como `if`, `while`, o `for`, y por lo tanto no pueden ser usadas para identificar variables porque eso crearía ambigüedad. Dicho problema no existe en Raku: dado que los nombres de las variables comienzan con un sigilo, el compilador es siempre capaz de diferenciar entre una palabra clave y una variable. Nombres tales como `$if` o `$while` son sintácticamente identificadores válidos de variables en Raku (si estos nombres hacen sentido es un asunto diferente).

2.3 Expresiones y Sentencias

Una **expresión** es una combinación de términos y operadores. Los términos pueden ser variables o literales, es decir, valores constantes tales como un número o una cadena de texto. Al igual que un valor, una variable es también considerada una expresión. Así que todo las siguientes son expresiones legales:

```
> 42
42
> my $n = 17;
17
> $n;
17
> $n + 25;
42
>
```

Cuando escribes una expresión en el prompt, el interpretador la **evalúa**, lo que significa que encuentra el valor de la expresión. En este ejemplo, `$n` tiene el valor 17 y `$n + 25` tiene el valor 42.

Una **sentencia** es una unidad de código que tiene un efecto, tal como crear una variable o mostrar un valor, y usualmente necesita terminar con un punto y coma ; (aunque el punto y coma puede algunas veces ser omitido como veremos más adelante):

```
> my $n = 17;
17
> say $n;
17
```

La primera línea es una sentencia de asignación que asigna un valor a `$n`. La segunda línea es una sentencia de impresión que muestra el valor de `$n`.

Cuando escribes una sentencia y después presiona Enter, el interpretador la **ejecuta**, lo que significa que hace lo que la sentencia dicta.

Una sentencia puede ser combinada con expresiones usando los operadores aritméticos. Por ejemplo, podrías escribir:

```
> my $respuesta = 17 + 25;
42
> say $respuesta;
42
```

El símbolo `+` es obviamente el operador de adición y, después de la sentencia de asignación, la variable `$respuesta` contiene el resultado de la adición. Los términos en cada lado del operador (aquí 17 y 25) son usualmente llamados los *operandos* de la operación (una adición en este caso).

Nota que el REPL actualmente muestra el resultado de la asignación (la primera línea con “42”), así que la sentencia de impresión no era realmente necesaria en este ejemplo *en el REPL*; desde aquí en adelante, para ser breve, generalmente omitiremos las sentencias de impresión en los ejemplos donde el REPL muestra los resultados.

En algunos casos, puedes querer añadir algo a una variable y asignar el resultado a la misma variable. Esto se puede escribir así:

```
> my $respuesta = 17;
17
> $respuesta = $respuesta + 25;
42
```

Aquí, `$respuesta` es primero declarada con un valor de 17. La siguiente sentencia asigna a `$respuesta` el valor actual de `$respuesta` (por ejemplo, 17) + 25. Esta operación es tan común en Raku, como en otros lenguajes de programación, que tiene un atajo:

```
> my $respuesta = 17;
17
> $respuesta += 25;
42
```

El operador `+=` combina el operador de la adición aritmética y el operador de asignación para modificar un valor y aplicar el resultado a una variable en un solo paso, así que `$n += 2` quiere decir: toma el valor actual de `$n`, agrega 2, y asigna el resultado a `$n`. Esta sintaxis funciona con todos los operadores aritméticos. Por ejemplo, `--` realiza una sustracción y una asignación, `*` una multiplicación y una asignación, etc. Además de los operadores aritméticos, también puede ser usado con otros operadores tal como el operador de la concatenación de cadena de texto que veremos más adelante.

Agregar 1 a una variable es una versión muy común de esto, y como resultado, existe un atajo, el operador de *incremento*, el cual incrementa su argumento por uno, y devuelve el valor incrementado:

```
> my $n = 17;
17
> ++$n;
18
> say $n;
18
```

Esto se conoce como el operador de incremento prefijo, porque el operador `++` se coloca antes que la variable a ser incrementada. También hay una versión sufija (posfija), `$n++`, la cual devuelve el valor actual y después incrementa la variable por uno. No haría ninguna diferencia en el fragmento de código más arriba, pero el resultado puede ser diferente en expresiones un poco más complejas.

También hay un operador de decremento `--`, el cual disminuye su argumento por uno y existe en la forma prefija y sufija.

2.4 Modo Script

Hasta ahora hemos ejecutado Raku en el **modo interactivo**, lo que significa que tu interactúas directamente con el interpretador (REPL). El modo interactivo es una buena manera de comenzar, aunque si estás trabajando con más de varias líneas de código, puede ser un poco torpe y hasta tedioso.

La alternativa es usar un editor de texto y guardar el código en un archivo de texto conocido como un **script** y después ejecutar el interpretador en el **modo script** para ejecutar el script. Por convención, los scripts de Raku tienen nombres que terminan con `.raku`, pero en teoría podrías usar extensión que deseases (aunque esto es probablemente recomendado en aras de claridad).

Asegúrate de usar un *editor de texto* y no un *programa de procesamiento de texto* (como MS Word, OpenOffice o LibreOffice Writer). Hay un gran número de editores de texto gratis disponibles. En Linux, podrías usar *vi* (o *vim*), *emacs*, *gEdit*, o *nano*. En Windows, puedes usar *notepad* (muy limitado), o *notepad++*. También hay editores multiplataforma o entorno de desarrollo integrado (IDEs por su siglas en inglés) los cuales proveen la funcionalidad de un editor de texto. Entre ellos están *padre*, *eclipse*, o *atom*. Muchos de estos proveen varias funcionalidades de resaltado de sintaxis, que pueden ayudarte a usar la sintaxis correcta (y encontrar algunos errores sintácticos).

Una vez que has guardado tu código en un archivo de texto (por ejemplo, `mi_script.raku`), puedes ejecutar el programa mediante el siguiente comando en el

prompt del sistema operativo (por ejemplo en una consola de Linux or en una ventana cmd en Windows):

```
raku mi_script.raku
```

Dado que Raku provee ambos modos, puedes ejecutar piezas de código en el modo interactivo antes que las pongas en un script. Pero existen diferencias entre el modo interactivo y el modo script que pueden confundir.

Por ejemplo, si estás usando el interpretador de Raku como una calculadora, podrías escribir:

```
> my $millas = 26.2;
26.2
> $millas * 1.61;
42.182
```

La primera línea asigna un valor a `$millas` y muestra ese valor. La segunda línea es una expresión, así que el interpretador la evalúa y muestra el resultado. Resulta que un maratón es alrededor de 42 kilómetros.

Pero si escribes el mismo código en un script y lo ejecutas, no obtienes ningún resultado. En el modo script, una expresión por sí misma no tiene ningún efecto visible. Raku actualmente evalúa la expresión, pero no muestra el valor a menos que se lo indiques:

```
my $millas = 26.2;
say $millas * 1.61;
```

Este comportamiento puede ser confuso al principio. Examinemos el por qué.

Un script usualmente contiene una secuencia de sentencias. Si hay más de una sentencia, los resultados aparecen uno por uno al tiempo que las sentencias de impresión se ejecutan.

Por ejemplo, considera el siguiente script:

```
say 1;
my $x = 2;
say $x;
```

Produce el siguiente resultado:

```
1
2
```

La sentencia de asignación no produce ningún resultado.

Para comprobar tu entendimiento, escribe las siguientes sentencias en el interpretador de Raku y observa lo que hacen:

```
5;
my $x = 5;
$x + 1;
```

Ahora pon la mismas sentencias en un script y ejecútalo. ¿Cuál es el resultado? Transforma cada expresión en una sentencia de impresión y después ejecuta el script modificado otra vez.

2.5 Modo de una sola línea

Raku también tiene un *modo de una sola línea* (modo one-liner), el cual te permite escribir un script bien corto directamente en el prompt del sistema operativo. Debido a esto, dichos scripts son conocidos como *one-liners*. En Windows, podría lucir de la siguiente manera:

```
C:\Users\Laurent>raku -e "my $valor = 42; say 'La respuesta es ', $valor;"
La respuesta es 42
```

La opción `-e` le dice al compilador que el script a ser ejecutado no está guardado en un archivo de texto sino que en cambio está escrito en el prompt entre comillas dobles inmediatamente después de dicha opción.

En Unix y Linux, reemplazarías las comillas dobles con apóstrofes (o comillas simples) y los apóstrofes con comillas dobles:

```
$ raku -e 'my $valor = 42; say "La respuesta es $valor";'
La respuesta es 42
```

El one-liner más arriba puede no parecer útil, pero los one-liners desechables pueden ser muy prácticos para realizar operaciones simples, tal como modificar un archivo que no está formateado apropiadamente, sin tener que guardar un script en un archivo separado antes de ejecutarlo.

No daremos ningunos detalles adicionales sobre el modo de una sola línea, pero daremos más ejemplos útiles más adelante en este libro; por ejemplo, Subsección 8.4.1, Subsección A.5.9 (resolviendo el ejercicio “rot-13”), o Subsección A.6.1 (resolviendo el ejercicio sobre letras dobles consecutivas).

2.6 Orden de Operaciones

Cuando una expresión contiene más de un operador, el orden de evaluación depende en el **orden de las operaciones** o la *precedencia de los operadores*. Para los operadores matemáticos, Raku sigue la convención matemática. El acrónimo **PEMDAS**² es una manera muy útil de recordar las reglas:

- **Paréntesis** tienen la mayor precedencia y pueden ser usados para forzar una expresión a evaluar en el orden que deseas. Dado que las expresiones en paréntesis son evaluadas primero, $2 * (3-1)$ es 4, y $(1+1)**(5-2)$ es 8. También puedes usar paréntesis para hacer una expresión más legible, como en $(\$minuto * 100) / 60$, aunque no cambia el resultado.
- **Exponente** (potenciación) es la siguiente en el nivel de precedencia, así que $1 + 2**3$ es 9 ($1 + 8$), no 27, y $2 * 3**2$ es 18, no 36.
- **Multiplicación y División** tiene mayor precedencia que **Adición y Sustracción**. Por lo tanto, $2*3-1$ es 5, no 4, y $6+4/2$ es 8, no 5.

²Los estudiantes estadounidenses se les enseña a usar el mnemónico "Please Excuse My Dear Aunt Sally" para recordar el orden correcto de las letras en el acrónimo.

En español, podrías utilizar "Por Favor, Excusa Mi Dragón Azul, Sancho" para recordar el orden correcto.

- Operadores con la misma precedencia son usualmente evaluados de izquierda a derecha (excepto la potenciación). Así que en la expresión `$grados / 2 * pi`, la división ocurre primero y el resultado es multiplicado por `pi`, el cual no es el resultado esperado. (Nota que `pi` no es una variable, sino una constante predefinida en Raku, y por lo tanto no requiere un sigilo.) Para dividir por 2π , puedes usar paréntesis:

```
my $resultado = $grados / (2 * pi);
```

o escribir `$grados / 2 / pi` o `$grados / 2 /  $\pi$` , lo cual divide `$grados` por 2, y después divide el resultado de esa operación por π (el cual es equivalente a `$grados` dividido por 2π).

Trato de no recordar la precedencia de los operadores. Si no puedo determinar la precedencia al mirar la expresión, uso paréntesis para hacerlo obvio. Si no sé cuál de dos operadores tiene la mayor precedencia, entonces la siguiente persona manteniendo mi código podría tampoco saberlo.

2.7 Operaciones de Cadena de Texto

En general, no puedes realizar operaciones matemáticas con las cadenas de texto, a menos que las cadenas de texto se parezcan tanto a los números que Raku las transforma o *coacciona* en números y todavía hace sentido. Así que los siguientes casos son ilegales:

```
'2'-'1a'    'eggs'/'easy'    'third'*'a charm'
```

Por ejemplo, esto produce un error:

```
> '2'-'1a'
Cannot convert string to number: trailing characters after number
in '1?a' (indicated by ?)
in block <unit> at <unknown file>:1
```

Pero las siguientes expresiones son válidas porque estas cadenas de texto pueden ser coaccionadas en números sin ninguna ambigüedad:

```
> '2'-'1'
1
> '3'/'4'
0.75
```

El operador `~` realiza la **concatenación de cadenas de texto**, lo cual quiere decir que une las cadenas de texto al enlazarlas de extremo a extremo. Por ejemplo:

```
> my $primera = 'hidro'
hidro
> my $segunda = 'avión'
avión
> $primera ~ $segunda
hidroavión
```


2.9 Depuración de Programas

Tres tipos de errores pueden ocurrir en un programa: errores sintácticos, errores al tiempo de ejecución, y errores semánticos. Es muy útil saber distinguirlos para rastrearlos más rápidamente.

Error sintáctico La “syntaxis” se refiere a la estructura de un programa. Por ejemplo, los paréntesis deben estar en parejas, así que $(1 + 2)$ es legal mientras $8)$ es un *error sintáctico*.³

Si en tu programa hay un error sintáctico, Raku muestra un mensaje de error y abandona la ejecución sin siquiera comenzar a ejecutar el programa. Durante las primeras semanas de tu carrera de programación, podrías invertir mucho tiempo rastreando errores sintácticos. A medida que ganas experiencia, cometerás menos errores y los encontrarás más rápido.

Error al tiempo de ejecución El segundo tipo de error es un error al tiempo de ejecución, llamado así porque el error no aparece hasta que el programa ha comenzado a ejecutarse. Estos errores también son conocidos como *excepciones* porque ellos usualmente indican que algo excepcional (y malo) ha ocurrido.

Los errores al tiempo de ejecución son raros en los programas simples que verás en los primeros capítulos, así que podría pasar un rato antes que encuentres uno. Ya hemos visto un ejemplo de tales errores, aunque, al inicio de la Sección 2.7 (p. 25), cuando intentamos sustraer `'2' - '1a'`.

Error semántico El tercer tipo de error es *semántico*, lo que significa que está relacionado con el significado. Si en tu programa hay un error semántico, el programa se ejecutará sin generar mensajes de error. Específicamente, hará lo que le *dijiste* que hiciera, pero no lo que habías *previsto*.

La identificación de errores semánticos puede ser complicada porque requiere que trabajes de atrás hacia adelante mediante la observación de la salida del programa y tratando de descifrar lo que el programa está haciendo.

2.10 Glosario

Variable Informalmente, un nombre que hace referencia a un valor. En términos precisos, una variable es un contenedor que tiene un nombre y almacena un valor.

Asignación Una sentencia que asigna un valor a una variable.

Diagrama de estado Una representación gráfica de un conjunto de variables y los valores a los que hacen referencia.

Palabra clave Una palabra reservada que es usada para parsear un programa; en muchos lenguajes, no puedes usar palabras claves tales como `if`, `for`, y `while` como nombres de variables. Este problema usualmente no ocurre en Raku porque los nombres de variables comienzan con un *sigilo*.

³Aquí estamos usando “error sintáctico” como un cuasi-sinónimo para “error al tiempo de compilación”; ellos no son exactamente la misma cosa (en teoría, puedes tener errores sintácticos que no son errores al tiempo de compilación y viceversa), pero se pueden considerar aquí lo mismo por razones prácticas. En Raku, errores al tiempo de compilación tienen la cuerda de texto `“===SORRY!===”` al inicio del mensaje de error.

Operando Un valor o término al lado de un operador que es usado en su evaluación.

Término Una variable o valor literal.

Expresión Una combinación de operadores y términos que representan un valor único.

Evaluar Simplificar una expresión al realizar las operaciones para obtener un valor único.

Sentencia Una sección de código que representa un comando o una acción. Hasta ahora, las sentencias que hemos visto son las asignaciones y las sentencias de impresión. Las sentencias usualmente terminan con un punto y coma.

Ejecutar Poner una sentencia en acción y hacer lo que dice.

Modo interactivo (o modo interpretador) Una manera de usar el interpretador de Raku al escribir código en el prompt.

Modo script Una manera de usar el interpretador de Raku para leer código desde un script y ejecutarlo.

Modo de una sola línea (one-liner) Una manera de usar el interpretador de Raku para leer código pasado al prompt del sistema operativo y ejecutarlo.

Script Un programa almacenado en un archivo de texto.

Order de operaciones Reglas que dictan el orden en el que las expresiones que contienen varios operadores y operandos son evaluadas. También se conoce como precedencia de los operadores.

Concatenar Unir dos cadenas de texto de extremo a extremo.

Comentario Información en una programa que está dirigida a otros programadores (o cualquier persona leyendo el código fuente) y que no tiene ningún efecto en la ejecución del programa.

Error sintáctico Un error en un programa que hace imposible analizarlo sintácticamente (y por lo tanto imposible de compilarlo y ejecutarlo).

Excepción Un error que es detectado mientras que el programa se ejecuta.

Semántica El significado de un programa.

Error semántico Un error en un programa que causa que haga algo diferente a lo que programador pretendía hacer.

2.11 Ejercicios

Ejercicio 2.1. Repetimos nuestro consejo mencionado en el capítulo anterior, cuando aprendas algo nuevo, deberías intentarlo en el modo interactivo (de REPL) y cometer errores a propósito para ver que es lo que no funciona.

- Hemos visto que $\$n = 42$ es legal. ¿Y $42 = \$n$?

- ¿Qué acerca de $\$x = \$y = 1$? (Pista: nota que tendrás que declarar ambas variables, por ejemplo con una sentencia tal como `my $x; my $y;` o posiblemente `my ($x, $y);`, antes de que ejecutes lo anterior.)
- En algunos lenguajes, las sentencias no tienen que terminar con un punto y coma, `;`. ¿Qué pasa en el modo `script` si omites un punto y coma al final de una sentencia de Raku?
- ¿Qué pasa si pones un punto al final de una sentencia?
- En notación matemática puedes multiplicar m y n así: mn . ¿Qué pasa si tratas de hacerlo en Raku?

Ejercicio 2.2. Practica usando el interpretador de Raku como una calculadora:

1. El volumen de una esfera con radio r es $\frac{4}{3}\pi r^3$. ¿Cuál es el volumen de una esfera con radio 5?
2. Supón que el precio inicial de un libro es \$24.95, pero la librería obtiene un 40% de descuento. El envío cuesta \$3 por la primera copia y 75 centavos por cada copia adicional. ¿Cuál es el precio total de 60 copias?
3. Si dejo la casa a la 6:52 a.m. y corro 1 milla a paso suave (8 min. 15 seg. por milla), después 3 millas a paso rápido (7 min. 12 seg. por milla) y 1 milla a paso suave otra vez. ¿Qué hora es cuando termino con mi ejercicio?

Funciones

En el contexto de la programación, una **función** es usualmente una secuencia de sentencias con nombre que realiza una computación. En Raku, las funciones son usualmente llamadas **subrutinas** y los dos términos (por ahora) pueden considerarse más o menos equivalentes. Cuando defines una función, especificas el nombre y la secuencia de sentencias. Después, cuando quieres realizar la computación, puedes “invocar” la función por el nombre y esto ejecutará la secuencia de sentencias dentro de la definición de la función.

Raku tiene muchas funciones integradas que son muy útiles. Ya has visto algunas de ellas: por ejemplo, `say` es una función integrada, y veremos más en el curso de este libro. Y si Raku no tiene una función que hace lo que deseas hacer, tú mismo puedes construirla. Este capítulo te enseña lo básico de las funciones y cómo construirlas.

3.1 Llamadas de Funciones

Ya hemos visto algunos ejemplos de **llamadas de funciones**:

```
> say 42;  
42
```

El nombre de la función es `say`. La expresión que sigue el nombre de la función se llama el **argumento** de la función. La función `say` causa que el argumento sea mostrado en la pantalla. Si necesitas pasar varios argumentos a la función, solo debes separarlos con comas:

```
> say "La respuesta a la pregunta absoluta es ", 42;  
La respuesta a la pregunta absoluta es 42
```

Muchos lenguajes de programación requieren que los argumentos de la función se encuentren dentro de paréntesis. Esto no es requerido (y usualmente no recomendado) en Raku para la mayoría de las funciones integradas (excepto cuando se necesita para precedencia), pero si usas paréntesis, deberías evitar insertar espacios entre el nombre de la función

y el paréntesis izquierdo. Por ejemplo, la función `round` usualmente toma dos argumentos: el valor a ser redondeado y la unidad o escala. Puedes invocarla en cualquiera de las siguientes formas:

```
> round 42.45, 1;
42
> round 42.45, .1;
42.5
> round(42.45, .1);      # Pero no: round (42.45, .1);
42.5
> round( 42.45, .1);     # Espacio *después* del paréntesis izq. es OK
42.5
```

Los programadores de Raku con experiencia prefieren omitir los paréntesis cuando pueden hacerlo. Esto hace posible encadenar varias funciones con una sintaxis más clara. Considera por ejemplo la diferencia entre estas dos llamadas:

```
> say round 42.45, 1;
42
> say(round(42.45, 1));
42
```

La segunda sentencia dice explícitamente lo que está pasando, pero la acumulación de paréntesis actualmente no hace las cosas muy claras. Por el contrario, la primera sentencia puede ser considerada como una línea de tuberías que se puede leer de derecha a izquierda: la última función a la derecha, `round`, toma dos argumentos, `42.45`, `1`, y el valor producido por `round` es pasado como un argumento a `say`.

Es muy común decir que una función “toma” uno o varios argumentos y “devuelve” un resultado. El resultado es también llamado **valor de retorno**.

Raku provee varias funciones que convierten valores de un tipo a otro. Cuando se invocan con un solo argumento, la función `round` toma cualquier valor y lo convierte en un número entero, o por el contrario se queja:

```
> round 42.3;
42
> round "yes"
Cannot convert string to number: base-10 number must begin with valid
digits or '.' in '<HERE>yes' (indicated by <HERE>)
in block <unit> at <unknown file> line 1
```

Nota que, en Raku, muchas funciones integradas pueden también usar la sintaxis de la *llamada de método* con la tal llamada de notación de punto. Las siguientes sentencias muestran el mismo resultado:

```
> round 42.7;      # Sintaxis de la llamada de función
43
> 42.7.round;     # Sintaxis de la llamada de método
43
```

La función `round` puede redondear valores racionales y valores de coma flotante. Hay un método `Int` que también convierte valores numéricos no enteros en enteros, pero no redondea sino que descarta la parte fraccional:

```
> round 42.7
43
> 42.7.Int
42
```

Volveremos a los métodos en la siguiente sección.

La función integrada `Rat` convierte números enteros y cadenas de texto en números racionales (si es posible):

```
> say 4.Rat;
4
> say 4.Rat.WHAT;
(Rat)
> say Rat(4).WHAT;
(Rat)
> say Rat(4).nude;
(4 1)
> say Rat('3.14159');
3.14159
> say Rat('3.14159').nude
(314159 100000)
```

(Como podrás recordar de la Sección 1.5, el método `nude` muestra el *numerador* y *denominador* de un número racional.)

Finalmente, `Str` convierte su argumento en una cadena de texto:

```
> say 42.Str.WHAT
(Str)
> say Str(42).WHAT;
(Str)
```

Nota que estas funciones de conversión de tipo no necesitan ser invocadas explícitamente, dado que Raku intentará hacer lo correcto en muchos de los casos. Por ejemplo, si tienes una cadena de texto que se parece a un número entero, Raku *coaccionará* la cadena de texto en un entero si tratas de aplicarle una operación aritmética:

```
> say "21" * "2";
42
```

Igualmente, los enteros serán coaccionados en cadenas de texto si aplicas el operador de la concatenación de cadena de texto sobre ellos:

```
> say 4 ~ 2;
42
> say (4 ~ 2).WHAT;
(Str)
```


La coacción puede ocurrir hasta dos veces dentro de la misma expresión si es necesario:

```
> say (4 ~ 1) + 1;
42
> say ((4 ~ 1) + 1).WHAT;
(Int)
```

3.2 Funciones y Métodos

Un método es similar a una función—toma argumentos y devuelve un valor—pero la sintaxis de llamada es diferente. Con una función, especificas el nombre de la función seguida de sus argumentos. Un método, por el contrario, usa la notación de punto: especificas el nombre del objeto sobre el cual el método es invocado, seguido por un punto y el nombre del método (y posiblemente argumentos adicionales).

Una llamada de método es usualmente conocida como una **invocación**. Las diferencias más profundas entre las funciones y los métodos se volverá aparente más adelante, cuando estudiemos la programación orientada a objetos (en el Capítulo 12).

Por el tiempo presente, podemos considerar que la diferencia es esencialmente un asunto de una sintaxis diferente de llamada cuando se usan las funciones integradas de Raku. La mayoría de las funciones integradas en Raku aceptan la sintaxis de la llamada de función y la sintaxis de la invocación de método. Por ejemplo, las siguientes sentencias son equivalentes:

```
> say 42;                # sintaxis de la llamada de función
42
> 42.say;                # sintaxis de la invocación de método
42
```

También puedes encadenar subrutinas con ambas formas sintácticas:

```
> 42.WHAT.say;          # sintaxis de método
(Int)
> say WHAT 42;          # sintaxis de función
(Int)
> say 42.WHAT;          # sintaxis mixta
(Int)
```

Depende de ti cuál de las dos formas prefieres usar, pero aquí usaremos ambas formas, aún solo sea para acostumbrarnos a ambas.

3.3 Funciones Matemáticas

Raku provee muchas de las funciones matemáticas que nos son familiares.

Para algunas funciones no tan comunes, podría necesitar un módulo especializado tal como `Math::Matrix` o `Math::Trig`. Un **módulo** es un archivo de texto que contiene una colección de funciones que están relacionadas.

Antes de que usemos las funciones en un módulo, tenemos que importarlo con la sentencia **use**:

```
use Math::Trig;
```

Esta sentencia importará un número de funciones que serás capaz de usar como si tu las hubieses definido en el archivo de código principal, por ejemplo `grad2rad` convierte un valor angular desde grados a radianes o `rad2grad` que hace la conversión opuesta.

Para la mayoría de funciones matemáticas comunes, sin embargo, no necesitas tener ningún módulo matemático, dado que dichas funciones ya están incluidas en el núcleo del lenguaje:

```
> my $nivel-ruido = 5.5;
5.5
> my $nivel-señal = 125.6;
125.6
> my $decibelios = 10 * log10 $nivel-señal / $nivel-ruido;
13.5862694990693
```

El primer ejemplo usa `log10` (logaritmo común) para computar la razón entre la señal y el ruido en decibelios (asumiendo que `nivel-señal` y `nivel-ruido` sean definidos en las unidades correctas). Raku también provee una función `log`, la cual, cuando recibe un argumento, calcula el logaritmo en base e del argumento y, cuando recibe dos argumentos, computa el logaritmo del primer argumento en base del segundo argumento:

```
> say e;                # e es predefinida como la constante de Euler
2.71828182845905
> my $val = e ** e;
15.1542622414793
> say log $val;          # logaritmo natural
2.71828182845905
> say log $val, e;       # logaritmo en base e o logaritmo natural
2.71828182845905
> say log 1024, 2;       # logaritmo binario o logaritmo en base 2
10
```

Raku también provee muchas de las funciones trigonométricas comunes:

```
> my $radianes = 0.7;
0.7
> my $altura = sin $radianes;
0.644217687237691
```

Este ejemplo encuentra el seno de `$radianes`. El nombre de la variable es una pista a que el `sin` y otras funciones trigonométricas (`cos`, `tan`, etc.) toman argumentos en radianes. Para convertir de grados a radianes, puedes usar la función `deg2rad` del módulo `Math::Trig`, o simplemente divide por 180 y multiplica por π :

```
> my $grados = 45;
45
> my $radianes = $grados / 180.0 * pi;    # pi, constante predefinida
0.785398163397448
> say sin $radianes;                    # debería ser la raíz cuadrada de 2 dividida por 2
0.707106781186547
```

La expresión `pi` es una constante predefinida de una aproximación de π , precisa alrededor de 14 dígitos:

Si sabes trigonometría, puedes verificar el resultado anterior al compararlo con la raíz cuadrada de 2 dividida por 2:

```
> say sqrt(2) / 2;
0.707106781186548
```

3.4 Composición

Hasta ahora, hemos discutido los elementos de un programa—variables, expresiones, y sentencias—aisladamente, sin discutir sobre cómo combinarlos.

Una de las características más útiles de los lenguajes de programación es su habilidad de tomar pequeños componentes y **unir** estos componentes, o sea, combinarlos de tal manera que el resultado de uno es la entrada de otro. Por ejemplo, el argumento de una función puede ser cualquier tipo de expresión, incluyendo operadores aritméticos:

```
> my $grados = 45;
45
> my $altura = sin($grados / 360.0 * 2 * pi);
0.707106781186547
```

Aquí, hemos usados paréntesis para el argumento de la función `sin` para clarificar que todas las operaciones aritméticas sean completadas antes que la función `sin` sea actualmente llamada, para que así use el resultado de estas operaciones como su argumento.

Puedes componer llamadas de funciones:

```
> my $x = 10;
10
> $x = exp log($x+1)
11
```

Casi en cualquier lado que puedes poner un valor, puedes poner una expresión arbitraria, con una excepción: el lado izquierdo de la sentencia de asignación tiene que ser el nombre de una variable, posiblemente con su declaración. Casi cualquier otra expresión en el lado izquierdo es un error sintáctico ¹:

```
> my $horas = 1;
1
> my $minutos = 0;
0
> $minutos = $horas * 60;          # correcto
60
> $horas * 60 = $minutos;          # incorrecto!!
Cannot modify an immutable Int
in block <unit> at <unknown file> line 1
```

¹Veremos una rara excepción a esta regla más tarde

3.5 Agregar Nuevas Funciones (o Subrutinas)

Hasta ahora, hemos solo usado las funciones que vienen con Raku, pero también es posible agregar nuevas funciones. En Raku, las funciones definidas por el usuario son usualmente conocidas como **subrutinas**, pero podrías elegir cualquiera de las dos palabras.

Una **definición de función** comienza con la palabra clave `sub` (abreviación de subrutina) y especifica el nombre de una nueva subrutina y la secuencia de sentencias que serán ejecutadas cuando la función sea llamada.

Este es un ejemplo de una subrutina que cita el famoso discurso “I Have a Dream” de Martin Luther King en el Lincoln Memorial en Washington (1963):

```
sub imprimir-discurso() {  
    say "Let freedom ring from the prodigious hilltops of New Hampshire.";  
    say "Let freedom ring from the mighty mountains of New York.";  
}
```

`sub` es la palabra clave que indica que esto es una definición de subrutina. El nombre de la función es `imprimir-discurso`. Las reglas para los nombres de subrutinas son las mismas reglas para los nombres de variables: letras, números, y guiones bajos son legales, al igual que un guion o un apóstrofo entre letras, pero el primer carácter debe ser una letra o un guion bajo. No deberías utilizar una palabra clave usada por el lenguaje (tal como `if` o `while`) como el nombre de una función (en algunos casos, podría actualmente funcionar pero sería algo confuso, por lo menos para el lector humano).

Los paréntesis vacíos después del nombre indican que esta función no toma ningún argumento. En tal caso los argumentos son opcionales, pero son requeridos cuando los parámetros necesitan ser definidos durante la definición de la subrutina.

La primera línea de la definición de una subrutina es algunas veces llamada la **cabecera**; el resto es conocido como el **cuerpo** de la subrutina. El cuerpo tiene que ser un bloque de código colocado entre llaves y puede contener una cantidad indefinida de sentencias. Aunque no existe una regla que lo requiera, es buena práctica (y muy recomendado) indentar las sentencias en el cuerpo con varios espacios delanteros, debido a que facilita descifrar visualmente donde el cuerpo de la función inicia y termina.

Ten presente que no puedes utilizar la sintaxis de invocación de método con las subrutinas (tal como `imprimir-discurso`) que escribas: debes llamarlas con la sintaxis de llamada de función.

Las cadenas de texto en las sentencias de impresión están rodeadas por comillas dobles. En este caso específico, las comillas simples se podrían haber usado para obtener el mismo resultado, pero hay otros casos donde los dos tipos de comillas no harían lo mismo, así que tendrás que elegir una de las dos dependiendo de las circunstancias.

Muchas personas usan las comillas dobles en aquellos casos donde una comilla simple (la cual también es un apóstrofo) aparece en una cadena de texto:

```
say "And so we've come here today to dramatize a shameful condition.";
```

De igual manera, podrías utilizar comillas simples cuando las comillas dobles aparecen en una cadena de texto:

```
say 'America has given the Negro people a bad check,
    a check which has come back marked "insufficient funds."';
```

Sin embargo, existe una diferencia más importante entre las comillas simples y las comillas dobles: las comillas dobles permiten la interpolación de variables, lo cual no es posible con el uso de las comillas simples. La interpolación de variables se refiere a que si el nombre de una variable aparece dentro de una cadena de texto rodeada por comillas dobles, dicho nombre será reemplazado por el valor de la variable; dentro una cadena de texto rodeada por comillas simples, el nombre de la variable aparecerá literalmente. Por ejemplo:

```
my $var = 12;
say "La edad del niño es $var.";           # -> La edad del niño es 12.
say 'La edad de la niña es $var.';         # -> La edad de la niña es $var.
```

No es que la edad de la niña no se muestra porque se quiera mantener como un secreto. En la primera cadena de texto, \$var es simplemente reemplazada dentro de la cadena por su valor, 12, porque la cadena de texto está rodeada con comillas dobles; en la segunda cadena de texto, \$var no es reemplazada por su valor porque las comillas simples están supuestas a proveer un citado más literal. Existen otros tipos de citados que ofrecen un mayor control sobre la manera en que las variables y los caracteres especiales son mostrados en la salida, pero las comillas simples y las comillas dobles son las más útiles.

La sintaxis para llamar una nueva subrutina es la misma usada para llamar las funciones integradas:

```
> imprimir-discurso();
Let freedom ring from the prodigious hilltops of New Hampshire.
Let freedom ring from the mighty mountains of New York.
```

No obstante, no puedes utilizar la sintaxis de invocación de método con tales subrutinas. Más tarde veremos en este libro (ve Capítulo 12) cómo crear métodos. Por el tiempo presente, permaneceremos con la sintaxis de llamada de función.

Una vez que has definido una subrutina, puedes usarla dentro de otra subrutina. Por ejemplo, para repetir el fragmento anterior del discurso de King, podríamos escribir una subrutina llamada `repetir-discurso`:

```
sub repetir-discurso() {
  imprimir-discurso();
  imprimir-discurso();
}
```

Y después llamar a `repetir-discurso`:

```
> repetir-discurso();
Let freedom ring from the prodigious hilltops of New Hampshire.
Let freedom ring from the mighty mountains of New York.
Let freedom ring from the prodigious hilltops of New Hampshire.
Let freedom ring from the mighty mountains of New York.
```

Sin embargo, no es así como prosigue el discurso.

3.6 Definiciones y Usos

Juntando los fragmentos de código de la sección anterior, el programa entero luce así:

```
sub imprimir-discurso() {  
    say "let freedom ring from the prodigious hilltops of New Hampshire.";  
    say "Let freedom ring from the mighty mountains of New York.";  
}  
sub repetir-discurso() {  
    imprimir-discurso();  
    imprimir-discurso();  
}  
repetir-discurso();
```

Este programa contiene dos definiciones de subrutinas: `imprimir-discurso` y `repetir-discurso`. Las definiciones de funciones pueden ser ejecutadas como otras sentencias, pero el efecto es crear la función. Las sentencias dentro de la función no son ejecutadas hasta que la función es llamada, y la definición de la función no genera ninguna salida.

No necesitas crear una subrutina antes de ejecutarla, la definición de la función puede venir después de la llamada de la función:

```
repetir-discurso;  
sub repetir-discurso() {  
    imprimir-discurso;  
    imprimir-discurso;  
}  
sub imprimir-discurso() {  
    # ...  
}
```

3.7 Flujo de Ejecución

Para asegurar, por ejemplo, que una variable está definida (o poblada) antes de su uso, necesitas saber el orden en el que las sentencias se ejecutan. Dicho orden es conocido como el **flujo de ejecución**.

La ejecución siempre comienza con la primera sentencia del programa (bueno, *casi* siempre, pero digamos siempre por el tiempo presente). Las sentencias se ejecutan una por una, de arriba hacia abajo.

Las definiciones de subrutinas no cambian el flujo de ejecución del programa, pero debes recordar que las sentencias dentro de una función no son ejecutadas hasta que la función es llamada.

Una llamada de función es como un desvío en el flujo de ejecución. En vez de continuar con la siguiente sentencia, el flujo salta al cuerpo de la función, ejecuta las sentencias ahí dentro, y regresa para continuar donde se quedó.

Esto suena simple hasta que recuerdas que una función puede llamar a otra función. Mientras se encuentra dentro de una función, el programa podría ejecutar las sentencias en otra

función. ¡Después, mientras se encuentra en dicha función, el programa podría aún ejecutar otra función!

Afortunadamente, Raku es lo suficientemente bueno para rastrear dónde se encuentra, así que cada vez una función se completa, el programa continúa donde se quedó en el programa que lo llamó. Cuando llega al final del programa, entonces termina.

En resumen, cuando leas un programa, no siempre quieras leerlo de arriba hacia abajo. En algunas ocasiones hace más sentido si sigues el flujo de ejecución.

3.8 Parámetros y Argumentos

Algunas de las funciones que hemos visto requieren argumentos. Por ejemplo, cuando llamas a `say` le pasas un número como argumento. Algunas funciones toman más de un argumento: por ejemplo, la función `round` que vimos al principio de este capítulo tomó dos, el número a ser redondeado y la escala (aunque la función `round` puede aceptar un solo argumento. En dicho caso, el valor de la escala por defecto es 1).

Dentro de la subrutina, los argumentos son asignados a variables llamadas **parámetros**. Aquí está la definición de una subrutina que toma un solo argumento:

```
sub doble-impresión( $valor ) {
    say $valor;
    say $valor
}
```

Esta subrutina asigna el argumento al parámetro llamado `$valor`. Otra manera común de interpretarlo es decir que la subrutina *ata* el parámetro definido en su cabecera al argumento con el cual es llamada. Cuando la subrutina anterior es llamada, la misma imprime el contenido del parámetro (cualquier cosa que sea) dos veces.

Esta función funciona con cualquier argumento que pueda ser imprimido:

```
> doble-impresión("Let freedom ring")
Let freedom ring
Let freedom ring
> doble-impresión(42)
42
42
> doble-impresión(pi)
3.14159265358979
3.14159265358979
```

Las mismas reglas de composición que aplican a las funciones integradas también aplican a las funciones definidas por el programador, así que podemos llamar a `doble-impresión` con cualquier tipo de expresión como un argumento:

```
> doble-impresión('Let freedom ring! ' x 2)
Let freedom ring! Let freedom ring!
Let freedom ring! Let freedom ring!
> doble-impresión(cos pi)
-1
-1
```

El argumento es evaluado antes que la función sea llamada, así que en los ejemplos las expresiones 'Let freedom ring! ' x 2 y cos pi son evaluadas una sola vez.

También puedes usar una variable como argumento:

```
> my $declaración = 'When in the Course of human events, ...'
> doble-impresión($declaración)
When in the Course of human events, ...
When in the Course of human events, ...
```

El nombre de la variable que pasamos como un argumento (\$declaración) no tiene nada que ver con el nombre del parámetro (\$valor). No importa cual sea el nombre de la variable cuando fue llamada; aquí, dentro de doble-impresión, llamamos al parámetro \$valor sin importar el nombre o contenido del argumento que se pasó a la subrutina.

3.9 Las Variables y los Parámetros son Locales

Cuando creas una variable dentro de una subrutina con la palabra clave **my**, dicha variable es entonces **local**, o más precisamente, la variable está en un ámbito léxico con respecto al bloque de la función. Esto significa que la variable solo existe dentro de la función. Por ejemplo:

```
sub doble_concat( $parte1, $parte2 ) {
  my $concatenación = $parte1 ~ $parte2;
  doble-impresión($concatenación)
}
```

Esta función toma dos argumentos, los concatena, e imprime el resultado dos veces. Este es un ejemplo que la usa:

```
> my $inicio = 'Let freedom ring from ';
> my $final = 'the mighty mountains of New York.';
> doble_concat($inicio, $final);
Let freedom ring from the mighty mountains of New York.
Let freedom ring from the mighty mountains of New York.
```

Cuando doble-concat termina, la variable \$concatenación es destruida. Si intentamos imprimirla, obtenemos una excepción:

```
> say $concatenación;
===SORRY!=== Error while compiling <unknown file>
Variable '$concatenación' is not declared
at <unknown file>:1
-----> say <HERE>$concatenación;
```

Los parámetros también están en un ámbito léxico en la subrutina. Por ejemplo, fuera de doble-impresión, no existe tal variable como \$valor.

main	\$inicio	→ 'Let freedom ring from'
	\$final	→ 'the mighty mountains of New York.'
doble_concat	\$part1	→ 'Let freedom ring from'
	\$part2	→ 'the mighty mountains of New York.'
	\$concatenación	→ 'Let freedom... mountains of New York.'
doble_impresión	\$valor	→ 'Let freedom ring.'

Fig. 3.1: Diagrama de estado.

3.10 Diagramas de Pila

Para rastrear cual de las variables puede ser usada, algunas veces es muy útil dibujar un **diagrama de pila**. Al igual que los diagramas de estado, los diagramas de pila muestran el valor de cada variable, pero también muestran la función a la cual cada variable pertenece.

Cada función se representa gráficamente por un **marco**. Un marco es un cuadro con el nombre de la función al lado y los parámetros y las variables de la función dentro. El diagrama de pila para el ejemplo anterior se muestra en la Figura 3.1.

Los marcos están organizados en la pila para indicar cual función llama a cual. En este ejemplo, `doble-impresión` fue llamada por `doble_concat`, y `doble_concat` fue llamada por `main`, el cual es un nombre especial para el marco superior. Cuando creas una variable fuera de cualquier función, dicha variable le pertenece a `main`.

Cada parámetro hace referencia al mismo valor que su argumento correspondiente. Así que, `$parte1` tiene el mismo valor que `$inicio`, `$parte2` tiene el mismo valor que `$final` y `$valor` tiene el mismo valor que `$concatenación`.

3.11 Funciones Fructuosas y Funciones Voids

Algunas de las funciones que hemos usado, tal como las funciones matemáticas, devuelven resultados y son útiles en cuanto podamos usar el valor devuelto; por falta de un mejor nombre, las podemos llamar **funciones fructuosas**. Otras funciones, tal como `doble-impresión`, realizan una acción pero no parecen devolver un valor (aunque en verdad devuelve un valor, `True`, pero no estamos interesados en este valor). Estas funciones son algunas veces llamadas funciones vacías o **funciones void** en otros lenguajes de programación.

En algunos lenguajes de programación, tales como Pascal o Ada, existe una gran distinción entre una *función* (la cual devuelve un valor) y un *procedimiento* (el cual no devuelve ningún valor); ellas son definidas con diferentes palabras claves. Dicha distinción no aplica en Raku ni en la mayoría de los lenguajes de programación modernos.

De hecho, desde un punto de vista sintáctico, las funciones de Raku siempre devuelven un valor. Así que la distinción entre una función “fructuosa” y “void” no existe sintáctica, sino semánticamente, o sea, desde el punto de vista del significado del programa: tal vez necesitamos usar el valor devuelto, o tal vez no.

Otra distinción que se hace comúnmente entre funciones y mutadores es que las funciones no cambian el estado inicial de los argumentos que se les pasa mientras que los mutadores modifican los argumentos. No usaremos esta distinción aquí, pero es útil mantenerla presente.

Cuando llamas una función fructuosa, es muy común que quieras hacer algo con el resultado; por ejemplo, podrías asignarlo a una variable o usarlo como parte de una expresión:

```
my $altura = sin $radianes;  
my $aureo = (sqrt(5) + 1) / 2;
```

Cuando llamas una función en el modo interactivo (en el REPL), Raku usualmente muestra el resultado inmediatamente:

```
> sqrt 5;  
2.23606797749979
```

¡Pero en un script, si llamas una función fructuosa por sí misma, el valor de retorno se pierde para siempre! En algunos casos, el compilador será capaz de advertirte, pero no siempre. Por ejemplo, considera el siguiente programa:

```
my $cinco = 5;  
sqrt $cinco;  
say $cinco;
```

Este ejemplo produce la siguiente advertencia:

```
WARNINGS for /home/Laurent/raku_tests/sqrt.raku:  
Useless use of "sqrt $cinco" in expression "sqrt $cinco" in sink context (line 2)  
5
```

El script calcula la raíz cuadrada de 5, pero dado que no almacena o muestra el resultado, no es muy útil.

Las funciones void muestran algo en la pantalla, guardan algún dato en un archivo, modifican una variable o un objeto, o tienen algún otro efecto, pero generalmente no tienen un valor de retorno, o por lo menos algo no muy útil. Si asignas el resultado a una variable, puedes conseguir el valor de retorno de la subrutina, el valor de la última expresión que fue evaluada en la función, o un valor especial tal como Any, el cual esencialmente se refiere a algo que no ha sido definido, o Nil.

Las subrutinas que hemos escrito hasta ahora esencialmente muestran cosas en la pantalla. En ese caso, ellas devuelven True, por lo menos cuando la impresión fue exitosa. Aunque ellas devuelven un valor verdadero, lo que ellas devuelven no es muy útil y podemos considerarlas funciones void para nuestro propósito.

El siguiente es un ejemplo de una subrutina fructuosa bien simple:

```
> sub cuadrado($número) { return $número ** 2 }
sub cuadrado ($número) { #`(Sub|118134416) ... }
> say cuadrado 5;
25
```

El mensaje Sub|118134416 mostrado por el REPL es solo un identificador interno de la subrutina que hemos definido.

La sentencia `return` le indica a la función que termine la ejecución de la función en esta sentencia y devuelva el valor de la siguiente expresión a donde fue llamada. En tal caso donde el programa está ejecutando la última sentencia de una función, la palabra clave `return` puede ser omitida dado que la función devolverá el valor de la última sentencia evaluada, así que la subrutina `cuadrado` podría ser escrita de la siguiente manera:

```
sub cuadrado( $número ) {
  $número ** 2
}
```

Usaremos funciones fructuosas más extensivamente en los siguientes capítulos.

3.12 Signaturas de Funciones

Cuando una función recibe argumentos, los cuales son almacenados en los parámetros, la parte de la definición de función que describe los parámetros entre los paréntesis se llama la **signatura de la función**. Las signaturas de funciones que hemos visto hasta ahora son bien simples y consisten de un solo parámetro o posiblemente una lista de parámetros.

Las signaturas pueden proveer una plétora de información sobre los parámetros usados por la función. Primero, puedes definir el tipo de los parámetros. Algunas funciones hacen sentido solo si sus parámetros son numéricos y deberían probablemente levantar un error si ellas reciben una cadena de texto que no puede convertirse a un valor numérico. Por ejemplo, si defines una función `mitad` que computa el valor igual a su argumento dividido por 2, no hace sentido tratar de computar la mitad de una cadena de texto que no es numérica. Dicha función podría escribirse así:

```
sub mitad( Int $número ) {
  return $número / 2
}
say mitad 84; # -> 42
```

Si esta función es llamada con una cadena de texto, conseguimos el siguiente error:

```
> say mitad "Douglas Adams"
===SORRY!=== Error while compiling <unknown file>
Calling mitad(Str) will never work with declared signature (Int $number)
at <unknown file>:1
-----> say <HERE>mitad "Douglas Adams"
```

El tipo `Int` incluido en la signatura de la función es una restricción de tipo que puede ayudar a prevenir errores sutiles. En algunos casos, también pueden ser una molestia. Considera el fragmento de código:

```
sub mitad( Int $numero ) { $numero / 2 }
say mitad "84"; # -> ERROR
```

Dado que el argumento de la subrutina `mitad` es "84", es decir, una cadena de texto, este código fallará con un error de tipo. Si no hubiésemos incluido el tipo `Int` en la signatura, el script habría convertido (o *coaccionado*) la cadena de texto "84" en un número, dividido por 2, y mostrado el resultado esperado:

```
sub mitad( $numero ) { $numero / 2 }
say mitad "84"; # -> 42
```

En algunos casos, quieres que esta conversión ocurra, en otros, no quieres esto. Es tu deber decidir si quieres tipado estricto o no, dependiendo en la situación específica y necesidad. Es probablemente útil usar tipos en los parámetros en muchos casos, pero puede volverse un obstáculo en algunas situaciones. Raku te deja decidir qué tan estricto deseas ser sobre estas cosas.

Nuestra subrutina original `mitad` tiene otra limitación: solo funciona con números enteros. Pero una función que devuelve la mitad de su argumento debería supuestamente ser útil para números racionales o hasta otros números. Puedes usar los tipos `Real` o `Numeric` para hacer la función más general (la diferencia entre los tipos es que el tipo `Numeric` aceptará no solo números Reales pero también números complejos (`Complex`)²). Como sucede con números complejos, el elegir un tipo `Numeric` abre más posibilidades:

```
sub mitad( Numeric $numero ) { $numero / 2 }
say mitad(3+4i); # -> 1.5+2i
```

La siguiente tabla resume e ilustra algunos de los varios tipos que hemos visto hasta ahora.

Tipo	Ejemplo
String	"Una cadena de texto (string)", 'Hola', "42"
Integer	-3, -2, 0, 2, 42
Rational	1/2, 0.5, 3, 14159, 22/7, 42.0
Real	π , pi, $\sqrt{2}$, e , log 42, sin 0.7
Complex	$5.4 + 3i$

3.13 Parámetros Inmutables y Mutables

Por defecto, los parámetros de una subrutina son apodos **inmutables** para los argumentos pasados a la subrutina. En otras palabras, ellos no pueden ser cambiados dentro de la función y tú no puedes modificar el argumento accidentalmente donde la función es llamada:

```
sub más-tres( Int $numero ) { $numero += 3 }
my $valor = 5;
say más-tres $valor; # ERROR: Cannot assign to an immutable value
```

² Los números complejos son un concepto matemático poderoso. Ellos son números de la forma $a + bi$, donde a y b son un números reales y i es un número imaginario de tal modo que i^2 es igual a -1 .

En algunos otros lenguajes, este comportamiento es conocido como una “llamada por valor”: en términos generales, la subrutina recibe (por defecto) un valor y no una variable, y el parámetro por lo tanto no puede ser modificado.

Si quieres cambiar el valor del parámetro dentro de la subrutina (pero sin cambiar el argumento), puedes agregar el *rasgo* `is copy` a la signatura:

```
sub más-tres( Int $número is copy ) { $número += 3 }
my $valor = 5;
say más-tres $valor;      #-> 8
say $valor;               #-> 5 (valor de la variable intacto)
```

Un **rasgo** (**en.** *trait*) es una propiedad del parámetro definida al tiempo de compilación. Aquí, el parámetro `$número` es modificado dentro de la subrutina y el valor incrementado es devuelto e imprimido como 8, pero, dentro de la función que hace la llamada, la variable usada como un argumento para la función, `$valor`, no es modificada (es todavía 5).

Aunque esto puede ser algunas veces peligroso, puedes también querer escribir una subrutina que modifica su argumento en el lugar donde la función es llamada. Para esto, puedes usar el *rasgo* `is rw` en la signatura:

```
sub más-tres( Int $número is rw ) { $número += 3 }
my $valor = 5;
say más-tres $valor;      #-> 8
say $valor;               #-> 8 ($valor es modificada)
```

Con el *rasgo* `is rw`, el parámetro `$número` está ahora *vinculado* al argumento `$valor`, así que cualquier cambio hecho al usar `$número` dentro de la subrutina será inmediatamente aplicado a `$valor` en lugar donde la función es llamada, porque `$número` y `$valor` son solo nombres diferentes para la misma cosa (ambos hacen referencia a la misma dirección de memoria). El argumento es ahora totalmente *mutable*.

En algunos otros lenguajes de programación, esto se conoce como “llamada por referencia”, porque, en aquellos lenguajes, si tu pasas una referencia (o un puntero) de una variable a una función, entonces es posible que la función pueda modificar la variable referida por la referencia.

3.14 Funciones y Subrutinas: Ciudadanos de Primera Clase

Las subrutinas y otros objetos de código se pueden pasar como los valores, como cualquier variable, literal u objeto. Entonces se dice que las funciones son **objetos de primera clase** o algunas veces, ciudadanos de primera clase o funciones de orden superior. Esto significa que una función de Raku (no el valor que devuelve sino su código) es un valor que puedes asignar a una variable o pasarlo como un argumento. Por ejemplo, `do-twice` es una subrutina que toma una función como argumento y la llama dos veces:

```
sub do-twice( $código ) {
  $código();
  $código();
}
```

Aquí, el parámetro `$código` se refiere a una función o cualquier otro objeto que se pueda llamar. Esto es un ejemplo que usa `do-twice` para llamar una función `saludar` dos veces:

```
sub saludar {
  say "¡Hola Mundo!";
}
do-twice &saludar;
```

Esto imprimirá:

```
¡Hola Mundo!
¡Hola Mundo!
```

El sigilo `&` se coloca antes del nombre de la subrutina en la lista de argumentos para dejarle saber a Raku que estás pasando una subrutina o cualquier objeto de código que se puede llamar (y no estás llamando la subrutina en el momento).

De hecho, sería más idiomático usar también el sigilo `&` en la definición de la subrutina `do-twice` para especificar que el parámetro es un objeto de código que se puede llamar:

```
sub do-twice( &código ) {
  &código();
  &código();
}
```

De igual manera:

```
sub do-twice( &código ) {
  código();
  código();
}
```

La sintaxis con el sigilo `&` tiene el beneficio de proveer un mejor mensaje de error si cometes un error y le pasas a `do-twice` algo que no se puede llamar.

Todas las funciones que hemos visto hasta ahora han tenido nombres, pero no es necesario que una función tenga un nombre y por lo tanto, puede ser una función *anónima*. Por ejemplo, se puede almacenar directamente en una variable escalar:

```
my $saludar = sub {
  say "¡Hola Mundo!";
};
$saludar();           # imprime "¡Hola Mundo!"
do-twice $saludar;    # imprime "¡Hola Mundo!" dos veces
```

Se podría argumentar que la subrutina `$saludar` no es realmente anónima, debido a que está almacenada en una variable escalar que podría de alguna manera ser considerada su nombre. Pero realmente la subrutina no posee un nombre; solo sucede que está asignada a una variable escalar. Para demostrar que la subrutina no tiene nombre alguno, considera lo siguiente:

```
do-twice(sub {say "¡Hola Mundo!"} );
```

Esto imprimirá "¡Hola Mundo!" sin ningún problema. Si la función `do-twice` se ha declarado posteriormente, puedes hasta simplificar la sintaxis y omitir los paréntesis:

```
do-twice sub {say "¡Hola Mundo!"};
```

Para casos simples donde no hay necesidad de pasar un argumento o devolver un valor, puedes omitir la palabra clave `sub` y pasar un bloque de código directamente a la función:

```
do-twice {say "¡Hola Mundo!"};
do-twice {say "¡Buen día!"};
```

Como puedes observar, `do-twice` es una subrutina *genérica* cuyo trabajo es ejecutar dos veces cualquier función o bloque de código que se le pasa, sin ningún conocimiento sobre lo que la función o el bloque de código hace. Este es un concepto poderoso usado en algunas técnicas de programación avanzadas que discutiremos más adelante.

Las subrutinas se pueden también pasar como valores de retorno desde otras subrutinas:

```
> sub crear-función ($persona) { return sub { say "¡Hola $persona!"}}
# Crear dos funciones de saludos
sub crear-función ($persona) { #`(Sub|176738440) ... }
> my $saludar_mundo = crear-función "Mundo";
sub () { #`(Sub|176738592) ... }
> my $saludar_amigo = crear-función "querido amigo";
sub () { #`(Sub|176739048) ... }
# Usar las funciones de saludos
> $saludar_mundo();
¡Hola Mundo!
> $saludar_amigo();
¡Hola querido amigo!
```

Aquí, la función `crear-función` devuelve una subrutina que saluda a alguien. Se llama dos veces con dos argumentos diferentes para crear dos funciones diferentes al tiempo de ejecución, `$saludar_mundo` y `$saludar_amigo`. Una función como `crear-función` es algunas veces conocida como una *factoría de funciones* porque puedes crear un número determinado de funciones con solo llamar a `crear-función`. Este ejemplo puede parecer una manera complicada de hacer algo tan simple. Lo que pasa es que en este momento es muy prematuro dar ejemplos útiles, no obstante esta es una técnica de programación muy poderosa.

Regresaremos a estas técnicas a lo largo de este libro y hasta dedicaremos un capítulo entero (Capítulo 14) para este asunto y otros temas relacionados.

3.15 ¿Por qué Funciones y Subrutinas?

Debido a que no está claro el por qué de dividir un programa en funciones o subrutinas, aquí discutimos varias razones:

- La creación de una subrutina nueva te da la oportunidad de nombrar un grupo de sentencias, lo cual hace tu programa más fácil de leer y depurar. Las subrutinas también ayudan a hacer el flujo de ejecución más claro para el lector.
- Las subrutinas pueden reducir el tamaño de un programa al eliminar código repetitivo. Más adelante, si haces un cambio, solamente tienes que hacerlo en un solo lugar.
- La división de un programa largo en subrutinas te permite depurar las partes una a una y después ensamblarlas en un todo que funciona.
- Las subrutinas que son bien diseñadas usualmente son útiles para muchos programas. Una vez que escribes y depuras una, la puedes reusar.
- La creación de subrutinas es una de las maneras más útiles de descomponer un problema difícil en tareas pequeñas más fáciles y de crear capas sucesivas de abstracción, las cuales son las llaves para resolver problemas complejos.
- Escribir buenas subrutinas te permite crear cajas negras, con una entrada y una salida conocida. Así que no tienes que pensar sobre ellas cuando trabajas en algo más. De alguna forma, se han convertido en herramientas. Una vez que ensamblas un taladro, no necesitas pensar sobre cómo trabaja internamente cuando lo usas para construir o reparar algo, solamente lo usas.
- En el mundo actual de código abierto, existe la oportunidad que tu código tendrá que ser comprendido, mantenido o mejorado por otras personas. La programación se ha vuelto más y más una actividad social y la descomposición de tu código en pequeñas subrutinas cuyo propósito es fácil de entender hará el trabajo de las otras personas más fácil. Y tú estarás más deleitado cuando la persona que tenga que mantener o refactorizar tu código sea ... tú.

3.16 Depuración de Programas

Una de las más importantes habilidades de la programación que adquirirás es la depuración. Aunque algunas veces puede ser frustrante, la depuración es una de las partes más intelectualmente rica, estimulante e interesante de la programación.

En alguna forma, la depuración se parece al trabajo detectivesco. Eres confrontado con pistas y debes inferir los procesos y eventos que conllevaron a los resultados que observas.

La depuración es como una ciencia experimental. Una vez que tienes una idea de lo que no funciona, modificas el programa e intentas nuevamente. Si la hipótesis era correcta, puedes predecir el resultado de la modificación, y te acercas más a un programa que funciona. Si la hipótesis era incorrecta, tienes que producir una nueva hipótesis. Como Sherlock Holmes señaló, “Cuando todo aquello que es imposible ha sido eliminado, lo que quede, por muy improbable que parezca, es la verdad” (A. Conan Doyle, *El signo de los cuatro*).

En los casos donde no eres capaz de producir una hipótesis sobre lo que no funciona, puedes intentar introducir código del cual espera un determinado tipo de error, una “hipótesis negativa”. Algunas veces puedes aprender mucho del hecho de que no creó el error que se esperaba. Crear una hipótesis no significa necesariamente que tienes una idea sobre cómo hacerlo funcionar, también podría ser una hipótesis sobre cómo hacerlo fallar.

Para algunas personas, la programación y la depuración son la misma cosa. Es decir, la programación es el proceso de depurar gradualmente un programa hasta que hace lo que quieres. La idea es que deberías comenzar con un programa que funciona y hacer pequeñas modificaciones, y depurarlas según avanzas.

Por ejemplo, Linux es un sistema operativo que contiene millones de líneas de código. Sin embargo, todo comenzó con un simple programa que Linus Torvalds usó para explorar el chip Intel 80386. De acuerdo a Larry Greenfield, “Uno de los proyectos iniciales de Linus fue un programa que cambiaba entre la impresión de AAAA y BBBB. Luego éste evolucionó en Linux.” (*The Linux Users’ Guide Beta Version 1*).

3.17 Glosario

Función Una secuencia de sentencias que realiza un trabajo útil. Las funciones pueden tener cero o más argumentos y pueden producir o no producir un resultado. Raku viene con muchas funciones integradas, y tú puedes crear tus propias funciones. En Raku, las funciones definidas por el usuario son usualmente conocidas como subrutinas.

Definición de una función Una sentencia que crea una función nueva, especifica su nombre, sus parámetros, y las sentencias que contiene.

Cabecera La primera línea de la definición de la función.

Cuerpo La secuencia de sentencias dentro de la definición de la función, usualmente en un bloque de código delimitado por llaves.

Parámetro Un nombre que se usa dentro de la subrutina para referirse al valor que se pasa como un argumento.

Llamada de función Una sentencia que ejecuta la función. Consiste del nombre de la función seguido por una lista de argumentos, la cual puede o no puede estar encerrada por paréntesis.

Argumento Un valor que se provee a la función cuando ésta se llama. Este valor es asignado al parámetro correspondiente en la función.

Variable léxica Una variable definida dentro de una subrutina o un bloque de código. Una variable léxica definida dentro de una función se puede usar solo dentro de la función.

Valor de retorno El resultado de una función. Si una llamada de función se usa como una expresión, el valor de retorno es el valor de la expresión.

Any Un valor especial típicamente encontrado en las variables a las cuales no se le ha asignado un valor. Es también un valor especial devuelto por algunas funciones que llamamos “void” (porque ellas devuelven algo que no es generalmente útil tal como “Any”).

Nil También un valor especial devuelto algunas veces por las subrutinas “void”.

Módulo Un archivo que contiene una colección de funciones relacionadas y otras definiciones.

Sentencia use Una sentencia que lee un módulo y usualmente exporta algunas funciones.

Composición Usar una expresión como parte de una expresión más compleja o una sentencia como parte de una sentencia más larga.

Flujo de ejecución El orden en cual las sentencias son ejecutadas.

Diagrama de pila Una representación gráfica de una pila de subrutinas, sus variables, y los valores a los cuales hacen referencia.

Marco Una caja en un diagrama de pila que representa una llamada de subrutina. Contiene las variables locales y los parámetros de la subrutina.

Función fructuosa Una función que devuelve un valor útil.

Función void Una función o subrutina que no devuelve un valor útil.

Signatura de una función La parte de la definición de una función (usualmente entre paréntesis) que define sus parámetros y posiblemente sus tipos y otras propiedades.

Parámetros inmutables Un parámetro de una función o subrutina que no se puede cambiar dentro del cuerpo de la función. Por defecto, los parámetros de una subrutina son inmutables.

Rasgo Una propiedad de un parámetro de una función o subrutina que es definido al tiempo de compilación.

Objeto de primera clase Las subrutinas de Raku se dice que son objetos de orden superior o objetos de primera clase porque ellas se pueden pasar como argumentos a otras subrutinas o como valores de retorno, como cualquier otro objeto.

Función anónima Una función que no tiene nombre.

Factoría de funciones Una función que produce otras funciones como valores de retorno.

3.18 Ejercicios

Ejercicio 3.1. *Escribe una subrutina llamada `alinear-a-derecha` que toma una cadena de texto llamada `$cadena-entrada` como parámetro e imprime la cadena de texto con suficiente espacio blanco de tal modo que la última letra de la cadena de texto esté en la columna 70 de la pantalla.*

```
> alinear-a-derecha('Larry Wall')
```

Larry Wall

Pista: Usa la concatenación de cadena de texto y repetición. De igual manera, Raku provee una función integrada llamada `chars` que devuelve la longitud de una cadena, así que el valor de `chars 'Larry Wall'` or `'Larry Wall'.chars` es 10. Solución: A.1.1.

Ejercicio 3.2. *Hemos visto que las funciones y otros objetos de código se pueden pasar como valores, tal como cualquier objeto. Se dice que las funciones objetos de primera clase. Por ejemplo, `do-twice` es una función que toma una función como un argumento y la llama dos veces:*

```

sub do-twice( $código ) {
    $código();
    $código();
}
sub saludar {
    say "¡Hola Mundo!";
}
do-twice(&saludar);

```

1. Escribe este ejemplo en un script y pruébalo.
2. Modifica la función `do-twice` para que tome dos argumentos, una función y un valor, y llama la función dos veces, pasando el valor como un argumento.
3. Copia la definición de `doble-impresión` del inicio de este capítulo en tu script.
4. Usa la versión modificada de `do-twice` para llamar a `doble-impresión` dos veces, pasando como argumento la cadena de texto "What's up doc".
5. Define una función nueva llamada `do-four` que toma una función y un valor y llama la función cuatro veces, pasando el valor como un argumento. Deberían haber solo dos sentencias en el cuerpo de la función, no cuatro.

Solución: A.1.2.

Ejercicio 3.3. Nota: Debes resolver este ejercicio con solo el uso de las sentencias y otras características que hemos discutido hasta ahora.

1. Escribe una subrutina que dibuje una cuadrícula como la siguiente:

```

+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +
|           |           |
|           |           |
|           |           |
|           |           |
+ - - - - + - - - - +

```

Pista: para imprimir más de un valor en una línea, puedes imprimir una secuencia de valores separados por coma:

```
say '+', '-';
```

La función `say` imprime sus argumentos con una nueva línea al final (la cual avanza el cursor a la siguiente línea). Si no quieres colocar el cursor en la siguiente línea, entonces usa la función `print`:

```
print '+', ' ';
print '-';
```

La salida de estas sentencias es "+ -".

Una sentencia `say` con una cadena de texto vacía como argumento termina la actual línea y va a la siguiente línea.

2. *Escribe una subrutina que dibuje una cuadrícula similar con cuatro filas y cuatro columnas.*

Solución: A.1.3.

Crédito: Este ejercicio está basado en un ejercicio en Oualline, Practical C Programming, Third Edition, O'Reilly Media, 1997.

Bucles, Condicionales y Recursión

El tema principal de este capítulo es la sentencia `if`, la cual ejecuta código diferente dependiendo del estado del programa. Pero primero quiero introducir dos operadores nuevos: división de enteros y módulo.

4.1 División de Enteros y Módulo

El operador **división de enteros**, `div`, divide dos números y los redondea a un entero. Por ejemplo, supón que una película se tarda 105 minutos. Podrías querer saber cuántas horas hay en 105 minutos. En Raku, la división convencional devuelve un número racional (en muchos lenguajes, devuelve un número de coma flotante, que es otra forma de representación interna de números que no son enteros):

```
> my $minutos = 105;
> $minutos / 60;
1.75
```

Sin embargo, normalmente nosotros no escribimos las horas con puntos decimales. La división de enteros devuelve el número entero de horas, descartando la parte fraccional:

```
> my $minutos = 105;
> my $horas = $minutos div 60;
1
```

En aritmética, la división de enteros es algunas veces llamada *división euclidiana*, la cual calcula el cociente y el residuo.

Para conseguir el residuo, podrías sustraer una hora de los minutos:

```
> my $residuo = $minutos - $horas * 60;
45
```

Una alternativa es usar el **operador de módulo**, %, el cual divide dos números y devuelve el residuo:

```
> my $residuo = $minutos % 60;
45
```

El operador de módulo es muy común en los lenguajes de programación y es más útil de lo que parece. Por ejemplo, puedes chequear si un número es divisible por otro—si `$dividendo % $divisor` es cero, entonces `$dividendo` es divisible por `$divisor`. Esto es usado comúnmente, por ejemplo, con un divisor igual a 2 para determinar si un número es par o impar. Veremos un ejemplo de esto más adelante en este capítulo (ver Sección 4.5).

Para ser sincero, Raku también tiene un operador específico para la divisibilidad, `%%`. La expresión `$dividendo %% $divisor` devuelve verdadero si `$dividendo %% $divisor` es igual a 0, es decir si `$dividendo` es divisible por `$divisor` (de lo contrario, devuelve falso):

```
> 42 %% 2;
True
```

De igual manera, puedes extraer el dígito (o dígitos) más a la derecha de un número con el operador de módulo. Por ejemplo, `$x % 10` extrae el dígito más a la derecha de `$x` (en base 10). Similarmente, `$x % 100` extrae los últimos dos dígitos:

```
> 642 % 100;
42
```

4.2 Expresiones Booleanas

Una **expresión booleana** es una expresión que es verdadera o falsa. Los siguientes ejemplos usan el operador `==` para comparar los operandos numéricos y producir `True` si son iguales y de lo contrario, `False`:

```
> 5 == 5;
True
> 5 == 6;
False
```

`True` y `False` son valores especiales que pertenecen al tipo `Bool`; ellos no son cadenas de texto:

```
> say True.WHAT
(Bool)
> say False.WHAT
(Bool)
```

El operador `==` es uno de los **operadores de relación numérica** y su función es chequear si los operandos son iguales; los otros son:

```

$x != $y      # $x no es numéricamente igual a $y
$x > $y       # $x es numéricamente mayor que $y
$x < $y       # $x es numéricamente menor que $y
$x >= $y      # $x es numéricamente mayor o igual a $y
$x <= $y      # $x es numéricamente menor o igual a $y
$x === $y     # $x y $y son verdaderamente idénticos

```

Aunque estas operaciones probablemente son familiares, los símbolos en Raku son diferentes a los símbolos matemáticos. Un error común es usar un solo signo de igualdad (=) en vez de dos signos de igualdad (==). Recuerda que = es el operador de asignación y == es un operador de relación. No existe tal cosa como =<, aunque existe el operador =>, el cual no es un operador de relación sino algo completamente distinto (como veremos más adelante, dicho operador es el constructor de pares).

La diferencia entre == y === es que el primer operador chequea si los valores de los operandos son iguales y el último chequea si los operandos son verdaderamente idénticos. Como un ejemplo, considera esto:

```

say 42 == 42;      # True
say 42 == 42.0;    # True
say 42 === 42;     # True
say 42 === 42.0;   # False

```

Estos operadores de relación pueden solo comparar valores numéricos (números o variables que contienen números) o valores que se pueden coaccionar en valores numéricos, tal como, por ejemplo, la cadena de texto "42" la cual, si se usa con estos operadores (excepto ===), serán coaccionada en el número 42.

Para la comparación de cadenas de texto (en un tipo de comparación lexicográfica o “pseudo-alfabética”), necesitas usar los **operadores de relación de cadena de texto**:

```

$x eq $y      # $x es una cadena de texto igual a $y
$x ne $y      # $x no es una cadena de texto igual a $y
$x gt $y      # $x es mayor que $y (alfabéticamente después)
$x lt $y      # $x es menor que $y (alfabéticamente antes)
$x ge $y      # $x es mayor o igual a $y
$x le $y      # $x es menor o igual a $y
$x eqv $y     # $x es verdaderamente equivalente a $y

```

Por ejemplo, puedes comparar (alfabéticamente) los nombres de dos presidentes de los Estados Unidos:

```

> 'FDR' eq 'JFK';
False
> 'FDR' lt 'JFK';    # comparación alfabética
True

```

A diferencia de otros lenguajes de programación, Raku te permite encadenar varios operadores de relación de forma transitiva, al igual que en la notación matemática:

```

say 4 < 7 < 12;      # True
say 4 < 7 < 5;       # False

```


Es necesario aclarar que los operadores de relación numérica y los operadores de relación de cadena de texto no funcionan de la misma manera (y esta es una buena razón para tener operadores diferentes), porque ellos no poseen la misma idea de lo que es *mayor que* o *menor que*.

Al comparar dos enteros positivos, un número con cuatro dígitos es siempre mayor que un número con solo dos o tres dígitos. Por ejemplo, 1110 es mayor que 886.

Por lo contrario, la comparación de cadenas de texto sigue básicamente reglas (pseudo) alfabéticas: "b" es mayor que "aaa", porque la regla comúnmente aceptada para la comparación de cadenas de texto es comenzar a comparar la primera letra de cada cadena de texto: si las dos letras son diferentes se sabe cuál de las dos cadenas es mayor sin importar el carácter que viene a continuación; necesitas proceder con la comparación de la segunda letra de cada palabra solo si la comparación de la primera letra de cada cadena termina en un empate, etc. Así que cualquier palabra que inicia con "a" es menor que cualquier palabra que inicie con "b", sin importar la longitud de estas palabras. Esto puede parecer algo minucioso, no obstante esto se vuelve esencial cuando comienzas a ordenar elementos: realmente tienes que pensar qué tipo de orden (numérico o alfabético) quieres usar.

También existen los operadores de relación de "tres sentidos", `cmp`, `<=>` y `leg`, pero regresaremos a ellos cuando estudiemos cómo ordenar los elementos de una lista. Similarmente, necesitamos aprender otras cosas sobre Raku antes que podamos justificar el increíblemente poderoso y expresivo operador de *coincidencia inteligente*, `~~`.

Un punto final acerca de la comparación de cadenas de texto es que las letras mayúsculas son siempre menores que las letras minúsculas. Así que "A", "B", "BB" y "C" son *todas* menores que "a", "b", "bb", y "c". No discutiremos los detalles aquí pero esto se vuelve complicado (y algo confuso) cuando las cadenas de texto a ser comparadas contienen caracteres no alfabéticos (o letras Unicode que no son ASCII).

4.3 Operadores Lógicos

Hay tres pares principales de **operadores lógicos**:

- lógico *y* : "and" y `&&`
- lógico *o* : "or" y `||`
- lógico *no* : "not" y `!`

La semántica (significado) de estos operadores es similar a sus significados en inglés (y en español). Por ejemplo, `$x > 0 and $x < 10` es verdadera solo si `$x` es mayor que 0 *and* (y) menor que 10.

`$n % 2 == 0 and $n % 3 == 0` es verdadero si *ambas* condiciones son verdaderas, es decir, si el número es divisible por 2 *and* (y) por 3. En otras palabras, el número es divisible por 6 (que podría ser mejor escrito así: `$n % 6 == 0` or `$n %% 6`).

`$n % 2 == 0 or $n % 3 == 0` es verdadero si *una o ambas* condiciones son verdaderas, es decir, si el número es divisible por 2 *or* (o) por 3 (o ambos).

Finalmente, el operador `not` niega una expresión Booleana, así que `not (x > y)` es verdadero si `x > y` es falso, es decir, si `x` es menor o igual a `y`.

Los operadores `&&`, `||`, y `!` tienen el mismo significado, respectivamente, como `and`, `or`, y `not`, pero una precedencia más estricta, lo que significa que cuando están en una expresión con otros operadores, ellos tienen una mayor precedencia de ejecución. Volveremos con la precedencia más adelante, pero digamos por el tiempo presente que, en los casos más comunes, los operadores `and`, `or`, y `not` usualmente serán suficientes para hacer lo que desees.

En sentido estricto, los operandos de los operadores lógicos deberían ser expresiones Booleanas, pero Raku, como muchos otros lenguajes derivados parcialmente de C, no es estricto con respecto a eso. Los números `0` y `0.0` son falsos (`False`); y cualquier número que no sea cero o una cadena de texto que no esté vacía es interpretado como `True`:

```
> 42 and True;  
True
```

Aunque esta flexibilidad puede ser muy útil, existen sutilezas que pueden ser confusas. A menos que sepas lo que estás haciendo, esto es algo que deberías evitar.

La función integrada `so` devuelve una evaluación de sus argumentos:

```
> say so (0 and True);  
False
```

Aquí la expresión `(0 and True)` es falsa porque `0` es falso y la expresión sería verdadera solo si ambos argumentos del operador `and` son verdaderos.

Cuando varias condiciones Booleanas son enlazadas con algunos operadores lógicos, Raku solo realizará la comparaciones que son estrictamente necesarias para descifrar el resultado final, comenzando con aquellas en la izquierda. Por ejemplo, si escribes:

```
> False and $numero > 0;  
False
```

no hay necesidad de evaluar la segunda expresión Booleana para saber que la expresión completa será falsa. En este caso, Raku no trata de chequear si el número es positivo o hasta si está definido. Se dice que estos operadores realizan una evaluación de “corto-circuito” con las condiciones innecesarias.

De la misma manera, en el siguiente código, la subrutina `calcular-pensión` no será llamada si la edad de la persona es menos de 65:

```
$edad >= 65 and calcular-pensión();
```

Lo mismo ocurre con el operador `or`, pero de manera completamente distinta: si la primera expresión booleana de una sentencia `or` es verdadera, entonces la siguiente expresión no será evaluada. El siguiente código es equivalente al anterior:

```
$edad < 65 or calcular-pensión();
```

Esta *puede* ser una forma de ejecutar la subrutina `calcular-pensión` condicionalmente, dependiendo del valor de la edad, y esto se usa usualmente en construcciones tales como:

```
haz-algo() or die "no pude hacer algo";
```

la cual aborta el programa si `haz-algo` devuelve un valor falso, lo cual quiere decir que fue incapaz de hacer algo tan esencial que no vale la pena continuar con su ejecución.

Más adelante examinaremos formas más claras y comunes de ejecutar código con condicionales.

4.4 Ejecuciones Condicionales

Para escribir programas útiles, casi siempre necesitamos la habilidad de evaluar condiciones y cambiar el comportamiento del programa adecuadamente. Las **sentencias condicionales** nos brindan esta habilidad. La forma más simple es la sentencia `if`:

```
if $numero > 0 {  
    say '$numero es positivo';  
}
```

La expresión Booleana después de `if` se conoce como la **condición**. Si la condición es verdadera, el bloque de código subsecuente se ejecuta. Si es falsa, nada pasa. El bloque de código puede contener cualquier cantidad de sentencias.

Es una convención y muy recomendado (aunque no es obligatorio desde la perspectiva del compilador) indentar las sentencias del bloque, para así ayudar con la visualización del *flujo de control* del programa, es decir, su estructura de ejecución: con tal indentación, podemos ver mucho mejor que las condiciones dentro del bloque serán ejecutadas solo si la condición es verdadera.

La condición puede ser una expresión Booleana compuesta:

```
if $n > 0 and $n < 20 and $n %% 2 {  
    say '$n es un número par y positivo menor que 20'  
}
```

Nota que en la sentencia de impresión anterior, el punto y coma final se omitió. Cuando una sentencia es la última línea de código de un bloque, inmediatamente antes de la llave derecha `}` que cierra el bloque, el punto y coma final es opcional y puede ser omitido, aunque sería recomendable incluirlo.

En teoría, el fragmento de código anterior es por sí mismo una sentencia y debería también terminar con un punto y coma después de la llave derecha. Pero una llave derecha seguida por un carácter de nueva línea implica un separador de sentencia, así que no necesitas un punto y coma ahí y por lo tanto, es generalmente omitido.

4.5 Ejecución Alternativa

Una segunda forma de la sentencia `if` es la “ejecución alternativa”, en la cual existen dos posibilidades y la condición determina cual de ellas ejecutar. Dada una variable `$numero` que contiene un entero, el siguiente código muestra dos mensajes diferentes dependiendo si el valor del entero es par o impar:

```
if $número % 2 == 0 {  
    say 'La variable $número es par'  
} else {  
    say 'La variable $número es impar'  
}
```

Si el residuo del \$número dividido por 2 es 0, entonces sabemos que el \$número es par, y el programa muestra el mensaje apropiado. Si la condición es falsa, el segundo conjunto de sentencias se ejecuta. Dado que la condición debe ser verdadera o falsa, exactamente una de las alternativas será ejecutada. Las alternativas son conocidas como **ramas**, porque son ramas en el flujo de ejecución.

Nota que si \$número divide a dos exactamente, se imprimirá este código:

```
La variable $número es par
```

El valor de la variable \$número no es interpolado, porque hemos usado las comillas simples con el propósito de imprimir el nombre de la variable y no su valor. Habríamos usado las comillas dobles si hubiésemos querido imprimir el valor de la variable y no el nombre.

4.6 Condicionales Encadenadas

Algunas veces existen más de dos posibilidades y necesitamos más de dos ramas en el flujo de ejecución. Una forma de expresar una computación como ésta es con el uso de una **condicional encadenada**:

```
if $x < $y {  
    say 'La variable $x es menor que la variable $y'  
} elsif $x > $y {  
    say 'La variable $x es mayor que la variable $y'  
} else {  
    say 'Las variables $x y $y son iguales'  
}
```

La palabra clave `elsif` es una abreviación de “else if” que tiene la ventaja de prevenir los bloques anidados. Otra vez, exactamente una de las ramas será ejecutada. No hay límite con el número de sentencias `elsif`.

Si existe una cláusula `else`, tiene que ir al final. Sin embargo, no tiene que haber una:

```
if $selección eq 'a' {  
    draw_a()  
} elsif $selección eq 'b' {  
    draw_b()  
} elsif $selección eq 'c' {  
    draw_c()  
}
```

Cada condición es examinada en orden. Si la primera es falsa, la siguiente es examinada, etc. Si una de ellas es verdadera, la rama correspondiente se ejecuta y la sentencia finaliza. Aún si más de una condición es verdadera, solo la primera rama que es verdadera se ejecuta.

4.7 Condicionales Anidadas

Una condicional puede también ser anidada dentro de otra. Podríamos haber escrito el ejemplo anterior de la siguiente manera:

```
if $x == $y {
    say 'Las variables $x y $y son iguales'
} else {
    if $x < $y {
        say 'La variable $x es menor que la variable $y'
    } else {
        say 'La variable $x es mayor que la variable $y'
    }
}
```

La condicional exterior contiene dos ramas. La primera rama contiene una sentencia simple. La segunda rama contiene otra sentencia `if`, la cual contiene dos ramas. Estas dos ramas son sentencias simples, aunque pudieran haber sido sentencias condicionales. Se dice que la condición `if $x < $y` está anidada dentro de la rama `else` de la condicional exterior.

Tales condicionales anidadas muestran lo crítico que es para tu propia comprensión intentar apropiadamente las sentencias condicionales, debido a que sería difícil entender la estructura sin la ayuda visual proveída por la correcta indentación.

Aunque la indentación de las sentencias ayuda a entender la estructura, la lectura de las **condicionales anidadas** se complica rápidamente. Es una buena idea evitarlas cuando sea posible. Los operadores lógicos usualmente proveen una manera de simplificar las condicionales anidadas. Por ejemplo, considera el siguiente código (el cual asume que `$x` es un entero):

```
my Int $x;
# ... $x = ...;
if 0 < $x {
    if $x < 10 {
        say 'El valor de $x es un número positivo de un solo dígito.'
    }
}
```

La sentencia `say` se ejecuta solo si pasamos ambas condicionales, así que podemos obtener el mismo efecto con el operador Booleano `and`, y el código puede escribirse usando una sola condicional:

```
if 0 < $x and $x < 10 {
    say '$x es un número positivo de un solo dígito.'
}
```

Para este tipo de condición, Raku provee una opción más concisa usando los operadores de relación encadenados discutidos anteriormente:

```
if 0 < $x < 10 {
    say '$x es un número positivo de un solo dígito.'
}
```

4.8 Condicionales if como Modificadores de Sentencias

También existe una forma de if conocida como un **modificador de sentencia** (o algunas veces “condicional sufija”) cuando hay solo una sentencia condicional. En este caso, la sentencia if y la condición vienen después del código que quieres ejecutar condicionalmente. Nota que la condición es siempre la primera en ser evaluada:

```
say '$número es negativo.' if $número < 0;
```

Esto es equivalente a:

```
if $número < 0 {  
    say '$número es negativo.'  
}
```

Esta forma sintáctica es más concisa debido a que toma una sola línea de código en vez de tres. La ventaja de esto es que puedes ver más de tu código en una sola pantalla, sin tener que desplazarte hacia arriba o hacia abajo. Sin embargo, esta sintaxis es clara y limpia solo cuando la condición y la sentencia son cortas y simples. Por lo tanto, es mejor usarlas en estos casos.

El modificador de sentencia no permite el uso de las sentencias else y elsif.

4.9 Sentencia Condicional Unless

Si no te gusta escribir condiciones negativas en una sentencia condicional if tal como:

```
if not $número >= 0 {  
    say '$número es negativo.'  
}
```

podrías escribirla así:

```
unless $número >= 0 {  
    say '$número es negativo.'  
}
```

Esta palabra clave unless tiene el mismo significado que la palabra en inglés: mostrará la sentencia “\$número es negativo.” *a menos que* (unless) el número sea mayor o igual a 0.

No puedes usar las sentencias else o elsif con unless, porque eso se volvería confuso.

La condicional unless se usa más en su forma de modificador de sentencia (o notación de sufijo):

```
say '$número es negativo.' unless $número >= 0;
```

4.10 Bucles For

Supón que necesitas computar e imprimir el producto de los primeros cinco dígitos positivos (de 1 a 5). Este producto es conocido en matemática como el *factorial* de 5 y se denota por 5!. Podrías escribir el siguiente programa:

```
my $producto = 1 * 2 * 3 * 4 * 5;
say $producto;           # imprime 120
```

Podrías hacerlo un poco más simple:

```
say 2 * 3 * 4 * 5;       # imprime 120
```

El problema es que esta construcción sintáctica no escala muy bien y se vuelve tediosa para el producto de los primeros diez enteros (o factorial de 10). Y se vuelve una pesadilla para el factorial de 100. La computación del factorial de un número es algo común en matemáticas (especialmente en los campos de la combinatoria y probabilidad) y la ciencia de la computación. Necesitamos automatizarlo, y el uso del bucle `for` es una de las maneras más obvia de hacerlo:

```
my $producto = 1;
for 1..5 {
    $producto *= $_
}
say $producto;           # imprime 120
```

Ahora, si necesitas calcular el factorial de 100, solo necesitas reemplazar el 5 en el código más arriba con el 100. Ten presente que la función factorial es conocida por crecer extremadamente rápido, y obtendrás un número realmente grande, con 158 dígitos (o sea, un número mucho más grande que el número total estimado de átomos en el universo conocido).

En este script, `1..5` es el operando de rango, el cual es usado aquí para generar una lista de números consecutivos entre 1 y 5. La palabra clave `for` es usada para iterar sobre una lista, y la variable `$_` es una variable especial que toma cada valor sucesivo de esta lista: primero 1, después 2, etc. hasta 5. En el bloque de código que forma el cuerpo del bucle, la variable `$producto` es multiplicada sucesivamente por cada valor de `$_`. El bucle termina con 5 y el resultado, 120, se imprime en la última línea.

Esto es un uso simple de la sentencia `for`, pero probablemente no la más usada en Raku; veremos más formas de hacerlo adelante. También veremos otros tipos de bucles. Pero esto debería ser suficiente para que escribas algunos bucles. Los bucles se encuentran en todas partes en la ciencia de la computación.

La variable especial `$_` es conocida como la *variable tónica*. No necesita ser declarada y muchas de las construcciones sintácticas le asignan un valor sin mencionarla explícitamente. De igual manera, la variable `$_` es un argumento implícito de los métodos que se llaman sin un invocante. Por ejemplo, para imprimir los primeros cinco dígitos, podrías escribir:

```
for 1..5 { .say }; # imprime los números del 1 al 5, cada uno en su línea
```

Aquí `.say` es un atajo de sintaxis equivalente a `$_ .say`. Y debido a que, como vimos, `$_` toma cada valor sucesivo del rango introducido por la palabra clave `for`, este atajo imprime cada número entre 1 y 5, cada uno de ellos en una línea distinta. Esto es un ejemplo típico del uso de la variable tópica `$_` sin ser explícitamente mencionada. Más adelante veremos otros usos de la variable especial `$_`.

Algunas veces, no necesitas la variable `$_` dentro de un bucle, por ejemplo si quieres hacer algo cinco veces pero no te importa a cuál iteración del bucle has llegado. Una subrutina que imprime un mensaje un número n de veces podría lucir de la siguiente manera:

```
sub imprime-n-veces( Int $n, Str $mensaje ) {
    for 1..$n { say $mensaje }
}
```

El bucle `for` tiene una forma de modificador de sentencia (o forma sufijo), usada aquí para calcular otra vez el factorial de 5:

```
my $producto = 1;
$producto *= $_ for 1..5;
say $producto;           # imprime 120
```

Existe otra forma de sintaxis del bucle `for`, usando una variable de bucle explícita:

```
sub factorial( Int $num ) {
    my $producto = 1;
    for 1..$num -> $x {
        $producto *= $x
    }
    return $producto
}
say factorial 10;      # 3628800
```

El bucle `for` en esta subrutina usa lo que se llama una sintaxis de “bloque puntiagudo”. Es esencialmente la misma idea del bucle `for` anterior, excepto que, en vez de usar la variable tópica `$_`, ahora declaramos una variable de bucle explícita `$x` con la sintaxis `1..$num -> $x` para iterar sobre el rango de valores. El uso de una variable de bucle explícita puede hacer tu código más claro cuando las cosas se complican, por ejemplo cuando necesitas anidar varios bucles `for`. Más adelante veremos más ejemplos.

También veremos otras formas de calcular el factorial de un número en este libro.

4.11 Recursión

Es legal que una función o subrutina llame a otra; también es legal que una subrutina se llame a sí misma. Podría no ser obvio porque esto es algo bueno, sin embargo resulta que es una de las cosas más brillante y mágica que un programa puede hacer. Por ejemplo, observa la siguiente subrutina:

```
sub cuenta-regresiva( Int $tiempo-restante ) {
    if $tiempo-restante <= 0 {
```



```

        say '¡Despegue!';
    } else {
        say $tiempo-restante;
        cuenta-regresiva($tiempo-restante - 1);
    }
}

```

Si `$tiempo-restante` es 0 o negativo, muestra la palabra “¡Despegue!”. Por lo contrario, muestra el valor de `$tiempo-restante` y después llama a la subrutina `cuenta-regresiva`—ella misma—y pasa a `$tiempo-restante - 1` como argumento.

¿Qué pasa si llamamos la subrutina de esta manera?

```
cuenta-regresiva(3);
```

La ejecución de `cuenta-regresiva` comienza con `$tiempo-restante = 3`, y dado que `$tiempo-restante` es mayor que 0, la subrutina muestra el valor 3, y después se llama a sí misma...

La ejecución de `cuenta-regresiva` comienza con `$tiempo-restante = 2`, y dado que `$tiempo-restante` es mayor que 0, la subrutina muestra el valor 2, y después se llama a sí misma...

La ejecución de `cuenta-regresiva` comienza con `$tiempo-restante = 1`, y dado que `$tiempo-restante` es mayor que 0, la subrutina muestra el valor 1, y después se llama a sí misma...

La ejecución de `cuenta-regresiva` comienza con `$tiempo-restante = 0`, y dado que `$tiempo-restante` no es mayor que 0, la subrutina muestra la palabra, “¡Despegue!”, y después devuelve.

La `cuenta-regresiva` que obtuvo `$tiempo-restante = 1` devuelve.

La `cuenta-regresiva` que obtuvo `$tiempo-restante = 2` devuelve.

La `cuenta-regresiva` que obtuvo `$tiempo-restante = 3` devuelve.

Y entonces, te encuentras en el programa principal. Así que la salida de texto luce de la siguiente manera:

```

3
2
1
¡Despegue!

```

Una subrutina que se llama a sí misma es **recursiva**; el proceso de ejecutarla se conoce como **recursión**.

Como otro ejemplo, podemos escribir una subrutina que imprime una cadena de texto `$n` veces:

```

sub imprime-n-veces( Str $sentencia, Int $n ) {
    return if $n <= 0;
    say $sentencia;
    imprime-n-veces($sentencia, $n - 1);
}

```

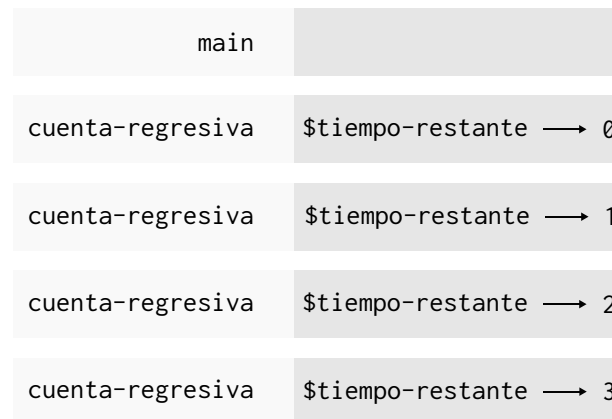


Fig. 4.1: Diagrama de pila.

Si $n \leq 0$, la **sentencia return** termina la ejecución de la subrutina. El flujo de ejecución inmediatamente se devuelve a la función que hace la llamada, y las líneas restantes de la subrutina no se ejecutan. Esto ilustra una característica de la sentencia `return` que no hemos visto antes: La sentencia `return` puede usarse como un control de flujo, es decir, para terminar la ejecución de la subrutina y devolver el control a la subrutina que hace la llamada. Nota también que, aquí, la sentencia `return` no devuelve ningún valor a la función que hace la llamada; la función `imprime-n-veces` es una función *void*.

El resto de la subrutina es similar a `cuenta-regresiva`: la subrutina muestra a `$sentencia` y después se llama a sí misma para mostrar a `$sentencia` $n - 1$ veces adicionales. Por lo tanto el número de líneas de salida de texto es $1 + (n - 1)$, lo que equivale a n .

Para ejemplos simples como este, puede parecer fácil usar un bucle `for`. Pero más adelante veremos ejemplos que son difíciles de escribir con un bucle `for` pero son fáciles de escribir con recursión, por lo tanto es bueno empezar temprano.

4.12 Diagramas de Pilas para Subrutinas Recursivas

En la Sección 3.10, usamos un diagrama de pila para representar el estado de un programa durante la llamada de una subrutina. El mismo tipo de diagrama puede ayudarnos a interpretar una subrutina recursiva.

Cada vez que una subrutina es llamada, Raku crea un marco para contener las variables locales de la subrutina y los parámetros. Para una subrutina recursiva, podría haber más de un marco en la pila al mismo tiempo.

La Figura 4.1 muestra un diagrama de pila para `cuenta-regresiva` llamada con $n = 3$.

Como siempre, la parte superior de la pila es el marco de del programa principal.

Está vacío porque no creamos ninguna variable dentro del marco. Tampoco pasamos ningún argumento al marco.

Los cuatro marcos de cuenta-regresiva tienen diferentes valores para el parámetro `$tiempo-restante`. La parte inferior de la pila, donde `$tiempo-restante = 0`, se conoce como el **caso base**. El caso base no realiza una llamada recursiva y por lo tanto, no hay más cuadros.

Como un ejercicio, dibuja un diagrama de pila para la subrutina `imprime-n-veces` la cual se llama con los argumentos `$sentencia = 'Hola'` y `$n = 2`. Después escribe una función llamada `hazlo-n-veces` que toma una función y un número, `$num`, como argumentos, y llama la función provista `$num` veces.

Solución: ver Solución: A.2

4.13 Recursión Infinita

Si una recursión nunca alcanza un caso base, se mantiene haciendo llamadas recursivas, y el programa nunca termina. Esto se conoce como **recursión infinita**, y generalmente no es una buena idea. De hecho, tu programa no se ejecutará por siempre sino que terminará en algún punto cuando la computadora agote la memoria disponible o algún otro recurso crítico.

Tienes que ser cuidadoso al escribir subrutinas recursivas. Asegúrate de tener un caso base, y comprueba que el programa lo alcanzará. Actualmente, aunque esto no es absolutamente requerido por el lenguaje, te aconsejo que crees el hábito de tratar el caso base primero.

4.14 Entrada del Teclado

Los programas que hemos escrito hasta ahora no aceptan la entrada de texto por el usuario. Ellos hacen las mismas cosas una y otra vez. Raku provee funciones integradas para parar el programa y esperar que el usuario escriba algo.

Por ejemplo, la función `prompt` incita al usuario a escribir algo a través de una pregunta o una instrucción. Cuando el usuario presiona Return o Enter, el programa resume y la función `prompt` devuelve lo que el usuario escribió como una cadena de texto (sin el carácter de nueva línea que corresponde a la tecla Return que el usuario pulsa):

```
my $usuario = prompt "Por favor escribe tu nombre: ";
say "Hola $usuario";
```

Esta es probablemente una de las maneras más común de obtener entrada interactiva de texto desde el usuario, porque es una buena idea dejarle saber lo que se espera.

Otra posibilidad es usar el método `get` (el cual lee una sola línea) en la entrada estándar:

```
say "Por favor escribe tu nombre: ";
my $usuario = $*IN.get;
say "Hola $usuario";
```

o la función `get`, la cual lee una línea de la entrada estándar por defecto:

```
say "Por favor escribe tu nombre: ";
my $usuario = get;
say "Hola $usuario";
```

4.15 Argumentos del Programa y la Subrutina MAIN

Hay otra forma (y usualmente mejor) de hacer que un programa use entradas variadas definidas por el usuario, lo cual se refiere a pasar argumentos desde la línea de comando al programa, de la misma manera en la que pasamos argumentos a nuestras subrutinas.

La manera más fácil de extraer argumentos que han sido pasados al programa es con el uso de la subrutina especial MAIN. Un programa que tiene una subrutina MAIN definida llamará automáticamente a dicha subrutina, y los argumentos de la línea de comando suministrados al programa serán pasados como argumentos a MAIN. La signatura MAIN te permitirá extraer los argumentos suministrados en la línea de comando y posiblemente chequear la validez de los argumentos.

Por ejemplo, el programa `saludar.raku` podría lucir así:

```
sub MAIN( Str $nombre ) {  
    say "Hola $nombre";  
}
```

Puedes llamar este programa dos veces con diferentes argumentos en la línea de comando de tal forma:

```
$ raku saludar.raku Larry  
Hola Larry  
  
$ raku saludar.raku mundo  
Hola mundo
```

Es muy fácil cambiar el argumento, debido a que todo lo que necesitas hacer en la línea de comando del sistema operativo es usar la flecha y editar el final de la línea de comando anterior.

Si olvidas suministrar un argumento (o provees el número equivocado de argumentos, o los argumentos no coinciden con la signatura), el programa terminará y Raku generará y mostrará el modo de uso:

```
$ raku saludar.raku  
Usage:  
saludar.raku <nombre>
```

4.16 Depuración de programas

Cuando ocurre un error de sintaxis o un error al tiempo de ejecución, el mensaje de error contiene mucha información, aunque puede ser agobiante. Las preguntas más útiles son usualmente:

- ¿Qué tipo de error fue?
- ¿Dónde ocurrió?

Los errores sintácticos son usualmente fáciles de encontrar, sin embargo existen algunos trucos. En general, los mensajes de error indican donde se descubrió el problema, aunque el actual error podría encontrarse un poco antes, algunas veces en la línea previa o hasta varias líneas antes.

Por ejemplo, el objetivo del siguiente código era mostrar la tabla de multiplicación:

```
# ADVERTENCIA: código con fallas
sub tabla-multiplicación {
    for 1..10 -> $x {
        for 1..10 -> $y {
            say "$x x $y\t= ", $x * $y;
            say "";
        }
    }

    tabla-multiplicación();
}
```

Pero falló al tiempo de compilación con el siguiente error:

```
$ raku tabla_mult.raku
===SORRY!=== Error while compiling /home/Laurent/tabla_mult.raku
Missing block (taken by some undeclared routine?)
at /home/Laurent/tabla_mult.raku:9
-----> tabla-multiplicación();<HERE><EOL>
```

El mensaje de error reporta un error en la línea 9 del programa (la última línea del código), al final de la línea, pero el error actual es que falta una llave izquierda después de la línea 4 y antes de la línea 5. La razón de esto es que, mientras el programador cometió el error en la línea 4, el interpretador de Raku no pudo detectar este error antes de alcanzar el final del programa. El programa correcto para mostrar la tabla de multiplicación podría ser:

```
sub tabla-multiplicación {
    for 1..10 -> $x {
        for 1..10 -> $y {
            say "$x x $y\t= ", $x * $y;
        }
        say "";
    }
}

tabla-multiplicación();
```

Cuando un error es reportado en la última línea de un programa, es comúnmente causado por la falta de un paréntesis, un corchete, una llave, o una comilla varias líneas más arriba. Un editor con coloración de sintaxis puede algunas veces ayudarte a resolver este error.

Lo mismo sucede con errores al tiempo de ejecución. Considera este programa cuyo objetivo es calcular 360 grados divididos sucesivamente por los enteros entre 2 y 5:

```
# ADVERTENCIA: código con fallas
my ($a, $b, $c, $d) = 2, 3, 5;
my $valor = 360;
$valor /= $_ for $a, $b, $c, $d;
say $valor;
```

Este programa compila correctamente pero muestra una advertencia y después una excepción al tiempo de ejecución:

```
Use of uninitialized value of type Any in numeric context
in block at producto.raku line 3
Attempt to divide 12 by zero using div
in block <unit> at producto.raku line 4
```

El mensaje de error indica una excepción “division by zero” en la línea 4, pero no hay nada incorrecto con esa línea. La advertencia en la línea 3 podría darnos una pista sobre el intento del script de usar un valor indefinido, pero el error real se encuentra en la primera línea del script, donde uno de los cuatros enteros necesarios fue omitido por equivocación en la asignación de la lista.

Deberías prestarle atención a los mensajes de error y leerlos cuidadosamente. Sin embargo, no asumas que ellos apuntan a la causa de la excepción; usualmente ellos apuntan a problemas subsecuentes.

4.17 Glosario

División de enteros Una operación, denotada por `div`, que divide dos números y redondea el resultado.

Operador de módulo Un operador, denotado por un signo de porcentaje (%), que funciona con números enteros y devuelve el residuo al dividir un número por otro.

Expresión Booleana Una expresión cuyo valor es `True` o `False`.

Operador de relación Uno de los operadores que comparan sus operandos. Los operadores de relación numérica más comunes son `==`, `!=`, `>`, `<`, `>=`, y `<=`. Los operadores equivalentes de relación de cadenas de texto son `eq`, `ne`, `gt`, `lt`, `ge`, and `le`.

Operador lógico Uno de los operadores que combinan expresiones Booleanas: `and`, `or`, y `not`. Los operadores equivalentes de precedencia superior son `&&`, `||`, y `!`.

Sentencia condicional Una sentencia que controla el flujo de ejecución dependiendo en algunas condiciones.

Condición La expresión booleana en una sentencia condicional que determina cual de las ramas se ejecuta.

Rama Una de las secuencias alternativas de sentencias en una sentencia condicional.

Condicional encadenada Una sentencia condicional con una serie de ramas alternativas.

Condicional anidada Una sentencia condicional que aparece en una de las ramas de otra sentencia condicional.

Modificador de sentencia Una expresión condicional sufija, es decir, una expresión condicional (usando por ejemplo `if`, `unless` o `for`) que es colocada después de la sentencia cuya ejecución controla. También puede referirse a una expresión de bucle sufijo.

Sentencia de retorno Una sentencia que fuerza a una función a terminar de inmediato y devolver a la función que realiza la llamada.

Recursión El proceso de llamar una función que aún está en ejecución.

Caso base Una rama condicional en una función recursiva que no realiza una llamada recursiva.

Recursión infinita Una recursión que carece de un caso base, o que nunca lo alcanza. Eventualmente, una recursión infinita causa un error al tiempo de ejecución, por el cual no querrás esperar dado que podría tardar demasiado.

4.18 Ejercicios

Ejercicio 4.1. Usando los operadores división de enteros y módulo:

1. Escribe una subrutina que calcule cuantos días, horas, minutos y segundos hay en el número de segundos pasado como argumento a la subrutina.
2. Escribe un script que calcule cuantos días, horas, minutos y segundos hay en 240 000 segundos.
3. Cambia el script para calcular el número de días, horas, minutos y segundos que hay en un número de segundos proveído por el usuario cuando se le pide proveer un número de segundos.

Solución: A.2.2.

Ejercicio 4.2. El último teorema de Fermat enuncia que no existen números enteros positivos a , b , y c tales que se cumpla:

$$a^n + b^n = c^n$$

para todos los valores de n mayores que 2.

1. Escribe una función con el nombre `chequear-fermat` que toma cuatro parámetros— a , b , c , y n —y chequea si se cumple el teorema de Fermat. Si n es mayor que 2 y

$$a^n + b^n = c^n$$

el programa debería imprimir, “¡Santo dios, Fermat estaba equivocado!” Por lo contrario, el programa debería imprimir “No, eso no funciona.”

2. Escribe una subrutina que incita al usuario a entrar valores para las variables a , b , c , y n , los convierte a enteros, y usa la función `chequear-fermat` para chequear si ellos violan el teorema de Fermat.

Solución: A.2.3

Ejercicio 4.3. Si se te provee con tres palillos, podrías o no podrías ser capaz de organizarlos en un triángulo. Por ejemplo, si uno de los palillos tiene una longitud de 12 pulgadas y los restantes tienen 1 pulgada, no podrás conseguir que los palillos pequeños se encuentren en el medio. Para tres longitudes cualquiera, existe una simple prueba para ver si es posible crear un triángulo:

Si una de las tres longitudes es mayor que la suma de las otras dos, entonces no puedes formar un triángulo. De lo contrario, puedes formar un triángulo. (Si la suma de dos lados es igual al tercero, ellos forman lo que se conoce como un triángulo “degenerado”.)

1. Escribe una función llamada `es-triángulo` que toma tres números positivos como argumentos, e imprime “Sí” o “No”, dependiendo si puedes formar un triángulo con las longitudes dadas.
2. Escribe una función que incite al usuario a entrar tres longitudes y use la función `es-triángulo` para chequear si palillos con las longitudes dadas pueden formar un triángulo.

Solución: A.2.4

Ejercicio 4.4. Los números Fibonacci fueron inventados por Leonardo Fibonacci (también conocido como Leonardo de Pisa o simplemente, Fibonacci), un matemático italiano del siglo XIII.

Los números Fibonacci son una secuencia de números como tal:

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

en la cual los primeros dos números son 1 y cada número subsecuente de la secuencia está definido como la suma de los dos números anteriores (por ejemplo, $5 = 2 + 3$, $8 = 3 + 5$, etc.).

En notación matemática, los números Fibonacci podrían ser definidos por una relación de recurrencia de la siguiente manera:

$$F_1 = 1, F_2 = 1, \text{ and } F_n = F_{n-1} + F_{n-2}$$

1. Escribe un programa usando el bucle `for` que imprime en la pantalla los primeros 20 números Fibonacci.
2. Escribe un programa que incite al usuario a entrar un número n y, usando el bucle `for`, calcule y muestre el número Fibonacci n° .

Solución: A.2.5

Ejercicio 4.5. ¿Cuál es la salida del siguiente programa? Dibuja un diagrama de pila que muestre el estado del programa cuando imprime el resultado.

```
sub recurse( $n, $s ) {
    if ($n == 0) {
        say $s;
    } else {
        recurse $n - 1, $n + $s;
    }
}
```

```
recurse 3, 0;
```

1. ¿Qué pasaría si llamaras la función de la siguiente forma: `recurse(-1, 0)`?
2. Escribe un comentario de documentación (quizás en la forma de un comentario multilínea) que explique todo lo que alguien necesita saber para usar esta función (y nada más).

Solución: A.2.6

Subrutinas Fructuosas

La mayoría de funciones de Raku que hemos usado, tales como las funciones matemáticas, producen valores de retorno. Aún así, la mayoría de funciones que hemos escrito hasta ahora son funciones void: ellas tienen un efecto, como imprimir un valor, pero no tienen un valor de retorno. En este capítulo, aprenderás a escribir funciones fructuosas.

5.1 Valores de Retorno

Al llamar una función fructuosa se genera un valor de retorno, el cual usualmente asignamos a una variable o usamos como parte de una expresión:

```
my $pi = 4 * atan 1;  
my $altura = $radio * sin $radianes;
```

Muchas de las subrutinas que hemos escrito hasta ahora son void. Hablando casualmente, ellas no tienen un valor de retorno útil; más precisamente, su valor de retorno puede ser Any, Nil, (), o True.

En este capítulo, finalmente escribiremos subrutinas fructuosas. El primer ejemplo es `area`, la cual devuelve el área de un círculo con un radio en particular:

```
sub area( $radio ) {  
    my $area_circular = pi * $radio**2;  
    return $area_circular;  
}
```

Hemos visto la sentencia `return` anteriormente, pero en una subrutina fructuosa la sentencia `return` incluye una expresión. Esta sentencia quiere decir: “Devuelve inmediatamente desde esta función y usa la siguiente expresión como un valor de retorno.” La expresión puede ser arbitrariamente complicada, así que podríamos escribir esta función en una manera más concisa:

```

sub area( $radio ) {
    return pi * $radio**2;
}

```

Por otro lado, las **variables temporales** como \$area_circular hace la depuración mucho más fácil. Además ayudan a documentar lo que está sucediendo en la función.

Algunas veces es útil tener varias sentencias return, por ejemplo una en cada rama de una condicional:

```

sub valor_absoluto( $num ) {
    if $num < 0 {
        return -$num;
    } else {
        return $num;
    }
}

```

Dado que estas sentencias return están en una condicional alternativa, solo una se ejecuta.

Esto podría también escribirse de forma concisa usando la sintaxis del modificador de sentencia:

```

sub valor_absoluto( $num ) {
    return -$num if $num < 0;
    return $num;
}

```

Otra vez, solo una de las sentencias return se ejecuta: si el número es negativo, la primera sentencia return es ejecutada y la ejecución de la subrutina termina ahí; en cambio, si el número es positivo o cero, solo la segunda sentencia return es ejecutada.

Tan pronto una sentencia return es ejecutada, la función termina sin ejecutar las sentencias subsiguientes. Cualquier código que aparece después de una sentencia return incondicional, o cualquier otro lugar que el flujo de ejecución no puede alcanzar es conocido como **código muerto**.

En una función fructuosa, es una buena idea asegurar que cada posible ruta a través del programa se alcance con una sentencia return. Por ejemplo:

```

# ADVERTENCIA: código con fallas
sub valor_absoluto( $num ) {
    if $num < 0 {
        return -$num;
    }
    if $num > 0 {
        return $num;
    }
}

```

Esta subrutina es incorrecta porque si \$num es 0, ninguna de las condiciones es verdadera, y la subrutina termina sin alcanzar una sentencia return. Si el flujo de ejecución llega al final de una función, el valor de retorno es (), que básicamente significa “no definido” y que claramente no es el valor absoluto de 0:

```
> valor_absoluto(0)
()
```

Casualmente, Raku provee una función integrada llamada `abs` que calcula valores absolutos.

Como ejercicio, escribe una subrutina `comparar` que toma dos números, x y y , y devuelve 1 si $x > y$, 0 si $x == y$, y -1 si $x < y$.

Solución: A.3.1

5.2 Desarrollo Incremental

A medida que escribes funciones más complejas, puedes pasar más tiempo depurando.

Para tratar con programas complejos, podrías querer usar un proceso llamado **desarrollo incremental**. El objetivo del desarrollo incremental es evitar sesiones largas y tediosas de depuración al añadir y evaluar una pequeña pieza de código a la vez.

Como un ejemplo, supón que quieres encontrar la distancia entre dos puntos, dadas las coordenadas cartesianas o rectangulares (x_1, y_1) y (x_2, y_2) de los puntos. Por el teorema de Pitágoras, la distancia es:

$$\text{distancia} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

El primer paso es considerar cómo la función `distancia` debería lucir en Raku. En otras palabras, cuáles son las entradas (parámetros) y cuál es la salida (valor de retorno)?

En este caso, las entradas son los puntos, los cuales puedes representar usando cuatro números. El valor de retorno es la distancia representada por un valor numérico.

Inmediatamente puedes escribir un bosquejo de la función:

```
sub distancia( $x1, $y1, $x2, $y2 ) {
    return 0.0;
}
```

Obviamente, esta versión no calcula la distancia; siempre devuelve cero. Sin embargo, es sintácticamente correcta, y puede ser ejecutada, lo que significa que la probaste antes de hacerla más compleja.

Para probar la nueva función, llámala con algunos argumentos:

```
> distancia(1, 2, 4, 6);
0.0
```

Elegí estos valores para que la distancia horizontal sea 3 y la distancia vertical sea 4; de esa manera, el resultado es 5, la hipotenusa de un triángulo 3-4-5. Al probar una función, es siempre útil saber la respuesta correcta con antelación.

En este momento, hemos confirmado que la función es sintácticamente correcta, y podemos comenzar a añadir código al cuerpo de la función. El paso siguiente razonable es encontrar las diferencias $x_2 - x_1$ and $y_2 - y_1$. La siguiente versión almacena esos valores en variables temporales y los imprime:

```

sub distancia( $x1, $y1, $x2, $y2 ) {
    my $dx = $x2 - $x1;
    my $dy = $y2 - $y1;
    say '$dx es ', $dx;
    say '$dy es ', $dy;
    return 0.0;
}

```

Si la subrutina funciona como debe, debería mostrar \$dx es 3 y \$dy es 4 (y todavía devolver 0.0). Si esto sucede, sabemos que la función está recibiendo los argumentos correctos y realizando la primera computación correctamente. Por lo contrario, si esto no sucede solo tenemos que depurar algunas líneas.

El siguiente paso es computar la suma de los cuadrados de \$dx y \$dy:

```

sub distancia( $x1, $y1, $x2, $y2 ) {
    my $dx = $x2 - $x1;
    my $dy = $y2 - $y1;
    my $dist-cuadrada = $dx**2 + $dy**2;
    say '$dist-cuadrada es: ', $dist-cuadrada;
    return 0.0;
}

```

Otra vez, ejecutarías el programa en esta etapa y verificarías la salida (que debería ser 25). Finalmente, puedes usar la función integrada `sqrt` para calcular la raíz cuadrada y devolver el resultado:

```

sub distancia( $x1, $y1, $x2, $y2 ) {
    my $dx = $x2 - $x1;
    my $dy = $y2 - $y1;
    my $dist-cuadrada = $dx**2 + $dy**2;
    my $resultado = sqrt $dist-cuadrada;
    return $resultado;
}

```

Si eso funciona correctamente, ya acabaste. Por lo contrario, podrías imprimir el valor de `$resultado` antes de la sentencia `return`.

La versión final de la subrutina no muestra nada cuando se ejecuta; solo devuelve un valor. Las sentencias `print` que escribimos son útiles para la depuración, pero una vez que consigues que la subrutina funcione correctamente, puedes remover dichas sentencias. Código como ese es algunas veces llamado **andamiaje** porque es útil para construir el programa pero no forma parte del producto final.

Cuando empiezas a programar, debes agregar una o dos líneas de código al mismo tiempo. A medida que ganas experiencia, puedes entonces escribir y depurar piezas más grandes. De cualquier manera, el desarrollo incremental puede salvarte muchas horas de depuración.

Los aspectos claves del proceso son:

1. Comienza con un programa que funcione y haz pequeños cambios incrementales. En cualquier punto, si hay un error, deberías entonces tener una buena idea donde se encuentra.

2. Usa variables para almacenar los valores intermedios para que puedas mostrarlos y chequearlos.
3. Una vez que el programa funciona, podrías remover algo del código de andamiaje o consolidar sentencias múltiples en una expresión compuesta, pero solo si no dificulta la lectura del programa.

Nota que, por lo menos para casos relativamente simples, puedes usar el REPL para probar expresiones y hasta sentencias multilíneas o subrutinas en el modo interactivo antes de incluirlas en tu código. Esto es usualmente rápido y puede ahorrarte algo de tiempo.

Como un ejercicio, usa el desarrollo incremental para escribir una función llamada *hipotenusa* que devuelve la longitud de la hipotenusa de un triángulo rectángulo, dadas las longitudes de los otros dos lados como argumentos. Registra cada etapa del desarrollo a medida que avanzas.

Solución: A.3.2

5.3 Composición

Como deberías saber por ahora, puedes llamar una función desde otra. Como un ejemplo, escribiremos una función que toma dos puntos, el centro del círculo y un punto en el perímetro, y calcula el área del círculo.

Asume que el centro es almacenado en las variables `$x-c` y `$y-c`, y el perímetro está en `$x-p` y `$y-p`. El primer paso es encontrar el radio del círculo, el cual es la distancia entre los dos puntos. Anteriormente escribimos una función, *distancia*, que hace exactamente eso:

```
my $radio = distancia($x-c, $y-c, $x-p, $y-p);
```

El siguiente paso es encontrar el área de un círculo con ese radio; también ya escribimos algo para hacer eso mismo:

```
my $resultado = area($radio);
```

Al encapsular estos pasos en una función, tenemos:

```
sub area-círculo( $x-c, $y-c, $x-p, $y-p ) {
  my $radio = distancia($x-c, $y-c, $x-p, $y-p);
  my $resultado = area($radio);
  return $resultado;
}
```

Las variables temporales `$radio` y `$resultado` son útiles para el desarrollo y la depuración, pero una vez que el programa ya funciona adecuadamente, podemos hacer todo más conciso al componer las llamadas de las funciones en la siguiente manera:

```
sub area-círculo( $x-c, $y-c, $x-p, $y-p ) {
  return area distancia($x-c, $y-c, $x-p, $y-p);
}
```

La última línea del ejemplo anterior ahora funciona como una tubería (del inglés *pipeline*) de datos desde la derecha hacia la izquierda; la función `distancia` toma los cuatros argumentos y devuelve una distancia (el radio) el cual se pasa como un argumento a `area`; con este argumento, `area` es ahora capaz de devolver el área, la cual es entonces devuelta por `area-círculo` al código que hace la llamada. Regresaremos más tarde a este modelo de tubería de datos.

5.4 Funciones Booleanas

Las funciones pueden devolver valores Booleanos, lo cual es usualmente conveniente para ocultar pruebas complicadas dentro de las funciones. Por ejemplo:

```
sub es-divisible( Int $x, Int $y ) {
    if $x % $y == 0 {
        return True;
    } else {
        return False;
    }
}
```

Es común darle a las funciones Booleanas nombres que suenan como preguntas con respuestas de sí/no; `es-divisible`, por ejemplo, devuelve `True` o `False` para indicar si `x` es divisible por `y`.

Este es un ejemplo:

```
> es-divisible(6, 4);
False
> es-divisible(6, 3);
True
```

El resultado del operador `==` es un valor Booleano, así que podemos escribir la subrutina de manera concisa al devolverlo directamente:

```
sub es-divisible( Int $x, Int $y ) {
    return $x % $y == 0
}
```

Si no hay una sentencia de retorno, una subrutina en Raku devuelve el valor de la expresión en la última línea de código de la subrutina (provisto que la última expresión sea una expresión que puede ser evaluada), así que la sentencia `return` no es requerida aquí. Además, dado que 0 es un valor falso y cualquier otro entero es un valor verdadero, la misma subrutina podría escribirse de otra forma:

```
sub es-divisible( Int $x, Int $y ) {
    not $x % $y
}
```

Las declaraciones de tipo `Int` en las signatures de la subrutina más arriba no son necesarias. La subrutina funcionaría sin ellas, pero ellas pueden proveer algún tipo de protección en contra del uso de esta subrutina con argumentos defectuosos.

Las funciones Booleanas se usan usualmente en los modificadores de sentencias:

```
say "$x es divisible por $y" if es-divisible($x, $y);
```

Podría ser tentativo escribir algo como:

```
say "$x es divisible por $y" if es-divisible($x, $y) == True;
```

Pero la comparación extra no es necesaria: `es-divisible` devuelve un valor Booleano que puede ser interpretado directamente por la condicional `if`.

Como un ejercicio, escribe una función `está-entre(x, y, z)` que devuelve `True` si $x \leq y \leq z$ o por lo contrario, `False`.

Solución: A.3.3

5.5 Un Lenguaje de Programación Completo

En la sección anterior vimos varias maneras de escribir una subrutina para chequear la divisibilidad de dos números enteros.

De hecho, como fue mencionado anteriormente, Raku tiene un operador “es-divisible”, `%%`, el cual devuelve `True` si el número en la izquierda es divisible por el de la derecha:

```
> 9 %% 3
True
> 9 %% 4
False
```

Por lo tanto no había necesidad de crear la subrutina `es-divisible`. Pero no te preocupes, es totalmente normal si no recordaste eso. A los hablantes de lenguajes naturales se les permite tener diferentes niveles de cualificación, aprender a medida que avanzan, y poner el lenguaje a buen uso antes de que sepan el lenguaje completo. Lo mismo pasa con Raku. Tú (y yo) no sabemos todo sobre Raku todavía, al igual que no sabemos todo sobre el inglés (o el español). Pero es de hecho “Oficialmente Okay en la Cultura de Perl” usar el subconjunto del lenguaje que sabes. De hecho, se fomenta a que uses lo que es a veces llamado “baby Perl” para escribir programas, aunque sean un poco toscos y rudimentarios al principio. Esa es la mejor manera de aprender Perl, del mismo modo que “hablar como un bebé” (“baby talk”) es la mejor manera para que un niño aprenda inglés (o español).

El número de maneras diferentes de llevar a cabo una tarea, tal como chequear si un número es divisible por otro, es un ejemplo del lema de Perl: *there is more than one way to do it* (Hay más de una forma para hacer algo), usualmente abreviado TIMTOWTDI. Algunas maneras pueden ser más concisas o más eficientes que otras, pero, en la filosofía de Perl, tú estás perfectamente facultado para hacerlo a tu manera, especialmente si eres un principiante, provisto que encuentres el resultado correcto.

Hasta ahora, solamente hemos cubierto un pequeño subconjunto de Raku, pero te sería interesante saber que este subconjunto es un lenguaje de programación *completo*, lo cual significa que esencialmente cualquier cosa que puede computarse podría expresarse en este lenguaje. Cualquier programa que se haya escrito jamás podría escribirse de nuevo solo usando las características del lenguaje que has aprendido hasta ahora (actualmente, solo necesitarías algunos comandos para controlar dispositivos como el ratón, discos, redes, etc., pero eso es todo).

La prueba de esa afirmación no es un ejercicio trivial. El primero en probarla fue Alan Turing, uno de los primeros científicos de la computación (algunas personas argumentan que él fue un matemático, sin embargo muchos de los primeros científicos de la computación comenzaron como matemáticos). Por esta razón, se le conoce como la tesis de Church-Turing (*Church-Turing Thesis*). Para una discusión más completa (y precisa) de la tesis de Church-Turing, recomiendo el libro *Introduction to the Theory of Computation* de Michael Sipser.

5.6 Más Recursión

Para darte una idea de lo que puedes hacer con las herramientas que has aprendido hasta ahora, evaluaremos algunas funciones matemáticas definidas de forma recursiva. Una definición recursiva es similar a una definición circular, en el sentido de que la definición contiene una referencia a la cosa que se define. Una definición verdaderamente circular no es muy útil:

Terrenal Un adjetivo usado para describir algo que es terrenal.

Si ves esa definición en el diccionario, podrías enojarte. Sin embargo, si busca la definición de la función factorial, denotada con el símbolo $!$, podrías conseguir algo como esto:

$$\begin{aligned} 0! &= 1 \\ n! &= n(n-1)! \end{aligned}$$

Esta definición dice que el factorial de 0 es 1, y que el factorial de cualquier otro valor (entero positivo), n , es n multiplicado por el factorial de $n-1$.

Así que $3!$ es 3 por $2!$, el cual es 2 por $1!$, el cual es 1 por $0!$. Si lo ponemos todo junto, $3!$ es igual a 3 por 2 por 1 por 1, lo cual es 6.

Si puedes escribir una definición recursiva de algo, puedes escribir un programa de Raku para evaluar dicha definición. El primer paso es decidir cuáles deberían ser los parámetros. En este caso, es claro que el factorial toma un número¹:

```
sub factorial( $n ) {
}
```

Si el argumento es 0, todo lo que tenemos que hacer es devolver 1:

```
sub factorial( $n ) {
  if $n == 0 {
    return 1;
  }
}
```

Por el otro lado, y ésta es la parte interesante, tenemos que hacer una llamada recursiva para encontrar el factorial de $n-1$ y después multiplicarlo por n :

¹Debería ser realmente un entero, pero regresaremos a esto más tarde en este capítulo.

```
sub factorial( $n ) {  
    if $n == 0 {  
        return 1;  
    } else {  
        my $recurse = factorial($n-1);  
        my $resultado = $n * $recurse;  
        return $resultado;  
    }  
}
```

El flujo de ejecución para este programa es similar al flujo de cuenta-regresiva en la Sección 4.11. Si llamamos a `factorial` con el valor 3:

Dado que 3 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

Dado que 2 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

Dado que 1 no es 0, tomamos la segunda rama y calculamos el factorial de $n-1$...

Dado que 0 es igual a 0, tomamos la primera rama y devolvemos 1 sin hacer más llamadas recursivas.

El valor de retorno, 1, es multiplicado por n , el cual es 1, y se devuelve el resultado.

El valor de retorno, 1, es multiplicado por n , el cual es 2, y se devuelve el resultado.

El valor de retorno, 2, es multiplicado por n , el cual es 3, el resultado, 6, se convierte en el valor de retorno de la llamada de subrutina que inició todo el proceso.

La Figura 5.1 muestra como el diagrama de pila luce para esta secuencia de llamadas de función.

Los valores de retorno se muestran siendo pasados devuelta a la pila. En cada cuadro, el valor de retorno es el valor de `$resultado`, el cual es producto de n y `$recurse`.

En el último cuadro, las variables locales `$recurse` y `$resultado` no existen, porque la rama que las crea no se ejecuta.

Un programador de Raku con experiencia podría escribir una subrutina más concisa o más idiomática²:

```
sub factorial( $n ) {  
    return 1 if $n == 0;  
    return $n * factorial $n-1;  
}
```

Esto no es nada mejor que nuestra primera versión, y probablemente no se ejecutará significativamente más rápido, pero es presumiblemente más clara, por lo menos cuando te acostumbra a este tipo de sintaxis.

²Luego veremos más formas idiomáticas de computar el factorial de un número.

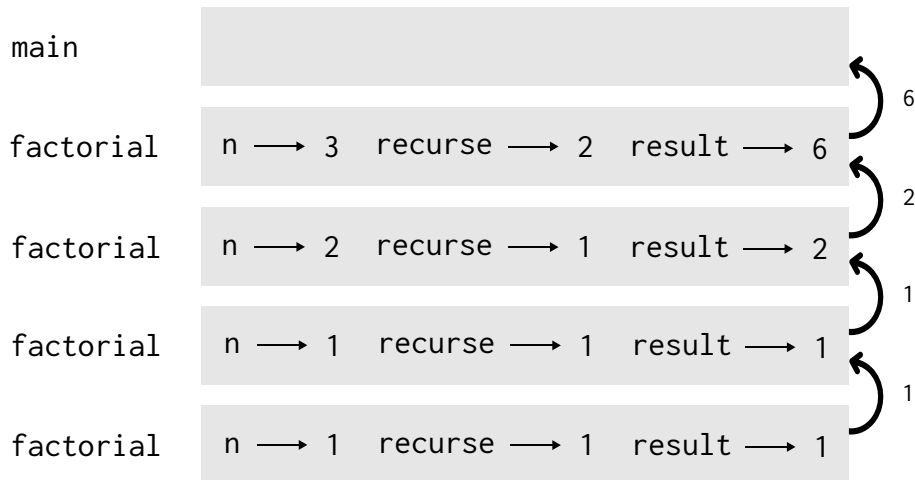


Fig. 5.1: Diagrama de pila.

5.7 Acto de Fe

Seguir el flujo de ejecución es una manera de leer un programa, pero se vuelve agobiante en poco tiempo. Una alternativa es lo que puede ser llamado el “acto de fe”. Cuando encuentras una subrutina, en lugar de seguir el flujo de ejecución, tú *asumes* que la subrutina funciona correctamente y devuelve el resultado correcto.

De hecho, ya estás practicando este acto de fe cuando usas las funciones integradas. Cuando llamas las funciones matemáticas tales como `cos` o `sqrt`, no examinas los cuerpos de esas funciones. Asumes que funcionan adecuadamente porque las personas que escribieron las funciones integradas eran probablemente buenos programadores (y porque puedes asumir seguramente que dichas funciones han sido probadas extensivamente).

Lo mismo es cierto cuando llamas una de tus subrutinas. Por ejemplo, en la Sección 5.4, escribimos una subrutina llamada `es-divisible` que determina si un número es divisible por otro. Una vez que nos convencemos que esta subrutina es correcta—al examinar y probar el código—podemos usar la subrutina sin examinar el cuerpo de la misma otra vez.

Lo mismo es cierto para programas recursivos. Cuando obtienes la llamada recursiva, en lugar de seguir el flujo de ejecución, deberías asumir que la llamada recursiva funciona (devuelve el resultado correcto) y después preguntarte, “Asumiendo que puedo encontrar el factorial de $n-1$, ¿puedo yo calcular el factorial de n ?”. Es bien claro que puedes, al multiplicar por n .

¡Por supuesto, es un poco extraño asumir que la subrutina funciona correctamente cuando no has terminado de escribirla, pero por eso es que se conoce como un acto de fe!

5.8 Otro Ejemplo Más

Después de `factorial`, el ejemplo más común de una función matemática definida recursivamente es `fibonacci`, la cual tiene la siguiente definición (ver

https://es.wikipedia.org/wiki/Sucesi3n_de_Fibonacci):

$$\begin{aligned}\text{fibonacci}(0) &= 1 \\ \text{fibonacci}(1) &= 1 \\ \text{fibonacci}(n) &= \text{fibonacci}(n-1) + \text{fibonacci}(n-2)\end{aligned}$$

En lenguaje llano, una sucesi3n de Fibonacci es una secuencia de n3meros tales como:

1, 1, 2, 3, 5, 8, 13, 21, ...

donde los dos primeros t3rminos son iguales a 1 y cualquier otro t3rmino es la suma de los dos n3meros que lo preceden.

Discutimos la secuencia de Fibonacci brevemente en el Ejercicio 4.4 del cap3tulo anterior y la implementamos con un bucle `for`.

Ahora traduzcamos la definici3n recursiva en Raku. As3 es como luce:

```
sub fibonacci( $n ) {
    return 1 if $n == 0 or $n == 1;
    return fibonacci($n-1) + fibonacci($n-2)
}
```

Si tratas de seguir el flujo de ejecuci3n aqu3, a3n para valores peque1os de `$n`, tu cabeza va a explotar. Sin embargo, de acuerdo al acto de fe, si asumimos que las llamadas recursivas funcionan correctamente, entonces es claro que obtendr3s el resultado correcto al a1adirlo todo junto.

5.9 Chequeo de Tipos

¿Qu3 pasa si llamamos `factorial` y pasamos a 1.5 como un argumento?

Al parecer conseguimos una recursi3n infinita. ¿C3mo puede ser? La subrutina tiene un caso base—cuando `$n == 0`. Pero si `$n` no es un entero, podemos *perder* el caso base y hacer la recursi3n por siempre.

En la primera llamada recursiva, el valor de `$n` es 0.5. En la siguiente, es -0.5. Desde ah3, se vuelve m3s peque1o (m3s negativo), pero nunca ser3 0.

Tenemos dos opciones. Podemos tratar de generalizar la funci3n `factorial` para funcionar con n3meros que no son enteros, o podemos hacer que `factorial` haga un chequeo de sus argumentos. La primera opci3n es llamada la funci3n gamma y est3 fuera del alcance de este libro. Por lo tanto, discutiremos la segunda opci3n.

Ya hemos visto ejemplos de subrutinas que usan la signature para verificar el tipo del argumento. As3 que podemos a1adir el tipo `Int` al par3metro en la signature. Mientras hacemos esto, tambi3n podemos asegurarnos de que el argumento sea positivo o cero:

```
sub factorial( Int $n where $n >= 0 ) {
    return 1 if $n == 0;
    return $n * factorial $n-1;
}
```

El tipo `Int` en la signatura chequea por números no enteros, lo cual ya lo sabemos. La parte `where $n >= 0` es una restricción de parámetro: si el parámetro es negativo, la subrutina debería fallar. Técnicamente, la restricción se implementa aquí dentro de la signatura usando una característica sintáctica llamada un *rasgo* (**en.** `trait`), que es una propiedad impuesta sobre el parámetro al tiempo de compilación. Si el argumento que se pasa a la función no es un entero o si es negativo, el programa imprime un mensaje de error para indicar que algo malo pasó:

```
> say factorial 1.5
Type check failed in binding $n; expected Int but got Rat
  in sub factorial at <unknown file> line 1
  in block <unit> at <unknown file> line 1

> say factorial -3
Constraint type check failed for parameter '$n'
> say factorial "Fred"
Type check failed in binding $n; expected Int but got Str
  in sub factorial at <unknown file> line 1
  in block <unit> at <unknown file> line 1
```

Si eludimos a ambos chequeos, sabemos que `$n` es un entero y que es positivo o cero, así que podemos probar que la recursión termina.

Otra modo de lograr un resultado similar es definir tu propio subconjunto de tipos integrados. Por ejemplo, puedes crear un subconjunto `Par-entero` de enteros y después usarlo más o menos como si fuera un tipo para declarar tus variables o asignarle un tipo a tus parámetros de las subrutinas:

```
subset Par-entero of Int where { $ _ %% 2 } # 0 : ... where { $ _ % 2 == 0 }
# Par-entero puede ahora usarse como un tipo

my Par-entero $x = 2; # OK
my Par-entero $y = 3; # Error de discordancia de tipo
```

De igual manera, en el caso de la subrutina `factorial`, podemos crear un subconjunto de enteros *no negativos* y usarlos para chequear el parámetro que se pasa a la subrutina:

```
subset Entero-no-neg of Int where { $ _ >= 0 }
# ...

# Usando nuestro subconjunto en el parámetro
sub factorial( Entero-no-neg $n ) {
  return 1 if $n == 0;
  return $n * factorial $n-1;
}
```

Si pasamos un número negativo a la subrutina, podemos obtener un error similar al anterior:

```
Constraint type check failed for parameter '$n'...
```

Este programa demuestra un patrón conocido como **guardián**. La signatura actúa como un guardián, que protege el código de valores que podrían causar un error. Los guardianes hacen posible probar la validez del código.

5.10 Subrutines multi

Es posible escribir versiones múltiples de una subrutina con el mismo nombre pero con firmas diferentes, por ejemplo una *aridad* diferente (una palabra sofisticada para referirse al número de argumentos) o diferentes tipos de argumentos, usando la palabra clave `multi`. En este caso, el interpretador elegirá la versión de la subrutina cuya firma coincide (o mejor coincide) con la lista de argumentos. Este proceso se conoce como *despacho*. El despacho ocurre dependiendo en el número (aridad), tipo, y nombre de los argumentos.

Por ejemplo, podríamos escribir de nuevo la función factorial como sigue:

```
multi sub fact( 0 ) { 1 };
multi sub fact( Int $n where $n > 0 ) {
    $n * fact $n - 1;
}
say fact 0;    # -> 1
say fact 10;   # -> 3628800
```

Aquí, no entramos en una recursión infinita porque, cuando el parámetro que se pasa a `fact` es 0, es la primera de la versión de la subrutina que se llama y devuelve un valor entero (1), y esto termina la recursión.

Similarmente, la función Fibonacci puede escribirse de nuevo con subrutinas `multi`:

```
multi fibonacci( 0 ) { 1 }
multi fibonacci( 1 ) { 1 }
multi fibonacci( Int $n where $n > 1 ) {
    fibonacci($n - 2) + fibonacci($n - 1)
}
say fibonacci 10;    # -> 89
```

Nota que en el ejemplo de `fibonacci` más arriba, hemos omitido la palabra clave `sub`, dado que no es necesaria después del declarador `multi`.

Muchas de las funciones integradas y muchos de los operadores de Raku son escritos como subrutinas `multi`.

En los dos ejemplos anteriores, solamente una de las firmas de la subrutina `multi` podría coincidir con los parámetros de entrada. Por ejemplo, si el argumento proveído a la subrutina `fact` es 0, entonces la primera versión es llamada; si el argumento es mayor que 0, entonces la segunda versión es llamada.

¿Qué sucede si más de una firma coincide con los argumentos proveídos? En este caso, el proceso de despacho se vuelve un poco más complicado. Como se discutió al inicio de esta sección, el entorno de ejecución seleccionará la subrutina *candidata* que *mejor* coincida con la firma. En tales casos, el entorno de ejecución primero analiza la aridad (número de argumentos) de la firma. Si solamente una subrutina coincide con el número de parámetros proveídos a la subrutina, entonces este es la subrutina candidata que se selecciona. De lo contrario, si varias subrutinas coinciden con el número de argumentos, entonces el tipo de los argumentos lleva a cabo el despacho.³

³Más adelante veremos (ver sección 11.4.1) que pueden haber otros tipos de firmas, con *parámetros nombrados*. Digamos aquí que, cuando hay argumentos nombrados, los mismos llevan a cabo el despacho aún cuando su tipo es el mismo que otro candidato.

5.11 Depuración de programas

La fragmentación de un programa grande en funciones o subrutinas más pequeñas crea puestos de control naturales para la depuración. Si una subrutina no está funcionando, existen tres posibilidades para considerar:

- Hay algo incorrecto con los argumentos que la subrutina recibe; una precondition es violada.
- Hay algo incorrecto con la subrutina; una poscondición es violada.
- Hay algo incorrecto con el valor de retorno o la forma en la cual es usado.

Para descartar la primera posibilidad, puedes agregar una sentencia de impresión al inicio de la función y mostrar los valores de los parámetros (y tal vez sus tipos). O puedes escribir código para chequear las precondiciones explícitamente.

Para el propósito de depuración, es usualmente útil imprimir el contenido de una variable o de un parámetro dentro de una cadena de texto con caracteres alrededor, para así visualizar caracteres que de otra manera son invisibles, tales como espacios o caracteres de nueva línea. Por ejemplo, piensas que la `$var` debería contener “two” y ejecutar la siguiente lo prueba:

```
if $var eq "two" {
    do-something()
}
```

Pero falla y la subrutina `do-something` nunca se ejecuta.

Quizás quieres usar una sentencia de impresión que corroborará el contenido de `$var`:

```
say "[$var]";
if $var eq "two" {
    do-something()
}
```

Esto podría imprimir:

```
[two ]
```

o:

```
[two
]
```

Ahora sabes que la prueba de igualdad falla porque `$var` contiene un carácter colgante (espacio o nueva línea) que podría ser difícil de detectar.

Si los parámetros están bien, agrega una sentencia `print` antes de cada sentencia `return` y muestra el valor de retorno. Si es posible, chequea el resultado manualmente. Considera llamar la función con valores que facilitan chequear el resultado (como en la Sección 5.2).

Si la función aparenta funcionar, observa la llamada de la función para asegurarte que el valor de retorno está siendo utilizado correctamente (o apenas siendo utilizado).

Agregar sentencias de impresión al comienzo y final de una función ayuda a hacer al flujo de ejecución más visible. Por ejemplo, la siguiente es una versión de factorial con sentencias de impresión:

```
sub factorial( Int $n ) {
  my $espacio = ' ' x (4 * $n);
  say $espacio, 'factorial ', $n;
  if $n == 0 {
    say $espacio, 'devolviendo 1';
    return 1;
  } else {
    my $resultado = $n * factorial $n-1;
    say $espacio, 'devolviendo ', $resultado;
    return $resultado;
  }
}
```

La variable \$espacio es una cadena de caracteres de espacio que controla la indentación de la salida. Este es el resultado para factorial(4):

```
        factorial 4
      factorial 3
    factorial 2
  factorial 1
factorial 0
devolviendo 1
  devolviendo 1
    devolviendo 2
      devolviendo 6
        devolviendo 24
```

Si estás confundido sobre el flujo de ejecución, este tipo de salida puede ser útil. Toma tiempo desarrollar andamiaje efectivo, pero un poco de andamiaje puede ahorrar mucho tiempo de depuración.

5.12 Glosario

Variable temporal Una variable usada para almacenar un valor intermedio en una operación compleja.

Código muerto Parte de un programa que nunca se ejecuta, usualmente porque aparece después de una sentencia return.

Desarrollo incremental Un plan de desarrollo de programa destinado a evitar la depuración al añadir y probar solo piezas pequeñas de código a la vez.

Andamiaje Código usado durante el desarrollo de un programa pero que no es parte de la versión final.

Guardián Un patrón de programación que usa una sentencia condicional para chequear y manejar circunstancias que podrían causar un error.

5.13 Ejercicios

Ejercicio 5.1. *Dibuja un diagrama de pila para el siguiente programa. ¿Qué imprime el programa? Por favor trata de contestar estas preguntas antes de ejecutar el programa.*

```
sub b( Int $z ) {
    my $prod = a($z, $z);
    say $z, " ", $prod;
    return $prod;
}
sub a( Int $x is copy, Int $y ) {
    $x++;
    return $x * $y;
}
sub c( Int $x, Int $y, Int $z ) {
    my $total = $x + $y + $z;
    my $square = b($total) ** 2;
    return $square;
}

my $x = 1;
my $y = $x + 1;
say c($x, $y + 3, $x + $y);
```

Ejercicio 5.2. *La función de Ackermann, $A(m, n)$, se define como sigue:*

$$A(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

Ver https://es.wikipedia.org/wiki/Funci%C3%B3n_de_Ackermann. Escribe una subrutina llamada `ack` que evalúa la función de Ackermann. Usa tu subrutina para evaluar `ack(3, 4)`, que debería ser 125. ¿Qué pasa con valores más grandes de `m` y `n`?

Solución: A.3.4

Ejercicio 5.3. *Un palíndromo es una palabra que se lee igual adelante que atrás, como “Ana” y “radar”. Recursivamente, una palabra es un palíndromo si las letras inicial y final son las mismas y el medio es un palíndromo.*

Las siguientes son subrutinas que toman un argumento de cadena de texto y devuelve las letras primera, última y del centro:

```
sub primera_letra( Str $palabra ) {
    return substr $palabra, 0, 1;
}

sub última_letra( Str $palabra ) {
    return substr $palabra, *-1, 1;
}

sub centro_letra( Str $palabra ) {
```

```
    return substr $palabra, 1, *-1;  
}
```

Por el momento, no es necesario que sepas cómo funcionan; veremos esto en el Capítulo 7 sobre cadenas de texto. Por ahora:

1. Escribe estas subrutinas en un archivo llamado `palindromo.raku` y pruébalas. ¿Qué pasas si llamas a `letra_centro` con una cadena de texto con dos letras? ¿Una letra? ¿Qué pasa con una cadena de texto, la cual se escribe `' '` y no contiene letras? Dado que el método `.chars` devuelve la longitud de una cadena de texto, cómo podrías añadir una restricción de signatura para rechazar una entrada inválida?
2. Escribe una subrutina llamada `es-palindromo` que toma una cadena de texto como argumento y devuelve `True` si es un palíndromo y `False` de lo contrario. Recuerdas que puedes usar el método integrado `.chars` para chequear la longitud de una cadena de texto.

Solución: A.3.5

Ejercicio 5.4. Un número entero a , es una potencia de b si es divisible por b y a/b es una potencia de b . Escribe una función llamada `es-potencia-de` que toma los parámetros a y b y devuelve `True` si a es una potencia de b . Nota: tendrás que pensar sobre el caso base.

Solución: A.3.6

Ejercicio 5.5. El máximo común divisor (MCD) de a y b es el mayor número entero que los divide sin dejar un residuo.

Una manera de encontrar el MCD de dos números está basada en la observación de que si r es el residuo cuando a es dividido por b , entonces $\text{mcd}(a, b) = \text{mcd}(b, r)$. Como un caso base, podemos usar $\text{mcd}(a, 0) = a$.

Escribe una función llamada `mcd` que toma los parámetros a y b y devuelve su máximo común divisor.

Crédito: este ejercicio está basado en un ejemplo de *Structure and Interpretation of Computer Programs* de Abelson y Sussman.

Solución: A.3.7

Iteración

Este capítulo es sobre la iteración, la cual es la habilidad de ejecutar un bloque de sentencias repetidamente. Vimos un tipo de iteración, usando recursión, en la Sección 4.11. También vimos otro tipo, usando un bucle `for`, en la Sección 4.10. En este capítulo veremos otro tipo, usando la sentencia `while`. Pero primero queremos discutir un poco más sobre la asignación de una variable.

6.1 Asignación Versus Igualdad

Antes de proceder, quiero hablar sobre una fuente común de confusión. Dado que Raku usa el signo de igualdad (`=`) para la asignación, es tentativo interpretar una sentencia tal como `$a = $b` como una proposición matemática de igualdad, eso es, la afirmación que `$a` y `$b` son iguales. Pero esta interpretación es incorrecta.

Primero, la igualdad es una relación simétrica y la asignación no lo es. Por ejemplo, en matemáticas, si $a = 7$, entonces $7 = a$. Pero en Raku, la sentencia `$a = 7` es legal y `7 = $a` no lo es.

De igual manera, en matemáticas, una proposición de igualdad es verdadera o falsa todo el tiempo. Si $a = b$ ahora, entonces a siempre será igual a b . En Raku, una sentencia de asignación puede hacer dos variables iguales, pero no tienen que permanecer de ese modo:

```
> my $a = 5;
5
> my $b = $a;    # $a y $b son iguales ahora
5
> $a = 3;        # $a y $b ya no son iguales
3
> say $b;
5
```

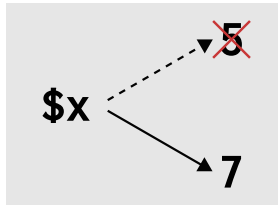


Fig. 6.1: Diagrama de estado.

La tercera línea cambia el valor de \$a pero no cambia el valor de \$b, así que ya no son iguales.

En resumen, recuerda que `=` es un operador de asignación y no un operador de igualdad; los operadores para probar igualdad entre dos términos son `==` para números y `eq` para cadenas de texto.

6.2 Reasignación

Como tal vez descubriste, es legal hacer más de una asignación a la misma variable. Una asignación nueva hace que una variable existente haga referencia a un nuevo valor (y deje de referirse al valor antiguo):

```
> my $x = 5;
5
> say $x;
5
> $x = 7;
7
> say $x
7
```

La primera vez mostramos a \$x, su valor es 5; la segunda vez, su valor es 7.

La Figura 6.1 muestra como la **reasignación** luce en un diagrama de estado.

La reasignación de variables es usualmente útil, pero deberías utilizar esta característica con cierta precaución. Si los valores de las variables se cambian frecuentemente, puede hacer el código difícil de leer y depurar.

6.3 Actualización de las Variables

Un tipo común de reasignación es una **actualización**, donde el nuevo valor de la variable depende en el antiguo:

```
> $x = $x + 1;
```

Esto significa “obtén el valor actual de \$x, agrégale uno, y después actualiza a \$x con el valor nuevo.”

Si intentas actualizar una variable a la cual no se le ha dado un valor, obtienes una advertencia, porque Raku evalúa el lado derecho de la sentencia de asignación antes de asignar un valor a `$x`:

```
> my $x;
> $x = $x + 1;
Use of uninitialized value of type Any in numeric context
in block <unit> at <unknown file> line 1
```

Antes de actualizar una variable, tienes que declararla e **inicializarla**, usualmente con una sentencia de asignación:

```
> my $x = 0;
> $x = $x + 1;
```

Actualizar una variable al agregar 1 se conoce como un **incremento**; substrair 1 se conoce como un **decremento**.

Como se mencionó anteriormente en la Sección 2.3, Raku tiene algunos atajos para el incremento y el decremento:

```
$x += 1; # equivalente a $x = $x + 1
$x++;   # también equivalente

$x -= 1; # equivalente a $x = $x - 1
$x--;   # también equivalente
```

6.4 La Sentencia while

Las computadoras usualmente automatizan tareas repetitivas. Repetir tareas idénticas o similares sin hacer errores es algo que las computadoras saben hacer bien y por el contrario, algo en lo cual las personas comúnmente cometen errores. En un programa de computadora, la repetición se conoce como **iteración**.

Ya hemos visto dos funciones, `cuenta-regresiva` y `imprime-n-veces`, que iteran usando recursión (ver Sección 4.11). Dado que la iteración es tan común, muchos lenguajes de programación incluyendo a Raku proveen características para hacerla más fácil. Una de ellas es la sentencia `for` que vimos en la Sección 4.10. Regresaremos a eso más tarde.

Otra es la sentencia `while`. Esta es una versión de `cuenta-regresiva` que usa la sentencia `while`:

```
sub cuenta-regresiva( Int $n is copy ) {
    while $n > 0 {
        say $n;
        $n--;
    }
    say '!Despegue!';
}
```

Puedes casi leer la sentencia `while` como si fuera inglés. Significa, “Mientras (`while`) n sea mayor que 0, muestra el valor de n y después reduce n . Cuando llegues a 0, muestra la palabra ¡Despegue!”

Aquí se presenta el flujo de ejecución de una sentencia `while` más formalmente:

1. Determina si la condición es verdadera o falsa.
2. Si es falsa, abandona la sentencia `while` y continúa la ejecución en la siguiente sentencia.
3. Si la condición es verdadera, ejecuta el cuerpo y después regresa al paso 1.

Este tipo de flujo se llama un bucle (**en.** `loop`) porque el tercer paso regresa nuevamente al principio.

El cuerpo del bucle debería cambiar el valor de una o más variables para que la condición se vuelva falsa eventualmente y el bucle termine. De lo contrario, el bucle se repetirá por siempre, lo cual se conoce como un **bucle infinito**. Una fuente de entretenimiento para los científicos de la computación es la observación de que las instrucciones en la botella de champú, “Enjabona, enjuaga, repite”, son un bucle infinito.

En el caso de cuenta-regresiva, podemos probar que el bucle termina: si n es cero o negativo, el bucle nunca se ejecuta. Por lo contrario, n se vuelve más pequeño cada vez a través del bucle, así que eventualmente tenemos que llegar a 0.

Con otros bucles, no es tan fácil decir si el bucle termina. Por ejemplo:

```
sub secuencia( $n is copy ) {
  while $n != 1 {
    say $n;
    if $n %% 2 {          # $n es par
      $n = $n / 2;
    } else {             # $n es impar
      $n = $n*3 + 1;
    }
  }
  return $n;
}
```

La condición de este bucle es $n \neq 1$, así que el bucle continuará hasta que n sea 1, lo cual hace la condición falsa.

Cada vez a través del bucle, el programa muestra el valor de n y después chequea si es par o impar. Si es par, n es dividido por 2. Si es impar, el valor de n es reemplazado con $n*3 + 1$. Por ejemplo, si el argumento que se pasa a `secuencia` es 42, los valores resultantes de n son 42, 21, 64, 32, 16, 8, 4, 2, 1.

Dado que n algunas veces incrementa y otras disminuye, no hay prueba obvia que n llegará a 1, o que el programa termina. Para algunos valores particulares de n , podemos probar que el programa termina. Por ejemplo, si el valor inicial es una potencia de dos, n será un par a través del bucle hasta que llega al 1. El ejemplo anterior termina con tal secuencia de potencias de dos, comenzando con 64.

¡La pregunta difícil es si podemos probar que este programa termina para *todos* los valores positivos de *n*. Hasta ahora, nadie ha sido capaz de probarlo o refutarlo! (Ver https://es.wikipedia.org/wiki/Conjetura_de_Collatz.)

Como un ejercicio, podrías escribir de nuevo la función `imprimir-n-veces` de la Sección 4.11 usando iteración en lugar de recursión.

La sentencia `while` puede también usarse como un modificador de sentencia (o sintaxis sufija):

```
my $val = 5;
print "$val " while $val-- > 0;    # imprime 4 3 2 1 0
print "\n";
```

La sentencia `while` del bucle ejecuta el bloque mientras la condición sea verdadera. También existe un bucle `until`, el cual ejecuta el bloque mientras su condición se falsa:

```
my $val = 1;
until $val > 5 {
    print $val++;                # imprime 12345
}
print "\n";
```

6.5 Variables Locales y Ámbito de una Variable

Hemos visto en la Sección 3.9 que las variables creadas dentro de una subrutina (con la palabra clave `my` son *locales* a esa subrutina. La palabra clave `my` es usualmente llamada un *declarador* porque se usa para declarar una variable nueva (u otro identificador). Es uno de los declaradores más comunes. Entre otros declaradores se encuentran `our` o `state`, los cuales se describen brevemente más tarde en este capítulo.

Similarmente, los parámetros de una subrutina son usualmente *locales* a la subrutina en la signatura donde son declarados.

Mencionamos brevemente que el término *ámbito léxico* es probablemente más preciso que *local*, pero era muy temprano en aquel entonces para realmente explicar lo que esto significa.

La declaración de una variable con `my` le da un *ámbito léxico*. Esto significa que solo existe dentro del bloque actual. En términos generales, un *bloque* es una pieza de código de Raku dentro de llaves. Por ejemplo, el cuerpo de una subrutina y el código de un bucle `while` o `for` o de una sentencia condicional `if` son bloques de códigos. Cualquier variable creada con el declarador `my` existe y está disponible para usarse solamente dentro del lugar donde fue declarada hasta el final del bloque de código.

Por ejemplo, este código:

```
if $condición eq True {
    my $foo = "bar";
    say $foo;    # imprime "bar"
}
say $foo;      # ERROR: "Variable '$foo' is not declared ..."
               # ERROR: "La variable '$foo' no está declarada ..."
```


fallará en la segunda sentencia de impresión, porque la llamada de la función `say` no está en el ámbito léxico de la variable `$foo`, el cual termina con la llave derecha que cierra el bloque de la condición. Si queremos que esta variable pueda accederse después que la condición termine, entonces tendríamos que declararla antes de la sentencia `if`.

Por ejemplo:

```
my $foo;
if $condición eq True {
    $foo = "bar";
    say $foo; # imprime "bar"
} else {
    $foo = "baz";
}
say $foo;      # imprime "bar" o "baz" dependiendo de $condición
```

Si una variable léxica no se declara dentro de un bloque, su ámbito se extiende hasta al final del archivo (esto se llama algunas veces una variable estática o global, aunque estos términos no son del todo precisos). Por ejemplo, en el último fragmento de código, el ámbito de la variable `$foo` se extenderá hasta al final del archivo, lo cual puede o no puede ser algo bueno, dependiendo de como pretendas usarla. Es usualmente mejor reducir el ámbito de las variables tanto como sea posible porque esto ayuda a reducir las dependencias entre las varias partes del código y limita el riesgo de errores sutiles. En el código anterior, si queremos limitar el ámbito de `$foo`, podemos agregar llaves para crear un bloque delimitador con el único propósito de limitar el ámbito de dicha variable:

```
{
    my $foo;
    if $condición eq True {
        $foo = "bar";
        say $foo; # imprime "bar"
    } else {
        $foo = "baz";
    }
    say $foo;      # imprime "bar" o "baz" dependiendo de $condición
}
```

Ahora las llaves exteriores crean un bloque delimitador que delimita el ámbito de `$foo` solamente donde lo necesitamos. Esto puede parecer un ejemplo un poco forzado, pero es común agregar llaves solo con el propósito de definir el ámbito de algo.

El ámbito léxico también significa que las variables con el mismo nombre pueden ser temporalmente redefinidas en un nuevo ámbito:

```
my $posición = "fuera";
sub exterior {
    say $posición;
}
sub interior {
    my $posición = "dentro";
    say $posición;
}
```

```
say $posición;      # -> fuera
exterior();         # -> fuera
interior();         # -> dentro
say $posición;      # -> fuera
```

Tenemos en efecto dos variables con el mismo nombre, \$posición, pero en diferentes ámbitos. Una es válida solo dentro de la subrutina interior donde ha sido redefinida, y la otra en otro lugar.

Si agregamos una subrutina nueva:

```
sub en-ningún-lugar {
  my $posición = "en ningún lugar";
  exterior();
}
en-ningún-lugar();      # -> fuera
```

está aún imprimirá “fuera” porque la subrutina exterior sabe sobre la versión “fuera” de la variable \$posición, la cual existía cuando exterior fue definida. En otras palabras, el código exterior que aludió a la variable exterior (“fuera”) sabe sobre la variable que existía cuando fue creado, pero no sobre la variable existente donde dicho código fue llamado. Así es como las variables *lexicales* funcionan. Este comportamiento es la base para construir *clausuras*, una forma de subrutina con algunas propiedades especiales que estudiaremos más tarde en este libro, pero que en efecto está presente en todas partes en Raku.

Mientras que tener variables diferentes con el mismo nombre puede darte mucho poder expresivo, te aconsejamos que evite crear variables distintas con el mismo nombre y diferentes ámbitos, por lo menos hasta que realmente entiendas estos conceptos lo suficiente para saber lo que estás haciendo, debido a que esto puede ser un poco complejo.

Mayormente, muchas de las variables usadas en Raku son variables lexicales, que se declaran con el declarador `my`. Aunque no son declarados con `my`, los parámetros declarados en la signature de una subrutina y los parámetros de un bloque puntiagudo también tienen un ámbito léxico limitado al cuerpo de la subrutina o el bloque de código.

Existen otros declaradores, tales como `our`, el cual crea una variable con ámbito de paquete, y `state`, que crea una variable con ámbito léxico pero con un valor persistente. Raramente son usados.

Un punto final: aunque usualmente no se declaran con un declarador `my`, las subrutinas tienen también un ámbito léxico por defecto. Si se definen dentro de un bloque, serán visibles solo dentro de ese bloque. Un ejemplo de esto se mostró al final de la solución al ejercicio MCD del capítulo anterior (ver Solución: A.3.7). Con eso dicho, *puedes* declarar una subrutina con un declarador `my` si así lo deseas:

```
my sub frobnicate {
  # ...
}
```

Esta técnica podría agregar consistencia o alguna forma de característica de auto-documentación, pero no obtendrá mucha funcionalidad con eso.

6.6 Sentencias de Flujo de Control (last, next, etc.)

Algunas veces no sabes cuando terminar un bucle hasta que llegas a la mitad del cuerpo. En ese caso, puedes usar una sentencia de flujo de control como `last` para salir del bucle.

Por ejemplo, supón que quieres tomar entrada del usuario hasta que entre `terminar`. Podrías escribir lo siguiente:

```
while True {  
  my $línea = prompt "Entra algo ('terminar' para salir)\n";  
  last if $línea eq "terminar";  
  say $línea;  
}  
say '¡Terminado!';
```

La condición de bucle es `True`, lo cual siempre es verdadero, así que el bucle se ejecuta hasta que alcanza la sentencia `last`.

Cada vez, el bucle incita al usuario a teclear algo. Si el usuario escribe `terminar`, la sentencia `last` termina el bucle. Por lo contrario, el programa refleja cualquier cosa que el usuario escribe y regresa a la parte superior del bucle. Esta es una muestra:

```
$ raku terminar.raku  
Entra algo ('terminar' para salir)  
Hola  
Hola  
Entra algo ('terminar' para salir)  
Estoy aprendiendo Raku  
Estoy aprendiendo Raku  
Entra algo ('terminar' para salir)  
terminar  
¡Terminado!
```

Esta manera de escribir bucles `while` es común porque puedes chequear la condición en cualquier parte en el bucle (no solo en la parte superior) y puedes expresar la condición para terminar afirmativamente (“termina cuando esto suceda”) en lugar de negativamente (“mantente haciendo esto hasta que esto pase”).

Usar un bucle `while` con una condición que es siempre verdadera es una manera natural de escribir un bucle infinito, es decir, un bucle que se ejecutará por siempre hasta que algo en el código (tal como la sentencia `last` usada anteriormente) force al programa a salir del bucle. Esto es comúnmente usado en lenguajes de programación, y funciona muy bien en Raku. No obstante, existe otra forma común y más idiomática de construir un bucle infinito en Raku: al usar la sentencia `loop`, la cual estudiaremos en la Sección 9.7 (p. 160). Por ahora, usaremos la sentencia `while True`, la cual es bastante legítima.

Algunas veces, en lugar de salir del bucle `while` con la sentencia de control `last`, por ejemplo, necesitas comenzar el cuerpo del bucle al principio. Por ejemplo, quieres chequear si la entrada del usuario es correcta con alguna subrutina `es-válido` (no especificada) antes de procesar los datos, e incitar al usuario a intentar nuevamente si la entrada no era correcta. En este caso, la sentencia de control `next` te deja comenzar al principio del bucle otra vez:

```

while True {
  my $línea = prompt "Entra algo ('terminar' para salir)\n";
  last if $línea eq "terminar";
  next unless es-válido($línea);
  # procesamiento posterior de $línea;
}
print(';Terminado!');

```

Aquí, el bucle termina si el usuario escribe “terminar”. Si no, la entrada del usuario se chequea con la subrutina `es-válido`; si la subrutina devuelve un valor verdadero, el procesamiento continúa; si devuelve un valor falso, entonces el flujo de control comienza nuevamente al principio del cuerpo del bucle, y por lo tanto se incita al usuario a escribir una entrada válida.

Las sentencias de control `last` y `next` también funcionan con los bucles `for`. Por ejemplo, el siguiente bucle `for` itera en teoría sobre un rango de números enteros entre 1 y 20, pero desecha los números impares por virtud de una sentencia `next` y sale del bucle con una sentencia `last` tan pronto la variable del bucle es mayor que `$max` (por ejemplo, 10):

```

my $max = 10;
for 1..20 -> $i {
  next unless $i %% 2; # mantiene solos los valores pares
  last if $i > $max;   # termina el bucle si $i es mayor que $max
  say $i;              # imprime 2 4 6 8 10
}

```

Puedes tener tantas sentencias `last` y `next` como desees, al igual que puedes tener tantas sentencias `return` como quieras en una subrutina. El uso de tales sentencias de flujo de control no es considerado mala práctica. Durante la infancia de la programación estructurada, algunas personas insistían que los bucles y las subrutinas tienen solo una entrada y una sola salida. La noción de una sola entrada es todavía una buena idea, pero la noción de una sola salida ha causado que muchas personas hagan lo imposible y escriban muchísimo código no tan natural. La mayor parte de la programación consiste en recorrer árboles de decisiones. Un árbol de decisiones naturalmente comienza con un solo tronco pero termina con muchas hojas. Escribe tu código con el número de controles de bucle (y salida de subrutinas) que es natural para el problema que estás intentando resolver. Si declaras tus variables con ámbitos razonables, todo se limpia automáticamente al momento apropiado, sin importar como dejes el bloque.

6.7 Raíces Cuadradas

Los bucles se usan a menudo en programas que calculan resultados numéricos comenzando con una respuesta aproximada y mejorándola iterativamente.

Por ejemplo, una manera de calcular las raíces cuadradas es con el método de Newton (también conocido como el método de Newton-Raphson). Supón que quieres saber la raíz cuadrada de a . Si comienzas con casi cualquier estimado, x , puedes calcular un mejor estimado con la siguiente fórmula:

$$y = \frac{x + a/x}{2}$$

Por ejemplo, si a es 4 y x es 3:

```
> my $a = 4;
4
> my $x = 3;
3
> my $y = ($x + $a/$x)/2;
2.166667
```

El resultado está mucho más cerca de 3 que a la respuesta correcta ($\sqrt{4} = 2$). Si repites el proceso con el estimado nuevo, se acerca aún más:

```
> $x = $y;
2.166667
> $y = ($x + $a/$x)/2;
2.006410
```

Después de varias actualizaciones, el estimado es casi exacto:

```
> $x = $y;
2.006410
> $y = ($x + $a/$x)/2;
2.000010
> $x = $y;
2.000010
> $y = ($x + $a/$x)/2;
2.000000000026
```

En general no sabemos con antelación cuántos pasos toma para obtener la respuesta correcta, pero sabemos cuando la obtenemos porque el estimado para de cambiar:

```
> $x = $y;
2.000000000026
> $y = ($x + $a/$x)/2;
2
> $x = $y;
2
> $y = ($x + $a/$x)/2;
2
```

Cuando $\$y == \x , podemos parar. Este es un bucle que comienza con un estimado inicial, x , y lo mejora hasta que para de cambiar:

```
my ($a, $x) = (4, 3);
while True {
    say "-- Valor Intermedio: $x";
    my $y = ($x + $a/$x) / 2;
    last if $y == $x;
    $x = $y;
}
say "Resultado final es $x";
```

Esto imprimirá:

```
-- Valor Intermedio: 3
-- Valor Intermedio: 2.166667
-- Valor Intermedio: 2.006410
-- Valor Intermedio: 2.000010
-- Valor Intermedio: 2.000000000026
-- Valor Intermedio: 2
Resultado final es 2
```

Para muchos valores de $\$a$ esto funciona bien, pero tenemos que considerar varias cosas. Primero, en la mayoría de lenguajes de programación, es peligroso chequear la igualdad de coma flotante (float), porque los valores de coma flotante son solo aproximadamente correctos. No tenemos este problema con Raku, porque, como hemos mencionado, Raku usa una mejor representación de los números racionales que la mayoría de los lenguajes de programación generalistas. (Deberías tener esto presente si estás usando algún otro lenguaje). Aún si no tenemos este problema en Raku, también pueden existir otros problemas con algoritmos que no se comportan tan bien como el algoritmo de Newton. Por ejemplo, algunos algoritmos podrían no convergir tan rápida y perfectamente como el algoritmo de Newton pero en lugar producir valores alternativos por encima y por debajo del resultado preciso.

En lugar de chequear si $\$x$ y $\$y$ son exactamente iguales, es más seguro usar la función integrada `abs` para calcular el valor absoluto, o magnitud, de la diferencia entre ellos:

```
last if abs($y - $x) < $epsilon;
```

donde `epsilon` tiene un valor pequeñísimo como 0.0000001 que determina qué tan cerca es suficientemente cerca.

6.8 Algoritmos

El método Newton es un ejemplo de un **algoritmo**, que es un proceso mecánico para resolver una categoría de problemas (en este caso, calcular raíces cuadradas).

Para entender lo que es un algoritmo, podría ser útil comenzar con algo que no es aún un algoritmo. Cuando aprendiste a multiplicar los números de un solo dígito, probablemente memorizaste la tabla de multiplicación. En efecto, memorizaste 100 soluciones específicas. Ese tipo de conocimiento no es algorítmico.

Pero si eres “perezoso”, posiblemente aprendiste algunos trucos. Por ejemplo, para encontrar el producto de n y 9, puedes escribir $n - 1$ como el primer dígito y $10 - n$ como el segundo dígito. Por lo tanto para calcular $9 * 7$, $n - 1$ es 6 y $10 - n$ es 3, así que el producto $9 * 7$ es 63. Es truco es una solución general para multiplicar un número de un solo dígito por 9. ¡Eso es un algoritmo!

Similarmente, las técnicas que aprendiste en la escuela para añadir (cargando), substraer (tomando prestado), y la división larga son todos algoritmos. Una de las características de los algoritmos es que no requieren ninguna inteligencia para llevarse a cabo. Ellos son procesos mecánicos donde cada paso que sigue precede al último de acuerdo a un conjunto simple de reglas.

Ejecutar los algoritmos es aburrido, pero diseñarlos es interesante, intelectualmente estimulante, y una parte central de la ciencia de la computación.

Algunas de las cosas que la gente hace naturalmente, sin ninguna dificultad o pensamiento consciente, son las más difíciles de expresar algorítmicamente. Entender un lenguaje natural es un buen ejemplo. Todos lo hacemos, pero hasta ahora nadie ha podido explicar *cómo* lo hacemos, por lo menos no en la forma de un algoritmo.

6.9 Depuración de Programas

A medida que empieces a escribir programas más grandes, podría ser que dediques más tiempo a la depuración. Más código significa que existen más posibilidades de cometer un error y más lugares en los cuales se ocultan los errores.

Una manera de reducir tu tiempo de depuración es “depurar por bisección”. Por ejemplo, si hay 100 líneas en tu programa y las chequeas una a la vez, te tomaría 100 pasos.

En lugar de eso, intenta dividir el problema a la mitad. Inspecciona la mitad del programa, o sus alrededores, con un valor intermedio que puedas chequear. Agrega una sentencia `say` (o algo más que tenga un efecto verificable) y ejecuta el programa.

Si el chequeo en la mitad del programa es incorrecto, entonces podría haber un problema en la primera mitad del programa. Si es correcto, entonces el problema está en la segunda mitad.

Cada vez que realizas un chequeo como este, divides el número de líneas por la mitad que tienes que inspeccionar. Después de seis pasos (lo cual es menos que 100), lo habrás reducido a uno o dos líneas de código, por lo menos en teoría.

En práctica no es siempre claro cual es la “mitad del programa” y no siempre es posible chequearla. No hace sentido contar el número de líneas y encontrar el punto exacto. En lugar, piensa sobre los lugares en el programa donde podría haber errores y los lugares donde es fácil chequear. Después, elige el sitio donde piensas que la posibilidad de encontrar el error antes o después del chequeo es la misma.

6.10 Glosario

Reasignación Asignar un valor nuevo a una variable que ya existe.

Actualización Una asignación donde el valor nuevo de la variable depende del viejo.

Inicialización Una asignación que provee una variable con valor inicial que puede ser actualizada más tarde.

Incremento Una actualización que aumenta el valor de una variable (usualmente por uno).

Decremento Una actualización que reduce el valor de una variable.

Iteración Ejecución repetida de un conjunto de sentencias usando una llamada de una función recursiva o un bucle.

Bucle infinito Un bucle en el cual la condición para terminar nunca es satisfecha.

Algoritmo Un proceso general para resolver una categoría de problemas.

6.11 Ejercicios

Ejercicio 6.1. Copia el bucle de la Sección 6.7 y encapsúlalo en una subrutina llamada `mi-r-cuad` que toma a `a` como un parámetro, elige un valor razonable de `x`, y devuelve un estimado de la raíz cuadrada de `a`.

Para probarla, escribe una función llamada `probar-raiz-cuadrada` que imprime una tabla como esta:

a	mi-r-cuad(a)	sqrt(a)	diff
1	1.00000000000000	1.00000000000000	1.110223e-15
2	1.4142135623747	1.4142135623731	1.594724e-12
3	1.7320508075689	1.7320508075689	0.000000e+00
4	2.00000000000000	2.00000000000000	0.000000e+00
5	2.2360679774998	2.2360679774998	0.000000e+00
6	2.4494897427832	2.4494897427832	8.881784e-16
7	2.6457513110647	2.6457513110646	1.025846e-13
8	2.8284271247494	2.8284271247462	3.189449e-12
9	3.00000000000000	3.00000000000000	0.000000e+00

La primera columna es un número, a , la segunda columna es la raíz cuadrada a que es calculada con `mi-r-cuad`, la tercera columna es la raíz cuadrada calculada con la función integrada `sqrt` de Raku, y la cuarta columna es el valor absoluto de la diferencia entre los dos estimados. No te preocupes mucho acerca de obtener un formato tabular organizado, todavía no hemos visto las funciones integradas para hacer eso.

Solución: A.4.1.

Ejercicio 6.2. El matemático Srinivasa Ramanujan encontró una serie infinita que puede ser usada para generar una aproximación numérica de $1/\pi$:

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}}$$

Escribe una función llamada `estimar-pi` que use esta fórmula para computar y devolver un estimado de π . Debería usar un bucle `while` para computar los términos de la sumatoria hasta que el último término sea menor que $1e-15$ (la cual es la notación de Raku para 10^{-15}). Puedes chequear el resultado al compararlo con la constante integrada `pi`.

Solución: A.4.2.

Cadenas de Texto

Las cadenas de texto no son como los enteros, los racionales, y los booleanos. Una cadena de texto es una **secuencia** de caracteres, lo cual significa que es una colección ordenada de otros valores, y algunas veces necesitas acceder algunos de estos valores individuales. En este capítulo verás cómo analizar, manejar, y modificar cadenas de texto, y aprenderás sobre algunos de los métodos que las cadenas de texto proveen. También aprenderás sobre una herramienta muy poderosa para la manipulación de texto, expresiones regulares (también conocidas como “regexes”).

7.1 Una Cadena de Texto es una Secuencia

Una cadena de texto es primariamente una pieza de datos textuales, pero es técnicamente una secuencia ordenada de caracteres.

Muchos lenguajes de programación te permiten acceder caracteres individuales de una cadena de texto con índice entre corchetes. Esto no es directamente posible en Raku, pero todavía puedes acceder los caracteres uno a la vez usando el método integrado `comb` y el operador corchete:

```
> my $cadena = "banana";  
banana  
> my $cad = $cadena.comb;  
(b a n a n a)  
> say $cad[1];  
a  
> say $cad[2];  
n
```

El método `comb` en la segunda sentencia divide la cadena de texto en una lista de caracteres que puedes acceder individualmente con los corchetes.

La expresión dentro de los corchetes se llama un **índice** (también llamado subíndice). El índice indica cuales caracteres en la secuencia quieres acceder (de ahí el nombre). Pero puede ser que esto no es lo que esperabas: el artículo con índice 1 es la segunda letra de la palabra. Para los científicos de la computación, el índice es usualmente un desplazo del inicio. Por ejemplo, el índice de la primera letra ("b") es 0, y el índice de la primera "a" es 1, no 2, etcétera.

También podrías extraer una "rebanada" de varios caracteres a la vez al usar el operador de rango dentro de los corchetes:

```
> say $cad[2..5]
(n a n a)
```

De nuevo, la subcadena "nana" comienza en la tercera letra de 'banana', pero esta letra está indexada 2, y la sexta letra tiene índice 5.

Aunque esto puede ser útil en ocasiones, esta no es la manera en la que manejarías cadenas de texto en Raku, el cual tiene herramientas superiores que son más poderosas y más expresivas, así que raramente necesitas usar índices o subíndices para acceder caracteres individuales.

También, si existe una necesidad real de acceder y manipular las letras individuales, haría más sentido almacenarlas en un array, pero no hemos hablado sobre arrays todavía, así que regresaremos a esto más tarde.

7.2 Operadores Comunes de las Cadenas de Texto

Raku provee un número de operadores y funciones para el manejo de cadenas de texto. Revisemos algunas de los más populares.

7.2.1 Longitud de una Cadena de Texto

La primera cosa que desearíamos saber sobre una cadena de texto es su longitud. El método (o función) `chars` devuelve el número de caracteres de una cadena de texto. Como podemos ver, `chars` puede usarse con una sintaxis de método o función:

```
> say "banana".chars; # sintaxis de invocación de método
6
> say chars "banana"; # sintaxis de llamada de función
6
```

Nota que, con el advenimiento de Unicode, la noción sobre la longitud de una cadena de texto se ha vuelto más complicada de lo que era con cadenas de texto ASCII. Hoy en día, un carácter puede estar compuesto de uno, dos, o más bytes. La rutina `chars` devuelve el número de caracteres (en el sentido de los grafemas de Unicode, los cuales son más o menos lo que los humanos perciben como caracteres) dentro de la cadena, aún cuando algunos caracteres requieren una codificación de más de 2, 3, o 4 bytes.

Una cadena de texto con una longitud cero (o sea, sin caracteres) se conoce como una *cadena de texto vacía*.

7.2.2 Búsqueda de una Subcadena Dentro de una Cadena de Texto

La función integrada `index` usualmente toma dos argumentos, una cadena de texto y una subcadena (algunas veces conocidas como la “paja” y la “aguja”), busca la subcadena en la cadena de texto, y devuelve la posición donde se encuentra la subcadena. Si la subcadena no se encuentra en la cadena, la función devuelve un valor indefinido:

```
> say index "naranja", "ra";  
2  
> say index "naranja", "je";  
Nil
```

Nuevamente, el índice es un desplazamiento del inicio de la cadena de texto, así que el índice de la primera letra (“n”) es cero, y el índice de la segunda “a” es 3, no 4.

También puedes invocar a `index` con una sintaxis de método:

```
> say "naranja".index("ra");  
2
```

La función `index` puede tomar un tercer argumento opcional, un entero que indica donde comenzar la *búsqueda* (y por lo tanto ignorando en la búsqueda cualquier carácter posterior a la posición de inicio):

```
> say index "naranja", "a", 2;  
3
```

Aquí la función `index` comenzó la búsqueda en la “r” y encontró la posición de la segunda ocurrencia de la subcadena “a”.

También existe una función `rindex`, la cual realiza la búsqueda de la subcadena de atrás hacia adelante y devuelve la última posición de la subcadena dentro de la cadena de texto:

```
> say rindex "naranja", "a";  
6
```

Nota que aunque `rindex` inspecciona la cadena de texto de atrás hacia adelante, la función devuelve una posición computada desde el inicio de la cadena de texto.

7.2.3 Extrayendo una Subcadena de una Cadena de Texto

La función opuesta a `index` es la función (o método) `substr`, la cual, dada una posición inicial y una longitud, extrae una subcadena de una cadena de texto:

```
> say substr "Hacer es la mejor manera de decir.", 0, 5;  
Hacer  
> say "Hacer es la mejor manera de decir.".substr(12, 5)  
mejor
```

Nota que, al igual que la función `chars`, la longitud es expresada en caracteres (o grafemas Unicode), no en bytes. De igual modo, como puedes observar, los espacios que separan las palabras dentro de la cadena de texto obviamente cuentan como caracteres. El argumento de la longitud es opcional; si no se provee, la función `substr` devuelve la subcadena comenzando en la posición de inicio al final de la cadena de texto:

```
> say "Hacer es la mejor manera de decir.".substr(9)
la mejor manera de decir.
```

Similamente, si el valor de la longitud es muy largo para que la subcadena comience en la posición inicial, la función `substr` devolverá la subcadena comenzando en la posición inicial hasta el final de la cadena:

```
> say substr "banana", 2, 10;
nana
```

Por supuesto, los parámetros de la posición inicial y la longitud no necesitan números literales como en los ejemplos anteriores; puedes también usar variables (o hasta una expresión o una función que devuelva un valor numérico), provisto que la variable o la función pueda ser coaccionada en un entero. Pero la posición de inicio debe estar dentro del rango de la cadena de texto, y si falla obtienes un error `Start argument to substr out of range ...`; por lo tanto podrías tener que verificarlo en contra de la longitud de la cadena de texto con antelación.

También puedes comenzar a contar de atrás hacia adelante con la siguiente sintaxis:

```
> say "I have a dream".substr(*-5)
dream
> say substr "I have a dream", *-5;
dream
```

Aquí, el asterisco `*` puede considerarse como una representación de la longitud total de la cadena de texto; `*-5` es por lo tanto la posición en la cadena de texto cinco caracteres antes del final de la cadena. Así que `substr(*-5)` devuelve los caracteres desde esa posición hasta el final de la cadena de texto, es decir, los últimos cinco caracteres de la cadena de texto.

7.2.4 Otras Funciones o Métodos Útiles de las Cadenas de Texto

Esto puede no ser obvio todavía, pero prontamente veremos que la combinación de las funciones de cadenas de texto que discutimos más arriba nos dan ya mucho poder para manipular cadenas de texto más allá de lo que piensas posible hasta el momento.

Déjanos mencionar brevemente varias funciones adicionales que pueden ser útiles de vez en cuando.

7.2.4.1 flip

La función (o método) `flip` invierte una cadena de texto:

```
> say flip "mango";
ognam
```

7.2.4.2 `split`

La función (o método) `split` divide una cadena de texto en subcadenas de texto, basado en los delimitadores en la cadena de texto:

```
> say $_ for split "-", "25-12-2016";
> #           ^ delimitador
25
12
2016
> for "25-12-2016".split("-") -> $val {say $val};
25
12
2016
```

El delimitador puede ser un solo carácter con comillas como en el ejemplo más arriba o una cadena de varios caracteres, tales como una coma y un espacio en el ejemplo más abajo:

```
> .say for split ", ", "Jan, Feb, Mar";
> #           ^^ delimitador
Jan
Feb
Mar
```

Recuerda que `.say` es un atajo para `$_ .say`.

Por defecto, los delimitadores no aparecen en la salida producida por la función (o método) `split`, pero este comportamiento puede cambiarse con el uso del adverbio apropiado. Un *adverbio* es básicamente un argumento nombrado de una función que modifica la manera en la cual la función se comporta. Por ejemplo, el adverbio `:v` (valores) le dice a `split` que también devuelva el valor del delimitador:

```
> .raku.say for split ', ', "Jan, Feb, Mar", :v;
"Jan"
", "
"Feb"
", "
"Mar"
```

Los otros adverbios que pueden usarse en este contexto son `:k` (claves), `:kv` (claves y valores), y `:p` (pares). Su significados detallados pueden encontrarse en la documentación para `split` (<https://docs.raku.org/routine/split>). El adverbio `skip-empty` remueve las partes vacías de la lista de resultados.

La función `split` pueden también usarse con un *patrón* de expresión regular como un delimitador, y esto puede hacerla mucho más poderosa. Estudiaremos expresiones regulares más adelante en este capítulo.

7.2.4.3 Concatenación de Cadenas de Texto

El operador `~` concatena dos cadenas de texto y forma una:

```
> say "ban" ~ "ana";
banana
```

Puedes tener cadenas de varias ocurrencias de este operador para concatenar más de dos cadenas:

```
> say "ba" ~ "na" ~ "na";
banana
```

Si se usa como un operador prefijo unario, `~` transforma su argumento en una cadena de texto:

```
> say (~42).WHAT;
(Str)
```

7.2.4.4 Separando en Palabras

La función `words` devuelve una lista de palabras que componen la cadena de texto:

```
> say "I have a dream".words.raku;
("I", "have", "a", "dream").Seq
> .say for "I have a dream".words;
I
have
a
dream
```

7.2.4.5 join

La función `join` toma un argumento separador y una lista de cadenas de texto como argumentos; la función las intercala con el separador, concatena todo en una sola cadena de texto, y devuelve la cadena de texto resultante.

Este ejemplo ilustra el uso de las funciones (o métodos) `words` y `join`:

```
say 'I have a dream'.words.join('|');    # -> I|have|a|dream
say join ";", words "I have a dream";    # -> I;have;a;dream
```

En ambos casos, `words` primero divide la cadena original en una lista de palabras, `splits` sutura los artículos de esta lista y los devuelve en una nueva cadena de texto intercalados con el separador.

7.2.4.6 Conversión a Minúscula y a Mayúscula

Las rutinas `lc` y `uc` devuelven respectivamente una versión en minúscula y una versión mayúscula de sus argumentos. También está la función `tc` la cual devuelve su argumento con la primera letra en mayúscula:

```
say lc "Abril";    # -> april
say "Abril".lc;    # -> april
say uc "abril";    # -> APRIL
say tc "abril";    # -> April
```

Recuerda que el operador `eq` chequea la igualdad de dos cadenas de texto.

7.3 Recorrido de una Cadena con un bucle while o for

Muchas computaciones involucran el procesamiento de una cadena de texto un carácter a la vez. Usualmente comienzan al principio, seleccionan un carácter iterativamente, hacen algo con dicho carácter, y continúan hasta el final. Este patrón de procesamiento se conoce como **recorrido**. Una manera de escribir un recorrido es con un bucle while y la función index:

```
my $indice = 0;
my $fruta = "banana";
while $indice < $fruta.chars {
    my $letra = substr $fruta, $indice, 1;
    say $letra;
    $indice++;
}
```

Esto imprimirá cada letra, una a la vez:

```
b
a
n
a
n
a
```

Este bucle recorre la cadena de texto y muestra cada letra en su propia línea. La condición del bucle es `$indice < $fruta.chars`, así que cuando `$indice` es igual a la longitud de la cadena de texto, la condición es falsa, y el cuerpo del bucle no se ejecuta. En otras palabras, el bucle termina cuando `$indice` es la longitud de la cadena de texto menos uno, lo cual corresponde al último carácter de la cadena de texto.

Como un ejercicio, escribe una función que toma una cadena de texto como un argumento y muestra las letras de atrás hacia adelante, una por línea. Hazlo por lo menos una vez sin utilizar la función `flip`. Solución: A.5.1.

Otra forma de escribir un recorrido es con un bucle for:

```
my $fruta = "banana";
for $fruta.comb -> $letra {
    say $letra
}
```

Cada vez a través del bucle, el siguiente carácter en la cadena de texto es asignado a la variable `$letra`. El bucle continúa hasta que no hayan más caracteres.

El bucle podría también usar la función `substr`:

```
for 0..$fruta.chars - 1 -> $indice {
    say substr $fruta, $indice, 1;
}
```


El siguiente ejemplo muestra cómo usar la concatenación y un bucle `for` para generar una tipo de abecedario (es decir, en orden alfabético). En el libro *Make Way for Ducklings* de McCloskey, los nombres de los patitos son Jack, Kack, Lack, Mack, Nack, Ouack, Pack, y Quack. Este bucle podría mostrar estos nombres en orden:

```
my $sufijo = 'ack';
for 'J'..'Q' -> $letra {
  say $letra ~ $sufijo;
}
```

La salida es:

```
Jack
Kack
Lack
Mack
Nack
Oack
Pack
Qack
```

Por supuesto, esto no es del todo correcto dado que “Ouack” y “Quack” están mal escritos. Como un ejercicio, modifica el programa para arreglar este error. Solución: A.5.2.

7.4 Bucles y Conteos

El siguiente programa cuenta el número de veces que la letra “a” aparece en una cadena de texto:

```
my $palabra = 'banana';
my $conteo = 0;
for $palabra.comb -> $letra {
  $conteo++ if $letra eq 'a';
}
say $conteo;                                # -> 3
```

Este programa demuestra otro patrón de la computación conocido como un **contador**. La variable `$conteo` se inicializa a 0 y después se incrementa por uno cada vez que se encuentra una “a”. Cuando el bucle termina, `$conteo` contiene el resultado—es decir, el número total de ocurrencias de la letra “a” en la cadena de texto.

Como un ejercicio, encapsula este código en una subrutina llamada `contar`, y generalízala para que acepte una cadena de texto y la letra a contarse como argumentos. Solución: A.5.3.

7.5 Expresiones Regulares (Regexes)

Las funciones y los métodos de cadenas de texto que hemos visto hasta ahora son realmente poderosos, y pueden usarse para un sin número de operaciones relacionadas con la

manipulación de cadenas de texto. Pero supón que quieres extraer de la cadena “yellow submarine” cualquier letra después de la letra “l” y antes de la letra “w”. Este tipo de “búsqueda difusa” puede hacerse con un bucle, pero no es práctico. Puedes hacerlo como un ejercicio si deseas, pero debo advertirte: es algo delicado y difícil. Aún si no lo haces, la solución puede interesarte: ver Solución: A.5.4.

Si agregas otra condición, por ejemplo que dicha letra debería extraerse o *capturarse* (es decir, ser almacenada para uso futuro) solo si el resto de la cadena de texto contiene la subcadena “rin”, esto comienza a volverse algo tedioso. También, cualquier cambio a los requerimientos conduce a que tengamos que hacer una reescritura substancial o hasta refactorizar el código completamente.

Para este tipo de trabajo, una **expresión regular** o un **regex** es una herramienta mucha más poderosa y expresiva. La siguiente es una manera de extraer letras usando el criterio descrito anteriormente:

```
> my $cadena = "yellow submarine";
yellow submarine
> say ~$0 if $cadena ~~ / l (.) w .*? rin /;
o
```

No te preocupes si no entiendes este ejemplo; afortunadamente todo se aclarará muy pronto.

El operador `~~` se conoce como el operador de coincidencia inteligente. Es un operador de relación muy poderoso que puede usarse para tareas de comparación muy avanzadas. En este caso, el operador chequea si la variable `$cadena` a su izquierda “coincide” con la expresión a su derecha, es decir, como una primera aproximación, si la expresión a su derecha describe la cadena de texto (o parte de la misma).

La parte `/ l (.) w .*? rin /` se llama un patrón de regex y puede describirse así: la letra “l”, seguida por cualquier otro carácter (el punto) para ser capturado (gracias a los paréntesis), seguida por la letra “w”, seguida por un número no específico de caracteres, seguida por la subcadena “rin”. ¡Uff! ¡Todo esto en una sola línea de código! Bien poderoso, no lo crees? Si la cadena de texto coincide con el patrón, entonces la coincidencia devolverá un valor verdadero y la variable `$0` será poblada con el carácter a ser capturado—la letra “o” en este caso.

A menos que sea especificado (veremos cómo hacerlo luego), los espacios en blanco no son significativos dentro de un patrón de regex. Así que puedes agregar espacios dentro de una patrón para separar sus piezas y hacer que tus intenciones sean mucho más claras.

La mayor parte de este capítulo discutirá los fundamentos de la construcción de tales patrones de regex y cómo usarlos. Pero el concepto de los regexes es tan crucial en Raku que dedicaremos un capítulo completo a este tema y otros asuntos relacionados (Capítulo 13).

La noción de las expresiones regulares es originalmente un concepto procedente de las teoría de lenguajes formales. El primer uso de expresiones regulares en la computación fue en las utilidades Unix, algunas de las cuales permanecen en extenso uso hoy en día, tales como `grep`, creada por Ken Thomson en 1973, `sed` (ca. 1974), y `awk`, desarrollada años más tarde (en 1977) por Aho, Weinberger, and Kernighan. Las versiones anteriores del lenguaje Perl, el lenguaje del cual Raku se deriva, en el decenio del 1980 incluían una versión extendida de las expresiones regulares, que ha sido desde tal entonces imitada

por muchos otros lenguajes de programación. La diferencia, no obstante, es que las expresiones regulares están profundamente arraigadas en el núcleo de los lenguajes Perl y Raku, mientras que los otros lenguajes las han adoptado como un add-on o plug-in, usualmente basado o derivado de una librería conocida como Expresiones Regulares Compatible con Perl (PCRE por sus siglas en inglés).

Las expresiones regulares de Raku han extendido estas nociones tanto que tienen muy poco que ver con el concepto original en la teoría de lenguaje, y por lo tanto se ha considerado apropiado parar de llamarlas *expresiones regulares* y en cambio hablar de *regexes*, es decir, un tipo de sub-lenguaje de coincidencia de patrón que funciona de manera similar a las expresiones regulares.

7.6 Uso de los Regexes

Una manera simple de usar un regex es utilizar el operador de coincidencia inteligente `~~`:

```
say "Coincidió" if "abcdef" ~~ / bc.e /;      # -> Coincidió
```

Aquí, el operador de coincidencia inteligente compara la cadena de texto "abcdef" con el patrón `/bc.e/` y reporta una comparación exitosa dado que, en este caso, "bc" en la cadena coincide con la parte `bc` del patrón, el punto en el patrón coincide con cualquier carácter en la cadena (coincidió en este caso con `d`) y, finalmente, la `e` de la cadena coincide con la `e` en el patrón.

La parte de la cadena que coincidió es almacenada en la variable `$/` que representa el **objeto de coincidencia**, el cual podemos convertir en una cadena de texto con el operador `~`. Podemos hacer un buen uso de esto para visualizar la parte de la cadena que coincidió con el patrón de regex:

```
say ~$/ if "abcdef" ~~ / bc.e /;      # -> bcde
```

El proceso de coincidencia podría describirse de la siguiente forma (pero ten presente que esto una simplificación): busca en la cadena de texto (desde izquierda a derecha) un carácter que coincide con el primer átomo (o sea, el primer artículo de coincidencia) del patrón; cuando se encuentra, busca si el segundo carácter puede coincidir el segundo átomo del patrón, etc. Si se usa el patrón entero, entonces el regex fue exitoso. Si falla durante el proceso, comienza nuevamente desde la posición inmediatamente después del punto de coincidencia inicial. Esto se conoce como **vuelta atrás** o **retroceso** (en. backtracking). Y se repite hasta que uno de los siguientes casos ocurra:

- Hay una coincidencia exitosa. En este caso, el proceso finaliza y se reporta éxito.
- La cadena de texto ha sido agotada sin encontrar la coincidencia. En este caso, el regex falla.

Examinemos un ejemplo de retroceso:

```
say "Coincidió" if "abcabcdef" ~~ / bc.e /;  # -> Coincidió
```

En este ejemplo, el motor de regex comienza a coincidir "bca" con *bc.*, pero el atento coincidencia inicial falla, porque la siguiente letra en la cadena de texto, "b", no coincide con la "e" del patrón. El motor de regex da vuelta atrás e inicia la búsqueda nuevamente desde la tercera letra ("c") de la cadena. Comienza una nueva coincidencia en la quinta letra de la cadena (la segunda "b"), logra coincidir exitosamente con "bcde" y sale con un estado exitoso (sin buscar ninguna otra coincidencia).

Si la cadena de texto a analizar está contenida en la variable tónica *\$_*, entonces el operador de coincidencia inteligente es implícito y la sintaxis es aún más simple:

```
for 'abcdef' {                                     # $_ ahora contiene a 'abcdef'
  say "Coincidió" if / cd.f /;                     # -> Coincidió
}
```

También puedes usar una sintaxis de invocación de método:

```
say "Coincidió" if "abcdef".match(/ b.d.f /); # -> Coincidió
```

En todos los casos que hemos visto hasta ahora, usamos directamente un patrón dentro de un par de delimitadores de barra oblicua /. Podemos utilizar otros delimitadores si precedemos nuestro patrón con la letra "m":

```
say "Coincidió" if "abcdef" ~~ m{ bc.e };         # -> Coincidió
```

o:

```
say "Coincidió" if "abcdef" ~~ m! bc.e !;         # -> Coincidió
```

El operador "m" no altera la manera en que un regex funciona; solo hace posible el uso de otros delimitadores (además de las barras oblicuas). Dicho diferentemente, el prefijo "m" es la manera estándar de introducir un patrón, pero es implícito y puede omitirse cuando el patrón es delimitado por barras oblicuas. Es probablemente mejor usar barras oblicuas, porque es lo que la gente comúnmente utiliza y reconoce inmediatamente, y usar otros delimitadores solo cuando el patrón de regex contiene barras oblicuas.

Un patrón puede también almacenarse en una variable (o, más precisamente, en un objeto regex), usando el operador *rx/*/:

```
my $regex = rx/c..f/;
say "Coincidió" if 'abcdef' ~~ $regex;             # -> Coincidió
```

7.7 Cómo Construir tus Patrones Regex

Es ahora tiempo para estudiar los componentes básicos de un patrón regex.

7.7.1 Coincidencia Literal

Como posiblemente ya has notado, el caso más simple de un patrón de regex es una cadena de texto literal. Coincidir una cadena de texto en contra de tal regex es más o menos equivalente a una búsqueda de esa cadena con la función `index`:

```
my $cadena = "superlative";
say "$cadena contiene 'perl'." if $cadena ~~ /perl/;
# -> superlative contiene 'perl'.
```

Sin embargo nota que, para tales coincidencias literales, la función `index` discutida anteriormente será probablemente más eficiente que un regex con cadenas de texto largas. El método `contains`, el cual devuelve verdadero si su argumento es una subcadena de su invocante, es también probablemente más rápido.

Los caracteres alfanuméricos y la raya baja `_` son coincidencias literales. Todos los otros caracteres deben escaparse con una barra invertida (por ejemplo `\?` para coincidir con un signo de interrogación derecho) o incluirse dentro de comillas:

```
say "Éxito" if 'nombre@org.uk' ~~ / nombre@or /;      # Falla en la compilación
say "Éxito" if 'nombre@org.uk' ~~ / 'nombre@or' /;    # -> Éxito
say "Éxito" if 'nombre@org.uk' ~~ / nombre\@or/ ;     # -> Éxito
say "Éxito" if 'nombre@org.uk' ~~ / nombre '@' org /; # -> Éxito
```

7.7.2 Comodines y Categorías de Caracteres

Los regexes no serían muy útiles si solo pudieran hacer coincidencias literales. Ahora nos acercamos a las partes más interesantes.

En un patrón de regex, algunos símbolos pueden coincidir no solo con un carácter específico, sino con una familia de caracteres, tales como letras, dígitos, etc. Estos símbolos se conocen como categorías de clases.

Ya hemos visto que el punto es un tipo de comodín que coincide con un solo carácter de la cadena de texto a coincidir:

```
my $cadena = "superlative";
say "$cadena contiene 'pe.l'." if $cadena ~~ / pe . l /;
# -> superlative contiene 'pe.l'.
```

El ejemplo más arriba ilustra otra característica de los regexes: los espacios en blanco usualmente no son significativos dentro de los patrones de regex (a menos que se especifiquen con el adverbio `:s` o `:sigspace`, como veremos más adelante).

Hay categorías de caracteres predefinidas de la forma `\w`. Su negación se escribe con una letra en mayúscula, `\W`. El carácter de palabra `\w` coincide con un solo carácter alfanumérico (o sea, entre los caracteres alfanuméricos, dígitos, y el carácter `_`). Por su parte, `\W` coincidirá cualquier otra carácter que no sea un carácter de palabra. Nota, sin embargo, que Raku es compatible con Unicode y que, por ejemplo, letras del alfabeto griego o cirílico o dígitos tailandeses coincidirán con `\w`:

```
say "Coincidió" if 'abcδ' ~~ / ab\w\w /; # -> Coincidió
```

En este ejemplo, las cadenas de texto coincidieron porque, de acuerdo al estándar Unicode, δ ("GREEK SMALL LETTER DELTA") es una letra y por lo tanto pertenece a la categoría de clase `\w`.

Otras categorías de caracteres incluyen:

- `\d` (dígitos) y `\D` (cualquier carácter menos un dígito)
- `\s` (espacio en blanco) y `\S` (cualquier carácter menos un espacio en blanco)
- `\n` (línea nueva) y `\N` (cualquier carácter menos una línea nueva).

```
say ~$/ if 'Bond 007' ~~ /\w\D\s\d\d\d/; # -> "nd 007"
```

Aquí, coincidimos con "nd 007", porque encontramos un carácter de palabra ("n"), seguido por un un carácter que no es un dígito ("d"), seguido por un espacio en blanco, seguido por tres dígitos.

También puedes especificar tus propias categorías de caracteres al insertar entre `<[ ]>` cualquier número de caracteres individuales y rangos de caracteres (expresado con dos puntos entre los puntos extremos), con o sin espacio en blanco. Por ejemplo, una categoría de caracteres para dígitos hexadecimales podría lucir así:

```
<[0..9 a..f A..F]>
```

Puedes negar cualquier categoría de caracteres al insertar un "-" después de la comilla angular izquierda. Por ejemplo, una cuerda no es un entero hexadecimal válido si contiene cualquier carácter que no está en `<[0..9a..fA..F]>`, es decir, cualquier carácter que coincide con la categoría de caracteres de hexadecimales negada:

```
say "No un número hex" if $cadena ~~ /<-[0..9 a..f A..F]>/;
```

Por favor nota que generalmente no tienes que escapar caracteres que no son alfanuméricos en tus categorías de caracteres:

```
say ~$/ if "-17.5" ~~ /(<[\d.-]>+)/; # -> -17.5
```

En este ejemplo, usamos el cuantificador "+" que discutiremos en la siguiente sección, pero el punto aquí es que no necesitas escapar el punto y la barra oblicua dentro de la definición de categoría de caracteres.

7.7.3 Cuantificadores

Un cuantificador hace que un átomo¹ precedente coincida no solamente una vez, sino un específico o variable número de veces. Por ejemplo, `a+` coincide una o más caracteres "a". En el siguiente código, `\d+` coincide con uno o más dígitos (tres dígitos en este caso):

```
say ~$/ if 'Bond 007' ~~ /\w\D\s\d+/; # -> "nd 007"
```

¹La palabra *átomo* se refiere a un carácter individual o varios caracteres u otros átomos agrupados juntos (por un conjunto de paréntesis o corchetes).

Los cuantificadores predefinidos incluyen:

- `+`: una o más veces;
- `*`: cero o más veces;
- `?`: cero o una coincidencia.

Los cuantificadores `+` and `*` se dice que son *voraces* (**en.** greedy), lo cual significa que ellos coinciden con tantos caracteres como sea posible. Por ejemplo:

```
say ~/ if 'aabaababa' ~~ / .+ b /;      # -> aabaabab
```

Aquí, `.+` coincide con tantos caracteres de la cadena de texto como sea posible, mientras que es capaz de coincidir con la final "b". Usualmente esto es lo que deseas, pero no siempre. Quizás tu intención era coincidir con todas las letras hasta la primera "b". En tales casos, usarías las versiones *frugales* (no voraces) de estos cuantificadores, las cuales se obtienen al precederlos con un signo interrogación derecho: `+`? y `*`?. Un cuantificador frugal coincidirá con tantos caracteres como sea posible para que el regex sea exitoso pero no más de lo necesario. Para coincidir con todas las letras hasta la primera *b*, podrías usar:

```
say ~/ if 'aabaababa' ~~ / .+? b /;      # -> aab
```

También podrías especificar un rango (min..max) para indicar el número de veces que un átomo debe coincidir. Por ejemplo, para coincidir con un entero menor que 1,000:

```
say 'Es un número < 1,000' if $cadena ~~ / ^ \d ** 1..3 $ /;
```

Esto coincide con uno a tres dígitos. Los caracteres `^` y `$` son anclas que representan el inicio y el final de la cadena de texto y los discutiremos en la siguiente sección.

Para coincidir con un número exacto de veces, solo reemplaza el rango con un número individual:

```
say 'Es un número de 3 dígitos' if $num ~~ / ^ \d ** 3 $ /;
```

7.7.4 Anclas y Aserciones

Algunas veces, coincidir con una subcadena no es suficientemente bueno; quieres coincidir con la cadena de texto completa, o quieres que la coincidencia ocurra al comienzo o al final de la cadena, o en algún lugar específico dentro de la cadena. Las anclas y las aserciones hacen posible especificar donde la coincidencia debe ocurrir. Necesitan coincidir exitosamente para que el regex completo pueda ser exitoso, pero no consumen los caracteres mientras la coincidencia ocurre.

7.7.4.1 Anclas

Las anclas más usadas comúnmente son `^` (inicio de la cadena de texto) y `$` (final de la cadena de texto):

```
my $cadena = "superlative";
say "$cadena comienza con 'perl'" if $cadena ~~ /^perl/; # (No salida)
say "$cadena termina con 'perl'" if $cadena ~~ /perl$/; # (No salida)
say "$cadena es igual a 'perl'" if $cadena ~~ /^perl$/; # (No salida)
```

Los tres regexes más arriba fallan porque, aunque la \$cadena contiene la subcadena “perl”, la subcadena no se encuentra al inicio, ni al final de la cadena de texto.

En el evento de que tengas que manejar cadenas de texto con líneas múltiples, podrías usar las anclas `^^` (inicio de línea) y `$$` (final de línea).

Existen otras anclas útiles, como las anclas `<<` (inicio de palabra o borde izquierdo de palabra) y `>>` (final de palabra o borde derecho de palabra).

7.7.4.2 Aserciones Look-Around

Las *aserciones look-around* (o “mirar alrededor”) hacen posible especificar reglas más complejas: por ejemplo, coincidir “foo” pero solamente si es precedida (o seguida) por “bar” (o ni precedida ni seguida por “bar”):

```
say "foobar" ~~ /foo <?before bar>/; # -> foo (aserción lookahead)
say "foobaz" ~~ /foo <?before bar>/; # -> Nil (regex fallido)
say "foobar" ~~ /<?after foo> bar/; # -> bar (aserción lookbehind)
```

Al usar un signo de exclamación derecho en lugar de un signo de interrogación derecho transforma estas aserciones look-around en aserciones negativas. Por ejemplo:

```
say "foobar" ~~ /foo <!before baz>/; # -> foo
say "foobaz" ~~ /foo <!before baz>/; # -> Nil (regex fallido)
say "foobar" ~~ /<!after foo> bar/; # -> Nil (regex fallido)
```

Asumo que los ejemplos anteriores demostraron el concepto claramente. Si quieres saber más, dirígete a la documentación (https://docs.raku.org/language/regexes#Look-around_assertions) para detalles adicionales.

7.7.4.3 Aserciones de Código

También puedes incluir una aserción de código `<?{...}>`, la cual coincidirá si el bloque de código devuelve un valor verdadero:

```
> say ~$/ if /\d\d <?{$/ == 42}>/ for <A12 B34 C42 D50>;
42
```

Una aserción de código negativa `<!{...}>` coincidirá siempre y cuando el bloque no devuelve un valor verdadero:

```
> say ~$/ if /\d\d <!{$/ == 42}>/ for <A12 B34 C42 D50>
12
34
50
```


Las aserciones de código son útiles para especificar condiciones que no pueden expresarse fácilmente como los regexes.

De igual manera, pueden usarse para mostrar algo, por ejemplo para el propósito de depurar un regex al imprimir información sobre las coincidencias parciales:

```
> say "Coincidió $/" if "A12B34D50" ~~ /(\D) <?{ say ~$0 }> \d\d$/;
A
B
D
Coincidió D50
```

La salida muestra los intentos múltiples de coincidencia que fallaron (“A” y “B”) antes que el proceso de vuelta atrás condujera al éxito (“D50” al final de la cadena).

No obstante, las aserciones de código son raramente necesarias para casos simples, porque siempre puedes añadir un bloque simple de código con el mismo propósito:

```
> say "Coincidió $/" if "A12B34D50" ~~ /(\D) { say ~$0 } \d\d$/;
```

Este código produce la misma salida, sin preocuparnos si el bloque devuelve un valor verdadero.

7.7.5 Alternancia

Las alternancias se usan para coincidir con una de varias alternativas.

Por ejemplo, para chequear si una cadena de texto representa uno de los tres colores básicos de una imagen (en JPEG y algunos otros formatos), podrías usar:

```
say 'Es un color JPEG' if $cadena ~~ /^ [ rojo | verde | azul ] $/;
```

Existen dos formas de alternancias. La alternancia de primera coincidencia (*first-match*) que usa el operador `||` y la cual termina en la primera alternativa que coincide con el patrón:

```
say ~$/ if "abcdef" ~~ /ab || abcde/;      # -> ab
```

En este ejemplo, el patrón coincide con “ab” sin tener que hacer coincidencia más adelante, aunque habría una coincidencia presumiblemente “mejor” (es decir, más larga) con la otra alternativa. Cuando se usa este tipo de alternación, tienes que pensar cuidadosamente acerca del orden en el cual colocas las múltiples alternativas, dependiendo en lo que necesitas hacer.

La alternancia de coincidencia más larga (*longest-match*) usa el operador `|` e intentará con todas las alternativas y coincidirá con la más larga:

```
say ~$/ if "abcdef" ~~ /ab | abcde/;      # -> abcde
```

Sin embargo, ten presente que esto funcionará como se explicó más arriba solo si las coincidencias alternativas comienzan en la misma posición dentro de la cadena de texto:

```
say ~$/ if "abcdef" ~~ /ab | bcde/;      # -> ab
```

Aquí, la coincidencia con la posición más a la izquierda triunfa (esta es una regla general con los regexes).

7.7.6 Grupos y Capturas

Los paréntesis y los corchetes pueden usarse para agrupar cosas o para anular la precedencia:

```
/ a || bc /      # Coincide con 'a' o 'bc'
/ ( a || b ) c / # Coincide con 'ac' o 'bc'
/ [ a || b ] c / # Igual: coincide con 'ac' o 'bc', grupo sin captura
/ a b+ /        # Coincide con una 'a' seguida por una 'b' o más
/ (a b)+ /      # Coincide con una o más secuencias de 'ab'
/ [a b]+ /      # Coincide con una o más secuencias 'ab', sin captura
/ (a || b)+ /   # Coincide con una secuencia de 'a' y 'b's (al menos una)
```

La diferencia entre los paréntesis y los corchetes es que los paréntesis no solo agrupan las cosas, sino que capturan datos: ellos permiten que la cadena de texto que coincide dentro de los paréntesis esté disponible como una variable especial (y también como un elemento del objeto de coincidencia resultante):

```
my $cad = 'número 42';
say "El número es $0" if $cad =~ /número\s+ (\d+)/; # -> El número es 42
```

En este ejemplo, el patrón coincidió con la cadena de texto \$cad y la parte del patrón dentro de los paréntesis fue capturada en la variable especial \$0. Si hay varios grupos con paréntesis, las capturas se encuentran en las variables \$0, \$1, \$2, etc. (contando los paréntesis abiertos de la izquierda a derecha):

```
say "$0 $1 $2" if "abcde" =~ /(a) b (c) d (e)/;      # -> a c e
# o: say "$/[0..2]" if "abcde" =~ /(a) b (c) d (e)/; # -> a c e
```

Las variables \$0, \$1, etc. son actualmente atajos para \$/[0], \$/[1], el primer y segundo elemento del objeto de coincidencia en un contexto de lista, así que imprimir "El número es \$/[0]" tendría el mismo efecto.

Como se indicó, los paréntesis tienen dos roles en los regexes: ellos agrupan elementos de regex y capturan lo que coincide con el subregex dentro de los paréntesis. Si solo deseas el comportamiento de agrupamiento, usa los corchetes [...]:

```
say ~$0 if 'cacbcd' =~ / [a||b] (c.) /; # -> cb
```

El uso de los corchetes cuando la captura de texto no es necesaria tiene la ventaja de no contaminar las variables \$0, \$1, \$2, etc. y es un poco más rápido.

7.7.7 Adverbios (o Modificadores)

Los adverbios modifican la manera en que el motor de regex funciona. Usualmente tiene una forma larga y una forma corta.

Por ejemplo, el adverbio :ignorecase (or :i) le dice al compilador que ignore la distinción entre letras mayúsculas y letras minúsculas:

```
> say so 'AB' ~~ /ab/;
False
> say so 'AB' ~~ /:i ab/;
True
```

La función integrada `so` usada aquí coacciona su argumento (es decir, el valor devuelto por la expresión de coincidencia de regex) a un valor Booleano.

Si se coloca en frente del patrón, el adverbio aplica al patrón entero:

```
> say so 'AB' ~~ m:i/ ab/;
True
```

El adverbio también puede colocarse después en el patrón y afectar en este caso solo la parte del regex posterior al mismo:

```
> say so 'AB' ~~ /a :i b/;
False
> say so 'aB' ~~ /a :i b/;
True
```

Anteriormente dije que los espacios en blanco usualmente no son significativos en los patrones de regex. El adverbio `:sigspace` o `:s` hace que los espacios en blanco sean significativos en un regex:

```
> say so 'ab' ~~ /a+ b/;
True
> say so 'ab' ~~ /:s a+ b/;
False
> say so 'ab' ~~ /:s a+b/;
True
```

En lugar de buscar por solo una coincidencia y devolver un objeto de coincidencia, el adverbio `:global` o `:g` le dice al compilador que busque cada coincidencia que no esté superpuesta y las devuelva en una lista:

```
> say "Conteo de palabras = ", $/.elems if "I have a dream" ~~ m:g/ \w+/;
Conteo de palabras = 4
> say ~$/[3];
dream
```

Estos son los adverbios más usados comúnmente. Otro adverbio, `:ratchet` o `:r`, le dice al motor de regex que no vuelva atrás y es muy importante para usos específicos, pero regresaremos a esto en otro capítulo (ver Sección 13.4).

7.7.8 Ejercicios con Regexes

Como un ejercicio simple, escribe algunos regexes para coincidir y capturar:

- Una sucesión de 10 dígitos dentro de otra cadena de texto más larga;

- Un número octal válido (números en el sistema octal solo usan los dígitos del 0 al 7);
- La primera palabra al inicio de una cadena de texto (para el propósito de estos pequeños ejercicios, el separador de palabra puede considerarse un espacio, pero podrías hacerlo sin esta suposición);
- La primera palabra de una cadena de texto que comienza con una “a”;
- La primera palabra de una cadena de texto que comienza con una letra minúscula;
- Un número telefónico francés (en Francia, los números telefónicos tienen 10 dígitos y comienzan con “06” o “07”); puedes asumir que los dígitos son consecutivos (sin espacios);
- La primera palabra de una cadena de texto que comienza con una vocal en mayúscula o minúscula;
- La primera ocurrencia de una letra doble (la misma letra dos veces);
- La segunda ocurrencia de una letra doble;

Solución: A.5.5.

7.8 Combinando Todas las Piezas

La intención de esta sección es proveerte con algunos ejemplos usando varias de las características de los regex que hemos visto para resolver problemas prácticos.

7.8.1 Extracción de Fechas

Asume que tenemos una cadena de texto que contiene en alguna parte una fecha en el formato YYYY-MM-DD:

```
my $cadena = "Christmas : 2016-12-25.";
```

Como mencionamos anteriormente, uno de los lemas en Raku es “Hay más de una forma para hacer algo” (TIMTOWTDI). Los ejemplos múltiples más abajo deberían ilustrar este principio bastante bien al mostrar varias maneras diferentes para extraer la fecha en la cadena:

- Uso de una categoría de caracteres (dígitos y guion):

```
say ~$0 if $cadena ~~ /(<[\d-]>+)/;      # -> 2016-12-25
```

- Uso de una categoría de caracteres y un cuantificador para evitar coincidir con algunos números pequeños en otra parte de la cadena si existen:

```
say ~$0 if $cadena ~~ /(<[\d-]> ** 10)/;  # -> 2016-12-25
```

- Uso de una descripción más detallada del formato de la fecha:

```
say ~$/ if $cadena ~~ /(\d ** 4 \- \d\d \- \d\d)/;
```

- El mismo regex, pero usando un agrupamiento adicional para evitar la repetición del subpatrón `\- \d\d`:

```
say ~/[0] if $cadena ~~ /(\d ** 4 [\- \d\d] ** 2 )/;
```

- Captura de elementos individuales de la fecha:

```
$cadena ~~ /(\d ** 4) \- (\d\d) \- (\d\d)/;
my ($año, $mes, $día) = ~$0, ~$1, ~$2;
```

Nota que el uso de la tilde como un prefijo más arriba causa que `$año`, `$mes`, y `$día` sean pobladas con cadenas de texto. Asumiendo que desees que estas variables contengan números enteros, podrías coaccionarlas en valores numéricos usando el operador prefijo `+`:

```
$cadena ~~ /(\d ** 4) \- (\d\d) \- (\d\d)/;
my ($año, $mes, $día) = +$0, +$1, +$2;
```

- Uso de subpatrones como componentes básicos:

```
my $a = rx/\d ** 4/;
my $m = rx/\d ** 2/;
my $d = rx/\d ** 2/;
$cadena ~~ /(<$a> \- (<$m>) \- (<$d>)/;
my ($año, $mes, $día) = ~$0, ~$1, ~$2;
```

El uso de subpatrones como componentes básicos es una manera bastante eficiente de construir un regex complicado paso a paso, pero como veremos en el Capítulo 13 hay mejores maneras de hacer este tipo de cosas.

- Podríamos mejorar el subpatrón `$m` (mes) para que coincida solo con “01” al “12” y así verificar que solo coincida con un número de mes válido:

```
my $m = rx { 1 <[0..2]> # 10 a 12
             || 0 <[1..9]> # 01 a 09
           };
```

Como puedes ver, el uso de comentarios y espacios en blanco facilita que la intención del regex sea más clara.

Otra forma de lograr la misma meta es usar una aserción de código para chequear que el valor se encuentre numéricamente entre 1 y 12:

```
my $m = rx /\d ** 2 <?[ 1 <= $/ <= 12 ]> /;
```

Como un ejercicio, podrías intentar validar que el subpatrón `$d` (día) cubra los números en el rango 01 y 31. Intenta usar ambas técnicas de validación que fueron discutidas anteriormente.

El objeto de coincidencia `$/` tiene los métodos `prematch` y `postmatch` para extraer lo que viene antes y después de la parte que coincidió en la cadena de texto:

```
$cadena ~~ /(\d ** 4) \- (\d\d) \- (\d\d)/;
say $/.prematch;           # -> "Christmas : "
say $/.postmatch;          # -> "."
```

Como un ejercicio, intenta adaptar los regexes anteriores para varios formatos de fecha (tales como DD/MM/YYYY o YYYY MM, DD) y pruébalos. Si intentas con el formato YYYY MM, DD, por favor recuerda que los espacios no son usualmente significativos en un patrón de regex, así que tendrías que especificar espacios en blanco explícitos (con el uso de la categoría de caracteres `\s`) o utilizar el adverbio `:s` para hacer los espacios significativos.

7.8.2 Extracción de una dirección IP

Asumamos que tenemos una cadena de texto que contiene una dirección IP-v4 en algún lugar. Las direcciones IP son usualmente escritas en la notación de punto decimal, la cual consiste de cuatro octetos de la dirección expresados individualmente en números decimales y separados por puntos, por ejemplo 17.125.246.28.

Para el propósito de estos ejemplos, nuestra muestra de cadena de texto será la siguiente:

```
my $cadena = "IP address: 17.125.246.28;"
```

Ahora intentemos diferentes maneras de capturar la dirección IP en esa cadena, en la misma manera que lo hicimos con las fechas:

- Uso de una categoría de caracteres:

```
say ~$0 if $cadena =~ /(<[\d.]>+)/;           # -> 17.125.246.28
```

- Uso de una categoría de caracteres y un cuantificador (nota que cada octeto puede tener de uno a tres dígitos, así que el total de números de caracteres puede variar de 7 a 15):

```
say ~$0 if $cadena =~ /(<[\d.]> ** 7..15)/;
```

- Uso de una descripción más detallada del formato IP:

```
say ~$/ if $cadena =~ /([\d ** 1..3 \.] ** 3 \d ** 1..3 )/;
```

- Uso de subpatrones como componentes básicos:

```
my $octeto = rx/\d ** 1..3/;
say ~$/ if $cadena =~ /(<$octeto> \.] ** 3 <$octeto>)/;
```

- El valor máximo de un octeto es 255. Podemos definir nuevamente la el subpatrón `$octeto`:

```
my $octeto = rx/<[1..2]>? \d ** 1..2/;
say ~$/ if $cadena =~ /(<$octeto> \.] ** 3 <$octeto>)/;
```

Con esta definición del subpatrón `$octeto`, el regex podría coincidir con cualquier número de uno o dos dígitos, o un número de tres dígitos comenzando con los dígitos del 1 al 2.

- Pero esto no es suficientemente bueno si realmente queremos chequear que la dirección IP es válida (por ejemplo, podría aceptar a 276 como un octeto válido). La definición del subpatrón `$octeto` puede ser redefinida más exhaustivamente para coincidir solo con valores autorizados:

```

my $octeto = rx { (    25 <[0..5]>      # 250 a 255
                    || 2  <[0..4]> \d    # 200 a 249
                    || 1  \d ** 2      # 100 a 199
                    || \d ** 1..2      # 0 a 99
                    )
};

say ~$/ if $cadena ~~ /([<$octeto> \.] ** 3 <$octeto>)/;

```

Esta definición de \$octeto ilustra una vez más cómo el uso abundante de espacios en blanco y comentarios ayuda a hacer la intención más clara.

- Podríamos usar una aserción de código para limitar el valor de un \$octeto al rango 0..255:

```

my $octeto = = rx{(\d ** 1..3) <?{0 <= $0 <= 255 }> };
say ~$/ if $cadena ~~ /([<$octeto> \.] ** 3 <$octeto>)/;

```

7.9 Sustituciones

El reemplazo de una parte de una cadena de texto con otra subcadena es un requerimiento muy frecuente en el manejo de cadenas de texto. Esto podría ser necesario para la corrección de errores ortográficos, formateo de datos, remoción de información personal de los datos, etc.

7.9.1 El Método subst

Raku tiene un método subst el cual reemplaza algún texto con otro texto:

```

my $cadena = "abcdefg";
$cadena = $cadena.subst("cd", "DC");          # -> abDCefg

```

El primer argumento de este método es la parte de búsqueda, y puede ser una cadena de texto literal, como en el ejemplo más arriba, o un regex:

```

my $cadena = "abcdefg";
$cadena = $cadena.subst(/c \w+ f/, "SUBST");  # -> abSUBSTg

```

7.9.2 La construcción s/búsqueda/reemplazo/

La manera más común de realizar una sustitución de texto en Raku es con la construcción s/búsqueda/reemplazo/, la cual es bien concisa, funciona muy bien con la sintaxis general de los regexes, y tiene la ventaja de habilitar la sustitución *in situ*.

Esto es un ejemplo de la sintaxis estándar para este tipo de sustitución:

```

my $cadena = "salviavodas";
$cadena ~~ s/ v \w+ o /VAVI/;                # -> salVAVIIdas

```

Aquí, la parte de búsqueda es un patrón de regex y el reemplazo es una simple cadena de texto (las comillas no son necesarias).

Si la cadena de texto de entrada se encuentra en la variable tópica \$_, no necesitas usar el operador de coincidencia inteligente:

```
$_ = "salviavodas";
s/ v \w+ o /VAVI/;           # -> salVAVIDas
```

Los delimitadores no tienen que ser barras oblicuas (y esto puede ser bastante útil si la búsqueda o el reemplazo contiene barras oblicuas):

```
my $cad = "<h2>Tema</h2> <h1>Subtema</h1>";
$cad =~ s!'<h1>Subtema</h1>'!<h3>Subtema</h3>!

# $cad es ahora: <h2>Tema</h2> <h3>Subtema</h3>
```

A menos que se especifique lo contrario (con un adverbio), la sustitución se realiza una sola vez, lo cual ayuda a prevenir resultados inesperados:

```
$_ = 'Hay caince casas.';
s/ca/qu/;           # Reemplaza 'ca' con 'qu' una sola vez
.say;              # Hay quince casas.
```

Si la sustitución se hubiese hecho en la cadena de texto completa, "casas" se hubiese reemplazado con "qusas", lo cual claramente no era el resultado esperado.

7.9.3 Uso de Capturas

Si el regex en el lado izquierdo contiene capturas, el reemplazo en el lado derecho puede usar las variables \$0, \$1, \$2, etc. en el lado derecho para insertar subcadenas capturadas en el texto de reemplazo. Un ejemplo típico de eso es el reformato de fechas:

```
my $cadena = "Xmas = 2016-12-25";
$cadena =~ s/(\d ** 4) \- (\d\d) \- (\d\d)/$2-$1-$0/;
# $cadena es ahora: Xmas = 25-12-2016
```

7.9.4 Adverbios

Los adverbios discutidos más arriba (Sección 7.7.7) pueden usarse con el operador de sustitución.

Los modificadores más usados comúnmente en sustituciones son los adverbios :ignorecase (o :i) y :global (o :g). Ellos funcionan como lo describimos en la Subsección 7.7.7 de la sección sobre regexes y coincidencia.

El punto que quiero hacer aquí es que las sustituciones se hacen usualmente solo una vez. Pero con el adverbio :global (o :g), se realizarán en toda la cadena de texto:

```
# Sin sustitución global:
my $cad = "...y tan a su gusto, a su caballo...";
$cad =~ s/su/mi/; # $cad es ahora: "...y tan a mi gusto, a su caballo..."

# Con sustitución global:
my $cad = "...y tan a su gusto, a su caballo...";
$cad =~ s:g/su/mi/; # $cad es ahora: "...y tan a mi gusto, a mi caballo..."
```


7.10 Depuración de Programas

Cuando usas los índices para recorrer los valores en una secuencia, es un poco difícil conseguir el inicio y el final del recorrido correctamente. Aquí tenemos una subrutina que está supuesta a comparar dos palabras y devolver True si una de las palabras es la inversa de la otras, pero contiene dos errores:

```
# ADVERTENCIA, ten cuidado: código con errores
sub es-inversa( Str $palabra1, Str $palabra2 ) {
    return False if $palabra1.chars != $palabra2.chars;

    my $i = 0;
    my $j = $palabra2.chars;

    while $j > 0 {
        say '$i = ', $i, ' $j = ', $j;
        return False if substr($palabra1, $i, 1) ne substr($palabra2, $j, 1);
        $i++; $j--;
    }
    return True;
}
say es-inversa "pots", "stop";
```

La primera sentencia sufijo `if` chequea si las palabras tienen la misma longitud. Si no, podemos devolver False inmediatamente. De lo contrario, por el resto de la subrutina, podemos asumir que las dos palabras tienen la misma longitud. Este es un ejemplo del patrón guardián descrito en la Sección 5.9 (p. 85).

Las variables `$i` y `$j` son los índices: `$i` recorre a `$palabra1` hacia adelante mientras `$j` recorre a `$palabra2` hacia atrás. Si encontramos dos letras que no coinciden, podemos devolver False inmediatamente. Si logramos pasar a través del bucle y todas las letras coinciden, devolvemos True.

Si probamos esta función con las palabras "stop" y "pots", esperamos el valor de retorno True, pero obtenemos False en lugar de eso. Así que, cuál es el problema?

Con este tipo de código, el sospechoso usual es un posible error en el manejo de los índices (quizás un error de un solo número). Para depurar este tipo de error, el primer paso podría ser la impresión de los valores de los índices inmediatamente antes de las líneas donde se usan:

```
sub es-inversa( Str $palabra1, Str $palabra2 ) {
    return False if $palabra1.chars != $palabra2.chars;

    my $i = 0;
    my $j = $palabra2.chars;

    while $j > 0 {
        say '$i = ', $i, ' $j = ', $j;
        return False if substr($palabra1, $i, 1) ne substr($palabra2, $j, 1);
        $i++; $j--;
    }
}
```

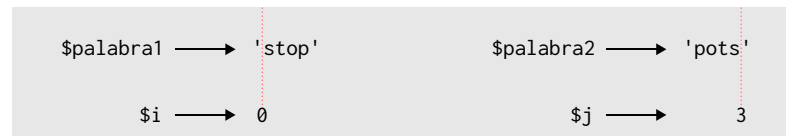


Fig. 7.1: Diagrama de estado.

```

    return True;
}

```

Ahora cuando ejecutemos el programa nuevamente, obtenemos más información:

```

$i = 0 $j = 4
False

```

La primera vez a través del bucle, el valor de `$j` es 4, el cual está fuera de rango para la cadena de texto `'pots'`. El índice del último carácter es 3, así que el valor inicial para `$j` debería ser `$palabra2.chars - 1`.

Nota que en el evento de no haber sido suficiente para rastrear el error de fuera de rango, podríamos haber ido un paso más allá e imprimir las letras. Al hacer esto, hubiésemos visto que no obteníamos la última letra de la segunda palabra.

Si arreglamos el error y ejecutamos el programa nuevamente, obtenemos:

```

$i = 0 $j = 3
$i = 1 $j = 2
$i = 2 $j = 1
True

```

Esta vez obtenemos la respuesta correcta, pero parece ser que el bucle solo se ejecutó tres veces, lo cual es sospechoso: parece que el programa no comparó la última letra de la primera palabra (indexada `$i = 3`) con la última letra de la segunda palabra (indexada `$j = 0`).

Podemos confirmar esto al ejecutar la subrutina con los siguientes argumentos: `"stop"` y `"lots"`, lo cual muestra:

```

$i = 0 $j = 3
$i = 1 $j = 2
$i = 2 $j = 1
True

```

Esto es obviamente incorrecto, dado que `"lots"` no es la inversa de `"stop"`, la subrutina debería devolver `False`. Así que tenemos otro *bug* en nuestras manos.

Para tener una mejor idea de lo que está pasando, es útil dibujar un diagrama de estado. Durante la primera iteración, el cuadro de la función `es-inversa` se muestra en la Figura 7.1.

Tomamos ciertas libertades para acomodar las variables en el cuadro y agregar líneas punteadas para mostrar que los valores de `$i` y `$j` indican los caracteres en `$palabra1` y `$palabra2`.

Comienza con este diagrama, ejecuta el programa en papel, cambia los valores de \$i y \$j durante cada iteración. Encuentra y arregla el segundo error en esta función.

Solución: A.5.6.

7.11 Glosario

Objeto Algo que una variable puede almacenar. Por el momento, puedes usar “objeto” y “valor” indistintamente.

Secuencia Una colección ordenada de valores donde cada valor es identificado por un índice entero.

Artículo Uno de los valores en una secuencia.

Índice Un valor entero que se usa para seleccionar un artículo en una secuencia, tal como un carácter en un cadena de texto. En Raku, los índices comienzan desde el 0.

Rebanada Una parte de una cadena de texto especificada por un rango de índices.

Cadena de texto vacía Una cadena de texto sin caracteres y con longitud 0, representada por dos comillas.

Recorrer Iterar a través de los artículos de una secuencia, realizando una operación similar en cada uno de ellos.

Búsqueda Una patrón de recorrido que cesa cuando encuentra lo que está buscando.

Contador Una variable que se usa para contar algo, usualmente inicializada a cero y posteriormente incrementada.

Expresiones regulares Un sublenguaje de computación derivado de la teoría de lenguajes formales.

Patrón Una secuencia de caracteres con sintaxis especial que describen de izquierda a derecha el contenido que se intenta coincidir dentro de una cadena de texto en particular.

Regexes Un sublenguaje de coincidencia de patrón de Raku derivado de las expresiones regulares.

Retroceso El proceso por el cual, dado un intento de coincidencia fallido en una cadena de texto, el motor de regex abandona parte del intento de coincidencia actual, vuelve atrás en la cadena de texto, e intenta nuevamente para ver si puede encontrar una nueva ruta a una coincidencia exitosa. El proceso de retroceso eventualmente termina tan pronto una coincidencia exitosa ocurre, o finalmente cuando todas las coincidencias posibles fallan. Este proceso se conoce como *backtracking* en inglés.

7.12 Ejercicios

Ejercicio 7.1. *Escribe una subrutina que use la función `index` en un bucle para contar el número de caracteres “a” en ``banana`` como hicimos en la Sección 7.4. Modifícala para contar cualquier letra en cualquier palabra pasadas como argumentos a la subrutina.*

Escribe otra subrutina para contar una letra dada en una palabra dada usando la función `substr`.

Solución: A.5.7.

Ejercicio 7.2. *La categoría de caracteres `<[a..z]>` coinciden con cualquier carácter en minúscula (solo caracteres ASCII en minúscula, no caracteres Unicode). La siguiente subrutina:*

```
sub es-minúscula( Str $char ) {
    return so $char ~~ /^[a..z]$/
}
```

debería devolver True si su argumento es una letra ASCII en minúscula y de lo contrario, False. Asegúrate de que funcione como se espera (y arrégla si es necesario). La función `so` coacciona el resultado de la coincidencia de regex en un valor Booleano.

Las siguientes subrutinas usan la subrutina `es-minúscula` y su trabajo es chequear si una cadena de texto contiene letras en minúsculas, pero algunas de ellas no funcionan adecuadamente. Analiza cada subrutina manualmente, determina si funciona bien, y describe lo que actualmente hace (puedes asumir que el parámetro es una cadena de texto). Después, pruébalas todas con varias cadenas de texto para chequear si tu análisis estaba en lo cierto.

```
# ADVERTENCIA: algunas de las siguientes subrutinas
#               no funcionan correctamente.
```

```
sub cualquier_minúscula1( Str $cadena ){
    for $cadena.comb -> $char {
        if es-minúscula $char {
            return True;
        } else {
            return False;
        }
    }
}
```

```
sub cualquier_minúscula2( Str $cadena ) {
    for $cadena.comb -> $char {
        if es-minúscula "char" {
            return True;
        } else {
            return False;
        }
    }
}
```

```
sub cualquier_minúscula3( Str $cadena ) {
    my $bandera;
    for $cadena.comb -> $char {
```

```
        $bandera = es-minúscula $char;
    }
    return $bandera;
}

sub cualquier_minúscula4( Str $cadena ) {
    my $bandera = False;
    for $cadena.comb -> $char {
        $bandera = $bandera or es-minúscula $char;
    }
    return $bandera;
}

sub cualquier_minúscula5( Str $cadena ) {
    my $bandera = False;
    for $cadena.comb -> $char {
        if es-minúscula $char {
            $bandera = True;
        }
    }
    return $bandera;
}

sub cualquier_minúscula6( Str $cadena ) {
    for $cadena.comb -> $char {
        if es-minúscula $char {
            return 'True';
        }
    }
    return 'False';
}

sub cualquier_minúscula7( Str $cadena ) {
    for $cadena.comb -> $char {
        return True if es-minúscula $char;
    }
    return False;
}

sub cualquier_minúscula8( Str $cadena ) {
    for $cadena.comb -> $char {
        return False unless es-minúscula $char;
    }
    return True;
}

sub cualquier_minúscula9( Str $cadena ) {
    for $cadena.comb -> $char {
        if not es-minúscula $char {
            return False;
        }
    }
    return True;
}
```

```

    }
    return True;
}
}

```

Solución: A.5.8.

Ejercicio 7.3. Un cifrado César es una forma débil de cifrado que involucra la “rotación” de cada letra por un número fijo de posiciones. La rotación de una letra se refiere a su desplazamiento a través del alfabeto, y puede envolverse alrededor si es necesario, así que “A” rotada por 3 corresponde a “D” y “Z” rotada por 1 corresponde a “A”.

Para rotar una palabra, rota cada letra por el mismo número de posiciones. Por ejemplo, “educar” rotada por 7 es “lkbjhy” y “carta” rotada por -10 es “sqhjq”. (Para el cifrado, usamos solo caracteres del alfabeto básico latino ISO.) En la película 2001: A Space Odyssey, la computadora de la nave se llama HAL, que es una rotación de IBM por -1.

Escribe una función llamada `rotar-palabra` que toma una cadena de texto y un número entero como parámetros, y devuelve una nueva cadena que contiene las letras de la cadena original rotadas por el monto dado.

Podrías desear utilizar las funciones integradas `ord`, la cual convierte un carácter a su código numérico (Unicode code point), y `chr`, la cual convierte tales código numérico devuelta a los caracteres:

```

> say 'c'.ord;
99
> say chr 99
c

```

Las letras del alfabeto están codificadas en orden alfabético, así que por ejemplo:

```

> ord('c') - ord('a')
2

```

dado que 'c' es la segunda letra después de 'a' en el alfabeto. Pero ten pendiente: los códigos numéricos para las letras mayúsculas son diferentes.

Bromas en la Internet que tienen el potencial de ser ofensivas son algunas veces codificadas en ROT13, el cual es un cifrado César con rotación 13. Dado que 13 es la mitad del número de letras en nuestro alfabeto, la aplicación de una rotación por 13 devuelve la palabra original, así que el mismo procedimiento puede usarse para cifrar y descifrar en rotación 13. Si no te ofendes fácilmente, encuentra y descifra algunas de estas bromas. (ROT13 se usa también para otros propósitos, tales como ocultar débilmente la solución de un acertijo.)

Solución: A.5.9.

Caso Práctico: Juego de Palabras

En lugar de introducir conceptos nuevos, el propósito de este capítulo es dejarte practicar y consolidar el conocimiento que has adquirido hasta ahora. Para ayudarte a ganar experiencia con la programación, discutiremos un caso práctico que involucra la solución de sopas de palabras al buscar palabras que tienen ciertas propiedades. Por ejemplo, encontraremos los palíndromos más largos en inglés y buscaremos palabras cuyas letras aparecen en orden alfabético. Y presentaré otro plan de desarrollo de programa: la reducción a un programa previamente solucionado.

8.1 Lectura de y Escritura a un Archivo

Para los ejercicios en este capítulo, necesitaremos que nuestros programas lean texto desde los archivos. En muchos lenguajes de programación, usualmente esto significa que necesitamos una sentencia que abra un archivo, después una sentencia o grupo de sentencias para leer el contenido del archivo, y finalmente una sentencia para cerrar el archivo (aunque esta operación puede realizarse automáticamente en algunas circunstancias).

Aquí estamos interesados en archivos de texto que están compuestos por líneas separadas por caracteres lógicos de nueva línea; dependiendo de tu sistema operativo, caracteres lógicos de nueva línea consisten de uno (Linux, Mac) o dos (Windows) caracteres físicos (bytes).

La función integrada de Raku `open` toma la ruta (**en.** `path`) la cual indica donde el archivo se encuentra y el nombre del archivo. `open` devuelve un **descriptor de archivo** (`IO::Handle` object) que puedes usar para leer (o para escribir) al mismo:

```
my $fh = open("ruta/a/mi_archivo.txt", :r);
my $datos = $fh.slurp-rest;
$fh.close;
```

La opción `:r` (`read`) es el modo de escritura. `$fh` es un nombre común para un descriptor de archivo (**en.** `file handle`). El *objeto de archivo* provee métodos para leer, tales como

`slurp-rest` el cual devuelve el contenido completo del archivo desde la posición actual hasta el final (y el contenido completo del archivo si lo hemos abierto por primera vez).

Esta es la manera tradicional de abrir y leer archivos en la mayoría de los lenguajes.

No obstante, el rol IO de Raku (en términos simples, un rol es una colección de métodos relacionados) ofrece métodos más simples los cuales pueden abrir y leerlo en una sola instrucción (o sea, sin tener que primero abrir un descriptor de archivo y después cerrarlo):

```
my $texto = slurp "ruta/a/mi_archivo.txt";
# alternativamente:
my $texto = "ruta/a/mi_archivo.txt".IO.slurp;
```

La función (o método) `slurp` toma la responsabilidad de abrir y cerrar el archivo por ti.

Podemos también leer el archivo línea por línea, lo cual es muy práctico si cada línea contiene una entidad lógica tal como un registro, y es especialmente útil para archivos muy grandes que podrían no caber en la memoria:

```
for 'ruta/a/mi_archivo.txt'.IO.lines -> $línea {
    # Haz algo con la $línea
}
```

Por defecto, el método `.lines` removerá los caracteres de nueva línea colgantes de cada línea, para que no tengas que preocuparte sobre ellos.

No hemos estudiados los arrays aún, pero puedes también leer todas las líneas de un archivo en un array, con cada línea del archivo volviéndose un artículo del array. Por ejemplo, puedes cargar el archivo `mi_archivo.txt` en el array `@líneas`:

```
my @líneas = "mi_archivo.txt".IO.lines;
```

El acceso a cada línea puede entonces hacerse con el operador bracket y un índice. Por ejemplo, para imprimir la primera y séptima línea:

```
say @líneas[0];
say @líneas[6];
```

Para escribir datos en un archivo, es posible abrirlo al igual que cuando queremos leer desde el mismo, excepto que la opción `:w` (write) (*escritura*) debe usarse:

```
my $fh = open("ruta/a/mi_archivo.txt", :w);
$fh.say("escribir esta línea en el archivo");
$fh.close;
```

Sé cuidadoso con esto. Si el archivo ya existía, cualquier contenido será sobrescrito. Así que sé cauteloso al abrir un archivo en el modo de escritura, debido a que podrías borrar todo el contenido de un archivo si no tienes cuidado.

También es posible abrir el archivo en modo de concatenación, usando la opción `:a` (append). Con esta opción, los datos nuevos serán entonces agregados al contenido existente.

La escritura a un archivo puede simplificarse con el uso de la función `spurt`, la cual abre el archivo, escribe los datos en el mismo, y consecuentemente lo cierra:

```
spurt "ruta/a/mi_archivo.txt", "escribir esta línea en el archivo\n";
```

Usado de esta manera, `spurt` sobrescribirá cualquier contenido existente en el archivo. También puede usarse en el modo de concatenación con la opción `:append`:

```
spurt "ruta/a/mi_archivo.txt",
      "escribir esta línea en el archivo\n ",
      :append;
```

8.2 Lectura de Listas de Palabras

Para los ejercicios en este capítulo necesitamos una lista de palabras en español. Hay muchas listas de palabras en la web, pero una de la más adecuada para nuestro propósito es una de las listas de palabras coleccionada y cedida al dominio público por Grady Ward como parte del proyecto léxico Moby (ver http://wikipedia.org/wiki/Moby_Project). Es una lista de 113,809 palabras oficiales de crucigramas; es decir, palabras que son consideradas válidas en un crucigrama y otros juegos de palabras. En la colección Moby, el nombre del archivo es *spanish.txt*; puedes descargar una copia desde <https://www.gutenberg.org/files/3206/files/spanish.txt>.

Nota del traductor: En lugar de utilizar la lista de palabras en *spanish.txt* del proyecto Moby, he decidido utilizar la lista en *lemario.txt* (<https://gitlab.com/uzluisf/piensa-raku/tree/master/suplementos/lemario.txt>) que contiene aproximadamente 88,000 palabras¹. La decisión se debe a que la lista de palabras en español del proyecto Moby contiene muchas palabras que no están formateadas apropiadamente y esto dificultaría el procesamiento adecuado de las palabras.

Algunos ejercicios y ejemplos más adelante usan la lista de palabras *words.txt* usada en *Think Raku* 6. Dicha lista puede encontrarse aquí (https://raw.githubusercontent.com/LaurentRosenfeld/think_raku/master/Supplementary/words.txt).²

Este archivo contiene texto simple (con cada palabra de la lista en su propia línea), así que puedes abrirlo con un editor de texto, pero también puedes leerlo con Raku. Hagamos esto en el modo interactivo (con el REPL):

```
> my $fh = open("lemario.txt", :r);
IO::Handle<lemario.txt>(opened, at octet 0)
> my $línea = get $fh;
aa
> say "<<$línea>>";
<<aa>>
```

La función `get` lee una línea del descriptor de archivo.

La primera palabra en esta lista en particular es "a".

¹También puedes encontrar el *lemario* y más información sobre el mismo aquí: (<http://olea.org/proyectos/lemarios/index.html>)

²O alternativamente aquí: <https://gitlab.com/uzluisf/piensa-raku/raw/master/suplementos/words.txt>

Imprimir la variable `$línea` entre los paréntesis angulares dentro de una cadena de texto nos muestra que la función `get` removi6 explícitamente los caracteres de nueva línea colgantes, en este caso una combinación de los caracteres `\r\n` (un retorno de carro y una nueva línea), dado que este archivo aparentemente fue preparado en Windows.

El descriptor de archivo mantiene un registro de lo que se ha leído desde el archivo y lo que debe leerse después, así que si llamas a `get` otra vez, obtienes la siguiente línea:

```
> my $línea = get $fh;
a-
```

La palabra siguiente es “a-”. Si llamas a `get` nuevamente, obtienes la palabra “aaronico”.

Esto es grandioso si queremos explorar las primeras líneas del archivo `lemario.txt`, pero no vamos a explorar las casi 88,000 líneas del archivo de esta forma.

Necesitamos que un bucle lo haga por nosotros. Podríamos insertar la nueva instrucción `get` en un bucle `while`, pero hemos visto una manera más fácil y más eficiente de hacerlo con un bucle `for` y el método `IO.lines`, sin la inconveniencia de abrir o cerrar el archivo:

```
for 'lemario.txt'.IO.lines -> $línea {
    say $línea;
}
```

Este código lee el archivo línea por línea, e imprime cada línea en la pantalla. Pronto haremos cosas más interesantes que solo mostrar las líneas en la pantalla.

8.3 Ejercicios

Este caso práctico consiste principalmente de ejercicios y soluciones dentro del cuerpo del capítulo porque solucionar los ejercicios es la manera principal de enseñar el material de este capítulo. Por lo tanto, las soluciones a estos ejercicios se encuentran en las siguientes secciones de este capítulo, no en el apéndice. Por lo menos deberías tratar de solucionar cada uno de ellos antes de leer las soluciones.

Ejercicio 8.1. *Escribe un programa que lee el archivo `lemario.txt` e imprime solo las palabras con más de 20 caracteres.*

Ejercicio 8.2. *En 1939, Ernest Vincent Wright publicó una novela con aproximadamente 50,000 palabras llamada *Gadsby* que no contiene la letra “e”. Dado que “e” es la letra más común en inglés, eso no es fácil de hacer. De hecho, es difícil construir un solo pensamiento sin usar la letra más común. Tales escritos donde se omite alguna letra son algunas veces conocidos como un lipogramas.*

Escribe una subrutina llamada `sin-e` que devuelve `True` si la palabra dada no contiene la letra “e”.

Modifica tu programa del ejercicio previo para imprimir solamente palabras que no tienen “e” y calcular el porcentaje de palabras en la lista que no tienen “e”.

(El diccionario de palabras que estamos usando, `lemario.txt`, tiene todas las letras en minúsculas, así que no tienes que preocuparte acerca de la “E” mayúscula.)

Ejercicio 8.3. *Escribe una subrutina llamada `evitar` que toma una palabra y una cadena de texto de letras prohibidas, y devuelve `True` si la palabra no contiene ninguna de las letras prohibidas.*

Después, modifica tu programa para encitar al usuario a entrar una cadena de texto de letras prohibidas y después imprimir el número de palabras que no contienen ningunas de ellas. ¿Puedes encontrar una combinación de cinco letras prohibidas que excluya el menor número de palabras?

Ejercicio 8.4. *Escribe una subrutina llamada `usa-solamente` que toma una palabra y una cadena de letras, y devuelve `True` si la palabra contiene solo las letras en la lista. ¿Puedes construir una oración usando solo las letras `acefhlo`?*

Ejercicio 8.5. *Escribe una subrutina llamada `usa-todas` que toma una palabra y una cadena de letras requeridas, y devuelve `True` si la palabra usa todas las letras requeridas por lo menos una vez. ¿Cuántas palabras se encuentran ahí que usan todas las vocales `aeiou`? ¿Y cuántas usan `aeiouy`?*

Ejercicio 8.6. *Escribe una función llamada `es_abecedaria` que devuelve `True` si las letras en una palabra aparecen en orden alfabético (letras dobles son aceptables). ¿Cuántas palabras son abecedarias?*

8.4 Búsqueda

La mayoría de los ejercicios en la sección previa tiene algo en común; pueden solucionarse con el patrón de búsqueda (y la función `index` que vimos en la Sección 7.2.2 p. 109). Muchos de ellos también pueden solucionarse con `regexes`.

8.4.1 Palabras con Más de 20 Caracteres (Solución)

La solución al ejercicio más simple —imprimir todas las palabras de `lemario.txt` que tienen más de 20 caracteres— es:

```
for 'lemario.txt'.IO.lines -> $línea {
  say $línea if $línea.chars > 20
}
```

Dado que el código es tan simple, esto es un ejemplo típico de un one-liner útil (como se describió en la Sección 2.5). Asumiendo que quieras saber las palabras con más de 20 caracteres, no necesitas ni siquiera escribir un script, guardarlo y ejecutarlo. Simplemente puedes teclear esto en la prompt de tu sistema operativo:

```
$ raku -n -e '$_.say if $_.chars > 20;' lemario.txt
```

La opción “-e” le dice a Raku que el script a ser ejecutado viene en la línea de comando entre comillas. La “-n” le indica a Raku que lea el archivo línea por línea después de la línea de comando, almacene cada línea en la variable tópico `$_`, y aplique el contenido del script a cada línea. Y el script en modo one-liner solo imprime el contenido de `$_` si su longitud es mayor que 20.

Para simplificarlo un poco, las dos opciones `-n` y `-e` pueden agruparse como `raku -ne`. En adición, la variable tópico `$_` puede omitirse en las llamadas de método (en otras palabras, si un método no tiene invocante, el invocante será `$_` por defecto). Finalmente, el punto y coma colgante puede igualmente removerse. El script en modo one-liner más arriba puede entonces simplificarse aún más:

```
$ raku -ne '.say if .chars > 20' lemario.txt
```

Recuerda que, si está intentando esto en Windows, necesita reemplazar las comillas simples con las comillas dobles (y viceversa si el script contiene comillas dobles):

```
C:\Users\Laurent>raku -ne ".say if .chars > 20" lemario.txt
```

8.4.2 Palabras Sin “e” (Solución)

Una subrutina que devuelve True para palabras que no tienen “e” (Ejercicio 8.2) es igualmente simple:

```
sub sin-e( Str $palabra ) {
    return True unless defined index $palabra, "e";
    return False;
}
```

La subrutina simplemente devuelve True si la función `index` no encontró una “e” en la palabra recibida como un parámetro y por lo contrario, devuelve False.

Nota que esto funciona correctamente porque la lista de palabras usada (*lemario.txt*) está completamente en letras minúsculas. La subrutina más arriba necesitaría ser modificada si fuera a ser llamada con palabras que contienen letras mayúsculas.

Dado que la función `defined` devuelve un valor Booleano, podríamos simplificar nuestra subrutina:

```
sub sin-e( Str $palabra ) {
    not defined index $palabra, "e";
}
```

Podríamos también haber usado un regex para comprobar la presencia de una “e” en la segunda línea de esta subrutina:

```
return True unless $palabra =~ /e/;
```

Esta sintaxis bastante concisa es atractiva, pero cuando se busca por una coincidencia literal exacta, la función `index` es probablemente más eficiente (más rápida) que un regex.

Buscar y contar las palabras sin “e” en nuestra lista de palabras no es muy difícil:

```
sub sin-e( Str $palabra ) {
    not defined index $palabra, "e";
}

my $cuenta-total = 0;
my $cuenta-sin-e = 0;
for 'lemario.txt'.IO.lines -> $línea {
    $cuenta-total++;
    if sin-e $línea {
        $cuenta-sin-e++;
        say $línea;
    }
}

say "=" x 24;
say "Cuenta total de palabras: $cuenta-total";
say "Palabras sin 'e': $cuenta-sin-e";
printf "Porcentaje de palabras sin 'e': %.2f %%\n",
    100 * $cuenta-sin-e / $cuenta-total;
```

El programa anterior mostrará lo siguiente al final de su salida:

```
=====
Cuenta total de palabras: 87899
Palabras sin 'e': 36671
Porcentaje de palabras sin 'e': 41.72 %
```

Así que más de un tercio de las palabras en nuestra lista no tienen “e”.

8.4.3 Evitar Otras Letras (Solución)

La subrutina evitar es una versión más general de sin-e, pero tiene la misma estructura:

```
sub evitar( Str $palabra, Str $prohibida ) {
  for 0..$prohibida.chars - 1 -> $idx {
    my $letra = substr $prohibida, $idx, 1;
    return False if defined index $palabra, $letra;
  }
  True;
}
```

Podemos devolver False tan pronto encontremos una letra prohibida; si llegamos al final del bucle, devolvemos True. Debido a que una subrutina devuelve la última expresión evaluada, no necesitamos usar la sentencia return True explícitamente en la última línea de código más arriba. He usado esta característica como un ejemplo; podrías encontrarlo más claro devolver los valores explícitamente, excepto quizás con subrutinas muy simples de una línea.

Nota que tenemos dos bucles anidados implícitamente. Podríamos invertir los bucles externo e interno:

```
sub evitar( Str $palabra, Str $prohibida ) {
  for 0..$palabra.chars - 1 -> $idx {
    my $letra = substr $palabra, $idx, 1;
    return False if defined index $prohibida, $letra;
  }
  True;
}
```

El código principal que llama la subrutina más arriba es similar al código que llama a sin-e y podría lucir así:

```
my $cuenta-total = 0;
my $cuenta-no-prohibidas = 0;

for 'lemario.txt'.IO.lines -> $línea {
  $cuenta-total++;
  $cuenta-no-prohibidas++ if evitar $línea, "eiou";
}

say "=" x 24;
```

```
say "Cuenta total de palabras: $cuenta-total";
say "Palabras sin letras prohibidas: $cuenta-no-prohibidas";
printf "Porcentaje de palabras sin letras prohibidas: %.2f %%\n",
      100 * $cuenta-no-prohibidas / $cuenta-total;
```

8.4.4 Usando Solo Algunas Letras (Solución)

`usa-solamente` es similar a nuestra subrutina `evitar` excepto que el sentido de la condición está invertido:

```
sub usa-solamente( Str $palabra, Str $disponible ) {
  for 0..$palabra.chars - 1 -> $idx {
    my $letra = substr $palabra, $idx, 1;
    return False unless defined index $disponible, $letra;
  }
  True;
}
```

En lugar de una lista de letras prohibidas, tenemos una lista de letras disponibles. Si encontramos una letra en `$palabra` que no está en `$disponible`, podemos devolver `False`. Y devolvemos `True` si llegamos al final del bucle.

8.4.5 Usando Todas las Letras de una Lista (Solución)

`usa-todas` es similar a las subrutinas previas, excepto que invertimos el rol de la palabra y la cadena de letras:

```
sub usa-todas( Str $palabra, Str $requerida ) {
  for 0..$requerida.chars - 1 -> $idx {
    my $letra = substr $palabra, $idx, 1;
    return False unless defined index $palabra, $letra;
  }
  return True;
}
```

En lugar de recorrer las letras en `$palabra`, el bucle recorre las letras requeridas. Si cualquiera de las letras requeridas no aparece en la palabra, podemos devolver `False`.

Si estabas realmente pensando como un científico de la computación, habrías reconocido que `usa-todas` era una instancia de un problema previamente resuelto en inverso: si la palabra A usa todas las letras de la palabra B, entonces la palabra B usa solo las letras de la palabra A. Así que podemos llamar la subrutina `usa-solamente` y escribir:

```
sub usa-toda( $palabra, $requerida ) {
  return usa-solamente $requerida, $palabra;
}
```

Este es un ejemplo de un plan de desarrollo de programa conocido como la **reducción a un problema previamente solucionado**, lo cual significa que reconoces el problema con el cual estás trabajando como una instancia de un problema ya solucionado y aplica una solución ya existente.

8.4.6 Orden Alfabético (Solución)

Para `es_abecedaria` tenemos que comparar las letras adyacentes. Cada vez en nuestro bucle `for`, definimos una letra como nuestra letra actual y la comparamos con la anterior:

```
sub es_abecedaria( $palabra ) {
  for 1..$palabra.chars - 1 -> $idx {
    my $letra-actual = substr $palabra, $idx, 1;
    return False if $letra-actual lt substr $palabra, $idx - 1, 1;
  }
  return True
}
```

Una alternativa es usar recursión:

```
sub es-abecedaria( Str $palabra ) {
  return True if $palabra.chars <= 1;
  return False if substr($palabra, 0, 1) gt substr($palabra, 1, 1);
  return es-abecedaria substr $palabra, 1;
}
```

Otra opción es usar bucle `while`:

```
sub palabra( Str $palabra ) {
  my $i = 0;
  while $i < $palabra.chars - 1 {
    if substr($palabra, $i, 1) gt substr ($palabra, $i+1, 1) {
      return False;
    }
    $i++;
  }
  return True;
}
```

El bucle comienza con `$i=0` y termina con `i=$palabra.chars - 1`. Cada vez a través del bucle, se compara el i .º carácter (el cual puedes considerar como el carácter actual) al carácter $(i + 1)$.º (el cual puedes considerar como el carácter siguiente).

Si el carácter siguiente es menor (alfabéticamente posterior) que el carácter actual, entonces hemos descubierto una falla en la tendencia del abecedario, y devolvemos `False`.

Si llegamos al final del bucle sin encontrar una falla, entonces la palabra pasa la prueba. Para convencerte que el bucle finaliza correctamente, considera un ejemplo como `'flossy'`. La longitud de la palabra es 6, así que la última vez que el bucle se ejecuta es cuando `$i` es 4, el cual es el índice del penúltimo carácter. En la última iteración, se compara el penúltimo carácter (la segunda “s”) al último (la “y”), que es lo que deseamos.

8.4.7 Otro Ejemplo de Reducción a un Problema Previamente Solucionado

Aquí tenemos una versión de `es_palindromo` (ver Ejercicio 5.3) que usa dos índices; uno comienza al inicio e incrementa, mientras que el otro comienza al final y disminuye:


```

sub un-car( Str $cadena, $idx ) {
    return substr $cadena, $idx, 1;
}
sub es-palindromo( Str $palabra ) {
    my $i = 0;
    my $j = $palabra.chars - 1;

    while $i < $j {
        return False if un-car($palabra, $i) ne
                        un-car($palabra, $j);

        $i++;
        $j--;
    }
    return True;
}

```

O podríamos reducirlo a un problema previamente solucionado y escribir:

```

sub es-palindromo( Str $palabra ) {
    return es-inversa($palabra, $palabra)
}

```

usando `es-inversa` de la Sección 7.10 (pero deberías probablemente elegir la versión corregida de la subrutina `es-inversa` dada en el apéndice: ver Solución: A.5.6).

8.5 Depuración de Programas

Hacer pruebas manuales de los programas es difícil. Las funciones en este capítulo son relativamente fáciles para probar porque puedes chequear el resultado manualmente. Aún así, es algo entre difícil e imposible elegir un conjunto de palabras que puedan satisfacer todos los errores posibles.

Por ejemplo, tomemos el caso de la subrutina `sin-e`. En esta función, hay dos casos que debemos chequear: palabras que tienen una “e” deberían devolver `False`, y palabras que no tienen ninguna “e” deberían devolver `True`. Esto no representa problema alguno debido a que es fácil encontrar palabras que representan cada caso.

Dentro de cada caso, existen casos que son menos obvios. Entre las palabras que tienen una “e”, deberías probar palabras con una “e” al inicio, al final, y en el medio. Deberías probar con palabras largas, palabras cortas y con palabras muy cortas, como una cadena de texto vacía. La cadena de texto vacía es un ejemplo de un **caso especial**, el cual es uno de los casos que no son obvios donde los errores usualmente merodean.

Además de las pruebas de casos que generes, puedes también someter tu programa a una prueba con una lista de palabras como *lemario.txt*. Al escanear la salida, podrías atrapar errores, pero debes ser cuidadoso: podrías atrapar un tipo de error (palabras que no deberían ser incluidas, pero están incluidas) y dejar pasar otros errores (palabras que deberían ser incluidas pero no están incluidas).

En general, las pruebas pueden ayudar a encontrar errores, pero no es fácil generar un buen conjunto de pruebas de casos, y aún si lo haces, no puedes estar seguro que tu programa es correcto.

Según un legendaro científico de la computación:

La depuración de programas puede usarse para mostrar la presencia de errores, pero jamás para mostrar la ausencia de los mismos³.

— Edsger W. Dijkstra

8.6 Glosario

Objeto de archivo Un valor que representa un archivo abierto.

Reducción a un problema previamente solucionado Una manera de solucionar un problema al expresarlo como una instancia de un problema que fue solucionado previamente.

Caso especial Un caso de prueba que es atípico o que no es evidente (y con pocas posibilidades de ser manejado correctamente). Los términos *edge case* y *corner case* expresan más o menos la misma idea.

8.7 Ejercicios

Ejercicio 8.7. Este problema está basada en un Puzzler (acertijo) que fue transmitido en el programa radial Car Talk (<http://www.cartalk.com/content/puzzlers>):

*Dame una palabra con tres letras dobles consecutivas. Te daré unas cuantas palabras que casi califican, pero no completamente. Por ejemplo, la palabra *committee*, c-o-m-m-i-t-t-e-e. Sería un gran ejemplo excepto por la "i" que se escabulle ahí. O Mississippi: M-i-s-s-i-s-s-i-p-p-i. Si pudieras sacar esas i, sería otro gran ejemplo. No obstante, existe una palabra que tiene tres pares de letras consecutivas y a mi saber, esta puede ser la única palabra que satisface dicha condición. ¡Solo bromeo! Existen probablemente alrededor de 500 de estas palabras pero solo puede pensar sobre una. ¿Cuál es la palabra?*

Escribe un programa para encontrarla.

Solución: A.6.1.

Ejercicio 8.8. Aquí presentamos otro Puzzler de Car Talk (<http://www.cartalk.com/content/puzzlers>):

"Estaba conduciendo en la autopista el otro día y sucede que noto mi odómetro. Como la mayoría de odómetros, este muestra seis dígitos, en millas en números enteros solamente. Así que, si mi carro tenía 300,000 millas, por ejemplo, yo vería 3-0-0-0-0-0.

Ahora bien, lo que ví ese día fue algo muy interesante. Noté que los últimos cuatro dígitos eran palíndromos; es decir, se pueden leer igual de atrás hacia adelante. Por ejemplo, 5-4-4-5 es un palíndromo, así que mi odómetro podría leerse 3-1-5-4-4-5.

Una milla más tarde, los últimos 5 números eran palíndromos. Por ejemplo, podría leerse 3-6-5-4-5-6. Una milla después de esa, los 4 números del medio eran palíndromos. ¿Y estás listo para esto? ¡Otra milla más tarde, todos los 6 números eran palíndromos!"

"La pregunta es, qué había en el odómetro la primera vez que lo miré?"

³"Program testing can be used to show the presence of bugs, but never to show their absence!" — Edsger W. Dijkstra

Escribe un programa que chequee todos los números de seis dígitos e imprime cualquier número que satisfice estos requerimientos.

Solución: A.6.2.

Ejercicio 8.9. Otro Puzzler de Car Talk que puedes solucionar con una búsqueda (<http://www.cartalk.com/content/puzzlers>):

Recientemente visité a mi madre y me di cuenta que los dos dígitos que componen mi edad al ser invertidos resultan en su edad. Por ejemplo, si ella tiene 73 años, entonces yo tengo 37. Nos preguntamos que tan a menudo esto había ocurrido a través de los años pero nos distrajimos con otros temas y jamás obtuvimos una respuesta.

Cuando llegué a la casa fui capaz de deducir que los dígitos de nuestras edades habían sido reversibles seis veces hasta ahora. También deducí que si tenemos suerte pasaría nuevamente dentro de algunos años, y si somos bien suertudos ocurriría una vez más después de eso. En otras palabras, pasaría 8 veces en total. Así que la pregunta es, cuál es mi edad ahora?

Escribe un programa de Raku que busca las soluciones para este Puzzler. Pista: podrías encontrar el método para el formateo de cadenas de texto `sprintf` muy útil.

Solución: A.6.3.

Arrays y Listas

Este capítulo presenta algunos de los tipos integrados de Perl más útiles, los arrays y las listas.

9.1 Las Listas y los Arrays Son Secuencias

Al igual que las cadenas de texto, las **listas** y los **arrays** son secuencias de valores. En una cadena de texto, los valores son caracteres; en una lista o en un array, los valores pueden ser de cualquier tipo. Los valores en una lista o en un array se conocen como **elementos** o algunas veces como **artículos**.

Existen varias diferencias importantes entre las listas y los arrays. La principal de ellas es que las listas son colecciones ordenadas e inmutables de elementos: no puedes cambiar el número de elementos en una lista y ni tampoco puedes cambiar los elementos individuales. Los arrays, por el contrario, son variables y son generalmente mutables: puedes agregar elementos a un array, o quizás removerlos. Y también puedes tener acceso a los elementos individuales de un array y modificarlos. Para que esto sea posible, los arrays usualmente tienen un nombre (como cualquier otra variable) aunque algunos arrays son anónimos, lo que significa que no tienen un nombre, pero aún tienen otras formas de acceso.

Una lista es también efímera (al menos que sea asignada a una variable o algo más): deja de existir tan pronto ha sido usada, usualmente tan pronto el flujo de ejecución del programa procede a la siguiente línea de código. Por el contrario, un array tiene alguna forma de persistencia: puedes usarlo en otras partes del programa si la variable que lo contiene aún está dentro del ámbito.

Hay varias maneras de crear una lista nueva; la forma más simple es enumerar sus valores, separados por comas:

```
> 3, 4, 5
(3 4 5)
> say (3, 4, 5).WHAT;
```

```
(List)
> say $_ for 1, 2, 3;
1
2
3
```

No necesitas usar paréntesis para crear una lista, pero son usualmente útiles para delimitar los valores, por ejemplo, para estipular dónde comienza y dónde termina, y en algunos casos, anular la precedencia.

Ya hemos usado listas en este libro. Si escribimos:

```
> print "$_ " for 1, 3, 5, 9, "\n";
1 3 5 9
>
> print "$_ " for 1..10;
1 2 3 4 5 6 7 8 9 10 >
```

estamos básicamente creando y usando una lista de números enteros (desde el punto de vista de la jerarquía de tipos en Perl; esta observación no es totalmente correcta para el segundo ejemplo, dado que `1..10` es del tipo *Range* y es transformado en el tipo *Seq*. Sin embargo, esta aproximación es suficientemente buena para nuestros propósitos aquí).

Los arrays son variables cuyos nombres comienzan con el sigilo `@`. Los arrays con nombres necesitan ser declarados antes de ser usados, como cualquier otra variable que hemos visto hasta ahora (excepto la variable tónica, `$_`). Una de las formas más fáciles de crear un array es asignar una lista a una variable:

```
> my @dígitos_impares = 1, 3, 5, 7, 9;
[1 3 5 7 9]
> say @dígitos_impares.WHAT;
(Array)
> my @números_solo_dígito = 0..9;
[0 1 2 3 4 5 6 7 8 9]
```

En el REPL de Perl, un array se muestra entre corchetes (`[` y `]`), mientras que las listas se muestran entre paréntesis.

Si los elementos no contienen caracteres de espacio, es bien útil construir una lista (y asignarla a cualquier array si es necesario) con el operador `quote-word` `<...>`:

```
> my @días-semana = <lun mar mie jue vie>;
[lun mar mie jue vi]
> my @fin-semana = <sab dom>;
[sab dom]
```

La ventaja de este método es que no hay necesidad de separar los elementos con comas y ni tampoco hay necesidad de insertar comillas cuando los elementos son cadenas de texto. Básicamente, el operador `quote-word` descompone su contenido por espacios en blanco y devuelve una lista de palabras, la cual puede ser entonces usada en un bucle o asignada a un array como en el ejemplo anterior.

La mayoría de este capítulo será dedicado a los arrays. No obstante, ten presente que muchas de las funciones y operadores de los arrays que estudiaremos aquí también funcionan con las listas (por lo menos todas aquellas que no violan la propiedad de inmutabilidad de las listas).

Los elementos de un array (o una lista) no necesitan ser del mismo tipo:

```
> my @array-heterogeneo = 1, 2.3, pi, "str", (1, 2, 4);  
[1 2.3 3.14159265358979 str (1 2 4)]
```

Aquí, el array está compuesto por un entero, un racional, un número con coma flotante (tipo Num), una cadena de texto, y una lista de tres enteros. Aunque esto no se recomendaría por la sanidad mental del programador quien tendría que usar un array con elementos tan heterogéneos, Perl no se queja: al final, depende de ti dar sentido a tus datos.

El array anterior hasta contiene una lista de artículos. Si iteras sobre los elementos de este array, por ejemplo con el bucle `for`, esta lista terminará como un elemento distinto; este elemento no será “aplanado” como tres elementos del array. Similarmente, `elems` es un método que cuenta el número de artículos de un array (o de una lista). Al usarlo en el array anterior produce el siguiente resultado:

```
> say @array-heterogeneo.elems;  
5
```

Como puedes observar, la lista (1, 2, 4) cuenta como un solo elemento en el array.

Una lista dentro de otra es una lista **anidada**.

Un array que carece de elementos se conoce como un array vacío; puedes crear un array vacío con paréntesis vacíos, `()`:

```
> my @vacío = ();  
[]
```

Este código está realmente asignando una lista vacía al array. Pero esta sintaxis no es normalmente necesaria para crear un nuevo array vacío, dado que solo con declarar un array sin definirlo tiene el mismo efecto:

```
> my @vacío;  
[]
```

Así que el uso de paréntesis vacíos (o sea, en la asignación de una lista vacía) sería necesario para restaurar un array existente a un array vacío.

9.2 Los Arrays Son Mutables

La sintaxis para acceder a los elementos de un array o una lista usa el operador de corchetes. La expresión dentro de los corchetes especifica el índice o subíndice, el cual puede ser un entero literal (o algún valor que puede ser coaccionado en un entero), una variable que contiene un valor numérico, una lista o un rango de valores numéricos, una expresión o una pieza de código que devuelve un valor numérico, etc. Comparado con el comienzo de una array o una lista (de la misma manera que los valores devueltos por la función `index`), los índices comienzan en la posición 0. Así que, el primer elemento de un array tiene índice 0, el segundo tiene índice 1, etc. Por ejemplo:

```
say <sab dom>[1];           # -> sab (accediendo un elemento de la lista)
my @días-semana = <lun mar mie jue vie>;   # asignando un array
say "El tercer día es @días-semana[2]";    # -> El tercer día es mie
```

También puedes usar rangos o listas de índices para acceder partes (o *rebanadas*) de un array o una lista:

```
> my @dígitos-pares = 0, 2, 4, 6, 8;
[0 2 4 6 8]
> my @pequeños-dígitos_pares = @dígitos-pares[0..2];
[0 2 4]
> my @min-max-dígitos-pares = @dígitos-pares[0, 4]
[0 8]
```

Si necesitas rebanadas en el orden opuesto, puedes usar la función `reverse` para revertir el rango:

```
> my @opuestos-pequeños-dígitos_pares = @dígitos-pares[reverse 0..2];
[4 2 0]
```

o también puedes revertir los datos que resultan de la expresión de rebanadas:

```
> my @opuestos-pequeños-dígitos_pares = reverse @dígitos-pares[0..2];
[4 2 0]
```

A diferencia de las listas, los arrays son mutables. Cuando el operador de corchetes aparece después de un array en el lado izquierdo de una asignación, el operador identifica el elemento del array que será asignado:

```
> my @dígitos-pares = 0, 2, 2, 6, 8;   # Oops, error en el segundo 2
[0 2 2 6 8]
> @dígitos-pares[2] = 4; # arreglando el tercer elemento
4
> say @dígitos-pares
[0 2 4 6 8]
```

El tercer elemento de `dígitos-pares`, el cual es (por error) 2, es ahora 4. Si el índice corresponde a un artículo que no existe aún en el array, el array se expandirá para incluir el nuevo elemento:

```
> my @impares = 1, 3, 5;
[1 3 5]
> @impares[3] = 7;
7
> say @impares;
[1 3 5 7]
```

La función o método `elems` devuelve el número de elementos de un array. La función o método `end` devuelve el índice del último elemento de un array:

```
my @nums = 1..5;      # -> [1 2 3 4 5]
say @nums.elems;      # -> 5
say elems @nums;      # -> 5
say @nums.end;        # -> 4
```

El método `end` devuelve el resultado del método `elems` menos uno porque, dado que los índices comienzan en la posición 0, el índice del último elemento es igual al número de elementos menos uno.

La función o método `unique` devuelve una secuencia de elementos únicos en la lista de entrada o el array (es decir, el método devuelve la lista original si no hay valores duplicados):

```
> say < a b d c a f d g>.unique;
(a b d c f g)
```

Si sabes que la entrada está ordenada (y que, por lo tanto, los duplicados son adyacentes), en vez de usar la función `unique`, usa la función `squish` debido a que termina siendo más eficiente. La función `squish` remueve los elementos duplicados y adyacentes.

Para saber si dos arrays son idénticos (estructuralmente los mismos, con el mismo tipo y los mismos valores), usa el operador de equivalencia `eqv`. Para saber si los arrays contienen los mismos elementos, usa el operador de coincidencia inteligente `~~`. Entre dos arrays o listas, el operador de equidad numérica `==` devolverá `True` si los arrays tienen el mismo número de elementos y de lo contrario, `False`. Esto se debe a que `==` coacciona sus argumentos a tipo numérico, así que el mismo compara el número de elementos:

```
> my @pares1 = 0, 2, 4, 6, 8;
[0 2 4 6 8]
> my @pares2 = reverse 8, 6, 4, 2, 0;
[0 2 4 6 8]
> say @pares1 eqv @pares2;          # mismos elementos, misma estructura
True
> say <1 2 3 4 5> eqv 1..5;          # mismos elementos, estructura diferente
False
> say <1 2 3 4 5> ~~ 1..5;           # mismos elementos, True
True
> my @array = 1..5;
[1 2 3 4 5]
> say <1 2 3 4 5> ~~ @array;         # mismos elementos, True
True
> say <1 2 3 4 6> ~~ @array;         # no los mismos elementos
False
> say <1 2 3 4 5> == <5 6 7 8 9>;    # compara el número de elementos
True
```

La sentencia `<1 2 3 4 5> eqv 1..5` devuelve `False`, debido a que aunque contienen los mismos elementos, los argumentos son estructuralmente entidades diferentes (uno es una lista y el otro es un rango).

9.3 Cómo Agregar o Remover Elementos de un Array

Hemos visto que la asignación de un elemento a un índice que no existe expande el array. Existen otras formas de expandir un array.

Perl tiene operadores para agregar elementos a un array, o remover un elemento del mismo:

- `shift`: elimina el primer elemento del array y lo devuelve;
- `pop`: elimina el último elemento del array y lo devuelve;
- `unshift`: agrega un elemento o una lista de elementos al principio del array;
- `push`: agrega un elemento o una lista de elementos al final del array;

Estos son algunos ejemplos sobre cada uno de ellos:

```
> my @números = <2 4 6 7>;
[2 4 6 7]
> push @números, 8, 9;
[2 4 6 7 8 9]
> unshift @números, 0, 1;
[0 1 2 4 6 7 8 9]
> my $num = shift @números
0
> $num = pop @números
9
> say @números
[1 2 4 6 7 8]
```

Como podrías esperar por ahora, estas subrutinas también tienen una sintaxis de invocación de método. Por ejemplo:

```
> my @números = <2 4 6 7>;
[2 4 6 7]
> @números.push(8, 9)
[2 4 6 7 8 9]
```

No obstante, debes tener presente que si usas las funciones `push` o `unshift` con un array como argumento, obtendrás algo diferente a lo que podrías esperar:

```
> my @números = <2 4 6 7>;
[2 4 6 7]
> my @agregar-array = 8, 10;
[8 10]
> @números.push(@agregar-array);
[2 4 6 7 [8 10]]
```

Como puedes observar, cuando `@agregar-array` se agrega como una entidad al array `@números`, `@agregar-array` se convierte en un elemento nuevo del array original. Si lo que quieres es agregar los elementos de `@agregar-array` al array original, puedes usar el método `append` en vez de `push`:

```
> my @números = <2 4 6 7>;
[2 4 6 7]
> @número.append(@agregar-array);
[2 4 6 7 8 10]
```

O puedes usar el operador prefijo `"|"`, el cual aplanar el array añadido a una lista de argumentos:

```
> my @números = <2 4 6 7>;
[2 4 6 7]
> @números.push(|@agregar-array);
[2 4 6 7 8 10]
```

También existe el método `prepend` que reemplaza a `unshift` para agregar elementos individuales de un array al principio de otro array existente (en vez de añadir el array como una sola identidad).

9.4 Pilas y Colas

Las pilas (**en.** *stacks*) y las colas (**en.** *queues*) son estructuras de datos muy usadas en la ciencia de la computación.

Una pila es una estructura de datos de tipo LIFO (del inglés *Last In / First Out*, “último en entrar, primero en salir”). Puedes pensar sobre una pila como un grupo de platos apilados. Cuando pones un plato limpio en la pila, usualmente lo pones en la parte superior; de igual manera cuando tomas un plato, lo tomas de la parte superior. Así que el primer plato que tomas fue el último en ser añadido. Una pila en la ciencia de la computación implementa la misma idea: usas una pila cuando la primera pieza de dato que necesitas de una estructura de datos fue la última en ser añadida.

Por el contrario, una cola (o fila) es una estructura de datos de tipo FIFO (del inglés *First In / First Out*, “primero en entrar, primero en salir”). Esta es la misma idea de un grupo de personas esperando en línea para pagar en el supermercado. La primera persona en ser atendida es la primera persona que entró en la fila.

Una pila puede ser implementada con un array y las funciones `push` y `pop`, las cuales agregan un elemento (o varios) al final de un array y remueven un elemento del final del array respectivamente. Esta es una implementación bien simple de una pila:

```
sub poner-en-pila( @pila, $new_elem ) {
    push @pila, $new_elem;
}
sub tomar-desde-pila( @pila ) {
    my $elem = pop @pila;
    return $elem;
}
my @una-pila = 1, 2, 3, 4, 5;
poner-en-pila @una-pila, 6;
say @una-pila;
say tomar-desde-pila @una-pila for 1..3;
```

Este ejemplo imprimirá esto:

```
[1 2 3 4 5 6]
6
5
4
```

Esta pila es simplista porque, por lo menos, una implementación más robusta debería hacer algo sensible cuando tratas de tomar-desde-pila cuando la pila está vacía. También sería sabio agregar signaturas a las subrutinas. En adición, podrías querer poner-en-pila más de un elemento al mismo tiempo. Échale un vistazo a la solución del ejercicio sobre colas más abajo (Solución: A.7.1) para descifrar cómo esta pila podría ser mejorada.

Podrías obtener las mismas funciones usando las subrutinas `unshift` y `shift` en vez de `push` y `pop`. Los elementos serán añadidos al principio del array y tomados desde el inicio, pero todavía tendrás el comportamiento LIFO.

Como un ejercicio, trata de implementar una cola de tipo FIFO en el mismo modelo. Pista: probablemente querrás usar un array y las funciones `unshift` y `pop` (o las funciones `push` y `shift`). Solución: A.7.1.

9.5 Otras Formas de Modificar un Array

Las funciones `shift` y `pop` eliminan el primer y el último elemento de un array respectivamente y devuelven dicho elemento. Es posible casi realizar la misma operación con cualquier elemento de un array, usando el *adverbio* `delete`:

```
my @frutas = <manzana banana fresa mango piña naranja>;
my $removida = @frutas[2]:delete;
say $removida;    # -> fresa
say @frutas;      # -> [<manzana banana (Any) mango piña naranja>]
```

Nota que el tercer elemento (“fresa”) ha sido removido y devuelto, pero el array no ha sido reorganizado; la operación deja un tipo de “hueco”, un artículo indefinido, en el medio del array. La sintaxis de los dos puntos (“:”) que se usa aquí es el operador para un adverbio (discutimos los adverbios en la Sección 7.5 sobre regexes); por el tiempo presente, puedes imaginarte este operador como un método especial que opera sobre un elemento de una colección de elementos.

Hemos visto cómo rebanar un array para tomar varios elementos de un array o una lista al mismo tiempo. La misma sintaxis de rebanada puede usarse en el lado izquierdo de una asignación para modificar algunos elementos de un array:

```
my @dígitos = <1 2 3 6 5 4 7 8 9>
@dígitos[2..4] = 4, 5, 6
say @dígitos;    # -> [1 2 4 5 6 4 7 8 9]
```

Por supuesto no puedes hacer esto con listas, debido a que las listas son inmutables.

La función `splice` puede ser considerada la Navaja Suiza de los arrays. Esta función puede añadir, remover, y devolver varios elementos de un array. La sintaxis general se presenta a continuación:

```
my @array_salida = splice @array_entrada, $inicio,
                        $numero_elems, @reemplazo;
```

Los argumentos para `splice` son el array de entrada, el índice del primer elemento sobre el cual se harán los cambios, el número de elementos afectados por la operación, y una lista de

reemplazos para los elementos a ser removidos¹. Por ejemplo, para realizar la asignación de rebanada mostrada más arriba, es posible hacer esto:

```
my @dígitos = <1 2 3 6 5 4 7 8 9>
my @dígitos_removidos = splice @dígitos, 3, 3, 4, 5, 6;
say @dígitos_removidos;    # -> [6 5 4]
say @dígitos;              # -> [1 2 3 4 5 6 7 8 9]
```

Aquí, la sentencia `splice` removió tres elementos (6, 5, 4) y los reemplazó con los reemplazos en los argumentos (4, 5, 6). También devolvió los artículos removidos, los cuales fueron almacenados en `@dígitos_removidos`. El número de reemplazos no necesita ser igual al número de elementos removidos. En tal caso, el array crecerá o se reducirá. Por ejemplo, si no se proveen reemplazos, entonces `splice` removerá y devolverá el número requerido de elementos y el array se reducirá por el mismo número:

```
my @dígitos = 1..9;
my @dígitos_removidos = splice @dígitos, 3, 2;
say @dígitos_removidos;    # -> [4 5]
say @dígitos;              # -> [1 2 3 6 7 8 9]
```

Por el contrario, si el número de elementos a removerse es cero, no se removerá ningún elemento, un array vacío será devuelto, y los elementos en la lista de reemplazos serán añadidos en el lugar correcto:

```
my @dígitos = <1 2 3 6 4 7 8 9>;
my @dígitos_removidos = splice @dígitos, 3, 0, 42;
say @dígitos_removidos;    # -> []
say @dígitos;              # -> [1 2 3 42 6 4 7 8 9]
```

Assumiendo que la función `shift` no existiera en Perl, podrías escribir una subrutina `mi-shift` para simularla:

```
sub mi-shift( @array ) {
    my @resultado = splice @array, 0, 1;
    return @resultado[0];
}
my @letras = 'a'..'j';
my $letra = mi-shift @letras;
say $letra;          # -> a
say @letras;         # -> [b c d e f g h i j]
```

Podrías levantar una excepción si el array que se pasa a la función `mi-shift` está vacío. Esto podría hacerse al modificar la subrutina de la siguiente manera:

```
sub mi-shift( @array ) {
    die "No se puede remover elemento de una array vacío" unless @array;
    my @resultado = splice @array, 0, 1;
    return @resultado[0];
}
```

¹Nota que la función `splice` en los arrays tiene casi la misma sintaxis que la función `substr` en cadenas de texto. Esto hará más fácil entender y recordar la sintaxis.

o al añadir una restricción al array en la signatura de la subrutina:

```
sub mi-shift( @array where @array > 0 ) {
  my @resultado = splice @array, 0, 1;
  return @resultado[0];
}
```

La expresión `@array > 0` evalúa a `True` si el número de elementos del array es mayor que 0, es decir, si no es un array vacío. Es equivalente a la expresión `@array.elems > 0`.

Como un ejercicio, escribe subrutinas usando `splice` para simular las funciones integradas `pop`, `unshift`, `push`, y `delete`. Solución: A.7.2.

9.6 Cómo Recorrer una Lista

La manera más común de recorrer los elementos de una lista o un array es con el bucle `for`. La sintaxis para un array es la misma que hemos usado en los capítulos anteriores para las listas:

```
my @colores-ingles = <red orange yellow green blue indigo violet>;
for @colores-ingles -> $color {
  say $color;
}
```

Esto funciona bien si solo necesitas leer los elementos de la lista. Pero si quieres escribir o actualizar los elementos de un array, tú necesitas un bloque de doble punta. Por ejemplo, podrías usar la función `tc` (**en.** title case) para capitalizar la primera letra de cada palabra de un array:

```
my @colores-ingles = <red orange yellow green blue indigo violet>;
for @colores-ingles <-> $color {
  $color = tc $color
}
say @colores-ingles; #-> [Red Orange Yellow Green Blue Indigo Violet]
```

Aquí la variable `$color` del bucle es un *apodo* de lectura-escritura (**en.** read-write) de los elementos del array, así que cualquier cambio que se hace a este apodo se refleja en el array. Esto funciona bien con los arrays, pero no funcionaría con las listas, debido a que las mismas son inmutables. Obtendrías el siguiente error al usarlo con una lista:

```
> for <red orange yellow> <-> $color { $color = tc $color }
Parameter '$color' expected a writable container, but got Str value...
```

También puedes usar la sintaxis de un bucle `for` con la variable tónica `$_`. Por ejemplo, esto usa la función `uc` (**en.** upper case) para capitalizar cada palabra del array anterior:

```
for @colores-ingles {
  $_ = $_.uc
}
say @colores-ingles; # -> [RED ORANGE YELLOW GREEN BLUE INDIGO VIOLET]
```

Algunas veces quieres recorrer un array y necesitas saber el índice de los elementos que estás visitando. Una forma común de hacer esto es con el operador de rango `..` para iterar sobre los índices. Por ejemplo, para imprimir el índice y el valor de cada elemento de un array:

```
for 0..@colores-inglés.end -> $idx {
  say "$idx @colores-inglés[$idx]";
}
```

Esto es útil, por ejemplo, para recorrer dos (o más) arrays en paralelo:

```
my @letras = 'a'..'e';
my @números = 1..5;
for 0..@letras.end -> $idx {
  say "@letras[$idx] -> @números[$idx]";
}
```

Esto imprimirá:

```
a -> 1
b -> 2
c -> 3
d -> 4
e -> 5
```

No necesitas especificar el rango de índices tú mismo, dado que la función `keys` devolverá una lista de índices para un array o una lista:

```
for keys @colores-inglés -> $idx {
  say "$idx @colores-inglés[$idx]";
}
```

Otra forma de iterar sobre los índices y los valores de un array es con el uso de la función o método `kv` ("keys values"-"llaves valores") que devuelve el índice y el valor de cada elemento del array:

```
for @letras.kv -> $idx, $val {
  say "$idx $val";
}
```

En un contexto de lista, `@letters.kv` simplemente devuelve un secuencia entrelazada de índices y valores:

```
my @letras = 'a'..'e';
say @letras.kv;      # -> (0 a 1 b 2 c 3 d 4 e)
```

El bloque de doble punta con las variables de iteración hace posible procesar un índice y un valor a cada paso del bucle. Por supuesto, puedes tener más de dos variables de iteración si es necesario.

9.7 Nuevas Construcciones de Repetición

Dado que el tema de este capítulo son los arrays y las listas, es probablemente el tiempo perfecto para estudiar brevemente dos construcciones de repetición que hemos dejado a un lado hasta ahora.

La primera usa la misma palabra clave `for` usada anteriormente, pero con una sintaxis diferente para la iteración de variable(s):

```
my @letras = 'a'..'e';
for @letras {
    say $_-letra;
}
```

El símbolo `~` en la variable `$_-letra` es conocido como un *twigil*, es decir, un tipo de sigilo secundario. Cuando hay un twigil, el primer símbolo (en este caso, el signo `$`) posee el mismo significado de los sigilos usuales (aquí, denota una variable escalar) y el segundo (aquí, `~`) extiende la descripción de la variable y usualmente modifica su ámbito. En este caso específico, el segundo carácter especifica que la variable `$_-letra` es un *parámetro marcador* o un *parámetro auto-declarado de posición*. Es decir, es un parámetro de posición del bloque actual que no necesita ser declarado en la signature.

Si el bloque usa más de un marcador, ellos son asociados con la entrada de datos de acuerdo a su orden lexicográfico (alfabético):

```
my @letras = 'a'..'e';
for @letras.kv {
    say "$_a -> $_b";
}
```

Esto imprimirá:

```
0 -> a
1 -> b
2 -> c
3 -> d
4 -> e
```

Como vimos anteriormente, la función `kv` devuelve una secuencia entrelazada de índices y valores. Dado que `$_a` viene antes que `$_b` en el orden alfabético, `$_a` será atada al índice y `$_b` al valor de cada pareja de la entrada de datos.

Los marcadores pueden también ser usados en las subrutinas:

```
> sub dividir { $_primero / $_segundo }
sub dividir( $primero, $segundo ) { #^(Sub|230787048) ... }
> dividir 6, 4
1.5
```

Estos marcadores no se usan muy a menudo para recorrer los arrays, pero veremos más adelante lo útiles que son en casos donde no sería muy práctico tener que declarar los parámetros.

La segunda construcción de repetición que quiero introducir aquí usa la palabra clave `loop` y es similar al bucle `for` de estilo C (es decir, el bucle del lenguaje de programación C). En este tipo de bucle, tú declaras entre una pareja de paréntesis tres expresiones separadas por puntos y comas: el valor inicial de la variable de iteración, la condición por la cual el bucle debería terminar, y el cambio que se hace a la variable de iteración en cada iteración:

```
loop (my $i = 0; $i < 5; $i++) {
    say $i, " -> " ~ @letras[$i];
}
```

Para los bucles más comunes, el bucle `for` visto anteriormente es más fácil de escribir y usualmente más eficiente que esta construcción. Esta construcción especial `loop` debería ser probablemente usada solo cuando la salida de una condición o el cambio que se hace a la variable de iteración es bien inusual y sería algo difícil de expresar en un bucle `for` regular. Como una excepción, la construcción `loop` sin la especificación de tres partes es bien común y hasta idiomática para crear un bucle infinito:

```
loop {
    # haz algo
    # termina si ...
}
```

9.8 Asociaciones, Filtros y Reducciones

Al recorrer los elementos de un array (o una lista), lo que hemos hecho hasta ahora es procesar los elementos uno por uno con un bucle. Ahora estudiaremos formas de procesar todos los elementos al mismo tiempo.

9.8.1 Reducir una Lista a un Valor

Para añadir todo los números de una lista, puedes usar el bucle `for` de la siguiente manera:

```
sub agrega-todos( @números ) {
    my $total = 0;
    for @números -> $x {
        $total += $x;
    }
    return $total;
}
```

La variable `$total` es inicializada a 0. Cada vez que el bucle itera, la variable `$x` consigue uno de los elementos de la lista y el valor en dicha variable es agregado a `$total`. Mientras el bucle se ejecuta, `$total` acumula la suma de los elementos; una variable que es usada de esta forma se conoce como un **acumulador**.

Una operación que combina una secuencia de elementos en un solo valor es usualmente llamada una operación de reducción porque su efecto es **reducir** todos los artículos a un solo elemento (esto es conocido como “pliegues” (del inglés *folding*) en otros lenguajes de programación). Estas ideas son derivadas de los lenguajes de programación funcionales tal como LISP (cuyo nombre deriva de “LISt Processing”).

Raku tiene una función `reduce`, la cual genera un solo valor "combinado" de la lista de valores. Esto se lleva a cabo al aplicar de forma iterativa a cada artículo una función que sabe cómo combinar dos valores. El uso de la función `reduce` para calcular la suma de los primeros diez números luce así:

```
> my $suma = reduce { $^a + $^b }, 1..10;
55
```

¿Recuerdas la función *factorial* de la Sección 4.10? Esta función usaba un bucle `for` para calcular el producto de los primeros enteros hasta un cierto límite. Podría escribirse de nuevo usando la función `reduce`:

```
sub factorial( Int $num ) {
    return reduce { $^a * $^b }, 1..$num;
}
say factorial 10;    # -> 3628800
```

De hecho, el código para calcular el factorial es tan corto con la función `reduce` que se puede argumentar lo innecesario que es escribir una subrutina para eso. Podrías solo poner el código "en línea":

```
my $fact10 = reduce { $^a * $^b }, 1..10;    # -> 3628800
```

Podemos hacer muchísimas cosas poderosas con esto, pero regresaremos a esto más tarde, debido a que requiere algunas características sintácticas que no hemos visto todavía.

9.8.2 El Metaoperador de Reducción

Raku tiene un operador de reducción, o más bien un *metaoperador* de reducción. Un operador usualmente trabaja con variables o valores; un metaoperador actúa sobre otros operadores. Dada una lista y un operador, el metaoperador `[...]` aplica de forma iterativa el operador a todos los valores de la lista para producir un solo valor.

Por ejemplo, el código siguiente también imprime la suma de todos los elementos de una lista:

```
say [+] 1, 2, 3, 4;    # -> 10
```

Esto básicamente toma los primeros dos valores, los agrega, y agrega el resultado al siguiente valor, continuando hasta que haya recorrido la lista completa. Actualmente, existe una forma de este operador, con una barra invertida delante del operador, el cual devuelve los resultados intermedios de la operación:

```
say [\+] 1, 2, 3, 4;    # -> (1 3 6 10)
```

Este metaoperador puede usarse para transformar básicamente cualquier operador asociativo infijo² en un operador de lista que devuelve un solo valor.

La función factorial puede escribirse de nuevo en la siguiente forma:

²Un operador *infijo* es un operador que se coloca entre dos operandos.

```
sub fact( Int $x ) {
    [*] 1..$x;
}
my $factorial = fact(10);    # -> 3628800
```

El metaoperador de reducción puede también usarse con los operadores de relación para chequear si los elementos de un array o una lista están en el orden numérico o alfabético correcto:

```
say [<] 3, 5, 7;           # -> True
say [<] 3, 5, 7, 6;        # -> False
say [!t] <a c d f r t y>;  # -> True
```

9.8.3 Asociando una Lista a Otra Lista

Algunas veces quieres recorrer una lista mientras construyes otra. Por ejemplo, la siguiente función toma una lista de cadenas de texto y devuelve una lista nueva que contiene las cadenas de texto en letras mayúsculas:

```
sub mayúsculas( @palabras ) {
    my @resultado;
    push @resultado, $_.uc for @palabras;
    return @resultado;
}
my @palabras_minus = <one two three>;
my @palabras_mayus = mayúsculas(@palabras_minus); # -> [ONE TWO THREE]
```

La variable `@resultado` es declarada como un array vacío; cada vez que el bucle itera, agregamos el siguiente elemento. Así que `@resultado` es otro tipo de acumulador.

Una operación como `mayúsculas` es algunas veces conocida como un **map** porque “mapea” (asocia) una función (en este caso el método `uc`) a cada elemento en una secuencia.

Perl tiene una función `map` que hace posible todo eso con solo una sentencia:

```
my @palabras_minus = <one two three>;
my @palabras_mayus = map { .uc }, @palabras_minus; # -> [ONE TWO THREE]
```

Aquí la función `map` aplica el método `uc` a cada elemento del array `@palabras_minus` y lo devuelve en el array `@palabras_mayus`. Más precisamente, la función `map` asigna de forma iterativa cada elemento del array `@palabras_minus` a la variable tónica `$_`, aplica el bloque de código después de la palabra clave `map` a `$_` para crear nuevos valores, y devuelve una lista de estos valores.

Para generar una lista de números pares entre 1 y 10, podríamos usar el operador de rango para generar números entre 1 y 5 y usar `map` para multiplicarlos por dos:

```
my @pares = map { $_ * 2 }, 1..5; # -> [2 4 6 8 10]
```

En vez de usar la variable tónica `$_`, podríamos también usar la sintaxis del bloque puntificado con una variable de iteración explícita:

```
my @pares = map -> $num { $num * 2 }, 1..5; # -> [2 4 6 8 10]
```

o un bloque anónimo con una variable marcadora:

```
my @pares = map { $_ * 2 }, 1..5; # -> [2 4 6 8 10]
```

En vez de un bloque de código, el primer argumento para `map` puede ser una referencia de código (una referencia a una subrutina):

```
sub doble-raíz-más-uno( Numeric $x ) {
    1 + 2 * sqrt $x;
}
my @resultados = map &doble-raíz-más-uno, 4, 16, 42;
say @resultados; # -> [5 9 13.9614813968157]
```

El nombre de la subrutina necesita ser precedido con el sigilo `&` para clarificar que es un parámetro para `map` y no una llamada directa de la subrutina.

Si el nombre del array a la izquierda y a la derecha de la asignación es el mismo, entonces la modificación es hecha “en lugar”, o sea, lo cual aparece ser como si el array original es modificado en el proceso.

Esta es una función inmensamente poderosa y expresiva; regresaremos a ella más tarde.

9.8.4 Filtrando los Elementos de una Lista

Otra operación común de las listas es seleccionar algunos elementos de la lista y devolver una sublista. Por ejemplo, la siguiente función toma una lista de cadenas de texto y devuelve una lista que contiene solamente las cadenas de texto que contienen una vocal:

```
sub contiene-vocal( Str $cadena ) {
    return True if $cadena =~ /[aeiouy]/;
}

sub filtrar_palabras_con_vocales( @cadenas ) {
    my @cadena-retenida;
    for @cadena -> $cad {
        push @cadena-retenida, $cad if contiene-vocal $cad;
    }
    return @cadena-retenida;
}
```

La subrutina `contiene-vocal` devuelve `True` si la cadena de texto contiene por lo menos una vocal (para nuestro propósito, consideramos que “y” es una vocal).

La subrutina `filtrar_palabras_con_vocales` devolverá una lista de cadenas de texto que contienen por lo menos una vocal.

Una operación como la que `filtrar_palabras_con_vocales` lleva a cabo es conocida como un **filtro** (**en.** *filter*) porque selecciona algunos de los elementos y filtra el resto.

Perl tiene una función llamada `grep` que hace lo mismo con una sola sentencia:

```
my @filtradas = grep { /[aeiouy]/ }, @entrada;
```

El nombre de la función integrada `grep` usada para filtrar algunas entradas proviene del mundo de Unix, donde es una utilidad que filtra las líneas de un archivo de texto que coinciden con un patrón dado.

En el código de arriba, todas las cadenas de texto de `@entrada` serán comparadas contra el bloque `grep`, y aquellas que coinciden con el regex serán almacenadas en el array `@filtradas`. Al igual que `map`, la función `grep` asigna, de forma iterativa, cada elemento del array `@entrada` a la variable tónica `$_`, aplica el bloque de código después de la palabra clave `grep` a `$_`, y devuelve una lista de los valores para los cuales el bloque de código evalúa como verdadero. Aquí, el bloque de código es un simple regex que se aplica a la variable `$_`.

Al igual que `map`, podríamos haber usado una referencia a una función como el primer argumento para `grep`:

```
my @filtradas = grep &contiene-vocal, @entrada;
```

Para generar una lista de números pares entre 1 y 10, podríamos usar el operador de rango para generar números entre 1 y 10 y usar la función `grep` para filtrar los números impares:

```
my @pares = grep { $_ %% 2 }, 1..10; # -> [2 4 6 8 10]
```

Como un ejercicio, escribe un programa usando la función `map` para producir una array que contenga el cuadrado de los números de la lista de entrada y un programa que use `grep` para mantener solo los números de la lista de entrada que son cuadrados perfectos. Solución: A.7.3.

Muchas de las operaciones de listas pueden ser expresadas como una combinación de `map`, `grep`, y `reduce`.

9.8.5 Funciones de Orden Superior y la Programación Funcional

Además de sus utilidades inmediatas, las funciones `map`, `grep`, y `reduce` que usamos aquí hacen algo cualitativamente nuevo. Los argumentos de estas funciones no son solo datos: su primer argumento es un bloque de código o una función. No solo estamos pasando los datos que tendrán que usar, pero también pasamos el código que procesará los datos.

Las funciones `reduce`, `map`, y `grep` son usualmente conocidas como *funciones de orden superior* porque ellas no solo manipulan datos, sino que también manipulan otras funciones. Se puede pensar sobre estas funciones como funciones genéricas abstractas—ellas realizan una operación puramente técnica: procesan los elementos de una lista y aplican a cada de ellos un comportamiento definido en el bloque de código o la función del primer parámetro.

Estas ideas están basadas en gran medida en la programación funcional, un paradigma de programación que es muy diferente a lo que hemos visto hasta ahora y que ha sido implementado históricamente en lenguajes tales como Lisp, Caml, Ocaml, Scheme, Erlang o Haskell. Raku no es un lenguaje de programación funcional en el mismo sentido que estos lenguajes, porque puede usar otros paradigmas de programación, pero ha incorporado la mayoría de las funciones útiles. Por lo tanto, puedes usar el poder expresivo y la

seguridad inherente de este modelo de programación sin estar forzado a hacerlo y cuando prefieres un modelo diferente. Y todo esto lo puedes hacer sin tener que aprender una sintaxis totalmente nueva la cual puede lucir un poco abstrusa o hasta torpe.

Esto es inmensamente útil y puede darte un increíble poder expresivo para resolver ciertos tipos de problemas. Pero otros tipos de problemas podrían resolverse mejor con el modelo tradicional procedimental o imperativo, mientras que otros problemas se pueden beneficiar de una perspectiva orientada a objetos. Raku te deja elegir el modelo de programación que quieras usar, y hasta hace posible la combinación de varios paradigmas en el mismo programa.

La programación funcional es tan importante en mi opinión que he dedicado un capítulo completo de este libro a las capacidades de la programación funcional en Perl (ver Capítulo 14). Antes de leer ese capítulo, asegúrate de leer la Subsección A.7.1.6 en la sección de los arrays y listas en el capítulo sobre las soluciones de los ejercicios.

9.9 Arrays de Tamaños Fijos y Tipados

Por defecto, los arrays pueden contener elementos de cualquier tipo, incluyendo elementos de diferentes tipos, y se pueden auto-extender según lo necesites. Perl se encargará de los detalles innecesarios por ti, para que no preocupes por ellos. Esto es bien práctico pero viene con su costo: algunas operaciones de array pueden ser inesperadamente lentas, debido a que Perl puede tener que realizar un poco de limpieza entre bastidores, tales como gestionar la memoria, copiar un array completo a la memoria, etc.

En algunos casos, sin embargo, es posible saber de antemano el tamaño de un array, y el tipo de sus elementos. Si Perl tiene esta información, podría ser capaz de funcionar más rápidamente y usar muchísima menos memoria. También esto podría ayudar a prevenir errores sutiles.

Para declarar el tipo de los elementos de un array, solo necesitas especificarlo cuando declaras el array. Por ejemplo, para declarar un array de números enteros:

```
> my Int @números = 1..20;
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]
> @números[7] = 3.5;      # ERROR
Type check failed in assignment to @números; expected Int but got Rat
in block <unit> at <unknown file> line 1
```

Del mismo modo, puedes declarar el tamaño de un array. Los arrays con un tamaño fijo son conocidos como *arrays moldeados* (del inglés *shaped arrays*). Hay doce meses en un año, así que podrías decirle a Perl que tu array @meses nunca tendrá más de doce elementos:

```
> my @meses[12] = 1..7;
[1 2 3 4 5 6 7 (Any) (Any) (Any) (Any) (Any)]
> say @meses.elems
12
> say @meses[3];
4
> say @meses[12];
Index 12 for dimension 1 out of range (must be 0..11)
```

Aquí, Perl destinó 12 “espacios” para el array, aunque los último cinco están actualmente indefinidos. Perl no necesita destinar memoria cuando defines el décimo artículo del array. Y Perl te dejará saber tu equivocación cuando accidentalmente trates de acceder un elemento que está fuera de rango.

La definición del tipo de elementos y el tamaño máximo del array puede conducir a un mayor rendimiento en términos de la velocidad de ejecución (al menos para algunas operaciones) y reducir significativamente el consumo de memoria por el programa, especialmente cuando se manipulan los arrays grandes.

9.10 Arrays Multidimensionales

Los array que hemos visto hasta ahora han sido de una sola dimensión. En algunos lenguajes, tales arrays son conocidos como vectores. Pero los arrays también pueden ser multidimensionales (los cuales puedes llamar matrices).

Por ejemplo, podrías usar un array de dos dimensiones para almacenar una lista de empleados con sus respectivos salarios:

```
> my @empleados;
[]
> @empleados[0;0] = "Liz";
Liz
> @empleados[0;1] = 3000;
3000
> @empleados[1] = ["Bob"; 2500];
[Bob 2500]
> @empleados[2] = ["Jack"; 2000];
[Jack 2000]
> @empleados[3] = ["Betty"; 1800];
[Betty 1800]
> say @empleados[1;1];
2500
> say @empleados[2];
[Jack 2000]
> say @empleados;
[[Liz 3000] [Bob 2500] [Jack 2000] [Betty 1800]]
```

Es posible tener más de dos dimensiones. Por ejemplo, podríamos tener un array con tres dimensiones para almacenar las temperaturas de un reactor químico, medidas en varios lugares identificados por sus coordenadas x, y y z:

```
my @temp;
@temp[1;3;4] = 80;
```

Sin embargo, para este tipo de datos, es mejor usar la estructura de datos que cubriremos en el siguiente capítulo, los hashes.

Los arrays multidimensionales pueden también tener un tamaño fijo. Por ejemplo, esta puede ser la declaración de un array de dos dimensiones donde la primera dimensión es el mes del año y la segunda es el número de días del mes:

```
my @fecha[12, 31];
```

9.11 Ordenando Arrays o Listas

El ordenamiento (**en.** *sorting*) de datos es una operación muy común en la ciencia de la computación. Perl tiene una función `sort` que puede ordenar un array o una lista y devolver el resultado ordenado:

```
say sort <4 6 2 9 1 5 11>; # -> (1 2 4 5 6 9 11)
```

Existen varios tipos de ordenamiento. Los más comunes son los ordenamientos numéricos y lexicográficos (o alfabéticos). Ellos difieren en la manera en que comparan los elementos individuales a ser ordenados.

En el ordenamiento alfabético, tú debes comparar la primera letra de las palabras a ser comparadas; una palabra que comienza con una “a” siempre vendrá antes que una palabra que comienza con una “b” (o cualquier otra letra) en orden ascendente, sin importar el valor o número de los otros caracteres. Tú necesitas comparar el segundo carácter de las dos palabras solo si el primer carácter de las palabras es el mismo.

El ordenamiento numérico es muy diferente: el valor de interés es el valor total del número. Por ejemplo, si estamos ordenando números enteros, 11 es mayor que 9 porque tiene más dígitos. Sin embargo, el ordenamiento alfabético de 9 y 11 considera a 11 menor que 9, porque el primer dígito es menor.

Por lo tanto, el ordenamiento alfabético o lexicográfico de la lista de enteros anterior devuelve:

```
(1 11 2 4 5 6 9)
```

La consecuencia es que, con muchos lenguajes de programación, cuando quieres ordenar datos, necesitas especificar qué tipo de ordenamiento quieres. Con datos consistentes (cada artículo del mismo tipo), Raku es lo suficientemente astuto para encontrar qué tipo de ordenamiento es apto para tus necesidades. Así que, por ejemplo, este código hará el tipo de ordenamiento que esperas:

```
say sort <ac a bc ab abc cb ca>; # ->(a ab abc ac bc ca cb)
```

Hasta con tipos de datos mezclados, `sort` puede hacer un gran trabajo al proveer un resultado que puede ser lo que esperas:

```
say sort <1 b 11 5 cb 4 12 a ab abc 42 ac bc ca >;  
# -> (1 4 5 11 12 42 a ab abc ac b bc ca cb)
```

Existen otros casos donde el simple uso de la función `sort` fallará y no devolverá lo que probablemente esperas:

```
say sort <a ab abc A bc BAC AC>; # -> (A AC BAC a ab abc bc)
```

En este caso, `sort` pone todas las cadenas de texto que comienzan con una letra mayúscula antes que cualquier cadena de texto que comienza con una letra minúscula. Probablemente no deseas esto. Esto luce peor si las cadenas de texto usan caracteres ASCII extendidos:

```
say sort <a ab âb abc Ñ A bc BAC AC>;  
# -> (A AC BAC a ab abc bc Ñ âb)
```

La razón detrás de esto es que, al ordenar cadenas de texto, la función `sort` usa la codificación numérica interna de las letras. Esto fue llamado alguna vez orden "ASCIIbetical" (en contraste al orden alfabético), pero el término es ahora muy limitado y algo obsoleto, porque Raku usa Unicode y no ASCII.

Claramente, hay casos donde técnicas de ordenamiento más avanzadas son necesarias.

9.12 Técnicas de Ordenamiento Más Avanzadas

La rutina `sort` típicamente toma dos argumentos, un objeto de código y una lista de artículos a ser ordenados, y devuelve una nueva lista ordenada. Si no se especifica un objeto de código, como en el ejemplo que vimos más arriba, el operador integrado de comparación `cmp` se usa para comparar los elementos. Si el objeto de código es proveído, entonces se usa en la comparación, la cual informa a `sort` cuál de los elementos debería estar primero en el orden final.

Existen tres operadores integrados de comparación que pueden usarse para ordenar. Algunas veces son conocidos como los *comparadores de tres sentidos* porque ellos comparan sus operandos y devuelven un valor que informa si el primer operador debería ser considerado menor que, igual a o mayor que el segundo operador para el propósito de determinar el orden en que estos operadores deben ser ordenados. El operador `leg` coacciona sus argumentos a cadenas de texto y realiza una comparación lexicográfica. El operador `<=>` coacciona sus argumentos a números (Real) y realiza una comparación numérica. El operador `cmp` es el comparador "inteligente" de tres sentidos, el cual compara cadenas de texto con semántica de cadenas de texto y números con semántica de números.

La mayoría de nuestros simples ejemplos funcionaron bien con cadenas de texto y números porque ellos implícitamente usaron el operador `cmp` por defecto, el cual "adivina" el tipo de comparación a realizarse.

En otras palabras, esto:

```
say sort <4 6 2 9 1 5 11>; # -> (1 2 4 5 6 9 11)
```

es equivalente a esto:

```
say sort { $^a cmp $^b }, <4 6 2 9 1 5 11>;
# -> (1 2 4 5 6 9 11)
```

El bloque de código usado aquí como el primer argumento de la función `sort` usa los parámetros marcadores (parámetros auto-declarados de posición) los cuales vimos anteriormente en este capítulo. El operador `cmp` recibe dos argumentos y le pasa a la función `sort` información sobre cuál de los artículos debería aparecer primero en el orden resultante.

Si quieres ordenar en el orden inverso, podrías solo intercambiar el orden de los parámetros marcadores:

```
say sort { $^b cmp $^a }, <4 6 2 9 1 5 11>;
# -> (11 9 6 5 4 2 1)
```

Ten presente que discutimos este ejemplo solo con el propósito de explicar algunas funciones de los parámetros marcadores. Para ordenar el array que hemos presentado aquí en orden descendente, sería más fácil obtener el mismo resultado con el siguiente código:


```
say reverse sort <4 6 2 9 1 5 11>; # -> (11 9 6 5 4 2 1)
```

La razón por la cual `sort` hace un trabajo tan grandioso hasta con una mezcla de cadenas de texto y números es porque la función de comparación por defecto, `cmp`, es muy astuta y puede adivinar si debería realizar una comparación de orden lexicográfico o de orden numérico al examinar sus argumentos.

Si el ordenamiento se complica para `cmp`, o generalmente cuando se requiere un ordenamiento específico o personalizado, entonces debes crear tu propia subrutina ad-hoc de comparación.

Por ejemplo, si tomamos de nuevo el ejemplo de las cadenas de texto con letras mayúsculas y minúsculas, podemos lograr un ordenamiento alfabético que no es sensible a la diferencia entre letras mayúsculas y minúsculas:

```
say sort { $^a.lc cmp $^b.lc }, <a ab abc A bc BAC AC>;
# -> (a A ab abc AC BAC bc)
```

o de esta manera:

```
say sort { $^a.lc leg $^b.lc }, <a ab abc A bc BAC AC>;
# -> (a A ab abc AC BAC bc)
```

Cuando el bloque de código de comparación recibe sus dos argumentos, el método `lc` los convierte en letras minúsculas antes de realizar la comparación. Nota que esto no tiene ningún impacto en la salida de texto, dado que la transformación a letra minúscula es local al bloque de código de comparación y no tiene impacto alguno en los datos manejados por `sort`. Veremos en un rato que hay una manera más fácil y eficiente de realizar tal transformación antes de comparar los argumentos.

Si la especificación de la comparación es muy complicada, podría ser necesario escribirla en una subrutina separada y dejar que `sort` llame a esa subrutina. Supón que tenemos una lista de cadenas de texto que están todas formadas por dígitos iniciales, los cuales son seguidos por un grupo de letras y posiblemente seguidos por otros caracteres irrelevantes, y queremos ordenar las cadenas de texto de acuerdo al grupo de letras que siguen los dígitos.

Comencemos con escribir la subrutina de comparación:

```
sub mi_comp( $cad1, $cad2 ) {
  my $cmp1 = $0 if $cad1 =~ /\d+(\w+)/;
  my $cmp2 = $0 if $cad2 =~ /\d+(\w+)/;
  return $cmp1 cmp $cmp2;
}
```

Nada complicada: la subrutina toma dos argumentos, usa un regex para extraer el grupo de palabras de cada argumento, y devuelve el resultado de las cadenas extraídas a la función `cmp`. En el mundo real, se necesitaría hacer algo si la extracción falla, pero, para nuestro propósito, asumiremos que esto no sucederá.

El ordenamiento es ahora bastante directo, solo necesitamos pasar a la función `sort` la función anterior:

```
say sort &mi_comp, < 22ac 34bd 56aa3 12c; 4abc( 1ca 45bc >;
# -> (56aa3 4abc( 22ac 45bc 34bd 12c; 1ca)
```

Solo necesitamos preceder la subrutina de comparación con el sigilo & y funcionará bien: las cadenas de texto son ordenadas de acuerdo a los grupos de letras que siguen los dígitos al inicio de cada cadena de texto.

En todos los ejemplos anteriores, la subrutina de comparación aceptó dos parámetros, los artículos a ser comparados. La función sort puede funcionar con un objeto de código que toma un solo parámetro. En tal caso, el objeto de código no es un bloque de código de comparación, sino que es un objeto de código que implementa la transformación a ser aplicada a los artículos antes de usar la función de comparación por defecto cmp.

Por ejemplo, si tomamos una vez más el ejemplo de las cadenas de texto mezcladas con letras mayúsculas y minúsculas, podemos lograr un ordenamiento alfabético que no es sensible a la diferencia entre letras mayúsculas y minúsculas en una nueva forma:

```
say sort { $_.lc }, <a ab abc A bc BAC AC>;
# -> (a A ab abc AC BAC bc)
```

Esto podría también escribirse con un parámetro marcador:

```
say sort { $^a.lc }, <a ab abc A bc BAC AC>;
# -> (a A ab abc AC BAC bc)
```

En este ejemplo, debido a que el bloque de código de comparación toma un solo argumento, su tarea es transformar cada uno de los artículos a ser comparados antes de aplicar la función cmp a los argumentos. Esto no solo hace las cosas más simples, sino que también es probablemente más eficaz, especialmente si el número de artículos a ser ordenados es grande y si la subrutina de transformación es relativamente costosa (en término de procesamiento): los valores transformados son actualmente almacenados en *caché* (es decir, almacenados en la memoria para uso repetido), de tal manera que la transformación se hace solamente una vez por cada artículo, a pesar de que la subrutina de comparación se llama muchas veces por cada artículo en un ordenamiento.

De igual manera, podríamos ordenar los números de acuerdo a sus valores absolutos:

```
say sort { $_.abs }, <4 -2 5 3 -12 42 8 -7>; # -> (-2 3 4 5 -7 8 -12 42)
```

El ejemplo “más complejo” con dígitos y letras que requiere una subrutina separada aplica la misma transformación a ambos argumentos. Como un ejercicio, escribe un simple programa de ordenamiento que usa una subrutina de transformación y el operador de comparación por defecto cmp sobre los artículos transformados. Solución: A.7.4.

Sobra decir que los usos avanzados de la función sort presentados en esta sección son solo más ejemplos del estilo de programación funcional. Las subrutinas de comparación y las subrutinas de transformación pueden pasarse como argumentos a la función sort, y generalmente, todas las funciones, las subrutinas, y los bloques de códigos usados aquí son funciones de orden superior o funciones de primera clase.

9.13 Depuración de Programas

El uso descuidado de los arrays (y cualquier otro objeto mutable) puede conducir a largas horas de depuración. Estas son algunas de las dificultades y las maneras de evitarlas:

1. Algunas de las funciones y métodos integrados de los arrays modifican sus argumentos mientras otras no.

Puede ser tentativo escribir código así:

```
@array = splice @array, 1, 2, $nuevo-elem; # INCORRECTO!
```

La función `splice` devuelve los elementos que ha eliminado del array, no el array en sí mismo, el cual es modificado “en el lugar”.

Antes de usar los métodos y operadores de los arrays, deberías leer la documentación cuidadosamente y quizás probarlos en modo interactivo.

Al recorrer un array, por ejemplo con `for` o `map`, la variable tónica `$_` es un apodo para los elementos sucesivos del array, y no una copia de ellos. Esto significa que si cambias a `$_`, el cambio será reflejado en el array. Pueden haber casos donde este comportamiento es deseable y otros casos donde no importa lo que suceda con ellos (porque ya no necesitas el array original), pero esta técnica es propensa a errores y quizás debería evitarse (o por lo menos usarla con precaución).

```
my @números = <1 2 3>;
push @dobles, $_*=2 for @números; # INCORRECTO (probablemente)
say @números; # -> [2 4 6]
```

El error aquí es que la sentencia `$_*=2` está modificando a `$_`, así que el array `@números` es también modificado, mientras que la intención era ciertamente poblar los nuevos números en el array `@dobles`, no modificar a `@números`.

El mismo código aplica a una lista literal en vez de un array lo cual conduce a un error al tiempo de ejecución, debido a que una lista es inmutable:

```
> push @dobles, $_*=2 for <1 2 3>; # INCORRECTO (definitivamente)
Cannot assign to an immutable value
```

La solución en este caso es bien fácil y consiste en usar una expresión que no modifique a `$_` pero que devuelva el valor deseado:

```
push @dobles, $_ * 2 for @números; # OK
```

Lo mismo aplica a `map`:

```
my @números = <1 2 3>;
say map { ++$_ }, @números; # INCORRECTO (probablemente)
say @números; # -> [2 3 4]
```

Otra vez, el uso de una expresión que no modifique a `$_`, pero que en cambio devuelva el nuevo valor deseado, arreglará el problema:

```
my @números = <1 2 3>;
say map { $_ + 1 }, @números; # -> (2 3 4)
say @números; # -> [1 2 3]
```

9.14 Glosario

Lista Una secuencia inmutable de valores.

Array Una variable que contiene una secuencia mutable de valores.

Elemento Uno de los valores en una lista o array (o cualquier otra secuencia). También llamado artículos.

Array anidado Un array que es un elemento de otro array.

Acumulador Una variable que se usa en un bucle para agregar o acumular un resultado.

Reducir Un patrón de procesamiento que recorre una secuencia y acumula los elementos en un solo resultado.

Map Un patrón de procesamiento que recorre una secuencia y realiza una operación sobre cada elemento. También el nombre de una función integrada de Perl que realiza el mismo patrón de procesamiento.

Filtro Un patrón de procesamiento que recorre una lista y selecciona los elementos que satisfacen una condición. `grep` es una implementación en Perl de un filtro.

Apodo Una circunstancia donde un identificador se refiere directamente a alguna variable o valor, de tal manera que un cambio del identificador se traduce en un cambio de la variable o valor. Esto significa tener dos nombres para el mismo valor, contenedor u objeto.

9.15 Ejercicios

Ejercicio 9.1. *Escribe una subrutina llamada `suma-anidada` que toma un array de los arrays de enteros y añade todos los elementos de los arrays anidados. Por ejemplo:*

```
my @AoA = [[1, 2], [3], [4, 5, 6]];
say suma-anidada(@AoA);          # -> 21
```

Solución: A.7.5.

Ejercicio 9.2. *Escribe una subrutina llamada `suma-acumula` que toma una lista de números y devuelve la suma acumulativa, es decir, una nueva lista donde la suma del elemento i th es la suma de los primeros elementos $i + 1$ de la lista original. Por ejemplo:*

```
my @nums = [1, 2, 3, 4];
say suma-acumula(@nums);          # -> [1, 3, 6, 10]
```

Solución: A.7.6.

Ejercicio 9.3. *Escribe una subrutina llamada `medio` que toma una lista y devuelve una lista nueva que contiene todos los elementos excepto el primero y el último. Por ejemplo:*

```
say medio(1, 2, 3, 4);           # -> (2, 3)
```

Solución: A.7.7.

Ejercicio 9.4. *Escribe una subrutina llamada `corta` que toma un array, lo modifica al remover el primer y último elemento y devuelve nada útil. Por ejemplo:*

```
my @nums = 1, 2, 3, 4;
corta(@nums);
say @nums;                # -> [2, 3]
```

Solución: A.7.8.

Ejercicio 9.5. Escribe una subrutina llamada *está-ordenada* que toma una lista o un array de números como parámetro y devuelve `True` si la lista está ordenada en orden ascendente y `False` de lo contrario. Por ejemplo:

```
> está-ordenada (1, 2, 2);
True
> está-ordenada (1, 2, 1);
False
```

Solución: A.7.9.

Ejercicio 9.6. Una palabra es un anagrama de otra si las dos tienen las mismas letras, con el mismo número de apariciones, pero en un orden diferente. Escribe una subrutina llamada *es-anagrama* que toma dos cadenas de texto y devuelve `True` si son anagramas.

Solución: A.7.10.

Ejercicio 9.7. Escribe una subrutina llamada *tiene-duplicados* que toma una lista o un array y devuelve `True` si hay algún elemento que aparece más de una vez. La subrutina no debería modificar la entrada original.

Solución: A.7.11.

Ejercicio 9.8. Este ejercicio es acerca de la Paradoja del Cumpleaños. Encuentra más información sobre esta paradoja aquí: https://es.wikipedia.org/wiki/Paradoja_del_cumpleaños.

Si hay 23 estudiantes en tu clase, cuál es la probabilidad de que dos estudiantes cumplan año el mismo día? Puedes estimar esta probabilidad al generar muestras aleatorias de los 23 cumpleaños y chequear por cumpleaños duplicados. Pista: puedes generar cumpleaños aleatorios con la funciones `rand` e `int`.

Solución: A.7.12.

Ejercicio 9.9. Escribe una subrutina que lee el archivo `words.txt` y construye una lista con un elemento por palabra. Escribe dos versiones de esta función, una que usa el método `push` y la otra que usa la función `unshift`. ¿Cuál toma más tiempo en ejecutarse? ¿Por qué?

Solución: A.7.13.

Ejercicio 9.10. Para chequear si una palabra está en nuestra lista estándar de palabras, podrías chequear cada elemento por turno, pero sería lento porque buscaría a través de las palabras en orden.

Si las palabras están en orden alfabético (el cual es el caso en nuestra lista de palabras), podemos acelerar las cosas con una búsqueda binaria, la cual es similar a lo que haces cuando buscas una palabra en el diccionario. Tú comienzas en alguna parte en el medio del diccionario y chequea si la palabra que estás buscando se encuentra antes que la palabra en el medio de la lista. Si esto sucede, busca en la primera mitad de la lista del mismo modo. Por lo contrario, busca en la segunda mitad de la lista.

De cualquier manera, tú reduces el tiempo de búsqueda por la mitad. Si la lista de palabras contiene 113,809 palabras, tomará alrededor de 17 pasos para encontrar la palabra o concluir que la palabra no se encuentra en la lista.

Escribe una función llamada `búsqueda-binaria` que toma una lista ordenada y un valor a encontrar en la lista. La función debe devolver si el valor se encuentra en la lista o no.

Solución: A.7.14

Ejercicio 9.11. Dos palabras son una “pareja de inversas” si cada una es la inversa de la otra. Por ejemplo, “arroz” y “zorra” forman una pareja de inversas.; otro ejemplo incluye “amar” y “rama”, o “oír” y “río”.

Escribe un programa que encuentre todas las parejas de inversas en el archivo `word.txt`.

Solución: A.7.15.

Ejercicio 9.12. Dos palabras se “entrelazan” si al tomar letras alternadas de ambas palabras se forma una nueva palabra. Por ejemplo, “shoe” y “cold” se entrelazan para formar la palabra “schooled”.

Escribe un programa que encuentre todos los pares de palabras que se entrelazan en el archivo `word.txt`. Pista: ¡no enumeres todas la parejas, hay demasiadas!

Solución: A.7.16

Crédito: este ejercicio fue inspirado por un ejemplo en <http://puzzlers.org>.

Hashes

Este capítulo presenta otro tipo integrado conocido como un hash. Los hashes son unas de las mejores y más comunes características de Raku; ellos constituyen los componentes básicos de muchos algoritmos eficientes y elegantes.

10.1 Un Hash es un Mapeo

Un **hash** es como un array, pero más general. En un array, los índices o subíndices tienen que ser enteros; en un hash, pueden ser (casi) cualquier cosa.

Un hash contiene una colección de índices, los cuales se llaman **claves**, y una colección de valores. Cada llave está asociada a un valor único. Una clave y un valor juntos forman un par (un objeto del tipo `Pair`), o un **par clave-valor**. Un hash puede ser visto como una colección de parejas de clave-valor. Los valores en un hash puede también llamarse artículos o elementos, como con los arrays.

En otros lenguajes de programación, los hashes son algunas veces llamados diccionarios, tablas hash, mapas, o arrays asociativos.

En el lenguaje matemático, un hash representa un **mapeo** desde las claves a los valores, así que puedes también decir que cada clave “mapea” a un valor. Como un ejemplo, construiremos un hash que mapea palabras del inglés al español, y por lo tanto las llaves y los valores son todas cadenas de texto.

En Raku, el nombre de un hash comienza con el sigilo `%`. Para crear un nuevo hash, solo decláralo de esta manera:

```
> my %ingAesp;
```

Esto crea un hash vacío. Para agregar artículos al hash, puedes usar las llaves:

```
> %ingAesp{'one'} = 'uno';  
uno
```


Esta línea crea un artículo que relaciona la clave 'one' al valor 'uno'.

Si la clave es una cadena de texto que contiene una sola palabra (es decir, sin ningún espacio en el medio), existe un atajo más idiomático para crear la misma entrada de hash:

```
> %ingAesp<one> = 'uno';
uno
```

Si imprimimos el hash, vemos un par clave-valor con el operador constructor de pares => entre la clave y el valor:

```
> say %ingAesp;
one => uno
```

Este formato de salida es también un formato de entrada. Por ejemplo, puedes crear un hash nuevo con tres artículos:

```
> my %ingAesp = ('one' => 'uno', 'two' => 'dos', 'three' => 'tres');
one => uno, three => tres, two => dos
```

El uso del operador constructor de pares => entre las claves y los valores no es requerido; puedes usar comas también:

```
my %ingAesp = ('one', 'uno', 'two', 'dos', 'three', 'tres');
```

Pero el constructor de pares tiene la ventaja de mostrar gráficamente las relaciones clave-valor. El operador constructor de pares también hace que el uso de comillas no sea mandatorio en su lado izquierdo (provisto que la clave sea una cuerda de texto sin espacios):

```
> my %ingAesp = (one => 'uno', two => 'dos', three => 'tres');
one => uno, three => tres, two => dos
```

También podrías usar una sintaxis de lista más concisa para la asignación del hash y Raku convertirá felizmente la lista en un hash, provisto que el número de artículos en la lista de entrada sea par:

```
> my %ingAesp = <one uno two dos three tres>;
one => uno, three => tres, two => dos
```

Podrías sorprenderte con el resultado. El orden de los pares clave-valor usualmente no se encuentran en el orden en el cual lo poblaste. En general, el orden de los artículos en un hash es impredecible.

Pero eso no es un problema porque los elementos de un hash nunca son indexados con subíndices enteros. En lugar de esto, usas las claves para consultar los valores correspondientes:

```
> say %ingAesp<two>;
dos
```

La clave two siempre mapea al valor 'dos' así que el orden de los artículos no importa.

Si la clave no se encuentra en el hash, obtienes un valor indefinido:

```
> say %ingAesp<four>;
(Any)
```

El método o la función `elems` funciona con los hashes como con los array; devuelve el número de pares clave-valor:

```
> say %ingAesp.elems;
3
> say elems %ingAesp
3
```

El adverbio `:exists` también funciona con los hashes como con los arrays; te dice si algo aparece como una *clave* en el hash (aparecer como un valor no es suficiente)¹:

```
> %ingAesp<two> :exists;
True
> %ingAesp<four> :exists;
False
```

Para chequear si algo aparece como un valor en un hash, puedes usar el método `values`, el cual devuelve una colección de valores, y después usar un bucle (o posiblemente `grep`) para buscar el artículo:

```
my @vals = values %ingAesp;
for @vals -> $value {
    say ";Encontrado!" if $value eq 'uno';           # -> ;Encontrado!
}
```

O más concisamente:

```
say ";Encontrado!" if grep {$_ eq 'uno'}, %ingAesp.values;
```

Dado que `grep` usa por defecto una coincidencia inteligente, esto puede hacerse de una forma aún más concisa:

```
say ";Encontrado!" if grep 'uno', %ingAesp.values; # -> ;Encontrado!
```

Cuando se inspecciona los valores, el programa tiene que buscar los elementos de la lista en orden (o en secuencia), como en la Sección 7.2.2. A medida que la lista se vuelve más larga, el tiempo de búsqueda se extiende en proporción directa.

Por el contrario, cuando se inspecciona las claves, Raku usa un algoritmo de **hashing** que tiene una propiedad interesante: toma el mismo monto de tiempo sin importar cuantos artículos se encuentren en el hash. En otras palabras, funciona bastante rápido, comparado con el tamaño de la lista, cuando la lista que se inspecciona es larga. Ésta es la razón por la cual la solución al ejercicio de par reverso (Ejercicio 9.11) del capítulo anterior usando un hash fue casi tres veces más rápido que la solución de búsqueda binaria (ver Solución: A.7.15).

Como un ejercicio, usa la muestra de los datos de empleados del array multidimensional de la Sección 9.10 (p. 167), organízala en un hash, y busca algunos salarios. Pista: no necesitas una estructura multidimensional para hacer eso con un hash.

Solución: A.8.1.

¹Evaluar el valor en un contexto Booleano también funcionaría con nuestro ejemplo, pero esto devolvería algo erróneo cuando la clave existe, pero el valor no está definido o por lo contrario evalúa a un valor falso (por ejemplo, si es igual a `False`, cero, o cadena de texto vacía).

10.2 Operaciones Comunes con Hashes

Ya vimos que para poblar un hash, puedes asignarle una lista par. Las cuatro formas sintácticas siguientes son todas correctas:

```
my %primer_trimestre = ("ene" => 1, "feb" => 2, "mar" => 3);
my %segundo_trimestre = (abr => 4, may => 5, jun => 6);
my %tercer_trimestre = jul => 7, aug => 8, sep => 9;
my %cuarto_trimestre = < oct 10 nov 11 dec 12 >;
```

Para agregar un elemento a un hash, solo asigna el hash con una clave:

```
my %meses = ("ene" => 1, "feb" => 2, "mar" => 3);
%meses{'abr'} = 4;
say %meses;           # -> abr => 4, feb => 2, ene => 1, mar => 3
```

Recuerda que también puedes hacer lo mismo sin encerrar las claves en comillas si usas el operador quote-word con las comillas angulares (si las claves son cadenas de texto):

```
%meses<abr> = 4;      # igual que: %meses{'abr'} = 4;
```

o puedes usar la función push con un par:

```
> push %meses, (may => 5);
abr => 4, feb => 2, ene => 1, mar => 3, may => 5
> my $nuevo-par = jun => 6
jun => 6
> push %meses, $nuevo-par;
abr => 4, feb => 2, ene => 1, jun => 6, mar => 3, may => 5
```

Usar push para agregar un par a un hash no es exactamente lo mismo que hacer una asignación de hash: si la clave ya existe, el valor antiguo no es reemplazado por el valor nuevo—en lugar, ambos valores son colocados en un array (o si el valor antiguo se encuentra ya en el array, entonces el valor nuevo se agrega al array):

```
> push %meses, (ene => '01');
{abr => 4, feb => 2, ene => [1 01], jun => 6, mar => 3, may => 5}
```

Para chequear si un valor está definido para una clave dada, usa defined:

```
> say True if defined %meses<abr>;
True
```

Para obtener el número de artículos en un hash, usa el método elems:

```
say %meses.elems;           # -> 6
```

Para remover un artículo del hash, usa el adverbio :delete:

```
> push %meses, (jud => 7);      # ¡Oops, un error!
abr => 4, feb => 2, ene => 1, jud => 7, jun => 6, mar => 3, may => 5
> %meses{'jud'}:delete;        # error removido
7
> say %meses
abr => 4, feb => 2, ene => 1, jun => 6, mar => 3, may => 5
```

Nota que el adverbio `:delete` también devuelve el valor que ha sido removido.

Para interactuar sobre un hash, usa:

- `kv` para extraer las claves y los valores intercalados;
- `keys` para extraer las llaves;
- `values` para extraer los valores;
- `pairs` para extraer los pares clave-valor;

Por ejemplo:

```
> for %meses.kv -> $clave, $val { say "$clave => $val" }
ene => 1
abr => 4
mar => 3
jun => 6
may => 5
feb => 2
> say keys %meses;
(ene abr mar jun may feb)
> say values %meses;
(1 4 3 6 5 2)
> say %meses.pairs;
(ene => 1 abr => 4 mar => 3 jun => 6 may => 5 feb => 2)
```

10.3 Un Hash como una Colección de Contadores

Supón que se te provee con una cadena de texto y quieres contar el número de apariciones de cada letra. Existen varias maneras en las que puedes hacer esto:

- Podrías crear 26 variables, una por cada letra del alfabeto. Después podrías recorrer la cadena de texto y, por cada carácter, aumentar el contador correspondiente, probablemente usando una condicional encadenada grande y fea de 26 partes.
- Podrías crear un array con 26 elementos. Después podría convertir cada carácter a un número (usando la función integrada `ord`), usar el número como un índice en el array, e incrementar el contador apropiado.
- Podrías crear un hash con caracteres como claves y contadores como los valores correspondientes. La primera vez que ves un carácter, agregarías un artículo al hash. Después de eso, aumentarías el valor de un artículo existente.

Cada una de estas opciones realiza la misma computación, pero cada una de ellas implementa la computación en una manera distinta.

Una **implementación** es una manera de realizar una computación; algunas implementaciones son mejores que otras. Por ejemplo, una ventaja de la implementación con el hash es que no tenemos que saber con antelación cuales letras aparecen en la cadena de texto y solo tenemos que crear espacio para las letras que sí aparecen.

Aquí se muestra como el código podría lucir:

```

sub histograma( Str $cadena ) {
    my %histo;
    for $cadena.comb -> $letra {
        %histo{$letra}++;
    }
    return %histo;
}

```

El nombre de la función es `histograma`, el cual es un término estadístico para una colección de contadores (o frecuencias).

La primera línea de la función crea un hash vacío. El bucle `for` recorre (o traversa) la cadena. Cada vez a través del bucle, si el carácter `$letra` no está en el hash, Raku crea un artículo nuevo con la clave `$letra` y fija los valores 0 con el operador `++`, así que el primer valor inmediatamente después de eso es 1. Si la `$letra` ya se encuentra en el hash, el valor se incrementa.

Así es cómo funciona:

```

> say histograma("We all live in a yellow submarine")
W => 1, a => 3, b => 1, e => 4, i => 3, l => 5, (...) y => 1

```

El histograma indica que las letras 'W' y 'b' aparecen solo una vez; 'a' y 'i' aparecen tres veces, 'e' aparece cuatro veces, etc.

10.4 Bucles y Hashes

Si usas un hash en una sentencia `for`, el bucle traversa los pares del hash:

```

> for %ingAesp -> $par { say $par }
two => dos
three => tres
one => uno

```

Hemos nombrado la variable de iteración `$par` para resaltar más claramente que el programa está iterando sobre los pares clave-valor (actualmente objetos `Pair`). Puedes usar los métodos `key` y `value` (nota que están en singular) para acceder la clave y valor de un `Pair`. Por ejemplo, para revertir el orden en el cual cada línea es imprimida:

```

> for %ingAesp -> $par { say $par.value ~ " <= " ~ $par.key; }
dos <= two
tres <= three
uno <= one

```

Otra vez, las claves no están en un orden particular. Para recorrer las claves ordenadamente, puedes usar las funciones o métodos `keys` (en plural) y `sort`:

```

my %histo = histograma("We all live in a yellow submarine");
for %histo.keys.sort -> $clave {
    say "$clave\t%histo{$clave}";
}

```

10.5 Búsqueda Inversa

Dado un hash `%hash` y una clave `$k`, es fácil encontrar el valor correspondiente `$val = %hash{$k}`. Esta operación se conoce como una **búsqueda** y, como ya mencionamos, es bastante rápida hasta cuando el hash es bien grande.

¿Qué pasa si tienes a `$val` y quieres encontrar a `$k`? Sucede que tienes tres problemas: primero, podrían existir más de una clave que mapean al valor `$val`; dependiendo de la aplicación, podrías elegir una, o tendrías que crear un array que las contenga a todas. Segundo, no existe una sintaxis simple para hacer una **búsqueda inversa**; tienes que hacer la búsqueda. Tercero, podría consumir mucho tiempo si el hash es largo.

Aquí presentamos una función que toma un valor y devuelve la primera clave que mapea a ese valor:

```
sub busqueda-inversa( %hash, $val ) {
  for %hash -> $par {
    return $par.key if $par.value eq $val;
  }
  return;
}
```

Esta subrutina es otro ejemplo del patrón de búsqueda. Si llegamos al final del bucle, eso significa que `$val` no aparece en el hash como un valor, así que devolvemos un valor indefinido (`Nil`). Aquí, la responsabilidad para reaccionar a tal situación se deja a la función que hace la llamada. Una alternativa es levantar una excepción, la cual tendría que ser todavía la responsabilidad de la función que hace la llamada. Sin embargo, dado que la búsqueda directa con la clave no levanta una excepción pero simplemente devuelve un valor indefinido cuando la clave no existe, hace sentido que `busqueda-inversa` tenga el mismo comportamiento cuando el valor no se encuentra.

Este es un ejemplo de una búsqueda inversa exitosa:

```
> my %histo = histograma('parrot');
a => 1, o => 1, p => 1, r => 2, t => 1
> my $clave = busqueda-inversa %histo, "2";
r
```

Y una búsqueda sin éxito:

```
> say busqueda-inversa %histo, "3";
Nil
```

Otra forma más concisa de hacer la búsqueda inversa sería usar `grep` para extraer una lista de valores que satisfagan nuestra condición:

```
say grep { .value == 2 }, %histo.pairs; # -> (r => 2)
```

Otra opción es usar una expresión con la función integrada `first` para extraer solo el primer par:

```
my %histo = histograma('parrot');
say %histo.pairs.first: *.value == 1; # -> p => 1
```

Este último ejemplo usa el parámetro *whatever* "*" el cual no hemos discutido en este libro todavía. Digamos que, aquí, el "*" quiere decir devuelve sucesivamente el primer par que coincide con la condición del valor para cada par del hash y la función `first` (ver Sección 14.5.3 para detalles sobre el parámetro "*").

Una búsqueda inversa es mucha más lenta que una búsqueda directa; si tienes que hacerla constantemente, o si el hash se vuelve grande, el rendimiento de tu programa sufrirá.

10.6 Comprobando la Existencia

Una tarea bastante común es determinar si algo existe o si un valor dado ya se ha visto en un programa. El uso de un hash es usualmente la mejor solución porque encontrar si una entrada existe para un valor dado es muy simple y también muy eficiente: solo necesitas almacenar los valores que quieres inspeccionar como una entrada de clave, y después chequear por su existencia cuando sea necesario.

En tal caso, usualmente el valor no es importante y podrías colocar cualquier cosa. Es muy común en este caso usar "1" como un valor, pero también podrías almacenar `True` o cualquier otro valor que desees.

Supón que queremos generar 10 números enteros aleatorios entre 0 y 49, pero queremos asegurarnos que los enteros son únicos. Podemos usar el método `rand` 10 veces con el rango deseado. Pero la posibilidad de obtener el mismo número dos veces no es tan insignificante (ver Ejercicio 9.8 sobre la paradoja del cumpleaños y su solución, Solución: A.7.12, para una situación similar). Por ejemplo, al intentar esto:

```
> my @lista;
[]
> push @lista, 50.rand.Int for 1..10;
> say @lista;
[12 25 47 10 19 20 25 42 33 20]
```

se produjo un valor duplicado en la lista (25) en el primer intento. Y el segundo intento produjo tres pares de duplicados:

```
> say @lista;
[45 29 29 27 12 27 20 5 28 45]
```

Podemos usar un hash para rechazar cualquier entero aleatorio generado que ya se haya visto. La siguiente manera sería una forma de escribir esto en código:

```
my @lista;
my %visto;
while @lista.elems < 10 {
  my $aleatorio = 50.rand.Int;
  next if %visto{$aleatorio}:exists;
  %visto{$aleatorio} = 1;
  push @lista, $aleatorio;
}
say @lista;
```

Cada valor entero generado se agrega al hash %visto y la lista de salida. Pero antes de hacer eso, el entero generado se chequea con el hash %visto para verificar que no se ha visto todavía. Cuando el programa finaliza su ejecución, la lista contiene 10 enteros únicos y (pseudo)aleatorios.

Lo hicimos paso a paso y mantuvimos dos estructuras de datos separadas, el array de salida @lista y el hash %visto, para hacer el proceso tan claro como sea posible. Pero si lo piensas, @lista y %visto tienen esencialmente el mismo contenido a cada paso a través del bucle. Realmente no tenemos que mantener un registro de los mismos datos en dos lugares. Dado que tener un hash es importante para chequear que los valores de salida son únicos, podemos deshacernos de @lista y escribir una versión más concisa y probablemente más idiomática del mismo programa:

```
my %visto;
while %visto.elems < 10 {
    my $aleatorio = 50.rand.Int;
    push %visto, ($aleatorio => 1) unless %visto{$aleatorio}:exists;
}
say keys %visto;      # -> (39 12 46 27 14 21 4 38 25 47)
```

Esto puede simplificarse aún más. No es realmente necesario chequear si el entero generado existe en el hash: si existe, el elemento antiguo del hash será reemplazado con el nuevo, y el hash esencialmente no cambiará. Además, cuando se evalúa en un contexto numérico escalar, un hash devuelve el número de sus elementos, así que la invocación .elems no es necesaria. Esta es la nueva versión:

```
my %visto;
%visto{50.rand.Int} = 1 while %visto < 10;
say keys %visto;      # -> (46 19 5 36 33 1 20 45 47 30)
```

Esta última versión es probable más concisa y más idiomática, pero esto significa que es mejor. Es perfectamente aceptable si prefieres la segunda o la primera versión porque la encuentras más clara. Usa la versión que desees, o tu propia versión modificada provisto que realice la tarea prevista. Esto es Raku, *hay más de una forma para hacer algo* (TIMTOWTDI).

Sin embargo, nota que la versión pura de hash no mantiene el orden en el cual los números son generados, así que la (pseudo)aleatoriedad podría no ser tan buena.

También nota que Raku tiene una función o método pick para elegir elementos de forma aleatorio de una lista sin repetición.

10.7 Las Claves de un Hash son Únicas

No es posible tener la misma clave en un hash más de una vez. El intento de mapear un nuevo valor a una clave reemplazará el valor antiguo con el nuevo. Aquí presentamos un ejemplo de la creación de un hash con claves duplicadas:

```
> my %amigos = (Gabo => 5, Miguel => 6, Salomé => 5, Gabo => 7, Jorge => 3)
Miguel => 6, Jorge => 3, Salomé => 5, Gabo => 7
```


Dado que dos de nuestros amigos se llaman “Gabo”, perdemos los datos asociados con el primero. Esto es algo con lo cual deberías ser cuidadoso: las claves de un hash son únicas, así que perderás algunos artículos si los datos asociados con tus claves tienen duplicados. La siguiente sección mostrará algunas maneras de tratar este posible problema.

Pero esta propiedad sobre la singularidad de una clave tiene muchos aspectos positivos. Por ejemplo, una manera típica de remover los duplicados de una lista de artículos es asignar los artículos de una lista a las claves de un hash (el valor no importa); al final del proceso, la lista de claves no contiene duplicados:

```
> my @array = < a b c d s a z a r e s d z r a >
[a b c d s a z a r e s d z r a]
> my %único = map { $_ => 1 }, @array;
a => 1, b => 1, c => 1, d => 1, e => 1, r => 1, s => 1, z => 1
> my @array_único = keys %único;
[z a e s d c b r]
```

Como puedes ver, los duplicados han sido removidos del array de salida. En ejemplos simples como este, la función integrada `unique` habría sido suficiente para remover los duplicados de `@array`, pero dentro de un programa más complejo, es bien común usar un hash (usualmente llamado `%visto`) para chequear si un valor ya se ha visto.

10.8 Los Hashes y los Arrays

Invertir un hash puede ser muy fácil si se conoce que los valores ocurren solamente una sola vez (es decir, son únicos). Considera por ejemplo un hash que mapea los meses a sus números en el año (limitamos el ejemplo a cinco meses por brevedad):

```
> my %meses = ene => 1, feb => 2, mar => 3, abr => 4, may => 5;
abr => 4, feb => 2, ene => 1, mar => 3, may => 5
```

Podemos transformar los pares clave-valor en un lista plana, invertir la lista, y asignar la lista inversa a un nuevo hash:

```
> my %meses_inv = %meses.kv.reverse;
1 => ene, 2 => feb, 3 => mar, 4 => abr, 5 => may
```

Ahora tenemos un nuevo hash que mapea los números de los meses a sus nombres. Esto puede ser útil si se conoce que un hash es biyectivo, pero esta estrategia no funciona correctamente si un valor aparece más de una vez: en tal caso, algunos pares se perderán:

```
> my %meses = ene => 1, enero => 1, feb => 2, febrero => 2;
feb => 2, febrero => 2, ene => 1, enero => 1
> my %meses_inv = %meses.kv.reverse;
1 => enero, 2 => febrero
```

Los arrays pueden aparecer como valores en un hash. Por ejemplo, si te dan un hash que mapea las letras a las frecuencias, podrías querer invertirlo; es decir, crear un hash que mapea las frecuencias a las letras. Dado que podría haber varias letras con las misma frecuencia, cada valor en el hash invertido debería ser un array de letras.

La siguiente es una función que invierte un hash similar:

```

sub invertir-hash( %hash-entrada ) {
    my %hash-salida;
    for %hash-entrada.kv -> $clave, $val {
        push %hash-salida{$val}, $clave;
    }
    return %hash-salida;
}

```

Cada paso a través del bucle, un artículo del hash se asigna a las variables `$clave` y `$val`, y `$clave` se añade al valor `%hash-salida` para la clave `$val`; si dicho valor no existe todavía, entonces es creado. Al final del proceso, los valores de `%hash-salida` son todos arrays anónimos.

Este es un ejemplo:

```

my %rev-hist = invertir-hash histograma 'parrot';
say %rev-hist;
dd %rev-hist;

```

Esto mostrará:

```

1 => [p a o t], 2 => [r]
Hash %rev-hist = {"1" => $["p", "a", "o", "t"], "2" => $["r"]}

```

Nota que la función `say` provee una representación simple de los datos del hash, y que la nueva función `dd` (abreviatura de “data dumper” – “vertedero de datos”) usada en este ejemplo provee información más detallada. `dd` no es comúnmente usada en programas normales, pero puede ser útil durante la depuración de un programa para mostrar una descripción detallada de una estructura de datos compleja.²

Por ejemplo, `%hash-salida` contiene dos artículos (dos pares) cuyos valores son arrays anónimos. Puedes acceder el segundo elemento del primer array usando el valor del hash `%rev-hist{"1"}` como si fuera un nombre de array ordinario, con esta simple sintaxis:

```

say %rev-hist{"1"}[1]; # -> a

```

La Figura 10.1 es un estado de diagrama que muestra a `%hist` y a `%rev-hist`. Un hash se representa como una caja con el tipo hash encima y los pares clave-valor dentro.

Los arrays pueden ser valores en un hash, como este ejemplo muestra, pero no pueden ser claves. Si intentas hacerlo, probablemente termines con una clave que contiene solo un elemento del array, pero no lo que esperabas:

```

my @a = 'a' .. 'c';
my %h;
%h{@a} = 5;
say %h; # -> a => 5, b => (Any), c => (Any)

```

²Para ser honesto, `dd` no es Raku estándar, es una característica específica de Rakudo. Una implementación futura de Raku que no esté basada en Rakudo podría no tenerla.



Fig. 10.1: Diagrama de estado.

Aquí, Raku interpretó la asignación `%h{@a} = 5;` como una asignación de rebanada, o sea, asumió que intentamos poblar tres artículos de una vez, una para cada elemento del array.

Como se mencionó anteriormente, un hash se implementa usando una función hash y significa que las llaves tienen que ser hashable³. Una **función hash** es una función que toma un valor (de cualquier tipo) y devuelve un número entero. Los hashes usan estos enteros, llamados valores de hash, para almacenar y buscar pares clave-valor.

Este sistema funciona bien si las claves son inmutables. Pero si las claves son mutables, como con los arrays, cosas extrañas pasarían. Por ejemplo, cuando creas un par clave-valor, Raku aplicaría una función hash a la clave y la almacenaría en la ubicación correspondiente. Si modificas la clave y después aplica la función hash nuevamente, entonces sería almacenada en una ubicación diferente. En ese caso, podrías tener dos entradas para la misma clave, o no podría ser capaz de encontrar una clave. En ambos casos, el hash no funcionaría correctamente.

Esa es la razón por la cual las claves deben ser hashable, y porqué los tipos mutables como los arrays no lo son. Así que Raku hará algo más que puede ser útil (tal como crear tres artículos de hash distintos en el ejemplo anterior), pero no aplicará la función hash al array.

Dado que los hashes son mutables, ellos no pueden ser usados como claves, pero *pueden* usarse como valores, así que puedes tener hashes anidados.

10.9 Memos

Si jugueteaste con la subrutina `fibonacci` en la Sección 5.8, probablemente notaste que mientras más grande el argumento, más tiempo toma la subrutina para ejecutarse. Además, el tiempo de ejecución aumenta extremadamente rápido.

Para entender el porqué, considera la Figura 10.2, la cual muestra la **gráfica de llamada** para `fibonacci` con `n=4`.

Una gráfica de llamada muestra un conjunto de cuadros de una subrutina, con líneas que conectan cada cuadro a los cuadros de las funciones que llamas. En la parte superior de la

³Esto no es del todo cierto. Las claves de un hash “normal” deben ser del tipo hashable y por lo tanto, inmutables. Hay otro tipo de hash, objetos hash, para los cuales la necesidad de tener claves inmutables no aplica.

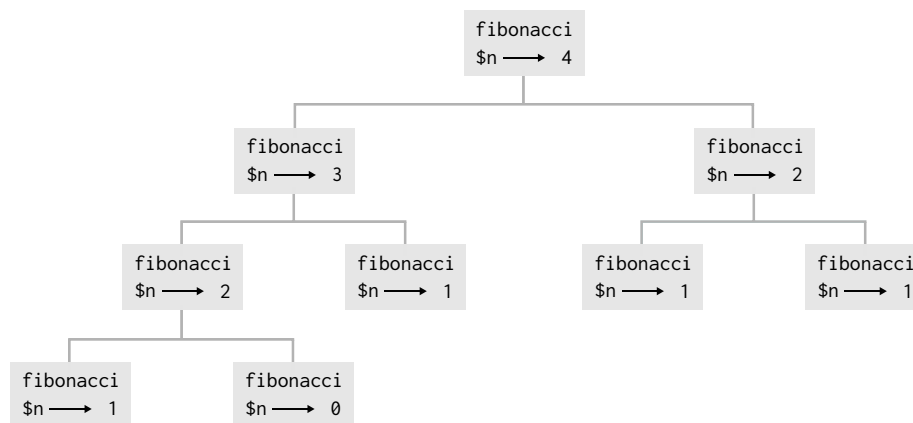


Fig. 10.2: Gráfica de llamada.

gráfica, fibonacci con $n=4$ llama a fibonacci con $n=3$ y con $n=2$. En cambio, fibonacci con $n=3$ llama a fibonacci con $n=2$ y a $n=1$, etcétera.

Cuenta cuantas veces fibonacci(0) y fibonacci(1) son llamadas. Esta es una solución ineficiente al problema, y se vuelve peor a medida que el argumento crece.

Una solución es mantener un registro de los valores que ya se han calculado al almacenarlos en un hash. Un valor que se ha computado y se almacena para uso posterior se conoce como un **memo**. Aquí presentamos una versión “memoizada” de fibonacci:

```

my %conocidos = 0 => 1, 1 => 1;
say fibonacci(10);
sub fibonacci( $n ) {
  return %conocidos{$n} if %conocidos{$n}:exists;
  %conocidos{$n} = fibonacci($n-1) + fibonacci($n-2);
  return %conocidos{$n};
}

```

%conocidos es un hash que mantiene un registro de los números Fibonacci que ya sabemos. Comienza con dos artículos: 0 y 1, los cuales mapean a 1.

Cuando se llama a fibonacci, la subrutina chequea a %conocidos. Si el resultado ya existe, lo devuelve inmediatamente. De lo contrario, tiene que computar el nuevo valor, agregar al hash, y devolverlo.

Si ejecutas esta versión de fibonacci y la comparas con la original, encontrarás que es mucho más rápida, especialmente para un argumento grande (por ejemplo, mayor que 30).

La memoización es una forma de *caché*, es decir, almacenar en memoria el resultado de una computación (presumiblemente costosa) para evitar computarla nuevamente. Este proceso es algunas veces llamado “negociación de memoria por ciclos de CPU”. En algunos casos, tal como nuestro ejemplo recursivo de Fibonacci, la ganancia es absolutamente inmensa: calcular el número Fibonacci 100^o tomaría billones de años con la subrutina recursiva original y toma solo segundos con la versión memoizada.

Nota que en el caso específico de la función Fibonacci, estamos almacenando valores para cada entero sucesivo; podría haber memoizado los números Fibonacci en un array en lugar de un hash (y podría ser hasta un poco más eficiente), pero usar un hash para tal propósito es una solución más general, lo cual funciona hasta cuando las claves del memo no son enteros consecutivos.

Como un ejercicio, intentar escribir la subrutina fibonacci usando un array en lugar de un hash para memoizar los números Fibonacci ya calculados.

10.10 Los Hashes como Tablas de Despacho

Puedes necesitar lanzar alguna acción dependiendo en el valor de un parámetro recibido por el programa. Para hacer eso, podrías usar una serie de sentencias `if {...} elsif {...} else {...}` como esta:

```
sub ejecuta-parar { ... };
sub ejecuta-iniciar { ... };
my $param = obtén-param();
if $param eq "parar" {
    ejecuta-parar;
} elsif $param eq "iniciar" {
    ejecuta-iniciar;
} elsif $param = "a" {
    say $ayuda;
} elsif $param = "ayuda" {
    say $ayuda;
} elsif $param = "v" {
    $verbose = True;
} else {
    die "Opción desconocida $param";
}
```

Esta estrategia es aburrida y sujeta a errores. El uso de una tabla de despacho es usualmente una solución más simple.

Una **tabla de despacho** es una estructura de datos que mapea identificadores a referencias de códigos u objetos de subrutinas. Si se aplica al escenario anterior, podría lucir así:

```
sub ejecuta-parar { ... };
sub ejecuta-iniciar { ... };
my %despacho = (
    parar => &ejecuta-parar,
    iniciar => &ejecuta-iniciar,
    a => { say $ayuda; },
    ayuda => { say $ayuda; },
    v => { $verbose = True; },
);
my $param = obtén-param();
die "Opción desconocida $param" unless %despacho{$param}:exists;
%despacho{$param}(); # ejecuta la acción especificada en %despacho
```

El hash `%despacho` define la acción dependiendo en el parámetro usado como una clave. La sentencia `%despacho{$param}()` llama la acción requerida.

Esta estrategia es un poco más concisa y más limpia, pero existen otras ventajas. Es más fácil de mantener: si necesitas agregar una opción, solo necesitas agregar una entrada al hash y evitas agregar código en el medio de una cadena complicada de sentencias `if {...} elsif {...} else {...}` lo cual podría estropear algo.

Otra ventaja es que la tabla de despacho puede modificarse dinámicamente durante el tiempo de ejecución, por ejemplo dependiendo en ciertas circunstancias externas (por ejemplo el día del mes cuando el programa se ejecuta) o de acuerdo a un archivo de configuración. Esto significa que es posible modificar el comportamiento de un programa dinámicamente después del tiempo de compilación, mientras todavía se ejecuta. Esto abre el camino a algunas técnicas avanzadas de programación que están fuera del alcance de este libro.

Nota que hemos usado hashes para nuestras tablas de despacho, y esta es la manera más común de implementarlas. Si hace sentido tener pequeños enteros como claves, podrías también implementar una tabla de despacho como un array. Este es el caso, por ejemplo, con los artículos numerados de un menú donde al usuario se le incita a teclear un número para indicar la opción de menú a activar.

10.11 Variables Globales

En el ejemplo anterior sobre Fibonacci memoizado, el hash `%conocidos` es creado fuera de la subrutina, así que pertenece al paquete principal entero. Tales variables son conocidas como **globales** porque pueden accederse desde cualquier otra función. A diferencia de variables lexicales (o “locales”), las cuales desaparecen cuando sus ámbitos terminan, las variables globales persisten de una llamada de subrutina a la siguiente.

Es común usar variables globales como **banderas** (**en. flags**); es decir, variables booleanas que indican si una condición es verdadera. Por ejemplo, algunos programas usan banderas nombradas `$verbose` para controlar el nivel de detalle en la salida de texto:

```
my $verbose = True;
sub ejemplo1 {
  say 'Ejecutando ejemplo1' if $verbose;
  # ...
}
```

Las variables globales son también usadas para variables de entorno y parámetros que se pasan al programa, como para almacenar una estructura de datos que es la pieza central de un programa, para evitar copiarla cuando se pasa como un argumento a las subrutinas.

Pero, a parte de esos casos específicos, es usualmente considerado mala práctica usar variables globales, porque crea dependencias y comportamientos de “acciones a distancia” inesperados entre varias partes de un programa y puede conducir a errores que son difícil de encontrar.

En el caso de nuestra subrutina fibonacci memoizada, el hash `%conocidos` es útil solo dentro de la subrutina. Podemos mejorar la implementación al usar el declarador `state` dentro de la subrutina:

```

say fibonacci(10);
sub fibonacci( $n ) {
    state %conocidos = 0 => 1, 1 => 1;
    return %conocidos{$n} if %conocidos{$n}:exists;
    %conocidos{$n} = fibonacci($n-1) + fibonacci($n-2);
    return %conocidos{$n};
}

```

El declarador `state` hace la variable local a la subrutina y persistente de una llamada de la subrutina a la otra: la línea de código con la sentencia `state` se ejecuta una sola vez (en la primera llamada de la subrutina) y el contenido de la variable, el hash `%conocidos` en este caso, se mantiene de una llamada a la siguiente.

10.12 Depuración de Programas

A medida que trabajas con conjuntos de datos (*datasets*) más grandes, puede volverse tedioso e inmanejable depurar con solo la impresión y chequeo de la salida manualmente. Aquí presentamos algunas sugerencias para depurar conjuntos de datos grandes:

Reduce la entrada Si es posible, reduce el tamaño de tu conjunto de datos. Por ejemplo, si el programa lee un archivo de texto, comienza con las primeras 10 líneas, o con el ejemplo más pequeño que puedas encontrar. Puedes editar los archivos o (mejor aún) modificar el programa para que lea solo las primeras `n` líneas.

Si hay un error, puedes reducir `n` al valor más pequeño que manifiesta el error, y después incrementarlo gradualmente a medida que encuentras y arreglas los demás errores.

Chequea los resúmenes y tipos En lugar de imprimir y chequear el conjunto de datos, considera imprimir resúmenes de los datos: por ejemplo, el número de artículos en un hash o el total de una lista de números.

Una causa común de los errores al tiempo de ejecución es un valor que no es del tipo correcto. Para depurar este tipo de error, es usualmente suficiente con imprimir el tipo de un valor (piensa sobre el método `.WHAT` o `~name`).

Es usualmente útil agregar tipos a tus variables. Donde espera una cadena de texto, asegúrate de escribir la variable o el parámetro de la subrutina con `Str`. Si espera un entero, escríbelo con `Int`. Si espera un `Int` de un cierto rango, crea un subconjunto para el mismo como en la Sección 5.9 (p. 85) y agrega ese tipo a la variable.

Escribe autochequeos Algunas veces puedes escribir código para chequear errores automáticamente. Por ejemplo, si estás calculando el promedio de una lista de números, podrías chequear que el resultado no sea mayor que el elemento más grande en la lista o menor que el elemento más pequeño. Esto se conoce como “comprobación de validez” porque detecta aquellos resultados que son “inválidos”.

Otro tipo de chequeo compara los resultados de dos computaciones diferentes para ver si son consistentes. Esto se llama “comprobación de consistencia”.

Formatea la salida Formatear la salida de un proceso de depuración puede facilitar la detección de errores. Vimos un ejemplo en la Sección 5.11. La función `dd` muestra detalles útiles de una estructura de datos compuesta o compleja.

Nuevamente lo repito: el tiempo que inviertes en andamiaje puede reducir el tiempo que pierdes en depuración.

10.13 Glosario

Mapeo Una relación en la cual cada elemento de un conjunto corresponde a un elemento de otro.

Hash Un mapeo de las claves a sus valores correspondientes.

Par clave-valor La representación de un mapeo de una sola clave a su valor.

Artículos En un hash, otro nombre para un par clave-valor.

Clave Un objeto que aparece en un hash como la primera parte de un par clave-valor.

Valor Un objeto en un hash que aparece como la segunda parte de un par clave-valor. Esto es más específico que nuestro uso previo de la palabra “valor”.

Implementación Una forma de realizar una computación.

Tabla hash El algoritmo usado para implementa hashes.

Función hash Una función usada por una tabla hash para computar la ubicación de una clave.

Hashable Un tipo que contiene una función hash. Los tipos inmutables como los números y las cadenas de texto son hashables; los tipos mutables como los arrays y los hashes no lo son.

Búsqueda Una operación de un hash que toma una clave y encuentra el valor correspondiente.

Búsqueda inversa Una operación de un hash que toma un valor y encuentra una o más claves que mapean al valor.

Gráfica de llamada Una diagrama que muestra cada marco creado durante la ejecución de un programa, con una flecha de cada función que hace la llamada a la función que se llama.

Memo Un valor que se calcula para evitar computaciones futuras innecesarias.

Memoizar Almacenar un valor calculado en la memoria para evitar cálculos posteriores del mismo. La memoización es una forma de caché.

Variable global Una variable definida fuera de cualquier subrutina u otro bloque. Las variables globales pueden accederse desde cualquier subrutina.

Bandera Una variable booleana que se usa para indicar si una condición es verdadera.

10.14 Ejercicios

Ejercicio 10.1. Escribe una subrutina que lea las palabras en `words.txt` y las almacene como claves de un hash. (No importa cuales sean los valores.) Después puedes usar el adverbio `exists` como una manera rápida de chequear si una cadena de texto se encuentra en el hash.

Si hiciste el Ejercicio 9.10, puedes computar la rapidez de esta implementación con un hash y la búsqueda binaria.

Solución: A.8.2.

Ejercicio 10.2. Memoiza la función Ackermann del Ejercicio 5.2 y observa si la memoización hace posible evaluar la subrutina con argumentos más grandes. Pista: no.

Solución: A.8.3.

Ejercicio 10.3. Si hiciste el Ejercicio 9.7, ya tienes una función llamada `tiene-duplicados` que toma una lista como un parámetro y devuelve `True` si cualquier objeto aparece más de una vez en la lista.

Usa un hash para escribir una versión más rápida y simple de `tiene-duplicados`.

Solución: A.8.4.

Ejercicio 10.4. Dos palabras son “pares rotativos” si puedes rotar una de ellas y conseguir la otra (ver `rotar-palabra` en Ejercicio 7.3) usando el cifrado César.

Escribe un programa que una lista de palabras (por ejemplo, `words.txt` y encuentre todos los pares rotativos).

Solución: A.8.5.

Ejercicio 10.5. Este es otro Puzzler de Car Talk (<http://www.cartalk.com/content/puzzlers>):

Esto fue enviado por una persona llamada Dan O’Leary. Recientemente, él encontró una palabra monosilábica de cinco letras que tiene la siguiente propiedad única. Cuando se remueve la primera letra, las letras restantes forman una homófona de la palabra original, es decir, una palabra que suena exactamente igual. Reemplaza la primera letra, es decir, colócala devuelta y remueve la segunda letra y el resultado es aún otra homófona de la palabra original. Y la pregunta es, cuál es la palabra?

Ahora te daré un ejemplo que no funciona. Miremos a la palabra de cinco letras, ‘`wrack`’. W-R-A-C-K, como en ‘`wrack with pain`’. Si remuevo la primera letra, me quedo con una palabra de cuatro letras, ‘`R-A-C-K`’. Como en, ‘`Holy cow, did you see the rack on that buck! It must have been a nine-pointer!`’ Es un homófono perfecto. Si pones la ‘`w`’ devuelta, y remueve la ‘`r`’, te quedas con la palabra ‘`wack`’, la cual es una palabra real, es solamente no una homófona de las otras dos palabras.

Pero existe, sin embargo, por lo menos una palabra que Dan y todos nosotros conocemos, que producirá dos homófonas si remueve una de las primeras dos letras para crear dos nuevas palabras de cuatro letras. La pregunta es, cuál es la palabra?

Puedes usar el hash del Ejercicio 10.1 más arriba para chequear si una cadena de texto se encuentra en `words.txt`.

Para chequear si dos palabras son homófonas, puedes usar el Diccionario de Pronunciación CMU. Puedes descargarlo de <http://www.speech.cs.cmu.edu/cgi-bin/cmudict>.

Escribe un programa que enumere todas las palabras en `words.txt` (o en el diccionario CMU) que resuelva el Puzzler.

Solución: A.8.6.

Caso Práctico: Selección de Estructura de Datos

A esta altura ya aprendiste sobre las estructuras de datos principales de Raku, y has visto algunos de los algoritmos que las usan.

Este capítulo presenta un caso práctico con ejercicios que te dejan pensar acerca de la elección de estructuras de datos y practicar al usarlas.

Pero primero, me gustaría discutir brevemente dos estructuras condicionales que hemos ignorado hasta ahora y proveer nuevas posibilidades sobre las signatures de una subrutina.

11.1 El Operador Condicional Ternario

Considera el siguiente código que verifica el valor de un número entero positivo:

```
my $result;  
if $num < 10 {  
    $result = "Un dígito";  
} else {  
    $result = "Más de un dígito";  
}  
say $result;
```

Esto es bien simple, pero un poco largo. Esto puede escribirse en una sola línea de código en la siguiente forma:

```
say $num < 10 ?? "Un dígito" !! "Más de un dígito";
```

El operador está conformado por dos partes: ?? y !!, las cuales separan tres expresiones (de ahí su nombre de “operador ternario”):

- La condición a ser evaluada (¿es \$num menor que 10?);
- La expresión que define el valor si la condición es verdadera;
- La expresión que define el valor si la condición es falsa.

Esta sentencia chequea si \$num es menor que 10 y, si esto es verdadero, imprime “Un dígito”; si la condición es falsa, entonces imprime “Más de un dígito”.

Realmente este operador no provee una nueva funcionalidad sino que ofrece una sintaxis más concisa para escribir una condicional.

Es posible anidar varios operadores ternarios para examinar múltiples opciones sucesivamente:

```
say $valor < 10    ?? "Un dígito"    !!
$valor < 100    ?? "Dos dígitos"    !!
$valor < 1000    ?? "Tres dígitos"  !!
                  "Más de tres dígitos";
```

Esta construcción es una forma de sentencia conocida algunas veces como una *sentencia de switch* porque el lenguaje C y muchos lenguajes derivados del mismo usan la palabra clave *switch* para describir una condicional con opciones múltiples.

Esto es mucho más conciso y usualmente más conveniente que las condicionales anidadas `if ... después ... else`, pero la siguiente sección provee un tipo de sentencia *switch* mucho más poderosa.

11.2 La Sentencia “Switch” `given ... when`

Raku tiene una sentencia “switch”, escrita con las palabras claves `given` y `when`. La palabra clave `given` introduce la variable o expresión que será puesta a prueba, y cada una de las sentencias `when` especifica una condición seguida por un bloque que se ejecutará si la condición es verdadera. Por defecto, el proceso termina cuando la primera condición es satisfecha.

El ejemplo de más arriba podría escribirse como sigue:

```
given $valor {
  when 0..9      { say "Un dígito" }
  when $_ < 99   { say "Dos dígitos" }
  when /\d**3$/ { say "Tres dígitos" }
  default       { say "Más de tres dígitos" }
}
```

La sentencia `given $valor` *topicaliza* el argumento, es decir, asigna el contexto de \$valor a la variable tópico \$_ (o, más precisamente, crea un sobrenombre del argumento a través de la variable \$_). El argumento de `given` es una variable simple en el ejemplo más arriba, pero puede ser una expresión compleja cuya evaluación se almacena en \$_. Cada una de las condiciones `when` es chequeada contra \$_. He escrito estas condiciones en tres formas sintácticas diferentes para ilustrar algunas de las varias posibilidades:

- La primera chequea contra \$_ (implícitamente) contra el rango 0..9.
- La segunda compara explícitamente \$_ con 99.
- La tercera usa un regex para chequear si \$_ tiene tres dígitos.
- La sentencia default se ejecuta solo si las otras condiciones han fallado.

Solo un mensaje será imprimido, porque el proceso de coincidencia termina tan pronto una de las condiciones ha sido satisfecha, y la cláusula default se ejecutará solo si las otras condiciones fallaron.

Si no hay un operador específico en la cláusula when, entonces realizará una coincidencia inteligente de la expresión en la la cláusula when contra \$_:

```
when $foo { ... }
# equivalente a: when $foo ~~ $_ { ... }
```

Nota que la palabra clave given no hace más que topicalizar su argumento para el resto del bloque. Las cláusulas when son las que hacen el trabajo pesado. De hecho, podrías usar las cláusulas when sin given provisto que asignes el valor correcto a \$_, lo cual, como podrás recordar, puede hacerse con un bloque for:

```
my $val = 7;
for $val {
    when 0..6 { say "menos que"}
    when 7 {say "Exacto";}
    when 8..* {say "mayor que";}
}
```

Es posible añadir una cláusula proceed al final de cualquiera de los bloques de código condicionales para prevenir que el proceso pare después que el bloque de código haya sido exitoso. Por ejemplo, podrías escribir esto:

```
given $valor {
    when 0..9      { say "Un dígito" }
    when 10..99   { say "Dos dígitos"; proceed }
    when 42       { say "La respuesta al sentido de la vida, el universo y todo lo demás" }
    when /\d**3$/ { say "Tres dígitos" }
    default      { say "Más de tres dígitos" }
}
```

En este ejemplo, si \$valor es 42, dos mensajes se imprimirán, "Dos dígitos" y "La respuesta al sentido de la vida, el universo y todo lo demás" porque la cláusula proceed en el segundo bloque de código evita que el proceso termine con la primera coincidencia exitosa.

Grandioso, pero hay un problema. La cláusula proceed debería usarse con cuidado, dado que puede fácilmente conducir a resultados inesperados. De hecho, *el código de más arriba es incorrecto*: si \$valor tiene dos dígitos pero no es 42 (si es, digamos, 43), el bloque default también se ejecutará, porque la única coincidencia exitosa tenía la cláusula proceed, y dirá que hay más "Más de tres dígitos" aunque esto es obviamente falso.

Como un ejercicio, prueba el código anterior con varios valores y intenta encontrar una manera de arreglar el error con la cláusula proceed.

Solución: A.9.1.

11.3 Condicionales Múltiples con Uniones

Algunas veces, necesitas probar varias posibles condiciones y escribes algo como lo siguiente:

```
if $value == 3 or $value == 5 or $value == 7 { ... }
```

Esto es un poco verboso, y no me gusta tener que repetir la parte `$value ==` varias veces. Preferías escribir algo como lo siguiente: “Si el valor es igual a 3, 5 o 7.” Esta sintaxis no funciona, pero las uniones te permiten hacer muy similar a lo anterior:

```
if $valor == 3|5|7 {
  say "$valor es 3, o 5, o 7"
}
```

La parte `3|5|7` es una unión.

Las uniones (**en**. junctions) son superposiciones de valores desordenados. Las operaciones sobre las uniones son ejecutadas por cada elemento de la unión separadamente (y hasta en paralelo), y los resultados se ensamblan en una unión del mismo tipo.

Los tipos de unión difieren cuando se evalúan en contexto booleano. Los tipos y los constructores asociados de las uniones son `any`, `all`, `one` y `none`.

Tipo	Constructor	Operador Infijo	Verdadero si ...
any	any		por lo menos un valor evalúa a True
all	all	&	no valor evalúa a False
one	one	^	exactamente un valor evalúa a True
none	none		no valor evalúa a True

Por ejemplo, `3|5|7` es lo mismo que `any(3, 5, 7)`.

Este es otro ejemplo de una unión `any`:

```
my Junction $días-laborables = any <lunes martes miércoles jueves viernes>
if $día eq $días-laborables {
  say "Este es un día laboral";
}
```

En este ejemplo, el operador `eq` se llama con cada par `$día`, ‘lunes’, `$día`, ‘martes’, etc., y el resultado se coloca en una unión `any` nuevamente. Tan pronto el resultado se determina (en este caso, tan pronto una comparación retorna True), se puede interrumpir la ejecución de las otras comparaciones.

Esto no solo funciona con los operadores, sino también con las subrutinas:

```
if 2 == sqrt(4 | 9 | 16) {
  say "YaY";
}
```

Las uniones pueden ser muy útiles para realizar tareas relativamente comunes. Imagina que tienes un array de números, y quieres verificar que todos ellos son no-negativos. En la mayoría de los lenguajes de programación, tendrías que crear un bucle para iterar sobre el array y chequear cada valor individualmente. En Raku, esto es bien simple:

```
my @valores = obtener-valores();
if all(@valores) >= 0 { ... }
```

O, si quieres evitar una excepción de división por cero:

```
my @valores = obtener-valores();
realizar-división(@valores) if none(@valores) == 0;
```

11.4 Subrutinas con Parámetros Nombrados y Opcionales

Las subrutinas que hemos visto hasta ahora han usado parámetros *de posición*, es decir, parámetros en los cuales la atadura (**en.** binding) con los argumentos de la subrutina depende en el orden de los mismos dentro de la lista de argumentos y en la signatura de la función. Usualmente esto es perfectamente normal cuando el número de argumentos pasado a la subrutina es pequeño (digamos, tres o menos).

Cuando la signatura de la subrutina se vuelve larga, el uso de argumentos de posición podría volverse complicado y propenso a errores.

11.4.1 Parámetros Nombrados

Los argumentos nombrados pueden suministrarse en cualquier orden: el nombre del parámetro está atado al argumento del mismo nombre. Por ejemplo:

```
sub dividir( :$dividendo, :$divisor where $divisor != 0 ) {
    return $dividendo/$divisor;
}
say dividir(dividendo => 2048, divisor => 128);      # -> 16
# o:
say dividir(divisor => 128, dividendo => 2048);      # -> 16
```

Los argumentos son suministrados en la llamada de la subrutina como una lista de pares usando la sintaxis de constructor de pares. En la signatura, los parámetros son extraídos en la signatura con la sintaxis de "colon-pair": el parámetro \$dividendo está atado al valor del par cuya clave es "dividendo" (2048), y \$divisor está similarmente atado a 128, sin importar el orden de los argumentos en la llamada de la subrutina.

Estos parámetros nombrados son especialmente útiles cuando el número de argumentos es grande. Por ejemplo, no hemos hablado de la programación orientada a objetos todavía (ver Capítulo 12), pero esto es como podríamos crear un objeto de la clase (definida por el usuario) Rectángulo:

```
my $rect = Rectángulo.new(
    origen_x => 100,
    origen_y => 200,
    ancho    => 23,
    longitud => 42,
    color    => 'negro'
);
```

Claramente, el uso de cinco parámetros de posición no sería práctico.

11.4.2 Parámetros Opcionales

Algunas veces, el número actual de argumentos no se conoce con antelación: por ejemplo, una subrutina puede llamarse con un número variable de argumentos. Tal subrutina se conoce como *variádica*. Puedes definir un parámetro para que sea opcional al colocar un signo de interrogación en frente del mismo en la signatura de la subrutina:

```
sub my-sub( $x, $y? ) { # $y es un simple parámetro opcional
  if $y.defined {
    say "El segundo parámetro ha sido suministrado y definido";
  } else {
    say "El segundo parámetro no ha sido suministrado";
  }
  # ...
}
```

Al usar parámetros de posición, los parámetros *opcionales* siempre tienen que ser los últimos en la lista (después de aquellos que son *mandatorios*).

Un parámetro puede también hacerse opcional al suministrar un **valor por defecto**:

```
sub mi-log( $número, $base = e ) { # e es una constante predefinida
                                   # $base es un parámetro opcional
  return log($número) / log($base);
}
say mi-log(4);           # Logaritmo natural (base e)  -> 1.38629436111989
say mi-log(32, 2);       # Logaritmo en base 2        -> 5
say mi-log(100, 10);     # Logaritmo común (base 10)  -> 2
```

Aquí, si el segundo argumento no se suministra, el valor por defecto (*e*) es usado. Inversamente, si hay un segundo argumento, este **anula** el valor por defecto.

En algunas ocasiones, tener parámetros opcionales o por defecto no es suficiente. Por ejemplo, la subrutina puede tener que procesar una lista que contiene un número indeterminado de valores. Para situaciones como esta, puedes usar un *parámetro slurpy*, es decir, un tipo de array colocado al final de la lista de parámetros que consumirá todos los argumentos restantes. Este tipo de parámetro slurpy usa el sigilo `*@`. En el ejemplo siguiente, la subrutina toma un parámetro mandatorio (el primer número de la lista) y un lista de argumentos adicionales que serán almacenados en el array `@resto`:

```
sub mi-suma( $primer-num, *@resto ) {
  say @resto;           # -> [3 4 5 12 17]
  return $primer-num + [+] @resto;
}
say mi-suma 1, 3, 4, 5, 12, 17;    # -> 42
```

Otros ejemplos de los parámetros slurpy se han proveído en la Solución: A.7.1.

11.5 Análisis de la Frecuencia de Palabras

Ahora, hablemos del caso práctico.

Como es habitual, deberías por lo menos intentar los ejercicios antes de leer las soluciones sugeridas, las cuales se proveen en las secciones siguientes de este capítulo.

Ejercicio 11.1. *Escribe un programa que lee un archivo, descompone cada línea en palabras, remueve los espacios en blanco y signos de puntuación de la palabra, y las convierte a palabras minúsculas.*

Ejercicio 11.2. *Dirígete al Project Gutenberg (<http://gutenberg.org>) y descarga tu copia de libro favorito en formato de texto simple.*

Modifica tu programa del ejercicio previo para que lea el libro que descargaste, salta sobre la información de la cabecera al principio del archivo, y procesa el resto de las palabras como lo hiciste anteriormente.

Después modifica el programa para que cuente el número total de palabras en el libro, y el número de veces que cada palabra es usada.

Imprime el número de palabras diferentes usadas en el libro. Compara libros diferentes por autores diferentes, escritos en eras diferentes. ¿Cuáles autores usan un vocabulario más extenso?

Ejercicio 11.3. *Modifica el programa del ejercicio anterior para que imprima las 20 palabras más usadas frecuentemente en un libro dado.*

Ejercicio 11.4. *Modifica el programa anterior para que lea una lista de palabras (ver Sección 8.2) y después imprima todas las palabras en el libro que no están en la lista de palabras. ¿Cuántas de ellas son errores tipográficos? ¿Cuántas de ellas son palabras comunes que deberían estar en la lista de palabras, y cuántas de ellas son realmente recónditas?*

11.6 Números Aleatorios

Dada la misma entrada, la mayoría de programas de computadoras generan la misma salida cada vez y, por lo tanto, se dice que son **deterministas**. El determinismo es usualmente algo bueno, dado que esperamos que la misma computación produzca el mismo resultado. No obstante, para algunas aplicaciones, queremos que la computadora sea impredecible. Los juegos de computadora son un ejemplo obvio, pero existen más.

Crear un programa verdaderamente no determinista resulta ser algo difícil, pero hay algunas maneras de hacerlo parecer por lo menos no determinista. Una de ellas es usar algoritmos que generan números **pseudo-aleatorios**. Los números pseudo-aleatorios no son verdaderamente aleatorios porque son generados a través de una computación determinista, pero solo con mirarlos es imposible distinguirlos de los números aleatorios.

Raku provee funciones tales como `rand` que generan números pseudo-aleatorios (los cuales llamaremos simplemente números “aleatorios” de aquí en adelante).

La función `rand` devuelve un número aleatorio (del tipo `Num`) entre 0.0 y 1.0 (incluyendo a 0.0 pero no a 1.0). Cada vez que llamas a `rand`, obtienes el número siguiente en una serie larga. Para ver una muestra, ejecuta este bucle en el REPL:

```
> say rand for 1..5;
```

Cuando se usa como un método, `rand` devuelve un número aleatorio entre 0.0 y el valor del invocante. Por ejemplo, `10.rand` devuelve un número aleatorio entre 0 y 10 (excluyendo al 10). Podrías intentarlo como un programa de una sola línea:

```
$ raku -e 'say 10.rand for 1..5'
8.23987158729588
9.83276889381497
```



```
2.52313276833335
3.44713459548771
1.82329894347025
```

Con suerte, deberías obtener una salida diferente a la que obtuve. Si quieres ejecutar un programa de una sola línea en Windows, recuerda que necesitarás reemplazar las comillas simples con comillas dobles.

Para obtener números enteros aleatorios entre 1 y 10, podrías usar los métodos `Int` y `rand`:

```
$ raku -e 'say 10.rand.Int + 1 for 1..5'
5
10
1
6
3
```

La función o método `pick` toma un número `$n` y una lista como argumentos y devuelve `$n` artículos escogidos aleatoriamente y sin repetición. Por ejemplo:

```
> say <1 3 4 5 7 9 10 12 14 42>.pick(5);
(5 42 3 4 7)
> say pick 5, <1 3 4 5 7 9 10 12 14 42>;
(42 12 5 1 9)
```

Si `$n` es mayor o igual al número de artículos de la lista, entonces todos los elementos de la lista serán devueltos en una secuencia aleatoria.

Para obtener enteros aleatorios únicos en un rango, podrías usar `pick` en un rango:

```
> say pick 5, 1..20;
(5 3 6 18 7)
> say (1..20).pick(5);
(20 4 18 2 7)
```

Si no especificas la cantidad de números aleatorios, obtendrás un solo número aleatorio:

```
> say (1..20).pick;
19
```

Ejercicio 11.5. *Escribe una función nombrada `escoge_del_hist` que toma un histograma como fue definido en la Sección 10.3 y devuelve un valor aleatorio del histograma, elegido con una probabilidad proporcional a la frecuencia. Por ejemplo, para tres artículos: ('a', 'a', 'b'), tu función debería devolver `a` con una probabilidad de 2/3 y `b` con una probabilidad de 1/3.*

11.7 Histograma de Palabras

Deberías intentar solucionar el ejercicio anterior antes de proceder.

Para el propósito de presentar las soluciones a los ejercicios anteriores, he usado el texto sencillo de *Don Quijote* (1605 y 1615), una novela escrita por Miguel de Cervantes, descargada desde el proyecto Gutenberg (<https://www.gutenberg.org/files/2000/2000-8>).

txt) y guardada en un archivo llamado *quijote.txt* ¹. Usa el mismo texto si quieres comparar tus soluciones y los resultados con los míos.

Nota del traductor: En *Think Raku*, el autor usa el texto de la novela *Emma*, escrita por Jane Austen². He decidido usar un libro en español porque creo que es más ameno y porque quería saber cuantas palabras tenía el libro *Don Quijote de la Mancha*.

Aquí está el programa que lee el archivo *quijote.txt* y construye un histograma de las palabras que pertenecen al libro en el archivo:

```
my %histograma;
my $saltar = True; # bandera para saltar la cabecera y
                   # marcar el final del libro

sub procesar-línea( Str $línea is copy ) {
    if defined index $línea, "*** START OF THIS PROJECT" {
        $saltar = False ;
        return;
    }

    return if $línea ~~ / Abela || anonymous /;
    return if $saltar;
    $línea ~~ s:g/<['-]>/ /; # Reemplazar guiones y apóstrofes con espacios
    $línea ~~ s:g/<[:,;,:;!¿?«»>()"_`>///; # Remover signos de puntuación
    $línea = $línea.lc; # Convertir cadena a minúscula
    for $línea.words -> $palabra {
        %histograma{$palabra}++;
    }

    $saltar = True if $línea ~~ /^fin$/; # Marcar el final del libro
}
procesar-línea $_ for "quijote.txt".IO.lines;
```

El programa lee cada línea del archivo *quijote.txt* y, por cada línea, llama a la subrutina *procesar-línea*.

La subrutina *procesar-línea* salta las líneas de la cabecera (o sea, todas las líneas hasta que se encuentra la línea que contenga la cadena de texto "*** START OF THIS PROJECT"). De igual manera, también salta cualquier línea que contenga las palabras "Abela" o "anonymous". La función reemplaza los guiones y los apóstrofes con espacios en blanco, remueve varios caracteres de puntuación, y convierte la línea a minúscula. Después, divide la línea en palabras individuales que son almacenadas y contadas con un acumulador en el hash *%histograma*. Al final, asigna verdadero a *\$saltar* si la línea contiene solo la palabra "fin", la cual marca el final del texto.

Para saber si el programa está haciendo lo que esperamos que haga, podemos mostrar unas entradas del hash *%histograma*:

¹Si tienes algún problema con el archivo, puede deberse a cambios hecho al formato del mismo por el proyecto Gutenberg. Para evitar tal problema asociado con cualquier cambio (y otros cambios futuros), puedes descargar el archivo original desde este repositorio de Gitlab: <https://gitlab.com/uzluisf/piensa-raku/tree/master/suplementos/quijote.txt>

²Puedes encontrar el archivo en este repositorio: https://github.com/LaurentRosenfeld/think_raku/blob/master/Supplementary/emma.txt. De igual manera, puedes consultar *Think Raku* para comparar tus resultados con los del autor si decides usar este texto.

```
# mostrando 20 líneas del histograma
my $cuenta;
for %histograma -> $par {
  say sprintf "%-24s %d", $par.key, $par.value;
  $cuenta++;
  last if $cuenta > 20;
}
```

Esto imprime lo siguiente:

ahorcase	1
descontente	1
regüeldos	1
libras	5
manteca	1
suerte	141
voy	51
desnudaba	1
xxviii	2
benevolencia	2
vieren	3
rebenque	1
asentado	2
profetizadas	2
mirábalas	1
lata	4
cesaría	1
preguntalle	1
bailado	2
elogio	2
codicio	1

Para contar el número total de palabras (pertenecientes al libro) en el archivo, podemos añadir los valores en el histograma:

```
my $total_palabras = [+] %histograma.values;
say "Hay $total_palabras palabras en el libro.";
```

El número de palabras diferentes es solo el número de artículos en el hash:

```
my $palabras-distintas = %histograma.elems;
say "Hay $palabras-distintas palabras distintas en el libro.";
```

Nota que podrías reducir lo anterior a una línea de código al interpolar un bloque de código dentro de la cadena de texto de salida:

```
say "Hay {%histograma.elems} palabras distintas en el libro.";
```

Y los resultados:

```
Hay 381215 palabras en el libro.
Hay 22950 palabras distintas en el libro.
```

11.8 Palabras Más Comunes

Para encontrar las palabras más comunes en `quijote.txt`, podemos ordenar el hash `%histograma` de acuerdo a los valores (frecuencia de palabras) y extraer las 10 palabras más frecuentes en un array.

```
my @más_comunes = (sort { %histograma{$^b} cmp %histograma{$^a} },
                  %histograma.keys)[0..9];
say $_ for map { "$_ \t%histograma{$_} "}, @más_comunes;
```

Las funciones `sort` recibe las claves del histograma y su función de comparación compara los valores asociados con esas claves. Dado que usamos la clave `$_b` antes que la clave `$_a`, `sort` producirá un ordenamiento en orden reverso (descendente). El procedimiento `sort` completo está colocado dentro de paréntesis, para que el rango de subíndices `[0..9]` actúe como una rebanada sobre la lista producida por `sort` y retenga solo las primeras 10 palabras más frecuentes. El resultado se almacena en el array `@más_comunes`. La siguiente línea de código solo procesa el array para mostrar las palabras y sus frecuencias respectivas.

Esto muestra la siguiente salida:

que	20626
de	18214
y	18188
la	10363
a	9823
en	8241
el	8210
no	6335
los	4748
se	4690

Si quieres ver más palabras interesantes, podrías, como un ejercicio adicional, filtrar el histograma y retener solo las palabras que tienen más de, digamos, cuatro letras.

El array temporario `@más_comunes` no es realmente necesario. Podríamos hacer todo el proceso con una sola sentencia:

```
say $_ for map { "$_ \t%histograma{$_} "},
              (sort { %histograma{$^b} cmp %histograma{$^a} },
               %histograma.keys)[0..9];
```

Este es un ejemplo de la programación de tuberías. Este tipo de sentencia necesita leerse de abajo hacia arriba y de derecha a izquierda. El primer paso es `%histograma.keys`, el cual produce una lista de las claves del histograma; esta lista se suministra a la sentencia `sort` para producir una lista de las llaves ordenadas (en orden descendente) de acuerdo a sus valores; una vez que esta parte entre los paréntesis se ha completado, el rango de subíndices `[0..9]` retiene las 10 palabras más frecuentes y las administra a la sentencia `map`, la cual produce la lista de palabras y las frecuencias para la salida final.

Déjame agregar una palabra de advertencia: ordenar el histograma por los valores y escoger los 10 más frecuentes para obtener las palabras más frecuentes es probablemente la

manera más fácil de resolver el problema y el código más corto para hacerlo. Esta es la razón por la cual he usado esta solución aquí. Pero no es la solución más eficiente desde el punto de vista del tiempo de ejecución, porque involucra el costo de ordenar un conjunto de datos relativamente grande, donde solo usamos una parte pequeña de los datos ordenados. Hay algoritmos mejores para hacer eso desde el punto de vista de la eficiencia de ejecución, pero son más complicados. Por lo tanto, hay un sacrificio entre la eficiencia de código y el rendimiento. Asumiendo que esto es código que queremos ejecutar solamente una vez, he favorecido la eficiencia de código.

11.9 Parámetros Opcionales

Vimos anteriormente en este capítulo que las subrutinas pueden tomar parámetros opcionales. Podemos usar esta funcionalidad para escribir una subrutina que imprime las palabras más comunes en el hash `%histograma` extraído desde *quijote.txt*.

```
mostrar-más-comunes(%histograma, 5);

sub mostrar-más-comunes( %hist, Int $num = 10 ) {
    say $_ for map { "$_ \t%hist{$_} " },
        (sort { %hist{$_b} cmp %hist{$_a} },
         %hist.keys)[0..$num - 1];
}
```

Esto mostrará las 5 palabras más comunes de la lista más arriba (Sección 11.8). Si la llamamos sin el segundo parámetro:

```
mostrar-más-comunes(%histograma);
```

obtendremos las 10 palabras más comunes (misma lista como la que mostramos en la Sección 11.8), porque el valor por defecto para el segundo parámetro es 10.

11.10 Diferencia de Hashes

Encontrar las palabras en *quijote.txt* que no están en la lista de palabras *lemario.txt* es un problema que podrías reconocer como una diferencia de conjuntos; es decir, queremos encontrar todas las palabras en un conjunto (las palabras en *quijote.txt*) que no están en el otro (las palabras en *lemario.txt*).

La subrutina *sustraer* toma los hashes `%princ` y `%dicc` y devuelve un nuevo hash que contiene todos las claves de `%princ` que no están en `%dicc`. Dado que no nos importan los valores, asignamos 1 a todas la claves.

```
sub sustraer( %princ, %dicc ) {
    my %diferencia;
    for %princ.keys -> $palabra {
        %diferencia{$palabra} = 1 unless %dicc{$palabra}:exists;
    }
    return %diferencia;
}
```

Para encontrar las palabras en *quijote.txt* que no están en *lemario.txt*, necesitamos cargar la lista de palabras en un hash y llamar a *sustraer*, pasando los dos hashes como argumentos. También agregamos algunas líneas de código para imprimir las primeras 20 palabras que no se encuentran en la lista de palabras:

```
my %lista-palabras = map { $_ => 1 }, "lemario.txt".IO.lines;
my %palabras-desconocidas = subtract(%histograma, %lista-palabras);
say %palabras-desconocidas.keys.head(20);
```

Nota que en lugar de usar una *rebanada*, he usado el método *head* para imprimir las primeras 20 palabras de la lista. Esta es solo otra manera de obtener los primeros “n” artículos de una lista o un array. También existe un método *tail* que extrae los últimos “n” artículos de una lista o un array.

Aquí están algunos de los resultados de *Don Quijote*:

```
(ahorcase descontente regüeldos manteca suerte voy desnudaba
xxviii benevolencia vieren rebenque asentado profetizadas
mirábalas lata cesaría preguntalle bailado elogio codicio)
```

Algunas de estas palabras son nombres y números romanos. Otras son raras y posiblemente no en uso. Otras son verbos conjugados. ¡Pero algunas son palabras comunes que deberían estar en la lista!

Nota que aquí he usado un hash (*%palabras-desconocidas*) para almacenar las palabras de *quijote.txt* que no se encuentran en la lista de palabras. Dado que estamos utilizando los datos solo para imprimir una muestra de 20 palabras, podríamos haber usado un array.

11.11 Construir Operadores Nuevos

Aprender sobre la diferencia de sustracción es una oportunidad excelente de introducir una funcionalidad muy interesante de Raku: la capacidad de construir operadores nuevos o redefinir aquellos existentes.

Dado que estamos sustrayendo dos listas de palabras, es muy tentador usar el signo de menos para denotar esta operación de sustracción. Es muy fácil crear tal operador en Raku:

```
multi sub infix:<-> (%princ, %dicc) {
    my %diferencia;
    for %princ.keys -> $palabra {
        %diferencia{$palabra} = 1 unless %dicc{$palabra}:exists;
    }
    return %diferencia;
}
```

Comparada con la definición de la subrutina *sustraer*, la única diferencia se encuentra en la línea cabecera. Discutiremos los detalles en un capítulo más adelante, pero los explicaremos brevemente aquí. La mayoría de los operadores de Raku son definidos como subrutinas “multis”, es decir, subrutinas que pueden ser definidas varias veces con diferentes firmas y cuerpos; Raku sabrá cual debe usar dependiendo en la firma. Aquí definimos el operador de menos como una subrutina *multi* cuyos parámetros son dos hashes;

esto será suficiente para que el compilador de Raku sepa que no queremos usar la sustracción regular entre dos valores numéricos, pero algo más que aplica a dos hashes. El operador de menos está definido como un operador “infijo”, lo cual significa que el operador será colocado entre sus dos operandos.

Llamar este operador es tan fácil como llamar el operador regular de sustracción entre dos números; solo necesitamos usar dos hashes como operandos:

```
my %palabras-desconocidas = %histograma - %lista-palabras;
```

El resto del programa funciona como antes.

Esta facilidad para crear operadores nuevos es una de las facilidades ofrecidas por Raku para *extender el lenguaje* desde sí mismo. Regresaremos a eso y otras maneras de extender el lenguaje en los capítulos siguientes.

Como un ejercicio, escribe una subrutina multi que crea un nuevo operador “!” para calcular el factorial de un entero:

```
say 5!;      # -> 120
```

También intenta pensar cómo probarías este nuevo operador “!” con varios valores de entrada.

Solución: A.9.2.

11.12 Sets, Bags y Mixes

Raku tiene una variedad de tipos de estructuras de datos llamadas Set, Bag y Mix que proveen muchas operaciones de conjuntos comunes. Estas estructuras son colecciones desordenadas de artículos únicos y con frecuencias. Dichas estructuras son inmutables (pero también hay versiones mutables de estas estructuras de datos, SetHash, BagHash y MixHash).

Puedes crear y usar un conjunto (set) en la siguiente forma:

```
> my $s = set <banana manzana naranja naranja banana pera manzana>;
set(banana, naranja, manzana, pera)
> say $s.raku
set("banana", "naranja", "manzana", "pera")
> say $s.elems;
4
> say $s{'manzana'}
True
> say $s{'cereza'}
False
```

Como puedes ver, los elementos duplicados han sido removidos. Los conjuntos solo nos dicen si al menos un elemento de un nombre dado se ha visto.

Un bag, al contrario, también mantiene un registro del número de cada artículo que se ha visto:

```
> my $b = bag <banana manzana naranja naranja banana pera manzana>;
Bag(banana(2), naranja(3), pera, manzana(2))
> say $b{'banana'}
2
```

Los mixes son similares a los bags, excepto que el peso de los elementos no tiene que ser entero.

La cosa interesante acerca de estas colecciones es que pueden usar muchas de las operaciones comunes de conjuntos usadas en matemáticas, tales como el operador de membresía de conjunto $\in$ (o usa `(elem)` si no sabes como teclear $a \in$ en tu editor ³), el operador de intersección de conjunto $\cap$ o `(&)`, y el operador de subconjunto $\subset$ o `(<)`:

```
> say "Encontrado!" if 'manzana' (elem) $s;
Encontrado!
> say "Es un subconjunto!" if <naranja banana> (<) $s;
Es un subconjunto!
> say <naranja banana piña> (&) $s;
set(banana, naranja)
```

Nota que no hemos usado la palabra clave `set` para definir la lista `<naranja banana>` en el segundo ejemplo más arriba. Esta lista ha sido coaccionada en un `Set` para chequear si era un subconjunto del conjunto `$s`. Esta es una de las cosas grandiosas de estas colecciones: la mayoría de estos operadores de conjuntos pueden usarse con listas, arrays, y hashes.

Podemos escribir el programa de la diferencia de hashes usando un conjunto para la lista de palabras y el operador de membresía de conjunto $\in$ (o `(elem)`):

```
my %histograma;
my $saltar = True; # bandera para saltar la cabecera
sub procesar-línea( Str $línea is copy ) {
  # (igual que antes)
}

procesar-línea $_ for "lemario.txt".IO.lines;
my $lista-palabras = set "lemario.txt".IO.lines;
my %palabras-desconocidas = sustraer(%histograma, $lista-palabras);
say %palabras-desconocidas.keys.head(20);

sub sustraer( %princ, $dicc ) {
  my %diferencia;
  for %princ.keys -> $palabra {
    %diferencia{$palabra} = 1 unless $palabra (elem) $dicc;
  }
  return %diferencia;
}
```

La línea de código en el bucle `for` podría también escribirse como sigue:

³No puedo enseñarte como teclear estos caracteres, porque cada editor requiere métodos diferentes.


```

        %diferencia{$palabra} = 1 unless $palabra ∈ $dicc;
#o:   %diferencia{$palabra} = 1 if $palabra ∉ $dicc;
#o:   %diferencia{$palabra} = 1 if $palabra !(elem) $dicc;

#o hasta con el operador (cont) o ∋:
        %diferencia{$palabra} = 1 unless $dicc (cont) $palabra;
#o:   %diferencia{$palabra} = 1 unless $dicc ∋ $palabra;
#o:   %diferencia{$palabra} = 1 if $dicc ∄ $palabra;
# etc.

```

Los operadores integrados $\setminus$ ⁴ y $(-)$ proveen la diferencia de conjunto, así que no tenemos ni que escribir una subrutina sustraer ni construir nuestro propio operador de menos:

```

procesar-línea $_ for "quijote.txt".IO.lines;
my $lista-palabras = set "lemario.txt".IO.lines;
my $palabras-desconocidas = %histograma.keys (-) $lista-palabras;
say $palabras-desconocidas.keys.head(20);

```

Hay más de 30 operadores de conjuntos disponibles, que cubren la mayoría de los operadores de conjuntos usados en las matemáticas. Solo he mostrado aquellos que pueden ser más útiles. Chequea la documentación oficial (<https://doc.raku.org/language/setbagmix>) si necesitas otros operadores.

Como un ejercicio, puedes escribir la subrutina `procesar-línea` de nuevo o reemplazarla para usar un set o un bag en lugar de un hash para almacenar las palabras de *quijote.txt* (y posiblemente adaptar el resto del programa donde sea necesario), para encontrar las palabras de *quijote.txt* que no se encuentran en *lemario.txt*.

Solución: A.9.3.

11.13 Palabras Aleatorias

Para elegir palabras aleatorias desde el histograma, el algoritmo más simple es construir una lista con copias múltiples de cada palabra, de acuerdo a la frecuencia observada, y después escoger elementos desde la lista con la función `pick`:

```

my @array = map { | (.key xx .value) }, %histograma;
say pick 30, @array;

```

La expresión `map { | (.key xx .value) }` lee cada par del histograma y crea una lista con copias `.value` de la cadena en `.key`. La función `pick` selecciona 30 palabras aleatorias desde el array.

Este algoritmo funciona, pero no es muy eficiente; cada vez que eliges una o más palabras aleatorias, se reconstruye la lista, la cual es tan grande como el libro original. Una mejora obvia es construir la lista una sola vez y después hacer selecciones múltiples, pero la lista es aún grande.

Una alternativa es:

⁴Ten presente que este es el carácter unicode 2216, no la barra invertida $\backslash$

1. Usar `keys` para conseguir una lista de palabras en `quijote.txt`.
2. Construir una lista que contiene la suma acumulativa de las frecuencias de las palabras (ver Ejercicio 9.2). El último artículo en esta lista es el número total de palabras en el libro, n .
3. Elegir un número aleatorio entre 1 y n . Usa la búsqueda binaria (ver Ejercicio 9.10) para encontrar el índice donde el número aleatorio sería entonces insertado en la suma acumulativa.
4. Usar el índice para encontrar la palabra correspondiente en la lista creada recientemente.

Ejercicio 11.6. *Escribe un programa que usa este algoritmo para elegir una palabra aleatoria desde `quijote.txt`.*

Solución: A.9.4.

Ten presente que Raku ofrece un atajo para realizar la tarea a mano: cuando se usa en un bag, la función `pick` devuelve una selección aleatoria de elementos, ponderados por los valores correspondientes a cada clave. Idealmente, deberíamos haber usado un bag en lugar de un hash para almacenar nuestro histograma `%histo`, pero no sabíamos nada sobre los bags en aquel entonces. Podemos, sin embargo, construir un bag desde el histograma `%histo`. Considera el siguiente ejemplo:

```
> my %histo = ( banana => 5, pera => 1, naranja => 12 );
{banana => 5, naranja => 12, pera => 1}
> my $canasto-frutas = bag map { $_ xx %histo{$_} }, %histo.keys;
Bag(banana(5), naranja(12), pera)
> for 1..10 { say $canasto-frutas.pick: 5}
(banana naranja naranja naranja naranja)
(naranja naranja banana naranja banana)
(naranja naranja banana naranja naranja)
(pera naranja banana banana naranja)
(naranja banana naranja naranja naranja)
...
```

Como puedes observar, el artículo más común, “naranja”, ha sido elegido más veces que los otros, y el menos común, “pera”, no ha sido elegido hasta el cuarto sorteo.

Como un ejercicio, podrías querer adaptar este código para elegir palabras aleatorias desde `lemario.txt`.

11.14 Análisis de Markov

Si eliges una palabra desde `quijote.txt` aleatoriamente, puedes ser que obtengas un sentido del vocabulario, pero probablemente no obtendrás una oración:

```
dicho libro del hidalgo original la escribir cervantes a el mancha
```

Una serie de palabras aleatorias raramente hace sentido porque no existe ninguna relación entre las palabras sucesivas. Por ejemplo, en una oración real en español esperas que un

artículo definido como “la” aparezca en frente de un sustantivo y que esté de acuerdo en género y número. Sin embargo, no esperas que aparezca en frente de un verbo o adverbio.

Una manera de medir estos tipos de relaciones es través del análisis de Markov, el cual caracteriza la probabilidad de las palabras que vienen después para una secuencia de palabras dadas. Es decir, la probabilidad de que ciertas palabras aparezcan después depende solamente en las palabras anteriores. Por ejemplo, el segundo capítulo de *El Principito* (1943), la novela famosa escrita por el escritor francés y aviador pionero Antoine de Saint-Exupéry, comienza:

Viví así, solo, sin alguien con quien poder hablar verdaderamente, hasta hace seis años cuando tuve una avería en el Sahara. Algo se había estropeado en el motor de mi avión. Como viajaba sin mecánico ni pasajero alguno, me dispuse a realizar yo sólo, una reparación difícil. Era para mí una cuestión de vida o muerte pues apenas tenía agua pura como para ocho días.

La primera noche me dormí sobre la arena, a unas mil millas de distancia del lugar habitado más próximo. Estaba más aislado que un naufrago en medio del océano. Imagínense, pues, mi sorpresa cuando al amanecer me despertó una vocecita que decía:

- ¡Por favor... píntame un cordero!

- ¿Eh?

- ¡Píntame un cordero!

Me puse en pie de un brinco y frotándome los ojos miré a mí alrededor. Descubrí a un extraordinario muchachito que me observaba gravemente. Ahí tienen el mejor retrato que más tarde logré hacer de él, aunque reconozco que mi dibujo no es tan encantador como el original. La culpa no es mía, las personas mayores me desanimaron de mi carrera de pintor a la edad de seis años, cuando sólo había aprendido a dibujar boas cerradas y boas abiertas.

Miré, fascinado, aquella aparición. No hay que olvidar que me encontraba a unas mil millas de distancia de cualquier región habitada y el muchachito no parecía ni perdido, ni muerto de cansancio, de hambre, de sed o de miedo. No tenía la apariencia de un niño perdido en el desierto a mil millas de distancia del lugar habitado más próximo. Cuando logré, por fin, poder hablar, pregunté:

- Pero... ¿qué haces tú aquí?

Y él repitió suave y lentamente, como algo muy importante:

- ¡Por favor... píntame un cordero!

Cuando el misterio es tan impresionante, uno no se atreve a contravenir. Por absurdo que aquello pareciera, a mil millas de distancia de algún lugar habitado y en peligro de muerte, saqué del bolsillo una hoja de papel y una pluma fuente. Recordé que yo había estudiado geografía, historia, cálculo y gramática y le dije al muchachito (algo malhumorado) que no sabía dibujar.

- No importa, ¡Píntame un cordero!

Como nunca había dibujado un cordero, repetí uno de los dos únicos dibujos que era capaz de realizar: el de la boa cerrada. Y quedé absorto al oírle decir:

- ¡No, no! No quiero un elefante dentro de una serpiente. La serpiente es muy peligrosa y el elefante ocupa mucho sitio. En mi tierra todo es muy pequeño. Necesito un cordero.

En este texto, la palabra “píntame” es siempre seguida por la palabra “un”, y “píntame un” es a su vez siempre seguida por “cordero”. Y la frase “unas mil” es siempre seguida por “millas”, pero la frase “unas mil millas” a su vez puede ser seguida por “de distancia del lugar habitado más próximo” o “de distancia de cualquier región habitada”.

El resultado del análisis de Markov es un mapeo de cada prefijo (como “píntame un” y “unas mil millas”) a todos los sufijos posibles (como “un cordero” y “de distancia del lugar habitado más próximo” o “de distancia de cualquier región habitada”).

Dado este mapeo, puedes generar un texto aleatorio al comenzar con cualquier prefijo y eligiendo aleatoriamente de los sufijos posibles. Después, puedes combinar el final del prefijo y el sufijo nuevo para formar el siguiente prefijo, y repetir.

Por ejemplo, si comienzas con el prefijo “píntame”, entonces la siguiente palabra tiene que ser “un” dado que la siguiente palabra es siempre “un” en el texto. Si el prefijo es “unas mil millas”, el sufijo siguiente podría ser “de distancia del lugar habitado más próximo” o “de distancia de cualquier región habitada”.

En este ejemplo, la longitud de los prefijos son dos o tres palabras pero puedes hacer un análisis de Markov con un prefijo de cualquier longitud.

Ejercicio 11.7. *Análisis de Markov:*

1. *Escribe un programa que lee un texto desde un archivo y realiza un análisis de Markov. El resultado debería ser un hash que mapea los prefijos a una colección de sufijos posibles. La colección podría ser una array, un hash, un set, un bag, etc.; hacer una elección apropiada depende de ti. Puedes probar tu programa con prefijos de longitud dos, pero sería bueno escribir el programa en una manera que facilite el proceso de intentar otras longitudes.*
2. *Agrega una función al programa previo para generar un texto aleatorio basado en el análisis de Markov. Aquí presentamos un ejemplo de Don Quijote con prefijo de longitud 2:*

o de ningún efecto tus fuerzas ni han de bastar todos los demás hicieron lo que le hizo volar por los rincones de los ojos cerrados y así en las florestas y despoblados buscando las aventuras que le había escrito el papel escrito y así creo que cuando entrasen en alguna fuerza y así le vio dijo ¡bendito sea el caballero la hallará con la cabeza

Para este ejemplo, la puntuación se ha dejado adjunta a las palabras. El resultado es casi sintácticamente correcto, pero no tanto. Semánticamente, casi hace sentido pero no del todo.

¿Qué pasa si incrementas la longitud de prefijo? ¿Hace más sentido el texto aleatorio?

3. *Una vez que tu programa funcione, podrías intentar hacer una mezcla: si combinas texto de dos o más libros, el texto aleatorio que generas mezclará el vocabulario y las frases de las fuentes en maneras interesantes.*

Crédito: este caso práctico está basado en un ejemplo del libro The Practice of Programming, escrito por Kernighan and Pike, Addison-Wesley, 1999.

Deberías intentar solucionar este ejercicio antes de proceder. Después puedes estudiar nuestra solución en Solución: A.9.5.

11.15 Estructuras de Datos

El uso del análisis de Markov para generar texto aleatorio es divertido, pero hay un propósito detrás de este ejercicio: la selección de estructura de datos. En tu solución al ejercicio previo, tenías que elegir:

- Cómo representar los prefijos.
- Cómo representar la colección de sufijos posibles.
- Cómo representar el mapeo desde cada prefijo a la colección de los sufijos posibles.

El último es fácil: un hash es la selección obvia para el mapeo desde las claves a los valores correspondientes.

Para los prefijos, las opciones obvias son un cadena de texto o una lista de cadenas de texto.

Para los sufijos, una opción es una lista; otra es un histograma (hash).

¿Cómo debes hacer la elección? El primer pasos es pensar sobre las operaciones que necesitarás implementar para cada estructura de datos. Para los prefijos, necesitamos ser capaz de remover palabras desde el inicio y agregar palabras al final. Por ejemplo, si el prefijo actual es “píntame”, y la siguiente palabra es “un”, necesitas ser capaz de formar el prefijo siguiente, “píntame un” para así encontrar el siguiente sufijo “cordero”.

Tu primera opción sería un array, dado que es fácil añadir y remover elementos, pero también necesitamos usar los prefijos como claves en un hash, así que este requerimiento elimina los arrays.

Para la colección de sufijos, las operaciones que necesitamos realizar incluyen agregar un sufijo nuevo (o incrementar la frecuencia de uno ya existente), y elegir un sufijo aleatorio.

Agregar un sufijo es igualmente fácil para la implementación de una lista o un hash. Elegir un elemento aleatorio desde una lista es fácil; elegirlo desde un hash es más difícil de hacer eficientemente (ver Solución: A.9.4).

Hasta ahora hemos estado discutiendo acerca de la facilidad de la implementación, pero hay otros factores a considerar en la elección de estructuras de datos. Uno de ellos es el tiempo de ejecución. Algunas veces existe una razón teórica al esperar que una estructura de datos sea más veloz que otra; por ejemplo, mencionamos que la operación de búsqueda es más rápida con los hashes que con los arrays, especialmente cuando el número de elementos es grande.

Pero usualmente no sabes con anticipación cual implementación será más rápida. Una opción es implementarlas a ambas y observar cual es mejor. Esta técnica se conoce como **benchmarking**. Una alternativa práctica es elegir la estructura de datos que es más fácil de implementar, y después ver si es lo suficientemente rápida para la aplicación a mano. Si lo es, no hay razón para proceder con la búsqueda. Si no es suficientemente rápida, existen herramientas, como módulos de perfil (**en. profile**), que pueden identificar los lugares en un programa que tardan más.

El otro factor a considerar es el espacio de almacenamiento. Por ejemplo, usar un histograma para la colección de sufijos podría tomar menos espacio porque tienes que almacenar cada palabra solamente una vez, sin importar cuantas veces aparezca. En algunos casos, ahorrar espacio puede hacer que tu programa se ejecute más rápido, y en el caso

extremo, tu programa podría no ejecutarse si tu memoria se agota. Pero para muchas aplicaciones, espacio es una consideración secundaria después del tiempo de ejecución.

Una reflexión final: en esta discusión, hemos insinuado que deberíamos utilizar una sola estructura de datos para el análisis y la generación. Pero debido a que estas son fases separadas, sería igualmente posible usar una estructura de datos para el análisis y después convertirla a otra para la generación. Esto sería una gran ventaja si el tiempo ahorrado durante la generación excede el tiempo que se emplea en la conversión.

11.16 Construyendo tus Propias Estructuras de Datos

Raku tiene un número de tipos compuestos tales como arrays y hashes que puedes combinar para construir arrays de arrays, arrays de hashes, hashes de arrays, hashes de hashes, arrays de arrays de arrays o hashes, etc., y esto es usualmente suficiente para la mayoría de las necesidades. Sin embargo, algunas veces necesitas algo muy específico que no está integrado en el lenguaje.

A través de los años, la ciencia de la computación ha estudiado y usado montones de estructuras de datos como listas enlazadas, pilas, colas, listas circulares, árboles de diferentes tipos, etc. Estudiaremos brevemente algunos de ellos.

11.16.1 Listas Enlazadas

Una lista enlazada es una colección de artículos en la cual cada artículo almacena un valor (o varios valores) y un enlace al siguiente artículo de la colección. En muchos lenguajes de programación, los arrays tienen un tamaño fijo (por el contrario, en Raku los arrays pueden usualmente crecer en tamaño de acuerdo a tus necesidades). En esos lenguajes de programación, una lista enlazada (**en.** linked list) es usualmente usada para representar una colección de artículos con tamaño variable.

Vimos en la Sección 9.4 cómo usar los arrays para construir pilas y colas. Fue relativamente fácil. En algunos lenguajes de programación de bajo nivel, necesitaría crear listas enlazadas para hacer eso.

En Raku, un artículo de una lista enlazada puede representarse por un par (un objeto del tipo `Pair`). El siguiente código es un ejemplo muy simple que muestra cómo podríamos implementar una pila usando una lista enlazada en Raku:

```
sub agregar-a-pila( Pair $pila-superior, $artículo ) {
    my $nuevo-par = $artículo => $pila-superior;
    return $nuevo-par;
}

sub tomar-desde-pila( Pair $pila-superior ) {
    my $nuevo-superior = $pila-superior.value;
    return $pila-superior.key, $nuevo-superior;
}

sub crear-pila( $artículo ) {
    return $artículo => Nil;
}
```

```

}

my $pila = crear-pila(0);

for 1..5 -> $artículo {
    $pila = agregar-a-pila($pila, $artículo);
}

say "La pila es: ", $pila.raku;

for 1..5 {
    my $valor;
    ($valor, $pila) = tomar-desde-pila($pila);
    say "$valor -- ", $pila;
}

```

Una vez que poblada, la pila resultante luce así:

```
5 => 4 => 3 => 2 => 1 => 0 => Nil
```

Esto se provee solo como un ejemplo para la construcción de una lista enlazada. Usualmente no hay necesidad de usar algo como esto en Raku, dado que es mucho más fácil implementar una pila usando un array, como se puede observar en la Sección 9.4, aunque el mismo principio puede ser usado para construir estructuras de datos más avanzadas.

Podrías aún querer, como un ejercicio, implementar una cola (ver Sección 9.4) usando una lista enlazada.

11.16.2 Árboles

Un árbol (**en.** *tree*) es usualmente una colección de artículos en la cual cada artículo (o *nodo*) almacena un valor (o posiblemente varios valores) y dos o varios enlaces a otros artículos de la colección, los hijos. Piensa sobre un árbol familiar o un árbol de directorios en un disco duro para tener una idea general. Los últimos nodos que no tienen sus propios hijos son usualmente llamados las hojas. Usualmente solo hay un nodo abuelo, llamado la raíz (si hay un más de un abuelo, entonces no es realmente un árbol pero varios árboles o un “bosque”).

Docenas de diferentes tipos de árboles han sido inventados y sus descripciones han cubierto libros completos sobre algoritmos en la ciencia de la computación. Algunos son diseñados para mantener tus datos ordenados, otros para mantener equilibrio entre las ramas de los árboles, etc. La estructura de datos es frecuentemente no tan complicada, pero los algoritmos necesarios para mantener las propiedades requeridas son algunas veces bien complicados.

Por ejemplo, un árbol típico podría retener un valor y dos enlaces, uno para el hijo izquierdo y uno para el hijo derecho. Podrías implementar tal árbol binario en una manera similar como la lista enlazada que describimos más arriba, excepto que el valor no sería ya un enlace al siguiente elemento sino que un array de dos elementos, los enlaces a los dos hijos. O podrías seguir el modelo de lista enlazada de más arriba y tener pares anidados. Por ejemplo, un árbol binario podría lucir de la siguiente manera:

```
my $árbol = 1 => (( 2 => 3 ) => (4 => (5 => ((6 => 7) => 8))));
```

La implementación se deja como un ejercicio al lector.

Muy a menudo un árbol puede implementarse en Raku como una estructura de datos más simple tal como un array o hash anidado. La siguiente sección examina un ejemplo así.

11.16.3 Montículos Binarios

Un montículo binario (**en.** binary heap) es un árbol binario que mantiene un orden parcial: cada nodo tiene un valor mayor que su padre y menor que sus dos hijos; no hay orden específico impuesto entre los hermanos. (Podrías también hacerlo de la otra forma: podrías diseñar montículos en los cuales cualquier nodo tiene un valor menor que su padre.)

Debido a que no hay un orden entre los hermanos, no es posible encontrar un elemento particular sin potencialmente escanear el montículo completo. Por lo tanto, un montículo no es muy bueno si necesitas acceso aleatorio a nodos específicos. Pero si estás interesado en siempre ser capaz de encontrar el artículo más pequeño, entonces un montículo es una estructura de datos muy eficiente.

Los montículos son usados para resolver un montón de problemas en la ciencia de la computación, y sirven como la base para una técnica de ordenamiento eficiente y muy popular llamada ordenamiento por montículo (**en.** heap sort).

Para un humano, es útil representar un montículo en forma de árbol. Pero una computadora puede almacenar un montículo como un array simple (ni siquiera como un array anidado). Para esto, el índice de un elemento es usado para calcular el índice de sus padres o sus dos hijos. Los hijos de un elemento son las dos localidades donde los índices son dos veces el índice del elemento padre. De forma inversa, el padre de un nodo está localizado a la mitad del índice del elemento nodo. Si el montículo inicia en el índice 0, las formulas exactas para un nodo con un índice n son comúnmente como se describen a continuación:

- padre: $\text{int}((n-1)/2)$
- hijo izquierdo: $2*n + 1$
- hijo derecho: $2*n + 2$

El nodo raíz comienza en el índice 0. Sus hijos se encuentran en las posiciones 1 y 2. Los hijos de 1 se encuentran en 3 y 4. Los hijos de 2 se encuentran en las posiciones 5 y 6. Los hijos de 3 se encuentran en 7 y 8, etc.

Si se interpreta como un montículo binario, el array:

[0, 1, 2, 3, 4, 5, 6, 7, 8]

está asociado con el árbol mostrado en la Figura 11.1.

Aquí presentamos una manera de construir un montículo (en orden alfabético parcial) desde una lista desordenada de letras:

```
sub construir-mont( @array, $indice is copy ) {
    my $valor-indice = @array[$indice];
    while ($indice) {
        my $padre = Int( ($indice - 1) / 2);
```

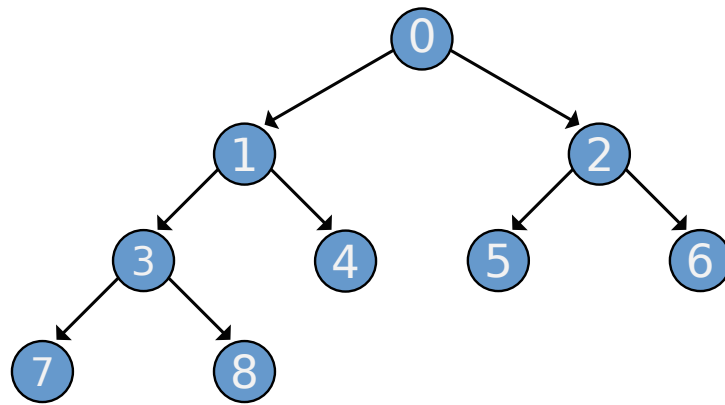



Fig. 11.1: El montículo correspondiente al array que contiene los dígitos del 0 al 8.

```

    my $valor-padre = @array[$padre];
    last if $valor-padre lt $valor-indice;
    @array[$indice] = $valor-padre;
    $indice = $padre;
  }
  @array[$indice] = $valor-indice;
}

sub heapify( @array ) {
  for @array.keys -> $i {
    construir-mont @array, $i;
  }
}

my @entrada = <m t f l s j p o b h v k n q g r i a d u e c>;
heapify @entrada;
say @entrada;

```

Nota que el montículo se construye en lugar (no hay una necesidad para un segundo array). El array resultante se muestra así:

```
[a b g d c k j l f h e m n q p t r o i u s v]
```

¿Es este montículo correcto? Es muy difícil de saber a primera vista y chequearlo manualmente es algo tedioso. Al escribir un programa para construir una estructura de datos como esta, es usualmente útil escribir algunas subrutinas para mostrar el contenido en una forma que facilite el entendimiento del resultado y el chequeo de exactitud. El código siguiente muestra dos ejemplos de tales subrutinas:

```

sub imprimir-mont( @array ) {
  my $inicio = 0;
  my $final = 0;
  my $último = @array.end;
  my $paso = 1;

```

```

loop {
    say @array[$inicio..$final];
    last if $final == $ultimo;
    $inicio += $paso;
    $paso *= 2;
    $final += $paso;
    $final = $ultimo if $final > $ultimo;
}
}

sub imprimir-mont2( @array ) {
    my $paso = 0;
    for @array.keys -> $actual {
        my $h_izquierdo = @array[2 * $actual + 1];
        say "$actual\tNodo = @array[$actual];\tNo hijos"
            and next unless defined $h_izquierdo;
        my $h_derecho = @array[2 * $actual + 2] // " ' ";

        say "$actual\tNodo = @array[$actual];\tHijos: " ~
            " $h_izquierdo y $h_derecho";
        $paso++;
    }
}

```

La primera subrutina muestra el árbol relacionado nivel por nivel:

```

(a)
(b g)
(d c k j)
(l f h e m n q p)
(t r o i u s v)

```

lo cual hace fácil dibujar el árbol (ver Figura 11.2).

La segunda subrutina muestra los hijos de cada nodo y hace posible chequear fácilmente que el orden alfabético parcial se satisface (o sea, cada nodo es menor que sus hijos):

0	Nodo = a;	Hijos: b y g
1	Nodo = b;	Hijos: d y c
2	Nodo = g;	Hijos: k y j
3	Nodo = d;	Hijos: l y f
4	Nodo = c;	Hijos: h y e
5	Nodo = k;	Hijos: m y n
6	Nodo = j;	Hijos: q y p
7	Nodo = l;	Hijos: t y r
8	Nodo = f;	Hijos: o y i
9	Nodo = h;	Hijos: u y s
10	Nodo = e;	Hijos: v y ' '
11	Nodo = m;	No hijos
12	Nodo = n;	No hijos
(...)		
21	Nodo = v;	No hijos

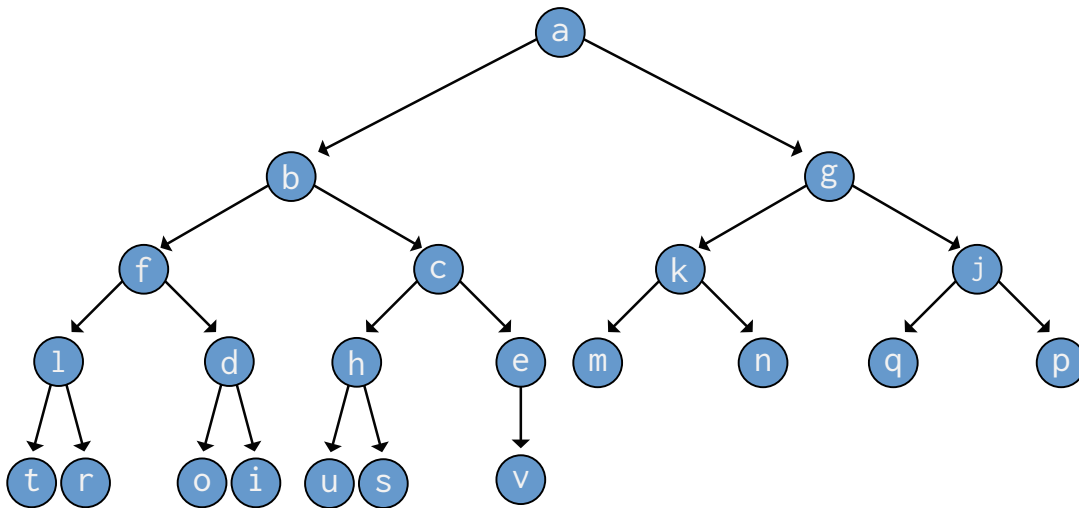


Fig. 11.2: El montículo correspondiente al array de letras.

11.17 Depuración de Programas

Cuando depuras un programa, y especialmente si estás trabajando con un error bien difícil, aquí presentamos algunas sugerencias que podrías intentar:

Leer Examina tu código, léelo, y chequea que diga lo que quieres decir.

Ejecutar Experimenta al hacer cambios y ejecutar las diferentes versiones. Usualmente si muestra la cosa correcta en el lugar correcto en el programa, el problema se vuelve obvio, pero algunas veces tienes que construir andamiaje para que esto ocurra.

Ejecutar con el depurador Un **depurador** es una utilidad que te permite ejecutar un programa paso a paso, para que puedas así seguir la ruta de ejecución y chequear el contenido de variables importantes a puntos cruciales en la ejecución del programa, para configurar puntos de interrupción, etc. Raku provee un depurador, llamado `raku-debug`, que hace todas estas cosas posible. Con la venida de lenguajes modernos de alto nivel, muchas personas se resisten al uso de un depurador. Esto es un error grave. Un depurador no te ayudará a solucionar cada tipo de problema, pero puede ser una utilidad inmensamente útil. Ver la Sección 12.14 para información adicional acerca del depurador de Raku.

Reflexionar ¡Toma tiempo para pensar! ¿Qué tipo de error es? ¿Sintáctico? ¿Al tiempo de ejecución? ¿Semántico? ¿Qué información puedes obtener de los mensajes de error, o desde la salida del programa? ¿Qué tipo de error podría causar el problema que estás observando? ¿Cuál fue el último cambio que hiciste, antes que el problema apareciera?

Depurar con el patito de goma Si explicas el programa a alguien más, algunas veces encuentras la solución antes de finalizar con la pregunta. Muy a menudo ni siquiera necesitas la otra persona; podrías conversar con un patito de goma. Ese es el origen de la muy conocida estrategia conocida como la **depuración del patito de goma**. ¡Y no estoy bromeando! Lee sobre la estrategia aquí https://es.wikipedia.org/wiki/M%C3%A9todo_de_depuraci%C3%B3n_del_patito_de_goma.

Retroceder Algunas veces, lo mejor que puedes hacer es retroceder, deshacer los cambios recientes, hasta que puedas obtener un programa que funciona y que entiendas. Después puedes comenzar la reconstrucción.

Los programadores principiantes algunas veces se atascan en una de estas actividades y se olvidan de las otras. Cada actividad viene con su propio modo de fallo.

Por ejemplo, leer tu programa podría ayudarte si el problema es un error tipográfico, pero no si el problema es un malentendido conceptual. Si no entiendes lo que tu programa hace, puedes leerlo más 100 veces y nunca verás el error, porque el problema se encuentra en tu cabeza.

Desarrollar experimentos puede ayudarte, especialmente si ejecutas pruebas pequeñas y simples. Pero si desarrollas experimentos sin pensar o leer tu código, puedes caer en un patrón que llamamos “programación de paso aleatorio”, el cual es el proceso de hacer cambios aleatorios hasta que el programa haga lo correcto. Obviamente, la programación de paso aleatorio puede tomar mucho tiempo.

Debes tomar tiempo para pensar. La depuración es como un experimento científico. Podrías tener por lo menos una hipótesis sobre el problema (por ejemplo, ¿cuál es el problema?). Si hay más de dos posibilidades, intenta pensar acerca de una prueba que eliminaría uno de ellos.

Aún así, la mejores técnicas de depuración fallarán si hay muchos errores, o si el código que estás intentando arreglar es demasiado grande y complicado. Algunas veces la mejor opción es retroceder, simplificar el programa hasta que obtengas algo que funcione y que entiendas.

Los programadores principiantes son usualmente reacios a retroceder porque no pueden tolerar el hecho de borrar una línea de código (aún si es algo erróneo). Si te hace sentir mejor, haz una copia de tu programa en otro archivo antes de hacer cambios. De esa manera puedes copiar las piezas una a la vez con cada progreso.

Encontrar un error difícil requiere leer, ejecutar (sin o con un depurador), reflexionar, y algunas veces retroceder. Si te atasca en una de estas actividades, prueba las otras.

11.18 Glosario

Determinista Perteneciente a un programa que hace la misma cosa cada vez que se ejecuta, dado la misma entrada.

Pseudo-aleatorio Perteneciente a una secuencia de números que aparece ser aleatoria, pero es generada por un programa determinista.

Valor por defecto El valor dado a un parámetro opcional si no se provee un argumento.

Sobrescribir Reemplazar un valor por defecto con un argumento.

Benchmarking El proceso de elegir entre varias estructuras de datos y algoritmos al implementar alternativas y someterlas a pruebas (especialmente sus tiempos de duración) con una muestra de las entradas posibles.

Depurador Un programa que permite ejecutar un programa línea por línea y chequea su estado a cualquier paso durante su ejecución.

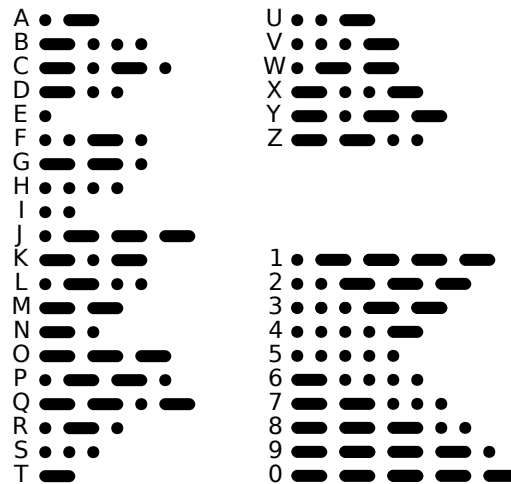


Fig. 11.3: Código Morse Internacional (dominio público)

Depuración del patito de goma Depuración que involucra explicar tu problema a un objeto inanimado tal como un patito de goma. Articular el problema puede ayudarte a solucionarlo, aún si el patito de goma no sabe Raku.

11.19 Ejercicio: Codificación Huffman

La codificación Huffman es una técnica usada para la compresión de datos, o sea, para reducir el tamaño de una pieza de datos (tal como, por ejemplo, comprimir un archivo).

11.19.1 Código de Longitud Variable

Si eres familiar con el código morse, ya sabes que es un sistema para codificar las letras del alfabeto en una serie de puntos y rayas. Por ejemplo, la famosa señal `...---` representa las letras SOS, la cual constituye una llamada de ayuda reconocida internacionalmente. La tabla en la Figura 11.3 muestra el resto de los códigos.

El código morse (inventado entre 1836 y 1844) fue uno de los primeros intentos de la codificación digital del alfabeto de un texto sencillo. El único otro intento conocido es el alfabeto braille (1824-1837).

Nota que algunos de los códigos son más largos que otros. Por diseño, las letras más comunes poseen códigos más cortos. Dado que hay un número ilimitado de códigos cortos, eso significa que las letras y símbolos menos comunes poseen códigos más largos. Un mensaje típico tendrás más códigos cortos que largos, lo cual minimiza el tiempo promedio de transmisión por letra.

Este tipo de código se conoce como códigos de longitud variable. En este ejercicio, estudiaremos un algoritmo para generar código de longitud variable llamado codificación Huffman. Es un algoritmo interesante en sí mismo, pero también hace de un ejercicio interesante porque su implementación usa una variedad de estructuras de datos.

Aquí presentamos un esquema de lo que haremos hasta al final de este capítulo:

1. Primeramente, usaremos una muestra de texto en inglés para generar una tabla de los caracteres y sus frecuencias.
2. Después usaremos esta tabla de frecuencia para generar una tabla de código.
3. Finalmente, codificaremos un mensaje con esta tabla de código y consiguientemente lo decodificaremos.

11.19.2 La Tabla de Frecuencias

Dado que la meta es proveer las letras comunes con códigos cortos, necesitamos saber la frecuencia con la cual cada letra ocurre. En la historia “The Gold Bug”⁵ de Edgar Allan Poe, uno de los personajes, William Legrand, usa la frecuencia de las letras para descifrar un criptograma. Él explica:

“Now, in English, the letter which most frequently occurs is e. Afterwards, the succession runs thus: a o i d h n r s t u y c f g l m w b k p q x z. E however predominates so remarkably that an individual sentence of any length is rarely seen, in which it is not the prevailing character.”⁶

Nuestra misión es chequear si Poe estaba en lo cierto. Para chequearlo, usaremos una muestra del texto del libro “The Gold Bug”, el cual puede descargarse desde el Proyecto Gutenberg (<http://www.gutenberg.org/files/2147/2147-0.txt>) y una variedad de otros sitios web.

Ejercicio 11.8. *Escribe un programa que cuenta el número de veces que cada letra aparece en una muestra de texto. Descarga el texto de “The Gold Bug” y analiza la frecuencia de las letras.*

Solución: A.9.6.1.

11.19.3 Construyendo el Código Huffman

Para nuestro propósito, el código morse tiene un defecto: no solamente usa dos símbolos como podrías pensar, sino que actualmente usa tres: además de los puntos y las rayas, también usa el espacio entre los dos símbolos implícitamente, como también espacios más largos entre dos letras.

La razón por la cual algunos espacios son necesarios es muy simple. Usa la tabla de código morse anterior y supón que recibes tres puntos (. . .). Esto podría ser interpretado como la letra “e” tres veces, o como “ie” o “ei” o como “s”, o como el inicio de “h”, “v”, “3”, “4”, o “5”. Los espacios agregados hacen posible distinguir entre las varias posibilidades. Pero también hacen la transmisión de código mucho más lenta.

En 1951, David A. Huffman inventó una técnica de construcción de código que evita este problema: provisto que sepas donde una letra dada comienza, no existe ninguna ambigüedad. Por ejemplo, trabajaremos más tarde con un código Huffman para un subconjunto pequeño del alfabeto que luce así:

⁵El Escarabajo de Oro

⁶“ Ahora bien: la letra que se encuentra con mayor frecuencia en inglés es la e. Después, la serie es la siguiente: a o i d h n r s t u y c f g l m w b k p q x z. La e predomina de un modo tan notable, que es raro encontrar una frase sola de cierta longitud de la que no sea el carácter principal.”

```

a => . .
e => . -
s => - . -
n => - . .
t => - - .
d => - - - .
r => - - - -

```

Si comienzas a leer una secuencia de puntos y rayas que representan un texto válido compuesto con estas siete letras, siempre puedes decodificarlo sin ninguna ambigüedad. Si el primer símbolo es un punto, entonces la letra es una “a” o un “e” dependiendo en el símbolo siguiente. Si el primer símbolo es una raya y el siguiente es un punto, entonces la letra deber ser una “s” o una “n” dependiendo del tercer símbolo. Si los primeros símbolos son rayas, puedes similarmente determinar que la letra actual es una “t” (si el tercer símbolo es un punto), o una “d” o una “r”, la cual puedes encontrar al revisar el cuarto símbolo. En resumen, no necesitas espacios entre los símbolos, siempre es posible decodificar una letra sin ambigüedad.

¿Cómo podemos construir un código Huffman? Hagámoslo manualmente con un alfabeto bien simple: las cuatros letras de las bases nitrogenadas del ADN: A, C, T y G. Supón que queremos codificar la siguiente cadena de texto de entrada:

CCTATCCTCGACTCCAGTCCA

Esto resulta en la tabla de frecuencia siguiente:

C :	10	47.62
T :	5	23.81
A :	4	19.05
G :	2	9.52

Para construir el código Huffman, comenzamos con las dos letras menos frecuentes y las mezclamos en un símbolo nuevo temporario, [GA], el cual pretendemos es una letra compuesta nueva con una frecuencia de 6. En este punto, decidimos que, entre dos letras, a la menos frecuente se le adjuntará un punto y a la otra se le adjuntará una raya (este es una decisión arbitraria, podría hacerse de la manera inversa). Así que decimos que el símbolo para la letra menos común de las dos letras (“G”) será [GA] . y [GA] - será el símbolo para la “A”.

Ahora tenemos tres letras, C, T, and [GA]. Mezclamos las dos letras menos frecuentes, “T” y “[GA],” y podemos ahora decir que el símbolo para la “T” será [TGA] . y el símbolo para [GA] será [TGA] -. Ahora tenemos solo dos letras restantes, “C” and “TGA”, con “C” siendo la menos frecuente; así que “C” será un punto y “TGA” será una raya.

Podemos ahora desenrollar nuestras letras ficticias: “T” es [TGA] ., así que, reemplazando [TGA] con su símbolo, o sea, una raya, el código final para “T” será - .; similarmente, [GA] . ahora puede traducirse como - -. Por el mismo proceso de sustitución, podemos ahora determinar que “A” y “G” son --- y --. respectivamente. Así que nuestra tabla final de código Huffman:

```

C => .
T => - .
G => - - .
A => - - -

```

Nota que, debido a la construcción, la letra más frecuente (C) tiene el código más corto mientras que las letras menos comunes (G y A) tienen los códigos más largos.

La codificación manual de cadena de texto de entrada CCTATCCTCGACTCCAGTCCA con este código rinde el código pseudo-Morse siguiente:

...-----...-----...-----...-----

Nota que nuestro código Huffman no tiene ambigüedad: el primer punto puede solo ser una "C", y el segundo también. El símbolo siguiente es una raya, que puede ser el inicio de las otras tres letras, pero solo la "T" puede tener un punto posterior. La secuencia siguiente de símbolos tiene cuatro rayas; esto puede solamente tres rayas de una "A", con la última raya siendo el inicio de la letra siguiente; y - . solo puede ser una "T", etc.

En una codificación de Huffman real para la compresión de archivos de texto, no usaríamos puntos y rayas, pero los bits 0 y 1; sin embargo, los punto y las rayas son solo otra manera conveniente de representar esos valores binarios. Por lo tanto, pretendamos que los puntos y las rayas son realmente los números binarios 0 y 1.

¿Realmente logramos la compresión de datos? Nuestra cadena de pseudo-Morse tiene 38 símbolos binarios. La cadena de texto de entrada original tenía 21 caracteres o bytes, eso son 168 bits. Los datos han sido comprimidos por un factor de 4.4.

¿Es la codificación de Huffman mejor que la codificación de longitud fija? Una representación de una cadena de texto donde cada letra sería representada por dos bits (dos bits pueden representar cuatros letras) requiere 42 símbolos. Por lo tanto, sí, obtuvimos una mejor compresión de datos que una codificación de longitud fija (alrededor de 10%). Este puede parecer un logro minúsculo, pero es actualmente un gran logro con un alfabeto tan pequeño. Con datos textuales actuales, la codificación de Huffman puede lograr una significativa compresión de datos.

Ejercicio 11.9. 1. *Escribe un programa que realiza la codificación de Huffman de una cadena de texto de caracteres simple. Puedes comenzar con el ejemplo de ADN más arriba. No te preocupes si no obtienes la misma tabla de Huffman de más arriba: pueden haber más código de Huffman para una cadena de texto dada; pero chequea que obtengas un código que no sea ambiguo.*

2. *Haz una prueba con cadenas de texto que tienen un alfabeto más grande (probablemente querrás comenzar con un alfabeto relativamente pequeño porque, por lo contrario, puede ser tedioso chequear el resultado manualmente).*

3. *Escribe una subrutina que codifique una cadena de texto de entrada en pseudo-Morse usando la tabla de Huffman generada anteriormente.*

4. *Escribe una subrutina que decodifique la salida pseudo-Morse que has generado para la pregunta anterior.*

Solución: A.9.6.2.

Parte II

Hacia Adelante

Ahora que has alcanzado el final de la primera parte de este libro, ya no deberías sentirte un principiante. Deberías de ser capaz de leer (y escudriñar) la documentación oficial de Raku (<https://docs.raku.org>) y encontrar tu camino.

Hay muchas cosas que decir acerca de la programación. Los siguientes tres capítulos serán dedicados a conceptos más avanzados y a nuevos paradigmas de la programación, incluyendo:

Programación orientada a objetos Describiremos cómo construir nuestros propios tipos y métodos, lo cual es una manera de extender el lenguaje.

Uso de gramáticas Esta es una forma de programación declarativa en la cual defines axiomas y reglas y derivas conocimientos de estos; las gramáticas son una forma muy poderosa de analizar contenido textual y se usan para transformar el código fuente de un programa en sentencias que se pueden ejecutar.

Programación funcional Este es aún otro tipo de paradigma de la programación en el cual una computación se expresa como la evaluación de funciones matemáticas.

Cada uno de estos capítulos probablemente merece un libro completo (y podrían tener uno en el futuro) pero esperamos decirte lo suficiente acerca de ellos para ponerte en marcha. En mi opinión, cada programador debería saber sobre estos conceptos poderosos para ser capaz de seleccionar la mejor forma de resolver un problema en particular.

Raku es un lenguaje multiparadigmático, así que podemos cubrir estas disciplinas usando el lenguaje Raku. Un número de temas que introducimos en los capítulos anteriores deberían facilitar la fácil adquisición de estas nuevas ideas, y esta es la razón por la cual pienso que es posible cubrirlas apropiadamente solo con un capítulo para cada una de estas disciplinas.

Habrán menos ejercicios en la segunda parte, porque esperamos por ahora que ya eres capaz de idear tus propios ejercicios y hacer tus propios experimentos. Y habrán solo algunas soluciones sugeridas, porque nos estamos acercando a un nivel donde no existe una única solución correcta, sino muchas maneras de enfrentar y resolver un problema.

Con respecto al lenguaje Raku, hemos cubierto muchísimo material, pero, como te advertí al principio, esto dista de ser exhaustivo. Los siguientes son los temas que no discutimos (y no cubriremos); si deseas saber más sobre ellos, este es el momento perfecto para explorar la documentación por ti mismo:

Programación concurrente Las computadoras de hoy en día poseen procesadores múltiples o procesadores con múltiples núcleos; Raku ofrece varias formas de tomar ventaja de estos para ejecutar procesos de computación en paralelo para así mejorar el rendimiento y reducir el tiempo de ejecución; ver <https://docs.raku.org/language/concurrency> para más detalles.

Manejo de excepciones El manejo de situaciones cuando algo no funciona correctamente es una parte importante de la programación. Raku ofrece varios mecanismos para el manejo de tales situaciones. Ver <https://docs.raku.org/language/exceptions> para más detalles.

Comunicación entre procesos Los programas usualmente tienen que ejecutar otros programas y comunicarse con los mismos. Ver <https://docs.raku.org/language/ipc>.

Módulos Cómo crear, usar, y distribuir módulos de Raku. Ver <https://docs.raku.org/language/modules>.

Interfaz para llamadas nativas Cómo llamar librerías que están escritas en otros lenguajes de programación y seguir las convenciones de llamadas del lenguaje C. Ver <https://docs.raku.org/language/nativecall>.

Clases y Objetos

A esta altura ya sabes cómo usar funciones para organizar código y tipos integrados para organizar datos. El siguiente paso es aprender sobre la “programación orientada a objetos,” la cual usa tipos definidos por el programador para organizar código y datos.

Cuando las aplicaciones de software comienzan a crecer, el número de detalles que se debe manejar puede volverse agobiante. La única manera de manejar esta complejidad es usar abstracción y encapsulamiento. La programación orientada a objetos es una manera muy popular y eficiente de implementar abstracción y encapsulamiento.

Raku es un **lenguaje de programación orientado a objetos**, lo cual significa que el lenguaje provee características que soportan la programación orientada a objetos, la cual posee las siguientes características definitivas:

- Los programas incluyen definiciones de clases y métodos.
- La mayor parte de la computación se expresa en términos de operaciones sobre los objetos.
- Los objetos usualmente representan cosas en el mundo real, y los métodos normalmente corresponden a las formas de interacción de las cosas en el mundo real.

La programación orientada a objetos en Raku es un tema enorme que merece un libro para sí mismo (y probablemente habrá un libro o dos sobre la materia en algún punto). Este capítulo hará más que analizar el tema superficialmente y te permitirá crear y usar objetos. Sin embargo, no cubrirá algunos detalles y características más avanzadas.

12.1 Objetos, Métodos y Programación Orientada a Objetos

Empecemos con una sinopsis general de la programación orientada a objetos y una breve introducción a la jerga asociada con la misma.

En la ciencia de la computación, un objeto puede describirse vagamente como una dirección de memoria o una entidad que posee un valor, y usualmente es conocido como un identificador. Esto puede ser una variable, una estructura de datos, un array, o posiblemente hasta una función. No usaremos esta concepción general con el mismo sentido en este capítulo.

En la programación orientada a objetos (POO), la palabra **objeto** tiene un significado mucho más específico: un objeto es una identidad que usualmente tiene:

- Una identidad (por ejemplo, su nombre).
- Algunas propiedades que definen su comportamiento (en la forma de funciones especiales que son comúnmente conocidas como **métodos**); este comportamiento usualmente no cambia con el tiempo y es generalmente común a todos los objetos del mismo tipo.
- Un **estado** el cual es definido por algunas variables especiales (conocidas como, dependiendo en el lenguaje, atributos, campos, o miembros); el estado puede cambiar a medida que pasa el tiempo y es generalmente específico a cada objeto. En Raku, nos referimos a dichas variables como **atributos**.

En resumen, un objeto es un conjunto de atributos y métodos todos juntos.

Los objetos son usualmente definidos dentro de un tipo de paquete de código conocido como una **clase**. Una clase define los métodos y la naturaleza de los atributos asociados a un objeto. En Raku, una clase hace posible definir nuevos tipos similares a los tipos integrados que hemos visto hasta ahora. Muy pronto comenzaremos a definir algunas clases y las usaremos para crear objetos.

Ya sabes informalmente lo que es un método, dado que hemos usado métodos integrados a lo largo del libro. Es como una función con una sintaxis sufija que usa la notación del punto sobre el invocante. Por ejemplo, puedes invocar el método `say` sobre una simple cadena de texto:

```
"foo".say;           # -> foo
```

Nota que “foo” no es un objeto, pero una simple cadena de texto. No obstante, puedes invocar el método `say` sobre la misma, debido a que Raku puede tratarla internamente como un objeto cuando es necesario. En algunos otros lenguajes POO, esta conversión implícita de un tipo nativo a un objeto es conocida como *autoboxing*.

Probablemente recuerdas que los métodos pueden ser encadenados en un proceso donde el valor devuelto por un método se convierte en el invocante para el siguiente método:

```
"foo".uc.say;        # -> FOO

my @alfabeto = <charlie foxtrot alpha golf echo bravo delta>;
@alfabeto.sort.uc.say;
# imprime: ALPHA BRAVO CHARLIE DELTA ECHO FOXTROT GOLF
```

En POO, los métodos que se pueden aplicar a los objetos son usualmente definidos dentro de las clases, regularmente la clase que también definió el objeto o alguna otra clase con

una cercana relación. En Raku, los métodos pueden también ser definidos en un **rol**, el cual es otro tipo de paquete de código que se parece a una clase, como veremos más adelante.

La idea básica de la programación orientada a objetos es que un objeto es un tipo de caja negra que oculta su estructura interna (datos y código) del usuario; el usuario puede consultar o cambiar el estado de un objeto a través de los métodos. El proceso de ocultar la estructura interna de los objetos se conoce como **encapsulación**. Esto usualmente permite tener una perspectiva más general y una mejor abstracción de datos con respecto a lo que hemos visto hasta ahora; esto resulta en programas con menos errores (especialmente programas extensos).

En adición, POO usualmente ofrece los siguientes conceptos:

- **polimorfismo**, es decir, la función o el método tiene la posibilidad de hacer cosas diferentes dependiendo del tipo de objeto que la llama o la invoca;
- **herencia**, es decir, la posibilidad de derivar una clase de otra clase, de tal manera que la clase hija hereda algunas de las propiedades de la clase **padre**, la cual es una herramienta poderosa para la reutilización de código.

Ahora estudiaremos cómo todo estos conceptos se implementan en Raku.

12.2 Tipos Definidos por el Programador

Hemos usado muchos tipos integrados de Raku; ahora vamos a definir un nuevo tipo. Como un ejemplo, crearemos un tipo llamado Punto2D que representa a un punto en el espacio bidimensional.

En notación matemática, los puntos son usualmente escritos en paréntesis con una coma que separa las coordenadas. Por ejemplo, en las coordenadas cartesianas o rectangulares, $(0,0)$ representa el origen, y (x,y) representa un punto x unidades a la derecha y y unidades hacia arriba desde el origen. x se llama la abscisa y y la ordenada.

Hay varias maneras de representar un punto en Raku:

- Podemos almacenar las coordenadas separadamente en dos variables, $\$x$ y $\$y$.
- De igual manera, podemos almacenar las coordenadas como elementos en una lista, un array, o una pareja.
- Así mismo, podemos crear un nuevo tipo que representa puntos como objetos.

Crear un nuevo tipo es un poco más complicado que las otras opciones, pero tiene ventajas que serán aparentes muy pronto.

Un tipo definido por el programador es usualmente creado con una **clase** (o un **rol**, pero regresaremos a esto más tarde). La definición de una clase bien simple para un tipo que representa a un punto luce así:

```
class Punto2D {  
    has $.abscisa;           # valor "x"  
    has $.ordenada;         # valor "y"  
}
```


La cabecera indica que la nueva clase se llama Punto2D. El cuerpo define dos atributos, es decir, propiedades con nombres asociadas con la clase, aquí la abscisa y la ordenada (o las coordenadas x y y) del punto.

```
class Punto2D {
  has $.abscisa;           # valor "x"
  has $.ordenada;         # valor "y"
}
```

La definición de una clase con el nombre Punto2D crea un **tipo objeto**.

El tipo objeto es como una factoría para crear objetos. Para crear un punto, llamas el método new sobre la clase Punto2D:

```
my $punto = Punto2D.new(
  abscisa => 3,
  ordenada => 4
);
say $punto.WHAT;           # -> (Punto2D)
say $punto.isa(Punto2D)   # -> True
say $punto.abscisa;       # -> 3
```

Por supuesto puedes crear tanto puntos como deseos.

El método new se llama un **constructor** y no se ha definido en este ejemplo; esto no es necesario dado que Raku provee un constructor new por defecto para cada clase (como veremos más adelante). La sintaxis de invocación de método, con la notación de punto, es la misma que hemos usados a lo largo del libro para invocar los métodos integrados. No estás forzado a usar este constructor; también puedes crear tu propio constructor (y puedes nombrarlo new o algo diferente), pero usaremos el método integrado new por el momento.

El proceso de crear un nuevo objeto con una clase es conocido como **instanciación**, y el objeto es una **instancia** de la clase.

Cada objeto es una instancia de alguna clase, así que los términos “objeto” e “instancia” son intercambiables. Pero usualmente usaremos “instancia” para indicar que estamos hablando de un objeto que pertenece a un tipo definido por el programador.

12.3 Atributos

Los atributos que hemos definido son las propiedades asociadas con las clase Punto2D pero, al mismo tiempo, son específicas a la instancia de la clase que hayamos creado. Ellos son los atributos de la instancia. Si creamos otro objeto Punto2D, también tendrá los mismos atributos, pero los valores de tales atributos serán probablemente diferentes.

La Figura 12.1 muestra el resultado de estas asignaciones. Un diagrama de estado que muestra un objeto y sus atributos es conocido como un **diagrama de objeto**.

La variable \$punto se refiere a un objeto Punto2D, el cual contiene dos atributos.

Cada atributo de la clase Punto2D debe referirse a un número, pero esto no es obvio en la definición actual de la clase. Al momento podemos crear un objeto Punto2D con un cadena de texto como la abscisa, lo cual no hace sentido. Podemos mejorar la definición de la clase al especificar un tipo numérico para los atributos:

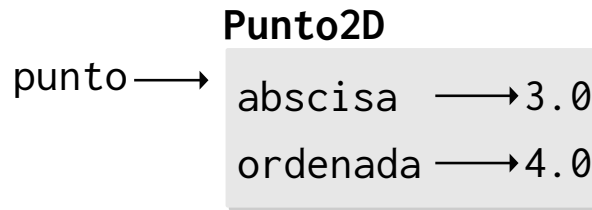


Fig. 12.1: Diagrama de objeto.

```
class Punto2D {
  has Numeric $.abscisa;           # valor "x"
  has Numeric $.ordenada;         # valor "y"
}
```

Los atributos de la instancia son privados para la clase, lo cual significa que no pueden accederse fuera de la clase: usualmente necesitarías invocar un método de la clase (es decir, un tipo de subrutina definida dentro de la clase), para obtener su valor. No obstante, cuando se define un atributo con un punto como en `$.abscisa`:

```
has $.abscisa;
```

Raku crea un método implícito *de acceso*, es decir, un método que comparte el nombre del atributo y devuelve el valor de dicho atributo. Así que, cuando escribimos:

```
say $punto.abscisa;                # -> 3
```

no estábamos accediendo el atributo `abscisa` del objeto `$punto` directamente, sino que estábamos llamando al método `abscisa` sobre el objeto, el cual devolvió el valor de ese atributo.

Puedes usar dicho método de acceso con la notación de punto como parte de cualquier expresión. Por ejemplo:

```
my $dist-al-centro = sqrt($punto.abscisa ** 2 + $punto.ordenada ** 2);
```

También hay otra manera de declarar un atributo en una clase, usando el twigil de signo de exclamación en vez de un punto:

```
has !$abscisa;
```

En ese caso, Raku no crea un método implícito de acceso y el atributo solo se puede acceder desde otros métodos dentro de la clase. Tal atributo es ahora completamente privado. Sin embargo, si declaras atributos de esta manera, no podrás asignarles ningún valor durante la creación del objeto con el constructor por defecto `new` y necesitarás crear tu propio constructor (o modificar `new` indirectamente). Así que no intentes esto por el tiempo presente, dado que no serás capaz de hacer mucho con tus objetos en este punto. Regresaremos a esto más tarde.

Por defecto, los atributos de un objeto no son mutables; solo se pueden leer. Esto significa que no puedes modificarlos una vez que se ha creado el objeto. Esto funciona bien para

algunos atributos: si un objeto representa a una persona, es poco probable que el nombre de la persona y su fecha de nacimiento cambien. Otros atributos necesitan ser actualizados, muchas veces muy frecuentemente. En tales casos, los atributos pueden declararse como mutables con el rasgo (del inglés *trait*) `is rw`:

```
class Punto2D {
  has Numeric $.abscisa is rw;          # valor "x"
  has Numeric $.ordenada is rw;        # valor "y"
}
```

Ahora es posible modificar estos atributos. Por ejemplo, podemos cambiar la abscisa del punto recién creado:

```
# Primero creamos un objeto Punto2D:
my $punto = Punto2D.new(abscisa => 3, ordenada => 4);
say $punto;    # -> Punto2D.new(abscisa => 3, ordenada => 4)

# Ahora movemos el objeto $punto dos unidades a la derecha:
$punto.abscisa = 5;
say $punto;    # -> Punto2D.new(abscisa => 5, ordenada => 4)
```

Casi toda la información presentada aquí acerca de los atributos está relacionada con los atributos de la instancia, es decir, propiedades relacionadas a objetos individuales. También puedes tener atributos que pertenecen a la clase entera, los cuales se conocen como *atributos de la clase*. Son menos comunes que los atributos de una instancia y son declarados con el declarador `my` (en vez de `has`). Un ejemplo típico de un atributo de una clase sería un contador al nivel de la clase para mantener un récord del número de objetos que se han instanciado.

12.4 Creando Métodos

La simple clase `Punto2D` y su instancia `$punto` no son muy útiles al momento. Complete-mos la definición de la clase con algunos métodos:

```
class Punto2D {
  has Numeric $.abscisa;
  has Numeric $.ordenada;

  method coordenadas {          # método de acceso para "x" y "y"
    return (self.abscisa, self.ordenada)
  }

  method distancia-al-centro {
    (self.abscisa ** 2 + self.ordenada ** 2) ** 0.5
  }

  method coordenadas-polares {
    my $radio = self.distancia-al-centro;
    my $theta = atan2 self.ordenada, self.abscisa;
    return $radio, $theta;
  }
}
```

Aquí declaramos tres métodos en la clase:

- `coordenadas`, un simple método de acceso a las coordenadas cartesianas;
- `distancia-al-centro`, un método que calcula y devuelve la distancia entre el objeto y el origen;
- `coordenadas-polares`, un método que calcula el radio y el azimuth (θ) del objeto en el sistema de coordenada polar (nota que `coordenadas-polares` invoca al método `distancia-al-centro` para encontrar el componente radio de las coordenadas polares).

La definición de un método no es muy distinta a la definición de una subrutina, excepto que usa la palabra clave `method` en vez de `sub`. Esto no es una sorpresa dado que un método es esencialmente una subrutina que se define dentro de una clase (o un rol) y sabe acerca de su *invocante*, es decir, el objeto que lo llama y su clase. Y, por supuesto, tiene una sintaxis de llamada diferente.

Otra diferencia importante entre una subrutina y un método es que, debido a que hay varios métodos con el mismo nombre definidos en diferentes clases (o diferentes roles), una invocación de método conlleva una fase de *despacho*, en la cual el sistema de objeto selecciona cuál método llamar, usualmente basado en la clase o tipo del invocante. Sin embargo, en Raku, esa diferencia no es muy clara por el hecho de que puedes tener multi subrutinas, es decir, subrutinas con el mismo nombre y signaturas diferentes que son resueltas al tiempo de ejecución, dependiendo en la *aridad* (número de argumentos) y el tipo de los argumentos.

Dentro de la definición de un método, `self` se refiere al *invocante*, el objeto que invocó al método. Existe un atajo para el mismo, `$.`, así que podríamos escribir el método `coordenadas` de la siguiente manera:

```
method coordenadas {          # método de acceso para "x" y "y"
    return ($.abscisa, $.ordenada)
}
```

Los dos formatos de sintaxis, `$.` y `self`, son esencialmente equivalentes.

Existe una tercera manera de lograr el mismo objetivo, la cual usa el signo de exclamación en vez del punto:

```
method coordenadas {          # método de acceso para "x" y "y"
    return (!$!abscisa, !$!ordenada)
}
```

Aquí, el resultado sería el mismo, pero esta nueva sintaxis no es equivalente: `$.abscisa` es una invocación de método, mientras que `(!$!abscisa)` provee acceso directo al atributo. La diferencia es que `(!$!abscisa)` está disponible dentro de la clase (y podría ser un poco más rápida), mientras la sintaxis de invocación de método puede ser usada en cualquier otra parte (por ejemplo dentro de otra clase). Veremos en la siguiente sección ejemplos de esta distinción y sus consecuencias.

Podemos ahora crear un objeto e invocar nuestros métodos:

```

my $punto = Punto2D.new(
  abscisa => 4,
  ordenada => 3
);
say $punto.coordenadas;           # -> (4 3)
say $punto.distancia-al-centro;   # -> 5
say $punto.coordenadas-polares;   # -> (5 0.643501108793284)

```

Podría ser que recuerdes que si usas un método sin nombrar el invocante explícitamente, entonces el método se aplica a la variable tópico:

```

.say for <one two three>;          # -> one two three (cada uno en su propia línea)

```

Ahora que hemos creado un objeto con algunos métodos, también podemos tomar ventaja de la misma sintaxis. Por ejemplo, si usamos `for` o `given` para poblar la variable tópico `$_` con el objeto `$punto`, podemos escribir:

```

given $punto {
  say .coordenadas;           # -> (4 3)
  say .distancia-al-centro;   # -> 5
  .coordenadas-polares.say;   # -> (5 0.643501108793284)
}

```

Como un ejercicio, podrías escribir una subrutina llamada `distancia-entre-puntos` que toma dos puntos como argumentos y devuelve la distancia entre ellos usando el Teorema de Pitágoras.

Los métodos de nuestra clase son todos *métodos de acceso*, lo cual significa que ellos proveen el estado de algunos de los atributos del invocante. Si los atributos son mutable (al ser declarados con el trait `is rw`), podemos también crear algunos *mutadores*, es decir, métodos que se invocan para cambiar aquellos atributos mutables:

```

class Punto2D-mutable {
  has Numeric $.abscisa is rw;
  has Numeric $.ordenada is rw;

  # quizás los mismos métodos de acceso como
  # en la clase anterior.

  method nueva-ordenada ( Numeric $ord ) {
    self.ordenada = $ord;
  }
}

# Creando un objeto Punto2D-mutable:
my $punto = Punto2D-mutable.new(abscisa => 3, ordenada => 4);
say $punto; # -> Punto2D-mutable.new(abscisa => 3, ordenada => 4)

# Modificando la ordenada:
$punto.nueva-ordenada(6);
say $punto; # -> Punto2D-mutable.new(abscisa => 3, ordenada => 6)

```

12.5 Rectángulos y Composición de Objetos

Algunas veces es obvio qué los atributos de un objeto deberían ser, pero otras veces debes hacer decisiones. Por ejemplo, imagínate que estás diseñando una clase para representar rectángulos. ¿Qué atributos usarías para especificar la ubicación y tamaño de un rectángulo? Puedes ignorar los ángulos; para mantener las cosas simples, asume que los bordes del rectángulo son verticales o horizontales.

Existen por lo menos dos posibilidades:

- Podrías especificar una esquina del rectángulo (o el centro), el ancho, y la altura.
- Podrías especificar dos esquinas opuestas.

En este momento es difícil decir cuál de las opciones es mejor, así que implementaremos la primera, como un ejemplo.

Aquí está la definición de la clase:

```
class Rectángulo {
  has Numeric $.ancho;
  has Numeric $.altura;
  has Punto2D $.esquina;      # vértice inferior izquierdo

  method area { return $.ancho * $.altura }
  method superior-izq { $.esquina.abcisa, $.esquina.ordenada + $.altura; }
  # otros métodos, e.g. para las coordenadas de las otras esquinas,
  # centro, etc.
}
```

La nueva característica comparada a la definición anterior de la clase Punto2D es que la clase Rectángulo ahora usa el tipo Punto2D creado anteriormente para definir el atributo esquina.

El método superior-izq devuelve las coordenadas del ángulo superior izquierdo del rectángulo. Este método superior-izq nos da la oportunidad de explicar un poco más acerca de la diferencia entre los twigils \$. y \$!. Hemos usado \$.esquina.abcisa para obtener la abcisa de la esquina, o sea, una invocación del método de acceso. Podríamos haber accedido los atributos esquina y altura de la clase Rectángulo directamente y usar la siguiente definición de método:

```
method superior-izq {
  $!esquina.abcisa, $!esquina.ordenada + $!altura;
}
```

Sin embargo, no sería posible \$!esquina!abcisa o \$.esquina!abcisa, dado que abcisa no es un atributo definido en la clase Rectángulo, y por lo tanto no se puede acceder directamente ahí. Puedes usar acceso directo del atributo (por ejemplo con la sintaxis \$!abcisa) solo dentro de la clase donde el atributo es definido, Punto2D. Así que, en Rectángulo, necesitamos invocar el método de acceso (o sea, la sintaxis con \$.) para obtener el valor de abcisa de la esquina.

Ahora podemos crear un objeto Rectángulo:

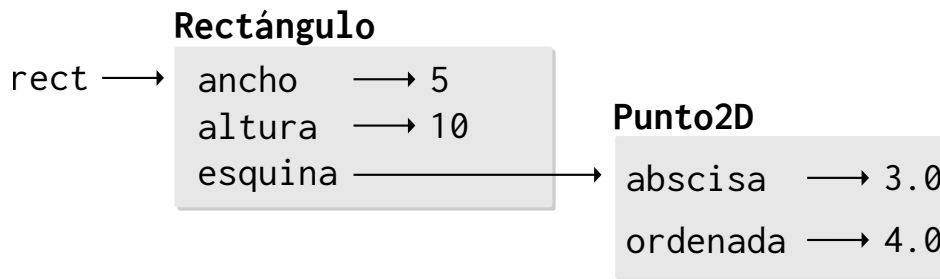


Fig. 12.2: Diagrama de objeto.

```
my $punto-inicio = Punto2D.new(abscisa => 4, ordenada => 3);
my $rect = Rectángulo.new(
  esquina => $punto-inicio,
  altura => 10,
  ancho => 5
);

say "Coord. de esquina superior-izq.: ", $rect.top-left;
# -> Coord. de esquina superior-izq.: (4 13)
say "Area del rectángulo: ", $rect.area;
# -> Area del rectángulo: 50
```

Podrás haber notado que los argumentos que pasan al constructor `Rectángulo.new` no están en el mismo orden como en la definición de la clase. Hice eso a propósito para mostrar que el orden de los argumentos no importan porque estamos usando argumentos nombrados.

La Figura 12.2 muestra el estado de este objeto.

El uso de un objeto como un atributo de otro objeto, posiblemente de otra clase, se conoce como **composición de objeto**. Un objeto que es un atributo de otro objeto es un objeto **embebido** (del inglés *embedded*). La composición de objeto hace posible definir capas anidadas de abstracción y es una característica poderosa de la programación orientada a objetos. En nuestro ejemplo de “geometría”, comenzamos a definir un objeto a un nivel inferior, una instancia `Punto2D`, y después usamos ese punto para construir un tipo a un nivel superior, `Rectángulo`.

12.6 Instancias como Valores de Retorno

Los métodos pueden devolver instancias de otras clase. Por ejemplo, la clase `Rectángulo` puede tener métodos que devuelven instancias de `Punto2D` para las otras esquinas:

```
method punto-superior-der {
  return Punto2D.new(
    abscisa => $!esquina.abscisa + $!ancho,
    ordenada => $!esquina.ordenada + $!altura
  );
}
# otros métodos para las otras esquinas
```

Nota que no tenemos que molestarnos en darle un nombre al punto superior derecho (aunque podríamos si quisiéramos); podemos crearlo con el constructor y devolverlo inmediatamente.

Podemos usar el nuevo método como sigue:

```
my $puntoSupDer = $rect.punto-superior-der;
say "Esquina superior derecha: ", $puntoSupDer;
# -> Esquina superior derecha: Punto2D.new(abscisa => 9, ordenada => 13)
```

Aunque esto no es muy útil en casos simples como este, podríamos asegurarnos y declarar un tipo `Punto2D` para `$puntoSupDer`:

```
my Punto2D $puntoSupDer = $rect.punto-superior-der;
```

De este modo, el código levantará un error si `punto-superior-der` devuelve algo más que una instancia `Punto2D`.

Similarmente, el método `encontrar-centro` invocado sobre un Rectángulo devuelve una instancia de `Punto2D` que representa el centro del Rectángulo:

```
method encontrar-centro { Punto2D.new(
    abscisa => $!esquina.abscisa + $!ancho / 2,
    ordenada => $!esquina.ordenada + $!altura / 2
);
}
```

Este nuevo método puede ser usado en la siguiente manera:

```
say "Centro = ", $rect.encontrar-centro;
# -> Centro = Punto2D.new(abscisa => 6.5, ordenada => 8.0)
```

12.7 Herencia

La herencia es probablemente la característica más emblemática de la programación orientada a objetos. Es un mecanismo a través del cual es posible derivar una clase de otra clase. La herencia es una de las maneras estándares de implementar la reutilización de código en la programación orientada a objeto. Es también otra manera útil de definir capas sucesivas de abstracción y una jerarquía de tipos.

12.7.1 La Clase Pixel

La clase `Punto2D` es muy general y podría usarse para una variedad de propósitos: geometría, gráficas vectoriales, mangas animados, etc. Podríamos querer usarlo para mostrar datos gráficos en la pantalla. Para este escenario, crearemos una nueva clase derivada `Pixel`, agregaremos nuevas propiedades al punto, tales como color, quizás transparencia, etc.

¿Necesitamos redefinir todos los atributos y los métodos para la nueva clase? No, no necesitamos hacer esto. Podemos definir una nueva clase que *hereda* las propiedades de la clase

base `Punto2D` y solo modificamos lo que no es adecuado o agregamos nuevas características si las necesitamos. Aquí, queremos un nuevo atributo para representar el color de un píxel y probablemente algunos otros métodos para manejar este nuevo atributo.

De acuerdo a los estándares más comunes, un color es definido por tres enteros (realmente tres octetos, es decir, enteros entre 0 y 255 en la notación decimal), que representan los componentes rojo, verde y azul de un píxel. Esta combinación de componentes es normalmente conocida como RGB (sigla de **R**ed, **G**reen y **B**lue en inglés):

```
class Pixel is Punto2D {
  has %.color is rw;

  method cambiar_color( %tono ) {
    %!color = %tono
  }
  method cambiar_color2( Int $rojo, Int $verde, Int $azul ) {
    # signatura usando parámetros de posición
    %!color = (rojo => $rojo, verde => $verde, azul => $azul)
  }
}
```

La nueva clase *hereda* las propiedades de la clase `Punto2D` gracias al rasgo `is Punto2D`, excepto posiblemente aquellos que son explícitamente modificados (o anulados) o agregados en la nueva clase. Esta nueva clase es algunas veces llamada una clase hija o subclase, mientras que `Punto2D` es la clase padre, clase base o superclase. La creación de esta nueva clase basada en `Punto2D` se conoce como **subclassing** de la clase base `Punto2D`.

La nueva clase hija hereda los atributos `abscisa` y `ordenada` de la clase padre `Punto2D` (y sus propiedades específicas si tienen algunas), al igual que los métodos tales como `coordenadas` el cual es definido en la clase padre. La clase hija tiene un nuevo atributo (el color) y dos métodos nuevos.

Instanciar un objeto `Pixel` es tan fácil como antes, solo necesitamos agregar un argumento adicional correspondiente al nuevo atributo cuando invoquemos el constructor `new`:

```
my $pix = Pixel.new(
  :abscisa(3.3),
  :ordenada(4.2),
  color => {rojo => 34, verde => 233, azul => 145},
);

say "El píxel original tiene los siguientes colores:", $pix.color;
```

En la definición de la clase `Pixel`, escribimos dos métodos diferentes para cambiar el color solo para ilustrar posibles formatos de sintaxis, para propósitos pedagógicos. El primer método recibe un hash como un parámetro, y el segundo usa parámetros de posición, los cuales forzan al usuario recordar el orden (RGB) en el cual los argumentos deben pasarse; esto puede ser una fuente de errores y debería evitarse cuando el número de parámetros excede un cierto límite (lo cual se dejará al lector para determinar). Por otro lado, cualquier persona que trabaja con gráficas sabe de memoria la convención estándar del orden de los colores (o sea, RGB). También, el segundo método tiene la ventaja de habilitar el chequeo

de tipo (los argumentos deben ser números enteros). Esto es un ejemplo simplificado; en la vida real, sería deseable chequear si los parámetros son octetos, es decir, enteros entre 0 y 255 (lo cual se podría hacer al añadir una restricción de tipo o al definir un subconjunto del tipo entero).

Usar la nueva clase `Pixel` es bien directo:

```
say "Colores originales: ", $pix.color;

$pix.cambiar_color({:rojo(195), :verde(110), :azul(70),});
say "Colores modificados: ", $pix.color;
say "Nuevas características del píxel:";
printf "\tAbscisa: %.2f\n\tOrdenada: %.2f\n\tColores: R: %d, G: %d, B: %d\n",
    $pix.abscisa, $pix.ordenada,
    $pix.color<rojo>, $pix.color{"verde"}, $pix.color{"azul"};

$pix.cambiar_color2(90, 180, 30); # argumentos de posición
say "Nuevos colores:
\tR: {$pix.color<rojo>}, G: {$pix.color<verde>}, B: {$pix.color<azul>} ";
```

Esto muestra lo siguiente en la pantalla:

```
Colores originales: {azul => 145, rojo => 34, verde => 233}
Colores modificados: {azul => 70, rojo => 195, verde => 110}
Nuevas características del píxel:
Abscisa: 3.30
Ordenada: 4.20
Colores: R: 195, G: 110, B: 70
Nuevos colores:
R: 90, G: 180, B: 30
```

Para decir la verdad, no era necesario usar dos métodos con nombres diferentes, `cambiar_color` y `cambiar_color2`, como hicimos en la definición de la clase `Pixel` para simplificar el asunto. Funcionaría de la misma manera si usáramos estas definiciones:

```
multi method cambiar_color( %tono ) {
    self.color = %tono
}
multi method cambiar_color( Int $rojo, Int $verde, Int $azul ) {
    # signature usando parámetros de posición
    self.color = (rojo => $rojo, verde => $verde, azul => $azul)
}
```

Debido a que el método `multi` se define dos veces, con el mismo nombre pero diferentes firmas, el sistema de objeto es capaz de despachar la invocación al método correcto.

12.7.2 La Clase `PuntoMovable`

Los atributos `$abscisa` y `$ordenada` de la clase `Punto2D` son por defecto atributos con solo acceso de lectura. Después de todo, cuando defines un punto en el plano, usualmente tiene una posición fija y generalmente no hay razón para cambiar su coordenada.

Sin embargo, supón que nuestra aplicación es acerca de la cinemática (la rama de la física que estudia el movimiento de puntos o cuerpos) o es un videojuego. En tal caso, probablemente queremos que nuestros puntos (o conjunto de puntos) puedan moverse. Necesitamos una nueva clase, `PuntoMovable`, para habilitar la modificación de las coordenadas.

No necesitamos redefinir todos los atributos y métodos para la nueva clase. Otra vez, podemos definir una nueva clase que *herede* las propiedades de la clase base `Punto2D` y solo modifique aquello que no es adecuado o agregue nuevas características que necesitemos, por ejemplo:

```
class PuntoMovable is Punto2D {
  has Numeric $.abscisa is rw;
  has Numeric $.ordenada is rw;

  method mover( Numeric $x, Numeric $y ) {
    $.abscisa += $x;
    $.ordenada += $y;
  }
}
```

La nueva clase hereda las propiedades de `Punto2D` gracias al rasgo `is Punto2D`, excepto que aquellos que son explícitamente modificados (o anulados) o agregados en la nueva clase. Los métodos que existen en la clase padre y se redefinen en la clase hija se dice que son *anulados* dentro esa clase.

Aquí, los atributos `$.abscisa` y `$.ordenada` se redefinen con acceso de lectura y escritura (a través del rasgo `is rw`) y un método nuevo, `mover`, es definido para modificar la posición de un punto al añadir los parámetros recibidos a las coordenadas del punto.

Nota que hemos usado parámetros de posición para el método `mover`. Dijimos que es usualmente mejor por motivos de claridad usar parámetros nombrados, pero hemos usado solo dos parámetros aquí; debido a que es bastante simple recordar que el parámetro `$x` debería venir antes que el parámetro `$y`. Esto fue una ocasión para ilustrar la posibilidad de usar parámetros de posición.

Ahora podemos probar nuestra clase hija, crear una instancia `PuntoMovable`, mostrar sus características, moverla a una posición diferente, y mostrar la posición nueva.

```
my $punto = PuntoMovable.new(
  abscisa => 6,
  ordenada => 7,
);

say "Coordenadas: ", $punto.coordenadas;
say "Distancia al origen: ", $punto.distancia-al-centro.round(0.01);
printf "%s: radio = %.4f, theta (rad) = %.4f\n",
  "Coordenadas polares", $punto.coordenadas-polares;

say "--> Moviendo el punto.";
$punto.mover(4, 5);
say "Nuevas coordenadas: ", $punto.coordenadas;
say "Distancia al origen: ", $punto.distancia-al-centro.round(0.01);
```

```
printf "%s: radio = %.4f, theta (rad) = %.4f\n",
    "Coordenadas polares", $punto.coordenadas-polares;
```

Esto produce la salida siguiente:

```
Coordenadas: (6 7)
Distancia al origen: 9.22
Coordenadas polares: radio = 9.2195, theta (rad) = 0.8622
--> Moviendo el punto.
Nuevas Coordenadas: (10 12)
Distancia al origen: 15.62
Coordenadas polares: radio = 15.6205, theta (rad) = 0.8761
```

Aquí, cuando el usuario invoca los métodos `coordenadas`, `distancia-al-centro`, y `coordenadas-polares`, Raku encuentra que dichos métodos no existen en la clase `PuntoMovable`. Pero, dado que `PuntoMovable` es una subclase de `Punto2D`, el programa hace una búsqueda de estos nombres en la clase padre, y los invoca si los encuentra. Si no los encuentra, entonces podría buscar en el padre de la clase padre para ver si se encuentran ahí, etc.

12.7.3 Herencia Múltiple: Atractiva Pero, es de Sabio Utilizarla?

En la programación orientada a objeto, el mecanismo de herencia es una manera tradicional de reutilizar código. Es probablemente la forma más común de hacerlo.

Un clase puede tener varias clases padres y, por lo tanto, ser una subclase de otras clases. Esto es lo que se conoce como **herencia múltiple**. Podríamos construir una nueva clase `PixelMovable` la cual hereda de `PuntoMovable` y de `Pixel` (e, indirectamente, de `Punto2D`). Técnicamente, puedes hacer esto fácilmente en Raku:

```
class PixelMovable is PuntoMovable is Pixel {
    # ...
}
```

Ahora, `PixelMovable` es una clase de `PuntoMovable` y `Pixel` y hereda de ambas clases padres.

Esto luce bastante prometedor, pero sucede que tiende a ser más complicado de lo esperado en situaciones reales. Si existe un conflicto (por ejemplo una colisión de nombre entre dos métodos), cuál debe prevalecer? Algunos mecanismo existen para manejar tales situaciones (por ejemplo en el lenguaje de programación C++), y Raku tiene métodos meta-objetos para encontrar información sobre el orden de resolución de los métodos (MRO por sus siglas en inglés), pero esto puede rápidamente conducir a varios problemas de diseño y a errores (*bugs*) que son sutiles y complicados. En resumen, mientras la herencia múltiple originalmente lucía como una idea atractiva, resultó ser en algo muy complicado de dominar, porque crea dependencias múltiples, y usualmente implícitas, que son difíciles de arreglar.

Esta es la razón por la cual, contrario a C++, lenguajes de programación orientados a objetos que son relativamente recientes como Java (el cual surgió no tan recientemente, en 1995) han decidido no implementar herencia múltiple.

Raku no quiere prohibir tales cosas y como resultado te permite hacer uso de la herencia múltiple si deseas, la cual puede ser muy útil para casos simples. Así que no la descarte innecesariamente, pero recuerda que, contrario a las expectativas originales, la herencia múltiple usualmente conduce a un desastre y resulta ser incontrolable.

Raku ofrece mejores conceptos para encargarse de tales situaciones, como veremos en un momento.

12.8 Roles y Composición

La herencia es un concepto poderoso para describir un árbol jerárquico de conceptos. Por ejemplo, puedes pensar de una jerarquía de figuras geométricas que tienen una o más propiedades específicas:

1. Polígono
2. Cuadrilátero (un polígono con cuatro lados y cuatro esquinas)
3. Trapezoido (un cuadrilátero con un par de lados paralelos)
4. Paralelogramo (un trapezoido con dos pares de lados paralelos y lados opuestos de la misma longitud)
5. Rectángulo (un paralelogramo con cuatro ángulos rectos)
6. Cuadrado (un rectángulo con cuatro lados iguales)

Es relativamente fácil de imaginar una serie de clases con un árbol de herencia jerárquica que refleje esas propiedades. Se vuelve más complicado, sin embargo, si agregamos el rombo (un paralelogramo con todos los lados iguales), porque el cuadrado es ahora *también* un rombo con cuatro ángulos rectos. La clase cuadrado sería una subclase del rectángulo y del rombo, y podríamos tener un posible caso de herencia múltiple.

Similarmente, podemos pensar acerca de un árbol de clases con herencia anidada que representan varios tipos de números (e.g., entero, racional, real, complejo) o especies de animales (e.g., vertebrado, mamífero, carnívoro, canino, perro, setter irlandés).

Estos son ejemplos grandiosos de herencia, pero el mundo real es raramente jerárquico, y es usualmente difícil de forzar cada caso a encajar en un modelo jerárquico.

Esta es una de las razones por la cual Raku introduce la noción de roles. Un rol es un conjunto de comportamientos o acciones que pueden compartirse entre varias clases. Técnicamente, un rol es una colección de métodos (posiblemente con algunos atributos); es por lo tanto similar a una clase, pero la primera diferencia obvia es que un rol no está diseñado para ser instanciado como un objeto (aunque los roles pueden ser promovidos al estado de clases). La segunda diferencia, quizás la más importante, es que los roles no se heredan: ellos se usan al ser aplicados a una clase o/y una composición.

12.8.1 Clases y Roles: Un Ejemplo

Volvamos con los vertebrados, mamíferos y perros. Un perro es un mamífero y hereda algunas características de los mamíferos tales como una neocórtex (una región del cerebro), pelo, y glándulas mamarias, también como una columna vertebral, la cual todos los mamíferos (incluyendo los peces, aves, reptiles, etc.) heredan de los vertebrados. Hasta ahora, la jerarquía de la clase parece simple y natural.

No obstante los perros pueden tener diferentes características y comportamientos. Para citar un artículo de Wikipedia sobre los perros: “Los perros tienen un sin número de *roles* para las personas entre los que figuran la caza, el pastoreo, halar cargas pesadas, asistir a la policía y el ejército, compañía y, recientemente, asistir a individuos incapacitados” (énfasis añadido). Los perros también pueden ser animales *salvajes* (i.e, animales que viven en ambientes silvestres pero que han descendido de individuos domesticados) o perros callejeros. Todos los comportamientos adicionales pueden añadirse a la clase perro. Similarmente, un gato, otro mamífero, puede ser una mascota o un animal salvaje. Los mustangos, caballos salvajes de Norteamérica, también son animales salvajes, descendientes en algún tiempo de caballos domesticados; pero un mustango puede capturarse y puesto devuelta en un estado domesticado. Este retorno a lo salvaje de animales silvestres no está limitado a los mamíferos: las palomas que viven en nuestras ciudades descendieron una vez de palomas mensajeras usadas en el pasado. Puede hasta ocurrir con invertebrados, tales como enjambres de abejas melíferas.

Es aparente que un modelo jerárquico de árboles de herencia no está adaptado para describir tales comportamientos.

Podemos definir clases para los perros, los gatos, y los caballos como subclases de mamíferos (los cuales también heredan de los vertebrados). Además de eso, podemos definir roles para mascotas o animales salvajes. En adición, podemos crear nuevas clases que son subclases de las clases perro, gato y caballo y que tienen algunos roles específicos. Del mismo modo, podemos asignar roles a instancias individuales de una clase. Esto podría lucir así (este un ejemplo ficticio que no se puede utilizar):

```
class Vertebrado { method hablar {say "vertebrado"}};
class Mamífero is Vertebrado { method hablar { say "mamífero" } }
class Ave is Vertebrado { method volar {} }
class Perro is Mamífero { method ladrar {} }
class Caballo is Mamífero { method relinchar {} }
class Gato is Mamífero { method maullar {} }
class Ratón is Mamífero { method chillar {} }
class Pato is Ave { method graznar {} }
# ...

role Animal-mascota {
  method es-compañero() {...}
  # otros métodos
}
role Ovejero { ... } # pastor de ovejas
role Salvaje { ... } # animal en un ambiente salvaje
role Guía { ... } # guía de los ciegos
role Humano-comp { ... } # animal que se comporta como un humano
# ...
```

```

class Perro-guía      is Perro    does Guía      { ... }
class Perro-ovejero   is Perro    does Ovejero    { ... }
class Perro-callejero is Perro    does Salvaje    { ... }
class Gato-mascota     is Gato     does Animal-mascota { ... }
class Gato-salvaje     is Gato     does Salvaje    { ... }
class Mustago         is Caballo  does Salvaje    { ... }
class Canario-dom     is Ave      does Animal-mascota { ... }
# ...
# Un rol se puede aplicar a instancias:
my $garfield = Gato-mascota.new(...);
my $mickey   = Ratón.new(...);
$mickey does Humano-comp;
my $donald   = Pato.new(...);
$donald does Humano-comp;
my $pluto    = Perro.new(...);
$pluto does Animal-mascota;
my $snoopy   = Perro.new(...);
$snoopy does Animal-mascota does Humano-comp;

```

Un rol se aplica a una clase o a un objeto con el rasgo `does` (opuesto a `is` para la herencia). Estas dos palabras claves diferentes reflejan la diferencia semántica asociada con ellos: componer un rol en una clase u objeto provee dicha clase u objeto con el *comportamiento suplementario* asociado con el rol, pero esto no significa que al objeto recibir el rol es la *misma cosa* o de la misma naturaleza como el rol.

Si los roles `Animal-mascota` y `Salvaje` hubiesen sido definidos como clase, entonces las clases `Gato-mascota` y `Gato-salvaje` hubiesen experimentado herencia doble, con los problemas potenciales asociados con eso. Al aplicar un rol a una clase, evitas construir un árbol de herencia múltiple que probablemente no se puede justificar y que puede ser difícil de conceptualizar y manejar. Un uso juicioso de clases y roles puede conducir a un modelo que es más simple, más natural, y más cercano a las relaciones reales entre las entidades y los comportamientos bajo escrutinio.

En adición, si compones varios roles inadvertidamente con dos métodos que poseen el mismo nombre, esto inmediatamente levanta un error (a menos que un método con el mismo nombre exista dentro de la clase y en tal caso, prevalece), en lugar de despachar silenciosamente a una de los métodos como en el caso de herencia múltiple. En ese caso, los conflictos de nombre son identificados inmediatamente (al tiempo de compilación), lo cual tiene el beneficio de encontrar un error inmediatamente, el cual podría no ser perceptible por un largo tiempo.

12.8.2 Composición de un Rol y Reutilización de Código

Las clases son para manejar instancias y los roles son para manejar comportamientos y reutilización de código. El siguiente ejemplo muestra cómo las clases y los roles funcionan bien juntos.

```

role Dibujable {
  has $.color is rw;
  method dibujar { ... }
}

```

```

}
class Figura {
  method area { ... }
}
class Rectángulo is Figura does Dibujable {
  has $.ancho;
  has $.altura;
  method area {
    $!ancho * $!altura;
  }
  method dibujar() {
    for 1..$.altura {
      say 'x' x $.ancho;
    }
  }
}
Rectángulo.new(ancho => 10, altura => 4).dibujar;

```

Ten presente que la intención de los puntos suspensivos ... usados en el código anterior es representar código que se deja para tu implementación. Sin embargo, esto es actualmente código válido y se compilará y hasta se ejecutará sin ningún problema. Los puntos suspensivos son usados para representar una funcionalidad que no se ha implementado todavía pero que se hará en el futuro. Esto funcionará siempre y cuando no invoques estos métodos (lo que producirá un error de ejecución) o crees una situación donde ellos necesitarían estar definidos (lo que ocasionará un error de compilación). En el caso del método dibujar en el rol Dibujable, la composición de rol en la clase Rectángulo funciona solo porque dibujar es redefinido en la clase Rectángulo; sin esta redefinición, hubiese levantado un error al tiempo de compilación. Similarmente, el código `method area { ... }` de la clase Figura levantaría un error al tiempo de ejecución si fuera llamado sin haber sido redefinido en la clase Rectángulo. Los puntos suspensivos se han usado aquí como una manera conveniente de representar código cuyo implementación no es importante para nuestro ejemplo porque se está redefiniendo de cualquier modo. En código real, es probablemente recomendado no usar puntos suspensivos, excepto como expediente temporario para código que no se ha desarrollado todavía pero que será implementado.

El código de más arriba dibuja un rectángulo ASCII:

```

$ raku test_drawable.raku
xxxxxxxxxx
xxxxxxxxxx
xxxxxxxxxx
xxxxxxxxxx

```

12.8.3 Roles, Clases, Objetos, y Tipos

Un rol se puede aplicar a una clase entera o solamente a algunas instancias de la clase:

```

role Guía { ... }
class Perro-guía is Perro does Guía {
  ...
}

```



```

} # Componiendo el rol Guía en la clase Perro-guía
  # la cual hereda de la clase Perro

my $perrito = new Perro; # creando un objeto Perro
$perrito does Guía;      # aplicando el rol al objeto

```

Los roles y las clases son diferentes, pero ambos son tipos o definen tipos. Esto significa que un rol puede usarse como un tipo para una declaración de variable donde esperaríamos el nombre de una clase. Por ejemplo, el rol `Guía` en el fragmento de código anterior efectivamente crea un tipo `Guía`. Así que un rol `Invidente` para un humano podría tener un atributo de tipo `Guía`, el cual representaría un perro-guía, un caballo-guía, un humano-guía o hasta un robot-guía.

```

class Humano {
  has Perro $perro; # Puede contener cualquier perro,
                   # con o sin un rol guía
}
role Invidente {
  has Guía $guía; # Puede contener un tipo Guía,
                 # un perro, un caballo, un humano, o un robot
}

```

Una gran mayoría de los tipos integrados de Raku son definidos por roles y no clases, tales como `IO`, `Iterable`, `Iterator`, `Numeric`, `Rational`, `Real`, etc.

12.9 Delegación de Método

La **delegación** es otra manera de enlazar un objeto a otra pieza de código. La técnica de delegación ha sido estudiada extensivamente al nivel teórico e implementada en algunos cuantos lenguajes especializados de investigación, pero la implementación de delegación en los lenguajes generalistas convencionales es bien inusual.

En lugar de definir métodos en una clase o en un rol, la idea es invocar los métodos perteneciente a otro objeto, como si fueran métodos de la clase actual. En Raku, la delegación puede realizarse al nivel de una clase o un rol. Un objeto delegado es simplemente un atributo definido en una clase o en un rol con la palabra clave `handles` que posibilita especificar cuales métodos del objeto delegado pueden ser usados en la clase actual:

```

class ClaseBase {
  method Don-Quijote() { "Cervantes" }
  method Hamlet()      { "Shakespeare" }
  method Three-Sisters () { "Chekhov" }
  method Don-Carlos()  { "Schiller" }
}

class Uses {
  has $.base is rw handles < Don-Quijote Hamlet Three-Sisters >;
}

my $user = Uses.new;

```

```

$user.base = ClaseBase.new(); # implementando un object-handler
say $user.Don-Quijote;
say $user.Hamlet;
say $user.Three-Sisters;
say $user.Don-Carlos;

```

Esto muestra lo siguiente:

```

Cervantes
Shakespeare
Chekhov
Method 'Don-Carlos' not found for invocant of class 'Uses'
  in block <unit> at delegate.raku line 16

```

El programa muestra apropiadamente los nombres de los escritores que los primeros tres métodos devuelve, porque ellos han sido más o menos “importados” en la clase `Uses`, pero falla con el último, porque “Don-Carlos” no es parte de la lista del handler. El error en el último método es una excepción al tiempo de ejecución y el programa terminaría la ejecución ahí aún hubiera algún código correcto después.

Nota que la clase `Uses` no sabe de donde los métodos serán importados; solo sabe sobre los nombres de los métodos que serán importados. Solamente cuando el objeto `$user` es creado y el atributo `$user.base` es añadido, el objeto es dinámicamente con los métodos definidos en `ClaseBase`. Casualmente, este proceso puede hacerse en un solo paso:

```

my $user = Uses.new( base => ClaseBase.new() );

```

No hay necesidad de enumerar los métodos que serán manipulados. La clase `Uses` puede importar todos los métodos de `ClaseBase`:

```

class Uses {
  has $.base is rw handles ClaseBase;
}

```

Esto funcionará como antes, excepto que no fallará con el método `Don-Carlos` ahora, debido a que este método fue importado también:

```

Cervantes
Shakespeare
Chekhov
Schiller

```

12.10 Polimorfismo

El polimorfismo es una manera de suplir una interfaz común o relacionada a tipos diferentes. En cierta manera, los ejemplos de herencia que estudiamos anteriormente ofrecen una forma de polimorfismo: los métodos coordenadas, distancia-al-centro, y coordenadas-polares son polimórficos, debido a que se pueden aplicar a los tipos `Punto2D`, `PuntoMovable` y `Pixel`. Hablaremos de polimorfismo cuando los métodos relevantes o funciones hacen cada uno algo diferente, por lo menos al nivel de implementación, aún compartan el mismo nombre y la misma interfaz.

Fuera de la programación orientada a objeto (POO), las subrutinas *multi* de Raku implementan una forma de polimorfismo, dado que se comportan de forma diferente dependiendo del tipo y número de sus argumentos. Dentro del contexto POO, es usualmente el tipo del invocante (su clase o posiblemente uno de sus roles) que determinará, usualmente al tiempo de ejecución, cuál de los posibles métodos será invocado.

Por ejemplo, podríamos desear crear una nueva clase para puntos en el espacio tridimensional. Los métodos tendrán que ser diferentes, pero sería interesante ofrecer al usuario un interfaz que es igual (o casi igual) a aquella de los puntos en dos dimensiones:

```
class Punto3D {
  has Numeric $.x;
  has Numeric $.y;
  has Numeric $.z;

  method coordenadas () {          # método de acceso a las 3 coordenadas
    return $.x, $.y, $.z
  }
  method distancia-al-centro () {
    return ($.x ** 2 + $.y ** 2 + $.z ** 2) ** 0.5
  }
  method coordenadas-polares () {
    return self.coordenadas-esféricas;
  }
  method coordenadas-esféricas {
    my $rho = $.distance2center;
    my $longitud = atan2 $.y, $.x;          # theta
    my $latitud = acos $.z / $rho;          # phi
    return $rho, $longitud, $latitud;
  }
  method coordenadas-cilíndricas {
    # ...
  }
}
```

Los métodos en esta nueva clase no son los mismos que en `Punto2D`, pero los métodos con semánticas similares tienen el mismo nombre; así que es posible usar cualquiera de las clases sin perderse con nombres diferentes.

El método `distancia-al-centro` tiene exactamente la misma interfaz. El método `coordenadas` devuelve una lista de tres valores en lugar de dos, pero la convención de llamada es la misma. Nota que podría haber sido posible diseñar `Punto2D` de tal manera que este método devolviera un tercer valor cero, para tener exactamente la misma interfaz (después de todo, un punto en el plano podría ser considerado como un punto en el espacio 3D con altura cero); cumplir exactamente con la misma interfaz no es mandatorio, pero solo una posible implementación que podría hacer una interfaz más intuitiva.

La noción de coordenadas polares no tiene un significado bien definido en el espacio 3D, pero he elegido mantener el nombre en nuestra interfaz porque es intuitivamente similar a la idea de coordenadas esféricas; no hace nada más que invocar el método `coordenadas-esféricas` sobre su invocante y devolver los valores de retorno.

Por favor nota que los matemáticos, físicos, astrónomos, ingenieros, geógrafos, y navegadores todos usan el mismo sistema básico para las coordenadas esféricas, pero sus convenciones son diferentes con respecto al origen, rango del ángulo, unidades medidas de ángulos y dirección de rotación, y el nombre de los varios valores o símbolos asociados con los mismos. Así que podrías encontrar diferentes fórmulas en un libro de texto. Las convenciones y fórmulas que hemos usado son comúnmente utilizadas en geografía y algunas ramas de las matemáticas. Una clase real de propósito general tendría que tomar en cuenta toda estas convenciones e implementar las convenciones necesarias.

12.11 Encapsulación

La encapsulación es la idea de ocultar los datos y código de una librería o un módulo del usuario. El concepto no es específico a la programación orientada a objeto, pero es parte fundamental de la misma.

En la programación orientada a objetos, la encapsulación consiste en proteger los datos en un objeto de la manipulación directa (y de manera inconsistente) por el usuario, quien accede tales datos a través de los métodos. Esto se logra al proveer al usuario métodos que son conocidos como *métodos de acceso* (o *getters*) y *mutadores* (o *setters*). Esto hace posible asegurar que las propiedades del objeto serán validadas por sus métodos.

La encapsulación es una forma poderosa de abstracción y abstracción procedimental. Visto desde afuera, un objeto es una caja negra que tiene propiedades y comportamientos específicos. De esta forma, estas propiedades y comportamientos están *ocultos* del usuario. No están ocultos en el sentido de que el usuario no puede saber sobre ellos (por lo menos en el mundo de código abierto (*open source*), es fácil saber esto), pero que están ocultos en el sentido de que es usualmente no posible usar ese conocimiento para eludir la interfaz proveída. Esto significa que la implementación interna del objeto puede cambiar sin tener que modificar el comportamiento externo. Si usas conocimiento privilegiado, tu código probablemente no funcionará correctamente cuando la implementación interna sea modificada. Por lo tanto, no hagas eso.

Varios lenguajes de programación no tienen las mismas reglas para garantizar encapsulación. Algunos son más estrictos que otros, algunos son menos estrictos sobre el acceso de lectura que el acceso de escritura, otros no hacen tal distinción pero más bien dependen en el nivel de visibilidad especificado por un atributo, por ejemplo “public” o “private” (algunas veces con un nivel intermedio “protegido”).

Raku te permite elegir el modelo de encapsulación que quieres aplicar a tus objetos y atributos. Todos los atributos son privados. Si declaras una clase así:

```
class Punto2D {  
    has $!abscisa;  
    has $!ordenada;  
    # ...  
    method valor_x { return $!abscisa }  
    method valor_y { return $!ordenada }  
}
```

las coordenadas `$!abscisa` y `$!ordenada` serán accesible solo dentro de la clase. Esta es la razón por la cual hemos añadido métodos de acceso. Además, los atributos son inmutables por defecto.

Pero como vimos anteriormente, si declaras esta clase así:

```
class Punto2D {
  has $.abscisa;
  has $.ordenada;
  # ...
}
```

las coordenadas serán todavía atributos privados, pero Raku generará métodos de acceso automáticamente con los mismos nombres de los atributos. De esta manera, se pueden acceder fuera de la clase casi como si fueran públicos:

```
class Punto2D {
  # ...
}
my $punto = Punto2D.new(abscisa => 2, ordenada => 3);
say $punto.abscisa;      # -> 2
```

El rasgo `is rw` maneja separadamente si un atributo es mutable o no. En resumen, Raku ofrece un modo acceso por defecto, pero puedes configurarlo y lo que necesitas.

12.11.1 Métodos Privados

Los métodos son la manera normal de usar objetos, ya sea con solo acceso de lectura o con acceso de lectura y escritura. Usualmente forman la *interfaz* de una clase, que es la parte de la clase que es pública y disponible para los programadores que deseen usarla. Es por lo tanto natural y legítimo que los métodos sean públicos, i.e., accesibles fuera de la clase.

Pero una clase puede también contener numerosos métodos que son parte de la receta de cocina interna de la clase, es decir, la forma en la que hace las cosas internamente, y que no están destinadas a ser usados fuera de la clase. Es posible prevenir su uso fuera de la clase al hacer estos métodos privados. Un método privado de Raku se prefija con un signo de exclamación:

```
method !comportamiento-privado( $x, $y ) {
  ...
}
```

También necesitarás usar un signo de exclamación para llamarlos:

```
$mi-objeto!comportamiento-privado($val1, $val2)
```

Los métodos privados son realmente internos de una clase dada. En particular, ellos no son heredados por las clases hijas.

12.11.2 Construir Objetos con Atributos Privados

Construir objetos con atributos privados trae consigo una pequeña dificultad. Consideremos el siguiente programa:

```
class Punto3D {
  has $.x;
  has $.y;
  has !$z;

  method coord_valores {
    return (!$x, !$y, !$z);
  }
};

my $a = Punto3D.new(x => 23, y => 42, z => 2);
say $_ for $a.coord_valores;
```

En este ejemplo, hemos declarado a \$.x y \$.y como atributos “públicos” (por así decirlo), y a !\$z como un atributo verdaderamente privado. La ejecución de este código muestra lo siguiente:

```
23
42
(Any)
```

¿Qué está sucediendo? Parece que el método coord_valores es incapaz de leer el contenido de !\$z, dado que devuelve un valor indefinido. Este método es definido dentro de la clase y debería ser capaz de acceder este atributo. De hecho, coord_valores no es el problema, sino que !\$z no está definido dentro del objeto, porque no se inicializó apropiadamente durante la creación del objeto. La culpa yace con el constructor implícito new que, por defecto, inicializa solo los atributos “públicos”.

Aquí la solución más simple es probablemente añadir el submétodo BUILD en la definición de la clase.

Un *submétodo* es un método público de una clase que no es heredado por sus clases hijas. Semánticamente, es realmente equivalente a una subrutina, pero se invoca con una sintaxis de método (de ahí su nombre). Los submétodos son especialmente útiles al realizar las tareas de construir y destruir un objeto que no deberían ser heredadas por las subclases, también como para las tareas que son tan específicas para tipo particular que las clases derivadas tendrán que seguramente modificarlas.

Inicializar los atributos privados durante la instanciación de un objeto podría lucir así:

```
class Punto3D {
  has $.x;
  has $.y;
  has !$z;

  submethod BUILD (:$!x, :$!y, :$!z) {
    say ";Inicialización!";
    $!x := $!x;
    $!y := $!y;
    $!z := $!z;
  }

  method coord_valores {
```

```

        return ($!x, $!y, $!z);
    }
};

my $a = Punto3D.new(x => 23, y => 42, z => 2);
say $_ for $a.coord_valores;

```

El programa ahora funciona como se deseaba y muestra todos los atributos:

```

;Inicialización!
23
42
2

```

Esto funciona porque el constructor por defecto `new`, un método definido la superclase `Mu` y heredado por cualquier clase de `Raku`, llama al submétodo por defecto `BUILD`. Si redefinimos a `BUILD` en nuestra clase, entonces reemplazará el submétodo por defecto que `new` llama. Al redefinir a `BUILD`, forzamos el constructor a tomar en cuenta el atributo privado que no se usó previamente.

Un poco de simplificación es posible. Dado que los argumentos que se pasan a una rutina enlazan los argumentos a los parámetros, un paso de enlace separado es innecesario si los atributos se usan como parámetros. Por lo tanto, el submétodo `BUILD` en el ejemplo anterior podría también escribirse tan simple como esto:

```

submethod BUILD( :$!x, :$!y, :$!z ) {
    say ";Inicialización!";
}

```

Mientras hablamos de las particularidades de la construcción de un objeto, nota que debido a que `new` es un método heredado desde la superclase `Mu`, puedes anularlo si lo prefieres. El constructor por defecto `new` puede usarse solo con argumentos nombrados. Asumiendo que realmente quieras parámetros de posición, puedes anular a `new` con tu propio método, de esta manera:

```

class Punto2D {
    has Numeric $.abscisa;
    has Numeric $.ordenada;

    method new ($x, $y) {
        self.bless(abscisa => $x, ordenada => $y);
    }
    method coordenadas {          # método de acceso a las coordenadas
        return (self.abscisa, self.ordenada)
    }
    # otros métodos
};

my $punto = Punto2D.new(3, 5);
say $_ for $punto.coordenadas;

```

Esto mostrará las dos coordenadas. `bless` es un método de nivel inferior para la construcción de objetos, heredado desde `Mu` y llamado automáticamente cuando invocas a `new` para construir un objeto. Usualmente no necesitas saber sobre ese método, excepto cuando quieres escribir tu propio constructor personalizado.

Puedes darle un nombre diferente al constructor, por ejemplo:

```
class Punto2D {
  has Numeric $.abscisa;
  has Numeric $.ordenada;

  method construir ($x, $y) {
    self.bless(abscisa => $x, ordenada => $y);
  }
  method coordenadas {          # método de acceso a las coordenadas
    return (self.abscisa, self.ordenada)
  }
  # otros métodos
};

my $punto = Punto2D.construir(3, 5);
say $_ for $punto.coordenadas;
```

Aunque deberías pensar dos veces antes de anular a `new` o crear tu propio constructor personalizado con un nombre diferente, dado que podría hacer más difícil la creación de subclases de la clase `Punto2D`.

12.12 Interfaz e Implementación

Una de las metas del diseño orientado a objetos es crear software más fácil de mantener, lo cual significa mantener el programa funcionando cuando otras partes del sistema cambia, y modificar el programa para satisfacer los requerimientos.

Un principio de diseño que ayuda a lograr esa meta es mantener la interfaz separada de las implementaciones. Para objetos, esto significa que la interfaz pública de los métodos proveídos por una clase no debería depender en cómo los atributos son representados.

Por ejemplo, diseñamos una clase `Punto2D` en la cual los atributos principales eran las coordenadas cartesianas del punto. Podemos descubrir que, para el propósito de nuestra aplicación, sería más fácil o más rápido almacenar las coordenadas polares del punto en los atributos del objeto. Es totalmente posible cambiar la implementación interna de la clase, y aún así mantener la misma interfaz. Para hacer esto, necesitamos que el constructor convierte los parámetros de entrada desde coordenadas cartesianas a polares, y almacenar las coordenadas polares en el atributo del objeto. El método `coordenadas-polares` devolvería los atributos almacenados, mientras que los métodos que devuelven las coordenadas cartesianas podrían tener que hacer la conversión opuesta (o quizás ser almacenados en atributos privados). En general, el cambio se podría hacer con la refactorización de la clase `Punto2D`, pero los usuarios de la clase todavía usarían la misma clase sin notar diferencia alguna.

Después de desplegar una nueva clase, podrías descubrir una mejor implementación. Si otras partes del programa están usando tu clase, podría ser un proceso largo y proclive a errores cambiar la interfaz.

Pero si diseñaste la interfaz cuidadosamente, puedes cambiar la implementación sin cambiar la interfaz, lo cual significa que las otras partes del programa no tienen que cambiar.

12.13 Programación Orientada a Objetos: Una Fábula

La mayoría de los libros y tutoriales que enseñan la programación orientada a objetos se enfocan en los aspectos técnicos de POO (como hemos hecho en este capítulo hasta ahora), y esa es una parte muy importante, pero algunas veces tienden a descuidar las razones de POO. Dicen “cómo”, pero no “porqué”. Hemos intentado explicar el “porqué” (y posiblemente logramos hacerlo bien), pero esta sección intenta explicar POO desde de la perspectiva de las razones y los beneficios de POO, independientemente de cualquier consideración técnica, en la forma de una parábola (los ejemplos y código son solo pseudocódigo y no se pueden compilar, menos ejecutarse).

12.13.1 La Fábula del Pastor

Había una vez un pastor que tenía un rebaño de ovejas. Su típico día de trabajo era así:

```
$pastor.mover_rebaño($pasto);  
$pastor.vigilar_rebaño();  
$pastor.mover_rebaño($casa);
```

Eventualmente, debido a una venta exitosa de lana, él expandió sus actividades granjeras y su día se volvió así:

```
$pastor.mover_rebaño($pasto);  
$pastor.vigilar_rebaño();  
$pastor.mover_rebaño($casa);  
$pastor.otro_tarea_importante();
```

Pero ahora el pastor quería dedicar más tiempo a `otro_tarea_importante()`, así que él decidió emplear un subordinado para hacerse cargo de las tareas relacionadas con las ovejas. De esta manera, el trabajo se dividió así:

```
$niño-pastor.mover_rebaño($pasto);  
$niño-pastor.vigilar_rebaño();  
$niño-pastor.mover_rebaño($casa);  
$pastor.otro_tarea_importante();
```

Esto otorgó más tiempo al pastor para la `otro_tarea_importante()`, pero desafortunadamente el `$niño-pastor` tenía la tendencia de gritar lobo. Como resultado, el pastor tuvo que reemplazarlo con un asistente nuevo:

```
$perro-ovejero.mover_rebaño($pasto);  
$perro-ovejero.vigilar_rebaño();  
$perro-ovejero.mover_rebaño($casa);  
$pastor.otro_tarea_importante();
```

El `$perro-ovejero` era más falible y demandaba un salario más bajo que el `$niño-pastor`, así que esto representaba una ganancia para el pastor.

12.13.2 La Moraleja

Podemos aprender varias cosas de esta parábola.

12.13.2.1 Delegación

Para manejar complejidad, delega a una entidad confiable, e.g., el granjero delegó parte de sus responsabilidades al \$niño-pastor.

12.13.2.2 Encapsulación

Indica a los objetos que hacer, en lugar de micro-gestionar, e.g.:

```
$perro-ovejero.vigilar_rebaño();
```

en lugar de algo así:

```
$perro-ovejero.cerebro.tarea.vigilar_rebaño();
```

A un nivel superior, no nos importa particularmente la estructura interna del objeto. Solo nos importa lo que el objeto puede hacer.

Mientras más se expone la estructura interna de un objeto, más difícil se vuelve cambiarlo.

12.13.2.3 Polimorfismo

\$perro-ovejero y \$niño-pastor ambos entienden los mismos comandos, así que el reemplazo del último con el primero fue más fácil de lo que hubiese sido por el contrario.

La fábula en esta sección es una adaptación de un post de "Arunbear" en el sitio web "PerlMonks": http://www.perlmonks.org/?node_id=1146129. Gracias a "Arunbear" por autorizarme a reutilizarla.

12.14 Depuración de Programas

Esta sección es sobre un depurador, un programa diseñado para ayudarte a depurar tus programas. "¿Qué? Tal herramienta existe, ¿y me lo dices solo ahora?" podrías quejarte. Bueno, no es realmente eso. Un depurador no a realizar tus tareas de depuración; todavía tienes que hacer el trabajo investigativo difícil, pero un depurador puede ayudarte a descubrir la razón por la cual tu programa no hace lo que crees que debería hacer. O, más bien, por qué lo que tu programa hace no es lo que quieres que haga.

Los depuradores son similares a personas con una fuerte personalidad: algunas personas los aman y otras los odian. Usualmente, las personas que no les gustan los depuradores simplemente nunca tomaron el tiempo de aprender cómo usarlos, pero existen algunos programadores expertos que no les gustan y de quienes no podemos sospechar de no intentar seriamente. Si te gustan o no te gustan los depuradores es probablemente un asunto de gusto personal, pero ellos pueden proveer una ayuda invaluable, si sabes cómo usarlos.

12.14.1 El Depurador de Raku

Rakudo incluye un depurador interactivo que llamas con el comando `raku-debug` (o, en algunas instalaciones, `raku-debug-m`). Puedes ejecutar este comando, seguido del nombre del programa a depurarse, como lo normalmente usaría `raku` con el nombre del programa para ejecutarlo. Una palabra de advertencia: puedes ejecutar el depurador con un programa solo si el programa compila sin ningún error; el objetivo de un depurador no es encontrar errores al tiempo de compilación, sino solo errores de ejecución o errores semánticos.

Una vez que has ejecutado el depurador, verás algo así:

```
>>> LOADING while_done.raku
+ while_done.raku (1 - 3)
| while True {
|     my $line = prompt "Enter something ('done' for exiting)\n";
|     last if $line eq "done";
|
>
```

Esto quiere decir que está cargando el programa `while_done.raku`, y muestra las primeras líneas del programa; la última línea al fondo ("`>`") es un prompt donde puedes entrar algunos comandos. El programa para en la primera sentencia que actualmente hace algo y espera por tu entrada. La línea de código que espera ser ejecutada es resaltada en un color diferente.

12.14.2 Cómo Conseguir Ayuda

El primer comando que probablemente quieres ejecutar es "`h`", el cual mostrará la ayuda del depurador y regresará al prompt. Abajo, hemos omitido la mayoría de la salida para ser breve:

```
> h
<enter>          single step, stepping into any calls
s                step to next statement, stepping over any calls
so               step out of the current routine
[...]
q[uit]           exit the debugger
>
```

Toma el tiempo para ejecutar ese comando y lee las posibles instrucciones que puedes entrar. Describiremos las más comunes. Como puedes ver arriba, solo usas "`q`" o "`quit`" para salir del depurador.

12.14.3 Recorriendo el Código

La característica principal de un depurador es que te deja ejecutar un programa paso a paso. Cada vez que presionas la tecla `Enter`, el programa se moverá un paso hacia adelante (e.g., una línea de código). Entrará en una subrutina si la línea de código es una llamada de subrutina, pero puedes pasar sobre la llamada de la subrutina al ejecutar el comando "`s`" en el prompt del depurador: esto ejecutará la subrutina y te colocará en la primera

línea de código después de la llamada de la subrutina (y cualquier llamada anidada de otras subrutinas) ha finalizado. Si entraste en la subrutina pero ya no estás interesado en recorrerla, solo entra el comando “so” para salir de ahí.

A cualquier punto del proceso, puedes mirar el contenido de las variables o hasta métodos sobre ellas. Para ver una variable, escribe su nombre y después presiona Enter:

```
> $line
"foo"
```

También puedes mirar un array o un hash, o usar el índice o la llave, por ejemplo @array[10] o %hash{"bar"}, para visualizar un elemento específico del array o del hash.

De igual modo, puedes utilizar “s” (o “say”) o “p” (o “print”) para evaluar y mostrar una expresión en el ámbito actual.

12.14.4 Parando en el Lugar Correcto con Puntos de Interrupción

Podría encontrarlo tedioso recorrer el programa paso a paso hasta que llegas a la parte interesante. Como sucede, tu puedes llegar ahí inmediatamente usando un *punto de interrupción* (*breakpoint*). Para agregar un punto de interrupción, escribe bp add línea, donde línea es el número de línea donde quieres que el programa pare la ejecución y vuelva de nuevo a recorrer el código línea a línea. Después entra el comando “r” y el programa se ejecutará hasta que alcance uno de los puntos de interrupción que has creado. La ejecución también parará si el programa llega a una excepción; en ese caso, puedes todavía acceder las variables para intentar descubrir lo que pasó. Si no alcanza ningún punto de interrupción o una excepción, se ejecutará hasta el final.

Puedes ver todos los puntos de interrupción (bp list), remover un punto de interrupción (bp rm línea), o remover todos los puntos de interrupción (bp rm all). También puedes crear un punto de interrupción en otro archivo (por ejemplo si estás usando un módulo) al usar la siguiente sintaxis: bp add archivo:línea, donde “archivo” es el nombre del archivo.

12.14.4.1 Ya Estás Listo para Usar el Depurador

Probablemente ya sabes lo suficiente para hacer buen uso del depurador de Raku, recorrer tu programa y descubrir donde hace algo que no está supuesto hacer. No fue tanto que aprender, cierto? ¡Inténtalo!

Aún discutiremos varios puntos adicionales.

12.14.5 Registrar Información con Puntos de Trace

Es posible crear puntos de trace en líneas específicas de código y variables (o expresiones), con el comando tp add línea \$var. Esto registrará el valor de \$var cada vez que el programa alcanza la línea elegida. Después simplemente ejecutas el programa por un momento y, en algún punto, puedes visualizar cómo la variable cambió a lo largo del tiempo, usando el comando tp show.

Por ejemplo, lo usamos para registrar la variable \$palabra-rotada en la solución del ejercicio sobre el cifrado César (ver Solución: A.5.9) para la cadena de texto de entrada “ABCD-abcd” con una rotación de 25 letras; el comando `tp show` mostró cómo la cadena de texto codificada fue progresivamente poblada letra por letra:

```
> tp show
>>> rotar.raku:23
*
* Z
* ZA
* ZAB
* ZABC
* ZABCz
* ZABCza
* ZABCzab
```

12.14.6 Recorriendo una Coincidencia Regex

El depurador puede también proveer información útil cuando el código está intentando coincidir un regex. Por ejemplo, supón que ejecutamos un programa con el depurador con el siguiente código:

```
"foobar" ~~ /f.+b/;
```

Si ejecutas el regex paso a paso, la coloración de sintaxis mostrará átomo por átomo donde está en el regex y que parte de la cadena de texto coincide. (No podemos mostrar la coloración de sintaxis aquí, pero deberías intentarlo para verlo.)

Con el regex arriba, verás que el motor de regex intenta coincidir el carácter “f” del patrón y encuentra un “f” al principio de la cadena de texto; siguiente, verás que el motor de regex intenta coincidir el subpatrón “.” y que coincide la cadena de texto entero; después, cuando el motor de regex intenta coincidir la última “b” del patrón, verás que el motor de regex retrocede, se deshace de la “r” y después de la “a”; finalmente, el motor de regex tiene éxito con “foob”.

Si tienes dificultad entendiendo cómo los regexes trabajan o estás mistificado por el retroceso (*backtracking*), solo ejecuta el depurador sobre algunos regexes y observa lo que sucede paso a paso. No tienes ni que escribir un programa; puedes usarlo en el modo de una sola línea (*one-liner*). Por ejemplo, para probar el regex de arriba como un one-liner en Windows, solo escribe el comando siguiente en el prompt:

```
C:\Users\Laurent>raku-debug-m -e "'foobar' ~~ /f.+b/;"
```

Como es usual, cambia las comillas dobles a comillas simples si estás usando cualquier plataforma de Unix.

Nuestra nota final sobre el depurador: recuerda siempre entrar “h” para conseguir ayuda sobre el comando que necesitas.

12.15 Glosario

Objeto Una entidad que encapsula su estado (atributos) y su comportamiento (métodos).

Clase Un tipo definido por el programador. La definición de una clase crea un nuevo tipo de objeto (una forma de abstracción) y hace posible la instanciación de objetos concretos que representan datos reales.

Método Un tipo especial de subrutina dentro de una clase o un rol, que puede llamarse usando la sintaxis de la notación del punto.

Tipo objeto Un objeto que contiene información sobre un tipo definido por el programador. El objeto de tipo puede usarse para crear instancias del tipo.

Instancia Un objeto que pertenece a una clase y contiene datos reales.

Instanciar Crear un nuevo objeto.

Atributo Una propiedad de estado dentro de la estructura POO que es similar a una variable. El atributo de una instancia es uno de los valores nombrados asociados con un objeto. Los atributos de una clase son las variables asociadas con la clase entera.

Objeto embebido Un objeto que es almacenado como un atributo de otro objeto.

Composición de objeto Uso de un objeto como parte de la definición de otro objeto, especialmente el uso de un objeto como un atributo de otro objeto.

Diagrama de objeto Un diagrama que muestra objetos, sus atributos, y los valores de dichos atributos.

Rol Una colección de métodos similar a una clase pero que no está diseñada para construir objetos. Un rol contiene métodos que pueden aplicarse a una clase o a un objeto para añadir comportamientos a ellos.

Polimórfico Perteneciente a una función que funciona con más de un tipo.

Encapsulación El principio que dicta como la interfaz proveída por un objeto no debería depender en su implementación, en particular la representación de sus atributos. Esto también se conoce como *ocultación de información*.

Herencia La habilidad de crear una nueva clase la cual es una versión modificada de otra clase previamente definida.

Clase padre La clase desde la cual la clase hija hereda. También conocida como *superclase* o *clase base*.

Clase hija Una nueva clase que hereda de una clase existente; también llamada *subclase*.

Subclassing El proceso de crear una clase hija derivada de una clase padre.

Anular Cuando un método de una clase padre es redefinido en una clase hija, se dice que ha sido anulado dentro de la clase hija.

Herencia múltiple Una situación en la cual una clase hija es derivada y hereda desde más de una clase padre.

Delegación Definir una clase o un rol en el cual es posible invocar métodos que pertenecen a otro objeto.

Regexes y Gramáticas

Las expresiones regulares o regexes fueron introducidas de la Sección 7.5 a la Sección 7.9. Es recomendable que leas esas secciones nuevamente antes de leer este capítulo si no recuerdas mucho sobre los regexes. No necesitas recordar los detalles de todo lo que discutimos anteriormente y explicaremos de nuevo, sino brevemente, las partes específicas de la funcionalidad que usaremos, pero se espera que entiendas cómo los regexes funcionan en general.

13.1 Una Breve Actualización

Los regexes, como los estudiamos hasta ahora, tratan sobre la exploración de cadenas de texto usando patrones. Un patrón es una secuencia de caracteres (usualmente especiales) que describen una cadena de texto o parte de una cadena de texto. Un patrón coincide con una cadena de texto si existe una correspondencia entre el patrón y la cadena de texto.

Por ejemplo, el siguiente fragmento de código inspecciona la cadena de texto y trata de encontrar la letra “a”, seguida por cualquier número (pero por lo menos una) de letras “b” o “c”, seguida por cero o más dígitos seguidos por una “B” o una “C”:

```
my $cad = "foo13abbccbcbb42Cbar";
say ~$/ if $cad ~~ /a <[bc]>+ (\d*) [B|C]/; # -> abbccbcbb42C
say ~$0;                                     # -> 42
```

Este fragmento de código usa el operador de coincidencia inteligente `~~` para chequear si la cadena `$cad` coincide con el patrón `/a <[bc]>+ (\d*) [B|C]/`. Recuerda que los espacios en blanco no son usualmente significativos en un patrón de regex (a menos que se especifiquen).

El patrón está compuesto por los siguientes componentes:

- `a`: una coincidencia literal de la letra “a”

- `<[bc]>+`: el átomo `<[bc]>` es una categoría de caracteres que se refiere a la letra “b” o “c”; el cuantificador `+` dicta que los caracteres que coinciden con la categoría de caracteres “b” o “c” pueden repetirse una o más veces.
- `(\d*)`: el átomo `\d` es una categoría de caracteres que son dígitos, el cuantificador `*` significa 0 o más ocurrencias del átomo anterior, y los paréntesis requieren una captura de estos dígitos (si existen) en la variable `$0` (una variable especial que es realmente un atajo para `$/[0]`)
- `[B|C]`: `B|C` es una alternación (una “B” o un “C”), y los corchetes reagrupan esta alternación en un (y también habilita precedencia adecuada).

Si la coincidencia es exitosa (como en este ejemplo), el resultado se almacena en el *objeto de coincidencia*, `$/`. La impresión de `~$/` muestra una versión en cadena de texto del objeto de coincidencia. Y la impresión de `$0` (o `$/[0]`) muestra la captura (la porción de la coincidencia entre los paréntesis, en este caso el número “42”).

Esto es lo que se llama coincidencia a un bajo nivel: el reconocimiento del patrón se hace mayormente al nivel individual de un carácter. Raku ofrece varias maneras de agrupar y nombrar los patrones de regex para que estos patrones individuales puedan utilizarse posteriormente como componentes básicos para coincidencia a un alto nivel: el reconocimiento de palabras y secuencias de palabras (más que caracteres), para el propósito de hacer lo que se conoce como análisis léxico (o **lexing**) y análisis sintáctico (o **parsing**) en una pieza de texto.

Este capítulo está mayormente dedicado este tipo de coincidencia, que conduce a la creación de gramáticas completas que analizan textos estructurados tales como textos XML o HTML, documentos JSON o YAML, o hasta programas de computadora: los programas de Raku son actualmente analizados sintácticamente usando una gramática de Raku escrita en Raku.

Las gramáticas son un tema muy importante en la ciencia de la computación pero, obviamente, no todos los programadores escriben gramáticas completas para analizar sintácticamente lenguajes de programación. No obstante, la escritura de una gramática simple y un analizador sintáctico simple podría ser, o quizás debería ser, una actividad mucho más común.

A menudo, la gente invierte demasiado esfuerzo en el descifrado de archivos simples de configuración con técnicas de bajo nivel, mientras que la escritura de un simple analizador sintáctico podría ser mucho más fácil y mucho más eficiente. Raku ofrece todas las herramientas para hacer eso muy fácilmente.

Algunas veces, también necesitas desarrollar un lenguaje de dominio específico (DSL por sus siglas en inglés), es decir, un sub-lenguaje relativamente pequeño (también conocido como una jerga) adaptado a un campo específico del conocimiento (ciencia, ingeniería, negocios, arte, u otro) con sus propias convenciones, símbolos, operadores, etc. Con una gramática y la habilidad de Raku para crea sus propios operadores, puedes usualmente expresar conocimiento especializado dentro del marco de terminología de los expertos de una área del conocimiento en particular.

13.2 Programación Declarativa

Los regexes y las gramáticas son ejemplos de aún otro tipo de paradigma de programación que no hemos explorado hasta ahora: **programación declarativa**. Este es un modelo de programación en el cual, contrario a la programación imperativa o procedimental, no indicas cómo hacer algo ni tampoco eliges tu flujo de control. En lugar de eso, tú especificas un conjunto de definiciones, reglas, propiedades, y posiblemente algunas restricciones y acciones, y dejas al programa aplicar estas definiciones para derivar nueva información sobre los datos de entrada.

Esta forma de programación se usa extensivamente en la programación lógica (e.g., Prolog), inteligencia artificial, sistemas expertos, análisis de datos, lenguajes para consultar de bases de datos (e.g., SQL), reconocimiento de texto y código fuente (e.g., Lex y Flex), compilación de programas (e.g., Yacc o Bison), manejo de configuraciones, makefiles, y también de alguna forma, la programación funcional.

13.3 Capturas

Como explicamos en los ejemplos de regex al comienzo de este capítulo, los paréntesis no solo agrupan cosas sino que también **capturan** datos: ellos hacen que la cadena de texto que coincide con el subpatrón dentro de los paréntesis se encuentre disponible dentro de una variable especial:

```
my $cad = 'número 42';
say "El número es $0" if $cad =~ /número \s+ (\d+)/; # -> El número es 42
```

Aquí, el patrón coincidió la cadena de texto \$cad, y la parte del patrón dentro de los paréntesis fue capturada en la variable especial \$0. Si hay varios grupos con paréntesis, ellos serán capturados en las variables \$0, \$1, \$2, etc. (de izquierda a derecha):

```
say "$0 $1 $2" if "abcde" =~ /(a) b (c) d (e)/; # -> a c e
```

Esto funciona muy bien con capturas simples, pero el número de capturas puede volverse tedioso si hay muchas capturas y algo complicado cuando hay paréntesis anidados en el patrón:

```
if 'abc' =~ / ( a (.) (.) ) / {
  say "Fuera: $0";           # Fuera: abc
  say "Dentro: $0[0] y $0[1]"; # Dentro: b y c
}
```

Cuando se vuelve complicado, es usualmente mejor usar otra característica conocida como *capturas nombradas*. La manera estándar de nombrar una captura es de la siguiente forma:

```
if 'abc;%' =~ / $<nombre_de_captura> = \w+ / {
  say ~$<nombre_de_captura>;           # abc
}
```

El uso de la captura nombrada, \$<nombre_de_captura>, es un atajo para acceder el objeto de coincidencia \$/ como un hash, en otras palabras: \$/{ 'nombre_de_captura' } o \$/<nombre_de_captura>.

Las capturas nombradas pueden anidarse usando la sintaxis regular de captura de grupo:

```

if 'abc' =~ / $<todo>=( a $<parte1>=(.) $<parte2>=(.) ) / {
  say "Todo: $<todo>";           # Todo: abc
  say "Parte 1: $<todo><parte1>"; # Parte 1: b
  say "Parte 2: $<todo><parte2>"; # Parte 2: c
}

```

Asignar el objeto de coincidencia a un hash ofrece acceso programático fácil a todas las capturas nombradas:

```

if 'abc' =~ / $<todo>=( a $<parte1>=(.) $<parte2>=(.) ) / {
  my %captura = $/.hash;
  say ~%captura<todo>;           # -> abc
  for kv %captura<todo> -> $clave, $val {
    say $clave, " ", ~$val;      # -> parte2 c \n parte1 b
  }
}

```

Pero podrías hacer la misma cosa directamente en el objeto de coincidencia sin tener que hacer una asignación de hash extra:

```

if 'abc' =~ / $<todo>=( a $<parte1>=(.) $<parte2>=(.) ) / {
  say "Todo: $<todo>";           # -> Todo: abc
  for kv %<todo> -> $clave, $val {
    say $clave, " ", ~$val;      # -> parte2 c \n parte1 b
  }
}

```

Recuerda que, en el código más arriba, \$<todo> es realmente un atajo para \$/<todo>, o sea, para un tipo de acceso hash al objeto de coincidencia \$/.

No obstante, existe una manera más conveniente de obtener capturas nombradas la cual discutimos en la siguiente sección.

13.4 Reglas Nombradas (o Subreglas)

Es posible almacenar piezas de regexes dentro de *reglas nombradas*. El ejemplo siguiente usa un regex nombrado, el cual es un tipo de reglas nombradas, para coincidir con una línea de texto:

```

my regex línea { \N* \n } # cualquier número de caracteres excepto
                          # una nueva línea, seguida por 1 nueva línea

if "abc\ndef" =~ /<línea> def/ {
  say "Primera línea: ", $<línea>.chomp; # Primera línea: abc
}

```

Nota que la sintaxis con un bloque de código se parece a la definición de una subrutina o un método. Esto no es una casualidad; veremos que las reglas nombradas son muy similares a los métodos. Notablemente, las reglas pueden llamarse unas a las otras (o hasta algunas

veces ellas mismas de forma recursiva) al igual que los métodos y las subrutinas, y también veremos que esto es una característica muy poderosa y expresiva.

Un regex nombrado puede declararse con `my regex nombre { cuerpo del regex }`, y llamarse con `<nombre>`.

Como puedes ver en el ejemplo más arriba, un regex nombrado que es exitoso crea una captura nombrada con el mismo nombre. Si necesitas un nombre diferente para la captura, puedes hacer esto con la siguiente sintaxis `<nombre-captura=nombre-regex>`. En este ejemplo, llamamos el mismo regex nombrado dos veces y, por conveniencia, usamos un nombre diferente para distinguir las dos capturas:

```
my regex línea { \N* \n }
if "abc\ndef\n" ~~ / <primera=línea> <segunda=línea> / {
  say "Primera línea: ", $<primera>.chomp; # -> Primera línea: abc
  say "Segunda línea: ", $<segunda>.chomp; # -> Segunda línea: def
  print $_.chomp for $<línea>.list;      # -> abcdef
}
```

En este ejemplo, usamos las invocaciones del método `chomp` para remover el carácter de nueva línea de las capturas. Existe de hecho una manera para coincidir con el carácter de nueva línea pero excluirla de la captura:

```
my regex línea { \N* >> \n }
if "abc\ndef\n" ~~ / <primera=línea> <segunda=línea> / {
  say "Primera línea: ", ~$<primera>; # -> Primera línea: abc
  say "Segunda línea: ", ~$<segunda>; # -> Segunda línea: def
  print $<línea>.list;                # -> abc def
}
```

Este poco conocido token, `>>`, marca el punto final de la captura completa de una coincidencia. Cualquier cosa después del token participará en la coincidencia pero no será capturada por el regex nombrado. Similarmente, el token `<<` indica el inicio de la captura.

Los regexes nombrados son solo una forma (y probablemente no la más común) de las reglas nombradas, que vienen en tres sabores distintos:

- Regexes nombrados, en la cual el regex se comporta como los regexes ordinarios
- Tokens nombrados, en la cual el regex tiene un adverbio `:ratchet` implícito, lo cual significa que no hay retroceso o vuelta atrás
- Reglas nombradas, en la cual el regex tiene un adverbio `:ratchet` implícito, como los tokens nombrados, y también un adverbio `:sigspace` nombrado, lo cual significa que los espacios en blanco dentro del patrón (o, más específicamente, entre los caracteres de palabra) no son ignorados

En los dos ejemplos arriba, no necesitamos que los regexes volvieran atrás. Podríamos (y probablemente deberíamos) haber usado un token nombrado en lugar de un regex nombrado:

```

my token línea { \N* \n }
if "abc\ndef" ~~ /<línea> def/ {
  say "Primera línea: ", $<línea>.chomp;    # Primera línea: abc
}

```

Pero, para que una regla coincida, tendríamos que remover el espacio desde *dentro del patrón*:

```

my rule línea { \N*\n } # Nota que no hay espacio entre \N* y \n
if "abc\ndef" ~~ /<línea> def/ {
  say "Primera línea: ", $<línea>.chomp;    # Primera línea: abc
}

```

Independientemente de la palabra clave específica usada para sus definiciones, es usual referirse a estos tres tipos de reglas nombradas colectivamente como reglas.

¿Recuerdas los varios regexes con los cuales experimentamos para extraer las fechas de una cadena de texto en la Subsección 7.8.1 (p. 125)? El último ejemplo usó subpatrones como componentes básicos para construir el patrón completo. Podríamos ahora escribirlo, con la característica añadida para reconocer formatos de fechas múltiples, en la siguiente manera:

```

my $cadena = "Christmas : 2016-12-25.";
my token año { \d ** 4 }
my token mes {
  1 <[0..2]>                                # 10 to 12
  || 0 <[1..9]>                             # 01 to 09
};
my token día { (\d ** 2) <?[1 <= $0 <= 31 ]> }
my token separador { '/' || '-' || '.' }
my rule fecha { <año> (<separador>) <mes> $0 <día>
  || <día> (<separador>) <mes> $0 <año>
  || <mes>\s<día>',' <año>
}

if $cadena ~~ /<fecha>/ {
  say ~$/;                                # -> 2016-12-25
  say "Día\t= " , ~$/<fecha><día>;        # -> 25
  say "Mes\t= " , ~$/<fecha><mes>;         # -> 12
  say "Año\t= " , ~$/<fecha><año>;         # -> 2016
}

```

Los primeros cuatro tokens nombrados definen los componentes básicos para coincidir con el año, el mes, el día, y los posibles separados. Después, la regla nombrada *fecha* usa los componentes básicos para definir una alternancia entre los posibles tres formatos de fecha.

Este código chequea que el día en el mes esté entre 1 y 31 y que el mes esté entre 01 y 12, y esto es probablemente suficiente para reconocer fechas en un texto en la mayoría de los casos, pero esto coincidiría con “2016-11-31” aunque Noviembre tiene solo 30 días. Podemos ser un poco más estrictos sobre las fechas válidas y prevenir eso al agregar una inserción de código negativa a la regla nombrada *fecha*:

```

my rule fecha { [ <año> (<separador>) <mes> $0 <día>
                || <día> (<separador>) <mes> $0 <año>
                || <mes>\s<día>',' <año>
                ] <![ $<día> > 30 and $<mes> == 2|4|6|9|11]>
}

```

Esto es mejor, pero todavía podemos coincidir con una fecha inválida tal como “2016-02-30”.

Ejercicio 13.1. Como un ejercicio, cambia la aserción de código para rechazar una fecha como “Feb. 30”. Si te sientes valiente, podrías hasta querer chequear el número de días en febrero dependiendo si la fecha ocurre en un año bisiesto. También podrías intentar definir y probar otros formatos de fecha. Solución: A.10.1.

Las reglas pueden (y usualmente deberían) agruparse en gramáticas; esa es la razón por la cual fueron diseñadas.

13.5 Gramáticas

Las gramáticas son una herramienta poderosa que se utilizan para analizar datos textuales y usualmente devuelven estructuras de datos que se crean al interpretar dichos textos.

Por ejemplo, cualquier programa de Raku se analiza sintácticamente y se ejecuta usando una gramática de Raku escrita en Raku, y podrías escribir una gramática para analizar sintácticamente (casi) cualquier otro lenguaje de programación. Para ser honesto, los programadores raramente escriben gramáticas para analizar sintácticamente lenguajes de programación. No obstante, las gramáticas son muy útiles para realizar muchas tareas que son mucho más comunes que analizar programas sintácticamente.

Si alguna vez intentaste usar regexes para analizar una pieza de texto HTML (o XML)¹, probablemente encuentres que se volvió casi imposible, excepto quizás por los datos de HTML más fáciles. Para analizar cualquier pieza de tales datos, necesitas un analizador sintáctico el cual, por su parte, estará basado en una gramática.

Si no te gustó la gramática en la escuela, no dejes que eso te intimide. Las gramáticas de Raku no son complicadas; ellas solo te permiten agrupar reglas nombradas, en la misma forma que las clases te permiten agrupar métodos de código regular.

Una gramática crea un espacio de nombres (del inglés *namespace*) y se introduce con la palabra clave `grammar`. Usualmente agrupa un número de reglas nombradas, en la misma manera que una clase agrupa un número de métodos. Una gramática es actualmente una clase que hereda de la superclase `Grammar`, la cual provee métodos tales como `parse` para analizar sintácticamente una cadena de texto y `parsefile` para analizar un archivo. Además, puedes actualmente escribir algunos métodos en una gramática, e incluso importar algunos roles. Y, como veremos, las gramáticas son usualmente asociadas con algunas **clases de acciones** o **objetos de acciones**.

A menos que se diga lo contrario, los métodos de análisis sintáctico buscarán una regla nombrada por defecto llamada “TOP” (la cual puede ser un regex nombrado, un token, o una regla) para comenzar el análisis. Las reglas de análisis sintáctico de una fecha que usamos más arriba pueden ensamblarse en una gramática de la siguiente manera:

¹No intentes hacerlo. Ya te lo advertí: no lo hagas.

```

grammar Mi-fecha {
  rule TOP { \s*?
    [   <año> (<separador>) <mes> $0 <día>
      || <día> (<separador>) <mes> $0 <año>
      || <mes>\s<día>',' <año>
    ] \s*
    <![ (<día> > 30 and <mes> == 2|4|6|9|11)]>
  }

  token año    { \d ** 4 }
  token mes    { 1 <[0..2]> || 0 <[1..9]> }
  token día    { (\d ** 2) <?[1 <= $0 <= 31 ]> }
  token separador { '/' || '-' || '.' }
}

for " 2016/12/25 ", " 2016-02-25 ", " 31/04/2016 " -> $cadena {
  my $coincidencia = Mi-fecha.parse($cadena);
  say ~$coincidencia if defined $coincidencia;
}

```

Esto imprimirá:

```

2016/12/25
2016-02-25

```

La aserción de código dentro de la regla “TOP” previene que fechas como “31/04/2016” coincidan; necesitarías añadir más código para manejar los finales de las fechas de Febrero, como hicimos en la solución del ejercicio anterior (ver Solución: A.10.1) si esto es importante. Podrías querer hacerlo como un ejercicio.

Además de eso, este código no es tan diferente de nuestro código anterior, pero hay algunos cambios que son significativos.

Nombré la regla `fecha` como `TOP` porque este es el nombre por defecto que el método `parse` busca para la regla en el nivel superior. Una gramática crea su propio espacio de nombres y ámbito léxico, y ya no necesito declarar las reglas con el declarador `my` (el cual se requiere para las reglas que se declaran fuera de una gramática).

Dentro de una gramática, el orden en el cual las reglas se definen es usualmente irrelevante, así que yo podría definir la regla `TOP` primero, aún si usa los tókenes que se definen después (lo cual no habría sido posible con las reglas usadas fuera de una gramática). Esto es importante porque, dentro de una gramática, puedes tener reglas que se llaman unas a las otras (o reglas que se llaman a ellas mismas de forma recursiva), lo cual no sería práctico si el orden de las definiciones de las reglas fuera importante.

Si analizas la cadena de texto de entrada con el método `.parse`, la regla `TOP` es automáticamente anclada al inicio y al final de la cadena de texto, lo que significa que la gramática tiene que coincidir con la cadena de texto completa para ser exitosa. Esta es la razón por la cual tuvimos que añadir patrones para los espacios en blanco al inicio y al final de nuestra regla `TOP` para coincidir con nuestras cadenas de texto que tienen espacios en blanco antes y después de la fecha. Existe otro método, `.subspace`, el cual no tiene que alcanzar el final de la cadena de texto para ser exitoso, pero todavía debemos tener el patrón para los espacios en blanco al inicio de la regla.

13.6 Herencia de una Gramática

Una gramática puede heredar de otra gramática, de la misma manera en que una clase puede heredar de otra clase.

Considera esta gramática simple (casi simplista) para analizar sintácticamente un correo electrónico:

```
grammar Mensaje {
  rule TOP      { <saludo> $<cuerpo>=<línea>+? <final> }
  rule saludo   { [Saludos|Hola|Hey] $<a>=\S+? ', ' }
  rule final    { Nos vemos luego ', ' $<desde>=+.+ }
  token línea   { \N* \n }
}
```

Podemos probarla con el siguiente código:

```
my $msg = "Saludos Tomás,
Espero que todo esté bien y que hayas reparado tu carro.
Nos vemos luego, Liz";

my $coincidencia = Mensaje.parse($msg);
if defined $coincidencia {
  say "Saludo      \t= ", ~$coincidencia<saludo>.chomp;
  say "Destinatario\t= $coincidencia<saludo><a>";
  say "Autor       \t= $coincidencia<final><desde>";
  say "Contenido   \t= $coincidencia<cuerpo>";
}
```

Esto imprimirá lo siguiente:

```
Saludo      = Saludos Tomás,
Destinatario = Tomás
Autor       = Liz
Contenido   = Espero que todo esté bien y que hayas reparado tu carro.
```

Ahora supón que queremos una gramática similar para analizar un mensaje más formal y notamos que podemos reutilizar parte de la gramática Mensaje. Podemos tener nuestra propia gramática hija que hereda de la gramática padre:

```
grammar MensajeFormal is Mensaje {
  rule saludo { [Estimad[a]o]] $<a>=\S+? ', ' }
  rule final  { [Sinceramente|Cordiales saludos] ', ' $<desde>=+.+ }
}
```

El rasgo `is Mensaje` en la cabecera le dice a Raku que `MensajeFormal` debería heredar de la gramática `Mensaje`. Solo dos reglas, `saludo` y `final`, necesitan definirse nuevamente; las otras (la regla `TOP` y el token `línea`) serán heredadas desde la gramática `Mensaje`.

Veamos si funciona:


```

my $mens_formal = "Estimado Tomás,
dentro se encuentra nuestro factura para junio 2016.
Sinceramente, Elizabeth.";
my $coincid2 = MensajeFormal.parse($mens_formal);
if defined $coincid2 {
    say "Saludo \t= ", ~$coincid2<saludo>.chomp;
    say "Destinatario\t= $coincid2<saludo><a>";
    say "Autor \t= $coincid2<final><desde>";
    say "Contenido \t= $coincid2<cuerpo>";
}

```

Esto imprimirá:

```

Saludo          = Estimado Tomás,
Destinatario    = Tomás
Autor           = Elizabeth.
Contenido       = dentro se encuentra nuestro factura para junio 2016.

```

13.7 Objetos de Acciones

Una coincidencia de gramática exitosa te da un árbol de análisis sintáctico de objetos de coincidencia (objetos del tipo Match). Este árbol recapitula todas las "subcoincidencias" individuales que contribuyeron a la coincidencia en su totalidad, así que puede volverse muy larga y complicada. A medida que el árbol de coincidencia se profundiza, y mientras hay más ramas en la gramática, la navegación del árbol de coincidencia para obtener información que te interesa se vuelve más difícil.

Para evitar la necesidad de adentrarse en un árbol de coincidencia, puedes proveer un objeto de acciones. Después de cada coincidencia exitosa de una regla nombrada de tu gramática, el objeto de acciones intenta invocar un método con un nombre similar a la regla de gramática, dándole el objeto de coincidencia recién creado como un argumento de posición. Si tal método no existe, se salta (Los métodos de una acción son algunas veces llamados métodos de reducción). Si existe, el método de acción es usualmente usado para construir un *árbol de sintaxis abstracta* (AST por sus siglas en inglés), es decir, una estructura de datos presumiblemente más simple de explorar y de usar que un árbol de objeto de coincidencia, o puede hacer cualquier otra cosa considerada útil.

En este ejemplo un poco simplista de una calculadora de operaciones aritméticas básicas, las acciones no tratan de construir un AST completo, pero simplemente hacen que el grupo de cálculo funcione entre los varios tókenes que coinciden con la gramática:

```

grammar GramAritmética {
    token TOP { \s* <número> \s* <operación> \s* <número> \s* }
    token operación { <[~*+/-]> }
    token número { \d+ | \d+\.\d+ | \.\d+ }
}

class AccionesAritm {
    method TOP($/) {
        given $<operación> {

```

```

        when '*' { $/.make([*] $<número>)}
        when '+' { $/.make([+] $<número>)}
        when '/' { $/.make($<número>[0] / $<número>[1]) }
        when '-' { $/.make([-] $<número>) }
        when '^' { $/.make($<número>[0] ** $<número>[1]) }
    }
}
}
for ' 6*7 ', '46.2 -4.2', '28+ 14.0 ',
    '70 * .6 ', '126 /3', '6.4807407 ^ 2' -> $op {
    my $coinc = GramAritmética.parse($op, :actions(AccionesAritm));
    say "$coinc\t= ", $coinc.made;
}

```

Esto imprime lo siguiente:

```

6*7          = 42
46.2 -4.2    = 42
28+ 14.0     = 42
70 * .6      = 42
126 /3       = 42
6.4807407 ^ 2 = 42.00000002063649

```

El objetivo de este ejemplo no es describir cómo implementar una calculadora básica (hay maneras mejores de hacer eso, regresaremos a esto más tarde), pero solo mostrar cómo las acciones pueden usarse en conjunción con una gramática.

La gramática es bien simple y busca dos números decimales separados por un operador aritmético infijo. Si hay una coincidencia, `$<número>` (o `$<número>` para ser breve) hará referencia a un array que contiene los dos números (y `$/operación` almacenará el operador aritmético).

El método `parse` se invoca con un argumento nombrado `actions:`, la clase `AccionesAritm`, que le dice a Raku cual objeto de acciones debe utilizar con la gramática. En este ejemplo, actualmente no pasamos un objeto de acciones, pero simplemente el nombre de la clase de acciones (actualmente un tipo de objeto), porque no hay necesidad de instanciar un objeto. En otros casos, por ejemplo si hubiese una necesidad de inicializar o de algún modo usar algunos atributos de un objeto, necesitaríamos pasar un objeto actual que tendría que ser construido con antelación.

Cuando la regla TOP es exitosa, el método TOP de la clase `AccionesAritm` es invocado con el objeto de coincidencia para la regla actual como el argumento. Este método llama al método `make` sobre el objeto de coincidencia, implícitamente puebla el AST, y devuelve el resultado de la operación aritmética entre los dos números. Después, el método `make` en el código de la construcción que hace la llamada (dentro del bucle `for`) devuelve el resultado que fue asignado por `make`.

13.8 Una Gramática para Analizar JSON

JSON (*JavaScript Object Notation*) es un formato estándar-abierto para datos textuales derivado de la notación de objeto en el lenguaje de programación JavaScript. Se ha convertido en uno de los estándares más usado para serializar estructuras de datos, lo cual hace

posible, por ejemplo, el intercambio de las mismas entre plataformas diferentes y lenguajes de programación diferentes, para enviarlas a través de la red, y almacenarlas permanentemente en archivos en los discos.

13.8.1 El Formato JSON

El formato JSON es bien simple y está compuesto de dos tipos de entidades estructurales:

- Objetos o listas desordenadas de pares nombre-valor (básicamente correspondiente a los hashes en Raku);
- Arrays, o una listas ordenadas de valores.

Los valores pueden ser objetos o arrays como se definieron anteriormente, o tipos básicos de datos, tales como: cadenas de texto, números, Booleano (true o false), y *null* (valor vacío o valor indefinido). Una cadena de texto es una secuencia de caracteres Unicode entre comillas, y los números son números decimales con signo que pueden contener una parte fraccional y pueden usar la notación "E" de exponentes.

13.8.2 Nuestra Muestra de JSON

Para ilustrar la descripción del formato más arriba y para el propósito de utilizar nuestras pruebas, usaremos un ejemplo tomado del artículo de Wikipedia sobre JSON (<https://es.wikipedia.org/wiki/JSON>), el cual es una posible descripción de una persona. Esta será nuestra cadena de texto de JSON:

```
my $cadena-JSON = '  
{  
  "nombre": "Juan",  
  "apellido": "Duarte",  
  "estaVivo": true,  
  "edad": 25,  
  "direccion": {  
    "calle": "21 2nd Street",  
    "ciudad": "New York",  
    "estado": "NY",  
    "codigoPostal": "10021-3100"  
  },  
  "numerosTelefono": [  
    {  
      "tipo": "hogar",  
      "numero": "212 555-1234"  
    },  
    {  
      "tipo": "oficina",  
      "numero": "646 555-4567"  
    },  
    {  
      "tipo": "movil",
```

```

        "número": "123 456-7890"
    },
    "infantes": [],
    "esposa": null,
    "cuentaBancaria": {
        "credito": 2342.25
    }
}
';

```

Comparado con el ejemplo de Wikipedia, hemos agregado un objeto `cuentaBancaria` para proveer la posibilidad de probar los números no enteros de JSON.

13.8.3 Creando la Gramática de JSON Paso a Paso

Tomaremos cada una de las entidades de JSON por turno y las manejaremos con reglas.

13.8.3.1 Números

El ejemplo del documento JSON más arriba solo tiene números enteros y decimales, pero debemos de ser capaces de reconocer números como "17", "-138.27", "1.2e-3", ".35", etc. Podemos usar el siguiente token para hacer eso:

```

token número {
    [ \+| |- ]?           # signo (+/-) opcional
    [ \d+ [ \. \d+ ]? ]   # parte entera y parte fraccional opcional
    | [ \. \d+ ]          # o solo parte fraccional
    [ <[eE]> [ \+| |- ]? \d+ ]? # exponente opcional
}

```

13.8.3.2 Cadenas de Texto JSON

Existen muchos patrones para definir una cadena de texto. Para nuestro documento de JSON, la siguiente regla será suficiente:

```

token cadena {
    \" <[ \w \s \- ' ]>+ \"
}

```

Esto coincidirá con una secuencia entre comillas dobles de caracteres alfanuméricos, espacios, guiones, y apóstrofes.

Para un analizador sintáctico real de JSON, una regla que usa una categoría negativa de caracteres que excluye cualquier cosa que no pertenece a una cuerda podría ser mejor, por ejemplo:

```

token cadena {
    \" <-[ \n \" \t ]>+ \"
}

```

es decir, una secuencia entre doble comillas de caracteres excepto comillas dobles, nuevas líneas y tabuladores.

Podrías desear estudiar los estándares de JSON² para averiguar exactamente lo que es aceptado o prohibido en una cadena de texto de JSON. Para nuestros propósitos, la primera regla más arriba será suficiente.

13.8.3.3 Objetos JSON

Los objetos de JSON son listas de pares clave-valor. Las listas son delimitadas por llaves y los pares son separados por comas. Un par clave-valor es una cadena de texto seguida por dos puntos, seguido por un valor (lo definiremos más adelante). Esto puede definirse de la siguiente manera:

```
rule objeto      { '{' <listaPar> '}' }
rule listaPar    { [<par> ',' <par>]* }
rule par         { <cadena> ':' <valor> }
```

Podemos ahora usar una característica de regex que no hemos visto. Esta es el modificador de cuantificador, que podemos usar para simplificar la regla `listaPar`. Para coincidir con cosas como valores separados por comas más fácilmente, puedes agregar un modificador `%` a cualquiera de los cuantificadores regulares para especificar un separador que debe ocurrir entre cada una de las coincidencias. Así que, por ejemplo, `/a+ % ', ' /` coincidirá con `"a"` o `"a,a"`, o `"a,a,a"`, etc.

Por lo tanto, la regla `listaPar` puede escribirse de la siguiente manera:

```
rule listaPar    { <par> + % \, }
```

o:

```
rule listaPar    { <par> * % \, }
```

si aceptamos que una `listaPar` puede estar también vacía.

13.8.3.4 Arrays JSON

Los arrays son listas de valores separados por comas entre corchetes:

```
rule array       { '[' <listaValor> ']' }
rule listaValor  { <valor> * % \, }
```

Aquí, también usamos el modificador de cuantificador que mostramos más arriba.

13.8.3.5 Valores JSON

Los valores son objetos, arrays, cadenas de texto, números, booleanos (verdadero (`true`) o falso (`false`)), o `null`:

```
token valor { | <objeto> | <array> | <cadena> | <número>
              | true      | false   | null
            }
```

²Dado que JSON no está completamente estandarizado, no proveeré un enlace específico; búscalo y decide.

13.8.4 La Gramática de JSON

Hemos definido todos los elementos de la gramática de JSON; solo necesitamos declarar una gramática y agregar una regla TOP para completarla:

```
grammar Gramática-JSON {
  token TOP      { \s* [ <objeto> | <array> ] \s* }
  rule objeto    { '{' \s* <listaPar> '}' \s* }
  rule listaPar   { <par> * % \, }
  rule par       { <cadena> ':' <valor> }
  rule array     { '[' <listaValor> ']' }
  rule listaValor { <valor> * % \, }
  token cadena   { \" <[ \w \s \- ' ]>+ \" }
  token número   {
    [\+|\-]?
    [ \d+ [ \. \d+ ]? ] | [ \. \d+ ]
    [ <[eE]> [\+|\-]? \d+ ]?
  }
  token valor    {
    | <objeto> | <array> | <cadena> | <número>
    | true     | false   | null
  }
}
```

Podemos ahora probar la gramática con nuestra muestra de cadena de texto de JSON y tratar de imprimir el objeto de coincidencia:

```
my $coinc = Gramática-JSON.parse($cadena-JSON);
say ~$coinc if $coinc;
```

Esto produce la siguiente salida:

```
{
  "nombre": "Juan",
  "apellido": "Duarte",
  "estaVivo": true,
  "edad": 25,
  "direccion": {
    "calle": "21 2nd Street",
    "ciudad": "New York",
    "estado": "NY",
    "codigoPostal": "10021-3100"
  },
  "numerosTelefono": [
    {
      "tipo": "hogar",
      "número": "212 555-1234"
    },
    {
      "tipo": "oficina",
      "número": "646 555-4567"
    }
  ]
}
```

```

    },
    {
      "tipo": "movil",
      "número": "123 456-7890"
    }
  ],
  "infantes": [],
  "esposa": null,
  "cuentaBancaria": {
    "credito": 2342.25
  }
}

```

La muestra del documento JSON ha coincidido completamente. Esta gramática de JSON funciona perfectamente, y toma menos de 20 líneas de código. Si piensas, esto es realmente poderoso. Pruébalo por ti mismo. Trata de cambiar la gramática en varios lugares para ver si aún funciona. También podrías tratar de introducir errores en el documento JSON (por ejemplo la remoción de una coma entre dos valores de una lista) y la coincidencia no ocurrirá (o, por lo menos, no debería ser la misma).

Puedes objetar que esta gramática cubre solo un subconjunto de JSON. Hay algo verdad en eso, pero no realmente: esta gramática está casi completa. Aún así, yo no recomendaría el uso de esta gramática en un entorno de producción para analizar documentos JSON, ya que fue construida con propósitos pedagógicos y puede no cumplir con cada detalle final de los estándares de JSON.

Mira la gramática del módulo `JSON::Tiny` (<https://github.com/moritz/json>) de Raku, el cual puede analizar cualquier documento de JSON válido. No es más complicado que lo que hemos mostrado aquí (excepto por el uso de regexes proto, un tema que no discutido aquí), y no es más extenso, dado que solo contiene 35 líneas de código.

13.8.5 Agregando Acciones

La gramática de JSON funciona bien, pero la impresión del árbol de los objetos de análisis solo para un documento de JSON pequeño mostrará alrededor de 300 líneas de texto, dado que provee todos los detalles de lo que coincidió, regla por regla y subpatrón por subpatrón. Esto es muy útil porque te ayuda a entender lo que la gramática hace (especialmente cuando no funciona como se esperaba), pero explorar dicho árbol para extraer los datos puede ser tedioso. Puedes usar *acciones* para poblar un árbol de estructura (usualmente llamado un árbol de sintaxis abstracta) más simple que contiene solo la información que necesitas.

Agreguemos una clase de acciones para construir un árbol de sintaxis abstracta (AST):

```

class acciones-JSON {
  method TOP($/) {
    make $/.values.[0].made;
  };
  method objeto($/) {
    make $<listaPar>.made.hash.item;
  }
}

```

```

method listaPar($/) {
  make $<par>».made.flat;
}
method par($/) {
  make $<cadena>.made => $<valor>.made;
}
method array($/) {
  make $<listaValor>.made.item;
}
method listaValor($/) {
  make [$<valor>.map(*.made)];
}
method cadena($/) { make ~$0 }
method número($/) { make +$/.Str; }
method valor($/) {
  given ~$/ {
    when "true" {make Bool::True;}
    when "false" {make Bool::False;}
    when "null" {make Any;}
    default { make $<val>.made;}
  }
}
}

```

Para que esta clase de acciones funcione, necesitamos hacer un pequeño cambio a la gramática. El método `valor` usa una captura nombrada `val` para acceder su contenido; necesitamos agregar las capturas nombradas relevante al token `valor` en la gramática:

```

token valor { <val=objeto> | <val=array> | <val=cadena>
              | <val=número> | true | false | null
}

```

También necesitamos añadir paréntesis de captura al token `cadena` para que el método `cadena` sea capaz de extraer la cadena de texto correspondiente:

```

token cadena { \" ( <[ \\w \\s \\- ' ]>+ ) \" }

```

Ahora podemos llamar nuestra gramática con la siguiente sintaxis:

```

my $j-acciones = acciones-JSON.new();
my $coinc = Gramática-JSON.parse($cadena-JSON, :actions($j-acciones));
say $coinc.made;

```

Observa que, aquí, hemos usado un objeto de acciones en lugar de simplemente una clase de acciones, pero esto es solo con el propósito de mostrar cómo hacerlo; podríamos haber usado la clase directamente como antes.

La última sentencia en el código más arriba imprime el AST. Hemos reformateado la salida para mostrar mejor la estructura del AST:

```

{
  cuentaBancaria => {

```



```

        credito => 2342.25
    },
    direccion => {
        ciudad => New York,
        codigoPostal => 10021-3100,
        estado => NY,
        calle => 21 2nd Street
    },
    edad => 25,
    infantes => [],
    nombre => Juan,
    estaVivo => True,
    apellido => Duarte,
    numerosTelefono => [
        {número => 212 555-1234, tipo => hogar}
        {número => 646 555-4567, tipo => oficina}
        {número => 123 456-7890, tipo => movil}
    ],
    esposa => (Any)
}

```

En este caso, la estructura superior es un hash (también podría ser una array con una diferente cadena de texto de JSON). Podemos ahora explorar este hash para encontrar los datos que son de interés para nosotros. Por ejemplo:

```

say "Las claves son: \n", $coinc.made.keys;
say "\nAlgunos Valores:";
say $coinc.made{$_} for <nombre apellido estaVivo>;
say $coinc.made<direccion><ciudad>;
say "\nNúmeros telefónicos:";
say $coinc.made<numerosTelefono>[$_]<tipo número>
    for 0..$coinc.made<numerosTelefono>.end;

```

Esto mostrará la siguiente salida:

```

Las claves son:
(apellido cuentaBancaria numerosTelefono infantes direccion edad nombre esposa estaVivo)

Algunos Valores:
Juan
Duarte
True
New York

Números telefónicos:
(hogar 212 555-1234)
(oficina 646 555-4567)
(movil 123 456-7890)

```

13.9 Herencia y las Gramáticas Mutables

La capacidad de una gramática de heredar de otra abre la puerta para muchas posibilidades en términos de la extensión del lenguaje Raku. Es posible, en el contexto de un módulo o una framework, crear una “subclase” de la gramática estándar de Raku, por ejemplo, para escribir una nueva gramática hija que hereda de la gramática estándar de Raku, pero agrega una nueva característica, sobrecarga un operador, o modifica algún otro elemento sintáctico, y ejecuta este programa con el mismo compilador de Raku, pero con una gramática modificada localmente.

Esto significa que es actualmente posible extender el lenguaje dinámicamente para necesidades nuevas, usualmente sin cambiar el compilador o la máquina virtual. No obstante, estos son temas avanzados que están dedicados más a los gurús del lenguaje que a los principiantes. Así que solo mencionamos estas posibilidades grandiosas con la esperanza de estimular tu apetito y empujarte a que los estudies más detalladamente, pero no hablaremos en más detalle sobre ellos en este libro.

13.10 Depuración de Programas

La escritura de gramáticas es muy divertida, pero también difícil o hasta tediosa cuando comienzas a hacerlo.

Cuando comenzaste a practicar la programación con este libro, probablemente cometiste muchos pequeños errores que inicialmente previnieron que tus programas se compilaran y se ejecutaran, o hicieran lo que querías. Con la práctica, sin embargo, con suerte cometiste menos errores gradualmente y gastaste menos tiempo persiguiendo errores.

Cuando comienzas a aprender sobre las gramáticas (y en menor medida los regexes), puedes sentirte que estás nuevamente en el punto de partida. Aún los programadores muy buenos usualmente cometen errores absurdos cuando comienzan a escribir gramáticas. Es un diferente paradigma de programación, y requiere una nueva fase de aprendizaje.

En este caso, pequeño es hermoso. Inicia con regexes pequeños y reglas pequeñas, y con una pequeña entrada de prueba. Prueba los regexes o las reglas individuales en el REPL, y agrégalos a tu código solo cuando estás confiado de que hacen lo que deseas.

Escribe casos de prueba al mismo tiempo que escribes tu código (o actualmente hasta antes de escribir tu código), y asegúrate de que pasen todas las pruebas relevantes antes de seguir adelante. Y agrega nuevas pruebas al agregar reglas nuevas.

Una técnica de depuración estándar es agregar sentencias de impresión al código para inquirir información sobre el estado del programa (tal como el valor de variables, el flujo de ejecución del programa, etc.). También puedes hacer eso con los regexes y las gramáticas.

Tomemos el ejemplo de la gramática simple para coincidir con las fechas de la Sección 13.5 y supongamos que escribiste esta gramática:

```
grammar Mi-Fecha {
  token TOP { \s* <año> '-' <mes> '-' <día> }
  token año  { \d ** 4 }
  token mes  { 1 <[0..2]> || 0 <[1..9]> }
  token día  { (\d ** 2) <?{1 <= $0 <= 31 }> }
```

```

}
my $cadena = " 2016-07-31 ";
say so Mi-Fecha.parse($cadena);           # -> False

```

Esta prueba falla.

A esta altura, ya se ha vuelto un poco difícil de inquirir por qué la gramática falla (a menos que hayamos probado cada uno de los tres tokens antes de construir la gramática, pero en aras de esta discusión asumamos que no lo hemos hecho). Intentemos no cambiar aleatoriamente cosas aquí o allá y observar si funciona; podríamos invertir horas haciendo eso y probablemente no llegaríamos a ninguna parte. Seamos más metódicos.

Primero probemos los tokens básicos, año, mes, y día. Hemos visto anteriormente que el método `parse` busca por defecto la regla `TOP` en la gramática, pero puedes especificar otra regla si así lo deseas, y eso es lo que necesitamos aquí. Podemos probar estos tokens individualmente:

```

say so Mi-Fecha.parse("2016", :rule<año>);    # -> True
say so Mi-Fecha.parse("07",   :rule<mes>);    # -> True
say so Mi-Fecha.parse("31",   :rule<día>);    # -> True

```

Estos tres tokens parecen funcionar apropiadamente. En este punto, podrías adivinar dónde se encuentra el problema, pero asumamos que no sabes.

Necesitamos depurar el token “TOP”. Podemos usar el método común de depuración de imprimir donde estamos a varias etapas del programa. Puedes insertar una sentencia de impresión en una regla nombrada. Intentemos cambiar el token `TOP` a esto:

```

token TOP { \s* <año> { say "año coincido"; }
            '-' <mes> { say "mes coincido"; }
            '-' <día> { say "día coincido"; }
}

```

Esto muestra la siguiente salida:

```

año coincido
mes coincido
día coincido

```

Incluso el token “TOP” parece funcionar hasta el final. En este punto, deberíamos ser capaz de deducir que nos falta el espacio final en el token “TOP”.

Así que podríamos agregar un espacio adicional al final del token:

```

token TOP { \s* <año> '-' <mes> '-' <día> \s* }

```

o cambiar la regla:

```

rule TOP { \s* <año> '-' <mes> '-' <día> }

```

o era posible probar la cadena de texto que era incorrecta (porque no podría tener espacios) y por lo tanto, necesitaba ser arreglada.

Si tiene una clase de acciones, también puedes agregar sentencias de impresión a los métodos de acciones.

Igualmente recuerda que el depurador de Raku (ver la Sección 12.14) puede ser muy útil. Brevemente mostramos en la Subsección 12.14.6 (p. 262) cómo recorrer una coincidencia de regex paso a paso. La mayor parte de lo que se describió ahí también aplica a la depuración de gramáticas.

Finalmente, vale la pena mencionar que existe un módulo grandioso, `Grammar::Tracer`, para la depuración de regexes y gramáticas (<https://github.com/jnthn/grammar-debugger/>), que funciona con Rakudo. Si agregas:

```
use Grammar::Tracer;
```

a tu programa, entonces cualquier gramática dentro del ámbito léxico imprimirá información de depuración sobre las reglas con la cual intentó coincidir, aquellas que fueron exitosas y aquellas que fallaron.

Puedes también usar lo siguiente:

```
use Grammar::Debugger;
```

para hacer lo mismo paso a paso. Solo escribe “h” en la prompt para la lista de comandos disponibles.

13.11 Glosario

Gramática Una herramienta de alto nivel para realizar análisis léxico y gramático de un texto estructurado. En Raku, una gramática es específicamente un espacio de nombres que contiene una colección de reglas nombradas destinada a este tipo de análisis.

Análisis semántico El proceso en el cual se realiza un análisis léxico de texto fuente, y especialmente dividirlo en “palabras” o tokens.

Análisis sintáctico El proceso en el cual se realiza un análisis gramático de un texto fuente, y especialmente se ensamblan palabras o tokens en expresiones y sentencias que tienen sentido semántico.

Programación declarativa Un modelo de programación donde especificas definiciones, reglas, propiedades, y restricciones, en lugar de sentencias e instrucciones, y dejas al programa derivar conocimiento nuevo de estas definiciones y reglas. Los regexes y las gramáticas son ejemplos de la programación declarativa.

Objeto de coincidencia En Raku, un objeto (de tipo `Match`), denotado usualmente por `$/`, el cual contiene información (algunas veces muy) detalla sobre lo que coincidió exitosamente con un regex o una gramática. El objeto de coincidencia `$/` se le asignará `Nil` si la coincidencia falla.

Captura El hecho por el cual partes de una cadena de texto que coinciden con un regex (o una gramática) pueden extraerse a través del uso de un número de variables especiales.

Regla En términos generales, las reglas nombradas son regexes que usan una sintaxis de método y usualmente se almacenan en una gramática. Más específicamente, una categoría de estas reglas nombradas (en conjunto con los regexes nombrados y los tókenes).

Árbol de sintaxis abstracta (AST) Una estructura de datos que usualmente resume el objeto de coincidencia y se usa para la explotación de datos útiles. El objeto de coincidencia es poblado automáticamente por Raku, donde el AST contiene información considerada útil y explícitamente insertada por el programador.

Clases de acciones Una clase usada en conjunción con una gramática para realizar ciertas acciones cuando una regla de una gramática coincide con algo en los datos de entrada. Si un método con el mismo nombre de una regla en la gramática existe en la clase de acciones, entonces será invocado siempre que la regla coincide.

13.12 Ejercicio: Una Gramática para una Calculadora Aritmética

La calculadora aritmética presentada en la Sección 13.7 más arriba es muy simple. En particular, puede analizar sintácticamente expresiones aritméticas compuestas de dos operandos separados por un operador infijo, pero no mucho más.

Nos gustaría ser capaces de analizar expresiones aritméticas complicadas. La calculadora debería también ser capaz de manejar:

- Expresiones con varios operadores diferentes (entre los cuatro operadores aritméticos básicos) y operandos múltiples
- Reglas estándares de precedencia entre los operadores (por ejemplo, las multiplicaciones deben hacerse antes que las adiciones)
- Paréntesis anulan las reglas usuales de precedencia

Estos son algunos ejemplos de expresiones que la calculadora debería analizar sintácticamente y calcular correctamente:

```
3 + 4 + 5;  
3 + 4 * 5;  # resultado debería ser 23  
(3 + 4) * 5; # resultado debería ser 35
```

Ejercicio 13.2. *Tu misión, [Juan|Salomé], así elijas aceptarla, es escribir tal gramática. Como es usual, si fallas, el Gobierno debería negar cualquier conocimiento de tu clase de acciones.*

Hay varias maneras posibles de lograr esto; la solución presentada en Solución: A.10.2 es solo una de ellas.

Programación Funcional en Raku

La programación funcional es un paradigma de programación que trata las computaciones como las evaluaciones de funciones matemáticas y evita el cambio de estado y datos mutables. Es un paradigma de programación declarativo, lo cual significa que la programación se lleva a cabo con expresiones o declaraciones en lugar de sentencias. En el código funcional, el valor de la salida de una función depende solamente de los argumentos que son la entrada de la función, así que llamar una función dos veces con el mismo argumento producirá el mismo resultado cada vez. La eliminación de los efectos secundarios, es decir, cambios en el estado que no dependen en las entradas de la función, hacen que entender y predecir el comportamiento de un programa sea más fácil, lo cual es una de las motivaciones principales para el desarrollo de la programación funcional.

Raku no es un lenguaje funcional dado que también usa otros paradigmas de programación que hemos visto en abundancia en este libro. Sin embargo, Raku ofrece características y capacidades extensivas de la programación funcional, algunas de las cuales han sido introducidas en varias secciones de este libro y que revisaremos brevemente antes de entrar en más detalles.

14.1 Funciones de Orden Superior

Desde el Capítulo 3 sobre funciones y subrutinas, en la Sección 3.14 (p. 46), hemos observado que las funciones, las subrutinas, y otros objetos de código son *objetos de primera clase* o *ciudadanos de primera clase* en Raku, lo cual significa que pueden pasarse como valores. Una función en Raku es un valor que puedes asignar a una variable o pasar como un argumento a otra función o un valor de retorno desde otra función.

14.1.1 Breve Actualización: Funciones como Objetos de Primera Clase

Nuestro simple ejemplo inicial de una función de orden superior fue algo así:

```

sub do-twice( $código ) {
    $código();
    $código();
}
sub saludar {
    say ";Hola Mundo!";
}
do-twice &saludar;

```

en el cual la subrutina `saludar` se pasa como un argumento a la subrutina `do-twice`, con el efecto de imprimir el saludo dos veces. Una función que se pasa como un argumento a otra función es llamada usualmente una *función de retrollamada* (del inglés *callback function*).

El sigilo `&` colocado antes del nombre de la subrutina `saludar` en la lista de argumentos (como también antes del parámetro `code` en la signatura y en el cuerpo de la subrutina `do-twice`) le dice a Raku que estás pasando una subrutina u otro objeto código que puede ser llamado.

En la ciencia de la computación, una subrutina que puede tomar otra subrutina como un argumento es algunas veces llamada una *función de orden superior*.

Más ejemplos interesantes de funciones de orden superior pueden encontrarse con las funciones `reduce`, `map`, y `grep` que estudiamos en la Sección 9.8 (p. 161), al igual que la función `sort` (Sección 9.11 y Sección 9.12).

Consideremos por ejemplo la tarea de ordenar registros por fechas, los cuales consisten de un identificador seguido por una fecha en el formato DD-MM-YYYY, tal como “id1;13-05-2015” o “id2;17-04-2015”. Los registros necesitan un poco de transformación antes que podamos compararlos con el propósito de encontrar el orden cronológico en el cual deberían ser ordenados, así que podríamos tener una función separada para la comparación:

```

sub comparar( $reg1, $reg2 ) {
    my $cmp1 = join " ", reverse split /<[; -]>/, $reg1;
    my $cmp2 = join " ", reverse split /<[; -]>/, $reg2;
    return $cmp1 cmp $cmp2;
}

```

Cada registro modificado se construye al encadenar tres funciones. Estas líneas deberían leerse de derecha a izquierda: primero, el valor de entrada se separa en cuatro artículos; estos artículos son después invertidos y consecuentemente unidos. Así que el resultado para “id1;13-05-2015” es “2015,05,13,id1”, el cual es adaptado para una comparación con el operador `cmp`. Más tarde regresaremos a esta forma de programación de tubería y otras maneras de realizar estas operaciones.

Ahora podemos pasar la subrutina `comparar` a la función `sort`:

```

.say for sort &comparar, <id1;13-05-2015 id2;17-04-2015
                        id3;21-02-2016 id4;12-01-2015>;

```

Esto muestra:

```
id4;12-01-2015
id2;17-04-2015
id1;13-05-2015
id3;21-02-2016
```

Por favor nota que esto se provee como un ejemplo de funciones de retrollamada usadas con la función integrada `sort`. Veremos al final de la siguiente subsección una manera más simple de lograr el mismo tipo de ordenamiento usando una función anónima.

14.1.2 Subrutinas Anónimas y Lambdas

Hemos visto que una subrutina no necesita tener un nombre y que puede ser *anónima*. Por ejemplo, puede almacenarse en una variable escalar directamente:

```
my $saludar = sub {
    say "Hola Mundo!";
};
do-twice $saludar;                # imprime "Hola Mundo!" dos veces
```

Ni siquiera necesitamos almacenar el código de la función anónima en la variable `$saludar`; de hecho, podemos pasar dicho código directamente como un argumento a la subrutina `do-twice`:

```
do-twice( sub {say "Hola Mundo!"} );
```

Dado que nuestra subrutina anónima no toma ningún argumento y tampoco devuelve un valor útil, podemos simplificar la sintaxis aún más y pasar un bloque de simple código anónimo a `do-twice`:

```
do-twice {say "Hola Mundo!"};    # imprime "Hola Mundo!" dos veces
```

Ya has visto varios ejemplos útiles de subrutinas anónimas en este libro (ver Sección 9.8 para más detalles):

- Con la función `reduce` para computar la suma de los primeros 20 números enteros:

```
my $suma = reduce { $_a + $_b }, 1..20; # -> 210
```

- Con la función `map` para convertir la primera letra de una lista de ciudades en mayúscula (usando la función integrada `tc`):

```
> .say for map { .tc }, <londres parís roma washington madrid>;
Londres
París
Roma
Washington
Madrid
```

- Con la función `grep` para generar una lista de números pares al filtrar los números impares:

```
my @pares = grep { $_ % 2 }, 1..17; # -> [2 4 6 8 10 12 14 16]
```


El ejemplo con `reduce` es interesante. En principio, contrario a una subrutina, no puedes pasar argumentos a un bloque de código tan fácilmente (porque no tiene una signatura). Pero el uso de los parámetros auto-declarados de posición (o parámetros marcadores) con el twigil `$~` hace posible usar los parámetros dentro del bloque.

Debido a esta posibilidad, el bloque de código anónimo se convierte en lo que usualmente se conoce como una *lambda* en la ciencia de la computación (y en las matemáticas), es decir, un tipo de función sin nombre. El cálculo lambda, una teoría matemática inventada en la década del 1930 por Alonzo Church, es la esencia de la mayoría de los lenguajes de programación funcional de hoy en día.

Actualmente, los dos otros ejemplos más arriba que usan la variable tópica `$_` son también lambdas. Aunque no lo mencionamos en aquellas ocasiones, otras construcciones que vimos anteriormente son también lambdas. En particular, considera la sintaxis del “bloque puntiagudo” usada en el siguiente bucle `for` que muestra la tabla de multiplicación:

```
for 1..9 -> $mult {
    say "Tabla de multiplicación del $mult";
    for 1..9 -> $val {
        say "$mult * $val = ", $mult * $val;
    }
}
```

Esta es otra forma de lambda donde el parámetro de la “función” es definido por la variable de bucle de un bloque puntiagudo.

El ejemplo de ordenamiento presentado en la Subsección 14.1.1 más arriba puede también escribirse con un bloque de código anónimo (tomando así ventaja de la sintaxis de `sort` y usando un bloque de código con un solo argumento descrito en la Sección 9.12):

```
my @in = <id1;13-05-2015 id2;17-04-2015 id3;21-02-2016>;
.say for sort { join ",", reverse split /<[;-]>/, $_ }, @in;
```

Aquí, el bloque de código un poco extenso pasado como un argumento a la función `sort` es una lambda.

14.1.3 Clausuras

En la programación de computadoras, una *clausura* (o *clausura léxica*) es una función que puede acceder el contenido de las variables que están lexicalmente disponibles donde la función es definida, aún si aquellas variables ya no se encuentran en el ámbito en la sección de código donde la función es llamada.

Considera el siguiente ejemplo:

```
sub crear-contador( Int $cuenta ) {
    my $contador = $cuenta;
    sub incrementar-cuenta {
        return $contador++
    }
    return &incrementar-cuenta;
}
```

```

}
my &contar-desde-cinco = crear-contador(5);
say &contar-desde-cinco() for 1..6; # imprime los números del 5 al 10

```

La subrutina `crear-contador` inicializa la variable `$contador` al valor del parámetro recibido, define la subrutina `incrementar-cuenta`, y devuelve esta subrutina. El código principal llama a `crear-contador` para dinámicamente crear la referencia de código `&contar-desde-cinco` (y podría llamarla muchas veces para crear otros contadores que cuenten desde 6, 7, etc.). Después, `&contar-desde-cinco` es llamada seis veces e imprime los números entre 5 y 10, cada uno en su propia línea.

Lo mágico de esto es que la variable `$contador` está fuera de ámbito cuando la función `&contar-desde-cinco` es llamada, pero `&contar-desde-cinco` puede aún acceder a su valor, devolverlo e incrementarlo porque `$contador` estaba dentro del ámbito léxico al tiempo que `incrementar-cuenta` fue definida. Se dice entonces que `incrementar-cuenta` contiene a la variable `$contador`. Por lo tanto, la subrutina `incrementar-cuenta` es una clausura.

El ejemplo anterior es un poco artificial y su sintaxis algo rara porque yo quería mostrar un ejemplo de una clausura nombrada (`incrementar-cuenta` es una subrutina nombrada). Es usualmente más simple e idiomático usar clausuras anónimas y escribir nuevamente el ejemplo de la siguiente manera:

```

sub crear-contador( Int $cuenta ) {
  my $contador = $cuenta;
  return sub {
    return $contador++
  }
}
my &contar-desde-cinco = crear-contador(5);
say &contar-desde-cinco() for 1..6; # imprime números del 5 al 10

```

Podríamos simplificar `crear-contador` aún más con el uso de sentencias `return` implícitas:

```

sub crear-contador( Int $cuenta ) {
  my $contador = $cuenta;
  sub { $contador++ }
}

```

pero esto es menos claro porque la intención del código es menos explícita.

El último ejemplo `crear-fifo` en la solución al ejercicio sobre la cola FIFO (Subsección A.7.1.6) es otro ejemplo que utiliza el mismo mecanismo:

```

sub crear-fifo {
  my @cola;
  return (
    sub {return shift @cola;},
    sub ($elem) {push @cola, $elem;}
  );
}
my ($fifo-consigue, $fifo-pone) = crear-fifo();
$fifo-pone($_) for 1..10;
print " ", $fifo-consigue() for 1..5; # -> 1 2 3 4 5

```

En Raku, todas las subrutinas son clausuras, lo cual significa que todas las subrutinas tienen acceso a las variables lexicales que existían en el entorno al momento de la definición de las subrutinas. Sin embargo, ellas no actúan necesariamente como clausuras.

De hecho, todos los objetos de código, incluyendo simples bloques de código anónimo, pueden actuar como clausuras, lo cual significa que pueden hacer referencia a variables lexicales desde el ámbito externo, y en efecto esto es lo que pasa con la variable de bucle de un bloque puntiagudo o en el siguiente bloque map:

```
my $multiplicador = 7;
say map { $multiplicador * $_ }, 3..6; # -> (21 28 35 42)
```

En este ejemplo, el bloque pasado a map hace referencia a la variable \$multiplicador desde el ámbito externo, convirtiendo al bloque en una clausura.

Los lenguajes sin clausuras no pueden proveer fácilmente funciones de orden superior que sean tan poderosas y fáciles de usar como map.

Aquí presentamos otro ejemplo de un bloque que actúa como una clausura para la implementación de un contador:

```
my &cuenta;
{
    my $contador = 10;
    &cuenta = { say $contador++ };
}
&cuenta() for 1..5;
```

Esta clausura guarda una referencia a la variable \$contador cuando la clausura es creada. La llamada al bloque de código &cuenta muestra y actualiza a \$contador exitosamente. Esto sucede aún cuando la variable ya no se encuentra en el ámbito léxico al momento que el bloque es ejecutado.

14.2 Procesamiento de Listas y Programación de Tuberías

A menudo, una computación puede expresarse como una serie de transformaciones de una lista de valores. Raku provee funciones capaces de trabajar con los artículos de una lista y aplicar simples acciones, funciones de retrollamada, o bloques de código a estos artículos. Ya hemos visto y usado abundantemente varias funciones de este tipo:

- map aplica una transformación a cada artículo de una lista.
- grep es un filtro que mantiene aquellos elementos para los cuales la función o el bloque de código asociado con grep evalúa a verdadero.
- reduce usa cada artículo de una lista para calcular un único valor escalar.
- sort ordena los elementos de una lista de acuerdo a reglas definidas en el bloque de código o la subrutina que se pasa.

Hemos discutido varios ejemplos donde estas funciones pueden usarse conjuntamente en un tipo de tubería de datos en la cual los datos producidos en cada paso de la tubería son suministrados al siguiente paso. Por ejemplo, anteriormente en este capítulo (Subsección 14.1.1), usamos esto:

Como destacamos anteriormente, este tipo de código debería leerse de derecha a izquierda (y de abajo hacia arriba si está escrito en varias líneas de código): \$reg1 es suministrada a split, la cual divide el dato en cuatro piezas; las piezas son después invertidas (con reverse) y suministradas a join para crear un solo dato donde las piezas están ahora en orden inverso.

```
my @mascotas-de-mujeres-solteras-de-kansas =
  map { .mascotas },
  grep { !.está-casada },
  grep { .género eq "Femenino" },
  grep { .estado eq "Kansas" },
  @ciudadanos;
```

Estas tuberías son muy poderosas y expresivas, y pueden hacer muchas cosas en pocas líneas de código.

Por ejemplo, al reutilizar el ejemplo sobre el ordenamiento de los registros por fechas que vimos al inicio de este capítulo, podrías escribirlo nuevamente así:

Por cierto, si estás usando tales operaciones de tubería con una entrada bien grande, dependiendo de la arquitectura de tu plataforma, Raku puede ser capaz de ejecutar estas múltiples operaciones en paralelo en diferentes CPUs o núcleos (cores), mejorando significativamente el rendimiento del proceso en su totalidad.

Existe también un operador backward feed (*suministro inverso*), `<==`, que posibilita la escritura de la tubería en orden inverso:

```
my $out <== join(",")
        <== reverse()
        <== split(/<[-]>/)
        <== "id1;13-05-2015";
```

14.2.2 El Metaoperador de Reducción

Ya conocimos este metaoperador en la Sección 9.8. Un metaoperador actúa sobre otros operadores. Dada una lista y un operador, el **operador de reducción** `[...]` aplica el operador iterativamente a todos los valores de la lista para producir un valor único.

Por ejemplo, lo siguiente imprime la suma de todos los elementos de una lista o un rango:

```
say [+] 1..10;      # -> 55
```

Similarmente, podemos escribir una función factorial así:

```
sub fact( Int $n where $n >= 0 ) {
    return [*] 1..$n;
}
say fact 20;      # -> 2432902008176640000
say fact 0;       # -> 1
```

(Nota que esto produce el resultado correcto hasta para el caso del factorial de 1, el cual es definido matemáticamente como 1.)

14.2.3 El Hiperoperador

Un hiperoperador aplica el operador especificado a cada artículo de una lista (o dos listas en paralelo) y devuelve una lista modificada (similar a la función `map`). Dicho hiperoperador usa las comillas francesas o alemanas, `<< >>` (Unicode codepoints U+00AB y U+00BB), pero puedes usar las comillas angulares, `<< >>`, si así lo deseas (o si no sabes como entrar estos caracteres Unicode con tu editor).

Nuestro primer ejemplo multiplicará cada elemento de una lista por un número dado (5):

```
my @b = 6..10;
my @c = 5 <<*>> @b;
say @c;      # imprime 30 35 ... 50
              # resultado (5*6, 5*7, ...)
```

Podemos también combinar dos listas y, por ejemplo, agregar los valores respectivos:

```
my @a = 1..5;
my @b = 6..10;
my @d = @a >>+<< @b;
say @d;      # -> [7 9 11 13 15]
```

Puedes también usar hiperoperadores con un operador unario:

```
my @a = 2, 4, 6;
say -<< @a;           # imprime: -2 -4 -6
```

Los hiperoperadores con operadores unarios siempre devuelven una lista con el mismo tamaño que la lista de entrada. Los hiperoperadores infijos tienen un comportamiento diferente dependiendo del tamaño de sus operandos:

```
@a >>+<< @b;  # @a y @b deben tener el mismo tamaño
@a <<+<< @b;  # @a puede ser más pequeño
@a >>+>> @b;  # @b puede ser más pequeño
@a <<+>> @b;  # Cualquiera puede ser más pequeño,
               # Raku probablemente hará lo que quieres
               # (principio DWIM)
```

Los hiperoperadores también funcionan con operadores modificadores de asignación:

```
@x >>+=<< @y  # Lo mismo que: @x = @x >>+<< @y
```

14.2.4 Los Operadores Cruz (X) y Zip (Z)

El operador cruz usa la letra mayúscula X. Dicho operador toma dos o más listas como argumentos y devuelve una lista de todas las listas que pueden construirse al combinar los elementos de cada lista (una forma de “producto cartesiano”):

```
my @a = 1, 2;
my @b = 3, 4;
my @c = @a X @b;      # -> [(1,3), (1,4), (2,3), (2,4)]
```

El operador cruz puede también usarse como un metaoperador y aplicar el operador modificador a cada combinación de artículos derivada de sus operandos:

```
my @a = 3, 4;
my @b = 6, 8;
say @a X* @b;         # -> 18 24 24 32
```

Si no se provee un operador adicional (como en el primer ejemplo), X actúa como si la coma fuera proveída como el operador adicional por defecto. Así que

```
say ('a', 'b') X (1, 2);  # -> ((a 1) (a 2) (b 1) (b 2))
say ('a', 'b') X, (1, 2); # -> ((a 1) (a 2) (b 1) (b 2))
```

son equivalentes.

El operador zip Z intercala las listas como una cremallera:

```
say 1, 2 Z <a b c> Z 9, 8;  # -> ((1 a 9) (2 b 8))
```

El operador Z también existe como un metaoperador, y en este caso, en lugar de producir listas anidadas internas como en el ejemplo más arriba, el operador zip aplicará el operador adicional y reemplazará estas listas anidadas con los valores generados. En el siguiente ejemplo, el operador de concatenación es usado para fusionar las listas internas producidas por el operador zip y crear una cadenas de texto:

```
say 1, 2, 3 Z~ <a b c> Z~ 9, 8, 7; # -> (1a9 2b8 3c7)
```

14.2.5 Un Resumen de Los Operadores de Listas

Los operadores de listas anteriores son poderosos y pueden combinarse para producir construcciones increíblemente expresivas.

Como un ejercicio, intenta resolver las siguientes pequeñas pruebas (por favor no prosigas con la lectura hasta que lo haya intentado):

- Dado que la función integrada `lcm` devuelve el mínimo común múltiplo entre dos números, escribe un programa que muestre el número positivo más pequeño divisible por todos los números entre 1 y 20.
- Escribe un programa que calcule la suma de todos los dígitos del factorial de 100.
- Encuentra la diferencia entre el cuadrado de la suma de los primeros 100 números enteros y la suma de los cuadrados de los primeros 100 números enteros.

Nuevamente, no prosigas con la lectura hasta que hayas tratado de resolver estos pequeños problemas (y con suerte lo hayas logrado).

El operador de reducción facilita la aplicación de un operador a todos los elementos de una lista. Así que si lo usamos con la función `lcm` nos dará el mínimo común múltiplo (MCM) entre 1 y 20:

```
say [lcm] 1..20; # -> 232792560
```

Para la suma de los dígitos del factorial de 100, usamos el metaoperador de reducción `[]` dos veces, una con el operador de multiplicación para calcular el factorial de 100, y otra con el operador de adición para añadir los dígitos del resultado:

```
say [+] split '', [*] 2..100; # -> 648
```

Para el cuadrado de la suma menos la suma de los cuadrados, es fácil calcular la suma de los 100 primeros números enteros con el operador de reducción. El hiperoperador `<<..>>` fácilmente suministra una lista de los cuadrados de estos enteros, y otra aplicación del operador de reducción reduce esta lista a una suma:

```
say ([+] 1..100)**2 - [+] (1..100) <<..>> 2; # -> 25164150
```

14.2.6 Creando Nuevos Operadores

Hemos visto brevemente (Sección 11.11) que podemos construir nuevos operadores o redefinir aquellos existentes con nuevos tipos.

El ejemplo que proveímos fue definir el signo de sustracción como un operador infijo entre dos hashes para realizar un tipo de sustracción matemática de conjuntos, o sea, para encontrar las llaves del primer hash que no se encuentran en el segundo hash.

En el párrafo anterior, la palabra *infijo* (**en.** infix) significa que este es un operador binario (dos operandos) que se colocará entre los dos operandos.

Existen otros sabores de operadores:

- Prefijo (*prefix*): un operador unario colocado antes del operando, por ejemplo el signo de menos en la expresión -1
- Sufijo (*postfix*): un operador unario colocado después del operando, por ejemplo el signo de exclamación usado como un símbolo matemático para el factorial: $5!$
- Circunfijo (*circumfix*): un operador compuesto de dos símbolos alrededor del/de los operando(s), por ejemplo los paréntesis (...) o los paréntesis angulares $< \dots >$
- Poscircunfijo (*postcircumfix*): un operador compuesto de dos símbolos colocados después de un operando y alrededor de otro operando, por ejemplo los corchetes en `@a[1]`

Para declarar un nuevo operador, usualmente necesitas especificar los siguientes elementos en el orden especificado:

1. El tipo (prefijo, sufijo, etc.) del operador
2. Dos puntos (:)
 3. El símbolo o nombre del operador entre paréntesis angulares
 4. La signatura y el cuerpo de la función que define el operador

Por ejemplo, podríamos definir un operador prefijo `%` de la siguiente manera:

```
multi sub prefix:<%> (Int $x) { # operador doble
    2 * $x;
}
say % 21;    # -> 42
```

Esto es solamente un ejemplo para mostrar cómo la construcción de un operador funciona; `%` no es probablemente un buen nombre para un operador doble. Lo interesante aquí es que hemos reutilizado un operador existente (el operador de módulo), pero el compilador no se confunde porque el módulo es un operador infijo y nuestro nuevo operador es definido como un operador prefijo.

Un ejemplo con un mejor nombre sería usar un signo de exclamación (!) como un operador sufijo para el factorial de un número, al igual que en la notación matemática:

```
multi sub postfix:<!> (Int $n where $n >= 0) {
    [*] 2..$n;
}
say 5!;    # -> 120
```

Observa que el signo de exclamación usado como un operador prefijo (es decir, colocado al frente de su operando) es el operador de negación, pero usualmente no es posible confundir los dos operadores porque uno es un operador prefijo y nuestro operador es un operador sufijo (aunque debes ser cuidadoso con la posición donde pones los espacios en blanco si tu expresión es algo complicada). La palabra `multi` no es estrictamente requerida en este ejemplo, pero es probablemente buena práctica colocarla, solo para cubrir los casos cuando es necesaria.

Al igual que el otro ejemplo, podríamos definir el operador Σ (suma) de la siguiente manera:


```
multi sub prefix:<Σ> (@*lista-num) {
    [+] @lista-num;
}
say Σ (10, 20, 12);          # -> 42
```

El beneficio de usar el operador Σ sobre + puede no ser directamente obvio, pero es algunas veces útil crear un “lenguaje de dominio específico”(DSL), es decir, un sublenguaje específicamente adaptado para un contexto o área de estudio específica (por ejemplo, matemáticas o química), lo cual permite que un problema o solución en particular sea expresada más claramente que lo que el lenguaje principal existente permitiría. En Raku, las gramáticas y la facilidad de crear nuevos operadores hace la creación de un DSL una tarea sencilla.

El nuevo operador no tiene que ser declarado entre los paréntesis angulares. Las siguientes declaraciones podrían usarse para definir un operador de adición:

```
infix:<+>
infix:<<+>>
infix:<+>
infix:('+')
infix:("+")
```

También puedes especificar la precedencia de tus nuevos operadores (relativa a aquellos existentes). Por ejemplo:

```
multi sub infix:<mult> is equiv(&infix:<*>) { ... }
multi sub infix:<plus> is equiv(&infix:<+>) { ... }
mutli sub infix:<zz> is tighter(&infix:<+>) { ... }
mutli sub infix:<yy> is looser(&infix:<+>) { ... }
```

En uno de sus artículos (“Structured Programming with go to statements”, Diciembre 1974), Donald Knuth, un científico de la computación muy famoso, usa el símbolo `:=:` como un operador de pseudocódigo para expresar el intercambio de dos valores en una variable, i.e, la siguiente operación:

```
# Advertencia: esto es pseudocódigo, no código funcional, por el momento
my $a = 1; my $b = 2;
$a :=: $b;
say "$a $b"; # -> 2 1
```

En el artículo de Knuth, esto es solo un atajo falso para discutir más fácilmente el algoritmo de ordenamiento rápido (**en.** quicksort) (descrito en el ejercicio), pero podemos implementar fácilmente ese símbolo:

```
multi sub infix:<:=:> ($a is rw, $b is rw) {
    ($a, $b) = $b, $a;
}
```

Nota que esto puede también escribirse de esta manera:

```
multi sub infix:<:=:> ($a is rw, $b is rw) {
    ($a, $b) .= reverse; # equivalente a: ($a, $b) = ($a, $b).reverse
}
```

Podemos ahora probarlo con los siguientes ejemplos:

```
my ($c, $d) = 2, 5;
say $c :=: $d;           # -> (5 2)
# usándolo para intercambiar dos elementos del array
my @e = 1, 2, 4, 3, 5;
@e[2] :=: @e[3];
say @e;                  # -> [1 2 3 4 5]
```

Ahora, el pseudocódigo de más arriba funciona tan bien como código real. Un algoritmo de ordenamiento como el presentado más abajo (Subsección 14.3.2) puede típicamente tener líneas como estas para intercambiar dos elementos en un array:

```
if $alguna-condición {
    my ($x, $y) = @array[$i], @array[$i + gap];
    @array[$i], @array[$i + gap] = $y, $x;
}
```

Si el operador `:=:` es definido, podríamos escribir estas líneas de la siguiente manera:

```
@array[$i] :=: @array[$i + gap] if $alguna-condicion;
```

Un punto final muy interesante. Supón que queremos usar el operador $\oplus$ para la unión matemática de conjunto entre dos hashes. Esto podría escribirse fácilmente como sigue:

```
multi sub infix:<⊕> (%a, %b) {
    my %c = %a;
    %c{$_} = %b{$_} for keys %b;
    return %c
}
```

Lo cual funciona maravillosamente:

```
my %trimestre1 = ene => 1, feb => 2, mar => 3;
my %trimestre2 = abr => 4, may => 5, jun => 6;
my %semestre1 = %trimestre1 ⊕ %trimestre2;
say %semestre1;
# {abr => 4, feb => 2, ene => 1, jun => 6, mar => 3, may => 5}
```

Hasta ahora, todo bien, nada realmente nuevo. Pero el nuevo operador infijo $\oplus$ se ha vuelto casi similar a un operador integrado de Raku, y por lo tanto podemos usarlo conjuntamente con el metaoperador de reducción:

```
my %trimestre1 = ene => 1, feb => 2, mar => 3;
my %trimestre2 = abr => 4, may => 5, jun => 6;
my %trimestre3 = jul => 7, ago => 8, sep => 9;
my %trimestre4 = oct => 10, nov => 11, dic => 12;
my %año = [⊕] %trimestre1, %trimestre2, %trimestre3, %trimestre4;
say %año;
# {abr => 4, ago => 8, dic => 12, feb => 2, ene => 1,
# jul => 7, jun => 6, mar => 3, may => 5, nov => 11,
# oct => 10, sep => 9}
```

Todo funciona como si este nuevo operador fuera parte de la gramática de Raku. Y eso es lo que, en efecto, ha pasado aquí: hemos extendido el lenguaje con un nuevo operador. Esta posibilidad de extender el lenguaje es una parte clave de la habilidad de Raku para afrontar futuras necesidades que ni siquiera tenemos en cuenta en el momento presente.

14.3 Creando tus Funciones Similares a Map

Hemos visto en este capítulo y en la Sección 9.8 (p. 161) cómo las funciones de orden superior tales como las funciones `reduce`, `map`, `grep`, y `sort` pueden ser poderosas y expresivas. También existen otros tipos de funciones integradas en Raku, pero nos gustaría también crear nuestras propias funciones.

14.3.1 Versiones Personalizadas de `map`, `grep`, etc.

Veamos cómo podríamos escribir nuestra propias versiones personalizadas de tales funciones.

14.3.1.1 `mapa`, Una Versión Pura de Raku de `map`

Primeramente, intentemos escribir en puro Raku la función `map`. Necesitamos tomar una subrutina o un bloque de código como su primer argumento, aplicarlo a un array o una lista, y devolver la lista modificada.

```
sub mapa( &codigo, @valores ) {  
    my @temp;  
    push @temp, &codigo($_) for @valores;  
    return @temp;  
}  
my @result = mapa { $_ * 2 }, 1..5;  
say @result;                # -> [2 4 6 8 10]
```

En el primer intento, esto funciona exactamente como se espera. (He intentado el mismo experimento con otros lenguajes de programación en el pasado, incluyendo Perl 5; tomó varios intentos antes de obtener la versión correcta del programa, especialmente lo concerniente a la sintaxis de la llamada. Aquí, todo cae en su lugar naturalmente.) Para ser honesto, las condiciones en este ejemplo son muy limitadas y podrían haber casos donde `mapa` no funciona de la misma forma exactamente que `map`; la lección a tomar es que es bien fácil construir una subrutina de orden superior que se comporta esencialmente en la misma manera que `map`.

14.3.1.2 `mi-grep`

Escribir en puro Raku nuestra versión de `grep` es igualmente fácil:

```
sub mi-grep( &código, @values ) {  
    my @temp;  
    for @values -> $val {
```

```

        push @temp, $val if &código($val);
    }
    return @temp;
}
my @pares = mi-grep { $_ %% 2 }, 1..10;
say @pares;                                # -> [2 4 6 8 10]
```

14.3.2 Nuestra Propia Versión de la Función sort

Similarmente podemos escribir nuestra propia versión de la función sort.

La función sort en Raku implementa un algoritmo de ordenamiento conocido como *merge sort* (**en.** merge sort)¹. Algunas versiones previas del lenguaje Perl (versiones anteriores a 5.8) implementaban otro algoritmo conocido como *quick sort*². La razón por este cambio fue que, aunque quick sort es en promedio un poco más rápido que merge sort, existen casos específicos donde quick sort es mucho menos eficiente que merge sort (notablemente cuando los datos están casi ordenados). Estos casos son muy raros con datos aleatorios, pero no en situaciones reales: es muy común que tengas que ordenar una lista previamente ordenada en la cual solo algunos elementos han sido añadidos o modificados.

En la teoría de la computación, se dice frecuentemente que, para ordenar n artículos, merge sort y quick sort tienen una *complejidad promedio* de $O(n \log n)$, lo cual significa que el número de operaciones a realizarse es proporcional a $n \log n$ si el número de artículos a ser ordenados es n , con quick sort siendo usualmente un poco más rápido; pero quick sort tiene una *complejidad de peor caso* de $O(n^2)$, mientras que merge sort tiene una *complejidad de peor caso* de $O(n \log n)$. Cuando el número n de artículos a ser ordenado crece inmensamente, n^2 se vuelve significativamente más grande que $n \log n$. En otras palabras, merge sort es considerado mejor porque permanece eficiente en todos los casos.

Supón ahora que queremos implementar otra algoritmo de ordenamiento cuyo rendimiento es alegadamente mejor. Para este ejemplo, usaremos un algoritmo algo exótico conocido *comb sort* (también llamado ordenamiento de Dobosiewicz), el cual se describe en esta página de Wikipedia: https://en.wikipedia.org/wiki/Comb_sort. Este algoritmo se dice que está *en lugar*, lo cual significa que no necesita copiar los artículos en una estructura de datos auxiliar, y tiene generalmente buen rendimiento (usualmente mejor que merge sort), pero no se usa comúnmente porque su análisis teórico es muy difícil (en particular, parece ser que tiene un buen rendimiento de peor caso, pero nadie lo ha probado formalmente hasta al momento). De hecho, realmente no nos importa el rendimiento real de este tipo de algoritmo; es muy improbable que una implementación pura de Perl del comb sort sobrepasará la función integrada sort la cual está implementada en C y probablemente optimizada muy cuidadosamente por sus autores. Queremos solamente mostrar cómo una subrutina de ordenamiento podría ser implementada.

Para que funcione como sort, una función de ordenamiento puede recibir como parámetros una función de comparación o bloque de código y un array para ser ordenado, y la rutina de comparación debería usar parámetros marcadores (\$^a y \$^b en el código más abajo). Esta es una posible implementación básica:

```

sub comb_sort( &código, @array ) {
    my $max = @array.elems;
```

¹Merge sort se presenta en más detalles en la Sección 14.7.1.

²Quick sort se presenta en más detalles en Sección 14.10

```

my $brecha = $max;
loop {
    my $intercambio = False;
    $brecha = Int($brecha / 1.3);    # 1.3: factor óptimo de retracción
    $brecha = 1 if $brecha < 1;
    my $lmax = $max - $brecha - 1;
    for (0..$lmax) -> $i {
        my ($x, $y) = (@array[$i], @array[$i+$brecha]);
        (@array[$i], @array[$i+$brecha], $intercambio) = ($y, $x, True)
            if &codigo($x, $y) ~~ More;    # o: if &brecha($x, $y) > 0
    }
    last if $brecha == 1 and ! $intercambio;
}
}

```

Esto puede probarse con el código siguiente:

```

my @v;
my $max = 500;
@v[$_] = Int(20000.rand) for (0..$max);

comb_sort {$^a <=> $^b}, @v;
.say for @v[0..10], @v[493..499]; # imprime comienzo y final del array
# imprime (por ejemplo):
# (14 22 77 114 119 206 264 293 298 375 391)
# (19672 19733 19736 19873 19916 19947 19967)

```

El bucle interno compara los artículos que están distantes de cada uno por los valores \$brecha, y los intercambia si no están en el orden correcto. Al comienzo, \$brecha es grande, y se divide por una factor de retracción en cada iteración del bucle externo. El rendimiento depende inmensamente en el valor del factor de retracción. Al final, la brecha es 1 y comb sort es equivalente a un ordenamiento de burbuja (**en.** bubble sort). El factor óptimo de retracción se encuentra entre 1.25 y 1.33; he usado un factor de retracción de 1.3, el cual es el valor sugerido por los autores de las publicaciones originales que presentan el algoritmo.

14.3.3 Una Versión Iterador de map

Estas funciones mapa, mi-grep, y comb_sort son pedagógicamente interesantes, pero no son muy útiles si hacen lo mismo que las funciones equivalentes integradas (y son probablemente más lentas). Sin embargo, ya que hemos visto cómo construirlas, podemos crear nuestras propias versiones que hacen cosas diferentes.

Digamos que queremos crear una función que actúa como map en el sentido que aplica una transformación a los artículos de la lista de entrada, pero hace eso con los artículos uno por uno, a la petición del proceso consumidor, y pausa mientras el proceso consumidor no necesita nada. Esto podría describirse como un **iterador** que devuelve elementos modificados a petición de la lista de entrada. Podrías pensar que esto no está íntimamente relacionado con map, pero podría igualmente ser considerado como una forma de map con evaluación retardada, la cual procesa solo los elementos de la lista de entrada que son necesarios para el programa y nada más.

La idea de procesar solo lo que es estrictamente necesario es usualmente conocido como *pereza* (**en.** laziness), y esta es una idea muy útil. El procesamiento de una lista perezosa puede ser muy útil no solo porque evita el procesamiento innecesario de datos, y por lo tanto puede contribuir a un mejor uso de recursos y mejor rendimiento, pero también hace posible considerar listas *infinitas*: mientras puedas garantizar que solo usarás un número limitado de elementos, no tienes ningún problema en considerar listas que son potencialmente ilimitadas. Raku provee los conceptos y las herramientas para hacer todo esto.

Para reflejar estas consideraciones, llamaremos nuestra subrutina `iter-map`. Dado que podríamos también escribir una subrutina `iter-grep` y posiblemente otras, escribiremos un iterador y un transformador de datos separadamente.

Podemos usar una clausura para manufacturar un iterador:

```
sub crear-iter( @array ) {
    my $indice = 0;
    return sub { @array[$indice++]; }
}
my $iterador = crear-iter(1..200);
say $iterador() for 1..5; # -> 1, 2, 3, 4, 5
```

Ahora que el iterador devuelve un valor a la vez, podemos escribir la subrutina `iter-map`:

```
sub iter-map( &ref-codigo, $iter ) {
    return &ref-codigo($iter);
}
my $iterador = crear-iter(1..200);
say iter-map { $_ * 2 }, $iterador() for 1..5; # -> 2, 4, 6, 8, 10
```

Dado que hemos llamado la función `iter-map` solo 5 veces, ha hecho el trabajo de multiplicar los valores por 2 solo 5 veces, en lugar de hacerlo 200 veces, de las cuales 195 serían para nada. Por supuesto, multiplicar un número por 2 no es una operación costosa y el array no es tan largo, pero esto muestra como la pereza puede prevenir computaciones innecesarias. Regresaremos a esta idea, dado que Raku ofrece soporte nativo de las listas perezosas y el procesamiento perezoso.

Como ya notamos, una ventaja adicional de usar una función como `iter-map` es la posibilidad de usar listas infinitas virtualmente. Esta implementación que usa una lista infinita funciona como antes:

```
my $iterador = crear-iter(1..*);
say iter-map { $_ * 2 }, $iterador() for 1..5;
# imprime 2, 4, 6, 8, 10
```

14.3.4 Una Versión Iterador de grep

Si intentamos escribir una subrutina `iter-grep` con el mismo modelo:

```
my $iterador = crear-iter(reverse 1..10);
sub iter-grep( &ref-codigo, $iter ) {
```

```

    my $val = $iter();
    return $val if &ref-codigo($val);
}
# simulando diez llamadas
say iter-grep { $_ % 2 }, $iterador for 1..10;

```

no funciona como lo esperamos, porque esto imprimirá valores impares alternativamente (9, 7, 5, etc.) y valores indefinidos (para los valores pares del array). Aunque no lo hemos especificado todavía, preferiríamos que `iter-grep` suministrara el siguiente valor para el cual `&ref-codigo` devuelve verdadero. Esto implica que `iter-grep` tiene que iterar sobre los valores por el iterador hasta que reciba un valor apropiado.

Eso podría lucir así:

```

my $iterador = crear-iter(reverse 1..10);
sub iter-grep( &ref-codigo, $iter ) {
    loop {
        my $val = $iter();
        return unless defined $val; # evita bucle infinito
        return $val if &ref-codigo($val);
    }
}
# simulando 10 llamadas
for 1..10 {
    my $val = iter-grep { $_ % 2 }, $iterador;
    say "¡Array de entrada agotado!" and last unless defined $val;
    say $val;
}

```

Esto ahora funciona como se esperaba:

```

9
7
5
3
1
¡Array de entrada agotado!

```

Sin embargo, todavía tenemos un problema si el array contiene algunos valores indefinidos (o “celdas vacías”). Esto podría interpretarse como el final del array de entrada, mientras que podrían haber valores adicionales en el array. Esto es algunas veces conocido en la ciencia de la computación como el problema del “semi-predicado”. Aquí, `iter-grep` no tiene manera de saber la diferencia entre una celda vacía en el array y el final del array. Una implementación más robusta necesita una mejor versión de `crear-iter` que devuelve algo diferente para un artículo indefinido del array y algo diferente para el agotamiento del array. Por ejemplo, el iterador podría devolver un valor falso cuando termine con el array, y por lo contrario, un par con el artículo del array como un valor. Un par será considerado verdadero, aún si su valor no está definido:

```

sub crear-iter( @array ) {
    my $indice = 0;

```

```

my $indice-max = @array.end;
return sub {
    return False if $indice >= $indice-max;
    return ("un_par" => @array[$indice++]);
}
}
my @array = 1..5;
@array[7] = 15;
@array[9] = 17;
push @array, $_ for 20..22;
.say for 'El @array ahora: ', @array;
my $iterador = crear-iter(@array);
sub iter-grep( &codigo_ref, $iter ) {
    loop {
        my $par-devuelto = $iter();
        return unless $par-devuelto; # evita bucle infinito
        my $val = $par-devuelto.value;
        return $val if defined $val and &codigo_ref($val);
    }
}
for 1..10 {
    my $val = iter-grep { $_ % 2 }, $iterador;
    say "¡Array de entrada agotado!" and last unless defined $val;
    say $val;
}

```

Al ejecutar este script, obtenemos la siguiente salida:

```

El @array ahora:
[1 2 3 4 5 (Any) (Any) 15 (Any) 17 20 21 22]
1
3
5
15
17
21
¡Array de entrada agotado!

```

Esto ahora funciona completamente como se deseaba.

Aunque `iter-map` no sufrió del mismo problema, podrías querer modificar a `iter-map` como un ejercicio para que use nuestra nueva versión de `crear-iter`.

La ventaja de las funciones iteradores vistas más arriba es que ellas procesan solo los artículos que son requeridos por el código del usuario, y por lo tanto solo realizan las computaciones estrictamente requeridas y no gastan ciclos de CPU y tiempo haciendo trabajo innecesario. Hemos estudiado las versiones iteradores de las funciones `map` y `grep` como una forma práctica para propósitos pedagógicos de explicar en términos prácticos la idea de la pereza (*laziness*).

Esto es lo que habría sido necesario para implementar iteradores perezosos en las versiones anteriores de Perl (e.g., Perl 5), pero no se requiere todo esto con Raku, dado que el mismo

tiene soporte integrado para las listas perezosas y operadores perezosos, como veremos más adelante.

14.4 Las Construcciones `gather` y `take`

Una construcción útil para crear listas (posiblemente perezosas) es `gather { take }`. Un bloque `gather` actúa más o menos como un bucle y se ejecuta hasta que `take` suministre un valor. Esta construcción es también una forma de iterador.

Por ejemplo, el código siguiente devuelve una lista de números igual a los números pares entre 1 y 10 multiplicados por 3:

```
my @lista = gather {
    for 1..10 {
        take 3 * $_ if $_ %% 2
    }
};
say @lista;                # -> [6 12 18 24 30]
```

Aquí, `gather` itera sobre los valores del rango y `take` “devuelve” los valores deseados.

Si lo contemplas, el código de más arriba parece ser una forma de combinación de las funciones `map` y `grep`.

Podemos de hecho simular un `map`. Por ejemplo:

```
my @pares = map { $_ * 2 }, 1..5;
```

podría escribirse con un bloque `gather { take }`:

```
my @pares = gather { take $_ * 2 for 1.. 5 }; # [2 4 6 8 10]
```

Y podríamos simular un `grep` de forma similar:

```
my @pares = gather { take $_ if $_ %% 2 for 1..10 };
```

Dado que `take` también acepta una sintaxis de método, esto podría ser escrito así:

```
my @pares = gather { .take if $_ %% 2 for 1..10};
```

Estos ejemplos de código no aportan ninguna ventaja sobre `map` o `grep`, pero ilustran cómo un bloque `gather { take }` puede ser considerado como una generalización de las funciones `map` y `grep`. Y, como ya lo mencionamos, el primer ejemplo en esta sección actualmente combina las acciones de una función `map` y una función `grep`.

De hecho, podemos escribir una nueva versión de `mapa`:

```
sub mapa( &ref_codigo, @valores ) {
    return gather {
        take &ref_codigo($_) for @valores;
    };
}
say join " ", mapa { $_ * 2 }, 1..10;
# imprime: 2 4 6 8 10 12 14 16 18 20
```

Escribir una nueva versión de `my-grep` es bien fácil y se deja como un ejercicio al lector.

Llamar la función `take` solo hace sentido dentro del contenido de un bloque `gather`, pero no tiene que ser dentro del bloque (o dentro del ámbito léxico del bloque `gather`); puede ser dentro del *ámbito dinámico* del bloque `gather`.

Aunque no hemos tratado este concepto anteriormente, Raku tiene la noción de *ámbito dinámico*: contrario al ámbito léxico, el ámbito dinámico no solo contiene el bloque actual, sino que también las subrutinas llamadas desde dentro del bloque actual. Las variables con ámbito dinámico usan el twigil `"*"`. Este es un ejemplo:

```
sub escribir-result () { say $*valor; }
sub caller( Int $val ) {
    my $*valor = $val * 2;
    escribir-result();
}
caller 5;                # -> 10
```

En el código más arriba, la variable dinámica `$*valor` es declarada y definida en la subrutina `caller` y usada en la subrutina `escribir-result`. Esto no funcionaría con una variable léxica, pero funciona con una variable dinámica como `$*valor`, porque el ámbito de `$*valor` se extiende a la subrutina `escribir-result` la cual es llamada por `caller`.

Similarmente, la función `take` puede funcionar dentro del ámbito dinámico de `gather`, lo cual esencialmente significa que la función `take` puede ser llamada dentro de una subrutina llamada desde el bloque `gather`. Por ejemplo:

```
my @lista = gather {
    computar-val($_) for 1..10;
}
sub computar-val( Numeric $x ) {
    take $x * $x + 2 * $x - 6;
}
say @lista[0..5];        # -> (-3 2 9 18 29 42)
```

Como puedes ver, la función `take` no es llamada dentro del bloque `gather`, pero funciona normalmente porque está dentro del bloque dinámico del bloque `gather`, o sea, dentro de la subrutina `computar-val`, la cual es llamada en el bloque `gather`.

Un último ejemplo mostrará lo poderosa que la construcción `gather { take }` puede ser.

Consideremos este problema publicado en el sitio Rosetta Code (http://rosettacode.org/wiki/Same_Fringe): escribe una subrutina que comparará las hojas de dos árboles binarios para determinar si son las mismas listas de hojas cuando se visitan de izquierda a derecha. La estructura o balance de los árboles no importa; solo el número, orden, y valor de las hojas es importante.

La solución en Raku usa un bloque `gather { take }` y consiste de solo seis líneas de código:

```
sub margen( $arbol ) {
    multi sub periferia( Pair $nodo ) {periferia $_ for $nodo.kv;}
    multi sub periferia( Any $hoja ) {take $hoja;}
    gather periferia $arbol;
}
sub mismo-margen( $a, $b ) { margen($a) eqv margen($b) }
```

Raku es el ganador indisputable en términos del código más conciso que soluciona el problema.

Como una comparación, el ejemplo en Ada es casi 300 líneas en tamaño, los programas en C y Java tienen alrededor de 100 líneas. Por cierto, las soluciones más concisas a parte de Raku (Clojure, Picolisp, Racket) tienen alrededor de 20 líneas y todos son lenguajes de programación funcional, o (para Perl 5 por ejemplo) están escritas usando conceptos de la programación funcional. Aunque el número de líneas de código es solamente uno de muchos criterios para comparar programas y lenguajes, esto es mi opinión honesta un testimonio en favor del paradigma de programación funcional y su expresividad intrínseca.

14.5 Listas Perezosas y el Operador de Secuencia

Regresemos ahora a la idea de listas perezosas y estudiemos cómo Raku puede manejarlas y usarlas.

14.5.1 El Operador de Secuencia

Raku provee el operador de secuencia `...` para construir listas perezosas. Por ejemplo, esto:

```
my $lista-perezosa := (0, 1 ... 200);
say $lista-perezosa[42];
```

-> 42

produce una lista perezosa de enteros sucesivos entre 0 y 200. El compilador de Raku puede o no puede alocar algunos de los números (dependiendo en la implementación), pero se requiere que produzca una lista completa inmediatamente. Los números que aún no han sido generados pueden ser creados y suministrados más tarde, cuando el programa intente usar dichos valores.

Como se explica más abajo, si quieres generar enteros consecutivos, puedes actualmente simplificar la definición de la lista perezosa:

```
my $lista-perezosa:= (0 ... 200);
```

Si asignas una secuencia a un array, todos los valores de la secuencia se generarán inmediatamente, dado que la asignación a un array es ansiosa (**en**. eager) (no perezosa). No obstante, puedes forzar la pereza con la función integrada `lazy` al asignar a un array:

```
my @array-perezoso = lazy 1 ... 200;      # -> [...]
say @array-perezoso.elems;                # -> No .elems con una lista perezosa
say @array-perezoso[199];                 # -> 200
say @array-perezoso[200];                 # -> (Any)
say @array-perezoso.elems;                # -> 200
```

Aquí, `@array-perezoso` es originalmente perezoso. La evaluación de un artículo después del último elemento del array forza a Raku a actualmente generar el array completo (y el array entonces no es perezoso). Después de eso, no elementos adicionales pueden ser

generados, y `.elems` permanece en 200 (a menos que actualmente asignes valores a los elementos después del elemento 200°).

Cuando dos enteros son suministrados, uno para el primero y el otro para el último artículo de una lista, el operador de secuencia generará una lista de enteros consecutivos entre los dos números suministrados. Si suministra dos artículos iniciales que definen un paso implícitamente, esto generará una secuencia aritmética:

```
my $impares = (1, 3 ... 15);      # (1 3 5 7 9 11 13 15)
my $pares = (0, 2 ... 42);        # (0 2 4 6 8 ... 40 42)
```

Puede ser que recuerdes que, en la Sección 9.1 del capítulo sobre arrays y listas, dijimos que los paréntesis no son usualmente necesarios para construir una lista, a menos que sean necesarios por razones de precedencia. El código más arriba es uno de tales ejemplos: intenta ejecutar ese código sin los paréntesis y observa el contenido de las variables `$impares` y `$pares`.

Cuando los tres números iniciales en una progresión geométrica son suministrados, el operador de secuencia producirá una secuencia geométrica, como en este ejemplo que produce las potencias de dos:

```
say (1, 2, 4 ... 32);             # -> (1 2 4 8 16 32)
```

El operador de secuencia puede también ser usado para producir números que no son enteros, como mostramos en este ejemplo en la REPL:

```
> say (1, 1.1 ... 2);
(1 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2)
```

Contrario al operador de rango, el operador de secuencia puede también contar de forma inversa:

```
say (10 ... 1);                  # (10 9 8 7 6 5 4 3 2 1)
```

14.5.2 Listas Infinitas

Una de las cosas grandiosas sobre las listas perezosas es que, dado que la evaluación de artículos es aplazada, ellas pueden ser infinitas sin consumir recursos infinitos de la computadora:

```
my $pares = (0, 2 ... Inf);      # (...)
say $pares[18..21];              # -> (36 38 40 42)
```

El operando `Inf` se conoce como “Texas” o su equivalente ASCII del símbolo de infinidad ∞ .

El código anterior podría escribirse:

```
my $pares = (0, 2 ... ∞);
say $pares[21];                  # -> 42
```

La manera más común de indicar una lista perezosa infinita es con el uso del argumento `Whatever *`:

```
my $pares = (0, 2 ... *);
say $pares[21];                  # -> 42
```

14.5.3 Uso de un Generador Explícito

El operador de secuencia `...` es una herramienta muy poderosa para generar listas perezosas. Dado un número, el operador comienza a contar desde ese número (a menos que el final de la secuencia sea un número menor. En tal caso, cuenta en reversa.). Dado dos números para comenzar una secuencia, la tratará como una secuencia aritmética y añadirá la diferencia entre esos dos números al último número generado para generar el siguiente número. Dado tres números, verificará si representan el inicio de una secuencia aritmética o una secuencia geométrica, y continúa la secuencia desde ahí.

No obstante, muchas secuencias interesantes no son ni aritméticas ni geométricas. Ellas pueden aún generarse con el operador de secuencia provisto que un término pueda deducirse usando el número previo (o los números previos). Para esto, necesitas proveer explícitamente el bloque de código para generar el número siguiente en la secuencia. Por ejemplo, la lista de números enteros impares podría también generarse con un generador de la siguiente manera:

```
say (1, { $_ + 2 } ... 11);           # -> (1 3 5 7 9 11)
```

Tenemos ahora otra manera de definir la función factorial:

```
my $a;
my @fact = $a = 1, { $_ * $a++ } ... *;
say @fact[0..8];                     # -> (1 1 2 6 24 120 720 5040 40320)
```

o, en otra forma más legible:

```
my @fact = 1, { state $a = 1; $_ * $a++ } ... *;
say @fact[0..8];                     # -> (1 1 2 6 24 120 720 5040 40320)
```

Esta estrategia es mucha más eficiente que aquellas que hemos visto antes para uso repetitivo, dado que automáticamente almacena (caché) los valores previamente computados en una array perezoso. Como podrás recordar de la Sección 10.9 (p. 188), *caché* es el proceso por el cual un valor se almacena en la memoria para evitar tener que calcularlo nuevamente, con el objetivo de ahorrar tiempo y ciclos de CPU.

Y podemos de igual forma construir una lista infinita perezosa de los números Fibonacci:

```
my @fibo = 0, 1, -> $a, $b { $a + $b } ... *;
say @fibo[0..10];                    # -> (0 1 1 2 3 5 8 13 21 34 55)
```

Esto puede escribirse en una manera más concisa (aunque posiblemente menos explícita y menos clara) usando el parámetro marcador `Whatever *`:

```
my @fibo = 0, 1, * + * ... *;
say @fibo[~10];                      # -> (0 1 1 2 3 5 8 13 21 34)
```

Al igual que el factorial, esto es más eficiente que las implementaciones que hemos visto previamente, porque los valores computados son almacenados en el array perezoso.

Similarmente la secuencia de números enteros impares al principio de esta sección podría generarse en una forma un poco más concisa con el parámetro `Whatever "*"` :

```
say (1, * + 2 ... 11);           # -> (1 3 5 7 9 11)
```

Esta sintaxis con un asterisco se llama la *clausura Whatever*; regresaremos a ella más adelante.

Sin embargo, al usar el operador de secuencia, hay algo que se debe tener en cuenta: el valor final (cota superior) tiene que ser uno de los números generados para que la lista pare en ese valor final. Por lo contrario, el operador de secuencia construirá una lista infinita:

```
my $nums = (0, { $_ + 4 } ... 10);
say $nums[0..5];           # -> (0 4 8 12 16 20)
```

Como podemos observar en este ejemplo, el generador “salta sobre el valor final” (va más allá de 10), y la lista es de hecho infinita. Usualmente, esto no es un problema en términos de los recursos de la computadora, dado que es una lista infinita perezosa, pero es probablemente un error si esperabas que la lista no sobrepasara el número 10. En este caso específico, es muy fácil computar un punto final que coincidirá (e.g., 8 or 12), pero puede ser más complicado encontrar un punto final válido. Por ejemplo, no es algo trivial determinar el máximo número Fibonacci menor que 10,000 sin primero calcular la serie de tales números hasta el primer número que sea mayor que 10,000.

En tales casos donde es difícil predecir cuál debería ser el punto final, podemos definir otro bloque de código para determinar si la secuencia debería terminar o continuar. La secuencia parará si el bloque devuelve un valor verdadero. Por ejemplo, para computar los números Fibonacci hasta 100, podríamos usar esto en el REPL:

```
> my @fibo = 0, 1, -> $a, $b { $a + $b } ... -> $c { $c > 100 }
[0 1 1 2 3 5 8 13 21 34 55 89 144]
```

Esto es mejor, dado que termina la serie de números, pero no donde queríamos: queríamos que la serie parara en el último número Fibonacci inmediatamente menor que 100, y estamos obteniendo uno más. Sería bien fácil remover o filtrar el último número Fibonacci generado, pero es aún mejor si no lo generamos. Un pequeño cambio en la sintaxis nos ayuda a lograr el resultado esperado:

```
> my @fibo = 0, 1, -> $a, $b { $a + $b } ...^ -> $c { $c > 100}
[0 1 1 2 3 5 8 13 21 34 55 89]
```

Hacer el cambio de ... a ...^ significa que la lista resultante no incluye el primer elemento para el cual la prueba de terminación devolvió un valor verdadero.

Similarmente, podemos limitar la sintaxis de la *clausura Whatever* en la siguiente forma:

```
> say 0, 1, * + * ...^ * > 100;
(0 1 1 2 3 5 8 13 21 34 55 89)
```

14.6 Currificación y el Operador Whatever

El proceso de **currying** (o aplicación parcial) es una técnica básica de la programación funcional, especialmente en lenguajes de programación funcional puros como Haskell. El nombre “curry” proviene del matemático americano Haskell Curry, uno de los fundadores

(con Alonzo Church) de las teorías de la lógica matemática, incluyendo el cálculo lambda y otros. (Y, como pudiste adivinar, el lenguaje de programación Haskell derivó su nombre del primer nombre de Curry).

La currificación consiste en tomar una función con varios argumentos, reemplazarla con una función que tiene un solo argumento y devolver otra función (usualmente una clausura) cuyo rol es procesar los argumentos restantes.

En algunos lenguajes de programación puros, una función puede solo tomar un argumento y devolver un resultado. La currificación es una técnica creada para afrontar tal limitación. Raku no posee dicha limitación, pero la currificación puede todavía ser muy útil para reducir y simplificar las listas de argumentos en las llamadas de una subrutina, notablemente en casos de llamadas recursivas repetidas.

14.6.1 Creando una Subrutina Currificada

El ejemplo estándar de una subrutina currificada es una función de *adición*. Supón que tenemos una función matemática de adición, `añadir(x, y)`, que toma dos argumentos y devuelve la suma de los mismos.

En Raku, la definición de la subrutina `añadir` es muy simple:

```
sub añadir( Numeric $x, Numeric $y ) {return $x + $y}
```

Una versión currificada de esto sería otra función `añadir_y(x)` que devuelve una función que añade *y* a su argumento.

Esto podría lograrse con una clausura en la siguiente manera:

```
sub hacer-adición( Numeric $val-añadido ) {
    return sub ($param) {$param + $val-añadido;}
    # alternativamente: return sub {$^a + $val-añadido;}
}
my &añadir_2 = hacer-adición 2;
say añadir_2(3);           # -> 5
say añadir_2(4.5);         # -> 6.5
```

La referencia de código `&añadir_2` es una versión currificada de nuestra función matemática `añadir`. Solo toma un argumento y devuelve un valor equivalente al argumento más 2.

Por supuesto podemos crear otras subrutinas currificadas usando `hacer-adición` con otros argumentos:

```
my &añadir_3= hacer-adición 3;
say &añadir_2(6);           # -> 9
```

Esto no es nada nuevo: `&añadir_2` y `&añadir_3` son solo clausuras que recuerdan el valor de incremento pasado a la subrutina `hacer-adición`. Esto puede ser útil cuando algunas funciones se llaman muchas veces (o recursivamente) con muchos argumentos, algunos de los cuales son los mismos: la currificación de las mismas permite simplificar las llamadas de las subrutinas.

14.6.2 Currificar una Subrutina Existente con el Método `assuming`

Si una subrutina ya existe, usualmente no hay necesidad de crear una clausura nueva con la ayuda de una “función factoría” (tal como `hacer-añadir`) como hicimos anteriormente. Es posible currificar una función existente, usando el método `assuming` sobre la misma:

```
sub añadir( Numeric $x, Numeric $y ) {return $x + $y}
my &añadir_2 = &añadir.assuming(2);
añadir_2(5);                # -> 7
```

El método `assuming` devuelve un objeto *callable* (es decir, que puede llamarse) que implementa el mismo comportamiento de la subrutina original, pero que tiene los valores pasados a `assuming` ya atados a los parámetros correspondientes.

Es también posible currificar funciones integradas. Por ejemplo, la función integrada `substr` toma normalmente tres argumentos: la cadena de texto sobre la cual opera, la posición inicial, y la longitud de la subcadena de texto a ser extraída. Podrías tener que realizar un número de extracciones sobre la misma cadena de texto. Puedes crear una versión currificada de `substr` siempre trabajando sobre la misma cadena de texto:

```
my $cad = "Cogito, ergo sum";
my &cadena-inicio = &substr.assuming($cad, 0);
say &cadena-inicio($_) for 6, 13, 16;
```

Esto imprimirá:

```
Cogito
Cogito, ergo
Cogito, ergo sum
```

Nota que hemos “asumido” dos parámetros aquí, así que la subrutina currificada “recuerda” los dos primeros argumentos y solo el tercer argumento necesita pasarse a `&cadena-inicio`.

Incluso puedes currificar los operadores de Raku (o aquellos que creas) si así lo deseas:

```
my &añadir_2 = &infix:<+>.assuming(2);
```

14.6.3 Currificando con el Parámetro Whatever Star

Una manera más flexible de currificar una subrutina o una expresión es con el uso del argumento *Whatever star* (`*`):

```
my &tercio = * / 3;
say tercio(126);                # -> 42
```

El *Whatever star* (`*`) es un marcador para un argumento, así que la expresión devuelve una clausura.

Puede usarse en una forma similar a la variable tópica `$_` (excepto que no tiene que existir cuando se hace la declaración):


```
> say map 'foo' x * , (1, 3, 2);
(foo foofoofoo foofoo)
```

Es también posible usar múltiple términos Whatever en la misma expresión. Por ejemplo, la subrutina añadir podría escribirse como una expresión Whatever con dos parámetros:

```
my $añadir = * + *;
say $añadir(4, 5);          # -> 9
```

o:

```
my &añadir = * + *;
say añadir(4, 5);           # -> 9
```

Podrías hacer lo mismo con el operador de multiplicación:

```
my $mult = * * *;
say $mult(6, 7);           # -> 42
```

El compilador no se confundirá y será capaz de determinar correctamente que el primer y tercer asterisco son términos Whatever y que el segundo asterisco es el operador de multiplicación; en otras palabras, esto es más o menos equivalente a esto:

```
my $mult = { $^a * $^b };
say $mult(6, 7);           # -> 42
```

o a esto:

```
my $mult = -> $a, $b { $a * $b };
say $mult(6, 7);           # -> 42
```

Para ser honesto, el compilador no se confunde, pero el usuario podría, a menos que ella o él haya sido previamente expuesta/o a algunos lenguajes de programación funcional que comúnmente usan este tipo de construcción sintáctica.

Estas ideas son poderosas, pero se te aconseja que preste atención para que no caigas en la trampa de la ofuscación de código.

Con eso dicho, el paradigma de programación funcional es extremadamente expresivo y puede hacer tu código mucho más corto. Y, sobre todo, código más corto, provisto que permanezca claro y fácil de entender, tendrá menos errores que código más largo.

14.7 Usando un Estilo de Programación Funcional

En este capítulo, hemos discutido cómo usar técnicas derivadas de la programación funcional para hacer nuestro código más simple y más expresivo. Aunque, de cierta manera, no hemos aplicado la programación funcional completamente. Todas las técnicas que hemos visto tienen sus orígenes en la programación funcional, pero la verdadera esencia de la programación funcional no es acerca del uso de funciones de orden superior, procesamiento de listas y la programación de tuberías, subrutinas anónimas y clausuras, listas perezosas y curificación, etc. La verdadera esencia de la programación funcional es una

mentalidad que trata las computaciones como las evaluaciones de funciones matemáticas y evita el cambio de estado y los datos mutables.

En lugar de simplemente usar las técnicas derivadas de la programación funcional, podemos ir un paso más allá y actualmente escribir código en un estilo de programación funcional. Si vamos a evitar el cambio de estado y los datos mutables, esto significa que no usaremos variables (o por lo menos no las modificaremos, y las trataremos como datos inmutables) y haremos las cosas diferentemente.

14.7.1 El Algoritmo de Ordenamiento por Mezcla

Considera el ejemplo de una técnica clásica y eficiente de ordenamiento conocida como el ordenamiento por mezcla (**en.** merge sort), inventada por John von Neumann en 1945. Dicha técnica está basada en el hecho de que si tienes dos arrays ordenados, es significativamente más rápido mezclar los dos arrays en un único array, al leer cada array en paralelo y escoger el artículo apropiado de cualquiera de los dos arrays, que lo que sería si ordenamos los datos de los dos arrays ciegamente.

El ordenamiento por mezcla es un algoritmo de “división y conquista” que consiste en dividir el array de entrada desordenado recursivamente en sub-listas pequeñas y más pequeñas, hasta que cada sub-lista contenga solo un artículo (por definición, en este punto la sub-lista está ordenada), y después mezclar las sub-listas nuevamente en un array ordenado.

Para evitar agregar complejidad innecesaria, aquí discutiremos implementaciones que simplemente ordenan números en orden numérico ascendente.

14.7.2 Una Implementación No Funcional de Ordenamiento por Mezcla

Así es cómo podríamos implementar un algoritmo de ordenamiento por mezcla usando la programación puramente imperativa/procedimental:

```
# ATENCIÓN: código defectuoso
sub ord-mezcla( @salida, @a-ordenarse, $ini = 0, $fin = @a-ordenarse.end ) {
  return if $fin - $ini < 2;
  my $medio = ($fin + $ini) div 2;
  my @primero = ord-mezcla(@a-ordenarse, @salida, $ini, $medio);
  my $segundo = ord-mezcla(@a-ordenarse, @salida, $medio, $fin);
  mezclar-listas(@salida, @a-ordenarse, $ini, $medio, $fin);
}

sub mezclar-listas( @entrada, @salida, $ini, $medio, $fin ) {
  my $i = $ini;
  my $j = $medio;
  for $ini..$fin -> $k {
    if $i < $medio and ($j >= $fin or @entrada[$i] <= @entrada[$j]) {
      @salida[$k] = @entrada[$i];
      $i++;
    } else {
      @salida[$k] = @entrada[$j];
    }
  }
}
```

```

        $j++;
    }
}

my @array = reverse 1..10;
my @salida = @array;
ord-mezcla @salida, @array;
say @salida;

```

Este programa siempre funciona sobre el array completo (y su copia) y las sub-listas no son extraídas; la extracción es simulada por el uso de los subíndices de rango.

Este código no es muy largo, sin embargo es un poco complicado. Si intentas ejecutarlo, encontrarás que hay un error (*bug*): el último artículo del array original no es ordenado apropiadamente. Por ejemplo, si intentas ejecutarlo sobre una lista de 10 números enteros en orden inverso (e.g., ordenados del 10 al 1, @array) usado en la prueba al final del código más arriba, obtendrás el siguiente array:

```
[2 3 4 5 6 7 8 9 10 1]
```

Como un ejercicio, intenta arreglar el error antes de continuar (el error será arreglado más adelante).

Es probable que notarás que identificar y corregir el error es difícil, aunque el error es relativamente simple (cuando escribí este código, encontré errores más complicados antes de encontrar este). Es bastante difícil de usar los subíndices de array apropiadamente e insertar los artículos en el lugar correcto, y así evitar “errores por uno” (**en.** off-by-one errors) y otros errores.

Esta es la versión corregida:

```

sub ord-mezcla( @salida, @a-ordenarse, $ini = 0, $fin = @a-ordenarse.elems ) {
    return if $fin - $ini < 2;
    my $medio = ($fin + $ini) div 2;
    ord-mezcla(@a-ordenarse, @salida, $ini, $medio);
    ord-mezcla(@a-ordenarse, @salida, $medio, $fin);
    mezclar-listas(@salida, @a-ordenarse, $ini, $medio, $fin);
}

sub mezclar-listas( @entrada, @salida, $ini, $medio, $fin ) {
    my $i = $ini;
    my $j = $medio;
    for $ini..$fin - 1 -> $k {
        if $i < $medio and ($j >= $fin or @entrada[$i] <= @entrada[$j]) {
            @salida[$k] = @entrada[$i];
            $i++;
        } else {
            @salida[$k] = @entrada[$j];
            $j++;
        }
    }
}

```

```

}

my @array = pick 20, 1..100;
my @salida = @array;
ord-mezcla @salida, @array;
say @salida;

```

El cambio principal se encuentra en la signatura de la subrutina `ord-mezcla`: el valor por defecto para el parámetro `$fin` es el tamaño (número de elementos) del array, y no el subíndice del último elemento del array (así que el error fue un error por uno). Hacer esta corrección causa que sea necesario el cambio del bloque puntiagudo (`for $ini..$fin - 1 -> ...`) en la subrutina `mezclar-listas`.

Para 20 números enteros aleatorios entre 1 y 100, esto imprime lo siguiente:

```
[11 13 14 15 19 24 25 29 39 46 52 57 62 68 81 83 89 92 94 99]
```

La lección a tomar es que es difícil entender la implementación detallada del algoritmo, y bastante complicada de depurar, hasta usando el depurador de Raku presentado en la Sección 12.14.

14.7.3 Una Implementación Funcional de Ordenamiento por Mezcla

En lugar de modificar el array completo en cada paso a través del proceso (y confundirnos con el manejo de los subíndices), podemos dividir recursivamente los datos en sub-listas y proseguir con estas sub-listas.

Esto nos puede conllevar a la siguiente implementación:

```

sub ord-mezcla( @a-ordenarse ) {
    return @a-ordenarse if @a-ordenarse < 2;
    my $medio = @a-ordenarse.elems div 2;
    my @primero = ord-mezcla(@a-ordenarse[0 .. $medio - 1]);
    my @segundo = ord-mezcla(@a-ordenarse[$medio .. @a-ordenarse.end]);
    return mezclar-listas(@primero, @segundo);
}

sub mezclar-listas( @uno, @dos ) {
    my @result;
    loop {
        return @result.append(@dos) unless @uno;
        return @result.append(@uno) unless @dos;
        push @result, @uno[0] < @dos[0] ?? shift @uno !! shift @dos;
    }
}

```

El código es más corto que la implementación previa, pero ese no es el punto principal.

La subrutina `ord-mezcla` es algo similar a la implementación previa, excepto que crea las sub-listas recursivamente y después mezcla dichas sub-listas.

La parte más simplificada de esta implementación ahora es la subrutina `mezclar-listas` (la cual realiza la mayor parte del trabajo en ambas implementaciones): recibe dos sub-listas y las mezcla. La mayor parte de este trabajo se realiza en la última línea de código; las dos penúltimas líneas solo se encargan de devolver la lista mezclada cuando una de las sub-listas de entrada no contiene elementos.

Esta versión funcional del programa captura la esencia del algoritmo de ordenamiento por mezcla:

- Si el array contiene menos de dos artículos, ya se encuentra ordenado. Por lo tanto, devuélvelo inmediatamente (este es el caso base que detiene la recursión).
- De lo contrario, escoge la posición central del array para dividirlo en dos sub-listas, y llama a `mezclar-listas` con dichas sub-listas recursivamente;
- Mezcla las sub-listas ordenadas que han sido generadas hasta el momento.
- Devuelve la lista mezclada a la función que realiza la llamada.

Espero que puedas ver lo limpia y concisa que es la implementación al estilo funcional. Para darte una idea, escribir y depurar este último programa me tomó alrededor de 15 minutos, o sea, cerca de 10 minutos menos que la versión que no es funcional. Si no lo crees, intenta implementar estas dos versiones por ti mismo. (Es un ejercicio excelente aún si me crees.)

La sección de ejercicios de este capítulo (Sección 14.10) proveerá otro (y probablemente más relevante) ejemplo sobre la simplicidad de la programación funcional comparada a estrategias más imperativas o procedimentales.

14.8 Depuración de Programas

Esta vez, no hablaremos sobre la depuración en sí misma, sino acerca de una actividad relacionada, pruebas de software (**en. testing**).

Las pruebas de software es una parte integral del desarrollo de software. En Raku, el módulo estándar `Test` (instalado conjuntamente con Rakudo) provee una plataforma de pruebas la cual habilita la verificación automatizada y repetitiva del comportamiento del código.

Las funciones de pruebas emiten salidas que se ajustan al *Test Anything Protocol* or TAP (<http://testanything.org/>), un formato estandarizado de pruebas que tiene implementaciones en Perl, C, C++, C#, Ada, Lisp, Erlang, Python, Ruby, Lua, PHP, Java, Go, JavaScript, y otros lenguajes.

Un archivo de pruebas típico luce así:

```
use v6;
use Test;          # un módulo estándar incluido con Rakudo
use lib 'lib';

# ...
```

```
plan $num-tests;

# .... pruebas

done-testing; # opcional con 'plan'
```

Nos aseguramos que estamos utilizando Raku, a través del uso del pragma `v6`, después cargamos el módulo `Test` y especificamos donde se encuentran nuestras librerías. Consiguientemente, especificamos el número de pruebas que planeamos en ejecutar (así la plataforma de pruebas nos dejará saber cuantas pruebas se ejecutaron) y cuando finalizamos con las pruebas, usamos `done-testing` para decirle a la framework que hemos acabado.

Ya hemos visto un pequeño ejemplo sobre el uso del módulo `Test` en Solución: A.9.2 (solución al ejercicio de la Sección 11.11).

El módulo `Test` exporta varias funciones que chequean el valor devuelto por una función dada, y produce una salida estandarizada de prueba.

En la práctica, la expresión usualmente será una llamada a la función o método al cual quieres aplicar la prueba. Podrías querer chequear:

- Veracidad:

```
ok($valor, $descripción?);
nok($condición, $descripción?);
```

La función `ok` marca una prueba como aprobada (o pasada) si el `$valor` dado evalúa a verdadero (*true*) en un contexto Booleano. Al contrario, la función `nok` marca una prueba como aprobada si el valor dado evalúa a falso (*false*). Ambas funciones aceptan una descripción opcional de la prueba. Por ejemplo:

```
ok $respuesta.success, 'La respuesta HTTP fue exitosa';
nok $búsqueda.error, 'Búsqueda completada sin error';
```

- Comparación de cadenas de texto:

```
is($valor, $esperado, $descripción?)
```

La función `is` marca una prueba como aprobada si `$valor` y `$esperado` comparan positivamente con el operador `eq`. La función acepta una descripción opcional de la prueba.

- Aproximar comparación numérica:

```
is-approx($valor, $esperado, $descripción?)
```

La función `is-approx` marca una prueba como aprobada si los valores numéricos `$valor` y `$esperado` son aproximadamente iguales. La subrutina puede llamarse en varias maneras que te permiten hacer la prueba usando tolerancia relativa o absoluta de diferentes valores. (Si no se especifica una tolerancia, la tolerancia por defecto será una tolerancia absoluta de 10^{-5} .)

- Regex:

```
like($valor, $regex-esperado, $descripción?)
unlike($valor, $regex-esperado, $descripción?)
```

La función `like` marca una prueba como aprobada si el `$valor` coincide con el `$regex-esperado`. Dado que estamos hablando de regexes, “coincide”, en la oración anterior, realmente se refiere a “coincidencias inteligentes”. La función `unlike` marca una prueba como aprobada si el `$valor` no coincide con `$regex-esperado`.

Por ejemplo:

```
like 'foo', /fo/, 'foo se parece a fo';
unlike 'foo', /bar/, 'foo no se parece a bar';
```

- Y un sin número de otras funciones que puedes estudiar en la siguiente documentación: <https://docs.raku.org/language/testing.html>.

En principio, podrías usar a `ok` para cada tipo de prueba de comparación, al incluir la comparación en la expresión que se pasa como un valor:

```
ok factorial(4) == 24, 'Factorial - entero pequeño';
```

No obstante, es mejor (donde sea posible) usar una de las funciones especializadas de comparaciones, porque pueden imprimir diagnósticos más útiles en casos en los cuales la comparación falla.

Si una prueba falla (aunque parezca ser exitosa), y no entiendes la razón por la cual falla, podrías querer usar la función `diag` para obtener información adicional. Por ejemplo, asume que la prueba:

```
ok $foo, 'simple prueba';
```

está fallando y que no posees información suficiente para entender el por qué; podrías intentar esto:

```
diag "comentario extenso" unless
  ok $foo, 'simple prueba';
```

Esto te podría proveer con la información adicional que necesitas.

Supón que queremos probar una subrutina para determinar si una cadena de texto dada es palíndroma (como se discutió en varios capítulos de este libro, ver por ejemplo Ejercicio 5.3 y Subsección 8.4.7). Podrías llevar la prueba a cabo al escribir algo como esto:

```
# archivo es-palindroma.raku
use v6;

sub es-palindroma( $s ) { $s eq $s.flip }

multi sub MAIN( $entrada ) {
  if es-palindroma( $entrada ) {
    say "'$entrada' es palíndroma.";
  }
  else {
    say "'$entrada' no es palíndroma.";
  }
}
```

```
multi sub MAIN(:$test!) {
    use Test;
    plan 4;
    ok es-palindroma(''), 'cadena vacía';
    ok es-palindroma('aba'), 'ejemplo con tamaño impar';
    ok es-palindroma('abba'), 'ejemplo con tamaño par';
    nok es-palindroma('blabba'), 'contraejemplo';
}
```

Usualmente, las pruebas están almacenadas en diferentes archivos que se encuentran en un subdirectorio “t”. En este ejemplo corto, todo está en el mismo archivo, pero dos subrutinas multi MAIN son suministradas para probar si un argumento pasado es un palíndromo o para ejecutar el plan de prueba. Ver la Sección 4.15 (p. 69 y Subsección A.9.5.1 (p. 399) si necesitas un repaso sobre la subrutina MAIN.

Puedes ejecutar estas pruebas en la siguiente manera:

```
$ raku es-palindroma.raku abba
'abba' es palíndroma.
$ raku es-palindroma.raku abbaa
'abbaa' no es palíndroma.
$
$ raku es-palindroma.raku --test
1..4
ok 1 - cadena vacía
ok 2 - ejemplo con tamaño impar
ok 3 - ejemplo con tamaño par
ok 4 - contraejemplo
```

Prueba este ejemplo, juega con el mismo, cambia algunas líneas, agrega nuevas pruebas, y observa lo que pasa.

Escribir tales unidades de pruebas puede ser una tarea tediosa. La verdad es que solo las pruebas manuales son algo tediosas, y si solo intentas, encontrarás que la escritura y uso de tales pruebas hace que la tarea de pruebas sea menos complicada. Usualmente escribes las pruebas una sola vez, y las ejecutas un sin número de veces. ¡Y te sorprenderás cuantos errores encontrarás aún si estás seguro que tu código está correcto! También, una vez que escribes un conjunto de pruebas para algo, podrías usarlo años más tarde, por ejemplo para pruebas no regresivas después de un cambio de software. Esto no es solo ahorra tiempo sino que también garantiza que estás suministrando software de buena calidad.

Muchas organizaciones actualmente escriben sus pruebas antes de escribir los programas. Este proceso se conoce como *desarrollo guiado por pruebas* y hay muchas áreas donde es muy exitoso. De hecho, el compilador de Rakudo/Raku tenía un inmenso conjunto de pruebas (más de 40,000 pruebas) mucho antes que el compilador estuviera listo. En cierta manera, el conjunto de pruebas hasta se convirtió en la especificación verdadera del proyecto, para que así cualquier persona pudiera usar el mismo conjunto de pruebas para verificar otra implementación de Raku.

Una ventaja adicional de tal estrategia es que medir la proporción de pruebas que pasan puede ser una mejor medida de la realización de software que los estimados fortuitos, tales como, por ejemplo, una proporción del número de líneas de código escrito versus el estimado del número final de líneas de código.

14.9 Glosario

Objeto de primera clase Un objeto que puede pasarse como un argumento a una subrutina o devolverse como un valor de retorno de la misma. En Raku, las subrutinas son objetos de primera clase (también llamados ciudadanos de primera clase).

Función de retrollamada Una función o subrutina que se pasa como un argumento a otra función.

Función de orden superior Una función o subrutina que toma otra subrutina (o un simple bloque de código) como un argumento. Las funciones integradas *map*, *grep*, *reduce*, y *sort* son ejemplos de funciones de orden superior.

Subrutina anónima Una subrutina que carece de nombre. También llamada comúnmente una *lambda*. Aunque no son exactamente lo mismo, los bloques puntiagudos pueden también ser asimilados en subrutinas anónimas.

Clausura Una función que puede acceder variables que están lexicalmente disponible donde la función es definida, incluso aquellas variables que ya no se encuentran en el ámbito donde la función es llamada.

Programación de tubería Un modelo de programación en el cual piezas de datos (usualmente listas) experimentan transformaciones sucesivas como en una tubería o línea de ensamblaje.

Reducción Un proceso a través del cual una lista de valores es reducida a un valor único. Por ejemplo, una lista de números puede reducirse para encontrar el promedio, un valor máximo, o una mediana. Algunos lenguajes conocen este proceso como *folding*.

Metaoperador Un operador que actúa sobre otro operador para proveer nuevas funcionalidades.

Complejidad algorítmica Una medida aproximada del número de operaciones (y tiempo) necesarias para realizar una computación sobre un conjunto de datos relativamente grande, y, más precisamente, una medida de cómo un algoritmo escala cuando el conjunto de datos crece.

Evaluación perezosa Un proceso de evaluación aplazada donde, por ejemplo, se puebla una lista o procesa los artículos de una lista solamente a petición, cuando es requerido, para evitar el procesamiento innecesario.

Iterador Una pieza de código que devuelve valores a petición y mantiene un récord de donde ha llegado, para de este modo ser capaz de saber cuál valor debería ser proveído en la siguiente iteración.

Cache El proceso de caché se refiere al almacenamiento de un valor en la memoria (en una variable, un array, un hash, etc.) para de esta forma evitar la necesidad de calcularlo nuevamente, y como resultado ahorrar algún tiempo de computación.

Currificación Esta técnica hace referencia al hecho de tomar una función que acepta varios argumentos y transformarla en una serie de funciones que toman un menor número de argumentos.

Desarrollo guiado por pruebas Una metodología de desarrollo donde las pruebas de software se escriben de las especificaciones antes que el programa actual. Esto se hace para que sea más fácil chequear que el programa cumple con las especificaciones.

14.10 Ejercicio: Ordenamiento Rápido

Ejercicio 14.1. El ordenamiento rápido es un algoritmo de ordenamiento “división y conquista” inventado por Tony Hoare en 1959. Se basa en la división del array a ser ordenado. Para dividir un array, un elemento conocido como el pivote es seleccionado. Todos los elementos más pequeños que el pivote están en frente del mismo y todos aquellos que son más grandes son movidos después del mismo. Las sublistas compuestas de los elementos más pequeños y más grandes son entonces recursivamente ordenadas a través del mismo procedimiento y finalmente reagrupadas.

Una de las dificultades es seleccionar el pivote correcto. Idealmente debería ser el valor mediano de los artículos del array, dado que esto daría particiones de igual tamaño, por consiguiente haciendo el algoritmo óptimamente eficiente, pero encontrar el valor mediano para cada partición tomaría tiempo y últimamente penalizaría dicho rendimiento. Múltiples variantes de ordenamiento rápido se han intentado, con diferentes estrategias para seleccionar (usualmente de forma arbitraria) un pivote. Aquí, seleccionamos un elemento en o cerca del medio de la partición.

La siguiente es una implementación típica del algoritmo de ordenamiento rápido que no usa la programación funcional.

```
sub ordrapido( @entrada ) {
  sub intercambiar( $x, $y ) {
    (@entrada[$x], @entrada[$y]) = @entrada[$y], @entrada[$x];
  }
  sub rap_ord( $izq, $der ) {
    my $pivote = @entrada[(($izq + $der) div 2)];
    my $i = $izq;
    my $j = $der;
    while $i < $j {
      $i++ while @entrada[$i] < $pivote;
      $j-- while @entrada[$j] > $pivote;
      if $i <= $j {
        intercambiar $i, $j;
        $i++;
        $j--;
      }
    }
    rap_ord($izq, $j) if $izq < $j;
    rap_ord($i, $der) if $j < $der;
  }
  rap_ord(0, @entrada.end)
}
my @array = pick 20, 1..100;
ordrapido @array;
say @array;
```

El array es modificado en lugar (lo cual tiene la ventaja de requerir memoria limitada), lo cual significa que el array original es modificado.

Para la programación funcional, los datos internos son inmutables, y por lo tanto copias fragmentos de datos en nuevas listas, en lugar de modificarlos “directamente”.

Con el mismo espíritu que lo hicimos en la Sección 14.7 para el algoritmo de ordenamiento de mezcla, trata de escribir una implementación del algoritmo de ordenamiento rápido. Pista: esto se puede hacer en la mitad de una docena de líneas de código.

Solución: A.11.1.

Algunos Consejos Finales

Todo el mundo sabe que la depuración es dos veces más difícil que escribir un programa en primer lugar. Así que si eres tan astuto como puedas cuando lo escribes, cómo podrás depurarlo?
— Brian Kernighan, "The Elements of Programming Style".

15.1 Hazlo Claro, Mantenlo Simple

Escribir un programa en la vida real no es lo mismo que aprender el arte de la programación o aprender un lenguaje nuevo.

Debido a que la meta de este libro es conducirte al aprendizaje de conceptos más avanzados o sintaxis nueva, en la mayoría de ocasiones he tratado de hacer las cosas de maneras diferentes. Pero esto no significa que deberías utilizar tu conocimiento más avanzado en cada uno de tus programas. Es todo lo opuesto.

La regla de oro es “KISS”: Keep It Simple, Stupid (Mantenlo Simple, Estúpido). El principio de ingeniería KISS (originado en el US NAVY alrededor de 1960) establece que la mayoría de sistemas funcionan mejor si se mantienen simples en lugar de complejos; por lo tanto la simplicidad debería ser una meta en el diseño y la complejidad innecesaria debería evitarse. No obstante, esto no significa que deberías escribir código simplista.

Como un ejemplo, si estás buscando una subcadena de texto literal dentro de una cadena, usa la función integrada `index`, en lugar de utilizar el armamento completo del motor de regex para eso. Similarmente, si sabes la posición y la longitud de la subcadena de texto, usa la función `substr`. Pero si necesitas una coincidencia más “parcial” (**en.** fuzzy) con quizás algunas alternativas o una clase de caracteres, entonces un regex podría ser la herramienta indicada para el trabajo.

Otro principio relacionado es “YAGNI”: you aren’t gonna need it (no lo vas a necesitar). Este acrónimo viene de la escuela de la programación conocida como la “programación extrema” ((**en.** extreme programming), XP). Aunque no sigas todos los principios de XP,

esta idea es válida y justificada: no agregues una funcionalidad hasta que realmente la necesites.

Trata de hacer tus programas tan claros como sea posible, y tan simples como puedas. Usa conceptos más avanzados solo si es realmente necesario, pero no lo hagas solo con el propósito de presumir. No trates de ser astuto ó, por lo menos, no tan *astuto*.

Ten presente que el código no solo es usado por el compilador, sino que también por humanos. Piensa en ellos.

Contempla sobre la persona que mantendrá tu código. Como algunas personas le gusta decir: “Siempre escribe código como si la persona que termina manteniendo tu código es un psicópata violento que sabe donde vives.” Y, si esto no te convence, recuerda que la persona que mantenga tu código dentro de un año podría ser tú. Para ese entonces, puede ser que no recuerdes cómo ese truco astuto que usaste en aquel entonces funciona realmente.

Una cita final de Edsger Dijkstra sobre el tema: “La simplicidad es un prerequisite para la fiabilidad.”

15.2 Qué Hacer y Qué No Hacer

No te repitas Evita la duplicación de código. Si tienes el mismo código en partes diferentes de tu programa, entonces algo malo puede suceder. Quizás el código repetido puede ir dentro de un bucle o una subrutina separada, o a lo mejor en un módulo o una librería. Recuerda que copiar y pegar es una fuente de maldad.

No invente la rueda de nuevo Usa librerías y módulos existentes cuando puedas; es posible que hayan sido probadas y funcionarán mejor que el pequeño arreglo que escribirás. El ecosistema de Raku tiene una gran colección de módulos de software (ver `modules.raku.org`) que crece día a día. Estos módulos puedes utilizarlos en tus programas.

Usa identificadores significativos Si tus variables, métodos, clases, gramáticas, módulos, y programas tienen nombres sensibles que expresan claramente lo que son o lo que hacen, entonces tu código será más claro y podría necesitar menos comentarios. Nombres muy cortos como `$i` o `$n` son usualmente ideales para las variables de un bucle, pero todo lo demás necesita un nombre que explique el contenido o el propósito. Nombres como `$array` o `%hash` se han usados a lo largo de este libro para indicar con más claridad la naturaleza de la estructura de datos, pero no es recomendado utilizarlos en programas en la vida real. Si tu hash contiene una colección de palabras, llámalo `%palabras` o `%lista-palabras`, no `%hash`. De cualquier modo, el sigilo `%` ya indica que es un hash.

Escribe comentarios útiles y evita los inútiles Un comentario como este:

```
my $count = 1;      # asigna 1 a $count
```

es completamente inútil. En general, tus comentarios no deberían explicar lo que tu código está haciendo o cómo lo está haciendo (esto debería ser obvio si tu código es claro), sino *la razón por la cual estás haciendo eso*: quizás deberías hacer referencia a un teorema matemático, un ley de física, una decisión de diseño, o una regla de negocio.

Remueve código muerto y código de andamiaje Al escribir código nuevo, puedes crear variables que no usas en la versión final de tu código. Si esto sucede, remuévelas; no las dejes distraer la atención de tu lector. Si modificas un programa existente, limpia el lugar después de modificarlo. Recuerda la regla de los niños exploradores: deja el lugar mejor y más limpio de como lo encontraste.

Prueba agresivamente Nadie puede escribir una pieza de código significativa sin tener un número inicial de errores. Se cita a Edsger Dijkstra diciendo que: “Si la depuración es el proceso de remover errores de software, entonces la programación debe ser el proceso de crearlos.” Lamentablemente es cierto. Aunque Dijkstra también dijo que “las pruebas de software muestran la presencia, no la ausencia de errores,” las pruebas de software son una parte esencial del desarrollo de software. Escribe conjuntos de pruebas extensos, úsalos a menudo, y actualízalos a medida que la funcionalidad evoluciona. Ve la Sección 14.8 (p. 318) para aprender sobre algunas herramientas de pruebas automatizadas.

Evita la optimización prematura En las palabras de Donald Knuth: “La optimización prematura es la fuente de todo lo malvado (o por lo menos la mayoría de ello) en la programación.”

No use números mágicos: Considera esto:

```
my $tiempo-restante = 31536000;
```

¿Cuál es el significado del número 31,536,000 que aparece de la nada? No hay manera de saber con solo mirar esta línea de código. Compáralo con esto:

```
my $segundosEnUnAño = 365 * 24 * 60 * 60;  
# ...  
my $tiempo-restante = $segundosEnUnAño;
```

¿No es la segunda versión más clara?¹ Bueno, para ser sincero, sería mejor usar una constante en tal caso:

```
constant SEGUNDOS-POR-AÑO = 365 * 24 * 60 * 60;
```

Evita valores literales Los valores literales son malos. Si tienes que usar algunos, defínelos como variables o constantes al comienzo de tu programa, y usa esas variables o constantes. Las rutas literales de archivos son especialmente malas. Si tienes que usar algunas, usa algunas variables con rutas relativas:

```
my $dir-base = '/path/a/datos/aplicacion';  
my $dir-entrada = "$dir-base/ENTRADA";  
my $dir-result = "$dir-base/RESULT";  
my $dir-temp = "$dir-base/TEMP";  
my $dir-log = "$dir-base/LOG";
```

Por lo menos, si la ruta debe cambiar, solo tienes que cambiar la línea de código superior.

¹Esta calculación se refiere al año *común* (es decir, un calendario que no es bisiesto o año civil). No es la misma definición de otros años astronómicos, los cuales más de un cuarto de día más largo.

No ignore los errores devueltos por las subrutinas o funciones integradas No todos los valores de retorno son útiles; por ejemplo, usualmente no chequeamos el valor de retorno de una sentencia de impresión, pero eso es usualmente bueno porque estamos solamente interesados en el efecto secundario tal como el hecho de imprimir algo a la pantalla o a un archivo, en lugar de un valor de retorno. En la mayoría de los casos, necesitas saber si algo malo pasó para tomar los pasos necesarios para recuperarte de la condición de error si es posible o abortar el programa con gracia (e.g., con un mensaje de error informativo) si el error es muy serio para que el programa continúe.

Formatea tu código clara y consistentemente El compilador puede no importarle la indentación de código, sin embargo eso es importante para los lectores humanos. El formato de tu código debería ayudar a clarificar la estructura y flujo de control de tus programas.

Sé bueno y diviértete.

15.3 Usa Expresiones Idiomáticas

Cualquier lenguaje tiene sus métodos de “buenas prácticas”. Estas son las expresiones idiomáticas que los programadores con experiencia usan y se han convertido en las maneras preferidas de hacer las cosas. Estos modismos son importantes. Ellos te protegen de inventar la rueda nuevamente. También son los modismos que los programadores con experiencia esperan leer; son familiares y te permiten enfocarte en el diseño de código en lugar de atascarte en preocupaciones detalladas de código. Usualmente, formalizan patrones que evitan equivocaciones o errores comunes.

Aunque Raku es un lenguaje relativamente nuevo, un número de tales modismos se han perfeccionados a lo largo del tiempo. Las siguientes son algunas de estas construcciones idiomáticas ².

Crear un hash de una lista de llaves y una lista de valores: Usando rebanadas (*slices*)

```
my %hash;
%hash{@llaves} = @valores;
```

Usando el operador *zip* y un metaoperador con el constructor pareja:

```
my %hash = @llaves Z=> @valores;
```

Para las pruebas existentes, los valores de los hashes solo necesitan ser verdaderos. Esta es una manera buena de crear un hash de una lista de llaves:

```
my %existentes = @llaves X=> True;
```

O, mejor aún, usa un conjunto:

```
my $existentes = @llaves.Set;
say "existe" if $existentes{$objeto};
```

Hacer los atributos mandatorios (o parámetros de la subrutina): Esto es una forma buena de hacer un atributo mandatorio en una clase:

²Cuando se sugiere dos soluciones, la segunda es usualmente la más idiomática.

```
has $.atr = die "El atributo 'atr' es mandatorio";
```

Este código usa el mecanismo de valor por defecto: si un valor es provisto, entonces el código para el valor por defecto no se ejecuta. Si valor no es provisto, entonces el código muere con el mensaje de error apropiado. El mismo mecanismo puede usarse para los parámetros de las subrutinas.

O puedes usar el rasgo `is required`:

```
> class A { has $.a is required };
> A.new;
The attribute '$!a' is required,
but you did not provide a value for it.
```

Hasta puedes pasar un mensaje con una explicación:

```
> class A { has $.a is required("Lo necesitamos") };
> A.new;
The attribute '$!a' is required because Lo necesitamos,
but you did not provide a value for it.
```

Iterar sobre los subíndices de un array La primera solución que viene a la mente podría ser:

```
for 0 .. @array.end -> $i {...}
```

Eso está bien, pero esto es probablemente lo mejor:

```
for @array.keys -> $i {...}
```

Iterar sobre los subíndices y valores de un array El método `.kv`, en combinación con un bloque puntiagudo que toma dos parámetros, te permite fácilmente iterar sobre un array:

```
for @array.kv -> $i, $valor {...}
```

Imprimir el número de artículos en un array Dos posibles soluciones:

```
say +@array;
# o:
say @array.elems;
```

Hacer algo cada tercera vez Usa el operador de divisibilidad `%%` en la variable del bucle:

```
if $i %% 3 {...}
```

Hacer algo cada *n* vez: Usa el operador de rango de derecha exclusiva:

```
for 0 ..^ $n {...}
# o, más simple:
for ^$n {...}
```

Divide una cadena de texto en palabras (dividiendo en espacio): Un invocación de método sin un invocante explícito siempre usa la variable tópica `$_` como un invocante implícito. Así que, asumiendo que la cadena de texto ha sido asignada a `$_`:

```
@palabras = .split(/\s+/);
# o, más simple:
@palabras = .words;
```


Un bucle infinito Una sentencia `loop` sin paréntesis y sin argumentos se ejecuta indefinidamente:

```
while True {...}
# o, más idiomático:
loop {...}
```

Por supuesto, el cuerpo de la sentencia `loop` debe tener algún tipo de sentencia de flujo de control para salir del bucle en un momento determinado.

Retornar los elementos únicos de una lista El método `unique` remueve los elementos duplicados de la lista de entrada:

```
return @array.unique;
```

O, si sabes que la lista está ordenada, puedes usar la función `squish` (la cual remueve los elementos duplicados adyacentes).

Añadir los elementos de una lista Usa la función `reduce` o el metaoperador de reducción:

```
my $suma = @a.reduce(* + *);
# o, más simple:
my $suma = [+] @a;
# o hasta más simple, usando la suma integrada:
my $suma = @a.sum;
```

Intercambiar dos variables Usa la invocación mutante del método `.=` con la función `reverse`:

```
( $x, $y ) = $y, $x;
# o:
( $x, $y ) .= reverse; # equivalente a: ($x, $y) = ($x, $y).reverse
```

Generar enteros aleatorios entre 2 y 6 Usa el operador de rango `..` y el método `pick`:

```
$z = 2 + Int(5.rand);
# o, mejor:
$z = (2..6).pick;
```

Contar progresivamente de 3 en 3 en un bucle infinito Usa el operador de secuencia con el operador `whatever star` `"*"` y el operador `puntiagudo`:

```
for 3, * + 3 ... * -> $n {...}
# o:
for 3, 6, 9 ... * -> $n {...}
```

Recorrer un rango de valores, ignorando los límites del rango: Usa el operador de rango exclusivo:

```
for ($comienzo+1) .. ($final-1) -> $i {...}
# o, mejor aún:
for $comienzo ^..^ $final -> $i {...}
```

15.4 ¿Qué sigue?

En un libro como este no se te puede decir todo sobre la programación ni todo sobre Raku. A esta altura, deberías saber cómo escribir un programa para resolver un problema de dificultad promedia, pero mucho se ha hecho en la última década para resolver problemas más complejos. Así que, hacia dónde deberíamos dirigirnos desde aquí?

Lee libros sobre algoritmos. Muchos libros excelentes tratan el tema, pero especialmente recomendando los siguientes dos (aunque deberías saber que no son fáciles):

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, The MIT Press
- Donald Knuth, *The Art of Computer Programming*, Addison Wesley (many volumes, many editions).

Lee otros libros sobre la programación, aún aquellos cuyos objetivos son otros lenguajes de programación o no lenguaje en específico. Es probable que tengan una perspectiva diferente sobre varios temas; esto te ofrecerá un punto de vista diferente y quizás un mejor entendimiento, y complementará lo que leíste aquí. Lee tutoriales, artículos, blogs, y foros sobre la programación. Participa cuando puedas. Leer la *Guía de Raku*, por Naoum Hankache, la cual se ha traducido a docenas de lenguajes al momento de escribir este libro (<http://raku.guide/>). Lee la documentación oficial de Raku (<https://docs.raku.org/>).

Este libro tiene más de mil ejemplos de código, lo cual es mucho, pero podría ser no suficiente si quieres aprender mucho más. Deberías leer fragmentos de código escritos por otros. Chequea las librerías o módulos de código abierto e intenta entender lo que hacen y cómo lo hacen. Intenta usarlas.

Habiendo dicho eso, debería enfatizar que puedes leer tantos libros como quieras sobre la teoría de la natación, pero nunca aprenderás a nadar hasta que lo intentes. Lo mismo es cierto sobre el aprendizaje de la programación y un lenguaje de programación. Escribe código nuevo. Modifica los ejemplos existentes y observa lo que sucede. Intenta cosas nuevas. Continúa, sé valiente, sumérgete en la piscina y nada. El resultado es que: aprenderás al hacer.

¡Aprender el arte de la programación es divertido. Disfrútalo!



Soluciones a los Ejercicios

Este (largo) capítulo provee soluciones a los ejercicios sugeridos en los capítulos principales del libro. No obstante, también contiene muchísimo más.

Primero, en muchos casos, provee varias soluciones diferentes, ilustrando así estrategias distintas para solucionar un problema, discutiendo sus méritos respectivos o desventajas y usualmente mostrando soluciones que pueden ser más eficientes que otras.

Segundo, usualmente provee muchísima información adicional o ejemplos complementarios.

Solo el alto volumen de ejemplos de código de este capítulo podría enseñarte muchísimo sobre la programación en general y acerca del lenguaje Raku en particular.

Finalmente, este capítulo algunas veces introduce (con ejemplos) conceptos nuevos serán discutidos en capítulos posteriores de este libro. Haber visto tales ejemplos puede ayudarte a tener una mayor comprensión de estas nuevas ideas al leer estos capítulos posteriormente. En algunos casos, este capítulo discute o introduce nociones que no será discutidas en otra parte del libro.

Cuando resuelvas un ejercicio, aún si crees que lo hiciste maravillosamente, por favor asegúrate de consultar las soluciones en este capítulo e intentarlas: es altamente probable que aprenderás algo de ellas.

A.1 Ejercicios del Capítulo 3: Funciones y Subrutinas

A.1.1 Ejercicio 3.1: Subrutina `alinear-a-derecha` (p. 51)

El objetivo es escribir una subrutina que imprima una cadena de texto con suficiente espacio en blanco inicial para que la última letra de la cadena de texto esté en la columna 70 de la pantalla (o terminal).

Este es el primer ejercicio real de este libro, así que hagámoslo paso a paso:

```

use v6;
sub alinear-a-derecha( $cadena-entrada ) {
    my $long_cadena = chars $cadena-entrada;
    my $long_restante = 70 - $long_cadena;
    my $espacios_delanteros = " " x $long_restante;
    say $espacios_delanteros, $cadena-entrada;
}
alinear-a-derecha("Larry Wall");
alinear-a-derecha("The quick brown fox jumps over the lazy dog");

```

Esta subrutina:

- Asigna la cadena de entrada a la variable `$long_cadena`;
- Computa el número de espacios en blanco que necesitará para agregarse al inicio de la línea a mostrarse para que finalice en la columna 70. Asigna este resultado a la variable `$long_restante`;
- Crea la cadena de texto `$espacios_delanteros` con el número necesario de espacios en blanco;
- Imprime los `$espacios_delanteros` y la `$cadena-entrada` uno detrás de otro para obtener el resultado deseado.

Esto muestra lo siguiente:

```

                                Larry Wall
The quick brown fox jumps over the lazy dog

```

Sin embargo, podemos hacer el código más conciso al componer algunas de las sentencias y expresiones:

```

sub alinear-a-derecha( $cadena-entrada ) {
    my $espacios_delanteros = " " x (70 - $cadena-entrada.chars);
    say $espacios_delanteros, $cadena-entrada;
}

```

Podríamos hasta reducirla a una subrutina más corta con una sola línea:

```

sub alinear-a-derecha( $cadena-entrada ) {
    say " " x 70 - $cadena-entrada.chars, $cadena-entrada;
}

```

Esto funciona bien, pero podría argumentarse que esta última versión es menos clara. De hecho, la variable temporaria `$espacios_delanteros` usada en la versión previa tenía un nombre que documentaba lo que la subrutina estaba haciendo. Puedes hacer el código muy conciso en el último ejemplo, pero algunas veces se puede volver un poco terso, así que hay una compensación entre lo conciso y la claridad.

Nota que hay dos funciones integradas, `printf` y `sprintf`, que pueden realizar tareas similares. También está el método `.fmt` para producir salida formateada.

A.1.2 Ejercicio 3.2: Subrutina do-twice (p. 51)

Para agregar un destinatario al saludo, necesitamos:

- Pasar un segundo argumento en la llamada a do-twice (la cadena de texto “Mundo”)
- Agregar un parámetro nuevo en la signature de la subrutina (\$destinatario)
- Agregar este parámetro nuevo como un argumento en las llamadas a \$código
- Agregar una signature con un parámetro (\$dest) en la definición de la subrutina saludar
- Usar este parámetro nuevo en la sentencia de impresión

Esto resulta en el código siguiente:

```
sub do-twice( $código, $destinatario ) {  
    $código($destinatario);  
    $código($destinatario);  
}  
sub saludar( Str $dest ) {  
    say "¡Hola $dest!";  
}  
do-twice &saludar, "World";
```

Esto muestra:

```
¡Hola World!  
¡Hola World!
```

Para la pregunta siguiente, reemplazamos la subrutina saludar con la subrutina doble-impresión:

```
sub do-twice( $código, $mensaje ) {  
    $código( $mensaje );  
    $código( $mensaje );  
}  
sub doble-impresión( $valor ) {  
    say $valor;  
    say $valor;  
}  
do-twice &doble-impresión, "What's up doc";
```

Esto imprime “What’s up doc” cuatro veces.

Finalmente, agregamos la subrutina nueva do-four y las dejamos llamar la subrutina do-twice dos veces, lo cual imprime el mensaje ocho veces:

```
sub do-twice( $código, $mensaje ) {  
    $código( $mensaje );  
    $código( $mensaje );  
}
```

```

sub doble-impresión( $valor ) {
    say $valor;
    say $valor;
}
sub do-four( $código, $mensaje ) {
    do-twice $código, $mensaje;
    do-twice $código, $mensaje;
}
do-four &doble-impresión, "What's up doc";

```

A.1.3 Ejercicio 3.3: Subrutina imprimir-cuadr (p. 52)

Para imprimir una cuadrícula como la requerida en el ejercicio, necesitamos imprimir cada línea una por una, y tenemos básicamente dos tipos de línea: las tres líneas “con rayas” y las ocho líneas sin rayas, las cual llamaremos “líneas vacías” porque están parcialmente vacías (sin rayas).

Para evitar la repetición de código, una manera de hacerlo es creando una cadena de texto para cada uno de los dos tipos de líneas e imprimir estas cadenas de texto de acuerdo a las necesidades.

Esta es una de las soluciones posibles:

```

use v6;

my $cuatro-rayas = "- " x 4;
my $línea-rayada = ("+ " ~ $cuatro-rayas ) x 2 ~ "+" ;
my $espacios = " " x 9;
my $línea-vacía = ("|" ~ $espacios ) x 2 ~ "|" ;

sub decir-cuatro-veces( $valor ) {
    say $valor;
    say $valor;
    say $valor;
    say $valor;
}

sub imprimir-cuadr {
    say $línea-rayada;
    decir-cuatro-veces $línea-vacía;
    say $línea-rayada;
    decir-cuatro-veces $línea-vacía;
    say $línea-rayada;
}

imprimir-cuadr;

```

Obviamente hay maneras mejores de hacer algo cuatro veces que solo repetir `say $valor;` cuatro veces con en la subrutina `decir-cuatro-veces` más arriba, pero esto será discutido en el Capítulo 4 (ver Sección 4.10).

Para dibujar una cuadrícula similar con cuatro filas y cuatro columnas, primero necesitamos modificar las cadenas usadas para imprimir las líneas:

```

my $línea-rayada = (" " ~ $cuatro-rayas ) x 4 ~ "+" ;
# ...
my $línea-vacia  = ("|" ~ $espacios ) x 4 ~ "|" ;

```

Además de eso, podríamos modificar a `imprimir-cuadr` para que solo imprima cada línea el número requerido de veces. Pero eso involucra un poco de repetición de código, y el propósito de este ejercicio es usar las subrutinas para permitir la reutilización de código.

Hay dos cosas que necesitamos hacer cuatro veces repetidamente. Hace sentido escribir una subrutina `hacer-cuatro-veces` que se usará para crear la subrutina `decir-cuatro-veces` y para llamar filas completas cuatro veces. A esta subrutina se le pasará la referencia de código para las acciones específicas requeridas:

```

my $cuatro-rayas  = "-" x 4;
my $línea-rayada  = (" " ~ $cuatro-rayas ) x 4 ~ "+" ;
my $espacios      = " " x 9;
my $línea-vacia   = ("|" ~ $espacios ) x 4 ~ "|" ;

sub hacer-cuatro-veces( $código ) {
    $código();
    $código();
    $código();
    $código();
}

sub decir-cuatro-veces( $valor ) {
    hacer-cuatro-veces(sub {say $valor});
}

sub impr-cuadr-sin-fondo {
    say $línea-rayada;
    decir-cuatro-veces $línea-vacia;
}

sub imprimir-cuadr {
    hacer-cuatro-veces(&impr-cuadr-sin-fondo);
    say $línea-rayada;
}

imprimir-cuadr;

```

Además, en lugar de declarar las variables globales para las cadenas de texto que representan las líneas, es mejor práctica declararlas y definir las dentro de las subrutinas donde son usadas. Ya no necesitamos la subrutina `decir-cuatro-veces`; podemos solo pasar los argumentos relevantes a la subrutina `hacer-cuatro-veces` para obtener el mismo efecto. Esto resulta en el programa siguiente:

```

sub hacer-cuatro-veces( $código, $val ) {
    $código($val);
    $código($val);
    $código($val);
    $código($val);
}

sub impr-cuadr-sin-fondo( $línea-punteada ) {
    say $línea-punteada;
}

```



```

    my $espacios = " " x 9;
    my $línea-espacio = ("|" ~ $espacios ) x 4 ~ "|" ;
    hacer-cuatro-veces(&say, $línea-espacio);
}
sub imprimir-cuadr {
    my $cuatro-rayas = "- " x 4;
    my $línea-rayada = ("+ " ~ $cuatro-rayas ) x 4 ~ "+" ;
    hacer-cuatro-veces(&impr-cuadr-sin-fondo, $línea-rayada);
    say $línea-rayada;
}
imprimir-cuadr;

```

A.2 Ejercicios del Capítulo 4: Condicionales y Recursión

A.2.1 Subrutina hacer-n-veces, Ejercicio Sugerido en la Sección 4.12 (p. 68)

Necesitamos una subrutina que tome una función y un número, \$num, como argumentos, y que llame la función dada \$num veces.

La subrutina hacer-n-veces es recursiva y se llama a sí misma con un argumento disminuido. La recursión se para cuando este argumento es 0. \$subref es una subrutina anónima llamada dentro del cuerpo de hacer-n-veces:

```

sub hacer-n-veces( $refcódigo, Int $num) {
    return if $num <= 0;
    $refcódigo();
    hacer-n-veces $refcódigo, $num - 1;
}

my $subref = sub { say "Carpe diem";}

hacer-n-veces $subref, 4;

```

Esto imprime:

```

Carpe diem
Carpe diem
Carpe diem
Carpe diem

```

A.2.2 Ejercicio 4.1: Días, Horas, Minutos, y Segundos (p. 72)

La siguiente es una manera posible de convertir un número de segundos a números de días, horas, minutos y segundos:

```

días-HMS(240_000);

sub días-HMS( Int $segundos ) {

```

```

my $minutos = $segundos div 60;
my $seg_restantes = $segundos mod 60;
my ($horas, $min_restantes) = $minutos div 60, $minutos mod 60;
my ($días, $h_restantes) = $horas div 24, $horas mod 24;
say "$días $h_restantes $min_restantes $seg_restantes";
    # imprime: 2 18 40 0
}

```

Las primeras dos líneas realizan la división entera y la operación de módulo separadamente. Para los dos casos siguientes, hacemos las dos operaciones en una sola línea, usando una sintaxis de lista.

Las variables \$minutos, \$horas, y \$días se computan todas en una manera similar. El código podría hacerse más modular al usar una subrutina para calcular los \$minutos, las \$horas, y las \$días. Aunque las subrutinas fructuosas serán realmente usadas en el capítulo siguiente, hemos ya visto varios ejemplos de ellas y pueden proveer un resumen acerca de cómo podrían ser utilizadas:

```

sub div_mod( Int $entrada, Int $num-base ) {
    return $entrada div $num-base, $entrada mod $num-base;
}
sub días-HMS( Int $segundos ) {
    my ($minutos, $seg_restantes) = div_mod $segundos, 60;
    my ($horas, $min_restantes) = div_mod $minutos, 60;
    my ($días, $h_restantes) = div_mod $horas, 24;
    say "$días $h_restantes $min_restantes $seg_restantes";
}

```

Para incitar al usuario a entrar un número de segundos, podrías hacer esto:

```

my $seg = prompt "Por favor entre el número de segundos: ";
días-HMS $seg.Int;

```

En la vida real, sería bueno verificar que la entrada proveída por el usuario es un entero positivo y preguntar nuevamente si no lo es. Como un ejercicio adicional, podrías insertar el código anterior en una subrutina recursiva que imprime un mensaje de error y se llama a sí misma otra vez si la entrada del usuario no es válida. La solución al ejercicio siguiente (Solución: A.2.3) provee un ejemplo de una subrutina recursiva diseñada para incitar al usuario a suministrar la entrada nuevamente; esto podría ayudarte a cómo hacerlo si encuentras alguna dificultad.

Intenta reemplazar la línea de código siguiente:

```
say "$días $h_restantes $min_restantes $seg_restantes";
```

con esta:

```
printf "%d días %d horas %d minutos %d segundos \n", días-HMS 240_000;
```

para mostrar una salida mejor formateada.

A.2.3 Ejercicio 4.2: Teorema de Fermat (p. 72)

La subrutina chequear-fermat chequea si:

$$a^n + b^n = c^n$$

es verdadera para los valores a , b , c , and n suministrados.

```
sub chequear-fermat( Int $a, Int $b, Int $c, Int $n ) {
  if $a**$n + $b**$n == $c**$n {
    if $n > 2 {
      say ";Santo dios, Fermat estaba equivocado!" if $n > 2;
    } elsif $n == 2 or $n == 1 {
      say "Correcto";
    }
    return True;
  }
  return False
}

say "Correcto para 3, 4, 5, 2" if chequear-fermat 3, 4, 5, 2;
obtener-entrada();

sub obtener-entrada {
  say "Tu misión, Jim, si la aceptas, es ";
  say "proveer los valores A, B, C y n que satisfacen la ecuación de Fermat:";
  say "  A ** n + B ** n = C * *n";
  my $a = prompt "Por favor proveer un valor para A: ";
  my $b = prompt "Por favor proveer un valor para B: ";
  my $c = prompt "Por favor proveer un valor para C: ";
  my $n = prompt "Por favor proveer un valor para el exponente: ";
  if chequear-fermat($a.Int, $b.Int, $c.Int, $n.Int) {
    say "La ecuación se satisface para tus valores";
  } else {
    say "Qué lata. La ecuación no está correcta."
  }
  my $intentar-nuevamente = prompt ";Quieres intentar nuevamente (S/N)?";
  obtener-entrada if $intentar-nuevamente eq 'S';
}
```

El último teorema de Fermat ha sido probado y, evidentemente, la misión es realmente imposible si $n > 2$; quizás esta vez Jim Phelps debería declinar la misión.

A.2.4 Ejercicio 4.3: ¿Es un triángulo? (p. 72)

Esta es una subrutina posible para verificar si puedes hacer un triángulo con tres longitudes de palillos dadas:

```
sub es-triángulo( Numeric $x, Numeric $y, Numeric $z ) {
  my $válido = True;
```

```

    $válido = False if $x > $y + $z;
    $válido = False if $y > $x + $z;
    $válido = False if $z > $x + $y;
    if $válido {
        say "Sí";
    } else {
        say "No";
    }
}
es-triángulo 1, 3, 4; # -> Sí
es-triángulo 1, 3, 6; # -> No

```

Otra manera de hacer esto sería comenzar a encontrar la longitud más larga y probar con esa solamente, pero eso no hace el algoritmo significativamente más simple.

Incitar al usuario a entrar tres longitudes se ha mostrado en los ejercicios previos; nada nuevo aquí. No obstante, esta es una forma nueva de hacerlo:

```

my ($a, $b, $c) = split " ",
    prompt "Por favor entra tres longitudes (separadas por espacios): ";
es-triángulo $a.Int , $b.Int , $c.Int;

```

A.2.5 Ejercicio 4.4: Los Números Fibonacci (p. 73)

Los números Fibonacci son una secuencia de números en la cual los primeros dos números son igual a 1 y cualquier número subsecuente es la suma de los dos números anteriores, por ejemplo:

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Imprimir los primeros 20 números Fibonacci:

```

sub n_fibonacci (Int $n) {
    my $fib1 = 1;
    my $fib2 = 1;
    say $_ for $fib1, $fib2;
    for 3..$n {
        my $nuevo_fib = $fib1 + $fib2;
        say $nuevo_fib;
        ($fib1, $fib2) = $fib2, $nuevo_fib;
    }
}
n_fibonacci 20;

```

Imprimir el n .º número Fibonacci:

```

my $n = prompt "Entra el número Fibonacci a buscarse: ";
$n = $n.Int;
say fibo($n);

sub fibo( Int $n ) {

```

```

my ($fib1, $fib2) = 1, 1;
for 3..$n {
    my $nuevo_fib = $fib1 + $fib2;
    ($fib1, $fib2) = $fib2, $nuevo_fib;
}
return $fib2;
}

```

A.2.6 Ejercicio 4.5: La Subrutina `recurse` (p. 73)

Examinando el código de la subrutina `recurse`, la primera cosa que deberías notar es cada vez que es llamada recursivamente, el primer argumento (`$n`) es reducido por uno comparado con la llamada previa. Si el valor inicial de `$n` es un número entero positivo, la sucesión de llamadas eventualmente llegará al caso base donde `$n == 0`, y la cascada de llamadas recursivas cesará.

Si `$n` no es un número entero o si es negativo, obtendremos una recursión infinita.

Una manera de visualizar cómo el programa se ejecuta es mostrar los parámetros de la subrutina en cada llamada:

```

sub recurse( $n, $s ) {
    say "Args : n = $n, s = $s";
    if ($n == 0) {
        say $s;
    } else {
        recurse $n - 1, $n + $s;
    }
}
recurse 3, 0;

```

Esto imprimiría:

```

Args : n = 3, s = 0
Args : n = 2, s = 3
Args : n = 1, s = 5
Args : n = 0, s = 6
6

```

Para evitar que los argumentos conduzcan a una recursión infinita, podemos agregar restricciones de tipo entero a la signature de la subrutina y algún código para cesar la recursión si el primer argumento es negativo, por ejemplo:

```

sub recurse( Int $n, Int $s ) {
    say "Args : n = $n, s = $s";
    if $n == 0 {
        say $s;
    } elsif $n < 0 {
        die '¡PARA! $n es negativo, nos damos por vencidos!';
    } else {

```

```

        recurse $n - 1, $n + $s;
    }
}

```

Ahora, si llamamos a `recurse` con un valor negativo para `$n`, obtenemos un error:

```

Args : n = -1, s = 0
¡PARA! $n es negativo, nos damos por vencidos
in sub recurse at recurse2.raku line 6
in block <unit> at recurse2.arku line 12

```

Si la llamamos con un valor que no es entero para `$n`:

```

===SORRY!=== Error while compiling recurse2.raku
Calling recurse(Rat, Int) will never work with declared
signature (Int $n, Int $s)
at recurse2.raku:12
-----> <BOL><HERE>recurse 1.5, 0;

```

Otra posibilidad sería usar una característica de Raku que no hemos discutido todavía, subrutinas *multi*, las cuales son descritas en la Sección 5.10 (p. 87). La idea es declarar dos versiones de la subrutina `recurse`, las cuales tienen el mismo nombre pero firmas diferentes. El compilador averiguará cual versión de `recurse` debe llamar dependiendo en cual firma aplica a los argumentos pasados a la subrutina:

```

multi recurse( Int $n where $n >= 0, $s ) {
    say "Args : n = $n, s = $s";
    if ($n == 0) {
        say $s;
    } else {
        recurse $n - 1, $n + $s;
    }
}

multi recurse( $n , $s ) {
    say "Args : n = $n, s = $s";
    # haz algo diferente para tal caso, por ejemplo:
    # recurse (abs $n.Int), $s; # llamando la primera versión de recurse
    # o simplemente:
    say '¡PARA! $n es negativo, nos damos por vencidos';
}

```

Si el primer parámetro es un entero positivo, la primera versión de `recurse` será llamada. Por lo contrario, la segunda versión se ejecutará:

```

$ raku recurse.raku
Args : n = 6.1, s = 0
¡PARA! $n es inválido, nos damos por vencidos

$ raku recurse.raku
Args : n = -1, s = 0
¡PARA! $n es negativo, nos damos por vencidos

```

Intentar ejecutar el código siguiente para la segunda definición de `recurse`:

```
multi recurse( $n , $s ) {
  say "Args : n = $n, s = $s";
  recurse (abs $n.Int), $s;
}
```

para observar lo que está ocurriendo en ese caso.

A.3 Ejercicios del Capítulo 5: Funciones Fructuosas

A.3.1 Comparar, ejercicio al final de la Sección 5.1 (p. 77)

Aquí presentamos una subrutina que toma dos números y los compara, y devuelve 1 si el primero es mayor que el segundo, 0 si son iguales, y -1 por lo contrario (es decir, si el segundo es mayor que el primero):

```
sub comparar( Numeric $x, Numeric $y ) {
  return 1 if $x > $y;
  return 0 if $x == $y;
  return -1;
}

say comparar 4, 7;    # -1
say comparar 7, 4;    # 1
say comparar 5, 5;    # 0
```

Nota: esto ejemplifica una función `comparar` de tres sentidos comúnmente usado para ordenar un número en los lenguajes de programación, incluyendo versiones más antiguas de Raku (tal como Perl 5). En Raku, los operadores que implementan esta funcionalidad (los comparadores de tres sentidos `cmp`, `leg` y `<=>`) devuelven tipos especiales de valores: `Order::More`, `Order::Less`, y `Order::Same`. (ver Sección 9.11 acerca de ordenamiento en el capítulo sobre arrays y listas para más detalles.)

A.3.2 Hipotenusa, ejercicio al final de la Sección 5.2 (p. 79)

El objetivo de este ejercicio es usar el plan de desarrollo incremental para calcular la hipotenusa de un triángulo rectángulo (usando el teorema de Pitágora).

Podríamos comenzar con un esbozo de la subrutina:

```
sub hipotenusa( Numeric $x, Numeric $y ) {
  return 0;
}

say hipotenusa 3, 4;
```

Esto imprimirá 0 obviamente.

Luego, calculamos la hipotenusa e lo imprime dentro de la subrutina:

```

sub hipotenusa( Numeric $x, Numeric $y ) {
  my $hipotenusa = sqrt ($x ** 2 + $y ** 2);
  say "hipotenusa = $hipotenusa";
  return 0.0;
}
say hipotenusa 3, 4;

```

Esto imprime:

```

hipotenuse = 5
0

```

La subrutina está calculando la hipotenusa (5) correctamente, pero está devolviendo el valor ficticio 0. Ahora podemos devolver seguramente el resultado (y remover el andamiaje):

```

sub hipotenusa( Numeric $x, Numeric $y ) {
  my $hipotenusa = sqrt ($x ** 2 + $y ** 2);
  return $hipotenusa;
}
say hipotenusa 3, 4;

```

Esto imprime correctamente el valor de la hipotenusa.

Finalmente, podemos, si así lo deseamos, remover la variable temporaria para simplificar la subrutina aún más:

```

sub hipotenusa( Numeric $x, Numeric $y ) {
  return sqrt ($x ** 2 + $y ** 2);
}
say hipotenusa 3, 4;

```

A.3.3 Operadores de Relación Encadenados (en la Sección 5.4)

Necesitamos una subrutina para averiguar si la expresión $x \leq y \leq z$ es verdadera o falsa. Simplemente necesitamos probarlo con un operador de relación encadenado y devolver el resultado:

```

sub está-entre( Numeric $x, Numeric $y, Numeric $z ) {
  return $x <= $y <= $z;
}
say está-entre 3, 5, 6; # True
say está-entre 3, 8, 7; # False
say está-entre 6, 5, 6; # False
say está-entre 6, 6, 7; # True

```

Nota que las pruebas proveídas aquí son solo un número limitado de ejemplos, con propósitos de ilustración. Un conjunto de pruebas más rigurosa podría ser necesario (probar por ejemplo con números negativos y positivos). Más tarde veremos mejores maneras de construir conjuntos de pruebas más robustos (ver por ejemplo la Sección 14.8 y la solución al ejercicio en la Solución: A.9.2).

A.3.4 La Función Ackermann (Ejercicio 5.2)

Escribe una subrutina para computar la función Ackermann. La función Ackermann, $A(m, n)$, se define como sigue:

$$A(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

Aquí presentamos una manera de computar la función Ackermann en Raku:

```
sub ack (Int $m, Int $n) {
    return $n + 1 if $m == 0;
    return ack($m - 1, 1) if $n == 0;
    return ack($m - 1, ack($m, $n-1));
}
say ack 3, 4;    # -> 125
```

Hemos usados paréntesis para mostrar mejor la estructura, pero funciona igualmente sin ellos. Aún en la última línea de código con dos llamadas a la subrutina, la signatura de la subrutina con dos números enteros es suficiente para que el compilador de Raku entienda cual argumento está asociado con una de la llamada:

```
sub ack (Int $m, Int $n) {
    # say "m n = $m, $n";
    return $n + 1 if $m == 0;
    return ack $m - 1, 1 if $n == 0;
    return ack $m - 1, ack $m, $n-1;
}
```

La función Ackermann está definida para números enteros que no son negativos. Como un ejercicio adicional, modifica la subrutina `ack` para prevenir que argumentos negativos sean suministrados. Discutimos dos maneras diferentes de hacer esto en la Sección 5.9.

A.3.5 Palíndromos (Ejercicio 5.3)

Escribe una subrutina recursiva que chequea si una palabra es un palíndromo:

```
sub primera_letra( Str $palabra ){
    return substr $palabra, 0, 1;
}

sub última_letra( Str $palabra ){
    return substr $palabra, *-1, 1;
}

sub centro_letra( Str $palabra ){
    return substr $palabra, 1, *-1;
}
```

```

sub es-palindromo( Str $palabra ) {
    return True if $palabra.chars <= 1;
    return False if primera_letra($palabra) ne última_letra($palabra);
    return es-palindromo(centro_letra($palabra))
}
for ("bob", "otto", "laurent", "ana", "reconocer") -> $x {
    say "¿Es $x un palíndromo? Respuesta: ", es-palindromo($x);
}

```

Result:

```

¿Es bob un palíndromo? Respuesta: True
¿Es otto un palíndromo? Respuesta: True
¿Es laurent un palíndromo? Respuesta: False
¿Es ana un palíndromo? Respuesta: True
¿Es reconocer un palíndromo? Respuesta: True

```

El tercer parámetro (longitud) de la función integrada `substr` es opcional. En ese caso, `substr` devolverá todos los caracteres desde una posición dada. Así que la subrutina `primera_letra` podría simplificarse así:

```

sub primera_letra( Str $palabra where $palabra.chars >= 2 ){
    return substr $palabra, 0;
}

```

Y la subrutina `última_letra` podría beneficiarse de la misma simplificación.

Nota: la función integrada `flip` o el método `.flip` de Raku devuelve una versión inversa de una cadena de texto y proveería una solución mucho más fácil:

```

sub es-palindromo( Str $palabra ) {
    return $palabra eq $palabra.flip;
}

```

A.3.6 Potencias (Ejercicio 5.4)

Escribe una subrutina recursiva que chequea si un número es una potencia de otro número:

```

sub es-potencia-de (Int $a, Int $b) {
    return False unless $a %% $b;
    return True if $a == $b;
    return es-potencia-de Int($a/$b), $b;
}

say es-potencia-de 16, 4;
say es-potencia-de 25, 5;
say es-potencia-de 125, 5;
say es-potencia-de 600, 20;
say es-potencia-de 8000, 20;

```

Esto resulta en:

```
True
True
True
False
True
```

Al agregar una sentencia say para visualizar las llamadas recursivas, obtenemos:

```
sub es-potencia-de (Int $a, Int $b) {
  return False unless $a %% $b;
  return True if $a == $b;
  say "$a\t$b";
  return es-potencia-de Int($a/$b), $b;
}
```

Si ejecutamos a es-potencia-de con los argumentos 1024 y 2, obtenemos los valores de \$a y \$b a lo largo de las llamadas recursivas:

```
1024    2
512     2
256     2
128     2
64      2
32      2
16      2
8       2
4       2
True
```

A.3.7 Encontrando el MCD de Dos Números, Ejercicio 5.5 (p. 91)

Escribe una subrutina que devuelva el máximo común divisor de dos números:

```
sub mcd( Int $a, Int $b ) {
  return $a if $b == 0;
  return $b if $a == 0;
  return mcd($b, $a mod $b);
}
```

```
say mcd 125, 25;
say mcd 2048, 256;
say mcd 256, 4096;
say mcd 2048, 1;
say mcd 0, 256;
say mcd 33, 45;
```

Nota que hay un método más simple para encontrar el MCD de dos números sin usar la función módulo. Se conoce como el algoritmo de Euclides y es considerado el algoritmo más antiguo (ver https://es.wikipedia.org/wiki/Algoritmo_de_Euclides). El algoritmo de Euclides se basa en la observación de que el MCD de dos números no cambia si el número mayor es reemplazado con la diferencia de tal número y el número menor.

Esto podría implementarse en Raku con la siguiente subrutina recursiva:

```
sub mcd( Int $a, Int $b ) {
    return mcd($b, $a - $b) if $a > $b;
    return mcd($a, $b - $a) if $b > $a;
    return $a;
}
```

Este código funciona perfectamente bien en *casi* todos los casos, por lo menos para todos los valores de entrada estrictamente positivos, pero intenta seguir el flujo de ejecución si uno de los argumentos pasados a la subrutina, digamos \$b, es cero. En este caso, mcd entra en una recursión infinita. Esto usualmente se conoce como un *edge case* o un *corner case*, es decir, un valor de entrada especial para el cual un programa aparentemente funcional cesa de funcionar apropiadamente.

Tenemos un problema similar para los valores de entrada negativos.

Una solución podría ser agregar una restricción de signatura (o usar un subconjunto de tipo):

```
sub mcd( Int $a where $a > 0, Int $b where $b > 0 ) {
    ...
}
```

pero esto no es realmente satisfactorio porque el MCD de cualquier entero que no es cero y 0 está matemáticamente bien definido y es igual al primer número.

Dejando al lado por un momento el caso de los números negativos, podrías escribir la subrutina nuevamente en la siguiente manera:

```
sub mcd( Int $a, Int $b ) {
    return $a if $b == 0;
    return $b if $a == 0;
    return gcd($b, $a - $b) if $a > $b;
    return gcd($a, $b - $a) if $b > $a;
    return $a;
}
```

Concerniente a los números negativos, hay un teorema que dice que el MCD de a y b es el mismo que el MCD de a y $-b$:

$$\text{mcd}(a, b) = \text{mcd}(-a, b) = \text{mcd}(a, -b) = \text{mcd}(-a, -b)$$

Podemos modificar la subrutina mcd aún más:

```
sub mcd( Int $a is copy, Int $b is copy ) {
    $a = -$a if $a < 0;
```

```

    $b = -$b if $b < 0;
    return $a if $b == 0;
    return $b if $a == 0;
    return mcd($b, $a - $b) if $a > $b;
    return mcd($a, $b - $a) if $b > $a;
    return $a;
}

```

Esto funciona bien ahora, pero recuerda que una subrutina recursiva puede ser llamada muchas veces y, por cada llamada, las primeras cuatro líneas de código en el programa más arriba son ejecutadas, aunque son realmente útiles solo en la primera llamada: una vez que estas condiciones han sido chequeadas durante la primera llamada de la subrutina, sabemos que los argumentos deben ser válidos y permanecer válidos en la cadena de llamadas recursivas, así que estos chequeos después de la primera llamada no son necesarios. Esto es un despilfarro y puede conducir a problemas innecesarios de rendimiento.

Idealmente, podría ser mejor separar estas cuatro líneas que chequea las precondiciones de las cascada de llamadas recursivas. Por ejemplo, podríamos escribir dos subrutinas:

```

sub mcd1( Int $c, Int $d ) {
    return mcd1($d, $c - $d) if $c > $d;
    return mcd1($c, $d - $c) if $d > $c;
    return $c;
}

sub mcd( Int $a is copy, Int $b is copy ) {
    $a = -$a if $a < 0;
    $b = -$b if $b < 0;
    return $a if $b == 0;
    return $b if $a == 0;
    return mcd1 $a, $b;
}

```

Ahora, `mcd` está haciendo todos los chequeos necesarios de los argumentos iniciales y llama subrutina recursiva `mcd1` con los argumentos que han sido higienizados y que no conducirán a una recursión infinita. Nota que hemos nombrados los parámetros dentro de `mcd1` diferentemente para mejor claridad, pero esto no era necesario; funcionaría igual si hubiésemos mantenido a `$a` y `$b`.

El código anterior trabaja perfectamente bien.

Aunque podría haber un último problema. Alguien descuidado (o queriendo ser muy astuto) podría tratar de llamar a `mcd1` directamente, y así destruyendo los beneficios de los chequeos realizados por `mcd`. Para prevenir eso, podemos hacer buen uso del hecho de que las subrutinas tiene ámbito léxico en Raku y pueden ser locales a otras subrutinas: podemos declarar y definir `mcd1` dentro del cuerpo de la subrutina `mcd`, para que `mcd1` pueda ser llamada solo desde el interior de la subrutina `mcd`:

```

sub mcd( Int $a is copy, Int $b is copy ) {
    sub mcd1($c, $d) {
        return mcd1($d, $c - $d) if $c > $d;
        return mcd1($c, $d - $c) if $d > $c;
    }
}

```

```

        return $c;
    }
    $a = -$a if $a < 0;
    $b = -$b if $b < 0;
    return $a if $b == 0;
    return $b if $a == 0;
    return mcd1 $a, $b;
}

say mcd 125, 25;      # 25
say mcd 2048, 256;    # 256
say mcd 256, 4096;    # 256
say mcd 2048, 1;      # 1
say mcd 0, 256;       # 256
say mcd 33, 45;       # 3
say mcd -4, 6;        # 2

```

El Capítulo 6 tendrá una discusión acerca del ámbito léxico.

Podrías estar interesada/o en saber que existe una función integrada en Raku que encuentra el MCD de dos números. Es la función `gcd`.

A.4 Ejercicios del Capítulo 6: Iteración

A.4.1 Ejercicio 6.1: Raíz Cuadrada (p. 105)

Necesitamos una subrutina que encuentra la raíz cuadrada de un número al computar sucesivamente las mejores aproximaciones de la raíz, usando el método de Newton.

Para este ejercicio, he hecho las siguientes decisiones aunque algo arbitrarias:

- He elegido un valor *epsilon* de 10^{-11} (o $1e-11$).
- He usado $\$a/2$ como un estimado inicial de $\sqrt{\$a}$.

Nota que podría tener más sentido declarar este estimado inicial dentro de la subrutina `mi-raíz-cuad`, en lugar de tener que pasarlo como un argumento. La justificación para hacer esto es que, en algunos casos, la subrutina que hace la llamada tiene información sobre el rango del valor de entrada y podría por lo tanto ser capaz de proveer un mejor estimado inicial, lo cual conduce el algoritmo a converger hacia la solución mucho más rápido.

Esta es una implementación del método de Newton para computar la raíz cuadrada de un número:

```

sub mi-raíz-cuad( $a, $estimado is copy ) {
    my $epsilon = 1e-11;
    while True {
        # say "-- Valor intermedio: $estimado";
        my $y = ($estimado + $a/$estimado) / 2;
        last if abs($y - $estimado) < $epsilon;
    }
}

```

```

        $estimado = $y;
    }
    return $estimado;
}

sub imprimir-result( $a, $r, $s, $d ) {
    printf "%d %.13f %.13f %.6e \n", $a, $r, $s, $d;
}

sub probar-raíz-cuadrada {
    say "a mi-raíz-cuad(a)\t sqrt(a)\t difer";
    for 1..9 -> $a {
        my $estimado-ini = $a/2;
        my $result = mi-raíz-cuad $a, $estimado-ini;
        my $sqrt = sqrt $a;
        my $difer = abs($result - $sqrt);
        imprimir-result($a, $result, $sqrt, $difer);
    }
}

probar-raíz-cuadrada;

```

La función `printf` ("formatted print") usada en `imprimir-result` es derivada del lenguaje de programación C. Su primer argumento es *formato de cadena de texto*, que describe cómo cada uno de los siguientes argumentos debería ser formateado. Aquí, el formato de cadena de texto requiere al compilador a producir el primer argumento subsecuente como un entero con signo (la parte `%d` formato de cadena de texto), los dos argumentos siguiente como números de coma flotante con 13 números después del punto decimal (la parte `%.13f`), y el último argumento como un número de coma flotante en notación científica con 6 dígitos después del punto decimal (`%.6e`).

A.4.2 Ejercicio 6.2: Estimado de Pi (p. 105)

El estimado de Pi de acuerdo al algoritmo de Srinivasa Ramanujan:

```

sub factorial( Int $n ) {
    return 1 if $n == 0;
    return $n * factorial $n-1;
}

sub pi-estimado {
    #`{ =====
        Algoritmo por Srinivasa Ramanujan
        (ver http://en.wikipedia.org/wiki/Pi)
        ===== }
    my $factor = 2 * 2.sqrt / 9801;
    my $k = 0;
    my $suma = 0;
    while True {
        my $num = factorial(4*$k) * (1103 + 26390*$k);
    }
}

```

```

        my $den = factorial($k)**4 * 396**(4*$k);
        my $term += $factor * $num / $den;
        # say "Término intermedio = $term";
        last if abs($term) < 1e-15;
        $suma += $term;
        $k++;
    }
    return 1 / $suma;
}

say pi-estimado;
# say pi - pi-estimado;

```

Esto imprime: 3.14159265358979.

Nota como hemos usado el comentario multilínea para proveer información adicional sobre la subrutina.

Eliminar el comentario de la sentencia de impresión muestra los pasos hacia la solución:

```

Término intermedio = 0.31830987844047
Término intermedio = 7.74332048352151e-009
Término intermedio = 6.47985705171744e-017
-4.44089209850063e-016

```

A.5 Ejercicios del Capítulo 7: Cadenas de Texto

A.5.1 Ejercicio en la Sección 7.3: Recorrido de una Cadena de Texto (p. 113)

El recorrido inverso de una palabra con un bucle `while` puede escribirse así:

```

my $fruta = "banana";
my $indice = $fruta.chars;
while $indice > 0 {
    $indice--;
    my $letra = substr $fruta, $indice, 1;
    say $letra;
}

```

El método `chars` devuelve la longitud de la cadena de texto. La función `substr` encontrará las letras debajo de `$indice` que se encuentran entre 0 y longitud - 1. Es por lo tanto práctico disminuir la variable `$indice` antes de usar la función `substr`.

El bucle `while` del código anterior puede escribirse más forma más concisa:

```

my $fruta = "banana";
my $indice = $fruta.chars;
while $indice > 0 {
    say substr $fruta, --$indice, 1;
}

```


Aquí, imprimimos el valor devuelto por `substr` directamente, sin usar una variable temporaria, y disminuimos la variable `$indice` dentro de la expresión usando una `substr`. Necesitamos usar la forma prefija del operador de decremento porque necesitamos que `$indice` sea reducido antes de ser utilizado por `substr`.

El bucle sería aún más conciso si usáramos un `while` con un modificador de sentencia (o la sintaxis sufija de `while`):

```
my $fruta = "banana";
my $indice = $fruta.chars;
say substr $fruta, --$indice, 1 while $indice;
```

Esta es la misma idea, usando la función `flip` para invertir la cadena de texto:

```
my $fruta_inv = flip "banana";
my $indice = 0;
say substr $fruta_inv, $indice++, 1 while $indice < $fruta_inv.chars;
```

El objetivo de este ejercicio es entrenarte a usar bucles para recorrer la cadena de texto. Combinar las funciones o métodos `flip` y `comb` haría nuestra solución mucho más simple (y probablemente más rápida):

```
.say for "banana".flip.comb;
```

A.5.2 Ejercicio en la Sección 7.3: The Ducklings (p. 113)

La primera idea que viene a la mente para este ejercicio es construir una lista modificada de prefijos de este manera:

```
for 'J' .. 'N', 'Ou', 'P', 'Qu' -> $letra { #...}
```

Pero esto no funciona apropiadamente porque crea una lista de cuatro elementos en la cual el primer elemento es una sub-lista desde “J” hasta “N”:

```
> say ('J'..'N', 'Ou', 'P', 'Qu').raku;
("J".."N", "Ou", "P", "Qu")
```

Regresaremos a esto más tarde en el libro, pero digamos que necesitamos aplanar esta combinación de listas en una sola lista iterable, lo cual puede hacerse con el método o la función `flat` o el operador “|”:

```
for ('J' .. 'N', 'Ou', 'P', 'Qu').flat -> $letra {#...}
# o: for flat 'J' .. 'N', 'Ou', 'P', 'Qu' -> $letra {...}
# o: for |('J' .. 'N'), 'Ou', 'P', 'Qu' -> $letra {...}
# Nota: los paréntesis en el último ejemplo son necesarios
# para resolver el problema de precedencia
```

Con esta pequeña dificultad removida, la solución es ahora fácil:

```
my $sufijo = 'ack';
for ('J' .. 'N', 'Ou', 'P', 'Qu').flat -> $letra {
  say $letra ~ $sufijo;
}
```

Aquí nuevamente, podríamos hacer el código un poco más conciso con la sintaxis sufija de `for` y la variable tónica `$_`:

```
my $sufijo = 'ack';
say "$_ $sufijo" for flat 'J' .. 'N', 'Ou', 'P', 'Qu';
```

Aquí introducimos otra manera simple y común de concatenar dos cadenas de texto: simplemente inserta las dos variables una después de la otra dentro de comillas dobles y permite que la interpolación de variables haga su trabajo.

A.5.3 Ejercicio en la Sección 7.3: Contando las Letras de una Cadena de Texto (p. 113)

Esta subrutina cuenta el número de ocurrencias de una letra específica dentro de una palabra (o cualquier otra cadena de texto):

```
sub contar( Str $palabra, Str $letra ) {
    my $cuenta = 0;
    for $palabra.comb -> $letra {
        $cuenta++ if $letra eq 'a';
    }
    return $cuenta;
}
say contar "banana", "a";    # -> 3
```

La solución al Ejercicio 7.1 (p. 133) más abajo usa las funciones `index` y `substr` para realizar la misma cuenta.

A.5.4 Sección 7.5: Simulando un Regex con un Bloque (p. 115)

El objetivo es encontrar en una cadena de texto cualquier letra que es inmediatamente seguida por la letra “l” y seguida por la letra “w”.

Si intentas hacer la búsqueda especificada con las técnicas que hemos visto hasta ahora, encontrarás que hay un número de casos especiales que hacen la tarea complicada.

Esto es una solución posible:

```
sub recorrer( Str $palabra, Str $inicio_letra, Str $final_letra ) {
    my $encontrada_inicio = False;
    my $captura_siguiente = False;
    my $letra_objetivo;
    for 0..$palabra.chars - 1 -> $idx {
        my $letra = substr $palabra, $idx, 1;
        next unless $letra eq $inicio_letra or $encontrada_inicio;
        if ($captura_siguiente) {
            $letra_objetivo = $letra;
            $captura_siguiente = False;
            next;
        }
    }
}
```

```

    if $letra eq $inicio_letra and not $encontrada_inicio {
        $encontrada_inicio = True;
        $captura_siguiente = True;
        next;
    }
    # si llegamos hasta aquí, hemos encontrado una letra objetivo
    # candidata
    if $letra eq $final_letra {
        return $letra_objetivo
    } else {
        # coincidencia errónea, iniciemos nuevamente,
        # necesitamos retroceder
        if $letra_objetivo eq $inicio_letra {
            $letra_objetivo = $letra;
            $captura_siguiente = False;
        } elsif $letra eq $inicio_letra {
            $captura_siguiente = True;
        } else {
            $captura_siguiente = False;
            $encontrada_inicio = False;
        }
    }
}
return; # ¡no encontrada!
}

for <s b l w l o s m y l a z> -> $ini, $fin {
    say "$ini $fin: ", recorrer "yellow submarine", $ini, $fin;
}

```

Como puedes ver, esto es algo un poco complicado porque los casos especiales que necesitamos manejar. Compara esto con un regex de una sola línea que hace lo mismo:

```
say ~$0 if "yellow submarine" ~~ /l(.)w/;
```

Para ser honesto, no he elegido la forma más simple de hacerlo.

Es más mucho más fácil de pasar a través de cada letra de la cadena de texto excepto la primera y la última, para cada letra como tal, para chequear cuales son la letra previa y la letra siguiente. Después simplemente necesitas devolver la letra actual si la previa y la siguiente coincide con las condiciones:

```

sub recorrer( Str $palabra, Str $inicio_letra, Str $final_letra ) {
    my $encontrada_inicio = False;
    my $captura_siguiente = False;
    my $letra_objetivo;
    for 1..$palabra.chars - 2 -> $idx {
        if $inicio_letra eq substr $palabra, $idx - 1, 1
            and $final_letra eq substr $palabra, $idx + 1, 1 {
            return substr $palabra, $idx, 1;
        }
    }
}

```

```

    }
    return; # no encontrada!
}

for <s b l w l o s m y l a z> -> $ini, $fin {
    say "$ini $fin: ", recorrer "yellow submarine", $ini, $fin;
}

```

En las pruebas de casos al final, uso bucle `for` con una construcción de bloque puntiagudo en la cual elijo *dos* de los artículos en la lista cada vez a través del bucle. Los números de espacios entre los artículos de la lista son técnicamente inservibles e irrelevante a la manera en la cual la construcción sintáctica funciona; ellos solo sirven para ayudar al lector a ver mejor cómo las letras serán agrupadas en el proceso.

Esto muestra:

```

s b: u
l w: o
l o: l
s m: Nil
y l: e
a z: Nil

```

Esto es mucho más simple que el intento anterior, pero sería aún un poco difícil de cambiar algo, por ejemplo agregar una nueva condición: la estructura del código necesitaría ser modificado un poco.

Aún comparada con esta solución más simple, la solución de regex realmente brilla en orden de magnitud.

A.5.5 Ejercicios en la Subsección 7.7.8: Ejercicios de Regexes (p. 124)

Como es usualmente el caso en Raku, y aún más con los regexes, hay más de una manera de hacer algo (TIMTOWTDI). La mayoría de los ejercicios sugeridos aquí tienen más de una solución (y algunas veces muchas).

Con los regexes, también tienes que pensar cuidadosamente acerca de los datos de entrada para saber lo que debe coincidir y lo que debe ser rechazado.

A.5.5.1 Diez dígitos en una fila

Aquí presentamos una manera de encontrar 10 dígitos consecutivos en una cadena de texto:

```

my $cadena = "567867 8778689 6765432 0123456789 897898";
say ~$0 if $cadena ~~ /\d ** 10/; # -> 0123456789

```

Simplemente estamos usando la categoría de caracteres `\d` (dígito) junto con un cuantificador que especifica esta clase 10 veces.

Nota que hemos usado los paréntesis de captura aquí para popular el número que coincidió en `$0`. Podríamos también omitir los paréntesis y extraer el número desde el objeto de coincidencia:

```
my $cadena = "567867 8778689 6765432 0123456789 897898";
say ~$/ if $cadena ~~ /\d ** 10/; # -> 0123456789
```

La solución anterior solo haría una coincidencia con una secuencia de 10-dígitos dentro de una secuencia más larga, lo cual puede o no puede ser lo que necesitas. Por ejemplo:

```
my $cadena = "567867 87786896765432 0123456789 897898";
say ~$0 if $cadena ~~ /(\d ** 10)/; # -> 8778689676
```

Si quieres coincidir más precisamente con una secuencia de 10-dígitos (no más de 10), necesitas especificar lo quieres tener alrededor de la coincidencia. Por ejemplo, para coincidir solo con la secuencia de 10-dígitos más arriba, podrías usar la categoría de caracteres de no dígitos:

```
my $cadena = "567867 87786896765432 0123456789 897898";
say ~$0 if $cadena ~~ /\D (\d ** 10) \D/; # -> 0123456789
```

Pero esto coincidiría con una secuencia de 10-dígitos al inicio o al final de la cadena de texto:

```
my $cadena = "5678670001 87786896765432 0123456789 897898";
say ~$0 if $cadena ~~ /\D (\d ** 10) \D/; # -> 0123456789
```

Un mejor solución podría usar anclas de borde de palabra:

```
my $cadena = "5678670001 87786896765432 0123456789 897898";
say ~$0 if $cadena ~~ /<< (\d ** 10) >>/; # -> 5678670001
```

Un poco de reflexión puede algunas veces ser necesario para asegurar que hacemos una coincidencia con lo que realmente queremos.

A.5.5.2 Un número octal

Esta es una solución posible para encontrar un número octal (es decir, un número compuesto solamente de dígitos del 0 al 7) en una cadena de texto:

```
my $cadena = "567867 8778689 6765432 0123456789 897898";
say ~$0 if $cadena ~~ /\D (<[0..7]>+) \D/; # -> 6765432
```

La categoría de caracteres es `<[0..7]>` para los dígitos entre 0 y 7. El cuantificador `+` significa: tantas categorías de caracteres como sea posible. Y los `\D` (no dígito) se encuentran ahí para prevenir el regex de coincidir parte de un número más largo que tiene dígitos que no forman parte de un número octal (por ejemplo coincidir 567 en el primer número). Dependiendo del requerimiento exacto, usar las anclas de borde de palabra como en la solución del ejercicio previo sería mejor.

A.5.5.3 Primera palabra al inicio de la cadena de texto

Para encontrar la primera palabra en una cadena de texto, podemos solo buscar la primera secuencia de caracteres de palabra (caracteres que pertenecen a la categoría de caracteres `\w`) en la cadena de texto:

```
my $cadena = "The greatest thing you'll ever learn " ~
    "is just to love and be loved in return. " ~
    "(Nature Boy, Nat King Cole)";
say ~$0 if $cadena ~~ /(\w +)/; # -> The
```

A.5.5.4 Primera palabra que comienza con una “a”

Esta es una manera de encontrar la primera palabra que comienza con la letra “a” en una sentencia:

```
my $cadena = "Four score and seven years ago our fathers ...";
say ~$0 if $cadena ~~ /\W (a \w+)/; # -> and
```

A.5.5.5 Primera palabra que comienza con una vocal minúscula

Para asegurarnos que la coincidencia no comience con una vocal en el medio de una palabra, podríamos comenzar el patrón con un \W (carácter de no palabra) o, mejor, con un borde izquierdo de palabra <<:

```
my $cadena = "Democracy is the worst form of government, " ~
    "except for all the others. (Churchill)";
say ~$0 if $cadena ~~ /<< (<[aeiouy]> \w*)/; # -> is
```

Aquí usamos un cuantificador * (en lugar de +) porque solo una palabra que contiene una vocal es eligible como una palabra que comienza con una vocal.

A.5.5.6 Un número telefónico

Para un número de 10-dígitos que comienza con “06” o “07”, la solución más fácil es probablemente usar una categoría de caracteres <[67]>:

```
my $cadena = "567867 8778689 0123456789 0723456789 3644";
say ~$0 if $cadena ~~ /(0<[67]>\d ** 8)/; # -> 0723456789
```

A.5.5.7 Primera palabra que comienza con una vocal (minúscula o mayúscula)

Podemos simplemente ignorar la distinción entre mayúscula y minúscula para la palabra completa:

```
my $cadena = " Ask not what your country can do for you - " ~
    " ask what you can do for your country. (JFK)";
say ~$0 if $cadena ~~ /:i << (<[aeiouy]> \w*)/; # -> Ask
```

A.5.5.8 Letras repetidas

Podemos capturar cualquier letra y chequear si la siguiente es la misma como la capturada:

```
say ~$0 if 'appeal' ~~ /((\w)$0)/; # -> pp
```

Para capturar el segundo grupo de letras repetidas:

```
say ~$1 if 'coffee' ~~ /(\w)$0.*((\w)$0)/; # -> ee
```

Y para el tercer grupo:

```
say ~$2 if 'Mississippi' ~~ /(\w)$0.*((\w)$0).*((\w)$0)/; # -> pp
```

o posiblemente:

```
say ~$2 if 'Mississippi' ~~ /(\w)$0.*(\w)$1.*((\w)$0)/; # -> pp
```

A.5.6 Ejercicio en la Sección 7.10: Subrutina es-inversa (p. 132)

El segundo error en la subrutina es-inversa está localizado en esta línea:

```
while $j > 0 {
```

El índice \$j debería ser capaz de repetir hasta llegar a 0 (inclusivo) si queremos comparar la primera letra de \$palabra2 con la última letra de \$palabra1.

La versión corregida de la subrutina es-inversa podría ser:

```
sub es-inversa( Str $palabra1, Str $palabra2 ) {
    return False if $palabra1.chars != $palabra2.chars;

    my $i = 0;
    my $j = $palabra2.chars - 1;

    while $j >= 0 {
        return False if substr($palabra1, $i, 1) ne substr($palabra2, $j, 1);
        $i++; $j--;
    }
    return True;
}
```

A.5.7 Ejercicio 7.1: Contando Letras (p. 133)

Contar el número de letras “a” en una palabra con la función index implica buscar la primera “a” desde el inicio de la cadena de texto, después buscar por la siguiente desde la posición inmediatamente después, y continuar así hasta que no se encuentren más letras “a”.

Aquí, creamos un bucle infinito desde el cual nos liberamos con la sentencia last cuando index ya no encuentra una “a”. El contador \$cuenta es incrementado cada vez que una “a” es encontrada, y la \$idx mantiene un registro de la posición actual dentro de la cadena de texto:

```
sub contar_a {
    my $palabra = "banana";
    my $cuenta = 0;
    my $idx = 0;
    while True {
        $idx = index $palabra, 'a', $idx;
        last unless $idx.defined;
        $idx++;
        $cuenta++;
    }
    return $cuenta;
}

say contar_a(); # -> 3
```

Si la adaptamos para cualquier cadena de texto y cualquier letra es solo una cuestión de pasar los argumentos correctos a la subrutina y usar dentro de la subrutina sus parámetros en lugar de valores literales:

```

sub contar_indice( Str $palabra, Str $letra ) {
    my $cuenta = 0;
    my $idx = 0;
    while True {
        $idx = index $palabra, $letra, $idx;
        last unless $idx.defined;
        $idx++;
        $cuenta++;
    }
    return $cuenta;
}
say contar_indice "When in the Course of human events...", "n"; # 4

```

Contar una letra dada en una palabra dada con la función `substr` es algo directo: solo necesitamos pasar a través de cada letra de la palabra e incrementar un contador cuando sea necesario:

```

sub contar_substr( Str $palabra, Str $letra ) {
    my $cuenta = 0;
    for 0..$palabra.chars - 1 {
        $cuenta++ if $letra eq substr $palabra, $_, 1;
    }
    return $cuenta;
}
say contar_substr "I have a dream that one day...", "a"; # -> 5

```

A.5.8 Ejercicio 7.2: Letras Minúsculas (p. 133)

Solo las subrutinas `cualquier_minúscula5` y `cualquier_minúscula7` chequean correctamente si la cadena de entrada contiene por lo menos una letra minúscula.

Si no lo determinaste por ti mismo, trata realmente de encontrar por ti mismo los errores en las otras antes de proseguir; deberías ser capaz de encontrar los errores en las otras subrutinas (excepto quizás `cualquier_minúscula4`, lo cual es sin duda un poco engañoso).

La subrutinas `cualquier_minúscula5` y `cualquier_minúscula7` realizan la búsqueda como sigue:

- `cualquier_minúscula5` asigna `False` a `$bandera` antes del bucle, lo cambia a `True` si cualquier carácter en la cadena de texto está en minúscula, y devuelve `$bandera` después de completar el bucle.
- `cualquier_minúscula7` es también correcta (y probablemente un poco mejor que `cualquier_minúscula5`). Devuelve `True` si cualquier carácter está en minúscula y devuelve `False` solo si se puede llegar al final del bucle.

Las otras subrutinas tienen los errores siguientes (algunas tienen varios errores; vamos a mencionar por lo menos uno de ellos):

- `cualquier_minúscula1` solo chequea el primer carácter de su argumento y sale del bucle inmediatamente después.

- `cualquier_minuscula2` llama la subrutina `es-minuscula` sobre la cadena de texto `"char"`, no sobre la variable `$char` (tiene el mismo defecto que `cualquier_minuscula1`).
- `cualquier_minuscula3` devuelve `True` o `False` dependiendo solo en el último carácter de la cadena de texto de entrada.
- `cualquier_minuscula4` sufre del problema de la precedencia de operadores: la asignación `$bandera = $bandera ...` se ejecuta antes que el operador de relación `or` sea ejecutado, así que la última parte no tiene efecto. Cambiando la línea defectuosa:

```
$bandera = $bandera || es-minuscula $char; # operador de precedencia más alta
# o
$bandera = ($bandera or es-minuscula $char); # los paréntesis anulan la precedencia
```

solucionaría el problema.

- `cualquier_minuscula6` es casi correcta en términos de su algoritmo, pero devuelve las cadenas `"True"` o `"False"` en lugar de los valores Booleanos `True` o `False`.
- `cualquier_minuscula8` devuelve `False` si el carácter no está en minúscula.
- `cualquier_minuscula9` también devuelve `False` si cualquier carácter no está en minúscula.

El siguiente es un ejemplo del bucle que podrías escribir para probar cada subrutina, cada una con tres cadenas de texto de entrada:

```
for <FOO bar Baz> -> $str {
  say "1. $str: ", cualquier_minuscula1 $str;
  say "2. $str: ", cualquier_minuscula2 $str;
  say "3. $str: ", cualquier_minuscula3 $str;
  say "4. $str: ", cualquier_minuscula4 $str;
  say "5. $str: ", cualquier_minuscula5 $str;
  say "6. $str: ", cualquier_minuscula6 $str;
  say "7. $str: ", cualquier_minuscula7 $str;
  say "8. $str: ", cualquier_minuscula8 $str;
  say "9. $str: ", cualquier_minuscula9 $str;
}
```

Sería posible reemplazar las nueve sentencias de impresión con un bucle simple, pero requiere usar características que no hemos estudiado todavía.

En otros capítulos verás maneras de organizar pruebas de casos, por ejemplo en la Sección 14.8 (p. p. 318).

A.5.9 Ejercicio 7.3: Cifrado César (p. 135)

Podemos implementar un cifrado de rotación de una letra:

```

sub rotar-una-letra( Str $letra, Int $despl ) {
    my $cota-sup = 'Z'.ord;      # última letra mayúscula
    my $cota-inf  = 'z'.ord;      # última letra minúscula

    my $ord-rotada = $letra.ord + $despl;
    if $letra ~~ /<[a..z]>/ {      # minúscula
        $ord-rotada -= 26 if $ord-rotada > $cota-inf;
    } elsif $letra ~~ /<[A..Z]>/ { # mayúscula
        $ord-rotada -= 26 if $ord-rotada > $cota-sup;
    } else {
        return $letra;
    }
    return $ord-rotada.chr;
}

sub rotar-palabra( Str $palabra, Int $despl is copy ) {
    $despl = $despl % 26;
    $despl = 26 + $despl if $despl < 0;
    my $palabra-rotada = "";
    for 0..$palabra.chars - 1 {
        $palabra-rotada ~= rotar-una-letra(
            substr($palabra, $_, 1),
            $despl
        );
    }
    return $palabra-rotada;
}

sub rot13( Str $palabra ) {
    return rotar-palabra $palabra, 13;
}

say rotar-palabra "ABDCabcd", 25;
say rotar-palabra "cheer", 7;
say rotar-palabra "melon", -10;

say rot13("Fbzr cebsnavgl");

```

Si estás interesado en decodificar solo ROT13, el operador de transliteración `tr` puede ofrecer un código mucho más corto. Por ejemplo, `tr/a..m/n..z/` transcribirá todas las letras en el rango `a..m` en sus equivalentes respectivos en el rango `n..z`.

Podemos escribir un ROT13 en un one-liner de Raku (ver Sección 2.5):

```

$ raku -e 'my $w = "foobar"; $w ~~ tr/a..mn..z/n..za..m/; say $w;'
sbbone

$ raku -e 'my $w = "sbbone"; $w ~~ tr/a..mn..z/n..za..m/; say $w;'
foobar

```

Es muy fácil agregar rango para las letras mayúsculas. Podrías hacerlo como un ejercicio adicional.

A.6 Ejercicios del Capítulo 8: Juego de Palabras

A.6.1 Ejercicio 8.7: Letras Dobles Consecutivas (p. 147)

Con la técnica de repetición usada en el Capítulo 8, podríamos escribir esto:

```
sub es_doble_triple( Str $palabra ) {
    # Examina si una palabra contiene tres pares de letras
    # consecutivas
    my $i = 0;
    my $cuenta = 0;
    while $i < $palabra.chars - 1 {
        if substr($palabra, $i, 1) eq substr($palabra, $i + 1, 1) {
            $cuenta++;
            return True if $cuenta == 3;
            $i += 2;
        } else {
            $cuenta = 0;
            $i++;
        }
    }
    return False;
}

for 'lemario.txt'.IO.lines -> $palabra {
    say $palabra if es_doble_triple $palabra;
}
```

Sin embargo, este es un caso típico donde los regexes pueden resultar más eficiente que un bucle (en términos de eficiencia de código, o sea, como un aproximación, cuántas líneas de código son necesarias para realizar una tarea dada).

Discutimos en Capítulo 7 que las capturas de regex poblan las variables especiales \$0, \$1, \$2, etc. Un patrón de regex que coincide con cualquier letra repetida podría por lo tanto ser `/(.) $0/`, donde el carácter encontrado por \$0 es el mismo carácter encontrado por el punto.

Similarmente, un patrón de regex que coincide con tres pares de letras dobles en una fila podría implementarse así:

```
say ~$/ if "abbccdde" ~~ /(.)$0 (.)$1 (.)$2/; # -> bbccdd
```

Con esto, el programa para encontrar las palabras con tres pares de letras dobles en el archivo *lemario.txt* puede escribirse en tres líneas de código:

```
for 'lemario.txt'.IO.lines -> $palabra {
    say $palabra if $palabra ~~ /(.) $0 (.) $1 (.) $2/;
}
```

Nota del traductor: Aunque el archivo *lemario.txt* tiene alrededor de 88,000 palabras, fui incapaz de encontrar una palabra (en español) que contenga tres pares de letras consecutivas. No obstante, hay palabras que contienen dos pares de letras consecutivas como *toffee*, *hemorro*, etc.

Debido a esto, el siguiente ejemplo usa el archivo *words.txt* el cual es utilizado en *Think Raku*¹.

Si usamos el archivo *words.txt*, ambos programas encuentran cuatro palabras, las cuales son variaciones de “bookkeeper” y “bookkeeping”.

La versión de regex es tan simple que puedes escribirla directamente en la línea de comando del sistema operativo como un one-liner: (ver Sección 2.5):

```
$ raku -ne '.say if /(.) $0 (.) $1 (.) $2/' words.txt
bookkeeper
bookkeepers
bookkeeping
bookkeepings
```

A.6.2 Ejercicio 8.8: Palíndromos en los Odómetros (p. 147)

La siguiente es un posible programa para solucionar el acertijo sobre el odómetro con palíndromos:

```
sub es-palindromo( $num, $ini, $long ) {
    # chequea si la sub-cadena relevante es un palíndromo
    my $subcad = substr $num, $ini, $long;
    return $subcad eq flip $subcad;
}

sub chequear( $num ) {
    # Chequea si el número entero tiene las
    # propiedades indicadas
    return (es-palindromo($num, 2, 4) and
            es-palindromo($num + 1, 1, 5) and
            es-palindromo($num + 2, 1, 4) and
            es-palindromo($num + 3, 0, 6));
}

say 'Las siguientes son las lecturas de odómetro posibles: ';
for 1e5..1e6 - 4 -> $número {
    say $número if chequear $número;
}
```

Otra forma de hacerlo sería con el uso de regexes para encontrar si tenemos palíndromos:

```
sub chequear( $num ) {
    # Chequea si el número entero tiene las
    # propiedades indicadas
    $num ~~ /^..(..)$1$0/ and
    $num + 1 ~~ /^..(..)$1$0/ and
    $num + 2 ~~ /^..(..)$1$0/ and
```

¹Puedes encontrar el archivo *words.txt* aquí:

https://raw.githubusercontent.com/LaurentRosenfeld/think_raku/master/Supplementary/words.txt

```

    $num + 3 ~~ /^(.)(.)(.)$2$1$0/;
}

say 'Las siguientes son las lecturas de odómetro posibles:';
for 1e5..1e6 - 4 -> $numero {
    say $numero if chequear $numero;
}

```

Este código es más corto, pero también más lento: toma casi el doble para ejecutarse en mi computadora. Así que hay un compromiso: La primera manera, la cual es más rápida, es probablemente mejor si necesitas ejecutar tu programa muchas veces o usualmente, pero puedes preferir la segunda versión si esto es una computación que haces raramente. La decisión es tuya.

A.6.3 Ejercicio 8.9: Palíndromos en las Edades (p. 148)

El siguiente programa itera sobre las posibles diferencias de edades entre 15 y 75 y, por cada edad, calcula todas las posibilidades de palíndromos.

```

say 'diferencias #instancias';
chequear_dif();
say 'hija madre';
instancia_num(18, True);

sub son_inversas( Int $i, Int $j ) {
    # $j (edad de la madre) tendrá siempre dos dígitos
    return $j eq flip sprintf '%02d', $i; # formatea $i con 2 dígitos
}

sub instancia_num( Int $dif, Bool $bandera ) {
    # computa y cuenta todas las posibilidades
    # para una diferencia de edad
    my $hija = 0;
    my $cuenta = 0;
    while True {
        my $madre = $hija + $dif;
        if son_inversas($hija, $madre) or
            son_inversas($hija, $madre+1) {
            $cuenta++;
            printf "%02d\t%d\n", $hija, $madre if $bandera;
        }
        last if $madre > 99;
        $hija++;
    }
    return $cuenta;
}

sub chequear_dif() {
    # enumera todas las posibles diferencias de edade
    for 15..75 -> $dif {

```

```

        my $nb_casos = instancia_num $dif, False;
        say "$dif    $nb_casos" if $nb_casos > 0;
    }
}

```

La sentencia `while True` crea un bucle infinito. El bucle es terminado por la sentencia de flujo de control, `last`, cuando la edad de la madre excede 99. Veremos en la Sección 9.7 una manera más idiomática de construir un bucle infinito, pero esto es suficiente por ahora.

La función `sprintf` usada en este ejercicio transforma cualquier número menor que 10 en una cadena de texto de dos dígitos con un 0 en frente. Su sintaxis es similar a aquella de la función `printf` que vimos anteriormente. La diferencia es que solo crea una cadena de texto nueva, pero no la imprime.

Usar el método `.fmt` en lugar de la función `sprintf`, como la sintaxis de método de `flip`, puede hacer que la subrutina `son_inversas` luzca algo mejor:

```

sub son_inversas( Int $i, Int $j ) {
    return $j eq $i.fmt('%02d').flip; # formatea $i con 2 dígitos
}

```

A.7 Ejercicios del Capítulo 9: Arrays y Listas

A.7.1 Ejercicio en la Sección 9.4: Implementando una Cola (Queue) (p. 156)

Esta es una implementación algo simple de una cola (o *queue*) que usa un array y las funciones `unshift` y `pop`:

```

# Añadir un elemento a la línea
# (o sea, última persona en unirse a la línea)
sub encolar( @cola, $nuevo_artículo ) {
    unshift @cola, $nuevo_artículo;
}

# Remover un elemento de la línea
#(i.e, persona en frente de la línea)
sub desencolar( @cola ) {
    my $artículo = pop @cola;
    return $artículo;
}

my @linea = 1, 2, 3, 4, 5;
encolar @linea, 6;
say @linea;
say desencolar @linea for 1..3;

```

A.7.1.1 Mejorando la cola con firmas de subrutinas

Intentemos mejorar nuestra cola y hacerla un poco más robusta.

Primero, queremos añadir algunas firmas a nuestras subrutinas. Podríamos ser tentados a escribir algo como esto:

```
sub encolar( Array @cola, $nuevo_artículo ) {
    unshift @cola, $nuevo_artículo;
}
```

Pero eso no funciona porque esencialmente eso le dice a Raku que el parámetro @cola es un array de arrays. Lo que necesitamos es la siguiente sintaxis de signatura:

```
sub encolar( @cola where Array, $nuevo_artículo ) {
    unshift @cola, $nuevo_artículo;
}

sub desencolar( @cola where Array ) {
    my $artículo = pop @cola;
    return $artículo;
}
```

Probablemente no queremos ningún tipo de signatura para el parámetro \$nuevo_artículo de desencolar, porque queremos que nuestra cola sea capaz de operar con cualquier tipo de datos para que sea tan genérica como sea posible. Pero, al igual que lo dijimos sobre pilas (Sección 9.4), podríamos querer añadir varios artículos a la estructura de datos de una sola vez.

A.7.1.2 Parámetros slurpy (o variádicos)

Hay varias maneras de insertar varios elementos a la cola, pero la más simple es probablemente usar una signatura con el parámetro *slurpy* (o parámetro *variádico*): un parámetro array o hash se marca como variádico con un asterisco al frente del mismo, lo cual significa que puede atarse a un número arbitrario de argumentos (cero o más). Estos parámetros se llaman "slurpy" porque sorbe los parámetros restantes a una función, al igual que un persona sorbiendo un helado. Esto también significa que un parámetro variádico de posición puede ser solamente el último en la signatura:

```
sub encolar( @cola where Array, *@nuevos_artículos ) {
    unshift @cola, $_ for @nuevos_artículos;
    # or: unshift @queue, |@new_items;
}

sub desencolar( @cola where Array ) {
    my $artículo = pop @cola;
    return $artículo;
}

my @linea = 4, 5, 6, 7, 8;
encolar @linea, 3, 2, 1;
say @linea;
say desencolar @linea for 1..3;
```

Esto mostrará:

```
[1 2 3 4 5 6 7 8]
8
7
6
```

Ver también la Sección 11.4.2 para más detalles sobre los parámetros variádicos.

Nota que, para una subrutina encolar, no podemos simplemente escribir:

```
sub encolar( @cola where Array, *@nuevos_artículos ) {
    unshift @cola, @nuevos_artículos;
}
```

porque, cuando se provee un array como un segundo argumento, unshift inserta los nuevos artículos como una sub-lista. Esta pequeña dificultad se puede solucionar con el uso de un bucle for o con el uso del operador “|” el cual aplana la lista.

Otra posibilidad es usar la función integrada prepend en lugar de unshift, dado que añade los elementos aplanados de una array al principio de la cola:

```
sub encolar( @cola where Array, *@nuevos_artículos ) {
    prepend @cola, @nuevos_artículos;
}
```

A.7.1.3 Un cola usando shift y append

```
sub encolar( @cola where Array, *@nuevos_artículos ) {
    prepend @cola, @nuevos_artículos;
}
```

A.7.1.4 Un cola usando shift y append

El orden en el cual los argumentos son pasados es un poco contradictorio. De igual manera, podríamos preferir no tener que usar un bucle para añadir los elementos nuevos. Es un poco más fácil usar la combinación push y shift, y reemplazar push con append, que hace casi lo mismo que push pero aplana la lista al igual que prepend que vimos anteriormente:

```
sub encolar( @cola where Array, *@nuevos_artículos ) {
    append @cola, @nuevos_artículos;
}

sub desencolar( @cola where Array ) {
    my $artículo = shift @cola;
    return $artículo;
}

my @linea = 1, 2, 3, 4;
encolar @linea, 6, 7, 8;
say @linea;
say desencolar @linea for 1..3;
```

Esto muestra:

```
[1 2 3 4 6 7 8]
1
2
3
```


A.7.1.5 Excepciones

Finalmente, una debilidad adicional necesita arreglarse: ¿Qué pasa si la cola está vacía cuando intentamos desencolar un artículo? Levantar una excepción o abortar el programa podría ser lo que necesitamos. Podríamos también decidir devolver un valor indefinido y dejar que la subrutina que hace la llamada haga lo necesario con el valor devuelto:

```
sub encolar( @cola where Array, *@nuevo_articulos ) {
    append @cola, @nuevo_articulos;
}

sub desencolar( @cola where Array ) {
    return unless @cola;
    my $articulo = shift @cola;
    return $articulo;
}

my @linea;
encolar @linea, 1, 2, 3;
say @linea;
for 1..4 -> $cuenta {
    my $artic = desencolar @linea;
    if defined $artic {
        say $artic;
    } else {
        say "ERROR: La cola está vacía!";
    }
}
```

Esto produce la siguiente salida:

```
[1 2 3]
1
2
3
ERROR: La cola está vacía!
```

La subrutina desencolar podría simplificarse con el uso del operador ternario (ver Sección 11.1) y devolver el valor Nil si la lista está vacía:

```
sub desencolar( @cola where Array ) {
    @cola ?? @cola.shift !! Nil
}
```

Como un ejercicio adicional, podrías aplicar los cambios que hemos hecho más arriba para el manejo de colas al ejemplo de código para las pilas (visto en la Sección 9.4 en la p. 155).

A.7.1.6 Encapsulando los datos

Otro problema con nuestra implementación de las colas es que la cola @linea es totalmente accesible al desarrollador, quien podría ser tentado a escudriñar el array directamente o

aún peor, modificarlo, sin usar las subrutinas encolar y desencolar diseñadas para mantener la cola consistente.

Podríamos querer prevenir eso y hacer imposible que el usuario altere la cola o por lo contrario la acceda por otros medios que no son las subrutinas adecuadas. Ocultar la información sobre la implementación o al contrario hacerla inaccesible por otros medios más que aquellos diseñados para ese propósito es usualmente conocido como la *encapsulación* de datos. Una manera común de lograr la encapsulación de datos es a través de la programación orientada a objetos, la cual discutimos en el Capítulo 12.

Podemos, sin embargo, obtener un resultado similar al combinar el ámbito de variables y material que se discute en la Sección 3.14 acerca de las subrutinas como objetos de primera clase.

Considera la siguiente implementación de una cola:

```
sub crear-fifo {
  my @cola;
  return (
    sub {return shift @cola;},
    sub ($artic) {push @cola, $artic;}
  ) ;
}
my ($fifo-obtener, $fifo-poner) = crear-fifo();
$fifo-poner($_) for 1..10;
print " ", $fifo-obtener() for 1..5; # -> 1 2 3 4 5
```

La pieza central aquí es la subrutina `crear-fifo`. El array `@cola` que almacena los datos tiene ámbito léxico en esta subrutina y no puede accederse directamente desde otra parte en el programa. `crear-fifo` devuelve dos subrutinas anónimas, una para desencolar artículos y otra para encolarlos. Estas subrutinas son *clausuras* lexicales, lo cual significa que pueden acceder `@cola`, porque han sido definidas dentro de su ámbito, aún si son llamadas desde otro lugar. Aún cuando `crear-fifo` ha finalizado, esas subrutinas pueden todavía accederlo porque ellas le ceden vida extra al array mientras las subrutinas son accesibles.

El resto del código debería ser claro: cuando `crear-fifo` es llamada, crean las dos subrutinas anónimas que son almacenadas en las variables `$fifo-obtener` y `$fifo-poner`. Una subrutina tal como `crear-fifo` es algunas veces llamada una *función factoría* porque genera otras subrutinas al tiempo de ejecución. Finalmente, `$fifo-poner` es llamada diez veces para poblar la cola con los enteros del 1 al 10, y `$fifo-obtener` es llamada cinco veces para obtener los primeros cinco artículos de la cola. La cola está encapsulada: aparte de usar las dos subrutinas anónimas, no hay manera de acceder a sus datos.

Encolar una lista de artículos (en lugar de uno solo) y manejar las excepciones (tal como intentar obtener un artículo de una cola vacía) se deja como un ejercicio al lector el .

Las técnicas usadas aquí toman prestado de un paradigma de programación llamado *programación funcional*, un modelo de programación usado por lenguajes tales como Lisp, Caml, Clojure, y Haskell. Este paradigma es bien diferente a todo lo que hemos visto hasta ahora, tal como la programación orientada a objetos es otro paradigma de programación diferente. A medida que ganas experiencia como un programador, deberías hacer un intento en entender estos diferentes paradigmas, porque ofrecen maneras diferentes de

hacer las cosas, y tienen ventajas específicas para tipos específicos de problemas. Saberlos todos te da mucho más poder expresivo. Una de las cosas maravillosas de Raku es que te ofrece una herramienta moderna y poderosa para usar cada uno de estos paradigmas de programación. Capítulo 14 es sobre la programación funcional. Mientras tanto, asegúrate de leer la Subsección 9.8.5 en el capítulo sobre arrays y listas.

A.7.2 Ejercicio de la Sección 9.5: Otras Maneras de Modificar un Array (p. 158)

A.7.2.1 Simulando la función pop

La subrutina `my-pop` usa la función `splice` para simular la función `pop`:

```
sub my-pop (@array where @array > 0) {
    my @result = splice @array, @array.end, 1;
    return @result[0];
}
my @letras = 'a'..'j';
my $letra = my-pop @letras;
say $letra;           # -> j
say @letras;          # -> [a b c d e f g h i]
```

Aquí, la expresión `@array.end` devuelve el índice del último elemento del array. También es posible contar el número de artículos en el array usando dicho índice y acceder al último y los penúltimos artículos de una lista o un array usando la siguiente sintaxis:

```
> say (1..9)[-1];
9
> say (1..9)[-2];
8
```

La subrutina `my-pop` podría escribirse como sigue:

```
sub my-pop( @array where @array > 0 ) {
    my @result = splice @array, *-1, 1;
    return @result[0];
}
```

No tienes que especificar el número de elementos con `splice` si quieres el resto. Podemos evitar el uso del array inmediato `@result`. Así que podríamos simplificar `my-pop`:

```
sub my-pop (@array where @array > 0) {
    @array.splice(*-1)[0]
}
```

A.7.2.2 Simulando la función push

La única dificultad en este ejercicio es manejar una signatura con una lista “variádica” de parámetros (o parámetros slurpy). Esto se explicó en la Subsección A.7.1.2: (p. 368).

```

sub my-push( @array, *@lista ) {
    my @result = splice @array, @array.end + 1, 0, @lista;
    return @array; # push devuelve la lista modificada
                    # (raramente usada por arrays)
}
my @letras = 'a'..'j';
my-push @letras, 'k', 'l', 'm';
say @letras;      # -> [a b c d e f g h i j k l m]

```

A.7.2.3 Simulando la función unshift

Para simular la función unshift, usamos los parámetros slurpy nuevamente:

```

sub my-unshift( @array, *@lista ) {
    my @result = splice @array, 0, 0, @lista;
    return @array; # unshift devuelve la lista modificada
                    # (raramente usada por arrays)
}
my @letras = 'd'..'j';
my-unshift @letras, 'a'..'c';
say @letras;      # -> [a b c d e f g h i j]

```

A.7.2.4 Simulando el adverbio de subíndice delete

Recuerda que el adverbio delete remueve le valor, pero deja el espacio indefinido dentro del array. La función splice también removería el espacio, que es algo distinto a lo que queremos si buscamos simular el comportamiento delete (aunque, en alguna manera, podría considerarse una mejora remover el espacio también). Para realmente simular la función delete, es probablemente mejor si solo dejamos el valor “indefinido”:

```

sub my-delete( @array, $idx ) {
    my $art = @array[$idx];
    @array[$idx] = Nil;
    return $art;
}
my @letras = 'a'..'j';
my $letra = my-delete @letras, 4;
say $letra;      # -> e
say @letras;     # -> [a b c d (Any) f g h i j]

```

A.7.3 Ejercicio de la Sección 9.8: Mapeando y Filtrando los Elementos de una Lista (p. 165)

Producir un array que contiene los cuadrados de los números de una lista de entrada es algo directo:

```

my @cuadrados = map { $_ ** 2 }, 3, 5, 7;      # -> [9 25 49]

```

Para mantener los elementos de una lista que son cuadrados perfectos, una manera de hacerlo es chequear si la raíz cuadrada de cada número es un número entero. Por ejemplo:

```
my @filtro = grep { my $sq = sqrt $_; $sq == $sq.Int }, 3, 9, 8, 16;
say @filtro;                                     # -> [9 16]
```

Esto funciona bien con la muestra de datos del ejemplo, pero este programa fallará si intentamos con un número negativo. Queremos evitar esa excepción y solo considerar que un número negativo nunca puede ser un cuadrado perfecto.

Debido a que el bloque de código aquí se complicaría, podríamos mejor usar una referencia de una función:

```
sub es-cuadrado( Numeric $num ) {
    return False if $num < 0;
    my $raiz_cuad = sqrt $num;
    return $raiz_cuad == $raiz_cuad.Int;
}
my @filtro = grep &es-cuadrado, 3, 9, -6, 16;    # -> [9 16]
```

A.7.4 Ejercicio de la Sección 9.12: Técnicas Avanzadas de Ordenamiento (p. 171)

La subrutina de transformación que puede extraer grupos de letras desde las cadenas de texto es algo directo:

```
sub my_comp( Str $cad ) {
    return $0 if $cad =~ /\d+ (\w+)/;
    Nil;    # devuelve Nil si el regex no coincidió
}
```

El ordenamiento es el mismo del capítulo original:

```
say sort &my_comp, <22ac 34bd 56aa3 12c; 4abc( 1ca 45bc>;
# -> (56aa3 4abc( 22ac 45bc 34bd 12c; 1ca)
```

La subrutina de transformación es lo suficientemente simple para ser fácilmente reemplazada con un bloque de código:

```
my @desordenados = <22ac 34bd 56aa3 12c; 42acd 12cd; 4abc( 1ca 45bc 3dab!>;
my @ordenados = sort { /\d+ (\w+)/; $0 // Nil }, @desordenados;
say @ordenados;
# -> [56aa3 4abc( 22ac 42acd 45bc 34bd 12c; 1ca 12cd; 3dab!]
```

Esto puede también escribirse con una sintaxis de invocación de método:

```
my @ordenados = @desordenados.sort: { /\d+ (\w+)/; $0 // Nil };
```

A.7.5 Sección 9.1: Suma Anidada (p. 173)

La manera más obvia de calcular la suma de todos los valores contenidos en listas anidadas o arrays anidados es usa bucles anidados. Por ejemplo:

```
my @AoA = [[1, 2], [3], [4, 5, 6]];
sub suma-anidada( @array ) {
    my $suma;
    for @array -> $elem {
        for $elem.flat -> $elem_anidado {
            $suma += $elem_anidado;
        }
    }
    return $suma
}
say suma-anidada @AoA; # -> 21
```

La única dificultad sintáctica aquí es que tenemos que aplanar las sub-listas \$elem para recorrerla. Esto podría también hacerse con el operado “|”:

```
for |$elem -> $elem_anidado { ... }
```

Esta es otra manera de hacerlo, que usa un bucle for para recorrer el array externo y el operador de reducción para agregar los elementos de las listas anidadas:

```
my @AoA = [[1, 2], [3], [4, 5, 6]];
sub suma-anidada( @array ) {
    my $suma;
    for @array -> $elem {
        $suma += [+] $elem;
    }
    return $suma
}
say suma-anidada @AoA; # -> 21
```

Usar la función map para aplanar las listas anidadas y el operador de reducción puede hacer este código considerablemente más corto:

```
my @AoA = [[1, 2], [3], [4, 5, 6]];
sub suma-anidada( @array ) {
    return [+] map {|$_}, @array;
}
say suma-anidada @AoA; # -> 21
```

Al comparar esta solución que necesita una sola línea de código con la primera muestra lo expresivo que el estilo de programación funcional puede ser para manejar arrays y listas y posiblemente entiendes la razón por la cual he recomendado este estilo de programación en este capítulo.

Estas soluciones funcionan bien porque se sabe que solo hay listas y listas anidadas (listas de listas). ¿Qué pasa si el nivel de anidamiento no se conoce con antelación? Una solución para esto sería el uso de una subrutina recursiva para explorar el árbol de listas:

```

my @AoA = [[1,2], [3], [4,5,6], [3, [7,6, [3,2]]]];
sub suma-anidada( $entrada ) {
    my $suma = 0;
    for |$entrada -> $elem {
        if $elem.WHAT ~~ Int {
            $suma += $elem;
        } else {
            $suma += suma-anidada $elem;
        }
    }
    return $suma;
}
say suma-anidada @AoA; # -> 42

```

Recuerda que una técnica recursiva es usualmente una herramienta eficiente para manejar datos encadenados o anidados.

A.7.6 Ejercicio 9.2: Suma Acumulativa (p. 173)

Para calcular la suma acumulativa de una lista de valores numéricos, solo necesitamos una acumulador y usamos el valor del acumulador cada vez a través de la iteración sobre el array:

```

my @números = <2 5 7 6 5 3 6 8>;
say suma-acumula(@números); # -> [2 7 14 20 25 28 34 42]

sub suma-acumula( @array ) {
    my @acumulativa;
    my $suma_partial = 0;
    for @array -> $elemento {
        $suma_partial += $elemento;
        push @acumulativa, $suma_partial;
    }
    return @acumulativa;
}

```

Pero adivina qué? El código puede ser mucho más corto con la programación funcional. Recuerda que el metaoperador de reducción puede darte una lista de resultados parciales:

```

my @números = <2 5 7 6 5 3 6 8>;
say [\+] @números; # -> (2 7 14 20 25 28 34 42)

```

Podrías pensar que he diseñado estos ejercicios para probar mi punto sobre el poder expresivo de la programación funcional. ¡No del todo! Ambos ejercicios, este ejercicio y el anterior, son tomados directamente del libro *Think Python* de Allen Downey en el cual este libro está basado. No he escrito estos dos ejercicios pero solamente las soluciones presentadas aquí.

A.7.7 Ejercicio 9.3: Medio (p. 173)

La forma más fácil de producir una lista nueva que contiene todos los elementos excepto el primer y último elemento de una lista dada es simplemente usar una rebanada (*slice*):

```
say medio(5..10);    # -> (6 7 8 9)
sub medio( @array ) {
    return @array[1..*-2]
}
```

Nota que `*-1` se refiere al índice del último elemento de una array. Para desechar el último elemento, podemos limitar el rango `*-2`.

A.7.8 Ejercicio 9.4: Corta (p. 173)

La diferencia básica con el ejercicio previo es que el array debería ser modificado en lugar, en vez de ser devuelto desde la función.

Esta es una posible solución, la cual las funciones `shift` y `pop` para remover respectivamente el primer y el último elemento de una array:

```
my @nums = 5..10;
corta(@nums);
say @nums;    # -> [6 7 8 9]

sub corta( @array ) {
    shift @array;
    pop @array;
    return;
}
```

Usar una rebanada es más simple; solo asegúrate de asignar la rebanada al array para modificar el array en lugar:

```
sub corta( @array ) {
    @array = @array[1..*-2];
    return;
}
```

A.7.9 Ejercicio 9.5: Subrutina está-ordenada (p. 174)

Para chequear si una lista está ordenada, solo necesitamos iterar sobre sus elementos, mantener un registro del valor previo y comparar el valor actual con el valor previo. Devolver falso si cualquier par de valores no satisface la comparación, y devolver verdadero si se llegas al final de la iteración:

```
sub está-ordenada( @array ) {
    my $previo = @array[0];
    for @array -> $actual {
        return False if $actual < $previo;
    }
}
```



```

        $previo = $actual;
    }
    return True;
}
say está-ordenada < 2 4 5 7 7 8 9>;    # -> True
say está-ordenada < 2 4 5 7 6 8 9>;    # -> False

```

Otra técnica podría simplemente comparar la lista de entrada con una versión de la misma:

```

sub está-ordenada(@array) {
    return @array.Seq eqv @array.sort;
}
say está-ordenada < 2 4 5 7 7 8 9>;    # -> True
say está-ordenana < 2 4 5 7 6 8 9>;    # -> False

```

Nota que debido a que la rutina `sort` retorna una secuencia, necesitamos coaccionarla la entrada en una `Seq` para que el operador `eqv` pueda comparar apropiadamente las secuencias de valores.

Mientras esto conduce a código más corto y simple, esta no es una solución óptima, porque requiere que el programa ordene el array de entrada, lo cual es significativamente más costoso que solamente recorrer el array, por lo menos cuando el array a ser chequeado es largo.

Una vez más, la programación funcional y especialmente el hiperoperador de reducción puede conducir a código mucho más corto que la primera solución, sin incurrir en el costo de un ordenamiento adicional:

```

sub está-ordenada( @array ) {
    return [<=] @array;
}
say está-ordenada < 2 4 5 7 7 8 9>;    # -> True
say está-ordenada < 2 4 5 7 6 8 9>;    # -> False

```

Por cierto, esta última versión de `está-ordenada` hará “corto circuito” y devolver `False` tan pronto haya encontrado el valor que no se encuentra en el orden correcto, sin iterar sobre el resto de la lista.

A.7.10 Ejercicio 9.6: Subrutina `es-anagrama` (p. 174)

Al comparar dos palabras para chequear si son anagramas, podemos iniciar con devolver falso si no tienen la misma longitud, dado que los anagramas obviamente tienen el mismo número de letra. Esto podría agilizar el proceso si el proceso detallado para comparar dos cadenas de texto requiere mucho tiempo, al evitar la parte que consume mucho tiempo para casos que obviamente no coincidirán.

No queremos intentar cada permutación de las letras dado que eso tomaría mucho tiempo. La forma más fácil de chequear por anagramas es probablemente comenzar con la *normalización* de las cadenas de texto de entrada, o sea, reorganizando las cadenas de texto de tal manera que puedan ser fácilmente comparadas. La manera más obvia es solo ordenar las letras de las dos palabras y comparar los resultados:

```

sub es-anagrama( Str $palabra1, Str $palabra2 ) {
    return False if $palabra1.chars != $palabra2.chars;
    return False if $palabra1.comb.sort ne $palabra2.comb.sort;
    True;
}

for <ban bane post stop post pots pots taps saco cosa> -> $w1, $w2 {
    say "$w1 $w2:\t", es-anagrama $w1, $w2;
}

```

Esto produce la siguiente salida:

```

$ raku anagramas.raku
ban bane:      False
post stop:     True
pots stop:     True
post pots:     True
pots taps:     False
saco cosa:     True

```

Nota que esto funciona correctamente porque el operador ne coacciona su argumento en una cadena de texto antes de realizar la comparación.

Este código puede reducirse aún más (pero posiblemente menos eficiente) al devolver directamente la comparación de las versiones ordenadas:

```

sub es-anagrama( Str $palabra1, Str $palabra2 ) {
    return $palabra1.comb.sort eq $palabra2.comb.sort;
}

```

A.7.11 Ejercicio 9.7: Subrutina tiene-duplicados (p. 174)

Dentro del contexto de lo que hemos visto hasta ahora, la forma más fácil de encontrar si una lista de cadenas de texto tiene duplicados es probablemente ordenar la lista, para que duplicados posibles sean adyacentes, y comparar cada elemento del array ordenado con el elemento previo (o siguiente):

```

say tiene-duplicados(< a b c df g xy z r e >); # -> False
say tiene-duplicados( < a b c df g xy z c e >); # -> True

sub tiene-duplicados( @array ) {
    my @ordenado = sort @array;
    for 1..@ordenado.end -> $i {
        return True if @ordenado[$i] eq @ordenado[$i - 1];
    }
    False;
}

```

En este ejemplo, el bucle comienza con el índice 1 (y no 0) por cada artículo es comparado con el artículo previo.

Otra forma es iterar sobre los elementos de una array ordenado y mantener un registro de los artículos previos para habilitar la comparación:

```

say tiene-duplicados( < a b c d f y z r e > ); # -> False
say tiene-duplicados( < a b c d f y z c e > ); # -> True

sub tiene-duplicados( @array ) {
    my @ordenado = sort @array;
    my $previo   = shift @ordenado;
    for @ordenado -> $artic {
        return True if $artic eq $previo;
        $previo = $artic;
    }
    False;
}

```

Otra posibilidad es usar la función `unique` de Raku, la cual devuelve una secuencia de valores únicos desde la lista o array de entrada. Comparar el número de elementos de la salida de output con el número de elementos de la lista original nos dirá si algunos duplicados fueron removidos por `unique`:

```

sub tiene-duplicados( @array ) {
    my @artic-unicos = unique @array;
    return False if @artic-unicos.elems == @array.elems;
    True;
}

```

Esto podría escribirse más concisamente al encadenar las invocaciones de método:

```

sub tiene-duplicados( @array ) {
    @array.unique.elems != @array.elems;
}

```

Nota que Raku también tiene una función integrada `repeated`, la cual es la contraparte de `unique` y devuelve los elementos duplicados de una lista:

```

say <a b c d b f d>.repeated; # -> (b d)

```

La subrutina `tiene-duplicados` puede también coaccionar la salida de `repeated` en un Booleano:

```

sub tiene-duplicados( @array ) {
    ?@array.repeated # -> ? coacciona su argumento a un Booleano
}

```

Otra forma eficiente de encontrar o remover los duplicados de una lista o un array es usar *hashes*, una estructura de datos integrada que discutimos en el Capítulo 10 (ver Ejercicio 10.3).

A.7.12 Ejercicio 9.8: Simulando la Paradoja del Cumpleaños (p. 174)

Para simular la paradoja del cumpleaños, necesitamos generar enteros aleatorios entre 1 y 365 (cada entero representa una fecha en el año). Por razones de simplicidad, generaremos enteros aleatorios entre 0 y 364, lo cual es equivalente para nuestro propósito.

Ejecutaremos la simulación 1,000 veces:

```

sub tiene-duplicados( @array ) {
    return ?@array.repeated
}

sub tiene-cumpleaños-duplicados( Int $num-estudiantes ) {
    my @blist;
    for 1..$num-estudiantes {
        push @blist, 365.rand.Int; # números entre 0 y 364
    }
    return tiene-duplicados(@blist);
}

my $cuenta-dupl = 0;
my $num-pruebas = 1000;
for 1..$num-pruebas{
    $cuenta-dupl++ if tiene-cumpleaños-duplicados 23; # 23 estudiantes
}
say "En $num-pruebas pruebas, $cuenta-dupl tenían "~
    "por lo menos un cumpleaños duplicado";

```

Nota que hemos reutilizado la subrutina `tiene-duplicados` del ejercicio anterior. Es tan corta que su código podría haberse utilizado en línea en la subrutina `tiene-cumpleaños-duplicados`, pero es considerado buena práctica utilizar componentes de software que han sido desarrollados y probados.

Ejecutamos el programa cuatro veces y esto resulta:

```

$ raku birthdays.raku
En 1000 pruebas, 498 tenían por lo menos un cumpleaños duplicado

$ raku birthdays.raku
En 1000 pruebas, 505 tenían por lo menos un cumpleaños duplicado

$ raku birthdays.raku
En 1000 pruebas, 527 tenían por lo menos un cumpleaños duplicado

$ raku birthdays.raku
En 1000 pruebas, 491 tenían por lo menos un cumpleaños duplicado

```

Esta simulación confirma que con una muestra de 23 personas, hay una posibilidad aproximada de 50% que al menos dos personas tendrán el mismo cumpleaños.

Nota que Raku tiene una función integrada `roll` que devuelve elementos seleccionados aleatoriamente desde una lista. Esto puede hacer la subrutina `tiene-cumpleaños-duplicados` significativamente más concisa:

```

sub tiene-cumpleaños-duplicados( Int $num-estudiantes ) {
    tiene-duplicados((~365).roll($num-estudiantes) )
}

```

A.7.13 Ejercicio 9.9: Comparando a push y unshift (p. 174)

Popular una array con push o unshift es algo que has visto anteriormente. La única cosa nueva aquí es comparar el tiempo de ejecución de varias soluciones. La función `now` devuelve el número de segundos que ha transcurrido desde un punto de partida teórico llamado “the Epoch”, usualmente 1 de Enero, 1970. Llamar a `now` una vez antes de ejecutar el script y otra vez después nos dirá el tiempo transcurrido a través de una sustracción simple.

```
my $inicio_push = now;
my @push_array;
for 'words.txt'.IO.lines -> $linea {
    push @push_array, $linea;
}
say "push tomó " ~ now - $inicio_push ~ " segundos.";
@push_array = ();

my $inicio_unsh = now;
my @unsh_array;
for 'words.txt'.IO.lines -> $linea {
    unshift @unsh_array, $linea;
}
say "unshift tomó " ~ now - $inicio_unsh ~ " segundos.";
@unsh_array = ();
```

Esta es una muestra de ejecución del programa:

```
push tomó 1.870107 segundos.
unshift tomó 2.2291266 segundos.
```

Inténtalo tu mismo y ejecútalo varias veces. Deberías notar que push es consistentemente más rápido que unshift, aunque la diferencia no es tan abismal.

Presumiblemente, esto se debe a que unshift cuando está insertando elementos al comienzo del array, Raku tiene que mover los datos muchas veces para reorganizar el array completo. Por lo contrario, usar push para insertar elementos al final del array implica menos mantenimiento interno de los datos.

Como un ejercicio adicional, puedes tratar de explorar otras maneras de poblar un array, tal como `append` o `splice`.

Si vas a insertar cada línea del archivo de entrada en un array sin hacer ningún cambio a esas líneas, sorber (*slurp*) todos los datos en el array sin un bucle será más simple y mucho más rápido:

```
my $sorbido_inicio = now;
my @array_sorbido = 'words.txt'.IO.lines;
say "slurp tomó " ~ now - $sorbido_inicio ~ " segundos.";
```

Esto es de cuatro a cinco veces más rápido:

```
slurp took 0.42602506 seconds.
```

Nota que realmente no necesitas llamar la función `now` al inicio del programa: puedes usar `INIT now` para extraer el tiempo cuando el programa comenzó a ejecutarse:

```
my @array_sorbido = 'words.txt'.IO.lines;
say "slurp tomó " ~ (now - INIT now) ~ " segundos.";
```

A.7.14 Ejercicio 9.10: Búsqueda Binaria en una Lista (p. 174)

Podemos comenzar con un algoritmo de bisección recursiva:

```
sub bisectar( @lista_palabras, Str $palabra ) {
  my $indice = (@lista_palabras.elems / 2).Int;
  return False if $indice == 0 and @lista_palabras[$indice] ne $palabra;
  my $encontrada = @lista_palabras[$indice];
  if $palabra lt $encontrada {
    # realiza la búsqueda en la primera mitad del array
    return bisectar @lista_palabras[0..$indice-1], $palabra;
  } elsif $palabra gt $encontrada {
    # realiza la búsqueda en la segunda mitad del array
    return bisectar @lista_palabras[$indice+1..*-1], $palabra;
  }
  True;      # si llegamos aquí, hemos encontrado la palabra
}

for <a f w e q ab ce> -> $búsqueda {
  if bisectar [<a b d c e f g>], $búsqueda {
    say "'$búsqueda' encontrada";
  } else {
    say "'$búsqueda' no encontrada";
  }
}
```

Esto mostrará la salida siguiente:

```
'a' encontrada
'f' encontrada
'w' no encontrada
'e' encontrada
'q' no encontrada
'ab' no encontrada
'ce' no encontrada
```

Existen varias debilidades en esta implementación. Primero, en cada llamada recursiva, `bisectar` pasa como un argumento un array que puede ser bien grande, y esto no es eficiente en términos de uso de memoria (para almacenar los subconjuntos sucesivos del array original) y términos de los ciclos de CPU (el tiempo tomado para copiar estos arrays).

Además, podemos saber si la palabra a buscarse puede encontrarse en la lista (y existen muchos casos donde no necesitamos más información que esa), pero no sabemos donde

se encontró (es decir, el índice original en el array original), que es lo que realmente necesitábamos.

Una opción involucra tener solamente una copia del array original, digamos una variable global, y pasar rangos de índices. Pero variables globales son usualmente desaprobadas porque tienden a ir en contra de los postulados de la programación estructurada y pueden ser peligrosa (aunque este sería un caso donde el uso de variables globales hace sentido). Podemos actualmente trabajar mejor sin el uso de variables globales y aún tener el beneficio de no pasar el array completo una y otra vez gracias al hecho que, en Raku, las subrutinas son *clausuras*, lo cual significa que pueden acceder variables que existen en el entorno donde fueron creadas.

En el código siguiente, *bisectar* no es un subrutina recursiva; ahora, es una subrutina muy simple que solo configura el entorno para *bisectar2*, la cual es la subrutina recursiva y está definida dentro del cuerpo de *bisectar*. Dado que el array y la palabra a ser buscada existen dentro de *bisectar*, *bisectar2* será capaz de tener acceso a ellos. Los parámetros de *bisectar2* son solamente dos subíndices que representan el rango en el cual tendrá que buscar la palabra:

```
sub bisectar( Str $palabra, @lista_palabras ) {
  sub bisectar2( $idx_inferior, $idx_superior ) {
    my $idx_medio = (($idx_inferior + $idx_superior) /2).Int;
    my $encontrada = @lista_palabras[$idx_medio];
    return $idx_medio if $palabra eq $encontrada;
    return -1 if $idx_inferior >= $idx_superior;
    if $palabra lt $encontrada {
      # realiza la búsqueda en la primera mitad del array
      return bisectar2 $idx_inferior, $idx_medio - 1;
    } else {
      # realiza la búsqueda en la segunda mitad del array
      return bisectar2 $idx_medio+1, $idx_superior;
    }
  }
  my $indice_max = @lista_palabras.end;
  return bisectar2 0, $indice_max;
}

for <a f w e q ab ce g> -> $búsqueda {
  my $result = bisectar $búsqueda, [<a b d c e f g>];
  if $result == -1 {
    say "'$búsqueda' no encontrada";
  } else {
    say "'$búsqueda' encontrada en la posición $result";
  }
}
```

Como un ejercicio adicional, adapta el programa anterior para buscar palabras en inglés en *words.txt* y palabras en español en *lemario.txt*. Nota lo rápido que es. Por favor ten presente que esto funciona porque las palabras en este archivo están ordenadas alfabéticamente.

Intenta cambiar el código para contar y mostrar el número de pasos necesarios para encontrar una palabra dada. Compara esto con el número de pasos que tomaría una búsqueda

lineal en promedio (o sea, recorrer el array linealmente hasta encontrar la palabra o pueda la palabra ser declarada ausente). Puedes adivinar por qué este algoritmo es también llamado *búsqueda logarítmica*?

Podrías también escribir una solución que no es recursiva usando un bucle.

A.7.15 Ejercicio 9.11: Invirtiendo Parejas de Palabras (p. 175)

Encontrar parejas de inversa requiere leer cada palabra de la lista y chequear la lista para ver si las palabras inversas existen en la lista. Esto significa que vas a buscar alrededor de 113,000 palabras en una lista que contiene 113,000 palabras. Tu método de búsqueda necesita ser eficiente. La solución obvia es usar la búsqueda binaria implementada en el ejercicio anterior:

```
sub bisectar( Str $palabra, @lista_palabras ) {
    # ver código en el ejercicio anterior
}

my @array = 'words.txt'.IO.lines;

for @array -> $palabra {
    my $inversa = $palabra.flip;
    my $res = bisectar $inversa, @array;
    say "$palabra y $inversa forman una pareja de inversas" if $res >= 0;
}

say now - INIT now;
```

En mi computador portátil (una computadora decente pero no una máquina poderosa), el proceso completo tomó 42 segundos, o sea, menos de 0.4 milisegundos por búsqueda.

Si piensa sobre ello, el bucle `for` en el código más arriba está realmente filtrando aquellas palabras que pertenecen a una pareja inversa de la lista de palabras. Esto podría implementarse con una función `grep` usando la subrutina `bisectar` para seleccionar las coincidencias:

```
say "$_ y $_.flip() forman una pareja de inversas"
    for @array.grep( { bisect( .flip, @array ) >= 0 } );
```

Con el algoritmo usado aquí, cada pareja de inversas se encuentra dos veces (una vez por cada palabra de la pareja). Al examinar cualquier palabra de la lista, realmente no necesitamos buscar hacia atrás en la parte de la lista anterior a esa palabra dado que si la palabra forma una pareja con otra palabra que viene antes en el orden alfabético, ya hemos encontrado la pareja cuando procesamos esa otra palabra. Por lo tanto sería más eficiente y más rápido si hacemos la búsqueda solamente hacia adelante, es decir, buscar la palabra en la parte de la lista que viene después de la palabra que está siendo examinada. Como un ejercicio adicional, modifica el bucle `for` para buscar palabras hacia adelante.

A.7.15.1 Comparando la búsqueda binaria con la búsqueda en un hash

La búsqueda binaria es bastante rápida, pero la búsqueda en una hash es mucho más rápida. Aunque aún no hemos discutido los hashes, intenta el código siguiente:


```

my %hash = map { $_ => 1}, 'words.txt'.IO.lines;
for %hash.keys -> $palabra {
    my $inversa = $palabra.flip;
    say "$palabra y $inversa forman una pareja de inversas"
    if %hash{$inversa}:exists;
}
say now - INIT now;

```

No te preocupes en entender el código por el tiempo presente, pero nota lo corto que es. Y lo rápido que se ejecute: en la misma computadora portátil, el tiempo de ejecución es alrededor de 16 segundos (menos de 0.15 milisegundos por búsqueda). Espero que esto estimulará tu apetito para el Capítulo 10.

Nota que la salida de este ejemplo no está ordenada porque un hash no mantiene el orden de los datos de entrada, como veremos en el Capítulo 10. Sería bien fácil mantener el ordenamiento, por ejemplo al usar un array en adición al hash, pero ese no es realmente el tema aquí.

A.7.15.2 Creando y usando un módulo

Regresando a la subrutina *bisectar*, copiar y pegar esta subrutina desde el programa del ejercicio anterior en el código de este ejercicio no es la mejor manera de reutilizar código. Supón que un error es encontrado en esa subrutina; ahora necesita ser arreglado en dos programas diferentes; la probabilidad que ese error sea corregido en un programa y olvidado en el otro es bien significativo. Aún si no se olvida, esto doblemente arduo, y esto es incrementa la probabilidad de cometer otro error en el proceso. Aún si no hay ningún error, podríamos necesitar un mejora y otra vez esto debe hacerse dos veces.

Una buena manera de reutilizar software mientras se mantiene una sola copia de la subrutina reutilizada es insertarla en un módulo de Raku, i.e, en un archivo separado que será cargado en nuestros programas para usarlo.

El módulo podría nombrarse *BusquedaBinaria.pm* y contener el código siguiente:

```

unit module BusquedaBinaria;

sub bisectar( Str $palabra, @lista_palabras ) is export {
    sub bisectar2( $idx_inferior, $idx_superior ) {
        my $idx_medio = (($idx_inferior + $idx_superior) /2).Int;
        my $encontrada = @lista_palabras[$idx_medio ];
        return $idx_medio if $palabra eq $encontrada;
        return -1 if $idx_inferior >= $$idx_superior;
        if $palabra lt $encontrada {
            # realiza la búsqueda en la primera mitad del array
            return bisectar2 $idx_inferior, $idx_medio - 1;
        }
        else {
            # realiza la búsqueda en la segunda mitad del array
            return bisectar2 $idx_medio+1, $idx_superior;
        }
    }
}

```

```

        my $indice_max = @lista_palabras.end;
        return bisectar2 0, $indice_max;
    }

    sub mostrar-resultado( Int $result, Str $busqueda ) is export {
        if $result == -1 {
            say "'$busqueda' no encontrada";
        }
        else {
            say "'$busqueda' encontrada: artículo # $result";
        }
    }
}

```

Nota que el nombre del módulo dado en la parte superior del código y el nombre del archivo tienen que corresponder:

- unit module **BusquedaBinaria**;
- **BusquedaBinaria**.pm

El otro cambio que se hizo al código de la subrutina fue agregar el rasgo `is export` a la signatura de la subrutina.

Ahora un programa de Raku será capaz de cargar este módulo y usar las subrutinas `bisectar` y `mostrar-resultado`. Por ejemplo:

```

use lib "."; # le dice a Raku que busque módulos en el directorio actual
use BusquedaBinaria;

my @array = 'a'..'m';
for < a f w e q a b c e g > -> $busqueda {
    my $result = bisectar $busqueda, @array;
    mostrar-resultado $result, $busqueda;
}

```

Raku tiene una lista de directorios que busca por módulos, los cuales pueden variar de una instalación de Raku a otra. La primera línea `use lib "."`; le dice a Raku que también busque por módulos en el directorio actual (es decir, el directorio donde se ejecuta el programa que utiliza el módulo). Esto es solo un ejemplo; podrías preferir usar un directorio para tus módulos. La segunda línea `use BusquedaBinaria`; le dice a Raku que cargue el módulo e importe las subrutinas marcadas para exportación (por ejemplo, con `is export`) en el módulo. Ahora, el programa puede usar las subrutinas `bisectar` y `mostrar-resultado` como si hubiesen sido definidas dentro del programa.

¡Eso es todo, amigos! ¿Bien simple, cierto? ¡Solo inténtalo! Aunque existen otras cosas más que saber sobre módulos, ya sabes lo suficiente para crear y usar módulos.

Podrías revisar algunas de las otras subrutinas que hemos creado hasta ahora y poner aquellas que podrían ser útiles en un módulo. Pista: algunas de las subrutinas relacionadas con cadenas de texto y arrays que hemos visto hasta ahora son buenas candidatas.

A.7.16 Ejercicio 9.12: Entrelazando Palabras (p. 175)

Primero, al parecer fue una buena idea crear el módulo `BusquedaBinaria`. Los reutilizaremos inmediatamente.

Segundo, necesitamos pensar. La primera idea para solucionar el problema podría ser el uso de un bucle anidado en la lista de palabras en orden para encontrar todas las parejas de las dos palabras, entrelazarlas, y chequear si la cadena resultante existe en la lista. Pero esto es bien ineficiente debido a que significa que debemos crear 113,000 parejas al cuadrado, o sea, más de 12.5 billones de parejas.

Aún si una parte relativamente grande de estas parejas puede eliminarse antes de hacer la búsqueda en la lista de palabras dado que una pareja puede ser entrelazada solo si la diferencia entre el número de letras de las dos palabras es 0 o 1, chequear todas estas parejas tomará eones.

Veamos que pasa si trabajamos en la otra dirección: por cada palabra en la lista de palabras, nosotros “entrelazamos” la palabra en una cadena de texto con las letras pares y las letras impares, y después chequeamos si estas sub-cadenas pertenecen a la lista. Como máximo, necesitamos 226,000 búsquedas—de hecho mucho menos porque no necesitamos buscar la segunda cadena de texto si la primera cadena de texto no coincidió con nada.

Esta es la solución que sugerimos:

```
use lib ".";
use BusquedaBinaria;

my @array = 'words.txt'.IO.lines;
for @array -> $palabra {
    my ($palabra1, $palabra2) = entrelazar($palabra);
    say "$palabra: $palabra1, $palabra2" if
        bisectar($palabra1, @array) >= 0 and
        bisectar($palabra2, @array) >= 0;
}

sub entrelazar( Str $palabra ) {
    my @letras = $palabra.comb;
    my $pares = join '', map {@letras[$_] if $_ %% 2}, @letras.keys;
    my $impares = join '', map {@letras[$_] if $_ % 2}, @letras.keys;
    return ($pares, $impares);
}
```

La subrutina `entrelazar` no es óptima en el sentido que recorre las letras del array `@letras` dos veces cada vez que es llamada. Podemos mejorarla usando un bloque punteggiado que toma dos parámetros (una letra impar y otra par):

```
sub entrelazar( Str $palabra ) {
    my (@pares, @impares);
    for $palabra.comb -> $par, $impar='' { # valor por defecto para $impar
        push @pares, $par;                # para cuando el número de letra es
        push @impares, $impar;            # impar
    }
}
```

```
@pares.join, @impares.join;
}
```

Como un ejercicio adicional, puedes encontrar palabras que están entrelazadas en tres formas, es decir, cada tercer letra forma una palabra, comenzando con la primera, segunda o tercera? Pista: será probablemente más fácil si comienzas con la versión revisada de entrelazar más arriba.

A.8 Ejercicios del Capítulo 10: Hashes

A.8.1 Ejercicio al Final de la Sección 10.1: Un hash es un mapeo (p. 179)

Esta es la manera en la cual se puebla un par a la vez:

```
my %salarios;
%salarios{"Liz"} = 3000;
%salarios{"Bob"} = 2500;
%salarios{"Jack"} = 2000;
%salarios{"Betty"} = 1800;
say "El salario de Bob es %salarios{'Bob'}";
for <Liz Jack> -> $empleado {
    say "El salario de $empleado es %salarios{$empleado};
}
```

Puedes evitar las comillas alrededor de las claves al usar el operador de comillas angulares <...>:

```
my %salarios;
%salarios<Liz> = 3000;
%salarios<Bob> = 2500;
# ...
say "El salario de Bob es %salarios<Bob>";
```

Y esto es cómo se asigna al hash completo en una sola vez:

```
my %salarios = Liz => 3000, Bob => 2500, Jack => 2000, Betty => 1800;
say %salarios; # -> Betty => 1800, Bob => 2500, Jack => 2000, Liz => 3000
```

A.8.2 Ejercicio 10.1: Almacenar la Lista de Palabras en un Hash (p. 194)

La forma estándar de almacenar la lista de palabras en un hash sería leer cada línea de un archivo en un bucle for y almacenar cada palabra como la clave del hash. El contenido del valor no es importante; almacenaremos 1 (haría también sentido si almacenamos el valor Booleano True):

```
my %palabras;
for 'lemario.txt'.IO.lines -> $linea {
    %palabras{$linea} = 1
}
```

Una técnica alternativa es asignar al hash la salida de una expresión map que devuelve una par por cada línea del archivo:

```
my %palabras = map { $_ => 1 }, 'lemario.txt'.IO.lines;
```

A.8.3 Ejercicio 10.2: Memoizando la Función Ackermann (p. 194)

La implementación original de la función Ackermann lucía así:

```
sub ack( $m, $n ) {
  return $n + 1 if $m == 0;
  return ack($m - 1, 1) if $n == 0;
  return ack($m - 1, ack($m, $n-1));
}
```

No es posible memoizar los casos donde m o n es cero, porque los otros valores son desconocidos. Solo el código correspondiente a la última línea de código puede ser memoizado, pero eso está bien porque hace casi todo el trabajo de cualquier manera.

El problema siguiente es que los hashes vistos hasta ahora tenían solamente una clave, pero la función Ackermann toma dos parámetros. La solución simple es crear un clave compuesta, es decir, concatenar los dos parámetros con un separador para crear las claves del hash. Esto conduce a esta posible solución:

```
my %ack-memo;
sub mem-ack( Int $m, Int $n ) {
  return $n + 1 if $m == 0;
  return mem-ack($m - 1, 1) if $n == 0;
  %ack-memo{"$m;$n"} = mem-ack($m - 1, mem-ack($m, $n-1))
    unless %ack-memo{"$m;$n"}:exists;
  return %ack-memo{"$m;$n"};
}
say mem-ack 3, 4;
```

Para hacer un benchmark de las soluciones, puedes usar el código siguiente:

```
my %ack-memo;
sub mem-ack (Int $m, Int $n) {
  return $n + 1 if $m == 0;
  return mem-ack($m - 1, 1) if $n == 0;
  %ack-memo{"$m;$n"} = mem-ack($m - 1, mem-ack($m, $n-1))
    unless %ack-memo{"$m;$n"}:exists;
  return %ack-memo{"$m;$n"};
}

my $inicio = now;
say mem-ack 3, 4;
say "mem-ack runtime: ", now - $inicio;
dd %ack-memo;

sub ack ($m, $n) {
  return $n + 1 if $m == 0;
  return ack($m - 1, 1) if $n == 0;
  return ack($m - 1, ack($m, $n-1));
}

$inicio = now;
say ack 3, 4;
say "tiempo de ejecución de ack: ", now - $inicio;
```

Pero no intentes ejecutarla con valores de `$m` mayores que 3; no es útil. Si fuéramos a encontrar un valor Ackermann para un par de números que ya hemos visto, eso significaría que hemos entrado en un bucle infinito. Así que no hace sentido tratar de memoizar la función Ackermann.

Hemos usado claves compuestas para `%ack-memo`, pero podemos tener hashes multidimensionales al igual que tenemos arrays multidimensionales (ver Sección 9.10). Solo necesitamos tener dos claves, cada una de dentro de un par de llaves:

```
my %h;
%h{'a'}{'b'}= 'ab';
%h{'a'}{'c'}= 'ac';
%h{'a'}{'d'}= 'ad';
%h{'b'}{'c'}= 'bc';
dd %h;
# -> Hash %h = {a(${:b("ab"), :c("ac"), :d("ad")}), :b(${:c("bc")})}
```

o usar un punto y medio para separar las llaves:

```
my %i;
%i{'a';'b'} = 'ab';
%i{'a';'c'} = 'ac';
%i{'b';'c'} = 'bc';
dd %i; # -> Hash %i = {a(${:b("ab"), :c("ac")}), :b(${:c("bc")})}
```

A.8.4 Ejercicio 10.3: Encontrando Duplicados con un Hash (p. 194)

Necesitamos un bucle sobre el array, almacenar los elementos del array en un hash y detectar si un elemento se encuentra en el hash. Esta es una manera de lograr eso:

```
sub tiene-duplicados( @array ) {
    my %vistos;
    for @array -> $elmt {
        return True if %vistos{$elmt}:exists;
        %vistos{$elmt} = 1;
    }
    return False;
}
```

Como un ejercicio adicional, genera una lista de 50,000 enteros aleatorios entre 0 y 1,000,000,000, y después, usando los varios métodos que hemos demostrado, chequea si esta lista contiene elementos duplicados y mide el tiempo de ejecución de estos varios métodos. Si encuentras dificultades haciendo esto, estudia las soluciones a los ejercicios “tiene duplicados” (ver Solución: A.7.11) y la “paradoja del cumpleaños” (ver Solución: A.7.12)) para obtener algunas pistas. Un ejemplo de un benchmarking simple es presentado en el ejercicio más arriba.

Una vez que tus subrutinas funcionan apropiadamente, ejecuta el proceso por lo menos 10 veces para ver si las diferencias son significantes.

A.8.5 Ejercicio 10.4: Rotar Pares (p. 194)

Considera la palabra “iron” y haz una rotación de tres letras. Esto resulta en la palabra “folk”. Esto también significa que si rotamos a “folk” por 23 letras, obtendremos “iron” de vuelta. Dado que vamos a escanear todas las palabras de una lista de palabras, encontraremos este “rotar par” cuando intentamos un desplazamiento (*shift*) de tres letras con “iron”, así que no hay necesidad de intentar un rotación de 23 letras con “folk”. Generalmente, solo necesitamos hacer rotaciones entre 1 y 13 letras.

El código siguiente itera a través de las palabras de la lista, rota cada una de ellas por un desplazamiento 1 y 13, y busca el resultado en el hash:

```
sub rotar-una-letra( Str $letra, Int $despl ) {
    my $última      = 'z'.ord;      # última letra minúscula
    my $ord-rotada = $letra.ord + $despl;
    if $letra =~ /[a..z]/ {
        $ord-rotada -= 26 if $ord-rotada > $última;
    } else {
        return $letra;
    }
    return $ord-rotada.chr;
}

sub rotar-una-palabra( Str $palabra, Int $despl ) {
    my $palabra-rotada = "";
    for 0..$palabra.chars - 1 {
        $palabra-rotada ~= rotar-una-letra substr($palabra, $_, 1), $despl;
    }
    return $palabra-rotada;
}

my %palabras = map { $_ => 1}, 'lemario.txt'.IO.lines;

for %palabras.keys -> $cadena {
    for 1..13 -> $despl {
        my $rotada = rotar-una-palabra $cadena, $despl;
        say " $cadena y $rotada están desplazadas por $despl"
        if %palabras{$rotada}:exists;
    }
}
```

Rotar cada palabra de una lista con alrededor de 88,000 palabras por cada desplazamiento entre 1 y 13 es algo bien largo. Ejecutar el programa tomará algún tiempo, probablemente de 10 a 15 minutos. Usar la función integrada `.trans` (ver documentación en <https://docs.raku.org/routine/trans>) podría acelerar el proceso. Inténtalo y juzga por ti mismo.

A.8.6 Ejercicio 10.5: Homófonas (p. 194)

Estamos buscando por palabras que *suenan* igual cuando removemos la primera o la segunda letra.

Esta es una solución que usa la lista de palabras *words.txt* usada anteriormente y el diccionario fonético CMU²:

```
my %fonético;

sub cargar-fonético( $nombre-archivo ) {
  for $nombre-archivo.IO.lines -> $linea {
    next if $linea !~ /^\\w/;
    my ($clave, $val) = $linea.split(" ", 2);
    $clave = lc $clave;
    %fonético{$clave} = $val;
  }
}

cargar-fonético('cmu_dict.txt');
my %palabras = map { $_ => 1}, 'words.txt'.IO.lines;

say "Iniciando la búsqueda";

for %palabras.keys -> $palabra {
  next unless %fonético{$palabra}:exists;
  my $más-corta = $palabra.substr(1);
  next unless %palabras{$más-corta}:exists;
  next unless %fonético{$más-corta}:exists;
  next unless %fonético{$palabra} eq %fonético{$más-corta};
  my $otra-más-corta = $palabra.substr(0, 1) ~ $palabra.substr(2);
  next unless %palabras{$otra-más-corta}:exists;
  next unless %fonético{$otra-más-corta}:exists;
  next unless %fonético{$otra-más-corta} eq %fonético{$más-corta};
  say "$palabra $más-corta $otra-más-corta %fonético{$más-corta}"
}
```

Pero esto es algo ineficiente porque actualmente no necesitamos la lista de palabras, dado que el diccionario CMU es otra lista de palabras que podemos usar (y no podemos usar palabras que aparecería en la lista de palabras y no en el diccionario CMU, porque el programa no sería capaz de averiguar cómo suenan). El programa siguiente usa solo el diccionario CMU y ahorra el tiempo de cargar la lista de palabras y hacer chequeos en la misma:

```
my %fonético;

sub cargar-fonético( $nombre-archivo ) {
  $nombre-archivo.IO.lines -> $linea {
    next if $linea !~ /^\\w/;
    my ($clave, $val) = $linea.split(" ", 2);
    $clave = lc $clave;
    %fonético{$clave} = $val;
  }
}
```

²Puedes descargar el archivo aquí: <http://www.speech.cs.cmu.edu/cgi-bin/cmudict>


```

cargar-fonético('cmu_dict.txt');

for %fonético.keys -> $palabra {
  my $más-corta = $palabra.substr(1);
  next unless %fonético{$más-corta}:exists;
  next unless %fonético{$palabra} eq %fonético{$más-corta};
  my $otra-más-corta = $palabra.substr(0, 1) ~ $palabra.substr(2);
  next unless %fonético{$otra-más-corta}:exists;
  next unless %fonético{$otra-más-corta} eq %fonético{$más-corta};
  say "$palabra $más-corta $otra-más-corta %fonético{$más-corta}"
}

```

A.9 Ejercicio del Capítulo 11: Estructuras de Datos

A.9.1 Ejercicio de la Sección 11.2: La Sentencia Switch given ... when (p. p. 197)

Para usar la sentencia switch con varios valores, podrías escribir algo como esto:

```

for <5 42 43 101 666 1024 2048> -> $valor {
  given $valor {
    when 0..9      { say "$_: Un dígito" }
    when 10..99    { say "$_: Dos dígitos" ; proceed; }
    when 42        { say "$_: Respuesta a la pregunta" }
    when /\d**3$/  { say "$_: Tres dígitos" }
    default        { say "$_: Más de tres dígitos" }
  }
  say ' ';
}

```

Esto mostrará el resultado siguiente:

```

5: Un dígito

42: Dos dígitos
42: Respuesta a la pregunta

43: Dos dígitos
43: Más de tres dígitos

101: Tres dígitos
(...)

```

Puedes ver el error cuando el valor de entrada es 43.

Como una solución, es posible cambiar el orden de las cláusulas when:

```

for <5 42 43 101 666 1024 2048> -> $valor {
  given $valor {
    when 0..9      { say "$_: Un dígito" }

```

```

        when 42      { say "$_: Respuesta a la pregunta" }
        when 10..99  { say "$_: Dos dígitos" ; proceed; }
        when /\d**3$/ { say "$_: Tres dígitos" }
        default      { say "$_: Más de tres dígitos" }
    }
    say '';
}

```

Esto funciona perfectamente ahora, pero la salida para 42 no está ya en el mismo orden. Si queremos mantener el orden original, necesitamos agregar una sentencia when con un bloque vacío:

```

for <5 42 43 101 666 1024 2048> -> $valor {
    given $valor {
        when 0..9      { say "$_: Un dígito" }
        when 10..99    { say "$_: Dos dígitos"; proceed; }
        when 42        { say "$_: Respuesta a la pregunta" }
        when 10..99    { }
        when /\d**3$/  { say "$_: Tres dígitos" }
        default        { say "$_: Más de tres dígitos" }
    }
    say '';
}

```

O podríamos remover la necesidad de proceed al insertar el código para el caso 42 en el bloque de dos dígitos;

```

for <5 42 43 101 666 1024 2048> -> $valor {
    given $valor {
        when 0..9      { say "$_: Un dígito" }
        when 10..99    { say "$_: Dos dígitos";
                        say "$_: Respuesta a la pregunta" if $_ == 42;
                        }
        when /\d**3$/  { say "$_: Tres dígitos" }
        default        { say "$_: Más de tres dígitos" }
    }
    say '';
}

```

Sería igualmente posible anidar la sub-expresión when dentro de la expresión when 10..99:

```

for <5 42 43 101 666 1024 2048> -> $valor {
    given $valor {
        when 0..9      { say "$_: Un dígito" }
        when 10..99    { say "$_: Dos dígitos";
                        when 42 {say "Respuesta a la pregunta";}
                        }
        when /\d**3$/  { say "$_: Tres dígitos" }
        default        { say "$_: Más de tres dígitos" }
    }
    say '';
}

```

A.9.2 Ejercicio de la Sección 11.11: Creando Nuevo Operadores p. 208)

El operador de negación "!" es un operador prefijo (o sea, colocado antes del término que niega). Para el operador factorial, necesitamos un operador sufijo (colocado después del término sobre el cual actúa), así que esta diferencia será suficiente para que el compilador de Raku pueda distinguir entre los dos operadores.

Usamos el metaoperador de reducción para computar el resultado:

```
sub postfix:<!> (Int $n) {
    [*] 2..$n;
}
say 5!; # -> 120
```

La signatura asegura que el operando es un entero (lo cual levanta un error cuando falla). Podemos protegernos de un número negativo, lo cual podemos hacer al levantar un error si un \$n es negativo. Además, podemos usar el módulo estándar Test para automatizar nuestras pruebas:

```
sub postfix:<!> (Int $n) {
    fail "El operando no es un número entero positivo" if $n < 0;
    [*] 2..$n
}
use Test;
plan 5;
dies-ok {(-1)!}, "Factorial falla para -1";
eval-dies-ok "(2.5)!", "Factorial falla para 2.5";
ok 0! == 1, "Factorial 0";
ok 1! == 1, "Factorial 1";
ok 5! == 120, "Factorial de un entero grande";
done-testing;
```

La línea `plan 5;` dice que el plan de prueba contiene cinco pruebas individuales. Después las dos primeras pruebas chequean el operador factorial falla para valores de entrada inválidos. Y chequea la salida por algunos valores de entrada.

La línea `done-testing` especifica que la prueba ha finalizado. Esta función es realmente útil cuando no tienes un plan, por ejemplo cuando no sabes todavía cuantas pruebas ejecutarás. Aquí, tenemos un plan, y el uso de `done-testing` no es necesario.

La siguiente es la salida de las pruebas:

```
1..5
ok 1 - Factorial falla para -1
ok 2 - Factorial falla para 2.5
ok 3 - Factorial 0
ok 4 - Factorial 1
ok 5 - Factorial de un entero grande
```

Si hubiésemos tenido un error de prueba en la prueba 3, habríamos obtenido algo como esto:

```

ok 1 - Factorial falla para -1
ok 2 - Factorial falla para 2.5
not ok 3 - Factorial 0

# Failed test 'Factorial 0'
# at test_fact.raku line 8
ok 4 - Factorial 1
ok 5 - Factorial de un entero grande
1..5
# Looks like you failed 1 test of 5

```

En este ejemplo, hemos colocado las pruebas en el mismo archivo que la definición de la subrutina por la simplicidad del ejemplo. Normalmente, las pruebas estarían en un archivo separado usualmente en un directorio llamado “t” y con una extensión .t en el archivo.

Pruebas y el módulo Test serán discutidos en la Sección 14.8 (p. 318). Más información sobre pruebas pueden encontrarse aquí: <https://doc.raku.org/language/testing>.

A.9.3 Ejercicio de la Sección 11.12: Sets, Bags y Mixes (p. 210)

No podemos solamente reemplazar el %histogram con un bag, porque los *bags* son inmutables (es decir, no pueden modificarse después de su creación) y el hash %histograma es poblado progresivamente a medida que las líneas del libro son leídas desde el archivo. Podrías usar un *baghash* (la versión mutable de un bag) y se te anima que lo intentes.

Sin embargo, el objetivo aquí es extraer las palabras del libro que no se encuentran en la lista de palabras. En otras palabras, no nos importa la frecuencia de las palabras sino que solo necesitamos una lista única de palabras que aparecen por lo menos una vez en el libro, así que un *set* sería suficiente para satisfacer nuestras necesidades. La pregunta es cómo poblamos el set al tiempo de creación.

Podemos cambiar la subrutina procesar-línea para que procese la línea al igual que antes, pero en lugar de poblar un hash, solo devuelve una lista de las palabras. Y podemos crear el set con una función map al llamar esa subrutina:

```

my $saltar = True; # bandera para saltar la cabecera
sub procesar-línea( Str $línea is copy ) {
    $saltar = False if defined index $línea, "*** START OF THIS PROJECT";
    return if $línea ~~ / Abela || anonymous / or $saltar;
    $línea ~~ s:g/<['-]>/ /; # Reemplazar guiones y
                           # apóstrofes con espacios
    $línea ~~ s:g/<[:;,;!¿?<>()."_`]>///; # Remover signos de puntuación
    $línea = $línea.lc; # Convertir cadena a minúscula
    $saltar = True if $línea ~~ /~fin$/; # Marcar el final del libro
    return $línea.words;
}

my $set-libro = set map { procesar-línea $_ }, "quijote.txt".IO.lines;
my $lista-palabras = set "lemario.txt".IO.lines;
my $palabras-desc = $set-libro (-) $lista-palabras;
say $palabras-desc.keys.head(20);

```

Esto funciona bien, pero una vez que hemos hecho eso, podemos también deshacernos de la estructura de datos `$set-libro` y solamente filtrar directamente las palabras extraídas desde el libro:

```
my $saltar = True; # bandera para saltar la cabecera
sub procesar-línea( Str $línea is copy ) {
    # (igual que la anterior)
}

my $lista-palabras = set "lemario.txt".IO.lines;
my @palabras-desc = unique grep {$_ ∉ $lista-palabras},
    grep { $_ },
    map { | procesar-línea $_ },
    "quijote.txt".IO.lines;
say @palabras-desc.head(20);
```

La prueba de tal programa puede tomar algún tiempo, porque tiene que procesar el libro completo cada vez. Para el propósito inicial de prueba, un consejo es reducir el monto de datos de entrada para acelerar las pruebas. Podrías lograr eso al prepara un archivo pequeño con un número limitado de líneas del archivo original *quijote.txt*. Otra forma simple de hacerlo es leer solamente algunas líneas desde el libro, lo cual puedes hacer con una rebanada en la línea de código que leer el archivo. Por ejemplo, para leer solo las primeras 2,000 líneas del libro:

```
my @palabras-desc = unique grep {$_ ∉ $lista-palabras},
    grep { $_ },
    map { | procesar-línea $_ },
    ("quijote.txt".IO.lines)[0..1999];
```

Esto puede también usarse para eliminar la cabecera. Dado que el texto actual del libro comienza en la línea 37, podemos tener algo como esto:

```
my @palabras-desc = unique grep {$_ ∉ $lista-palabras},
    grep { $_ },
    map { | procesar-línea $_ },
    ("quijote.txt".IO.lines)[37..1999];
```

y remover de `procesar-línea` el código para saltar la cabecera. Sin embargo, aún necesitamos una bandera que indique el final del libro:

```
my $final = False; # Marcar el final del texto
sub procesar-línea( Str $línea is copy ) {
    return if $final;
    $línea ~~ s:g/<['-]>/ /; # Reemplazar guiones y apóstrofes con espacios
    $línea ~~ s:g/<[:;,;!¿?«».()"_`]>///; # Remover signos de puntuación
    $línea = $línea.lc; # Convertir cadena a minúscula
    $final = True if $línea ~~ /^fin$/; # Marcar el final del libro
    return $línea.words;
}
```

A.9.4 Ejercicio de la Sección 11.13: Palabras Aleatorias (p. 211)

Hemos creado un módulo `BusquedaBinaria` que contiene una subrutina `bisectar`. Sería grandioso reutilizarla, pero no podemos porque hace actualmente comparaciones de cadenas de texto y necesitamos comparaciones numéricas.

La mejor solución ahora es probablemente hacer una copia de las subrutina y modificarla para hacer comparaciones numéricas. Las subrutinas pueden tener el mismo nombre provisto que sean declaradas como subrutinas multi y tenga signatura diferente: el primer parámetro de la nueva subrutina multi debería ser `Int` en lugar de `Str`. Debido que los cambios hechos son bien minúsculos y fáciles, esto se deja como un ejercicio al lector.

El programa usando ese módulo podría lucir así:

```
use lib ".";
use BusquedaBinaria;
my %histograma;

my $final = False;
sub procesar-línea( Str $línea is copy ) {
    return if $final;
    $línea ~~ s:g/<['-]>/ /;
    $línea ~~ s:g/<[:,;!,¿?«».()"_`>///;
    $línea = $línea.lc;
    $final = True if $línea ~~ /^fin$/;
    return $línea.words;
}

%histograma{$_}++ for grep {$_},
    map { | process-line $_},
    ("quijote.txt".IO.lines)[253..*];
my (@palabras, @frecuencias);
my $frec_total = 0;
for %histograma.kv -> $palabra, $frec {
    $frec_total += $frec;
    push @palabras, $palabra;
    push @frecuencias, $frec_total;
}
my $entero_aleat = $frec_total.rand.Int;
my $idx = bisectar ~$entero_aleat, @frecuencias;
say @palabras[$idx] if $idx >= 0;
```

A.9.5 Ejercicio de la Sección 11.14: Análisis de Markov (p. 213)

Antes de presentar nuestra solución al ejercicio, queremos hacer una introducción breve de una funcionalidad que es útil para extraer y validar los argumentos de la línea de comando pasados a un programa: la subrutina `MAIN`.

A.9.5.1 La subrutina `MAIN`

Los argumentos pasados a un programa son usualmente almacenados en el array especial `@*ARGS`. Puedes navegar los artículos de este array para extraer los argumentos. El

siguiente one-liner es un un ejemplo de esto:

```
$ raku -e 'say $_ for reverse @*ARGS' uno dos tres
tres
dos
uno
```

No obstante, hay otra forma de hacerlo, la subrutina MAIN que discutimos brevemente en la Sección 4.15(p. 69). Si hay una subrutina llamada MAIN en el programa, entonces el programa comenzará a ejecutar esta subrutina, cuyos parámetros serán los argumentos pasados al programa en la línea de comando. Esto significa que la signature de MAIN hará posible la extracción de parámetros y chequear su validez.

En nuestra solución más abajo, la subrutina MAIN está declarada como sigue:

```
sub MAIN( Str $libro, Int $cuenta-palabras, Int $orden = 2,
          Int $linea-inicio = 0) {
    # cuerpo de la subrutina va aquí
}
```

El programa chequeará que los argumentos pasados al mismo coincide con la signature de la subrutina MAIN. En el ejemplo, el primer parámetro tiene que ser una cadena de texto y el segundo un entero; el tercero y el cuarto son opcionales y tendrán valores por defecto de 2 y 0 respectivamente si los argumentos correspondientes no son proveídos.

Si los argumentos pasados al programa no coinciden con la signature de MAIN, el programa terminará después de imprimir un mensaje de uso generado automáticamente:

```
$ raku markov.raku emma.txt 100 2 foo
Usage:
markov.raku <libro> <cuenta-palabras> [<orden>] [<linea-inicio>]
```

El parámetro \$linea-inicio tiene que ser un número entero. Dado que el argumento correspondiente ("foo") no es un entero, el programa muestra un mensaje que detalla el uso del programa.

Validar los argumentos de la línea de comando pasados a un programa puede ser algunas veces una tarea tediosa. Pero, con el mecanismo de signature de la subrutina MAIN, puede usualmente reducirse a una sola línea de código, la signature MAIN.

A.9.5.2 Solución al ejercicio sobre el análisis de Markov

Esta es una manera posible de realizar una análisis de Markov de un archivo de texto:

```
my %prefijos;

sub MAIN( Str $libro, Int $cuenta-palabras, Int $orden = 2,
          Int $linea-inicio = 0)
{
    procesar-línea($orden, $_) for ($libro.IO.lines)[$linea-inicio..*];
    say crear-texto($orden, $cuenta-palabras);
}
```

```

my $final = False;
sub procesar-línea( $orden, Str $línea is copy ) {
    return if $final;
    $línea ~~ s:g/<[-']>/ /;
    $línea ~~ s:g/<[:,!?.()"_`]>\/;/; # removiendo signos de puntuación
    $línea = $línea.lc;                # convirtiendo a minúsculas
    return unless $línea ~~ /\w/;
    procesar-palabras($orden, $línea.words);
    $final = True if $línea ~~ /^fin$/;
}

sub procesar-palabras( $orden, @palabras-nuevas ) {
    state @buffer-palabras = ();
    push @buffer-palabras, |@palabras-nuevas;
    while (@buffer-palabras.elems >= $orden * 2) {
        my $clave = @buffer-palabras.shift ~ " " ~
            (join ' ', @buffer-palabras[0..$orden - 2]);
        my $valor = @buffer-palabras[$orden - 1];
        push %prefijos{$clave}, $valor;
    }
}

sub crear-texto( Int $orden, Int $cuenta-palabra ) {
    my @prefijo = %prefijos.keys.pick.words;
    my $cuenta = 0;
    my $texto = join " ", @prefijo;
    while $cuenta <= $cuenta-palabra {
        my @sufijos-posibles = |%prefijos{join " ", @prefijo};
        last unless @sufijos-posibles;
        my $palabra-nueva = |@sufijos-posibles.pick;
        $texto ~= " $palabra-nueva";
        shift @prefijo;
        push @prefijo, |$palabra-nueva;
        $cuenta++
    }
    return $texto;
}

```

Este programa puede ser llamado en el archivo *quijote.txt* con la siguiente sintaxis:

```
$ raku markov.raku quijote.txt 100 2 253
```

o usando *emma.txt*:

```
$ raku markov.raku emma.txt 50 3 300
```


A.9.6 Ejercicio sobre Código Huffman de la Sección 11.19 (p. 222)

A.9.6.1 La Tabla de Frecuencia (Sección 11.8)

Hemos ya visto problemas similares a este. Esta es una solución posible usando el modelo de programación de tuberías que describimos en la Sección 11.8 (p. 205):

```
my %frecuencias;
%frecuencias{$_}++ for grep {/<[a..z]>/}, map {.lc},
    "goldbug.txt".IO.lines.comb;
my $cuenta_total = [+] values %frecuencias;
say "$_ : \t%frecuencias{$_} \t",
    sprintf "%5.2f", %frecuencias{$_}*100/$cuenta_total
    for reverse sort {%frecuencias{$_}}, %frecuencias.keys;
```

Esto muestra:

e :	7625	13.10
t :	5485	9.42
a :	4477	7.69
o :	4208	7.23
i :	4183	7.18
n :	3912	6.72
s :	3516	6.04
h :	3372	5.79
r :	3278	5.63
d :	2533	4.35
l :	2324	3.99
u :	1893	3.25
c :	1523	2.62
m :	1499	2.57
f :	1392	2.39
w :	1303	2.24
p :	1169	2.01
y :	1146	1.97
g :	1143	1.96
b :	1031	1.77
v :	525	0.90
k :	351	0.60
x :	120	0.21
j :	111	0.19
q :	60	0.10
z :	44	0.08

Recuerda que el personaje de Edgar Allan Poe declaró que la sucesión de letras más comúnmente utilizadas en inglés es como sigue:

e a o i d h n r s t u y c f g l m w b k p q x z

Al parecer el personaje de Poe estaba aproximadamente correcto, pero no del todo cierto, en su estimados de las frecuencias de las letras en un texto en inglés. Aparentemente él

subestimó la frecuencia de la letra “t”. Si ejecutamos el mismo programa con el texto de la novela *Emma* de Jane Austen que hemos usado previamente, obtenemos resultados muy similares:

```
e :      87029    12.57
t :      60035     8.67
a :      54884     7.93
o :      53877     7.78
n :      47773     6.90
i :      47172     6.82
s :      42920     6.20
h :      42819     6.19
r :      41453     5.99
d :      28870     4.17
l :      27971     4.04
(...)
```

A.9.6.2 Código Huffman de una Cadena de ADN (Sección 11.9)

En cada paso del algoritmo, necesitamos buscar las dos letras menos frecuentes. En lugar de tener que iterar a través de todos los artículos en el hash de frecuencia (o ordenar los valores cada vez), usaremos una estructura de datos que mantiene los valores ordenados de acuerdo a nuestras necesidades.

Comenzamos con el hash `%frecuencia` construido en el ejercicio previo y lo transformamos en una colección ordenada de pares que mapean cada letra a su frecuencia respectiva. Creamos una subrutina `insertar-par` que agrega los pares creados recientemente (las letras ficticias) en el lugar correcto del array de pares para mantener el array ordenado de acuerdo a nuestras necesidades:

```
my %código;
my @pares;
push @pares, $_ => %frecuencias{$_} for
    sort {%frecuencias{$_}}, %frecuencias.keys;

sub insertar-par( @lista, $nuevo-elemento) {
    my $val = $nuevo-elemento.value;
    for @lista.keys -> $i {
        if @lista[$i].value >= $val {
            splice @lista, $i, 0, $nuevo-elemento;
            return;
        }
    }
    push @lista,$nuevo-elemento; # poner el elemento nuevo al final
                                # de la lista si el lugar correcto
                                # no se encontró anteriormente
}
```

Iteramos sobre los pares, elegimos los dos con las frecuencias más pequeñas, los unimos en un par nuevo, y agregamos dicho par en lugar correcto con la subrutina `insertar-par`. El bucle termina cuando hay solamente dos pares restantes. Al mismo tiempo, poblamos en cada paso del bucle el nuevo hash `%código` con los códigos parciales encontrados:

```

loop {
  my $menor1 = shift @pares;
  my $menor2 = shift @pares;
  my $par-nuevo = $menor1.key ~ $menor2.key => $menor1.value + $menor2.value;
  insertar-par @pares, $par-nuevo;
  %código{$menor1.key} = $menor1.key ~ $menor2.key ~ "|.";
  %código{$menor2.key} = $menor1.key ~ $menor2.key ~ "|-";
  last if @pares <= 2;
}
%código{@pares[0].key} = ".";
%código{@pares[1].key} = "-";

```

Al final del bucle, el array de pares contiene dos pares:

```
[c => 10 tga => 11]
```

y el hash %código contiene los códigos parciales para letra o letra ficticia:

```
{a => ga|-, c => ., g => ga|., ga => tga|-, t => tga|., tga => -}
```

Después usamos otro bucle para sustituir las letras falsas y deshacernos de ellas, hasta que quedemos con las letras actuales de la cadena de texto original:

```

loop {
  my $hecho = True;
  for %código.keys -> $letra {
    next if $letra.chars > 1;
    my ($val, $código) = split '|', %código{$letra};
    next unless defined $val and defined $código;
    $hecho = False;
    my $result = %código{$val} ~ $código;
    %código{$letra} = $result;
  }
  last if $hecho;
}
my %codificar;
%codificar{$_} = %código{$_} for grep {$_ .chars < 2 }, %código.keys;

```

El hash %codificar contiene la tabla de Huffman:

```

c => .
t => -.
g => --.
a => ---

```

A.9.6.3 Código Huffman para una Cadena de Texto Más Compleja (Sección 11.9)

Para esta pregunta, necesitaremos un párrafo pequeño escrito especialmente con un solamente algunas letras del alfabeto:

Eastern Tennessee anteaters ensnare and eat red ants, detest ant antennae (a tart taste) and dread Antarean anteater-eaters. Rare Andean deer eat tender sea reeds, aster seeds and rats' ears. Dessert? Rats' asses.

Como primer paso, simplificaremos un poco el problema al convertir todas la letras a minúsculas y usar solo las letras, eliminando espacios y puntuación de la computación de la tabla de frecuencia:

```
my $cadena = "Eastern Tennessee anteaters ensnare and eat red ants, detest ant
antennae (a tart taste) and dread Antarean anteater-eaters. Rare
Andean deer eat tender sea reeds, aster seeds and rats' ears. Dessert?
Rats' asses.";

my %frecuencias;
%frecuencias{$_}++ for grep { /\w/ }, $cadena.lc.comb;
```

Este tratado muy elocuente sobre el hábito alimenticio de varios animales produce la tabla de frecuencias siguiente:

e :	40	23.53
a :	32	18.82
t :	24	14.12
s :	22	12.94
n :	20	11.76
r :	19	11.18
d :	13	7.65

Usando el mismo código al igual que en la pregunta anterior genera la tabla de Huffman siguiente:

```
a => ..
e => .-
s => -.-
n => -..
t => --.
d => ---.
r => ----
```

A.9.6.4 Codificando la Cadena de Texto de Entrada (Sección 11.9)

No solo queremos codificar una cadena de texto de entrada con el código Huffman sino que también queremos ser capaces de decodificarla y reconocer la entrada original. Debido a eso, ya no queremos filtrar la puntuación de la tabla de traducción, la cual será más grande que antes. Los espacios (espacios horizontales y retornos de línea) serán manejados diferentemente: no los modificaremos en la cadena de texto codificada de pseudo-Morse, dado que esto hará más fácil de chequear y mostrar el resultado.

La tabla de frecuencias ahora incluye signos de puntuación que existen en la cadena de texto de entrada:

```
%frecuencias{$_}++ for grep {/(<[\w] + [.,()'?->\/]), $cadena.lc.comb;
```

La tabla de frecuencias tiene 14 entradas ahora:

e :	40	22.10
a :	32	17.68
t :	24	13.26
s :	22	12.15
n :	20	11.05
r :	19	10.50
d :	13	7.18
. :	3	1.66
, :	2	1.10
' :	2	1.10
) :	1	0.55
- :	1	0.55
? :	1	0.55
(:	1	0.55

Y la tabla de Huffman (hash %codificar) luce así:

```
e => .-
a => ---
s => -..
n => ...-
t => --.
r => ...
d => -.--
. => -.-.-.
( => -.-.-.-.
' => -.-.-.-.
? => -.-.-.-.
) => -.-.-.-.
- => -.-.-.-.
, => -.-.-.-.
```

La subrutina de codificación es bastante simple:

```
sub codificación( Str $entrada, %codificar ) {
    my $salida;
    for $entrada_lc_comb -> $letra {
        $salida ~= %codificar{$letra} // $letra;
    }
    return $salida;
}
```

Cada letra de la entrada es convertida a letras minúsculas (dado que hemos limitado nuestra tabla a minúsculas), traducida a su código equivalente de pseudo-Morse, y concatenada a la cadena de texto de salida. Si la letra no se encuentra en el hash %codificar, entonces se almacena en la salida tal como es: esto posibilita la inserción de espacios y caracteres de final de línea en la cadena de texto de salida.

Aquí presentamos el resultado (formateado para adaptarlo al libro):

Interesantemente, la cadena de texto de entrada tiene 213 caracteres y la cadena de texto de salida tiene 589 bits. Si estuviéramos almacenando 14 caracteres diferentes de la cadena de entrada con códigos de igual longitud, necesitaríamos cuatro bits por carácter, lo cual requeriría 1052 bits. Así que el código Huffman logró esto: un razón de compresión 1.78 mejor que los mejores códigos de igual longitud. Y la codificación ASCII de la cadena de texto de entrada requirió 213 bytes, es decir, 1704 bits; la salida codificada requirió menos de tres veces menos que eso.

Para decodificar eficientemente la cadena de texto pseudo-Morse, necesitamos invertir la tabla de Huffman, es decir, crear un hash en el cual los códigos pseudo-Morse son las claves y las letras son los valores. Invertir el hash %codificar es algo bien directo:

La expresión `%codificar .kv` produce una lista de claves y valores, y la sentencia `reverse` la transforma en una lista de valores y claves. Asignar esa lista a un hash nuevo produce un hash nuevo en el cual las claves y los valores son intercambiados. Nota que esto funciona porque sabemos que los valores son únicos, así que no hay ningún problema de duplicados cuando los convertimos en claves del hash.

Por ejemplo, con el inicio de la cadena de texto pseudo-Morse:

el primer punto no es una entrada pero la combinación ". ." es una "e". La siguiente raya no es una entrada y tampoco lo es "--", pero "---" es una "a".

La siguiente raya no es una entrada y tampoco lo es "-.", pero "-.." es una "s". Similarmen- te, los tres caracteres siguientes, "--.", forman una "t", y podemos decodificar la letra "eastern".

Podríamos implementar esto con dos bucles anidados: uno itera a través de la cadena de texto y el segundo consume el número necesario de puntos y rayas hasta el final de una letra:

```
sub decodificación( Str $entrada, %decodificar ) {
    my @códigos = $entrada.comb;
    my $salida;
    loop {
        last unless @códigos;
        my $actual = shift @códigos;
        $salida ~= $actual and next if $actual =~ /\s/;
        $salida ~= %decodificar{$actual} and
            next if %decodificar{$actual}:exists;

        loop {
            # necesitamos más caracteres para completar la letra
            $actual ~= shift @códigos;
            if %decodificar{$actual}:exists {
                $salida ~= %decodificar{$actual};
                last;      # finalizamos con una letra,
                           # devuelta al bucle principal
            }
        }
    }
    return $salida;
}
```

Esto funciona apropiadamente y la salida es la misma que la entrada original (excepto por el hecho de que todas las letras son minúsculas):

```
eastern tennessee anteaters ensnare and eat red ants, detest ant
antennae (a tart taste) and dread antarean anteater-eaters. rare
andean deer eat tender sea reeds, aster seeds and rats' ears. dessert?
rats' asses.
```

No obstante, si lo analizas, no necesitamos dos bucles anidados en la subrutina decodificación, la cual podemos hacerla más concisa en la siguiente manera:

```
sub decodificación( Str $entrada, %decodificar ) {
    my ($salida, $actual);
    for $entrada.comb -> $cod-ent {
        $salida ~= $cod-ent and next if $cod-ent =~ /\s/;
        $actual ~= $cod-ent;
        if %decodificar{$actual}:exists {
            $salida ~= %decodificar{$actual};
            $actual = "";
        }
    }
    return $salida;
}
```

Aquí, la variable `$actual` acumula los puntos y las rayas de la cadena de entrada hasta se determina que es una entrada en la tabla de traducción, en cuyo momento se reinicia a una cadena de texto vacía para prepararla para la letra siguiente.

La solución presentada más arriba para encontrar el código Huffman usa la subrutina `insertar-par` para mantener el array de pares `@pares` ordenado. Esto hace fácil encontrar las letras restantes menos comunes o pseudo-letras. Podrías recordar de la Sección 11.16.3 que los montículos binarios son estructuras de datos grandiosas cuyo objetivo es acceder rápidamente los artículos más pequeños de una colección. Como un ejercicio adicional, podrías escribir la solución nuevamente usando un montículo binario. La solución original de David Huffman actualmente usó un árbol (llamado el árbol de Huffman) muy similar a un montículo binario.

A.10 Ejercicios del Capítulo 13: Regexes y Gramáticas

A.10.1 Ejercicio de la Sección 13.1: Obteniendo las Fechas de Febrero Correctamente (p. 271)

Queremos chequear si las fechas de febrero son válidas.

Para comenzar, excluyamos las fechas de febrero que son mayores que 29. Esto puede hacerse al expandir la aserción de código mostrada en el código para reconocer las fechas:

```
my $cadena = "Día bisiesto : 2016-02-29.";
my token año { \d ** 4 }
my token mes {
    1 <[0..2]>                # 10 a 12
    || 0 <[1..9]>              # 01 a 09
};
my token día { (\d ** 2) <?{1 <= $0 <= 31 }> }
my token sep { '/' || '-' }
my rule fecha { [
    <año> (<sep>) <mes> $0 <día>
    || <día> (<sep>) <mes> $0 <año>
    || <mes>\s<día>',' <año>
] <![ ($<día> > 30 and $<mes> == 4|6|9|11) or
    $<día> > 29 and $<mes> eq '02' ]>
}

if $cadena ~~ /<fecha>/ {
    say ~$/;                # 2016-02-29
    say "Día\t= " , ~$/<fecha><día>;    # 29
    say "Mes\t= " , ~$/<fecha><mes>;    # 02
    say "Year\t= " , ~$/<fecha><año>;    # 2016
}
```

Esto funciona. Febrero tiene 29 días dado que 2016 es un año bisiesto. Pero este código validaría Feb. 29 para 2015 o 2017, lo cual es incorrecto dado que no son años bisiestos.

A.10.1.1 Reconociendo un año bisiesto

En el calendario Juliano antiguo (en honor a Julio César), los años bisiestos son años que son divisibles por 4. Resultó que el calendario Juliano tenía muchos años bisiestos para reflejar la realidad astronómica, así que el calendario retrocedió alrededor de 3 días por cada período de cuatro siglos.

El calendario Gregoriano, introducido por el Papa Gregorio XIII en 1582, corrigió el calendario Juliano con la siguiente regla adicional: años divisibles por 100 deberían considerarse años bisiestos solo si son divisibles por 400. Así que, para el calendario Gregoriano, 1700, 1800, 1900, y 2100 no son bisiestos pero 2000 y 2400 sí lo son.

Dependiendo del tipo de fechas que tu programa encontrará, puedes simplificar las reglas. Si estás escribiendo un módulo que está supuesto a ser preciso con cualquier fecha del pasado o futuro, probablemente quieres implementar la regla Gregoriana exacta. Pero si sabes que vas a encontrar fechas del período actual, podrías elegir una regla mucho más simple.

En particular, dado que 2000 es una excepción a la excepción y es bisiesto, cualquier año entre 1901 y 2099 es bisiesto si es divisible por 4 y no bisiesto por el contrario. Esta regla es probablemente suficiente para cualquier aplicación de negocios escrita en 2017. Probablemente no hay razón para hacerlo más complicado de lo que necesita ser (aunque podría argumentarse que es el tipo de razonamiento que condujo al gran terror del error “Y2K”).

Con esta simplificación en mente, una subrutina para encontrar si un año es bisiesto debería simplemente devolver verdadero si es divisible por 4 y podría lucir así:

```
sub es-bisiesto( $año ) { # funciona para los años entre 1901 y 2099
    return True if $año %% 4;
    return False;
}
```

O más simple:

```
sub es-bisiesto( $año ) { # funciona para los años entre 1901 y 2099
    return $año %% 4;
}
```

Si quieres implementar la regla Gregoriana completa, podría lucir así:

```
sub es-bisiesto( $año ) { # regla Gregoriana para cualquier año
    return False if $año % 4;      # no si no es divisible por 4
    return True  if $año % 100;    # sí si divisible por 4 y no por 100
    return False if $año % 400;    # no si divisible por 100 y no por 400
    return True;                   # sí si divisible por 400
}
```

o, si te gusta ser conciso (o la ofuscación):

```
sub es-bisiesto($a) { $a %% 400 or ($a %% 4 and not $a %% 100) }
```

El código más arriba es dado como un ejemplo acerca de cómo computar si un año es bisiesto, dado que es un problema interesante y clásico, pero Raku actualmente provee un método para eso en el rol `Dateish`. Por ejemplo:

```
> say DateTime.new(:year<2020>).is-leap-year;
True
> say DateTime.new(:year<2019>).is-leap-year;
False
```

A.10.1.2 De Regreso a la validación de fechas de febrero

Puedes agregar las reglas para Feb. 29 en el ejemplo de código más arriba, pero sugerimos que esto se está complicando para una aserción de código dentro de la regla fecha: agregar una condición Booleano en una aserción de código dentro de una regla está bien, pero cuando la condición se vuelve más complicada, esto causa que la regla sea difícil de entender. Piensa sobre la persona que tendrá que mantener tu código dentro de una año (y esa persona podría ser tú).

Preferimos mover el código que realiza la validación fuera de la regla fecha a una subrutina dedicada a chequear todas las fechas para febrero:

```
sub feb-fecha-inválida( $año, $día ) {
  return False if $día <= 28;
  return True  if $día > 29;
  return False if DateTime.new(year => $year).is-leap-year;
  True;
}
```

La regla fecha lucí ahora así:

```
my rule fecha { [ <año> (<sep>) <mes> $0 <día>
  || <día> (<sep>) <mes> $0 <año>
  || <mes>\s<día>', ' <año>
] <![{ ($<día> > 30 and $<mes> == 4|6|9|11) or
  $<mes> eq '02' and feb-fecha-inválida $<año>, $<día>}>
}
```

Originalmente llamé la subrutina nueva chequear-feb-29 pero la cambié a feb-fecha-inválida para mostrar mejor que devuelve un valor verdadero si la fecha no es válida. Esto puede parecer secundario, pero elegir nombres buenos para tus identificadores es importante porque eso documenta tus programas y clarifica sus semánticas.

Una vez que hemos introducido esta mínima subrutina, podríamos ir un paso más allá y mover el resto de la aserción de código dentro de la subrutina, para que la aserción de código final contenga solo una llamada a la nueva versión de la subrutina. Esto se deja como un ejercicio adicional al lector.

A.10.2 Ejercicio 13.12 (p. 286): Una Gramática para una Calculador Aritmética

Aquí presentamos una forma posible de implementar una calculadora aritmética.

A.10.2.1 La Gramática

Esta es una manera de escribir la gramática:

```
my grammar Calculadora {
  rule TOP          { <expr> }
  rule expr         { <término> + % <op-suma-sust> }
  token op-suma-sust { [< + - >] }
  rule término      { <átomo> + % <op-mult-div> }
  token op-mult-div  { [< * / >] }
  rule átomo {
    | <número>      { make +$<número> }
    | <expr-paren> { make $<expr-paren>.made }
  }
  rule número       { <signo> ? [\d+ | \d+\.\d+ | \.\d+ ] }
  rule expr-paren    { '(' <expr> ')' }
  token signo        { [< + - >] }
}
```

Esta solución es bien simple.

Una expresión (*expr*) está compuesta por uno o varios términos separados por los operadores “+” o “-”. Un término está compuesto por uno o varios átomos separados por los operadores “*” o “/”. Un átomo puede ser un número literal o una expresión entre paréntesis.

Esto garantiza que las reglas de precedencia sean satisfechas. Las multiplicaciones y las divisiones serán evaluadas antes que las adiciones y las sustracciones, dado que, al analizar sintácticamente una expresión, necesitas los términos individuales antes de completar la evaluación de la expresión. Similarmente, dado que la expresión entre paréntesis es un átomo, tendrá que ser evaluada antes que el término en el cual aparece sea completamente evaluado. Nota que, en el caso de una expresión entre paréntesis, la regla *expr* es llamada recursivamente.

A.10.2.2 Las Acciones

Nota que hemos incluido dos acciones en la gramática (en la regla *átomo*). Una razón para hacer eso es por conveniencia: dado que la regla *átomo* cubre dos sub-reglas nombradas diferentes, es un poco más fácil incluir la acción solo en el contexto de las sub-reglas. Si una acción se había adjuntado a la regla *átomo*, la regla hubiese requerido encontrar cual sub-regla había coincidido para saber cual acción debía realizar. Nada difícil, pero hacer esto habría hecho el código mucho más complejo. La otra razón para hacerlo es por propósito pedagógicos: aunque algunas veces hace sentido crear una clase de acciones, es igualmente útil saber que las acciones pueden insertarse en la parte de la gramática. Para una gramática bien simple, podría ser innecesario crear una clase de acciones con solo una o dos acciones.

La clase de acciones podría lucir así:

```
class AccionesCalc {
  method TOP ($/) {
    make $<expr>.made
```

```

    }
    method expr ($/) {
        $.calcular($/, $<término>, $<op-suma-sust>)
    }
    method término ($/) {
        $.calcular($/, $<átomo>, $<op-mult-div>)
    }
    method expr-paren ($/) {
        make $<expr>.made;
    }
    method calcular ($/, $operandos, $operadores) {
        my $result = (shift $operandos).made;
        while my $op = shift $operadores {
            my $número = (shift $operandos).made;
            given $op {
                when '+' { $result += $número; }
                when '-' { $result -= $número; }
                when '*' { $result *= $número; }
                when '/' { $result /= $número; }
                default { die "operador desconocido"}
            }
        }
        make $result;
    }
}

```

El método `calcular` computa las expresiones (términos separados por los operadores de adición o sustracción) y términos (átomos separados por los operadores de multiplicación o división) desde la izquierda hacia la derecha dado que los operadores son asociativos por la izquierda.

Esta gramática para una calculadora y su asociada clase de acciones puede probarse con el código siguiente:

```

for |< 3*4 5/6 3+5 74-32 5+7/3 5*3*2 (4*5) (3*2)+5 4+3-1/5 4+(3-1)/4 >,
    "12 + 6 * 5", " 7 + 12 + 23", " 2 + (10 * 4) ", "3 * (7 + 7)" {
    my $result = Calculadora.parse($_, :actions(AccionesCalc));
    # say $result;
    printf "%-15s %.3f\n", $_, $result.made if $result;
}

```

lo cual mostrará los resultados siguientes:

3*4	12.000
5/6	0.833
3+5	8.000
74-32	42.000
5+7/3	7.333
5*3*2	30.000
(4*5)	20.000
(3*2)+5	11.000

4+3-1/5	6.800
4+(3-1)/4	4.500
12 + 6 * 5	42.000
7 + 12 + 23	42.000
2 + (10 * 4)	42.000
3 * (7 + 7)	42.000

Puede preguntarte si este código funciona correctamente con expresiones entre paréntesis anidadas. Originalmente pensé, cuando escribí este código, que podría tener un fallo y que tal vez necesitaba cambiar o añadir algo para conseguir que las expresiones entre paréntesis funcionaran correctamente. Por ejemplo, considera la siguiente prueba de código con expresiones entre paréntesis relativamente anidadas:

```
for "(((2+3)*(5-2))-1)*3", "2 * ((4-1)*((3*7) - (5+2)))" {
  my $result = Calculadora.parse($_, :actions(AccionesCalc));
  printf "%-30s %.3f\n", $/, $result.made if $result;
}
```

El resultado es correcto:

(((2+3)*(5-2))-1)*3	42.000
2 * ((4-1)*((3*7) - (5+2)))	84.000

Como un ejercicio adicional, podrías añadir la potenciación a la lista de operadores permitidos. Recuerda que la potenciación tiene mayor precedencia que la multiplicación y la división (así que probablemente quieres poner dicho operador cerca del nivel del átomo). En el evento que quieras manejar operadores de potenciación anidados, también recuerda que usualmente son asociativos por la derecha:

$2^{3^{3^2}} = 2^{3^{(3^2)}} = 2^{81} = 512; \quad \# \text{ No: } (2^{3^3})^{3^2} \text{ o } 64$

A.11 Ejercicios del Capítulo 14: Programación Funcional

A.11.1 Ejercicio 14.10: Creando una Implementación Funcional de Ordenamiento Rápido

Aquí presentamos una manera de implementar el algoritmo de ordenamiento rápido en un estilo de programación funcional.

```
sub ord-rapido( @entrada ) {
  return @entrada if @entrada.elems <= 1;
  my $pivote = @entrada[@entrada.elems div 2];
  return flat ord-rapido(grep {$_ < $pivote}, @entrada),
    (grep {$_ == $pivote}, @entrada),
    ord-rapido(grep {$_ > $pivote}, @entrada);
}
```

Esta versión funcional del programa refleja directamente la estrategia del algoritmo de ordenamiento rápido:

- Si el array tiene menos de dos artículos, ya está ordenado, y por lo tanto devuélvelo inmediatamente (este es el caso base que parará la recursión).
- Por el contrario, elige un artículo como un pivote (aquí, nosotros elegimos el elemento del medio o uno inmediatamente cerca del medio).
- Divide el array en tres sub-listas que contienen artículos respectivamente más pequeños, mayores que e iguales al pivote.
- Ordena la dos primeras sub-listas usando una llamada recursiva a la función `ord-rapido`, pero no llame a `ord-rapido` sobre una sub-lista que contiene artículos iguales al pivote: a parte de estar ya ordenado (todos los elementos son iguales), fallaría en satisfacer el caso base y entraría en una recursión infinita.
- Devuelve la lista obtenida al concatenar las sub-listas.

Como se mencionó previamente, el pivote ideal sería la mediana de los valores, pero el costo de encontrar la mediana sería exorbitante.

En principio, podrías elegir cualquier artículo como el pivote, incluyendo el primer o último elemento del array. Pero para entradas específicas (tales como arrays que ya están casi ordenados, de forma ascendente o descendente), esto puede incrementar significativamente el tiempo de ejecución por las particiones (o divisiones) se vuelven desequilibradas, y por lo tanto perdiendo la ventaja de la estrategia de dividir y conquistar. Elegir un elemento en el medio, como hicimos aquí, reduce la probabilidad de tal comportamiento patológico. Otra posible forma posible de prevenir tal riesgo es seleccionar el pivote aleatoriamente entre los elementos del array.

Índice

- índice, 107, 130, 132, 151, 177
 - comienza con cero, 108, 109, 151–153
 - slice, 152
- ámbito, 41, 371
 - dinámico, 307
 - léxico, 41, 97, 272, 291, 292, 307
- árbol, 216
 - binario, 216, 307
 - de sintaxis abstracta (AST), 274, 280–282, 286
 - hoja, 216
 - nodo, 216
 - parse, 274, 280
 - tronco, 216
- épsilon, 103
- != (desigualdad numérica), 57
- <=> (comparación numérica), 57
- <= (numérico menor que o igual a), 57
- < (numérico menor que), 57
- === (identidad de valor), 56, 57
- => constructor de par, 57
- >= (numérico), 57
- > (numérico mayor que), 57
- cmp (comparación alfabética), 57
- eqv (alfabética y verdaderamente equivalente), 57
- eq (igualdad de cadena de texto), 57
- ge (alfabéticamente posterior o igual), 57
- grep, 164
- gt (alfabéticamente posterior), 57
- le (alfabéticamente inferior o igual), 57
- lt (alfabéticamente inferior), 57
- ne (desigualdad de cadena de texto), 57
- año bisiesto, 271, 409, 410
- abecedario, 114, 141
- Abelson, Harold, 91
- abstracción, 6, 49, 231
- acceso, 151
- accesor, 235, 238
- acciones
 - clase, 275, 280, 281, 285, 286
 - objeto, 275, 280, 281
- acertijo, 147, 148
- Ackermann
 - función, 194
- Ackermann, Wilhelm, 346
- acto de fe, 84
- actualización, 94, 102, 104
- acumulado
 - histograma, 203
- acumulador, 173
 - lista, 163, 376
 - suma, 161
- adición
 - sin cargar, 103
- ADN (ácido desoxirribonucleico), 224, 403, 404
- adverbio, 111, 123, 129
 - :delete, 180, 373
 - :exists, 179
 - :i, 123
 - :ignorecase, 123, 359
 - :r, 124
 - :ratchet, 124, 269
 - :s, 124, 127
 - :sigspace, 124, 269
 - :v, 111
- Aho, Alfred, 116
- alfabeto, 224, 225
 - braille, 222
- algoritmo, 103, 104, 211
 - de división y conquista, 315, 323
 - de Euclides, 349, 351
 - división y conquista, 415
 - raíz cuadrada, 105
- alias, 173, 196
- alinear a la derecha, 333

- all junction, 198
- alternación, 266
- alternancia, 122
 - de la coincidencia más larga, 122
 - de primera coincidencia, 122
- ambigüedad, 12
- análisis
 - de Markov, 211, 213, 399, 400
 - gramático, 266
 - HTML, 271
 - léxico, 266
 - semántico, 285
 - sintáctico, 12, 14, 271, 285
 - XML, 271
- anagrama, 174, 378
- ancla
 - al final de cadena de texto, 120
 - al final de la línea, 121
 - al final de la palabra, 121
 - al inicio de cadena de texto, 120
 - al inicio de la línea, 121
 - al inicio de la palabra, 121
 - borde de la palabra, 121
 - borde derecho de la palabra, 121
 - borde izquierdo de la palabra, 121
- anclas, 120
- andamiaje, 78, 89, 193
- animal
 - feral, 247, 248
 - mascota, 247, 248
- anulando un método, 242, 244, 256
- any junction, 198
- apóstrofo, 20, 24, 37
- archivo
 - de configuración, 266
 - escribir a un, 137
 - leer desde un, 137
 - sentencia close, 137
 - sentencia open, 137
- argumento, 32, 37, 40, 41, 50
- aridad, 87, 237
- array, 138, 149
 - de tamaño fijo, 166
 - moldeado, 166
 - multidimensional, 167, 391
 - tipado, 166
- artículo, 132, 149, 173, 177
 - hash, 193
- ASCII, 19
- aserción, 120
 - de código, 121, 126, 128, 270, 272
 - de código negativa, 121
 - look after, 121
 - look around, 121
 - look before, 121
 - look-around, 121
- asignación, 17, 21, 27, 94
 - de artículo, 152
 - de variable, 17
 - operador, 93
- atributo, 257
 - clase, 263
 - inmutable, 235, 254
 - instancia, 234, 263
 - mandatorio, 328
 - mutable, 235, 254
 - objeto, 232
 - público, 254, 255
 - privado, 235, 253–255
- autoboxing, 232
- awk, 116
- búsqueda, 132, 183, 193
 - binaria, 174, 211, 383, 385
 - con hash, 183
 - del intervalo medio, 383
 - hash, 183
 - inversa, 183, 184, 193
 - inversa de hash, 183
 - logarítmica, 385
 - patrón de, 132, 141
 - por bisección, 174
 - secuencial, 179
- baby Perl, 81
- backtracking, 262
- bag, 208, 211, 397
- baghash, 208, 397
- bandera, 191, 193
- barra baja, 11, 19
- benchmarking, 214, 221
- bisección, 175
- bisección, depuración por, 104
- bit, 225
- bloque
 - de código, 48
 - for, 197
 - puntiagudo, 65, 290, 357
- bosque, 216
- bucle, 96, 160
 - al estilo de C, 160

- for, 64, 65, 67, 101, 113, 158, 160, 238, 290, 357, 385, 389
- infinito, 96, 100, 104, 161, 330, 360, 367, 391
- palabra clave, 160
- recorrido, 113
- sentencia, 160
- until, 97
- while, 95, 113, 353
- byte, 225
- cálculo lambda, 290
- código
 - andamiaje, 327
 - de fuente abierta, 49
 - de longitud variable, 222
 - morse, 222, 223
 - muerto, 76, 89, 327
 - reuso de, 248
- cabecera, 37, 50
- cache, 171, 188, 189, 310, 322
- cadena de texto, 9, 14, 107, 214, 353
 - concatenación, 25, 111
 - longitud, 108
 - operación, 25
 - operador de relación, 57
 - operadores, 108, 110
 - recorrido inverso, 353, 355
 - vacía, 108, 132
- Caesar, Julius, 410
- caja negra, 6, 49, 233, 253
- calculadora, 15, 29, 286, 411
 - aritmética, 274, 275, 286, 411
 - gramática, 286
- calendario
 - Gregoriano, 410
 - Juliano, 410
- candidato
 - subrutinas multi, 87
- canino, 246
- captura, 266, 285, 359, 364
 - regex, 266
- capturas, 115, 123, 129, 267
 - enumeradas, 123
 - nombradas, 267, 269, 281
 - numeradas, 267
- Car Talk, 147, 148, 194, 364–366
- carácter, 107
 - comodín, 118
 - de nueva línea, 137
- carnívoro, 246
- caso
 - base, 67, 72, 85, 91, 318, 415
 - corner, 147, 349
 - edge, 147, 349
 - especial, 146, 147
 - kebab, 20
 - mayúscula, 19
 - minúscula, 19
 - título, 19
- categoría
 - de caracteres, 118, 265, 357–359
 - de caracteres negada, 119
 - de clases, 127, 133
- cero, índice comienza con, 108
- Cervantes, Miguel de, 250
- Chekhov, Anton, 250
- chequeo
 - de consistencia, 192
 - de error, 85
- Church, Alonzo, 290, 312
- cifrado, 223
- cifrado César, 135, 194, 262, 362
- ciudadano de primera clase, 287, 322
- clase, 231–233, 248, 249, 263
 - atributo, 236, 263
 - de acciones, 271, 412
 - definición, 233
 - herencia, 241
 - hija, 242, 244, 263
 - padre, 242, 244, 245, 263
 - Pixel, 241–243
 - Punto2D, 233, 234, 236
 - Punto3D, 252
 - PuntoMovable, 243
 - Rectángulo, 239
 - subclase, 242, 244
- clausura, 99, 290, 292, 303, 322, 371, 384
 - whatever, 311
- clave, 177, 193
- coerción, 25, 26, 33, 57, 151, 153, 169, 209, 379, 380
 - tipo, 45
- coincidencia
 - de cadena de texto, 116
 - inteligente, 197
 - literal, 118
- cola, 215
- coma, 11
- coma flotante, 9, 103

- comentario, 26, 28, 326, 353
 - multilínea, 353
- comillas, 10, 24
 - alemanas, 294
 - angulares, 180
 - dobles, 37, 61, 277
 - francesas, 294
 - simples, 20, 37, 61
- compilación, 267
- compilador, 13
- complejidad algorítmica, 301, 322
- composición, 36, 40, 51, 79, 334
 - objeto, 239, 263
- composición de función, 79
- compresión de datos, 225
- comprobación de validez, 192
- comuntatividad, 26
- Conan Doyle, Arthur, 49
- concatenación, 28, 41, 114
 - de cadenas de texto, 25, 111
- condición, 60, 71, 96
- condicional
 - anidada, 71
 - ejecución, 60
 - encadenada, 71, 345
 - sentencia, 60, 71
 - sufija, 63
- condicionales
 - anidadas, 62
 - encadenadas, 61
- conjetura de Collatz, 96
- conjunto, 9
- constante, 327
- construcción
 - gather y take, 306
- construcción de operadores, 207
- constructor, 234
 - de pares, 57, 178, 199
 - new, 255, 256
 - personalizado, 257
- contador, 114, 132, 133, 181, 182, 290
- contando letras, 360, 361
- contar y recorrer, 114
- contexto
 - escalar, 185
- conversión
 - tipo de, 32
- coordenadas
 - cartesianas, 77, 233, 237, 257
 - esféricas, 252
 - polares, 237, 245, 252, 257
 - rectangulares, 77, 233
- corchetes, 60, 107, 123
- corrida, 15
- creando nuevos operadores, 207, 296–298, 300
- creando un operador, 396
- crucigramas, 139
- cuadrícula, 52
- cuadro, 67, 83, 189
 - de una función, 67, 189
- cuantificador, 119, 127, 266, 358, 359
 - de rango, 120
 - frugal, 120
 - modificador, 278
 - número preciso de veces, 120
 - voraz, 120
- cuerpo, 37, 50, 96
- currificar, 311–313, 322
- Curry, Haskell, 312
- de Cervantes, Miguel, 203
- declaración
 - variable, 17
- declarador
 - my, 18, 41, 97, 99, 272
 - our, 99
 - state, 191
- declarando variables, 18
- defined, 180
- definición
 - circular, 82
 - función, 37
 - recursiva, 82
- delegación, 250, 259, 263
- delimitador, 111, 129
- depuración, 13, 14, 27, 69, 88, 130, 146, 172, 192, 220
 - con patito de goma, 220, 222
 - experimental, 49, 221
 - por bisección, 104
 - respuesta emocional, 13
- depurador, 220, 221, 259, 285, 317
 - accediendo variables, 261
 - ayuda, 260
 - depurando una gramática, 285
 - ejecutando código dentro de una subrutina, 260
 - ejecutando código fuera de una subrutina, 260

- ejecutando código paso a paso, 260
- ejecutando el depurador de Raku, 260
- el depurador de Raku, 260
- punto de interrupción, 261
- recorriendo la coincidencia de un regex, 285
- recorriendo una coincidencia Regex, 262
- trace point, 261
- usando el depurador de Raku, 260
- uso, 259
- dequeue, 367, 368, 370, 371
- desarrollo
 - guiado por pruebas, 321, 322
 - incremental, 77, 89
- descriptor de archivo, 137
- despacho
 - candidato, 87
 - múltiple, 87
 - subrutinas multi, 87
- desplazamiento, 109
- determinista, 201, 221
- diagrama
 - de estado, 27, 94, 131, 234, 240
 - de objeto, 234, 240, 263
 - de pila, 42, 51, 67, 83, 90
 - estado, 18, 187
 - gráfica de llamada, 193
- Diccionario de Pronunciación CMU, 194
- Dijkstra, Edsger, 147, 326, 327
- dirección IP
 - extracción de, 127
- diseño orientado a objetos, 257
- diversión, 214
- división
 - de coma flotante, 56
 - de enteros, 55, 56, 71
 - euclidiana, 55
 - por cero, 71
- divisibilidad, 56, 81, 329
 - operador, 56
- do-twice, 51, 335
- Don Carlos, 250
- Don Quijote, 203, 250
- DRY: don't repeat yourself, 326
- duplicado, 174, 185, 194, 379, 380, 391
 - chequeo, 184
- editor de texto
 - atom, 22
 - eclipse, 22
 - emacs, 22
 - gEdit, 22
 - nano, 22
 - notepad++, 22
 - padre, 22
 - vi, 22
 - vim, 22
- ejecución
 - alternativa, 60
 - condicional, 6
- ejecutar, 21, 28
- El Principito (Antoine de Saint-Exupéry), 212
- elemento, 149, 173
- elipsis, 249
- encadenado de operadores de relación, 58
- encapsulación, 79, 114, 231, 233, 253, 259, 263, 371
- enqueue, 367, 368, 370, 371
- entero, 9, 14
 - impar, 56, 61
 - par, 56, 61
- entrada, 6
- entrada del teclado, 68
- error, 13, 14, 27, 360
 - al tiempo de ejecución, 27, 68, 70
 - de un solo número, 130
 - excepción, 28
 - fuera de rango, 131, 167
 - ignorar, 328
 - off-by-one, 316, 317
 - semántico, 27, 28
 - sintáctico, 18, 20, 28, 36
 - sintaxis, 27
 - tipográfico, 221
 - Y2K, 410
- es-anagrama, 174, 378
- es-inversa, 130, 132, 360
- escalar, 18
- espacio
 - de dos dimensiones, 233
 - de nombres (namespace), 271, 272
- espacio en blanco, 140
- espacio en blanco en regexes, 118
- está-entre, 345
- está-ordenada, 377
- estado, 99, 191
 - objeto, 232
- estimado

- de pi, 352
- estringuificación, 112, 126
- estructura, 12
- estructura de datos, 214
 - selección, 214
- evaluación
 - de corto-circuito, 59, 378
 - retardada, 302
- evaluar, 21, 28
- excepción, 27, 28, 370
 - not declared, 41
- existencia
 - comprobando, 184
- expresión, 20, 28
 - anidada, 414
 - Booleana, 56, 71
 - regular, 111, 114, 132, 265
- extracción
 - de fechas, 125
 - de una dirección IP, 127
- fábula sobre OOP, 258, 259
- factoría, función, 51
- factorial, 64, 65, 162, 208, 320, 396
 - con una lista infinita perezosa, 310
 - creando un operador, 396
 - función, 82, 85
 - función recursiva con sentencias de depuración, 89
 - operador, 297, 396
 - usando el metaoperador de reducción, 162, 294
 - usando la función reduce, 162
 - usando recursión, 82
 - usando subrutinas multi, 87
 - usando un bloque puntiagudo, 65
 - usando un bucle for, 64
 - usando un modificador de sentencia, 65
- False
 - valor especial, 56, 130
- Febrero
 - número de días, 271, 409
- fecha
 - extracción de, 125
 - formato, 270, 271
 - gramática, 271
 - validación, 270, 411
- Fermat, El Último Teorema de, 72, 340
- Fermat, Pierre, 340
- Fibonacci, 188
 - función, 84, 188
 - función con subrutinas multi, 87
 - memoizada, 189
 - memoizada con una variable estado, 191
 - números, 73, 310
- Fibonacci, Leonardo, 73
- FIFO, 156, 292, 371
- FIFO (first in / first out), 155
- figura
 - cuadrado, 246
 - cuadrilátero, 246
 - paralelogramo, 246
 - rectángulo, 246
 - rombo, 246
 - trapezoide, 246
- filter (filtro), 164
- first in / first out (FIFO), 155
- flag, 191, 193
- flujo
 - control, 100
 - de ejecución, 39, 51, 85, 89, 96
- frecuencia, 182
 - de palabras, 201
 - tabla, 223, 224
- función, 31, 34, 37, 50
 - chars, 51
 - chr, 135
 - comparar, 344
 - contains, 118
 - dd, 187, 192
 - done-testing, 396
 - fmt, 334
 - gather, 306
 - gcd, 91, 348, 351
 - get, 68, 139
 - grep, 165, 288, 289, 292, 293
 - index, 109, 113, 118, 133, 325
 - is-leap-year, 410
 - join, 293
 - lcm, 296
 - log10 (logaritmo en base 10), 35
 - log (logaritmo natural), 35
 - map, 163, 165, 288, 289, 292, 293
 - now, 382, 383
 - open, 137
 - ord, 135
 - printf, 334, 339, 352
 - prompt, 68, 339
 - reduce, 165, 288, 289, 292, 330

- reverse, 293
- rindex, 109
- sin, 35
- slurp-rest, 137
- slurp, 138
- sort, 292
- so, 124
- splice, 172
- split, 293
- sprintf, 148, 334, 367
- spurt, 138
- sqrt, 36, 78
- squish, 330
- substr, 110, 133, 325
- take, 306, 307
- tc, 289
- unique, 330
- ack, 90
- Ackermann, 90, 194, 346
- anónima, 47, 51, 289
- argumento, 40
- Booleana, 80
- comparar, 77
- cuadro, 83
- de orden superior, 287, 288, 300, 322
- de retrollamada, 288, 322
- definición, 37, 39, 50
- definida por el programador, 40
- factoría, 48, 51, 313, 371
- factorial, 82
- Fibonacci, 84, 188
- fructuosa, 42, 51
- gama, 85
- grep, 374
- hash, 193
- llamada, 31, 37, 50
- map, 373
- marco, 42, 51
- matemática, 34
- orden superior, 46
- parámetro, 40
- recursiva, 66, 338
- signatura, 44, 51
- trigonométrica, 35
- void, 42, 51, 67
- función spurt
 - modo de concatenación, 139
- función de orden superior, 165
- función o método
 - abs, 77, 171
 - append, 369, 382
 - chars, 108, 109, 353
 - comb, 107, 113, 114, 182, 354
 - elems, 152, 179, 180, 185, 329, 380
 - flip, 110, 347, 354, 367
 - index, 360
 - join, 112
 - keys, 159, 181, 182, 211, 329
 - kv, 159, 181, 186, 329
 - lc, 112, 170
 - pairs, 181
 - pick, 185, 202, 210, 330
 - pop, 153, 155, 156, 367, 372, 377
 - prepend, 369
 - push, 153, 155, 156, 180, 185, 369, 372, 382
 - rand, 174, 184, 201
 - repeated, 380
 - reverse, 186, 407
 - roll, 381
 - round, 32
 - say, 8, 187
 - shift, 153, 156, 174, 369, 377
 - sort, 168, 182
 - splice, 372, 382
 - split, 111, 329
 - squish, 153
 - substr, 109, 353, 354, 361
 - sum, 330
 - tc, 112
 - uc, 112, 158
 - unique, 153, 186, 380
 - unshift, 153, 156, 174, 367, 373, 382
 - values, 179, 181
 - words, 112, 329
- función, razones para el uso de una, 48
- Gadsby, 140
- generador
 - operador de secuencia, 310
- generalización, 143
- gráfica de llamada, 189, 193
- grafema, 108, 109
- gramática, 265, 266, 271, 272, 285, 412
 - calculadora aritmética, 274, 286, 411, 413
 - de Raku, 266
 - depuración, 283
 - fecha, 271, 409
 - herencia, 273, 283

- JSON, 275
- métodos, 271
- Mensaje, 273
- MensajeFormal, 273
- mutable, 283
- subclase, 283
- Grammar::Debugger, 285
- Grammar::Tracer, 285
- Gregory XIII, Pope, 410
- grep, 116, 179, 292, 300, 306, 322, 385
- grupos, 123
- Hamlet, 250
- hash, 177, 193, 214, 380, 385, 389
 - artículo, 193
 - búsqueda, 183
 - búsqueda inversa, 183
 - función, 188, 193
 - invertir, 186
 - membresía, 179
 - multidimensional, 391
 - recorrer con, 182
 - sustracción de, 206, 207
 - tabla, 193
- hashable, 188, 193
- hay más de una forma para hacer algo, 81
- heap, 409
- heap sort, 217
- Hello World, 8
- herencia, 233, 241, 246, 263
 - clase, 241
 - múltiple, 248
- hipotenusa, 79, 344, 345
- histograma, 182
 - elección aleatoria, 202, 210
 - frecuencia de palabras, 203
 - selección aleatoria, 399
- Hoare, Charles Antony Richard, 298, 323
- hoja (árbol), 216
- Holmes, Sherlock, 49
- homófona, 194, 392
- Huffman
 - árbol, 409
 - código, 222, 223, 225, 403, 409
 - codificación, 405
 - decodificación, 407
 - tabla, 224, 404–406
- Huffman, David A., 223
- hyperoperador, 294–296
- identificadores significativos, 326
- idiomático, 47, 83, 100, 161, 178, 185, 291, 367
- idioma, 328
- igualdad y asignación, 93
- implementación, 181, 193, 214, 257
- imprimir cuadrícula, 52, 53
- imprimir-cuadr, 336, 338
- incremento, 114
- indentación, 37, 60, 62, 328
- infijo, 207
- inicialización
 - antes de la actualización, 95
 - variable, 104
- INIT now, 383
- inmutabilidad, 188
- instancia, 234, 263
 - como valor de retorno, 240
- instanciación, 234
- instanciar, 263
- instrucción, 6
- inteligencia artificial, 267
- interfaz, 251, 254, 257
- interpolación, 38, 61
- interpolando un bloque de código dentro de una cadena de texto, 204
- interpretador, 7, 13
- introspección, 9
- inventando la rueda nuevamente, 326
- inverso, 170
- invertiendo un hash, 186
- invocación, 9, 34
 - de método, 32, 34, 37, 38, 108, 117, 154, 234, 237, 239, 374, 380
- invocante, 9, 11, 64, 118, 141, 232, 237, 238, 251, 252, 329
 - de método por defecto, 64
- iter-map, 305
- iteración, 95, 104, 351
- iterador, 302, 303, 305, 322
 - grep, 303–305
 - map, 302, 303
- JSON
 - array, 276, 278
 - Booleano, 276
 - cadena de texto, 276, 277
 - formato, 276
 - gramática, 275, 279
 - muestra, 276
 - número, 276, 277
 - objeto, 276, 278

- tipos base, 276
- valor, 276, 278
- Kernighan, Brian, 116, 325
- KISS: keep it simple, stupid, 325
- Knuth, Donald, 298, 327
- lambda, 289, 290, 322
- last in / first out (LIFO), 155
- laziness, 302, 303, 305, 308, 322
- lazy
 - procesamiento de lista, 302
- lenguaje
 - de dominio específico (DSL), 266, 298
 - formal, 5, 11, 14
 - jerga (slang), 266
 - natural, 11, 14
 - para consultar bases de datos (SQL), 267
 - seguro, 27
 - Turing completo, 81
- letra
 - mayúscula, 112, 123, 135, 168, 362
 - minúscula, 112, 123, 168, 203, 362, 406
 - título, 112
- letras dobles, 147
- lexical, 18
- lexing, 266
- LIFO, 156
- LIFO (last in / first out), 155
- Linux, 50
- lipograma, 140
- lista, 149, 173
 - anidada, 151, 173, 375
 - aplanada, 354, 369, 375
 - de palabras, 139
 - elemento, 151
 - enlazada, 216
 - infinita, 303, 309
 - perezosa, 308
 - recorrido, 158
 - slice, 152
 - vacía, 151
- listas enlazadas, 215
- literal, 12, 20
- llamada por referencia, 46
- llaves, 37, 60, 97, 177, 278, 391
- máximo común divisor (MCD), 91, 348
- método, 34, 231, 232, 236, 237, 263
 - IO.lines, 138, 140
 - IO.slurp, 138
 - Int, 33, 202
 - Rat, 33
 - Str, 33
 - WHAT, 9, 192, 234
 - assuming, 313
 - denominator, 10
 - end, 329
 - fileparse, 271
 - fmt, 367
 - head, 207
 - isa, 234
 - made, 275
 - make, 275
 - match, 117
 - nude, 10
 - numerator, 10
 - parse, 271, 272, 275, 279
 - reduction, 274
 - slurp, 382
 - subparse, 272
 - subst, 128
 - tail, 207
 - accesor, 235, 253
 - anulación, 242, 244
 - de acción, 274
 - de Newton, 101, 351
 - despacho, 237, 243, 248
 - getter, 253
 - multi, 243
 - mutador, 253
 - público, 254
 - privado, 254
 - setter, 253
- métrica de software, 321
- módulo, 34, 50, 386, 387
 - BúsquedaBinaria, 386
 - creando un módulo, 386
 - de perfil, 214
 - de pruebas, 345, 362, 396
 - usando un módulo, 387
 - use, 387
 - use lib, 387
- módulo Test, 318
 - función diag, 320
 - función is-approx, 319
 - función is, 319
 - función like, 320
 - función nok, 319
 - función ok, 319

- función `unlike`, 320
- MAIN, 69, 321, 399
- mamífero, 246, 247
- mantenimiento, 257
- map, 163, 164, 186, 292, 294, 300, 302, 305, 306, 314, 322, 389
- mapeo, 177, 193, 213
- marcador, 160, 169, 171, 290
 - parámetro, 160
- marco, 42, 51
- Markov
 - análisis, 211
- mayúscula, 19, 123, 135
- mayúsculas
 - función `uc`, 112
- McCloskey, Robert, 114
- membresía
 - búsqueda binaria, 174
 - búsqueda por bisección, 174
- memo, 188, 189, 193
- memoizar, 188, 189, 193, 390
- mensaje de error, 14, 20, 27
- metaoperador, 162, 294, 295, 322, 328, 376
 - de reducción, 294, 296, 299
- mezcla, 213
- mezcla de arrays o listas, 315
- minúscula, 19, 123
 - categoría de clases, 133
- minúsculas
 - función `lc`, 112
- mix, 208
- mixhash, 208
- modelo jerárquico, 246
- modificador, 123, 129
 - de sentencia, 63, 65, 71, 80, 97, 354
- modo
 - de concatenación, 138
 - de escritura, 138
 - de lectura, 138
 - interactivo, 22, 23, 28, 43, 79, 139
 - one-liner, 24, 28, 141, 201, 262, 363, 365, 400
 - script, 22, 23, 28, 43
- modo de archivo
 - concatenación, 138
 - escritura, 138
 - lectura, 138
- montículo, 217, 409
- montículo binario, 217
- Morse, Samuel, 222
- multi
 - subrutina, 207, 251
 - subrutinas, 87
- mutabilidad, 152
- mutador, 238
- my-grep, 300, 307
- my-map, 300, 306
- número
 - aúreo, 19
 - aleatorio, 330
 - binario, 225
 - complejo, 246
 - entero, 246
 - impar, 56, 61
 - mágico, 327
 - par, 56, 61
 - racional, 246
 - real, 246
- números
 - aleatorios, 201
 - Fibonacci, 310, 341, 342
- new
 - constructor, 255, 256
 - constructor de objeto, 234
- Newton, Isaac, 351
- Nil, 370
- nodo
 - hijo (árbol), 216
 - padre (árbol), 216
- nodo (árbol), 216
- nombrado(a)
 - regex, 268, 269
 - regla, 268, 269, 274
 - token, 268, 269
- nombre de variable, 26
- none junction, 198
- notación
 - punto (dot), 232, 234, 235, 263
 - sufija, 65
- nuevos operadores
 - creando, 207
- numificación, 126
- objeto, 132, 231
- objeto, 231, 249, 263, 371
 - atributo, 232
 - clase, 233
 - comportamiento, 232
 - composición, 239, 240, 263

- constructor, 234
- de acción, 274
- de archivo, 137, 147
- de coincidencia, 266–268, 274, 285
- de primera clase, 46, 51, 287, 371
- embebido, 240, 263
- estado, 232
- instancia, 234
- interfaz, 253
- primera clase, 322
- octeto, 127, 242, 243
- ocultación de datos, 371
- ocultación de información, 263
- odómetro, 147, 365
- omitir el punto y coma, 60
- one junction, 198
- OOP (programación orientada a objetos), 231
 - una fábula, 258
- operador, 14, 21
 - ++ (incremento), 95, 104
 - (decremento), 95, 104
 - .. (rango), 197
 - <=> (tres sentidos), 58
 - == (igualdad numérica), 56, 93
 - [...] (corchetes), 107, 123
 - % (módulo), 56, 71
 - ∈ (membresía de conjunto), 209
 - ⊃ (set contain), 209
 - \ (diferencia de conjunto), 210
 - ⊕, 299
 - (coincidencia inteligente), 58
 - != (desigualdad numérica), 57
 - () , 24
 - ** (potenciación), 24
 - * (multiplicación), 24
 - * (whatever), 309, 310, 330
 - ++ (aumento), 22
 - ++ (incremento), 182
 - += (aumento y asignación), 22
 - +(adición), 21
 - , (coma), 149
 - (aumento), 22
 - ... (secuencia), 308, 310, 330
 - .. (rango), 64, 150, 329, 330
 - / (división), 24
 - <...> (quote-word), 150, 180
 - <== (backward feed), 294
 - <=> (comparación numérica), 169
 - <= (menor que o igual a), 57
 - < (numéricamente menor que), 57
 - === (identidad de valor), 57
 - ==> (feed), 293
 - == (igualdad numérica), 153
 - => (constructor de par), 57
 - = (asignación), 17
 - >= (mayor que o igual a), 57
 - > (numéricamente mayor que), 57
 - X (cross), 295
 - Z (zip), 295
 - [...] (corchetes), 150, 151
 - [...] (reducción), 162, 376
 - and, 58
 - cmp, 58, 288
 - cmp (comparación de cadenas de texto), 169–171
 - div, 338
 - div (división de enteros), 55, 72
 - eqv (alfabética y verdaderamente equivalente), 57
 - eqv (equivalencia), 153
 - eq, 112
 - eq (igualdad de cadena de texto), 57
 - ge (alfabéticamente posterior o igual), 57
 - gt (alfabéticamente posterior), 57
 - leg, 58
 - leg (comparación de cadenas de texto), 169
 - le (alfabéticamente inferior o igual), 57
 - lt (alfabéticamente inferior), 57
 - mod, 72, 338
 - ne (desigualdad de cadena de texto), 57
 - not, 58
 - or, 58
 - rx (regex), 117
 - s/// (sustitución), 128
 - slice, 152
 - tr, 363
 - x (repetición de cadenas de texto), 26
 - zip, 295, 328
 - (concatenación), 111, 112
 - (coincidencia inteligente), 115, 116, 128, 265
 - aritmético, 8
 - cadena de texto, 25
 - conjunto, 9
 - es-divisible, 81
 - factorial, 208
 - módulo, 297

- rango, 309
- redefinición, 207
- relacional encadenado, 345
- swap, 298
- ternario condicional anidado, 196
- operadores
 - booleanos de corto-circuito, 59
 - de relación encadenados, 62
 - lógicos, 58, 62
 - redefinidos, 207
- operando, 21, 28
- optimización prematura, 327
- orden
 - alfabético, 114, 135
 - de las operaciones, 28
- ordenamiento, 58, 168, 169, 344
 - alfabético, 168
 - ASCIIbetical, 169
 - avanzado, 169
 - de datos, 168
 - insensible a letras minúsculas y mayúsculas, 170, 171
 - lexicográfico, 168
 - numérico, 168
 - objeto de código, 169
 - orden inverso, 169
 - por montículo, 217
 - subrutina de comparación, 170
 - subrutina de transformación, 171, 374
- palíndrome, 346, 347, 365, 366
- palíndromo, 90, 91, 145, 147, 148, 320
- palabra
 - clave, 20, 27
 - reducible, 194
 - reservada, 20
- palabra clave
 - else, 60
 - elsif, 61
 - multi, 343
 - sub, 37
 - unless, 63
- palabras entrelazadas, 175
- par, 215, 304, 403
- par clave-valor, 177, 193, 278
- parámetro, 40, 41, 50
 - auto-declarado, 160, 290
 - con valor por defecto, 200
 - de posición, 199, 200
 - inmutable, 45, 51
 - marcador, 290, 301
 - mutable, 45
 - nombrado, 87, 199, 240, 242, 244, 256
 - opcional, 200, 206
 - posicional, 242, 244, 256
 - slurpy, 200, 368, 372, 373
 - variádico, 368, 372
- parámetros
 - de subrutina, 97
- paréntesis, 25, 286
 - anulando la regla de precedencia, 286
 - argumento, 32
 - grupos y capturas, 123
 - parámetros en, 40, 41
 - vacíos, 37
- paradigma de programación, 287
- paradoja del cumpleaños, 174, 380, 381
- pareja de palabras inversas, 175
- parse, 286
- parsing, 266
- paso suave, 29
- pastor, 258
- pastor (niño), 258
- patrón, 115, 117, 132, 265
 - de búsqueda, 132, 141
 - filter (filtro), 163, 173
 - guardián, 86, 89, 130, 342, 346
 - map (mapeo), 163, 173
 - reduce (reducción), 161, 173
- PEMDAS, 24
- pereza
 - procesamiento de lista, 302
- perro, 246, 247
 - ovejero, 258
- phi, 19
- pi, 19, 25, 36, 105
 - estimación, 105
- pila, 215, 216
- pivote
 - algoritmo quick sort, 323, 415
- plan de desarrollo
 - programación de paso aleatorio, 221
 - reducción, 144, 146, 147
- plan de pruebas, 396
- Poe, Edgar Allan, 223, 402
- poesía, 12
- polimorfismo, 233, 251, 259, 263
- post-condición, 88
- postmatch, 126
- potencia, 91, 347, 348

- potenciación, 414
- precedencia, 25, 286, 298, 362, 412, 414
 - de operadores, 59, 298, 362
 - operador, 24, 28
- precondición, 88
- prefijo, 213
- prematch, 126
- programa, 5, 14
 - argumento, 399
 - línea de comando, 399
 - pruebas de, 146
- programación, 5
 - de paso aleatorio, 221
 - de tubería de datos, 80, 206, 402
 - de tuberías de datos, 288, 292, 315, 322
 - declarativa, 267, 285
 - estilo funcional, 414
 - funcional, 165, 171, 267, 287, 308, 314, 315, 318, 371, 376, 378
 - lógica, 267
 - orientada a objetos (OOP), 231
 - orientada a objetos (OOP) - una fábula, 258
- programación declarativa
 - Bison, 267
 - makefile, 267
 - Yacc, 267
- prompt, 7, 14, 260
- prosa, 12
- Proyecto Gutenberg, 201, 203, 223
- Proyecto Moby, 139
- pruebas, 327, 396
 - acto de fe, 84
 - de programa, 283
 - desarrollo incremental, 77
 - módulo, 318
 - no regresivas, 321
 - pruebas automáticas, 318
 - saber la respuesta, 77
 - son difíciles, 146
 - y ausencia de errores, 147
- pseudo-aleatorio, 201, 221
- pseudo-code, 298
- pseudo-Morse, 225, 406, 407
- punto de interrupción, 261
- punto matemático, 233
- punto y coma, 21, 29
 - omitir, 60
- punto y medio, 8, 391
- Puzzler, 147, 148, 194
- queue, 155, 156, 215, 367, 370
- raíz cuadrada, 101, 105, 351
- racional, 14
- radián, 35
- Raku
 - diversión, 283, 328, 331
 - documentación, 331
 - en un navegador, 6
 - extendiendo el lenguaje, 208, 283, 300
 - idiomático, 328
 - la cultura de Perl, 81
 - PCRE (Perl Compatible Regular Expressions), 116
 - versión, 7
- rama, 61, 71
- Ramanujan, Srinivasa, 105, 352
 - estimación de pi, 105, 352
- rango, 152
 - operador, 64
 - tipo, 150
- rasgo, 46, 51, 387
 - does, 248
 - is export, 386
 - is rw, 236, 244
 - is (subclase), 242, 244, 248, 273
 - is copy, 46
 - is rw, 46
- rasgo (trait), 86
- ratchet, 269
- raya, 20
- reasignación, 94, 104, 152
- rebanada, 108, 132, 152, 377
 - asignación, 156
 - hash, 328
- recorrer y contar, 114
- recorrido, 113, 130, 132, 144, 161, 182
 - con cadenas de texto, 114
 - lista, 158
- rectángulo, 239
- recursión, 65, 66, 72, 73, 82, 84, 145, 342, 346, 375, 383, 384, 412, 415
 - caso base, 67, 318, 415
 - infinita, 68, 72, 85
- reducción, 294, 322
- reducción a un problema previamente solucionado, 144, 146, 147
- reduce, 292, 322
- redundancia, 12

- regex, 107, 111, 114, 132, 141, 170, 197, 262, 265, 325, 356, 357, 364
 - adverbio, 123
 - anclas, 120
 - captura, 266, 357
 - depuración, 262
 - paréntesis versus corchetes, 123
 - patrón delimitador, 117
- regla, 270, 285, 412
 - TOP, 271, 275
 - recursiva, 269, 272
- repetición, 6
 - de código, 49
 - de cadenas de texto, 26
- REPL, 7, 21, 28, 79, 139, 150
- representación, 233, 239
- resaltado de sintaxis, 22, 70
- residuo de la división, 55
- resolución de problemas, 5, 13
- restricción, 86
 - tipo, 44
- retorno, 43
 - sentencia de, 67, 75
 - valor de, 32, 50, 75, 240
- retroceso, 116, 117, 132, 262, 269
- reuso de código, 248, 386
- reutilización de código, 49
- RGB, 242, 243
- rol, 138, 232, 233, 246, 248, 249, 263
 - aplicación, 248
 - composición, 248
 - Guía, 250
 - IO, 138
 - tipo, 250
- rosettacode, 307
- rot13, 135, 362
- rotación de letras, 135, 194, 362
- símbolo de infinidad, 309
- Saint-Exupéry, Antoine de, 212
- salida, 6
- Schiller, Friedrich, 250
- script, 22, 28
- secuencia, 11, 107, 132, 149
 - aritmética, 309, 310
 - geométrica, 309, 310
- sed, 116
- selección de estructuras de datos, 214
- self, 237
- semántica, 28
- sentencia, 21, 28
 - asignación, 17
 - condicional, 60, 71
 - de asignación, 94
 - de impresión, 8, 14
 - de retorno, 67, 75
 - default, 197
 - for, 64, 101, 113, 158, 355
 - given, 196, 238, 394
 - if, 60
 - last, 100, 367
 - next, 100
 - proceed, 197, 394, 395
 - switch, 196
 - use, 50
 - when, 196, 394, 395
 - while, 95
- set (conjunto), 208, 397
 - membresía, 194
 - operador contain, 209
 - operador de diferencia, 210
 - operador de membresía, 209
- sethash, 208
- Shakespeare, William, 250
- SICP, 91
- sigilo, 18, 20, 25, 27, 150, 160, 177, 288
 - %, 177
 - @ (arroba), 150
 - y (ampersand), 47, 164, 171
- signatura, 44, 46, 51, 69, 160, 195, 199, 200, 207, 335, 342, 346, 349, 367, 368, 372, 387, 396, 399, 400
 - de subrutina, 156, 158, 207
 - de una subrutina, 367
- signo de interrogación, 8
- sigspace, 269
- simplicidad, 326
- singularidad, 174
- sintaxis, 11, 14, 27
 - colon-pair, 199
 - error, 27, 28
 - infija, 207
 - prefija, 354
 - sufija, 97
- slice, 205
 - asignación, 156
 - lista, 152
- sobreescribir, 221
- solución a los ejercicios, 333
- sort, 205, 288, 290, 292, 293, 374, 378, 379

- bubble sort, 302
 - comb sort, 301, 302
 - merge sort, 301, 315, 317, 318
 - quick sort, 298, 301, 323, 324, 414, 415
- stack, 155, 156, 215, 370
- sub, 37
 - palabra clave, 37
- sub-language, 266
- subíndice, 107
- subcadena, 108–110
- subclase, 244, 245, 255, 263
- subconjunto
 - tipo, 192
- submétodo, 255
- submétodo BUILD, 255, 256
- subpatrón, 126, 127, 266, 267, 270, 280
- subregla, 268
- subrutina, 37
 - añadir, 312
 - anónima, 289, 292, 322, 371
 - de comparación, 170
 - genérica, 48
 - léxica, 350
 - MAIN, 399
 - multi, 87, 207, 321
 - substr, 313
 - variádica, 200
- subset, 243
 - tipo, 86, 192
- subtarea, 6
- sufijo, 213
- suma
 - cumulativa, 173
- Sussman, Gerald Jay, 91
- sustitución, 128, 129
- sustracción
 - de hash, 206
 - sin tomar prestado, 103
- swap, 298, 299
- título, 19
 - función tc, 112
- término, 20, 28
- tabla
 - de frecuencia, 402, 405, 406
 - de multiplicación, 70, 290
 - hash, 193
- teorema de Pitágoras, 77
- terrenal, 82
- texto
 - aleatorio, 213
 - simple, 137, 201
- The Gold-Bug (Edgar Allan Poe), 223
- Thomson, Ken, 116
- Three Sisters, 250
- tiene-duplicados, 379
- tilde, 126
- TIMTOWTDI, 81, 125, 185, 357
- tipo, 9, 14, 234, 237, 241, 249, 250, 255
 - Bool, 56
 - Complex, 45
 - Int, 9, 14, 44, 45
 - Numeric, 45
 - Rat, 9, 14
 - Real, 45
 - Str, 9, 14, 44, 45
 - array, 149
 - bag, 208, 397
 - baghash, 208, 397
 - chequeo, 85
 - coerción, 25, 33, 45
 - construyendo un nuevo tipo, 232
 - de conversión, 32
 - definido por el programador, 233, 234, 263
 - definiendo un rol, 250
 - del parámetro, 44
 - del subset, 86
 - hash, 177
 - integrado, 250
 - lista, 149
 - mix, 208
 - mixhash, 208
 - objeto, 234, 263
 - rango, 150
 - restricción, 44
 - set, 208, 397
 - sethash, 208
 - subset, 192, 349
- tipo de operador
 - circunfijo, 297
 - infijo, 296
 - poscircunfijo, 297
 - prefijo, 296, 297
 - sufijo, 297
- token, 12, 14, 269, 270, 412
- traceback, 69
- triángulo, 72, 340
- tronco (árbol), 216
- True

- valor especial, 56, 130
- Turing
 - lenguaje completo, 81
 - tesis, 81
- Turing, Alan, 81
- twigil, 160, 200, 235, 239, 290
- unión, 198, 199
 - junction*, 198
 - tipos, 198
- Unicode, 19, 108, 109
- validación de entrada, 339
- validando
 - el número del día, 126
 - el número del mes, 126
- valor, 9, 14, 193
 - de retorno, 240
 - hard-coded, 327
 - no definido, 76
 - no inicializado, 71
 - por defecto, 200, 221
- valor especial
 - Any, 43, 50
 - False, 56
 - True, 56
- variable, 17, 19, 27
 - actualización, 94
 - declaración, 17, 18
 - dinámica, 307
 - escalar, 18
 - especial, 64
 - global, 191, 193
 - intercambio, 298, 330
 - interpolación, 38, 355
 - interpolación de, 61
 - léxica, 18, 97, 99, 191
 - local, 41, 50
 - tópica, 64, 117, 141, 150, 158, 163, 196, 238, 329, 355
 - temporaria, 76, 89
- Variable ... is not declared, 41
- vertebrado, 246, 247
- von Neumann, John, 315
- Weinberger, Peter, 116
- WHAT, 192
- whatever, 184
 - parámetro star, 313
 - término, 313, 314
- Wright, Ernest Vincent, 140
- YAGNI: you aren't gonna need it, 326

Raku Cheat Sheet



SIGILOS

\$escalar item list sink
@array Str flat/slice
%hash Num lazy/eager/
&código Bool hyper/race

CONTEXTOS MAYOR/MENOR

ACCESO

completo: @array[]
elemento: @array[0]
rebanada: @array[0,2]

ARRAYS

HASHES

%hash{}
%hash{'a'} o %hash<a>
%hash{'a','b'} o %hash<a b>

TWIGILS

\$normal-lexical
\$?constate-compilador
\$*global-o-dinámica
\$.accesor-público
\$!accesor-privado
\$^parámetro-posicional
\$:parámetro-nombrado
\$=información-pod
\$<captura-coinc-nombrada>
\$~variable-slang

COMPOSITORES

[] array
{ } bloque/hash
< > quotewords
(,) lista
:() signatura
\() captura

VARIABLES ESPECIALES

\$_ tópico actual
\$/ resultado de regex
\$! error de objeto
@*ARGS línea de comando
@*INC incluir ruta (path)
%*ENV entorno
\$*PID proceso id

DOMINIOS DE OPERADOR

Numérico: Stringy: Valor:
== eq eqv
!==(!=) !eq(ne) !eqv
+ ~
< lt before
> gt after
<=> leg cmp
<= le !after
>= ge !before

ObjectID: === !===

PRECEDENCIAS DE OPERADOR

tightest
↑
.method .[] i
++ --
**
unary + - ~ ! ? ^
* / % %% div
+ -
x xx
~
&
| ^
sleep abs sin temp let
<=> leg cmp .. but
~~ > == gt eq === eqv !op
&&
|| ^^ // min max
??!! ff
= := op= =>
so not
, :
X Xop Z Zop ...
say die map etc.
and
or xor
loosest
↓
<== ==>

DESREFERENCIAS AUTOMÁTICAS

&(\$foo)(1,2) == \$foo(1,2)
@(\$foo)[1] == \$foo[1]
%(\$foo){'bar'} == \$foo<bar>
@(@(\$foo)[1])[2] == \$foo[1][2]

METAOPERADORES

[op] reducir listop to A op B op C...
op= A = A op B
!op !(A op B)
»op« hyper/vectorizar
Zop zip con operador op
Xop cross con operador op
Rop invertir argumentos
Sop crear secuencia

SINTAXIS DE CONTROL

for LIST { }
for LIST -> \$a, \$b { }
while/until EXPR { }
repeat while/until EXPR { }
loop { } loop (a;b;c) { }
if EXPR { } elsif EXPR { } else { }
with EXPR { } orwith EXPR { } else { }
given EXPR { when EXPR { } default { } }
EXPR if EXPR for LIST;
next, last, redo
proceed, succeed

DECLARADORES DE ÁMBITO

my ámbito lexical
our ámbito de paquete
has ámbito de instancia
anon no ámbito
state lexical persistente
augment parasítico benigno
supersede parasítico maligno

METACARACTERES DE REGEX

^ \$ inicio/final de cadena
^^ \$\$ inicio/final de línea
+ uno o más
* cero o más
? cero o uno
**1..3 reptir en rango
() captura
[] sin captura
<[]> categoría de clases
<foo> sub-regla
| or paralelo
|| or serial
<< >> bordo de palabra

MODIFICADORES DE REGEX

:i ignorar letras MAY/min
:m ignore marcas
:g global
:r no retroceso
:s espacio significativo
:4th n° ocurrencia
:4x n veces

TIPOS BÁSICOS

AST Any Mu Label Block Code
Callable CallFrame Int atomicint Rat
FatRat Num Rational Real Complex Numeric Str Cool IntStr
RatStr NumStr ComplexStr Stringy Date DateTime Duration
Instant Dateish Sub Routine Macro Method Submethod Variable
Signature Parameter Whatever HyperWhatever WhateverCode
Proxy ObjAt Version Nil Bool Junction Scalar

CATEGORÍA DE CLASES DE REGEX

. == cualquier carácter, \N non \n
\s == <space>, \S non
\d == <digit>, \D non
\w == <alpha+digit+[_]>

REGEX, TOKEN Y RULE

- regex = regex ordinario
- token = regex + :r implícito
- rule = token + :s implícito

QUOTING

Datos

```
my ($a, $o, $h) = (3, 4, 5);
my @let = 'a', 'b';
my %bin = 0 => False, 1 => True;
sub greet { say "Hello" }
```

Cadenas de Textos Literales

Las cadenas de texto son usualmente representadas usando algún tipo de construcción de citación (*quoting*). La forma más minimalista es **Q**, la cual puede usarse con el atajo `Q[...]`, o **Q** seguida por un par de delimitadores.

```
Q[Una cadena literal]
Q[Otra cadena literal]
Q{Casi cualquier delimitador!}
❌ Q(Interpret. de llamada de función)
❌ Q'Esto no funciona'
✅ Q (¡Esto si funciona!)
✅ Q '¡Esto también!'
```

`()` y `'` no pueden ser usados como delimitadores regulares con **Q**. Para usarlos, coloca un espacio entre ellos y **Q**, **q**, y **qq**.

Q con cualquier delimitador puede usarse para citar cadenas verbatim sin interpolación o escape. No obstante, su comportamiento puede cambiarse a través del uso de adverbios:

Características Básicas

	corta	larga	significado	ejemplo
ADVERBIOS	:q	:single	Interpola <code>\</code> , <code>\qq</code> y escape de delim. con <code>\</code>	Q:q[\s] == Q:q[\\s]
	:qq	:double	Interpola con :s, :a, :h, :f, :c, :b	Q:qq!\$a, {2**3}, @let[]!
	:s	:scalar	Interpola variables \$	Q:s^\$c^
	:a	:array	Interpola variables @	Q:a (@let => @let[])
	:h	:hash	Interpola variables %	Q:h!%bin => %bin{}
	:f	:function	Interpola variables &	Q:f!&greet()!
	:c	:closure	Interpola expresiones {...}	Q:c/3{\$a**2 + \$o**2}/
	:b	:backslash	Habilita escapes backslash (<code>\n</code> , <code>\qq</code> , <code>\\$foo</code> , etc)	Q:b[\nHi]

Nota: Por defecto, solo las variables con el sigilo `$` son interpoladas. Aun si usas `Q:a{...}` o `Q:h{...}`, debes añadir `[]` al array o `{}` al hash para realizar la interpolación. Por ejemplo, `Q:a{@let[]}` y `Q:h!%bin{}` serán interpolado, pero no `Q:a{@let}` y `Q:h!%bin!`. Esto evita la interpolación indeseada en casos como `"docs@raku.org"` o `"es.wikipedia.org/wiki/%C3%91"`.

Características Avanzadas

	corta	larga	significado	ejemplo
ADVERBIOS	:x	:exec	Ejecutar comando y devolver resultado	Q:x{date}
	:w	:words	Dividir en palabras (no protección de quote)	.say for Q:w{'a b' c}
	:ww	:quotewords	Dividir palabras(con protección de quote)	.say for Q:ww{'a b' c}
	:v	:val	Convertir en alomorfo si es posible	Q:v{42}.^name
	:to	:heredoc	Analizar como terminador heredoc	Q:to{END}; Content starts in next line END

Algunos Atajos

forma adverbial	atajo	forma usual	significado
Q:q[...]	q[...]	'...'	
Q:qq[...]	qq[...]	"..."	
Q:q:w[...]	qw[...]	<...>	word quoting
Q:q:ww[...]	qww[...]		word quoting con protección de quote
Q:qq:w[...]	qqw[...]		word quoting con interpolación
Q:qq:ww[...]	qqww[...]	<<...>> or « »	word quoting con interpolación y protección de quote
Q:q:x[...]	qx[...]		shell quoting
Q:qq:x[...]	qqx[...]		shell quoting con interpolación

RECURSOS

ENLACES

raku.org
rakudo.org

IRC -- irc.freenode.net

#raku
#raku-dev
#moarvm

Forums

/r/rakulang

SOCIAL

#rakulang